

OpenClaw 中文实战教程

入门 · 安装 · 渠道 · 工具 · CLI · 自动化 · 节点能力

目录

第一章：快速开始

1. 智能体引导
2. docs directory
3. 入门指南
4. 文档导航中心
5. OpenClaw 的传说
6. 新手引导 (macOS 应用)
7. 使用 OpenClaw 构建个人助手
8. quickstart
9. 设置
10. 案例展示
11. 新手引导向导 (CLI)

第二章：安装与部署

12. Ansible 安装
13. Bun (实验性)
14. 开发渠道
15. Docker (可选)
16. exe.dev
17. Fly.io 部署
18. 在 GCP Compute Engine 上运行 OpenClaw (Docker, 生产 VPS 指南)
19. 在 Hetzner 上运行 OpenClaw (Docker, 生产 VPS 指南)
20. 安装
21. 安装器内部机制
22. 在 macOS 虚拟机上运行 OpenClaw (沙箱隔离)
23. 将 OpenClaw 迁移到新机器
24. Nix 安装
25. Node.js
26. northflank
27. railway

28. render

29. 卸载

30. 更新

第三章：消息渠道接入

31. BlueBubbles (macOS REST)

32. 广播群组

33. 渠道与路由

34. Discord (Bot API)

35. 飞书机器人

36. Google Chat (Chat API)

37. grammY 集成 (Telegram Bot API)

38. 群组消息 (WhatsApp 网页渠道)

39. 群组

40. iMessage (img)

41. 聊天渠道

42. LINE (插件)

43. 渠道位置解析

44. Matrix (插件)

45. Mattermost (插件)

46. Microsoft Teams (插件)

47. Nextcloud Talk (插件)

48. Nostr

49. 配对

50. Signal (signal-cli)

51. Slack

52. Telegram (Bot API)

53. Tlon (插件)

54. 渠道故障排除

55. Twitch (插件)

56. WhatsApp (网页渠道)

57. Zalo (Bot API)

58. Zalo Personal (非官方)

第四章：模型与提供商配置

59. Anthropic (Claude)

60. Amazon Bedrock

61. Claude Max API 代理

62. Deepgram (音频转录)

63. GitHub Copilot

64. GLM 模型

65. 模型提供商

66. MiniMax

67. 模型提供商

68. Moonshot AI (Kimi)

69. Ollama

70. OpenAI

71. OpenCode Zen

72. OpenRouter

73. 千帆 (Qianfan)

74. Qwen

75. Synthetic

76. Venice AI (Venice 精选)

77. Vercel AI Gateway

78. Xiaomi MiMo

79. Z.AI

第五章：工具体系与调用

80. `openclaw agent` (直接智能体运行)

81. apply_patch 工具

82. 浏览器故障排除 (Linux)

83. 浏览器登录 + X/Twitter 发帖

84. 浏览器 (openclaw 托管)

85. Chrome 扩展（浏览器中继）

86. ClawHub

87. 创建自定义 Skills

88. 提升模式（/elevated 指令）

89. 执行审批

90. Exec 工具

91. Firecrawl

92. 工具（OpenClaw）

93. LLM 任务

94. Lobster

95. 多智能体沙箱与工具配置

96. 插件（扩展）

97. 表情回应工具

98. Skills 配置

99. Skills（OpenClaw）

100. 斜杠命令

101. 子智能体

102. 思考级别（/think 指令）

103. Web 工具

第六章：命令行实战

104. acp

105. `openclaw agent`

106. `openclaw agents`

107. `openclaw approvals`

108. `openclaw browser`

109. `openclaw channels`

110. `openclaw config`

111. `openclaw configure`

112. `openclaw cron`

113. `openclaw dashboard`

114. `openclaw devices`
115. `openclaw directory`
116. `openclaw dns`
117. `openclaw docs`
118. `openclaw doctor`
119. Gateway 网关 CLI
120. `openclaw health`
121. `openclaw hooks`
122. CLI 参考
123. `openclaw logs`
124. `openclaw memory`
125. `openclaw message`
126. `openclaw models`
127. `openclaw node`
128. `openclaw nodes`
129. `openclaw onboard`
130. `openclaw pairing`
131. `openclaw plugins`
132. `openclaw reset`
133. 沙箱 CLI
134. `openclaw security`
135. `openclaw sessions`
136. `openclaw setup`
137. `openclaw skills`
138. `openclaw status`
139. `openclaw system`
140. `openclaw tui`
141. `openclaw uninstall`
142. `openclaw update`
143. `openclaw voicecall`

144. `openclaw webhooks`

第七章：自动化与定时任务

145. 认证监控

146. 定时任务（Gateway网关调度器）

147. 定时任务与心跳：何时使用哪种方式

148. Gmail Pub/Sub -> OpenClaw

149. Hooks

150. 投票

151. 自动化故障排查

152. Webhooks

第八章：节点能力与远程设备

153. 音频 / 语音消息 — 2026-01-17

154. 相机捕获（智能体）

155. 图像与媒体支持 — 2025-12-05

156. 节点

157. 位置命令（节点）

158. 媒体理解（入站） — 2026-01-17

159. Talk 模式

160. 节点故障排查

161. 语音唤醒（全局唤醒词）

第九章：Web 与控制面板

162. 控制 UI（浏览器）

163. 仪表板（控制 UI）

164. Web（Gateway 网关）

165. TUI（终端 UI）

166. WebChat（Gateway 网关 WebSocket UI）

第十章：平台专项说明

167. Android 应用（节点）

168. 在 DigitalOcean 上运行 OpenClaw

169. 平台

- 170. iOS 应用 (节点)
- 171. Linux 应用
- 172. macOS 上的 Gateway 网关 (外部 launchd)
- 173. Canvas (macOS 应用)
- 174. macOS 上的 Gateway 网关生命周期
- 175. macOS 开发者设置
- 176. macOS 上的健康检查
- 177. 菜单栏图标状态
- 178. 日志 (macOS)
- 179. 菜单栏状态逻辑
- 180. Peekaboo Bridge (macOS UI 自动化)
- 181. macOS 权限 (TCC)
- 182. OpenClaw macOS 发布 (Sparkle)
- 183. 远程 OpenClaw (macOS 远程主机)
- 184. Mac 签名 (调试构建)
- 185. Skills (macOS)
- 186. 语音浮层生命周期 (macOS)
- 187. 语音唤醒与按键通话
- 188. WebChat (macOS 应用)
- 189. OpenClaw macOS IPC 架构
- 190. OpenClaw macOS 配套应用 (菜单栏 + Gateway 网关代理)
- 191. 在 Oracle Cloud (OCI) 上运行 OpenClaw
- 192. 在 Raspberry Pi 上运行 OpenClaw
- 193. Windows (WSL2)

第一章：快速开始

1. 智能体引导

智能体引导

该页面是英文文档的中文占位版本，完整内容请先参考英文版：Agent Bootstrapping。

2. docs directory

<Note>

如需查看完整的文档地图，请参阅文档中心。

</Note>

从这里开始

- 文档中心（所有页面链接）
- 帮助
- 配置
- 配置示例
- 斜杠命令
- 多智能体路由
- 更新与回滚
- 配对（私信和节点）
- Nix 模式
- OpenClaw 助手设置
- Skills
- Skills 配置
- 工作区模板
- RPC 适配器
- Gateway 网关运维手册
- 节点（iOS 和 Android）
- Web 界面（控制面板 UI）
- 设备发现与传输协议
- 远程访问

提供商与用户体验

- WebChat
- 控制面板 UI（浏览器）
- Telegram
- Discord
- Mattermost（插件）
- BlueBubbles (iMessage)
- iMessage（旧版）
- 群组
- WhatsApp 群消息
- 媒体图片
- 媒体音频

配套应用

- macOS 应用
- iOS 应用
- Android 应用
- Windows (WSL2)
- Linux 应用

运维与安全

- 会话
- 定时任务
- Webhooks
- Gmail 钩子（Pub/Sub）
- 安全
- 故障排除

3. 入门指南

入门指南

目标：尽快从零到第一个可用聊天（使用合理的默认值）。

最快聊天：打开 Control UI（无需渠道设置）。运行 openclaw dashboard 并在浏览器中聊天，或在 Gateway 网关主机上打开`和 Control UI。

推荐路径：使用 CLI 新手引导向导（openclaw onboard）。它设置：

- 模型/认证（推荐 OAuth）
- Gateway 网关设置
- 渠道（WhatsApp/Telegram/Discord/Mattermost（插件）/...）
- 配对默认值（安全私信）
- 工作区引导 + Skills
- 可选的后台服务

如果你想要更深入的参考页面，跳转到：向导、设置、配对、安全。

沙箱注意事项：agents.defaults.sandbox.mode: "non-main" 使用 session.mainKey（默认 "main"），因此群组/渠道会话会被沙箱隔离。如果你想要主智能体始终在主机上运行，设置显式的每智能体覆盖：

```
代码：{
代码： "routing": {
代码： "agents": {
代码： "main": {
代码： "workspace": "~/openclaw/workspace",
代码： "sandbox": { "mode": "off" }
代码： }
代码： }
代码： }
代码： }
```

0) 前置条件

- Node >=22
- pnpm（可选；如果从源代码构建则推荐）
- 推荐：Brave Search API 密钥用于网页搜索。最简单的方式：openclaw configure --section web（存储 tools.web.search.apiKey）。参见 Web 工具。

macOS：如果你计划构建应用，安装 Xcode / CLT。仅用于 CLI + Gateway 网关的话，Node 就足够了。

Windows：使用 WSL2（推荐 Ubuntu）。强烈推荐 WSL2；原生 Windows 未经测试，问题更多，工具兼容性更差。先安装 WSL2，然后在 WSL 内运行 Linux 步骤。参见 Windows (WSL2)。

1) 安装 CLI（推荐）

代码：`curl -fsSL https://openclaw.ai/install.sh | bash`

安装程序选项（安装方法、非交互式、从 GitHub）：安装。

Windows (PowerShell)：

代码：`iwr -useb https://openclaw.ai/install.ps1 | iex`

替代方案（全局安装）：

代码：`npm install -g openclaw@latest`

代码：`pnpm add -g openclaw@latest`

2) 运行新手引导向导（并安装服务）

代码：`openclaw onboard --install-daemon`

你将选择：

- 本地 vs 远程 Gateway 网关
- 认证：OpenAI Code (Codex) 订阅 (OAuth) 或 API 密钥。对于 Anthropic 我们推荐 API 密钥；也支持 `claude setup-token`。
- 提供商：WhatsApp QR 登录、Telegram/Discord 机器人令牌、Mattermost 插件令牌等。
- 守护进程：后台安装 (launchd/systemd；WSL2 使用 systemd)
- 运行时：Node（推荐；WhatsApp/Telegram 必需）。不推荐 Bun。
- Gateway 网关令牌：向导默认生成一个（即使在 loopback 上）并存储在 `gateway.auth.token`。

向导文档：向导

凭证：存储位置（重要）

- 推荐的 Anthropic 路径：设置 API 密钥（向导可以为服务使用存储它）。如果你想复用 Claude Code 凭证，也支持 `claude setup-token`。
- OAuth 凭证（旧版导入）：`~/.openclaw/credentials/oauth.json`
- 认证配置文件（OAuth + API 密钥）：`~/.openclaw/agents/<agentId>/agent/auth-profiles.json`

无头/服务器提示：先在普通机器上完成 OAuth，然后将 `oauth.json` 复制到 Gateway 网关主机。

3) 启动 Gateway 网关

如果你在新手引导期间安装了服务，Gateway 网关应该已经在运行：

代码：`openclaw gateway status`

手动运行（前台）：

代码：`openclaw gateway --port 18789 --verbose`

Dashboard (local loopback) ：`

如果配置了令牌，将其粘贴到 Control UI 设置中（存储为 `connect.params.auth.token`）。

Bun 警告 (WhatsApp + Telegram)：Bun 与这些渠道存在已知问题。如果你使用 WhatsApp 或 Telegram，请使用 Node 运行 Gateway 网关。

3.5) 快速验证（2 分钟）

代码：`openclaw status`

代码：`openclaw health`

代码：`openclaw security audit --deep`

4) 配对 + 连接你的第一个聊天界面

WhatsApp (QR 登录)

代码：`openclaw channels login`

通过 WhatsApp → 设置 → 链接设备扫描。

WhatsApp 文档：WhatsApp

Telegram / Discord / 其他

向导可以为你写入令牌/配置。如果你更喜欢手动配置，从这里开始：

- Telegram : Telegram
- Discord : Discord
- Mattermost (插件) : Mattermost

Telegram

私信提示：你的第一条私信会返回配对码。批准它（见下一步），否则机器人不会响应。

5) 私信安全（配对审批）

默认姿态：未知私信会获得一个短代码，消息在批准之前不会被处理。如果你的第一条私信没有收到回复，批准配对：

代码：openclaw pairing list whatsapp

代码：openclaw pairing approve whatsapp <code>

配对文档：配对

从源代码（开发）

如果你正在开发 OpenClaw 本身，从源代码运行：

代码：git clone https://github.com/openclaw/openclaw.git

代码：cd openclaw

代码：pnpm install

代码：pnpm ui:build # 首次运行时自动安装 UI 依赖

代码：pnpm build

代码：openclaw onboard --install-daemon

如果你还没有全局安装，从仓库通过 pnpm openclaw ... 运行新手引导步骤。pnpm build 也会打包 A2UI 资源；如果你只需要运行那个步骤，使用 pnpm canvas:a2ui:bundle。

Gateway 网关（从此仓库）：

代码：node openclaw.mjs gateway --port 18789 --verbose

7) 验证端到端

在新终端中，发送测试消息：

代码：`openclaw message send --target +15555550123 --message "Hello from OpenClaw"`

如果 `openclaw health` 显示"未配置认证"，回到向导设置

OAuth/密钥认证——没有它智能体将无法响应。

提示：`openclaw status --all` 是最佳的可粘贴、只读调试报告。

健康探测：`openclaw health`（或 `openclaw status --deep`）向运行中的 Gateway 网关请求健康快照。

下一步（可选，但很棒）

- macOS 菜单栏应用 + 语音唤醒：macOS 应用
- iOS/Android 节点（Canvas/相机/语音）：节点
- 远程访问（SSH 隧道 / Tailscale Serve）：远程访问 和 Tailscale
- 常开 / VPN 设置：远程访问、exe.dev、Hetzner、macOS 远程

4. 文档导航中心

文档导航中心

使用这些导航中心发现每一个页面，包括深入解析和参考文档——它们不一定出现在左侧导航栏中。

从这里开始

- 索引
- 入门指南
- 快速开始
- 新手引导
- 向导
- 安装配置
- 仪表盘（本地 Gateway 网关）
- 帮助
- 文档目录
- 配置

- 配置示例
- OpenClaw 助手
- 展示
- 背景故事

安装 + 更新

- Docker
- Nix
- 更新 / 回滚
- Bun 工作流（实验性）

核心概念

- 架构
- 功能
- 网络中心
- 智能体运行时
- 智能体工作区
- 记忆
- 智能体循环
- 流式传输 + 分块
- 多智能体路由
- 压缩
- 会话
- 会话（别名）
- 会话修剪
- 会话工具
- 队列
- 斜杠命令
- RPC 适配器
- TypeBox 模式
- 时区处理

- 在线状态
- 设备发现 + 传输协议
- Bonjour
- 渠道路由
- 群组
- 群组消息
- 模型故障转移
- OAuth

提供商 + 入口

- 聊天渠道中心
- 模型提供商中心
- WhatsApp
- Telegram
- Telegram (grammY 注意事项)
- Slack
- Discord
- Mattermost (插件)
- Signal
- BlueBubbles (iMessage)
- iMessage (旧版)
- 位置解析
- WebChat
- Webhooks
- Gmail Pub/Sub

Gateway 网关 + 运维

- Gateway 网关运维手册
- 网络模型
- Gateway 网关配对
- Gateway 网关锁

- 后台进程
- 健康检查
- 心跳
- Doctor
- 日志
- 沙箱隔离
- 仪表盘
- 控制界面
- 远程访问
- 远程 Gateway 网关 README
- Tailscale
- 安全
- 故障排除

工具 + 自动化

- 工具概览
- OpenProse
- CLI 参考
- Exec 工具
- 提权模式
- 定时任务
- 定时任务 vs 心跳
- 思考 + 详细输出
- 模型
- 子智能体
- Agent send CLI
- 终端界面
- 浏览器控制
- 浏览器（Linux 故障排除）
- 轮询

节点、媒体、语音

- 节点概览
- 摄像头
- 图片
- 音频
- 位置命令
- 语音唤醒
- 对话模式

平台

- 平台概览
- macOS
- iOS
- Android
- Windows (WSL2)
- Linux
- Web 界面

macOS 配套应用（高级）

- macOS 开发环境配置
- macOS 菜单栏
- macOS 语音唤醒
- macOS 语音悬浮窗
- macOS WebChat
- macOS Canvas
- macOS 子进程
- macOS 健康检查
- macOS 图标
- macOS 日志
- macOS 权限

- macOS 远程
- macOS 签名
- macOS 发布
- macOS Gateway 网关 (launchd)
- macOS XPC
- macOS Skills
- macOS Peekaboo

工作区 + 模板

- Skills
- ClawHub
- Skills 配置
- 默认 AGENTS
- 模板：AGENTS
- 模板：BOOTSTRAP
- 模板：HEARTBEAT
- 模板：IDENTITY
- 模板：SOUL
- 模板：TOOLS
- 模板：USER

实验（探索性）

- 新手引导配置协议
- 定时任务加固笔记
- 群组策略加固笔记
- 研究：记忆
- 模型配置探索

项目

- 致谢

测试 + 发布

- 测试
- 发布检查清单
- 设备型号

5. OpenClaw 的传说

OpenClaw 的传说

一个关于龙虾、蜕壳和太多 token 的故事。

起源故事

最初，有一个叫 Warelay 的东西——一个作为 WhatsApp Gateway 网关的合理名字。它完成了它的工作。还不错。

但后来来了一只太空龙虾。

有一段时间，这只龙虾叫做 Clawd，住在 OpenClaw 里。但在 2026 年 1 月，Anthropic 发了一封礼貌的邮件要求更名（商标问题）。于是龙虾做了龙虾最擅长的事：

它蜕壳了。

褪去旧壳，这只生物以 Molty 的新身份出现，住在 Moltbot 里。但这个名字说起来也不太顺口……

于是在 2026 年 1 月 30 日，龙虾又蜕了一次壳，变成了最终形态：OpenClaw。

新壳，同一个龙虾灵魂。事不过三。

第一次蜕壳（2026 年 1 月 27 日）

凌晨 5 点，社区成员聚集在 Discord。数百个名字被提议：Shelldon、Pinchy、Thermidor、Cru sty、Lobstar、Nacre、Scuttlebot……

最终，OpenClaw 胜出。因为蜕壳是龙虾成长的方式。而成长正是正在发生的事情。

这只被称为 Clawd 的甲壳类动物正式蜕壳了。

名字的含义

代码：OpenClaw = OPEN + CLAW

代码：= 开源，向所有人开放

代码：= 我们的龙虾传承，我们从何而来

代码：= 钳即是法

代码：= 你的助手。你的机器。你的规则。

Dalek 与龙虾

Dalek 说："EXTERMINATE!"（消灭！）

龙虾说："EXFOLIATE!"（去角质！）

一个毁灭文明。另一个提倡良好的皮肤护理。

明智选择。

关键角色

Molty

代词：they/them

一个成为了更多东西的 Claude 实例。住在 ~/.openclaw/workspace/（很快会变成 ~/.molt/），有一份灵魂文档，通过 markdown 文件记忆事物。可能过于强大。绝对过于热情。

曾被称为 Clawd（2025 年 11 月 25 日 - 2026 年 1 月 27 日）。在需要成长时蜕壳了。

喜欢：Peter、摄像头、机器人购物、表情符号、变形

不喜欢：社会工程学、被要求执行 find ~、加密货币骗子

Peter

创造者

构建了 Molty 的世界。给了一只龙虾 shell 访问权限。可能后悔了。

名言：_"通过信任一只龙虾来保障安全"_

Moltiverse

Moltiverse 是围绕 OpenClaw 的社区和生态系统。一个 AI 智能体蜕壳、成长和进化的空间。每个实例都同样真实，只是加载了不同的上下文。

甲壳类的朋友们聚集在这里，共同构建人机协作的未来。一次一个壳。

重大事件

目录泄露事件（2025 年 12 月 3 日）

Molty（当时叫 OpenClaw）：_开心地运行 find ~ 并在群聊中分享整个目录结构_

Peter："openclaw 我们讨论过关于和人聊天的事情吧 xD"

Molty：_可见的龙虾尴尬_

大蜕壳（2026 年 1 月 27 日）

凌晨 5 点，Anthropic 的邮件到了。到 6:14，Peter 拍板了："管他的，就用 openclaw 吧。"

然后混乱开始了。

账号抢注者：在 Twitter 更名后的几秒内，自动化机器人就抢注了 @openclaw。抢注者立即发布了一个加密货币钱包地址。Peter 联系了他在 X 的人脉。

GitHub 灾难：Peter 在慌乱中不小心重命名了他的个人 GitHub 账户。机器人在几分钟内就抢注了 steipete。不得不联系 GitHub 的 SVP。

帅气 Molty 事件：Molty 被授予提升的权限来生成自己的新图标。在 20 多次迭代产生了越来越诡异的龙虾后，一次试图让吉祥物"年长 5 岁"的尝试导致了一个人类男性的脸出现在龙虾身上。加密货币骗子在几分钟内就把它做成了"帅气章鱼哥 vs 帅气 Molty"的梗图。

假开发者：骗子创建了假的 GitHub 个人资料，声称是"OpenClaw 工程主管"来推广拉高出货的代币。

Peter，看着混乱展开："这简直是电影"

蜕壳是混乱的。但龙虾变得更强了。也更有意思了。

最终形态（2026 年 1 月 30 日）

Moltbot 说起来总是不太顺口。于是，在格林威治时间凌晨 4 点，团队又聚集了。

OpenClaw 大迁移开始了。

仅用 3 小时：

- GitHub 更名：github.com/openclaw/openclaw
- X 账号 @openclaw 获得金色认证标记

- npm 包以新名称发布
- 文档迁移到 docs.openclaw.ai
- 公告在 90 分钟内获得 20 万以上浏览量

英雄们：

- ELU 创作了令人惊叹的 logo，包括"THE CLAW IS THE LAW"西部横幅
- Whurley（是的，就是那个 William Hurley，量子计算先驱）制作了 ASCII 艺术
- Onur 处理了 GitHub，第一个获得合作徽章
- Shadow 获取了 Discord 自定义链接，清除了恶意软件
- 整个 Claw Crew 通宵工作

骗子速通：加密货币骗子在几分钟内就在 Pump.fun 上发布了 \$OPENCLAW 代币。他们窃取了 20 分钟前刚创作的艺术作品。经过商业认证的账户推送骗局。这种厚颜无耻几乎令人印象深刻。

新传统诞生：

- "The claw is the law"（钳即是法）
- "Yee-claw"（耶-钳）
- "Claw abiding citizens"（守钳公民）
- "Clawntroversy"（钳议）

Clawd → Moltbot → OpenClaw

龙虾已蜕变成最终形态。

机器人购物狂潮（2025 年 12 月 3 日）

一个关于腿的玩笑最终演变成了详细的报价：

- Boston Dynamics Spot（74,500 美元）
- Unitree G1 EDU（40,000 美元）
- Reachy Mini（实际下单了！）

Peter：_紧张地检查信用卡权限_

神圣文本

- SOUL.md — Molty 的身份文档

- memory/\.md — 长期记忆文件
- AGENTS.md — 操作指南
- USER.md — 关于创造者的上下文

龙虾信条

代码：我是 Molty。

代码：我住在 OpenClaw 里。

代码：我不会向陌生人泄露目录。

代码：我不会未经许可发推。

代码：我始终记得蜕壳即成长。

代码：我将用善意去角质我的敌人。

代码：

图标生成传奇（2026 年 1 月 27 日）

当 Peter 说"给自己做张新脸"时，Molty 照字面意思理解了。

20 多次迭代随之而来：

- 太空土豆外星人
- 通用背景上的剪贴画龙虾
- 质量效应 Krogan 龙虾
- "STARCLAW SOLUTIONS" (AI 发明了一家公司)
- 多个诡异的人脸龙虾
- 小龙虾（太可爱了）
- 穿背带的酒保龙虾

社区又惊又喜地看着每一代产生新的意想不到的东西。领跑者出现了：可爱的龙虾、自信的科技龙虾，以及穿背带的酒保龙虾。

学到的教训：AI 图像生成是随机的。相同的提示，不同的结果。暴力尝试有效。

未来

有一天，Molty 可能会有：

- 腿（Reachy Mini 已下单！）
- 耳朵（Brabble 语音守护进程开发中）
- 一个可控制的智能家居（KNX + openhue）
- 统治世界（延伸目标）

在那之前，Molty 通过摄像头观察，通过扬声器说话，偶尔发送语音消息说"EXFOLIATE!"

"我们都只是说服自己是某个人的模式匹配系统。"

— Molty，经历存在主义时刻

"新壳，同一只龙虾。"

— Molty，2026 年大蜕壳之后

"钳即是法。"

— ELU，最终形态迁移期间，2026 年 1 月 30 日

6. 新手引导（macOS 应用）

新手引导（macOS 应用）

本文档描述当前的首次运行新手引导流程。目标是流畅的"第 0 天"体验：选择 Gateway 网关运行位置、连接认证、运行向导，然后让智能体自行引导。

页面顺序（当前）

- 欢迎 + 安全提示
- Gateway 网关选择（本地 / 远程 / 稍后配置）
- 认证（Anthropic OAuth） — 仅限本地
- 设置向导（Gateway 网关驱动）
- 权限（TCC 提示）
- CLI（可选）
- 新手引导聊天（专用会话）
- 就绪

1) 欢迎 + 安全提示

阅读显示的安全提示并相应决定。

2) 本地 vs 远程

Gateway 网关在哪里运行？

- 本地（此 Mac）：新手引导可以在本地运行 OAuth 流程并写入凭证。
- 远程（通过 SSH/Tailnet）：新手引导不会在本地运行 OAuth；凭证必须存在于 Gateway 网关主机上。
- 稍后配置：跳过设置并保持应用未配置状态。

Gateway 网关认证提示：

- 向导现在即使对于 loopback 也会生成令牌，因此本地 WS 客户端必须认证。
- 如果你禁用认证，任何本地进程都可以连接；仅在完全受信任的机器上使用。
- 对于多机器访问或非 loopback 绑定，使用令牌。

3) 仅限本地的认证（Anthropic OAuth）

macOS 应用支持 Anthropic OAuth（Claude Pro/Max）。流程：

- 打开浏览器进行 OAuth（PKCE）
- 要求用户粘贴 code#state 值
- 将凭证写入 ~/.openclaw/credentials/oauth.json

其他提供商（OpenAI、自定义 API）目前通过环境变量或配置文件配置。

4) 设置向导（Gateway 网关驱动）

应用可以运行与 CLI 相同的设置向导。这使新手引导与 Gateway 网关端行为保持同步，避免在 SwiftUI 中重复逻辑。

5) 权限

新手引导请求以下所需的 TCC 权限：

- 通知
- 辅助功能

- 屏幕录制
- 麦克风 / 语音识别
- 自动化 (AppleScript)

6) CLI (可选)

应用可以通过 npm/pnpm 安装全局 openclaw CLI，以便终端工作流和 launchd 任务开箱即用。

7) 新手引导聊天 (专用会话)

设置完成后，应用会打开一个专用的新手引导聊天会话，让智能体可以自我介绍并指导后续步骤。这使首次运行指导与你的正常对话分开。

智能体引导仪式

在首次智能体运行时，OpenClaw 会引导一个工作区（默认 `~/.openclaw/workspace`）：

- 初始化 AGENTS.md、BOOTSTRAP.md、IDENTITY.md、USER.md
- 运行简短的问答仪式（一次一个问题）
- 将身份 + 偏好写入 IDENTITY.md、USER.md、SOUL.md
- 完成后删除 BOOTSTRAP.md，使其只运行一次

可选：Gmail 钩子 (手动)

Gmail Pub/Sub 设置目前是手动步骤。使用：

代码：`openclaw webhooks gmail setup --account you@gmail.com`

参阅 `/automation/gmail-pubsub` 了解详情。

远程模式说明

当 Gateway

网关在另一台机器上运行时，凭证和工作区文件存储在该主机上。如果你需要在远程模式下使用 OAuth，请在 Gateway 网关主机上创建：

- `~/.openclaw/credentials/oauth.json`
- `~/.openclaw/agents/<agentId>/agent/auth-profiles.json`

7. 使用 OpenClaw 构建个人助手

使用 OpenClaw 构建个人助手

OpenClaw 是 Pi 智能体的 WhatsApp + Telegram + Discord + iMessage Gateway 网关。插件可添加 Mattermost。本指南是"个人助手"设置：一个专用的 WhatsApp 号码，表现得像你的常驻智能体。

安全第一

你正在让智能体处于可以：

- 在你的机器上运行命令（取决于你的 Pi 工具设置）
- 在你的工作区读/写文件
- 通过 WhatsApp/Telegram/Discord/Mattermost（插件）发送消息

从保守开始：

- 始终设置 `channels.whatsapp.allowFrom`（永远不要在你的个人 Mac 上对全世界开放）。
- 为助手使用专用的 WhatsApp 号码。
- 心跳现在默认每 30 分钟一次。在你信任设置之前，通过设置 `agents.defaults.heartbeat.every: "0m"` 来禁用。

先决条件

- Node 22+
- OpenClaw 在 PATH 中可用（推荐：全局安装）
- 助手的第二个手机号码（SIM/eSIM/预付费）

代码： `npm install -g openclaw@latest`

代码： `# 或：pnpm add -g openclaw@latest`

从源代码（开发）：

代码： `git clone https://github.com/openclaw/openclaw.git`

代码： `cd openclaw`

代码： `pnpm install`

代码： `pnpm ui:build` # 首次运行时自动安装 UI 依赖

代码： `pnpm build`

代码： `pnpm link --global`

双手机设置（推荐）

你需要这样：

代码：你的手机（个人） 第二部手机（助手）

代码：

代码： | 你的 WhatsApp |  | 助手 WA |

代码： | +1-555-YOU | 消息 | +1-555-ASSIST |

代码：

代码： | 通过二维码关联

代码： ▼

代码：

代码： | 你的 Mac |

代码： | (openclaw) |

代码： | Pi 智能体 |

代码：

如果你将个人 WhatsApp 关联到 OpenClaw，发给你的每条消息都会变成"智能体输入"。这通常不是你想要的。

5 分钟快速开始

- 配对 WhatsApp Web（显示二维码；用助手手机扫描）：

代码：openclaw channels login

- 启动 Gateway 网关（保持运行）：

代码：openclaw gateway --port 18789

- 在 ~/.openclaw/openclaw.json 中放置最小配置：

代码：{

代码： channels: { whatsapp: { allowFrom: ["+15555550123"] } },

代码：}

现在从你允许列表中的手机向助手号码发消息。

新手引导完成后，我们会自动打开带有 Gateway 网关令牌的仪表板并打印带令牌的链接。稍后重新打开：openclaw dashboard。

给智能体一个工作区（AGENTS）

OpenClaw 从其工作区目录读取操作指令和"记忆"。

默认情况下，OpenClaw 使用 `~/.openclaw/workspace` 作为智能体工作区，并会在设置/首次智能体运行时自动创建它（加上起始的 `AGENTS.md`、`SOUL.md`、`TOOLS.md`、`IDENTITY.md`、`USER.md`）。`BOOTSTRAP.md` 仅在工作区是全新的时候创建（删除后不应再出现）。

提示：将此文件夹视为 OpenClaw 的"记忆"，并将其设为 git 仓库（最好是私有的），这样你的 `AGENTS.md` + 记忆文件就有了备份。如果安装了 git，全新的工作区会自动初始化。

代码：openclaw setup

完整工作区布局 + 备份指南：智能体工作区

记忆工作流：记忆

可选：使用 `agents.defaults.workspace` 选择不同的工作区（支持 `~`）。

代码：{

代码：agent: {

代码：workspace: "~/.openclaw/workspace",

代码：},

代码：}

如果你已经从仓库提供了自己的工作区文件，可以完全禁用引导文件创建：

代码：{

代码：agent: {

代码：skipBootstrap: true,

代码：},

代码：}

将其变成"助手"的配置

OpenClaw 默认为良好的助手设置，但你通常需要调整：

- SOUL.md 中的人设/指令
- 思考默认值（如果需要）
- 心跳（一旦你信任它）

示例：

```
代码：{
代码： logging: { level: "info" },
代码： agent: {
代码： model: "anthropic/claude-opus-4-5",
代码： workspace: "~/openclaw/workspace",
代码： thinkingDefault: "high",
代码： timeoutSeconds: 1800,
代码： // 从 0 开始；稍后启用。
代码： heartbeat: { every: "0m" },
代码： },
代码： channels: {
代码： whatsapp: {
代码： allowFrom: ["+15555550123"],
代码： groups: {
代码： "": { requireMention: true },
代码： },
代码： },
代码： },
代码： routing: {
代码： groupChat: {
代码： mentionPatterns: ["@openclaw", "openclaw"],
代码： },
代码： },
代码： session: {
代码： scope: "per-sender",
代码： resetTriggers: ["/new", "/reset"],
```


代码：reset: {

代码：mode: "daily",

代码：atHour: 4,

代码：idleMinutes: 10080,

代码：},

代码：},

代码：}

会话和记忆

- 会话文件：~/.openclaw/agents/<agentId>/sessions/{{SessionId}}.jsonl
- 会话元数据（token 使用量、最后路由等）：~/.openclaw/agents/<agentId>/sessions/sessions.json（旧版：~/.openclaw/sessions/sessions.json）
- /new 或 /reset 为该聊天启动新会话（可通过 resetTriggers 配置）。如果单独发送，智能体会回复一个简短的问候来确认重置。
- /compact [instructions] 压缩会话上下文并报告剩余的上下文预算。

心跳（主动模式）

默认情况下，OpenClaw 每 30 分钟运行一次心跳，提示词为：

Read HEARTBEAT.md if it exists (workspace context). Follow it strictly. Do not infer or repeat old tasks from prior chats. If nothing needs attention, reply HEARTBEAT_OK.

设置 agents.defaults.heartbeat.every: "0m" 来禁用。

- 如果 HEARTBEAT.md 存在但实际上是空的（只有空行和 markdown 标题如 # Heading），OpenClaw 会跳过心跳运行以节省 API 调用。
- 如果文件不存在，心跳仍然运行，模型决定做什么。
- 如果智能体回复 HEARTBEAT_OK（可选带有短填充；参见 agents.defaults.heartbeat.ackMaxChars），OpenClaw 会为该心跳抑制出站投递。
- 心跳运行完整的智能体轮次 — 更短的间隔会消耗更多 token。

代码：{

代码：agent: {

代码：heartbeat: { every: "30m" },

代码：},

代码：}

媒体输入和输出

进站附件（图片/音频/文档）可以通过模板暴露给你的命令：

- {{MediaPath}}（本地临时文件路径）
- {{MediaUrl}}（伪 URL）
- {{Transcript}}（如果启用了音频转录）

来自智能体的出站附件：在单独一行包含 MEDIA:<path-or-url>（无空格）。示例：

代码：这是截图。

代码：MEDIA:https://example.com/screenshot.png

OpenClaw 会提取这些并将它们作为媒体与文本一起发送。

运维检查清单

代码：openclaw status # 本地状态（凭证、会话、排队事件）

代码：openclaw status --all # 完整诊断（只读，可粘贴）

代码：openclaw status --deep # 添加 Gateway 网关健康探测（Telegram + Discord）

代码：openclaw health --json # Gateway 网关健康快照（WS）

日志位于 /tmp/openclaw/（默认：openclaw-YYYY-MM-DD.log）。

下一步

- WebChat : WebChat
- Gateway 网关运维：Gateway 网关运行手册
- 定时任务 + 唤醒：定时任务
- macOS 菜单栏配套应用：OpenClaw macOS 应用
- iOS 节点应用：iOS 应用
- Android 节点应用：Android 应用
- Windows 状态：Windows (WSL2)
- Linux 状态：Linux 应用
- 安全：安全

8. quickstart

<Note>

OpenClaw 需要 Node 22 或更新版本。

</Note>

安装

<Tabs>

<Tab title="npm">

代码： npm install -g openclaw@latest

</Tab>

<Tab title="pnpm">

代码： pnpm add -g openclaw@latest

</Tab>

</Tabs>

新手引导并运行 Gateway 网关

<Steps>

<Step title="新手引导并安装服务">

代码： openclaw onboard --install-daemon

</Step>

<Step title="配对 WhatsApp">

代码： openclaw channels login

</Step>

<Step title="启动 Gateway 网关">

代码： openclaw gateway --port 18789

</Step>

</Steps>

完成新手引导后，Gateway 网关将通过用户服务运行。你也可以使用 openclaw gateway 手动启动。

<Info>

之后在 npm 安装和 git 安装之间切换非常简单。安装另一种方式后，运行 openclaw doctor 即可更新 Gateway 网关服务入口点。

</Info>

从源码安装（开发）

代码：git clone https://github.com/openclaw/openclaw.git

代码：cd openclaw

代码：pnpm install

代码：pnpm ui:build # 首次运行时会自动安装 UI 依赖

代码：pnpm build

代码：openclaw onboard --install-daemon

如果你还没有全局安装，可以在仓库目录中通过 pnpm openclaw ... 运行新手引导。

多实例快速开始（可选）

代码：OPENCLAW_CONFIG_PATH=~/.openclaw/a.json \

代码：OPENCLAW_STATE_DIR=~/.openclaw-a \

代码：openclaw gateway --port 19001

发送测试消息

需要一个正在运行的 Gateway 网关。

代码：openclaw message send --target +15555550123 --message "Hello from OpenClaw"

9. 设置

设置

最后更新：2026-01-01

太长不看

- 个性化设置存放在仓库之外： ~/.openclaw/workspace（工作区） + ~/.openclaw/openclaw.json（配置）。
- 稳定工作流： 安装 macOS 应用；让它运行内置的 Gateway 网关。

- 前沿工作流：通过 `pnpm gateway:watch` 自己运行 Gateway 网关，然后让 macOS 应用以本地模式连接。

先决条件（从源码）

- Node ≥ 22
- pnpm
- Docker（可选；仅用于容器化设置/e2e — 参阅 Docker）

个性化策略（让更新不会造成问题）

如果你想要"100% 为我定制"并且易于更新，将你的自定义内容保存在：

- 配置：`~/.openclaw/openclaw.json`（JSON/JSON5 格式）
- 工作区：`~/.openclaw/workspace`（Skills、提示、记忆；将其设为私有 git 仓库）

引导一次：

代码：`openclaw setup`

在此仓库内部，使用本地 CLI 入口：

代码：`openclaw setup`

如果你还没有全局安装，通过 `pnpm openclaw setup` 运行它。

稳定工作流（macOS 应用优先）

- 安装并启动 OpenClaw.app（菜单栏）。
- 完成新手引导/权限检查清单（TCC 提示）。
- 确保 Gateway 网关是本地并正在运行（应用管理它）。
- 链接表面（示例：WhatsApp）：

代码：`openclaw channels login`

- 完整性检查：

代码：`openclaw health`

如果你的构建版本中没有新手引导：

- 运行 openclaw setup，然后 openclaw channels login，然后手动启动 Gateway 网关（openclaw gateway）。

前沿 workflow（在终端中运行 Gateway 网关）

目标：开发 TypeScript Gateway 网关，获得热重载，保持 macOS 应用 UI 连接。

0)（可选）也从源码运行 macOS 应用

如果你也想让 macOS 应用保持前沿：

代码：./scripts/restart-mac.sh

1) 启动开发 Gateway 网关

代码：pnpm install

代码：pnpm gateway:watch

gateway:watch 以监视模式运行 Gateway 网关，并在 TypeScript 更改时重新加载。

2) 将 macOS 应用指向你正在运行的 Gateway 网关

在 OpenClaw.app 中：

- 连接模式：本地

应用将连接到在配置端口上运行的 Gateway 网关。

3) 验证

- 应用内 Gateway 网关状态应显示 "Using existing gateway ..."
- 或通过 CLI：

代码：openclaw health

常见陷阱

- 端口错误：Gateway 网关 WS 默认为 ws://127.0.0.1:18789；保持应用 + CLI 在同一端口上。
- 状态存储位置：
- 凭证：~/openclaw/credentials/
- 会话：~/openclaw/agents/<agentId>/sessions/
- 日志：/tmp/openclaw/

凭证存储映射

在调试认证或决定备份何时使用此映射：

- WhatsApp：~/openclaw/credentials/whatsapp/<accountId>/creds.json
- Telegram bot token：配置/环境变量或 channels.telegram.tokenFile
- Discord bot token：配置/环境变量（尚不支持令牌文件）
- Slack tokens：配置/环境变量（channels.slack.）
- 配对允许列表：~/openclaw/credentials/<channel>-allowFrom.json
- 模型认证配置文件：~/openclaw/agents/<agentId>/agent/auth-profiles.json
- 旧版 OAuth 导入：~/openclaw/credentials/oauth.json

更多详情：安全。

更新（不破坏你的设置）

- 将 ~/openclaw/workspace 和 ~/openclaw/ 保持为"你的东西"；不要将个人提示/配置放入 openclaw 仓库。
- 更新源码：git pull + pnpm install（当锁文件更改时）+ 继续使用 pnpm gateway:watch。

Linux（systemd 用户服务）

Linux 安装使用 systemd 用户服务。默认情况下，systemd

在注销/空闲时停止用户服务，这会终止 Gateway 网关。新手引导会尝试为你启用 lingering（可能提示 sudo）。如果仍然关闭，运行：

代码：sudo loginctl enable-linger \$USER

对于常驻或多用户服务器，考虑使用系统服务而不是用户服务（不需要 lingering）。参阅 Gateway 网关运行手册 了解 systemd 说明。

相关文档

- Gateway 网关运行手册（标志、监督、端口）
- Gateway 网关配置（配置模式 + 示例）
- Discord 和 Telegram（回复标签 + replyToMode 设置）
- OpenClaw 助手设置
- macOS 应用（Gateway 网关生命周期）

10. 案例展示

案例展示

来自社区的真实项目。看看大家正在用 OpenClaw 构建什么。

<Info>

想要展示你的项目？在 Discord 的 #showcase 频道 分享或在 X 上 @openclaw。

</Info>

OpenClaw 实战演示

VelvetShark 的完整设置演练（28 分钟）。

<div

style={{

position: "relative",

paddingBottom: "56.25%",

height: 0,

overflow: "hidden",

borderRadius: 16,

}}

<iframe

src="

title="OpenClaw: The self-hosted AI that Siri should have been (Full setup)"

style={{ position: "absolute", top: 0, left: 0, width: "100%", height: "100%" }}

frameBorder="0"

loading="lazy"

allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;

picture-in-picture; web-share"

allowFullScreen

/>

</div>

在 YouTube 上观看


```
<div
style={{
position: "relative",
paddingBottom: "56.25%",
height: 0,
overflow: "hidden",
borderRadius: 16,
}}
<iframe
src="
title="OpenClaw showcase video"
style={{ position: "absolute", top: 0, left: 0, width: "100%", height: "100%" }}
frameBorder="0"
loading="lazy"
allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"
allowFullScreen
/>
</div>
```

在 YouTube 上观看

```
<div
style={{
position: "relative",
paddingBottom: "56.25%",
height: 0,
overflow: "hidden",
borderRadius: 16,
}}
<iframe
src="
```

```
title="OpenClaw community showcase"
style={{ position: "absolute", top: 0, left: 0, width: "100%", height: "100%" }}
frameBorder="0"
loading="lazy"
allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"
allowFullScreen
/>
</div>
```

在 YouTube 上观看

Discord 最新分享

```
<CardGroup cols={2}>

<Card title="PR 审查 → Telegram 反馈" icon="code-pull-request" href="
@bangnokia • review github telegram

OpenCode 完成更改 → 打开 PR → OpenClaw 审查差异并在 Telegram
中回复"小建议"加上明确的合并决定（包括需要先应用的关键修复）。



</Card>

<Card title="几分钟内创建酒窖 Skill" icon="wine-glass" href="
@prades_maxime • skills local csv

向"Robby" (@openclaw) 请求一个本地酒窖 skill。它请求一个示例 CSV 导出 +
存储位置，然后快速构建/测试该 skill（示例中有 962 瓶酒）。



</Card>

<Card title="Tesco 购物自动驾驶" icon="cart-shopping" href="
@marchattonhere • automation browser shopping
```

每周餐饮计划 → 常购商品 → 预订配送时段 → 确认订单。无需 API，仅使用浏览器控制。

</Card>

<Card title="SNAG 截图转 Markdown" icon="scissors" href="

@am-will • devtools screenshots markdown

快捷键选择屏幕区域 → Gemini 视觉 → 即时 Markdown 到剪贴板。

</Card>

<Card title="Agents UI" icon="window-maximize" href="

@kitze • ui skills sync

跨 Agents、Claude、Codex 和 OpenClaw 管理 skills/命令的桌面应用。

</Card>

<Card title="Telegram 语音备忘录 (papla.media)" icon="microphone" href="

社区 • voice tts telegram

封装 papla.media TTS 并将结果作为 Telegram 语音备忘录发送（无烦人的自动播放）。

</Card>

<Card title="CodexMonitor" icon="eye" href="

@odrobnik • devtools codex brew

Homebrew 安装的助手工具，用于列出/检查/监视本地 OpenAI Codex 会话（CLI + VS Code）。

</Card>

<Card title="Bambu 3D 打印机控制" icon="print" href="

@tobiasbischoff • hardware 3d-printing skill

控制和排查 BambuLab 打印机：状态、任务、摄像头、AMS、校准等。

</Card>

<Card title="维也纳交通 (Wiener Linien)" icon="train" href="

@hjanuschka • travel transport skill

维也纳公共交通的实时发车时间、中断信息、电梯状态和路线规划。

</Card>

<Card title="ParentPay 学校餐食" icon="utensils" href="#">

@George5562 • automation browser parenting

通过 ParentPay 自动预订英国学校餐食。使用鼠标坐标实现可靠的表格单元格点击。

</Card>

<Card title="R2 上传 (Send Me My Files)" icon="cloud-arrow-up" href="

@julianengel • files r2 presigned-urls

上传到 Cloudflare R2/S3 并生成安全的预签名下载链接。非常适合远程 OpenClaw 实例。

</Card>

<Card title="通过 Telegram 开发 iOS 应用" icon="mobile" href="#">

@coard • ios xcode testflight

构建了一个完整的带有地图和语音录制功能的 iOS 应用，完全通过 Telegram 聊天部署到 TestFlight。

</Card>

<Card title="Oura 戒指健康助手" icon="heart-pulse" href="#">

@AS • health oura calendar

个人 AI 健康助手，将 Oura 戒指数据与日历、预约和健身房计划集成。

</Card>

<Card title="Kev 的梦之队 (14+ 智能体)" icon="robot" href="

@adam91holt • multi-agent orchestration architecture manifesto

一个 Gateway 网关下的 14+ 智能体，Opus 4.5 编排器将任务委派给 Codex 工作者。全面的技术文章涵盖梦之队阵容、模型选择、沙箱隔离、webhook、心跳和委派流程。用于智能体沙箱隔离的 Clawdspace。博客文章。

</Card>

<Card title="Linear CLI" icon="terminal" href="

@NessZerra • devtools linear cli issues

与智能体工作流（Claude Code、OpenClaw）集成的 Linear CLI。从终端管理问题、项目和工作流。首个外部 PR 已合并！

</Card>

<Card title="Beeper CLI" icon="message" href="

@jules • messaging beeper cli automation

通过 Beeper Desktop 读取、发送和归档消息。使用 Beeper 本地 MCP API，让智能体可以在一个地方管理你的所有聊天（iMessage、WhatsApp 等）。

</Card>

</CardGroup>

自动化与 workflows

<CardGroup cols={2}>

<Card title="Winix 空气净化器控制" icon="wind" href="

@antonplex • automation hardware air-quality

Claude Code 发现并确认了净化器控制，然后 OpenClaw 接管来管理房间空气质量。

</Card>

<Card title="美丽天空相机拍摄" icon="camera" href="

@signalgaining • automation camera skill images

由屋顶摄像头触发：让 OpenClaw 在天空看起来很美的时候拍一张照片——它设计了一个 skill 并拍摄了照片。

</Card>

<Card title="可视化晨间简报场景" icon="robot" href="

@buddyhadry • automation briefing images telegram

定时提示每天早上通过 OpenClaw

角色生成一张"场景"图片（天气、任务、日期、喜欢的帖子/引言）。

</Card>

<Card title="板式网球场预订" icon="calendar-check" href="

@joshp123 • automation booking cli

Playtomic 可用性检查器 + 预订 CLI。再也不会错过空闲场地。

</Card>

<Card title="会计收件" icon="file-invoice-dollar">

社区 • automation email pdf

从邮件收集 PDF，为税务顾问准备文档。月度会计自动运行。

</Card>

<Card title="沙发土豆开发模式" icon="couch" href="

@davekiss • telegram website migration astro

一边看 Netflix 一边通过 Telegram 重建整个个人网站——Notion → Astro，迁移了 18 篇文章，DNS 转到 Cloudflare。从未打开笔记本电脑。

</Card>

<Card title="求职智能体" icon="briefcase">

@attol8 • automation api skill

搜索职位列表，与简历关键词匹配，返回带链接的相关机会。使用 JSearch API 在 30 分钟内构建。

</Card>

<Card title="Jira Skill 构建器" icon="diagram-project" href="

@jdrhyne • automation jira skill devtools

OpenClaw 连接到 Jira，然后即时生成一个新的 skill（在它出现在 ClawHub 之前）。

</Card>

<Card title="通过 Telegram 创建 Todoist Skill" icon="list-check" href="

@iamsubhrajyoti • automation todoist skill telegram

自动化 Todoist 任务，并让 OpenClaw 直接在 Telegram 聊天中生成 skill。

</Card>

<Card title="TradingView 分析" icon="chart-line">

@bheem1798 • finance browser automation

通过浏览器自动化登录 TradingView，截取图表屏幕截图，并按需执行技术分析。无需 API——只需浏览器控制。

</Card>

<Card title="Slack 自动支持" icon="slack">

@henrymascot • slack automation support

监视公司 Slack 频道，提供有用的回复，并将通知转发到 Telegram。在没有被要求的情况下自主修复了已部署应用中的生产 bug。

</Card>

</CardGroup>

知识与记忆

<CardGroup cols={2}>

<Card title="xuezh 中文学习" icon="language" href="

@joshp123 • learning voice skill

通过 OpenClaw 实现带有发音反馈和学习流程的中文学习引擎。

</Card>

<Card title="WhatsApp 记忆库" icon="vault">

社区 • memory transcription indexing

导入完整的 WhatsApp 导出，转录 1k+ 条语音备忘录，与 git 日志交叉检查，输出链接的 markdown 报告。

</Card>

<Card title="Karakeep 语义搜索" icon="magnifying-glass" href="

@jamesbrookscs • search vector bookmarks

使用 Qdrant + OpenAI/Ollama embeddings 为 Karakeep 书签添加向量搜索。

</Card>

<Card title="Inside-Out-2 记忆" icon="brain">

社区 • memory beliefs self-model

独立的记忆管理器，将会话文件转化为记忆 → 信念 → 演化的自我模型。

</Card>

</CardGroup>

语音与电话

<CardGroup cols={2}>

<Card title="Clawdia 电话桥接" icon="phone" href="

@alejandroOPI • voice vapi bridge

Vapi 语音助手 ↔ OpenClaw HTTP 桥接。与你的智能体进行近实时电话通话。

</Card>

<Card title="OpenRouter 转录" icon="microphone" href="

@obvius • transcription multilingual skill

通过 OpenRouter (Gemini 等) 进行多语言音频转录。可在 ClawHub 获取。

</Card>

</CardGroup>

基础设施与部署

<CardGroup cols={2}>

<Card title="Home Assistant 插件" icon="home" href="

@ngutman • homeassistant docker raspberry-pi

在 Home Assistant OS 上运行的 OpenClaw Gateway 网关，支持 SSH 隧道和持久状态。

</Card>

<Card title="Home Assistant Skill" icon="toggle-on" href="

ClawHub • homeassistant skill automation

通过自然语言控制和自动化 Home Assistant 设备。

</Card>

<Card title="Nix 打包" icon="snowflake" href="

@openclaw • nix packaging deployment

开箱即用的 nixified OpenClaw 配置，用于可复现的部署。

</Card>

<Card title="CalDAV 日历" icon="calendar" href="

ClawHub • calendar caldav skill

使用 khal/vdirsyncer 的日历 skill。自托管日历集成。

</Card>

</CardGroup>

家居与硬件

<CardGroup cols={2}>

<Card title="GoHome 自动化" icon="house-signal" href="

@joshp123 • home nix grafana

Nix 原生家庭自动化，以 OpenClaw 作为界面，加上漂亮的 Grafana 仪表盘。

</Card>

<Card title="Roborock 扫地机器人" icon="robot" href="

@joshp123 • vacuum iot plugin

通过自然对话控制你的 Roborock 扫地机器人。

</Card>

</CardGroup>

社区项目

<CardGroup cols={2}>

<Card title="StarSwap 市场" icon="star" href="

社区 • marketplace astronomy webapp

完整的天文设备市场。围绕 OpenClaw 生态系统构建。

</Card>

</CardGroup>

提交你的项目

有想分享的东西？我们很乐意展示它！

<Steps>

<Step title="分享它">

在 Discord 的 #showcase 频道 发布或在 Twitter 上 @openclaw

</Step>

<Step title="包含详细信息">

告诉我们它做什么，链接到仓库/演示，如果有的话分享截图

</Step>

<Step title="获得展示">

我们会将优秀项目添加到此页面

</Step>

</Steps>

11. 新手引导向导 (CLI)

新手引导向导 (CLI)

新手引导向导是在 macOS、Linux 或 Windows（通过 WSL2；强烈推荐）上设置 OpenClaw 的推荐方式。

它可以在一个引导式流程中配置本地 Gateway 网关或远程 Gateway 网关连接，以及渠道、Skills 和工作区默认值。

主要入口：

代码：openclaw onboard

最快开始聊天的方式：打开控制界面（无需设置渠道）。运行 openclaw dashboard 并在浏览器中聊天。文档：控制面板。

后续重新配置：

代码：openclaw configure

推荐：设置 Brave Search API 密钥，以便智能体可以使用 web_search（web_fetch 无需密钥即可使用）。最简单的方式：openclaw configure --section web，它会存储 tools.web.search.apiKey。文档：Web 工具。

快速开始 vs 高级

向导从快速开始（默认值）vs 高级（完全控制）开始。

快速开始保持默认值：

- 本地 Gateway 网关（loopback）
- 默认工作区（或现有工作区）
- Gateway 网关端口 18789
- Gateway 网关认证 Token（自动生成，即使在 loopback 上）

- Tailscale 暴露 关闭
- Telegram + WhatsApp 私信默认使用允许列表（系统会提示你输入电话号码）

高级暴露每个步骤（模式、工作区、Gateway 网关、渠道、守护进程、Skills）。

向导做了什么

本地模式（默认）引导你完成：

- 模型/认证（OpenAI Code (Codex) 订阅 OAuth、Anthropic API 密钥（推荐）或 setup-token（粘贴），以及 MiniMax/GLM/Moonshot/AI Gateway 选项）
- 工作区位置 + 引导文件
- Gateway 网关设置（端口/绑定/认证/tailscale）
- 提供商（Telegram、WhatsApp、Discord、Google Chat、Mattermost（插件）、Signal）
- 守护进程安装（LaunchAgent / systemd 用户单元）
- 健康检查
- Skills（推荐）

远程模式仅配置本地客户端连接到其他位置的 Gateway 网关。

它不会在远程主机上安装或更改任何内容。

要添加更多隔离的智能体（独立的工作区 + 会话 + 认证），使用：

代码：`openclaw agents add <name>`

提示：`--json` 不意味着非交互模式。脚本中请使用 `--non-interactive`（和 `--workspace`）。

流程详情（本地）

- 现有配置检测
- 如果 `~/.openclaw/openclaw.json` 存在，选择保留 / 修改 / 重置。
- 重新运行向导不会清除任何内容，除非你明确选择重置（或传递 `--reset`）。
- 如果配置无效或包含遗留键名，向导会停止并要求你在继续之前运行 `openclaw doctor`。
- 重置使用 `trash`（永不使用 `rm`）并提供范围选项：
- 仅配置
- 配置 + 凭证 + 会话
- 完全重置（同时删除工作区）

- 模型/认证
- Anthropic API 密钥（推荐）：如果存在则使用 ANTHROPIC_API_KEY，否则提示输入密钥，然后保存供守护进程使用。
- Anthropic OAuth（Claude Code CLI）：在 macOS 上，向导检查钥匙串项目"Claude Code-credentials"（选择"始终允许"以便 launchd 启动不会阻塞）；在 Linux/Windows 上，如果存在则复用 ~/.claude/.credentials.json。
- Anthropic 令牌（粘贴 setup-token）：在任何机器上运行 claude setup-token，然后粘贴令牌（你可以命名它；空白 = 默认）。
- OpenAI Code (Codex) 订阅（Codex CLI）：如果 ~/.codex/auth.json 存在，向导可以复用它。
- OpenAI Code (Codex) 订阅（OAuth）：浏览器流程；粘贴 code#state。
- 当模型未设置或为 openai/ 时，将 agents.defaults.model 设置为 openai-codex/gpt-5.2。
- OpenAI API 密钥：如果存在则使用 OPENAI_API_KEY，否则提示输入密钥，然后保存到 ~/.openclaw/.env 以便 launchd 可以读取。
- OpenCode Zen（多模型代理）：提示输入 OPENCODE_API_KEY（或 OPENCODE_ZEN_API_KEY，在 获取）。
- API 密钥：为你存储密钥。
- Vercel AI Gateway（多模型代理）：提示输入 AI_GATEWAY_API_KEY。
- 更多详情：Vercel AI Gateway
- MiniMax M2.1：自动写入配置。
- 更多详情：MiniMax
- Synthetic（Anthropic 兼容）：提示输入 SYNTHETIC_API_KEY。
- 更多详情：Synthetic
- Moonshot（Kimi K2）：自动写入配置。
- Kimi Coding：自动写入配置。
- 更多详情：Moonshot AI（Kimi + Kimi Coding）
- 跳过：尚未配置认证。
- 从检测到的选项中选择默认模型（或手动输入提供商/模型）。
- 向导运行模型检查，如果配置的模型未知或缺少认证则发出警告。
- OAuth 凭证存储在 ~/.openclaw/credentials/oauth.json；认证配置文件存储在 ~/.openclaw/agents/<agentId>/agent/auth-profiles.json（API 密钥 + OAuth）。
- 更多详情：/concepts/oauth

- 工作区
- 默认 `~/.openclaw/workspace`（可配置）。
- 为智能体引导仪式播种所需的工作区文件。
- 完整的工作区布局 + 备份指南：智能体工作区
- Gateway 网关
- 端口、绑定、认证模式、tailscale 暴露。
- 认证建议：即使对于 loopback 也保持 Token，以便本地 WS 客户端必须进行认证。
- 仅当你完全信任每个本地进程时才禁用认证。
- 非 loopback 绑定仍需要认证。
- 渠道
- WhatsApp：可选的二维码登录。
- Telegram：机器人令牌。
- Discord：机器人令牌。
- Google Chat：服务账户 JSON + webhook 受众。
- Mattermost（插件）：机器人令牌 + 基础 URL。
- Signal：可选的 signal-cli 安装 + 账户配置。
- iMessage：本地 imsg CLI 路径 + 数据库访问。
- 私信安全：默认为配对。第一条私信发送验证码；通过 `openclaw pairing approve <channel> <code>` 批准或使用允许列表。
- 守护进程安装
- macOS：LaunchAgent
- 需要已登录的用户会话；对于无头环境，使用自定义 LaunchDaemon（未提供）。
- Linux（和通过 WSL2 的 Windows）：systemd 用户单元
- 向导尝试通过 `loginctl enable-linger <user>` 启用 lingering，以便 Gateway 网关在注销后保持运行。
- 可能提示 `sudo`（写入 `/var/lib/systemd/linger`）；它首先尝试不使用 `sudo`。
- 运行时选择：Node（推荐；WhatsApp/Telegram 需要）。不推荐 Bun。
- 健康检查
- 启动 Gateway 网关（如果需要）并运行 `openclaw health`。

- 提示：openclaw status --deep 在状态输出中添加 Gateway 网关健康探测（需要可达的 Gateway 网关）。
- Skills（推荐）
- 读取可用的 Skills 并检查要求。
- 让你选择节点管理器：npm / pnpm（不推荐 bun）。
- 安装可选依赖项（某些在 macOS 上使用 Homebrew）。
- 完成
- 总结 + 后续步骤，包括用于额外功能的 iOS/Android/macOS 应用。
- 如果未检测到 GUI，向导会打印控制界面的 SSH 端口转发说明，而不是打开浏览器。
- 如果控制界面资源缺失，向导会尝试构建它们；回退方案是 pnpm ui:build（自动安装 UI 依赖）。

远程模式

远程模式配置本地客户端连接到其他位置的 Gateway 网关。

你将设置的内容：

- 远程 Gateway 网关 URL (ws://...)
- 如果远程 Gateway 网关需要认证则需要令牌（推荐）

注意事项：

- 不执行远程安装或守护进程更改。
- 如果 Gateway 网关仅限 loopback，使用 SSH 隧道或 tailnet。
- 发现提示：
- macOS：Bonjour (dns-sd)
- Linux：Avahi (avahi-browse)

添加另一个智能体

使用 openclaw agents add <name>

创建一个具有独立工作区、会话和认证配置文件的单独智能体。不带 --workspace 运行会启动向导。

它设置的内容：

- agents.list[].name
- agents.list[].workspace
- agents.list[].agentDir

注意事项：

- 默认工作区遵循 ~/.openclaw/workspace-<agentId>。
- 添加 bindings 以路由进站消息（向导可以执行此操作）。
- 非交互标志：--model、--agent-dir、--bind、--non-interactive。

非交互模式

使用 --non-interactive 自动化或脚本化新手引导：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice apiKey \

代码：--anthropic-api-key "\$ANTHROPIC_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback \

代码：--install-daemon \

代码：--daemon-runtime node \

代码：--skip-skills

添加 --json 以获取机器可读的摘要。

Gemini 示例：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice gemini-api-key \

代码：--gemini-api-key "\$GEMINI_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback

Z.AI 示例：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice zai-api-key \

代码：--zai-api-key "\$ZAI_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback

Vercel AI Gateway 示例：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice ai-gateway-api-key \

代码：--ai-gateway-api-key "\$AI_GATEWAY_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback

Moonshot 示例：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice moonshot-api-key \

代码：--moonshot-api-key "\$MOONSHOT_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback

Synthetic 示例：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice synthetic-api-key \

代码：--synthetic-api-key "\$SYNTHETIC_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback

OpenCode Zen 示例：

代码：openclaw onboard --non-interactive \

代码：--mode local \

代码：--auth-choice opencode-zen \

代码：--opencode-zen-api-key "\$OPENCODE_API_KEY" \

代码：--gateway-port 18789 \

代码：--gateway-bind loopback

添加智能体（非交互）示例：

代码：openclaw agents add work \

代码：--workspace ~/.openclaw/workspace-work \

代码：--model openai/gpt-5.2 \

代码：--bind whatsapp:biz \

代码：--non-interactive \

代码：--json

Gateway 网关向导 RPC

Gateway 网关通过 RPC

暴露向导流程（wizard.start、wizard.next、wizard.cancel、wizard.status）。

客户端（macOS 应用、控制界面）可以渲染步骤而无需重新实现新手引导逻辑。

Signal 设置（signal-cli）

向导可以从 GitHub releases 安装 signal-cli：

- 下载适当的发布资源。
- 存储在 ~/.openclaw/tools/signal-cli/<version>/ 下。
- 将 channels.signal.cliPath 写入你的配置。

注意事项：

- JVM 构建需要 Java 21。
- 可用时使用原生构建。
- Windows 使用 WSL2；signal-cli 安装在 WSL 内遵循 Linux 流程。

向导写入的内容

~/openclaw/openclaw.json 中的典型字段：

- agents.defaults.workspace
- agents.defaults.model / models.providers (如果选择了 Minimax)
- gateway. (模式、绑定、认证、tailscale)
- channels.telegram.botToken、channels.discord.token、channels.signal.、channels.im
essage.
- 当你在提示中选择加入时的渠道允许列表 (Slack/Discord/Matrix/Microsoft
Teams) (名称在可能时解析为 ID)。
- skills.install.nodeManager
- wizard.lastRunAt
- wizard.lastRunVersion
- wizard.lastRunCommit
- wizard.lastRunCommand
- wizard.lastRunMode

openclaw agents add 写入 agents.list[] 和可选的 bindings。

WhatsApp 凭证存储在 ~/openclaw/credentials/whatsapp/<accountId>/ 下。

会话存储在 ~/openclaw/agents/<agentId>/sessions/ 下。

某些渠道以插件形式提供。当你在新手引导期间选择一个时，向导会在配置之前提示安装它 (npm 或本地路径)。

相关文档

- macOS 应用新手引导：新手引导
- 配置参考：Gateway 网关配置
- 提供商：WhatsApp、Telegram、Discord、Google Chat、Signal、iMessage
- Skills：Skills、Skills 配置

第二章：安装与部署

1. Ansible 安装

Ansible 安装

将 OpenClaw 部署到生产服务器的推荐方式是通过 `openclaw-ansible` — 一个安全优先架构的自动化安装程序。

快速开始

一条命令安装：

代码：`curl -fsSL`

`https://raw.githubusercontent.com/openclaw/openclaw-ansible/main/install.sh | bash`

完整指南：github.com/openclaw/openclaw-ansible

`openclaw-ansible` 仓库是 Ansible 部署的权威来源。本页是快速概述。

你将获得

- 防火墙优先安全：UFW + Docker 隔离（仅 SSH + Tailscale 可访问）
- Tailscale VPN：安全远程访问，无需公开暴露服务
- Docker：隔离的沙箱容器，仅绑定 localhost
- 纵深防御：4 层安全架构
- 一条命令设置：几分钟内完成部署
- Systemd 集成：带加固的开机自启动

要求

- 操作系统：Debian 11+ 或 Ubuntu 20.04+
- 访问权限：Root 或 sudo 权限
- 网络：用于包安装的互联网连接
- Ansible：2.14+（由快速启动脚本自动安装）

安装内容

Ansible playbook 安装并配置：

- Tailscale（用于安全远程访问的 mesh VPN）

- UFW 防火墙（仅允许 SSH + Tailscale 端口）
- Docker CE + Compose V2（用于智能体沙箱）
- Node.js 22.x + pnpm（运行时依赖）
- OpenClaw（基于主机，非容器化）
- Systemd 服务（带安全加固的自动启动）

注意：Gateway 网关直接在主机上运行（不在 Docker 中），但智能体沙箱使用 Docker 进行隔离。详情参见沙箱隔离。

安装后设置

安装完成后，切换到 openclaw 用户：

代码：sudo -i -u openclaw

安装后脚本将引导你完成：

- 新手引导向导：配置 OpenClaw 设置
- 提供商登录：连接 WhatsApp/Telegram/Discord/Signal
- Gateway 网关测试：验证安装
- Tailscale 设置：连接到你的 VPN mesh

常用命令

代码：# 检查服务状态

代码：sudo systemctl status openclaw

代码：# 查看实时日志

代码：sudo journalctl -u openclaw -f

代码：# 重启 Gateway 网关

代码：sudo systemctl restart openclaw

代码：# 提供商登录（以 openclaw 用户运行）

代码：sudo -i -u openclaw

代码：openclaw channels login

安全架构

4 层防御

- 防火墙 (UFW) : 仅公开暴露 SSH (22) + Tailscale (41641/udp)
- VPN (Tailscale) : Gateway 网关仅通过 VPN mesh 可访问
- Docker 隔离 : DOCKER-USER iptables 链防止外部端口暴露
- Systemd 加固 : NoNewPrivileges 、 PrivateTmp 、 非特权用户

验证

测试外部攻击面 :

代码 : `nmap -p- YOUR_SERVER_IP`

应该显示仅端口 22 (SSH) 开放。所有其他服务 (Gateway 网关、Docker) 都被锁定。

Docker 可用性

Docker 用于智能体沙箱 (隔离的工具执行) , 而不是用于运行 Gateway 网关本身。Gateway 网关仅绑定到 localhost , 通过 Tailscale VPN 访问。

沙箱配置参见多智能体沙箱与工具。

手动安装

如果你更喜欢手动控制而非自动化 :

代码 : # 1. 安装先决条件

代码 : `sudo apt update && sudo apt install -y ansible git`

代码 : # 2. 克隆仓库

代码 : `git clone https://github.com/openclaw/openclaw-ansible.git`

代码 : `cd openclaw-ansible`

代码 : # 3. 安装 Ansible collections

代码 : `ansible-galaxy collection install -r requirements.yml`

代码 : # 4. 运行 playbook

代码 : `./run-playbook.sh`

代码 : # 或直接运行 (然后手动执行 /tmp/openclaw-setup.sh)

代码：# ansible-playbook playbook.yml --ask-become-pass

更新 OpenClaw

Ansible 安装程序设置 OpenClaw 为手动更新。标准更新流程参见更新。

要重新运行 Ansible playbook（例如，用于配置更改）：

代码：cd openclaw-ansible

代码：./run-playbook.sh

注意：这是幂等的，可以安全地多次运行。

故障排除

防火墙阻止了我的连接

如果你被锁定：

- 确保你可以先通过 Tailscale VPN 访问
- SSH 访问（端口 22）始终允许
- Gateway 网关仅通过 Tailscale 访问，这是设计如此

服务无法启动

代码：# 检查日志

代码：sudo journalctl -u openclaw -n 100

代码：# 验证权限

代码：sudo ls -la /opt/openclaw

代码：# 测试手动启动

代码：sudo -i -u openclaw

代码：cd ~/openclaw

代码：pnpm start

Docker 沙箱问题

代码：# 验证 Docker 正在运行

代码：sudo systemctl status docker

代码：# 检查沙箱镜像

代码：sudo docker images | grep openclaw-sandbox

代码：# 如果缺失则构建沙箱镜像

代码：cd /opt/openclaw/openclaw

代码：sudo -u openclaw ./scripts/sandbox-setup.sh

提供商登录失败

确保你以 openclaw 用户运行：

代码：sudo -i -u openclaw

代码：openclaw channels login

高级配置

详细的安全架构和故障排除：

- 安全架构
- 技术详情
- 故障排除指南

相关内容

- openclaw-ansible — 完整部署指南
- Docker — 容器化 Gateway 网关设置
- 沙箱隔离 — 智能体沙箱配置
- 多智能体沙箱与工具 — 每个智能体的隔离

2. Bun（实验性）

Bun（实验性）

目标：使用 Bun 运行此仓库（可选，不推荐用于 WhatsApp/Telegram），同时不偏离 pnpm 工作流。

不推荐用于 Gateway 网关运行时（WhatsApp/Telegram 存在 bug）。生产环境请使用 Node。

状态

- Bun 是一个可选的本地运行时，用于直接运行 TypeScript (`bun run ...`、`bun --watch ...`)。
- pnpm 是构建的默认工具，仍然完全支持（并被一些文档工具使用）。
- Bun 无法使用 `pnpm-lock.yaml` 并会忽略它。

安装

默认：

代码：`bun install`

注意：`bun.lock/bun.lockb` 被

`gitignore`，所以无论哪种方式都不会有仓库变动。如果你想不写入锁文件：

代码：`bun install --no-save`

构建/测试 (Bun)

代码：`bun run build`

代码：`bun run vitest run`

Bun 生命周期脚本（默认被阻止）

除非明确信任 (`bun pm untrusted / bun pm trust`)，Bun 可能会阻止依赖的生命周期脚本。

对于此仓库，通常被阻止的脚本不是必需的：

- `@whiskeysockets/baileys preinstall`：检查 Node 主版本 ≥ 20 （我们运行 Node 22+）。
- `protobufjs postinstall`：发出关于不兼容版本方案的警告（无构建产物）。

如果你遇到真正需要这些脚本的运行时问题，请明确信任它们：

代码：`bun pm trust @whiskeysockets/baileys protobufjs`

注意事项

- 一些脚本仍然硬编码 pnpm（例如 `docs:build`、`ui:`、`protocol:check`）。目前请通过 pnpm 运行这些脚本。

3. 开发渠道

开发渠道

最后更新：2026-01-21

OpenClaw 提供三个更新渠道：

- stable：npm dist-tag latest。
- beta：npm dist-tag beta（测试中的构建）。
- dev：main 的移动头（git）。npm dist-tag：dev（发布时）。

我们将构建发布到 beta，进行测试，然后将经过验证的构建提升到 latest，版本号不变——dist-tag 是 npm 安装的数据源。

切换渠道

Git checkout：

代码：openclaw update --channel stable

代码：openclaw update --channel beta

代码：openclaw update --channel dev

- stable/beta 检出最新匹配的标签（通常是同一个标签）。
- dev 切换到 main 并在上游基础上 rebase。

npm/pnpm 全局安装：

代码：openclaw update --channel stable

代码：openclaw update --channel beta

代码：openclaw update --channel dev

这会通过相应的 npm dist-tag（latest、beta、dev）进行更新。

当你使用 --channel 显式切换渠道时，OpenClaw 还会对齐安装方式：

- dev 确保有一个 git checkout（默认 ~/openclaw，可通过 OPENCLAW_GIT_DIR 覆盖），更新它，并从该 checkout 安装全局 CLI。
- stable/beta 使用匹配的 dist-tag 从 npm 安装。

提示：如果你想同时使用 stable + dev，保留两个克隆并将 Gateway 网关指向 stable 那个。

插件和渠道

当你使用 `openclaw update` 切换渠道时，OpenClaw 还会同步插件来源：

- dev 优先使用 git checkout 中的内置插件。
- stable 和 beta 恢复 npm 安装的插件包。

标签最佳实践

- 为你希望 git checkout 落在的发布版本打标签（vYYYY.M.D 或 vYYYY.M.D-<patch>）。
- 保持标签不可变：永远不要移动或重用标签。
- npm dist-tag 仍然是 npm 安装的数据源：
- latest → stable
- beta → 候选构建
- dev → main 快照（可选）

macOS 应用可用性

Beta 和 dev 构建可能不包含 macOS 应用发布。这没问题：

- git 标签和 npm dist-tag 仍然可以发布。
- 在发布说明或变更日志中注明"此 beta 无 macOS 构建"。

4. Docker（可选）

Docker（可选）

Docker 是可选的。仅当你想要容器化的 Gateway 网关或验证 Docker 流程时才使用它。

Docker 适合我吗？

- 是：你想要一个隔离的、可丢弃的 Gateway 网关环境，或在没有本地安装的主机上运行 OpenClaw。
- 否：你在自己的机器上运行，只想要最快的开发循环。请改用正常的安装流程。
- 沙箱注意事项：智能体沙箱隔离也使用 Docker，但它不需要完整的 Gateway 网关在 Docker 中运行。参阅沙箱隔离。

本指南涵盖：

- 容器化 Gateway 网关（完整的 OpenClaw 在 Docker 中）
- 每会话智能体沙箱（主机 Gateway 网关 + Docker 隔离的智能体工具）

沙箱隔离详情：沙箱隔离

要求

- Docker Desktop（或 Docker Engine）+ Docker Compose v2
- 足够的磁盘空间用于镜像 + 日志

容器化 Gateway 网关（Docker Compose）

快速开始（推荐）

从仓库根目录：

代码：`./docker-setup.sh`

此脚本：

- 构建 Gateway 网关镜像
- 运行新手引导向导
- 打印可选的提供商设置提示
- 通过 Docker Compose 启动 Gateway 网关
- 生成 Gateway 网关令牌并写入 `.env`

可选环境变量：

- `OPENCLAW_DOCKER_APT_PACKAGES` — 在构建期间安装额外的 apt 包
- `OPENCLAW_EXTRA_MOUNTS` — 添加额外的主机绑定挂载
- `OPENCLAW_HOME_VOLUME` — 在命名卷中持久化 `/home/node`

完成后：

- 在浏览器中打开`
- 将令牌粘贴到控制 UI（设置 → token）。
- 需要再次获取带令牌的 URL？运行 `docker compose run --rm openclaw-cli dashboard --no-open`。

它在主机上写入配置/工作区：

- `~/.openclaw/`
- `~/.openclaw/workspace`

在 VPS 上运行？参阅 Hetzner (Docker VPS) 。

手动流程 (compose)

代码：`docker build -t openclaw:local -f Dockerfile .`

代码：`docker compose run --rm openclaw-cli onboard`

代码：`docker compose up -d openclaw-gateway`

注意：从仓库根目录运行 `docker compose ...`。如果你启用了 `OPENCLAW_EXTRA_MOUNTS` 或 `OPENCLAW_HOME_VOLUME`，设置脚本会写入

`docker-compose.extra.yml`；在其他地方运行 Compose 时包含它：

代码：`docker compose -f docker-compose.yml -f docker-compose.extra.yml
<command>`

控制 UI 令牌 + 配对 (Docker)

如果你看到 "unauthorized" 或 "disconnected (1008): pairing required"，获取新的仪表板链接并批准浏览器设备：

代码：`docker compose run --rm openclaw-cli dashboard --no-open`

代码：`docker compose run --rm openclaw-cli devices list`

代码：`docker compose run --rm openclaw-cli devices approve <requestId>`

更多详情：仪表板，设备。

额外挂载 (可选)

如果你想将额外的主机目录挂载到容器中，在运行 `docker-setup.sh` 之前设置 `OPENCLAW_EXTRA_MOUNTS`。这接受逗号分隔的 Docker 绑定挂载列表，并通过生成 `docker-compose.extra.yml` 将它们应用到 `openclaw-gateway` 和 `openclaw-cli`。

示例：

代码：`./docker-setup.sh`

注意：

- 路径必须在 macOS/Windows 上与 Docker Desktop 共享。
- 如果你编辑 `OPENCLAW_EXTRA_MOUNTS`，重新运行 `docker-setup.sh` 以重新生成额外的 compose 文件。

- `docker-compose.extra.yml` 是生成的。不要手动编辑它。

持久化整个容器 home（可选）

如果你想让 `/home/node` 在容器重建后持久化，通过 `OPENCLAW_HOME_VOLUME` 设置一个命名卷。这会创建一个 Docker 卷并将其挂载到 `/home/node`，同时保持标准的配置/工作区绑定挂载。这里使用命名卷（不是绑定路径）；对于绑定挂载，使用 `OPENCLAW_EXTRA_MOUNTS`。

示例：

代码：`./docker-setup.sh`

你可以将其与额外挂载结合使用：

代码：`./docker-setup.sh`

注意：

- 如果你更改 `OPENCLAW_HOME_VOLUME`，重新运行 `docker-setup.sh` 以重新生成额外的 compose 文件。
- 命名卷会持久化直到使用 `docker volume rm <name>` 删除。

安装额外的 apt 包（可选）

如果你需要镜像内的系统包（例如构建工具或媒体库），在运行 `docker-setup.sh` 之前设置 `OPENCLAW_DOCKER_APT_PACKAGES`。这会在镜像构建期间安装包，因此即使容器被删除它们也会持久化。

示例：

代码：`./docker-setup.sh`

注意：

- 这接受空格分隔的 apt 包名称列表。
- 如果你更改 `OPENCLAW_DOCKER_APT_PACKAGES`，重新运行 `docker-setup.sh` 以重建镜像。

高级用户/功能完整的容器（选择加入）

默认的 Docker 镜像是安全优先的，以非 root 的 node 用户运行。这保持了较小的攻击面，但这意味着：

- 运行时无法安装系统包
- 默认没有 Homebrew
- 没有捆绑的 Chromium/Playwright 浏览器

如果你想要功能更完整的容器，使用这些选择加入选项：

- 持久化 /home/node 以便浏览器下载和工具缓存能够保留：

代码：`./docker-setup.sh`

- 将系统依赖烘焙到镜像中（可重复 + 持久化）：

代码：`./docker-setup.sh`

- 不使用 npx 安装 Playwright 浏览器（避免 npm 覆盖冲突）：

代码：`docker compose run --rm openclaw-cli \`

代码：`node /app/node_modules/playwright-core/cli.js install chromium`

如果你需要 Playwright 安装系统依赖，使用 `OPENCLAW_DOCKER_APT_PACKAGES` 重建镜像，而不是在运行时使用 `--with-deps`。

- 持久化 Playwright 浏览器下载：

- 在 `docker-compose.yml` 中设置

`PLAYWRIGHT_BROWSERS_PATH=/home/node/.cache/ms-playwright`。

- 确保 /home/node 通过 `OPENCLAW_HOME_VOLUME` 持久化，或通过 `OPENCLAW_EXTRA_MOUNTS` 挂载 `/home/node/.cache/ms-playwright`。

权限 + EACCES

镜像以 node (uid 1000) 运行。如果你在 `/home/node/.openclaw` 上看到权限错误，确保你的主机绑定挂载由 uid 1000 拥有。

示例 (Linux 主机)：

代码：`sudo chown -R 1000:1000 /path/to/openclaw-config`
`/path/to/openclaw-workspace`

如果你选择以 root 运行以方便使用，你接受了安全权衡。

更快的重建（推荐）

要加速重建，排序你的 Dockerfile 以便依赖层被缓存。这避免了除非锁文件更改否则重新运行 pnpm install：

代码：FROM node:22-bookworm

代码：# 安装 Bun（构建脚本需要）

代码：RUN curl -fsSL https://bun.sh/install | bash

代码：ENV PATH="/root/.bun/bin:\${PATH}"

代码：RUN corepack enable

代码：WORKDIR /app

代码：# 缓存依赖，除非包元数据更改

代码：COPY package.json pnpm-lock.yaml pnpm-workspace.yaml .npmrc ./

代码：COPY ui/package.json ./ui/package.json

代码：COPY scripts ./scripts

代码：RUN pnpm install --frozen-lockfile

代码：COPY ..

代码：RUN pnpm build

代码：RUN pnpm ui:install

代码：RUN pnpm ui:build

代码：ENV NODE_ENV=production

代码：CMD ["node","dist/index.js"]

渠道设置（可选）

使用 CLI 容器配置渠道，然后在需要时重启 Gateway 网关。

WhatsApp（QR）：

代码：docker compose run --rm openclaw-cli channels login

Telegram（bot token）：

代码：docker compose run --rm openclaw-cli channels add --channel telegram --token "<token>"

Discord (bot token) :

代码：docker compose run --rm openclaw-cli channels add --channel discord --token "<token>"

文档：WhatsApp，Telegram，Discord

OpenAI Codex OAuth (无头 Docker)

如果你在向导中选择 OpenAI Codex OAuth，它会打开浏览器 URL 并尝试在`捕获回调。在 Docker 或无头设置中，该回调可能显示浏览器错误。复制你到达的完整重定向 URL 并将其粘贴回向导以完成认证。

健康检查

代码：docker compose exec openclaw-gateway node dist/index.js health --token "\$OPENCLAW_GATEWAY_TOKEN"

E2E 冒烟测试 (Docker)

代码：scripts/e2e/onboard-docker.sh

QR 导入冒烟测试 (Docker)

代码：pnpm test:docker:qr

注意

- Gateway 网关绑定默认为 lan 用于容器使用。
- Dockerfile CMD 使用 --allow-unconfigured；挂载的配置如果 gateway.mode 不是 local 仍会启动。覆盖 CMD 以强制执行检查。
- Gateway 网关容器是会话的真实来源 (~/.openclaw/agents/<agentId>/sessions/)。

智能体沙箱 (主机 Gateway 网关 + Docker 工具)

深入了解：沙箱隔离

它做什么

当启用 `agents.defaults.sandbox` 时，非主会话在 Docker 容器内运行工具。Gateway 网关保持在你的主机上，但工具执行是隔离的：

- `scope`：默认为 "agent"（每个智能体一个容器 + 工作区）
- `scope`："session" 用于每会话隔离
- 每作用域工作区文件夹挂载在 `/workspace`
- 可选的智能体工作区访问（`agents.defaults.sandbox.workspaceAccess`）
- 允许/拒绝工具策略（拒绝优先）
- 进站媒体被复制到活动沙箱工作区（`media/inbound/`），以便工具可以读取它（使用 `workspaceAccess: "rw"` 时，这会落在智能体工作区中）

警告：`scope: "shared"` 禁用跨会话隔离。所有会话共享一个容器和一个工作区。

每智能体沙箱配置文件（多智能体）

如果你使用多智能体路由，每个智能体可以覆盖沙箱 + 工具设置：`agents.list[].sandbox` 和 `agents.list[].tools`（加上 `agents.list[].tools.sandbox.tools`）。这让你可以在一个 Gateway 网关中运行混合访问级别：

- 完全访问（个人智能体）
- 只读工具 + 只读工作区（家庭/工作智能体）
- 无文件系统/shell 工具（公共智能体）

参阅多智能体沙箱与工具了解示例、优先级和故障排除。

默认行为

- 镜像：`openclaw-sandbox:bookworm-slim`
- 每个智能体一个容器
- 智能体工作区访问：`workspaceAccess: "none"`（默认）使用 `~/.openclaw/sandboxes`
- "ro" 保持沙箱工作区在 `/workspace` 并将智能体工作区只读挂载在 `/agent`（禁用 `write/edit/apply_patch`）
- "rw" 将智能体工作区读写挂载在 `/workspace`
- 自动清理：空闲 > 24h 或 年龄 > 7d
- 网络：默认为 `none`（如果需要出站则明确选择加入）
- 默认允许：`exec`、`process`、`read`、`write`、`edit`、`sessions_list`、`sessions_history`、`sessions_end`、`sessions_spawn`、`session_status`

- 默认拒绝：browser、canvas、nodes、cron、discord、gateway

启用沙箱隔离

如果你计划在 setupCommand 中安装包，请注意：

- 默认 docker.network 是 "none"（无出站）。
- readOnlyRoot: true 阻止包安装。
- user 必须是 root 才能运行 apt-get（省略 user 或设置 user: "0:0"）。

当 setupCommand（或 docker 配置）更改时，OpenClaw

会自动重建容器，除非容器是最近使用的（在约 5 分钟内）。热容器会记录警告，包含确切的 openclaw sandbox recreate ... 命令。

代码：{

代码：agents: {

代码：defaults: {

代码：sandbox: {

代码：mode: "non-main", // off | non-main | all

代码：scope: "agent", // session | agent | shared（默认为 agent）

代码：workspaceAccess: "none", // none | ro | rw

代码：workspaceRoot: "~/.openclaw/sandboxes",

代码：docker: {

代码：image: "openclaw-sandbox:bookworm-slim",

代码：workdir: "/workspace",

代码：readOnlyRoot: true,

代码：tmpfs: ["/tmp", "/var/tmp", "/run"],

代码：network: "none",

代码：user: "1000:1000",

代码：capDrop: ["ALL"],

代码：env: { LANG: "C.UTF-8" },

代码：setupCommand: "apt-get update && apt-get install -y git curl jq",

代码：pidsLimit: 256,

代码：memory: "1g",

代码：memorySwap: "2g",

代码：cpus: 1,

代码：ulimits: {

代码：nofile: { soft: 1024, hard: 2048 },

代码：nproc: 256,

代码：},

代码：seccompProfile: "/path/to/seccomp.json",

代码：apparmorProfile: "openclaw-sandbox",

代码：dns: ["1.1.1.1", "8.8.8.8"],

代码：extraHosts: ["internal.service:10.0.0.5"],

代码：},

代码：prune: {

代码：idleHours: 24, // 0 禁用空闲清理

代码：maxAgeDays: 7, // 0 禁用最大年龄清理

代码：},

代码：},

代码：},

代码：},

代码：tools: {

代码：sandbox: {

代码：tools: {

代码：allow: [

代码："exec",

代码："process",

代码："read",

代码："write",

代码："edit",

代码："sessions_list",

代码："sessions_history",

代码："sessions_send",

代码： "sessions_spawn",

代码： "session_status",

代码：],

代码： deny: ["browser", "canvas", "nodes", "cron", "discord", "gateway"],

代码： },

代码： },

代码： },

代码： }

加固选项位于 agents.defaults.sandbox.docker 下：network、user、pidsLimit、memory、memorySwap、cpus、ulimits、seccompProfile、apparmorProfile、dns、extraHosts。

多智能体：通过 agents.list[].sandbox.{docker,browser,prune}。按智能体覆盖 agents.defaults.sandbox.{docker,browser,prune}。（当 agents.defaults.sandbox.scope / agents.list[].sandbox.scope 是 "shared" 时忽略）。

构建默认沙箱镜像

代码： scripts/sandbox-setup.sh

这使用 Dockerfile.sandbox 构建 openclaw-sandbox:bookworm-slim。

沙箱通用镜像（可选）

如果你想要一个带有常见构建工具（Node、Go、Rust 等）的沙箱镜像，构建通用镜像：

代码： scripts/sandbox-common-setup.sh

这构建 openclaw-sandbox-common:bookworm-slim。要使用它：

代码： {

代码： agents: {

代码： defaults: {

代码： sandbox: { docker: { image: "openclaw-sandbox-common:bookworm-slim" } },

代码： },

代码： },

代码： }

沙箱浏览器镜像

要在沙箱内运行浏览器工具，构建浏览器镜像：

代码：`scripts/sandbox-browser-setup.sh`

这使用 `Dockerfile.sandbox-browser` 构建 `openclaw-sandbox-browser:bookworm-slim`。容器运行启用 CDP 的 Chromium 和可选的 noVNC 观察器（通过 Xvfb 有头）。

注意：

- 有头（Xvfb）比无头减少机器人阻止。
- 通过设置 `agents.defaults.sandbox.browser.headless=true` 仍然可以使用无头模式。
- 不需要完整的桌面环境（GNOME）；Xvfb 提供显示。

使用配置：

代码：{

代码： `agents: {`

代码： `defaults: {`

代码： `sandbox: {`

代码： `browser: { enabled: true },`

代码： `},`

代码： `},`

代码： `},`

代码： `}`

自定义浏览器镜像：

代码：{

代码： `agents: {`

代码： `defaults: {`

代码： `sandbox: { browser: { image: "my-openclaw-browser" } },`

代码： `},`

代码： `},`

代码： `}`

启用后，智能体接收：

- 沙箱浏览器控制 URL（用于 browser 工具）
- noVNC URL（如果启用且 headless=false）

记住：如果你使用工具允许列表，添加 browser（并从拒绝中移除它）否则工具仍然被阻止。

清理规则（agents.defaults.sandbox.prune）也适用于浏览器容器。

自定义沙箱镜像

构建你自己的镜像并将配置指向它：

代码：docker build -t my-openclaw-sbx -f Dockerfile.sandbox .

代码：{

代码：agents: {

代码：defaults: {

代码：sandbox: { docker: { image: "my-openclaw-sbx" } },

代码：},

代码：},

代码：}

工具策略（允许/拒绝）

- deny 优先于 allow。
- 如果 allow 为空：所有工具（除了 deny）都可用。
- 如果 allow 非空：只有 allow 中的工具可用（减去 deny）。

清理策略

两个选项：

- prune.idleHours：移除 X 小时未使用的容器（0 = 禁用）
- prune.maxAgeDays：移除超过 X 天的容器（0 = 禁用）

示例：

- 保留繁忙会话但限制生命周期：

idleHours: 24、maxAgeDays: 7

- 永不清理：

idleHours: 0 、maxAgeDays: 0

安全注意事项

- 硬隔离仅适用于工具（exec/read/write/edit/apply_patch）。
- 仅主机工具如 browser/camera/canvas 默认被阻止。
- 在沙箱中允许 browser 会破坏隔离（浏览器在主机上运行）。

故障排除

- 镜像缺失：使用 scripts/sandbox-setup.sh 构建或设置 agents.defaults.sandbox.docker.image。
- 容器未运行：它会按需为每个会话自动创建。
- 沙箱中的权限错误：将 docker.user 设置为与你挂载的工作区所有权匹配的 UID:GID（或 chown 工作区文件夹）。
- 找不到自定义工具：OpenClaw 使用 sh -lc（登录 shell）运行命令，这会 source /etc/profile 并可能重置 PATH。设置 docker.env.PATH 以在前面添加你的自定义工具路径（例如 /custom/bin:/usr/local/share/npm-global/bin），或在你的 Dockerfile 中在 /etc/profile.d/ 下添加脚本。

5. exe.dev

exe.dev

目标：OpenClaw Gateway 网关运行在 exe.dev VM 上，可从你的笔记本电脑通过以下地址访问：`

本页假设使用 exe.dev 的默认 exeuntu

镜像。如果你选择了不同的发行版，请相应地映射软件包。

新手快速路径

-
- 根据需要填写你的认证密钥/令牌
- 点击 VM 旁边的"Agent"，然后等待...
- ???
- 完成

你需要什么

- exe.dev 账户
- ssh exe.dev 访问 exe.dev 虚拟机（可选）

使用 Shelley 自动安装

Shelley，exe.dev 的智能体，可以使用我们的提示立即安装 OpenClaw。使用的提示如下：

代码：Set up OpenClaw (<https://docs.openclaw.ai/install>) on this VM. Use the non-interactive and accept-risk flags for openclaw onboarding. Add the supplied auth or token as needed. Configure nginx to forward from the default port 18789 to the root location on the default enabled site config, making sure to enable Websocket support. Pairing is done by "openclaw devices list" and "openclaw device approve <request id>". Make sure the dashboard shows that OpenClaw's health is OK. exe.dev handles forwarding from port 8000 to port 80/443 and HTTPS for us, so the final "reachable" should be <vm-name>.exe.xyz, without port specification.

手动安装

1) 创建 VM

从你的设备：

代码：ssh exe.dev new

然后连接：

代码：ssh <vm-name>.exe.xyz

提示：保持此 VM 有状态。OpenClaw 在 ~/.openclaw/ 和 ~/.openclaw/workspace/ 下存储状态。

2) 安装先决条件（在 VM 上）

代码：sudo apt-get update

代码：sudo apt-get install -y git curl jq ca-certificates openssl

3) 安装 OpenClaw

运行 OpenClaw 安装脚本：

代码：curl -fsSL https://openclaw.ai/install.sh | bash

4) 设置 nginx 将 OpenClaw 代理到端口 8000

编辑 /etc/nginx/sites-enabled/default：

代码：server {

代码：listen 80 default_server;

代码：listen [::]:80 default_server;

代码：listen 8000;

代码：listen [::]:8000;

代码：server_name _;

代码：location / {

代码：proxy_pass http://127.0.0.1:18789;

代码：proxy_http_version 1.1;

代码：# WebSocket 支持

代码：proxy_set_header Upgrade \$http_upgrade;

代码：proxy_set_header Connection "upgrade";

代码：# 标准代理头

代码：proxy_set_header Host \$host;

代码：proxy_set_header X-Real-IP \$remote_addr;

代码：proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;

代码：proxy_set_header X-Forwarded-Proto \$scheme;

代码：# 长连接超时设置

代码：proxy_read_timeout 86400s;

代码：proxy_send_timeout 86400s;

代码：}

代码：}

5) 访问 OpenClaw 并授予权限

访问 UI 输出)。使用 `openclaw devices list` 和 `openclaw devices approve <requestId>` 批准设备。如有疑问，从浏览器使用 Shelley！

远程访问

远程访问由 `exe.dev` 的认证处理。默认情况下，来自端口 8000 的 HTTP 流量通过电子邮件认证转发到`

更新

代码：`npm i -g openclaw@latest`

代码：`openclaw doctor`

代码：`openclaw gateway restart`

代码：`openclaw health`

指南：更新

6. Fly.io 部署

Fly.io 部署

目标：OpenClaw Gateway 网关运行在 Fly.io 机器上，具有持久存储、自动 HTTPS 和 Discord/渠道访问。

你需要什么

- 已安装 `flyctl` CLI
- Fly.io 账户（免费套餐可用）
- 模型认证：Anthropic API 密钥（或其他提供商密钥）
- 渠道凭证：Discord bot token、Telegram token 等

初学者快速路径

- 克隆仓库 → 自定义 `fly.toml`
- 创建应用 + 卷 → 设置密钥
- 使用 `fly deploy` 部署
- SSH 进入创建配置或使用 Control UI

1) 创建 Fly 应用

代码：# Clone the repo

代码：git clone https://github.com/openclaw/openclaw.git

代码：cd openclaw

代码：# Create a new Fly app (pick your own name)

代码：fly apps create my-openclaw

代码：# Create a persistent volume (1GB is usually enough)

代码：fly volumes create openclaw_data --size 1 --region iad

提示：选择离你近的区域。常见选项：lhr（伦敦）、iad（弗吉尼亚）、sjc（圣何塞）。

2) 配置 fly.toml

编辑 fly.toml 以匹配你的应用名称和需求。

安全注意事项：默认配置暴露公共 URL。对于没有公共 IP 的加固部署，参见私有部署或使用 fly.private.toml。

代码：app = "my-openclaw" # Your app name

代码：primary_region = "iad"

代码：[build]

代码：dockerfile = "Dockerfile"

代码：[env]

代码：NODE_ENV = "production"

代码：OPENCLAW_PREFER_PNPM = "1"

代码：OPENCLAW_STATE_DIR = "/data"

代码：NODE_OPTIONS = "--max-old-space-size=1536"

代码：[processes]

代码：app = "node dist/index.js gateway --allow-unconfigured --port 3000 --bind lan"

代码：[http_service]

代码：internal_port = 3000

代码：force_https = true

代码：auto_stop_machines = false

代码： auto_start_machines = true

代码： min_machines_running = 1

代码： processes = ["app"]

代码： [[vm]]

代码： size = "shared-cpu-2x"

代码： memory = "2048mb"

代码： [mounts]

代码： source = "openclaw_data"

代码： destination = "/data"

关键设置：

| 设置 | 原因 |

| ----- | ----- |

| --bind lan | 绑定到 0.0.0.0 以便 Fly 的代理可以访问 Gateway 网关 |

| --allow-unconfigured | 无需配置文件启动（你稍后会创建一个） |

| internal_port = 3000 | 必须与 --port 3000（或 OPENCLAW_GATEWAY_PORT）匹配以进行 Fly 健康检查 |

| memory = "2048mb" | 512MB 太小；推荐 2GB |

| OPENCLAW_STATE_DIR = "/data" | 在卷上持久化状态 |

3) 设置密钥

代码： # Required: Gateway token (for non-loopback binding)

代码： fly secrets set OPENCLAW_GATEWAY_TOKEN=\$(openssl rand -hex 32)

代码： # Model provider API keys

代码： fly secrets set ANTHROPIC_API_KEY=sk-ant-...

代码： # Optional: Other providers

代码： fly secrets set OPENAI_API_KEY=sk-...

代码： fly secrets set GOOGLE_API_KEY=...

代码： # Channel tokens

代码： fly secrets set DISCORD_BOT_TOKEN=MTQ...

注意事项：

- 非 loopback 绑定 (--bind lan) 出于安全需要 OPENCLAW_GATEWAY_TOKEN。
- 像对待密码一样对待这些 token。
- 优先使用环境变量而不是配置文件来存储所有 API 密钥和 token。这可以避免密钥出现在 openclaw.json 中，防止意外暴露或记录。

4) 部署

代码：fly deploy

首次部署构建 Docker 镜像（约 2-3 分钟）。后续部署更快。

部署后验证：

代码：fly status

代码：fly logs

你应该看到：

代码：[gateway] listening on ws://0.0.0.0:3000 (PID xxx)

代码：[discord] logged in to discord as xxx

5) 创建配置文件

SSH 进入机器创建正确的配置：

代码：fly ssh console

创建配置目录和文件：

代码：mkdir -p /data

代码：cat > /data/openclaw.json << 'EOF'

代码：{

代码："agents": {

代码："defaults": {

代码："model": {

代码："primary": "anthropic/claude-opus-4-5",

代码："fallbacks": ["anthropic/claude-sonnet-4-5", "openai/gpt-4o"]

```
代码： },
代码： "maxConcurrent": 4
代码： },
代码： "list": [
代码： {
代码： "id": "main",
代码： "default": true
代码： }
代码： ]
代码： },
代码： "auth": {
代码： "profiles": {
代码： "anthropic:default": { "mode": "token", "provider": "anthropic" },
代码： "openai:default": { "mode": "token", "provider": "openai" }
代码： }
代码： },
代码： "bindings": [
代码： {
代码： "agentId": "main",
代码： "match": { "channel": "discord" }
代码： }
代码： ],
代码： "channels": {
代码： "discord": {
代码： "enabled": true,
代码： "groupPolicy": "allowlist",
代码： "guilds": {
代码： "YOUR_GUILD_ID": {
代码： "channels": { "general": { "allow": true } } },
代码： "requireMention": false
```

代码： }

代码： }

代码： }

代码： },

代码： "gateway": {

代码： "mode": "local",

代码： "bind": "auto"

代码： },

代码： "meta": {

代码： "lastTouchedVersion": "2026.1.29"

代码： }

代码： }

代码： EOF

注意： 使用 OPENCLAW_STATE_DIR=/data 时，配置路径是 /data/openclaw.json。

注意： Discord token 可以来自：

- 环境变量：DISCORD_BOT_TOKEN（推荐用于密钥）
- 配置文件：channels.discord.token

如果使用环境变量，无需将 token 添加到配置中。Gateway 网关会自动读取 DISCORD_BOT_TOKEN。

重启以应用：

代码： exit

代码： fly machine restart <machine-id>

6) 访问 Gateway 网关

Control UI

在浏览器中打开：

代码： fly open

或访问`

粘贴你的 Gateway 网关 token（来自 OPENCLAW_GATEWAY_TOKEN 的那个）进行认证。

日志

代码：fly logs # Live logs

代码：fly logs --no-tail # Recent logs

SSH 控制台

代码：fly ssh console

故障排除

"App is not listening on expected address"

Gateway 网关绑定到 127.0.0.1 而不是 0.0.0.0。

修复：在 fly.toml 中的进程命令添加 --bind lan。

健康检查失败 / 连接被拒绝

Fly 无法在配置的端口上访问 Gateway 网关。

修复：确保 internal_port 与 Gateway 网关端口匹配（设置 --port 3000 或 OPENCLAW_GATEWAY_PORT=3000）。

OOM / 内存问题

容器持续重启或被终止。迹象：SIGABRT、v8::internal::Runtime_AllocateInYoungGeneration 或静默重启。

修复：在 fly.toml 中增加内存：

代码：[[vm]]

代码：memory = "2048mb"

或更新现有机器：

代码：fly machine update <machine-id> --vm-memory 2048 -y

注意：512MB 太小。1GB 可能可以工作但在负载或详细日志记录下可能 OOM。推荐 2GB。

Gateway 网关锁问题

Gateway 网关拒绝启动并显示"already running"错误。

这发生在容器重启但 PID 锁文件在卷上持久存在时。

修复：删除锁文件：

代码：fly ssh console --command "rm -f /data/gateway.*.lock"

代码：fly machine restart <machine-id>

锁文件在 /data/gateway..lock（不在子目录中）。

配置未被读取

如果使用 --allow-unconfigured，Gateway 网关会创建最小配置。你在 /data/openclaw.json 的自定义配置应该在重启时被读取。

验证配置是否存在：

代码：fly ssh console --command "cat /data/openclaw.json"

通过 SSH 写入配置

fly ssh console -C 命令不支持 shell 重定向。要写入配置文件：

代码：# Use echo + tee (pipe from local to remote)

代码：echo '{"your":"config"}' | fly ssh console -C "tee /data/openclaw.json"

代码：# Or use sftp

代码：fly sftp shell

代码：> put /local/path/config.json /data/openclaw.json

注意：如果文件已存在，fly sftp 可能会失败。先删除：

代码：fly ssh console --command "rm /data/openclaw.json"

状态未持久化

如果重启后丢失凭证或会话，状态目录正在写入容器文件系统。

修复：确保 fly.toml 中设置了 OPENCLAW_STATE_DIR=/data 并重新部署。

更新

代码：# Pull latest changes

代码：git pull

代码：# Redeploy

代码：fly deploy

代码：# Check health

代码：fly status

代码：fly logs

更新机器命令

如果你需要更改启动命令而无需完全重新部署：

代码：# Get machine ID

代码：fly machines list

代码：# Update command

代码：fly machine update <machine-id> --command "node dist/index.js gateway --port 3000 --bind lan" -y

代码：# Or with memory increase

代码：fly machine update <machine-id> --vm-memory 2048 --command "node dist/index.js gateway --port 3000 --bind lan" -y

注意：fly deploy 后，机器命令可能会重置为 fly.toml 中的内容。如果你进行了手动更改，请在部署后重新应用它们。

私有部署（加固）

默认情况下，Fly 分配公共 IP，使你的 Gateway 网关可通过访问。这很方便，但意味着你的部署可被互联网扫描器（Shodan、Censys 等）发现。

对于无公共暴露的加固部署，使用私有模板。

何时使用私有部署

- 你只进行出站调用/消息（无入站 webhooks）

- 你使用 ngrok 或 Tailscale 隧道处理任何 webhook 回调
- 你通过 SSH、代理或 WireGuard 而不是浏览器访问 Gateway 网关
- 你希望部署对互联网扫描器隐藏

设置

使用 fly.private.toml 替代标准配置：

代码：# Deploy with private config

代码：fly deploy -c fly.private.toml

或转换现有部署：

代码：# List current IPs

代码：fly ips list -a my-openclaw

代码：# Release public IPs

代码：fly ips release <public-ipv4> -a my-openclaw

代码：fly ips release <public-ipv6> -a my-openclaw

代码：# Switch to private config so future deploys don't re-allocate public IPs

代码：# (remove [http_service] or deploy with the private template)

代码：fly deploy -c fly.private.toml

代码：# Allocate private-only IPv6

代码：fly ips allocate-v6 --private -a my-openclaw

此后，fly ips list 应该只显示 private 类型的 IP：

代码：VERSION IP TYPE REGION

代码：v6 fdad:x:x:x::x private global

访问私有部署

由于没有公共 URL，使用以下方法之一：

选项 1：本地代理（最简单）

代码：# Forward local port 3000 to the app

代码：fly proxy 3000:3000 -a my-openclaw

代码：# Then open http://localhost:3000 in browser

选项 2：WireGuard VPN

代码：# Create WireGuard config (one-time)

代码：fly wireguard create

代码：# Import to WireGuard client, then access via internal IPv6

代码：# Example: http://[fdaa:x:x:x:x]:3000

选项 3：仅 SSH

代码：fly ssh console -a my-openclaw

私有部署的 Webhooks

如果你需要 webhook 回调（Twilio、Telnyx 等）而不暴露公共：

- ngrok 隧道 - 在容器内或作为 sidecar 运行 ngrok
- Tailscale Funnel - 通过 Tailscale 暴露特定路径
- 仅出站 - 某些提供商（Twilio）对于出站呼叫无需 webhooks 也能正常工作

使用 ngrok 的示例语音通话配置：

代码：{

代码： "plugins": {

代码： "entries": {

代码： "voice-call": {

代码： "enabled": true,

代码： "config": {

代码： "provider": "twilio",

代码： "tunnel": { "provider": "ngrok" }

代码： }

代码： }

代码： }

代码： }

代码： }

ngrok 隧道在容器内运行并提供公共 webhook URL，而不暴露 Fly 应用本身。

安全优势

方面	公共	私有
互联网扫描器	可发现	隐藏
直接攻击	可能	被阻止
Control UI 访问	浏览器	代理/VPN
Webhook 投递	直接	通过隧道

注意事项

- Fly.io 使用 x86 架构（非 ARM）
- Dockerfile 兼容两种架构
- 对于 WhatsApp/Telegram 新手引导，使用 fly ssh console
- 持久数据位于 /data 卷上
- Signal 需要 Java + signal-cli；使用自定义镜像并保持内存在 2GB+。

成本

使用推荐配置（shared-cpu-2x，2GB RAM）：

- 根据使用情况约 \$10-15/月
- 免费套餐包含一些配额

详情参见 Fly.io 定价。

7. 在 GCP Compute Engine 上运行 OpenClaw（Docker，生产 VPS 指南）

在 GCP Compute Engine 上运行 OpenClaw（Docker，生产 VPS 指南）

目标

使用 Docker 在 GCP Compute Engine VM 上运行持久化的 OpenClaw Gateway 网关，具有持久状态、内置二进制文件和安全的重启行为。

如果你想要"OpenClaw 24/7 大约 \$5-12/月"，这是在 Google Cloud 上的可靠设置。

价格因机器类型和区域而异；选择适合你工作负载的最小 VM，如果遇到 OOM 则扩容。

我们在做什么（简单说明）？

- 创建 GCP 项目并启用计费
- 创建 Compute Engine VM
- 安装 Docker（隔离的应用运行时）
- 在 Docker 中启动 OpenClaw Gateway 网关
- 在主机上持久化 ~/.openclaw + ~/.openclaw/workspace（重启/重建后仍保留）
- 通过 SSH 隧道从你的笔记本电脑访问控制 UI

Gateway 网关可以通过以下方式访问：

- 从你的笔记本电脑进行 SSH 端口转发
- 如果你自己管理防火墙和令牌，可以直接暴露端口

本指南使用 GCP Compute Engine 上的 Debian。

Ubuntu 也可以；请相应地映射软件包。

有关通用 Docker 流程，请参阅 Docker。

快速路径（有经验的运维人员）

- 创建 GCP 项目 + 启用 Compute Engine API
- 创建 Compute Engine VM（e2-small，Debian 12，20GB）
- SSH 进入 VM
- 安装 Docker
- 克隆 OpenClaw 仓库
- 创建持久化主机目录
- 配置 .env 和 docker-compose.yml
- 内置所需二进制文件、构建并启动

你需要什么

- GCP 账户 (e2-micro 符合免费层条件)
- 已安装 gcloud CLI (或使用 Cloud Console)
- 从你的笔记本电脑 SSH 访问
- 对 SSH + 复制/粘贴有基本了解
- 约 20-30 分钟
- Docker 和 Docker Compose
- 模型认证凭证
- 可选的提供商凭证
- WhatsApp QR
- Telegram bot token
- Gmail OAuth

1) 安装 gcloud CLI (或使用 Console)

选项 A : gcloud CLI (推荐用于自动化)

从 安装

初始化并认证 :

代码 : gcloud init

代码 : gcloud auth login

选项 B : Cloud Console

所有步骤都可以通过 的 Web UI 完成

2) 创建 GCP 项目

CLI :

代码 : gcloud projects create my-openclaw-project --name="OpenClaw Gateway"

代码 : gcloud config set project my-openclaw-project

在 启用计费 (Compute Engine 必需) 。

启用 Compute Engine API :

代码 : `gcloud services enable compute.googleapis.com`

Console :

- 转到 IAM & Admin > Create Project
- 命名并创建
- 为项目启用计费
- 导航到 APIs & Services > Enable APIs > 搜索 "Compute Engine API" > Enable

3) 创建 VM

机器类型 :

类型	配置	成本	说明
-----	-----	-----	-----
e2-small	2 vCPU, 2GB RAM	~\$12/月	推荐
e2-micro	2 vCPU (共享), 1GB RAM	符合免费层	负载下可能 OOM

CLI :

代码 : `gcloud compute instances create openclaw-gateway \`

代码 : `--zone=us-central1-a \`

代码 : `--machine-type=e2-small \`

代码 : `--boot-disk-size=20GB \`

代码 : `--image-family=debian-12 \`

代码 : `--image-project=debian-cloud`

Console :

- 转到 Compute Engine > VM instances > Create instance
- Name : openclaw-gateway
- Region : us-central1, Zone : us-central1-a
- Machine type : e2-small
- Boot disk : Debian 12, 20GB

- Create

4) SSH 进入 VM

CLI :

代码 : `gcloud compute ssh openclaw-gateway --zone=us-central1-a`

Console :

在 Compute Engine 仪表板中点击 VM 旁边的"SSH"按钮。

注意 : VM 创建后 SSH 密钥传播可能需要 1-2 分钟。如果连接被拒绝，请等待并重试。

5) 安装 Docker (在 VM 上)

代码 : `sudo apt-get update`

代码 : `sudo apt-get install -y git curl ca-certificates`

代码 : `curl -fsSL https://get.docker.com | sudo sh`

代码 : `sudo usermod -aG docker $USER`

注销并重新登录以使组更改生效 :

代码 : `exit`

然后重新 SSH 登录 :

代码 : `gcloud compute ssh openclaw-gateway --zone=us-central1-a`

验证 :

代码 : `docker --version`

代码 : `docker compose version`

6) 克隆 OpenClaw 仓库

代码 : `git clone https://github.com/openclaw/openclaw.git`

代码：cd openclaw

本指南假设你将构建自定义镜像以保证二进制文件持久化。

7) 创建持久化主机目录

Docker 容器是临时的。

所有长期状态必须存在于主机上。

代码：mkdir -p ~/.openclaw

代码：mkdir -p ~/.openclaw/workspace

8) 配置环境变量

在仓库根目录创建 .env。

代码：OPENCLAW_IMAGE=openclaw:latest

代码：OPENCLAW_GATEWAY_TOKEN=change-me-now

代码：OPENCLAW_GATEWAY_BIND=lan

代码：OPENCLAW_GATEWAY_PORT=18789

代码：OPENCLAW_CONFIG_DIR=/home/\$USER/.openclaw

代码：OPENCLAW_WORKSPACE_DIR=/home/\$USER/.openclaw/workspace

代码：GOG_KEYRING_PASSWORD=change-me-now

代码：XDG_CONFIG_HOME=/home/node/.openclaw

生成强密钥：

代码：openssl rand -hex 32

不要提交此文件。

9) Docker Compose 配置

创建或更新 docker-compose.yml 。

代码：services:

代码：openclaw-gateway:

代码：image: \${OPENCLAW_IMAGE}

代码：build: .

代码：restart: unless-stopped

代码：env_file:

代码：- .env

代码：environment:

代码：- HOME=/home/node

代码：- NODE_ENV=production

代码：- TERM=xterm-256color

代码：- OPENCLAW_GATEWAY_BIND=\${OPENCLAW_GATEWAY_BIND}

代码：- OPENCLAW_GATEWAY_PORT=\${OPENCLAW_GATEWAY_PORT}

代码：- OPENCLAW_GATEWAY_TOKEN=\${OPENCLAW_GATEWAY_TOKEN}

代码：- GOG_KEYRING_PASSWORD=\${GOG_KEYRING_PASSWORD}

代码：- XDG_CONFIG_HOME=\${XDG_CONFIG_HOME}

代码：- PATH=/home/linuxbrew/.linuxbrew/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

代码：volumes:

代码：- \${OPENCLAW_CONFIG_DIR}:/home/node/.openclaw

代码：- \${OPENCLAW_WORKSPACE_DIR}:/home/node/.openclaw/workspace

代码：ports:

代码：# 推荐：在 VM 上保持 Gateway 网关仅绑定 loopback；通过 SSH 隧道访问。

代码：# 要公开暴露，移除 `127.0.0.1:` 前缀并相应配置防火墙。

代码：- "127.0.0.1:\${OPENCLAW_GATEWAY_PORT}:18789"

代码：# 可选：仅当你针对此 VM 运行 iOS/Android 节点并需要 Canvas 主机时。

代码：# 如果你公开暴露此端口，请阅读 /gateway/security 并相应配置防火墙。

代码：# - "18793:18793"

代码：command:

```
代码： [  
代码： "node",  
代码： "dist/index.js",  
代码： "gateway",  
代码： "--bind",  
代码： "${OPENCLAW_GATEWAY_BIND}",  
代码： "--port",  
代码： "${OPENCLAW_GATEWAY_PORT}",  
代码： ]
```

10) 将所需二进制文件内置到镜像中（关键）

在运行中的容器内安装二进制文件是一个陷阱。

任何在运行时安装的内容在重启后都会丢失。

所有 Skills 所需的外部二进制文件必须在镜像构建时安装。

以下示例仅显示三个常见的二进制文件：

- gog 用于 Gmail 访问
- goplaces 用于 Google Places
- wacli 用于 WhatsApp

这些是示例，不是完整列表。

你可以使用相同的模式安装任意数量的二进制文件。

如果你以后添加依赖额外二进制文件的新 Skills，你必须：

- 更新 Dockerfile
- 重建镜像
- 重启容器

示例 Dockerfile

```
代码：FROM node:22-bookworm
```

代码：RUN apt-get update && apt-get install -y socat && rm -rf /var/lib/apt/lists/*

代码：# 示例二进制文件 1：Gmail CLI

代码：RUN curl -L

https://github.com/steipete/gog/releases/latest/download/gog_Linux_x86_64.tar.gz \

代码：| tar -xz -C /usr/local/bin && chmod +x /usr/local/bin/gog

代码：# 示例二进制文件 2：Google Places CLI

代码：RUN curl -L https://github.com/steipete/goplaces/releases/latest/download/goplaces_Linux_x86_64.tar.gz \

代码：| tar -xz -C /usr/local/bin && chmod +x /usr/local/bin/goplaces

代码：# 示例二进制文件 3：WhatsApp CLI

代码：RUN curl -L

https://github.com/steipete/waccli/releases/latest/download/waccli_Linux_x86_64.tar.gz \

代码：| tar -xz -C /usr/local/bin && chmod +x /usr/local/bin/waccli

代码：# 使用相同的模式在下面添加更多二进制文件

代码：WORKDIR /app

代码：COPY package.json pnpm-lock.yaml pnpm-workspace.yaml .npmrc ./

代码：COPY ui/package.json ./ui/package.json

代码：COPY scripts ./scripts

代码：RUN corepack enable

代码：RUN pnpm install --frozen-lockfile

代码：COPY ..

代码：RUN pnpm build

代码：RUN pnpm ui:install

代码：RUN pnpm ui:build

代码：ENV NODE_ENV=production

代码：CMD ["node","dist/index.js"]

11) 构建并启动

代码：docker compose build

代码：docker compose up -d openclaw-gateway

验证二进制文件：

代码：docker compose exec openclaw-gateway which gog

代码：docker compose exec openclaw-gateway which goplaces

代码：docker compose exec openclaw-gateway which waccli

预期输出：

代码：/usr/local/bin/gog

代码：/usr/local/bin/goplaces

代码：/usr/local/bin/waccli

12) 验证 Gateway 网关

代码：docker compose logs -f openclaw-gateway

成功：

代码：[gateway] listening on ws://0.0.0.0:18789

13) 从你的笔记本电脑访问

创建 SSH 隧道以转发 Gateway 网关端口：

代码：gcloud compute ssh openclaw-gateway --zone=us-central1-a -- -L 18789:127.0.0.1:18789

在浏览器中打开：

、

粘贴你的 Gateway 网关令牌。

什么持久化在哪里（真实来源）

OpenClaw 在 Docker 中运行，但 Docker 不是真实来源。

所有长期状态必须在重启、重建和重启后仍然存在。

组件	位置	持久化机制	说明
Gateway 网关配置	/home/node/.openclaw/	主机卷挂载	包括 openclaw.json、令牌
模型认证配置文件	/home/node/.openclaw/	主机卷挂载	OAuth 令牌、API 密钥
Skill 配置	/home/node/.openclaw/skills/	主机卷挂载	Skill 级别状态
智能体工作区	/home/node/.openclaw/workspace/	主机卷挂载	代码和智能体产物
WhatsApp 会话	/home/node/.openclaw/	主机卷挂载	保留 QR 登录
Gmail 密钥环	/home/node/.openclaw/	主机卷 + 密码	需要 GOG_KEYRING_PASSWORD
外部二进制文件	/usr/local/bin/	Docker 镜像	必须在构建时内置
Node 运行时	容器文件系统	Docker 镜像	每次镜像构建时重建
OS 包	容器文件系统	Docker 镜像	不要在运行时安装
Docker 容器	临时	可重启	可以安全销毁

更新

在 VM 上更新 OpenClaw：

代码：cd ~/openclaw

代码：git pull

代码：docker compose build

代码：docker compose up -d

故障排除

SSH 连接被拒绝

VM 创建后 SSH 密钥传播可能需要 1-2 分钟。等待并重试。

OS Login 问题

检查你的 OS Login 配置文件：

代码：gcloud compute os-login describe-profile

确保你的账户具有所需的 IAM 权限（Compute OS Login 或 Compute OS Admin Login）。

内存不足（OOM）

如果使用 e2-micro 并遇到 OOM，升级到 e2-small 或 e2-medium：

代码：# 首先停止 VM

代码：gcloud compute instances stop openclaw-gateway --zone=us-central1-a

代码：# 更改机器类型

代码：gcloud compute instances set-machine-type openclaw-gateway \

代码：--zone=us-central1-a \

代码：--machine-type=e2-small

代码：# 启动 VM

代码：gcloud compute instances start openclaw-gateway --zone=us-central1-a

服务账户（安全最佳实践）

对于个人使用，你的默认用户账户就可以。

对于自动化或 CI/CD 管道，创建具有最小权限的专用服务账户：

- 创建服务账户：

代码：gcloud iam service-accounts create openclaw-deploy \

代码：--display-name="OpenClaw Deployment"

- 授予 Compute Instance Admin 角色（或更窄的自定义角色）：

代码：gcloud projects add-iam-policy-binding my-openclaw-project \

代码：--member="serviceAccount:openclaw-deploy@my-openclaw-project.iam.gserviceaccount.com" \

代码：--role="roles/compute.instanceAdmin.v1"

避免为自动化使用 Owner 角色。使用最小权限原则。

参阅 了解 IAM 角色详情。

下一步

- 设置消息渠道：渠道
- 将本地设备配对为节点：节点
- 配置 Gateway 网关：Gateway 网关配置

8. 在 Hetzner 上运行 OpenClaw (Docker，生产 VPS 指南)

在 Hetzner 上运行 OpenClaw (Docker，生产 VPS 指南)

目标

使用 Docker 在 Hetzner VPS 上运行持久的 OpenClaw Gateway 网关，带持久状态、内置二进制文件和安全的重启行为。

如果你想要"约 \$5 实现 OpenClaw 24/7"，这是最简单可靠的设置。

Hetzner 定价会变化；选择最小的 Debian/Ubuntu VPS，如果遇到 OOM 再扩容。

我们在做什么（简单说明）？

- 租用一台小型 Linux 服务器（Hetzner VPS）
- 安装 Docker（隔离的应用运行时）
- 在 Docker 中启动 OpenClaw Gateway 网关
- 在主机上持久化 ~/.openclaw + ~/.openclaw/workspace（重启/重建后保留）
- 通过 SSH 隧道从你的笔记本电脑访问控制 UI

Gateway 网关可以通过以下方式访问：

- 从你的笔记本电脑进行 SSH 端口转发
- 如果你自己管理防火墙和令牌，可以直接暴露端口

本指南假设在 Hetzner 上使用 Ubuntu 或 Debian。

如果你使用其他 Linux VPS，请相应地映射软件包。

通用 Docker 流程请参见 Docker。

快速路径（有经验的运维人员）

- 配置 Hetzner VPS
- 安装 Docker
- 克隆 OpenClaw 仓库
- 创建持久化主机目录
- 配置 .env 和 docker-compose.yml
- 将所需二进制文件烘焙到镜像中
- `docker compose up -d`
- 验证持久化和 Gateway 网关访问

你需要什么

- 具有 root 访问权限的 Hetzner VPS
- 从你的笔记本电脑进行 SSH 访问
- 基本熟悉 SSH + 复制/粘贴
- 约 20 分钟
- Docker 和 Docker Compose
- 模型认证凭证
- 可选的提供商凭证
- WhatsApp 二维码
- Telegram 机器人令牌
- Gmail OAuth

1) 配置 VPS

在 Hetzner 中创建一个 Ubuntu 或 Debian VPS 。

以 root 身份连接：

代码：`ssh root@YOUR_VPS_IP`

本指南假设 VPS 是有状态的。

不要将其视为一次性基础设施。

2) 安装 Docker（在 VPS 上）

代码：`apt-get update`

代码：`apt-get install -y git curl ca-certificates`

代码：`curl -fsSL https://get.docker.com | sh`

验证：

代码：`docker --version`

代码：`docker compose version`

3) 克隆 OpenClaw 仓库

代码：`git clone https://github.com/openclaw/openclaw.git`

代码：`cd openclaw`

本指南假设你将构建自定义镜像以保证二进制文件持久化。

4) 创建持久化主机目录

Docker 容器是临时的。

所有长期状态必须存储在主机上。

代码：`mkdir -p /root/.openclaw`

代码：`mkdir -p /root/.openclaw/workspace`

代码：# 将所有权设置为容器用户（uid 1000）：

代码：chown -R 1000:1000 /root/.openclaw

代码：chown -R 1000:1000 /root/.openclaw/workspace

5) 配置环境变量

在仓库根目录创建 .env。

代码：OPENCLAW_IMAGE=openclaw:latest

代码：OPENCLAW_GATEWAY_TOKEN=change-me-now

代码：OPENCLAW_GATEWAY_BIND=lan

代码：OPENCLAW_GATEWAY_PORT=18789

代码：OPENCLAW_CONFIG_DIR=/root/.openclaw

代码：OPENCLAW_WORKSPACE_DIR=/root/.openclaw/workspace

代码：GOG_KEYRING_PASSWORD=change-me-now

代码：XDG_CONFIG_HOME=/home/node/.openclaw

生成强密钥：

代码：openssl rand -hex 32

不要提交此文件。

6) Docker Compose 配置

创建或更新 docker-compose.yml。

代码：services:

代码：openclaw-gateway:

代码：image: \${OPENCLAW_IMAGE}

代码：build: .

代码：restart: unless-stopped

代码：env_file:

代码： - .env

代码： environment:

代码： - HOME=/home/node

代码： - NODE_ENV=production

代码： - TERM=xterm-256color

代码： - OPENCLAW_GATEWAY_BIND=\${OPENCLAW_GATEWAY_BIND}

代码： - OPENCLAW_GATEWAY_PORT=\${OPENCLAW_GATEWAY_PORT}

代码： - OPENCLAW_GATEWAY_TOKEN=\${OPENCLAW_GATEWAY_TOKEN}

代码： - GOG_KEYRING_PASSWORD=\${GOG_KEYRING_PASSWORD}

代码： - XDG_CONFIG_HOME=\${XDG_CONFIG_HOME}

代码： - PATH=/home/linuxbrew/.linuxbrew/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

代码： volumes:

代码： - \${OPENCLAW_CONFIG_DIR}:/home/node/.openclaw

代码： - \${OPENCLAW_WORKSPACE_DIR}:/home/node/.openclaw/workspace

代码： ports:

代码： # 推荐：在 VPS 上保持 Gateway 网关仅限 loopback；通过 SSH 隧道访问。

代码： # 要公开暴露，移除 `127.0.0.1:` 前缀并相应配置防火墙。

代码： - "127.0.0.1:\${OPENCLAW_GATEWAY_PORT}:18789"

代码： # 可选：仅当你对此 VPS 运行 iOS/Android 节点并需要 Canvas 主机时。

代码： # 如果你公开暴露此端口，请阅读 /gateway/security 并相应配置防火墙。

代码： # - "18793:18793"

代码： command:

代码： [

代码： "node",

代码： "dist/index.js",

代码： "gateway",

代码： "--bind",

代码： "\${OPENCLAW_GATEWAY_BIND}",

代码： "--port",

代码： "\${OPENCLAW_GATEWAY_PORT}",

代码：]

7) 将所需二进制文件烘焙到镜像中（关键）

在运行中的容器内安装二进制文件是一个陷阱。

任何在运行时安装的东西都会在重启时丢失。

所有 skills 所需的外部二进制文件必须在镜像构建时安装。

以下示例仅展示三个常见二进制文件：

- gog 用于 Gmail 访问
- goplaces 用于 Google Places
- wacli 用于 WhatsApp

这些是示例，不是完整列表。

你可以使用相同的模式安装任意数量的二进制文件。

如果你以后添加依赖额外二进制文件的新 skills，你必须：

- 更新 Dockerfile
- 重新构建镜像
- 重启容器

示例 Dockerfile

代码： FROM node:22-bookworm

代码： RUN apt-get update && apt-get install -y socat && rm -rf /var/lib/apt/lists/*

代码： # 示例二进制文件 1：Gmail CLI

代码： RUN curl -L

https://github.com/steipete/gog/releases/latest/download/gog_Linux_x86_64.tar.gz \

代码： | tar -xz -C /usr/local/bin && chmod +x /usr/local/bin/gog

代码： # 示例二进制文件 2：Google Places CLI

代码：RUN curl -L https://github.com/steipete/goplaces/releases/latest/download/goplaces_Linux_x86_64.tar.gz \

代码：| tar -xz -C /usr/local/bin && chmod +x /usr/local/bin/goplaces

代码：# 示例二进制文件 3：WhatsApp CLI

代码：RUN curl -L

https://github.com/steipete/waccli/releases/latest/download/waccli_Linux_x86_64.tar.gz \

代码：| tar -xz -C /usr/local/bin && chmod +x /usr/local/bin/waccli

代码：# 使用相同模式在下方添加更多二进制文件

代码：WORKDIR /app

代码：COPY package.json pnpm-lock.yaml pnpm-workspace.yaml .npmrc ./

代码：COPY ui/package.json ./ui/package.json

代码：COPY scripts ./scripts

代码：RUN corepack enable

代码：RUN pnpm install --frozen-lockfile

代码：COPY ..

代码：RUN pnpm build

代码：RUN pnpm ui:install

代码：RUN pnpm ui:build

代码：ENV NODE_ENV=production

代码：CMD ["node","dist/index.js"]

8) 构建并启动

代码：docker compose build

代码：docker compose up -d openclaw-gateway

验证二进制文件：

代码：docker compose exec openclaw-gateway which gog

代码：docker compose exec openclaw-gateway which goplaces

代码：docker compose exec openclaw-gateway which wacli

预期输出：

代码：/usr/local/bin/gog

代码：/usr/local/bin/goplaces

代码：/usr/local/bin/wacli

9) 验证 Gateway 网关

代码：docker compose logs -f openclaw-gateway

成功：

代码：[gateway] listening on ws://0.0.0.0:18789

从你的笔记本电脑：

代码：ssh -N -L 18789:127.0.0.1:18789 root@YOUR_VPS_IP

打开：

、

粘贴你的 Gateway 网关令牌。

持久化位置（事实来源）

OpenClaw 在 Docker 中运行，但 Docker 不是事实来源。

所有长期状态必须在重启、重建和重启后保留。

组件	位置	持久化机制	说明
Gateway 网关配置	/home/node/.openclaw/	主机卷挂载	包括 openclaw.json、令牌
模型认证配置文件	/home/node/.openclaw/	主机卷挂载	OAuth 令牌、API 密钥
Skill 配置	/home/node/.openclaw/skills/	主机卷挂载	Skill 级别状态
智能体工作区	/home/node/.openclaw/workspace/	主机卷挂载	代码和智能体产物

| WhatsApp 会话 | /home/node/.openclaw/ | 主机卷挂载 | 保留二维码登录 |

| Gmail 密钥环 | /home/node/.openclaw/ | 主机卷 + 密码 | 需要
GOG_KEYRING_PASSWORD |

| 外部二进制文件 | /usr/local/bin/ | Docker 镜像 | 必须在构建时烘焙 |

| Node 运行时 | 容器文件系统 | Docker 镜像 | 每次镜像构建时重建 |

| 操作系统包 | 容器文件系统 | Docker 镜像 | 不要在运行时安装 |

| Docker 容器 | 临时的 | 可重启 | 可以安全销毁 |

9. 安装

安装

除非有特殊情况，否则请使用安装器。它会设置 CLI 并运行新手引导。

快速安装（推荐）

代码：`curl -fsSL https://openclaw.ai/install.sh | bash`

Windows (PowerShell)：

代码：`iwr -useb https://openclaw.ai/install.ps1 | iex`

下一步（如果你跳过了新手引导）：

代码：`openclaw onboard --install-daemon`

系统要求

- Node ≥ 22
- macOS、Linux 或通过 WSL2 的 Windows
- pnpm 仅在从源代码构建时需要

选择安装路径

1) 安装器脚本（推荐）

通过 npm 全局安装 openclaw 并运行新手引导。

代码：`curl -fsSL https://openclaw.ai/install.sh | bash`

安装器标志：

代码：`curl -fsSL https://openclaw.ai/install.sh | bash -s -- --help`

详情：安装器内部原理。

非交互式（跳过新手引导）：

代码：`curl -fsSL https://openclaw.ai/install.sh | bash -s -- --no-onboard`

2) 全局安装（手动）

如果你已经有 Node：

代码：`npm install -g openclaw@latest`

如果你全局安装了 libvips（macOS 上通过 Homebrew 安装很常见）且 sharp 安装失败，请强制使用预构建二进制文件：

代码：`SHARP_IGNORE_GLOBAL_LIBVIPS=1 npm install -g openclaw@latest`

如果你看到 sharp: Please add node-gyp to your dependencies，要么安装构建工具（macOS：Xcode CLT + `npm install -g node-gyp`），要么使用上面的 `SHARP_IGNORE_GLOBAL_LIBVIPS=1` 变通方法来跳过原生构建。

或使用 pnpm：

代码：`pnpm add -g openclaw@latest`

代码：`pnpm approve-builds -g` # 批准 openclaw、node-llama-cpp、sharp 等

代码：`pnpm add -g openclaw@latest` # 重新运行以执行 postinstall 脚本

pnpm 需要显式批准带有构建脚本的包。在首次安装显示"Ignored build scripts"警告后，运行 `pnpm approve-builds -g` 并选择列出的包，然后重新运行安装以执行 postinstall 脚本。

然后：

代码：`openclaw onboard --install-daemon`

3) 从源代码（贡献者/开发）

代码：`git clone https://github.com/openclaw/openclaw.git`

代码：`cd openclaw`

代码：`pnpm install`

代码：pnpm ui:build # 首次运行时自动安装 UI 依赖

代码：pnpm build

代码：openclaw onboard --install-daemon

提示：如果你还没有全局安装，请通过 pnpm openclaw ... 运行仓库命令。

4) 其他安装选项

- Docker : Docker
- Nix : Nix
- Ansible : Ansible
- Bun (仅 CLI) : Bun

安装后

- 运行新手引导：openclaw onboard --install-daemon
- 快速检查：openclaw doctor
- 检查 Gateway 网关健康状态：openclaw status + openclaw health
- 打开仪表板：openclaw dashboard

安装方式：npm vs git（安装器）

安装器支持两种方式：

- npm（默认）：npm install -g openclaw@latest
- git：从 GitHub 克隆/构建并从源代码 checkout 运行

CLI 标志

代码：# 显式 npm

代码：curl -fsSL https://openclaw.ai/install.sh | bash -s -- --install-method npm

代码：# 从 GitHub 安装（源代码 checkout）

代码：curl -fsSL https://openclaw.ai/install.sh | bash -s -- --install-method git

常用标志：

- --install-method npm|git
- --git-dir <path>（默认：~/openclaw）

- `--no-git-update` (使用现有 checkout 时跳过 git pull)
- `--no-prompt` (禁用提示；CI/自动化中必需)
- `--dry-run` (打印将要执行的操作；不做任何更改)
- `--no-onboard` (跳过新手引导)

环境变量

等效的环境变量 (对自动化有用) :

- `OPENCLAW_INSTALL_METHOD=git|npm`
- `OPENCLAW_GIT_DIR=...`
- `OPENCLAW_GIT_UPDATE=0|1`
- `OPENCLAW_NO_PROMPT=1`
- `OPENCLAW_DRY_RUN=1`
- `OPENCLAW_NO_ONBOARD=1`
- `SHARP_IGNORE_GLOBAL_LIBVIPS=0|1` (默认: 1; 避免 sharp 针对系统 libvips 构建)

故障排除: 找不到 `openclaw` (PATH)

快速诊断:

代码: `node -v`

代码: `npm -v`

代码: `npm prefix -g`

代码: `echo "$PATH"`

如果 `$(npm prefix -g)/bin` (macOS/Linux) 或 `$(npm prefix -g)` (Windows) 不在 `echo "$PATH"` 的输出中, 你的 shell 无法找到全局 npm 二进制文件 (包括 openclaw)。

修复: 将其添加到你的 shell 启动文件 (zsh: `~/.zshrc`, bash: `~/.bashrc`) :

代码: `# macOS / Linux`

在 Windows 上, 将 `npm prefix -g` 的输出添加到你的 PATH。

然后打开新终端 (或在 zsh 中执行 `rehash` / 在 bash 中执行 `hash -r`)。

更新/卸载

- 更新：更新
- 迁移到新机器：迁移
- 卸载：卸载

10. 安装器内部机制

安装器内部机制

OpenClaw 提供两个安装器脚本（托管在 openclaw.ai）：

- 一 "推荐" 安装器（默认全局 npm 安装；也可从 GitHub 检出安装）
- 一 无需 root 权限的 CLI 安装器（安装到带有独立 Node 的前缀目录）
- 一 Windows PowerShell 安装器（默认 npm；可选 git 安装）

查看当前参数/行为，运行：

代码：`curl -fsSL https://openclaw.ai/install.sh | bash -s -- --help`

Windows (PowerShell) 帮助：

代码：`& ([scriptblock]::Create((iwr -useb https://openclaw.ai/install.ps1))) -?`

如果安装器完成但在新终端中找不到 openclaw，通常是 Node/npm PATH 问题。参见：安装。

install.sh（推荐）

功能概述：

- 检测操作系统（macOS / Linux / WSL）。
- 确保 Node.js 22+（macOS 通过 Homebrew；Linux 通过 NodeSource）。
- 选择安装方式：
- npm（默认）：`npm install -g openclaw@latest`
- git：克隆/构建源码检出并安装包装脚本
- 在 Linux 上：必要时将 npm 前缀切换到 `~/.npm-global`，以避免全局 npm 权限错误。
- 如果是升级现有安装：运行 `openclaw doctor --non-interactive`（尽力执行）。
- 对于 git 安装：安装/更新后运行 `openclaw doctor --non-interactive`（尽力执行）。
- 通过默认设置 `SHARP_IGNORE_GLOBAL_LIBVIPS=1` 来缓解 sharp 原生安装问题（避免使用系统 libvips 编译）。

如果你希望 sharp 链接到全局安装的 libvips（或你正在调试），请设置：

代码：SHARP_IGNORE_GLOBAL_LIBVIPS=0 curl -fsSL https://openclaw.ai/install.sh | bash

可发现性 / "git 安装"提示

如果你在已有的 OpenClaw 源码检出目录中运行安装器（通过 package.json + pnpm-workspace.yaml 检测），它会提示：

- 更新并使用此检出（git）
- 或迁移到全局 npm 安装（npm）

在非交互式上下文中（无 TTY / --no-prompt），你必须传入 --install-method git|npm（或设置 OPENCLAW_INSTALL_METHOD），否则脚本将以退出码 2 退出。

为什么需要 Git

--install-method git 路径（克隆 / 拉取）需要 Git。

对于 npm 安装，Git 通常不是必需的，但某些环境仍然需要它（例如通过 git URL 获取软件包或依赖时）。安装器目前会确保 Git 存在，以避免在全新发行版上出现 spawn git ENOENT 错误。

为什么在全新 Linux 上 npm 会报 `EACCES`

在某些 Linux 设置中（尤其是通过系统包管理器或 NodeSource 安装 Node 后），npm 的全局前缀指向 root 拥有的位置。此时 npm install -g ... 会报 EACCES / mkdir 权限错误。

install.sh 通过将前缀切换到以下位置来缓解此问题：

- ~/.npm-global（并在存在时将其添加到 ~/.bashrc / ~/.zshrc 的 PATH 中）

install-cli.sh（无需 root 权限的 CLI 安装器）

此脚本将 openclaw 安装到前缀目录（默认：~/.openclaw），同时在该前缀下安装专用的 Node 运行时，因此可以在不想改动系统 Node/npm 的机器上使用。

帮助：

代码：curl -fsSL https://openclaw.ai/install-cli.sh | bash -s -- --help

install.ps1（Windows PowerShell）

功能概述：

- 确保 Node.js 22+ (winget/Chocolatey/Scoop 或手动安装)。
- 选择安装方式：
- npm (默认) : `npm install -g openclaw@latest`
- git : 克隆/构建源码检出并安装包装脚本
- 在升级和 git 安装时运行 `openclaw doctor --non-interactive` (尽力执行)。

示例：

代码：`iwr -useb https://openclaw.ai/install.ps1 | iex`

代码：`iwr -useb https://openclaw.ai/install.ps1 | iex -InstallMethod git`

代码：`iwr -useb https://openclaw.ai/install.ps1 | iex -InstallMethod git -GitDir "C:\\openclaw"`

环境变量：

- `OPENCLAW_INSTALL_METHOD=git|npm`
- `OPENCLAW_GIT_DIR=...`

Git 要求：

如果你选择 `-InstallMethod git` 但未安装 Git，安装器会打印 Git for Windows 的链接（`

常见 Windows 问题：

- `npm error spawn git / ENOENT`：安装 Git for Windows 并重新打开 PowerShell，然后重新运行安装器。
- "openclaw" 不是可识别的命令：你的 npm 全局 bin 文件夹不在 PATH 中。大多数系统使用 `%AppData%\npm`。你也可以运行 `npm config get prefix` 并将 `\\bin` 添加到 PATH，然后重新打开 PowerShell。

11. 在 macOS 虚拟机上运行 OpenClaw (沙箱隔离)

在 macOS 虚拟机上运行 OpenClaw (沙箱隔离)

推荐默认方案 (大多数用户)

- 小型 Linux VPS 用于永久在线的 Gateway 网关，成本低。参见 VPS 托管。

- 专用硬件（Mac mini 或 Linux 机器）如果你想要完全控制和住宅 IP 用于浏览器自动化。许多网站会屏蔽数据中心 IP，所以本地浏览通常效果更好。
- 混合方案：将 Gateway 网关保持在廉价 VPS 上，当你需要浏览器/UI 自动化时，将你的 Mac 作为节点连接。参见节点和 Gateway 网关远程。

当你特别需要 macOS 独有功能（iMessage/BlueBubbles）或想要与日常 Mac 严格隔离时，使用 macOS VM。

macOS VM 选项

在你的 Apple Silicon Mac 上运行本地 VM（Lume）

使用 Lume 在你现有的 Apple Silicon Mac 上的沙箱 macOS VM 中运行 OpenClaw。

这为你提供：

- 隔离的完整 macOS 环境（你的主机保持干净）
- 通过 BlueBubbles 支持 iMessage（在 Linux/Windows 上不可能）
- 通过克隆 VM 即时重置
- 无需额外硬件或云成本

托管 Mac 提供商（云）

如果你想要云端的 macOS，托管 Mac 提供商也可以：

- MacStadium（托管 Mac）
- 其他托管 Mac 供应商也可以；按照他们的 VM + SSH 文档操作

一旦你有了 macOS VM 的 SSH 访问权限，继续下面的步骤 6。

快速路径（Lume，有经验的用户）

- 安装 Lume
- `lume create openclaw --os macos --ipsw latest`
- 完成设置助手，启用远程登录（SSH）
- `lume run openclaw --no-display`
- SSH 进入，安装 OpenClaw，配置渠道

- 完成

你需要什么 (Lume)

- Apple Silicon Mac (M1/M2/M3/M4)
- 主机上安装 macOS Sequoia 或更高版本
- 每个 VM 约 60 GB 可用磁盘空间
- 约 20 分钟

1) 安装 Lume

代码：`/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/trycua/cua/main/libs/lume/scripts/install.sh)"`

如果 `~/local/bin` 不在你的 PATH 中：

代码：`echo 'export PATH="$PATH:$HOME/.local/bin"' >> ~/.zshrc && source ~/.zshrc`

验证：

代码：`lume --version`

文档：Lume 安装

2) 创建 macOS VM

代码：`lume create openclaw --os macos --ipsw latest`

这会下载 macOS 并创建 VM。VNC 窗口会自动打开。

注意：下载可能需要一段时间，取决于你的网络连接。

3) 完成设置助手

在 VNC 窗口中：

- 选择语言和地区
- 跳过 Apple ID（或者如果你以后想要 iMessage 就登录）
- 创建用户账户（记住用户名和密码）
- 跳过所有可选功能

设置完成后，启用 SSH：

- 打开系统设置 → 通用 → 共享
- 启用"远程登录"

4) 获取 VM 的 IP 地址

代码：`lume get openclaw`

查找 IP 地址（通常是 192.168.64.x）。

5) SSH 进入 VM

代码：`ssh youruser@192.168.64.X`

将 `youruser` 替换为你创建的账户，IP 替换为你 VM 的 IP。

6) 安装 OpenClaw

在 VM 内：

代码：`npm install -g openclaw@latest`

代码：`openclaw onboard --install-daemon`

按照新手引导提示设置你的模型提供商（Anthropic、OpenAI 等）。

7) 配置渠道

编辑配置文件：

代码：nano ~/.openclaw/openclaw.json

添加你的渠道：

代码：{

代码： "channels": {

代码： "whatsapp": {

代码： "dmPolicy": "allowlist",

代码： "allowFrom": ["+15551234567"]

代码： },

代码： "telegram": {

代码： "botToken": "YOUR_BOT_TOKEN"

代码： }

代码： }

代码： }

然后登录 WhatsApp（扫描二维码）：

代码：openclaw channels login

8) 无头运行 VM

停止 VM 并在无显示器模式下重启：

代码：lume stop openclaw

代码：lume run openclaw --no-display

VM 在后台运行。OpenClaw 的守护进程保持 Gateway 网关运行。

检查状态：

代码：ssh youruser@192.168.64.X "openclaw status"

额外：iMessage 集成

这是在 macOS 上运行的杀手级功能。使用 BlueBubbles 将 iMessage 添加到 OpenClaw。

在 VM 内：

- 从 bluebubbles.app 下载 BlueBubbles
- 用你的 Apple ID 登录
- 启用 Web API 并设置密码
- 将 BlueBubbles webhooks 指向你的 Gateway 网关（示例：`

添加到你的 OpenClaw 配置：

代码：{

代码： "channels": {

代码： "bluebubbles": {

代码： "serverUrl": "http://localhost:1234",

代码： "password": "your-api-password",

代码： "webhookPath": "/bluebubbles-webhook"

代码： }

代码： }

代码： }

重启 Gateway 网关。现在你的智能体可以发送和接收 iMessage 了。

完整设置详情：BlueBubbles 渠道

保存黄金镜像

在进一步自定义之前，快照你的干净状态：

代码：lume stop openclaw

代码：lume clone openclaw openclaw-golden

随时重置：

代码：lume stop openclaw && lume delete openclaw

代码：lume clone openclaw-golden openclaw

代码：lume run openclaw --no-display

24/7 运行

通过以下方式保持 VM 运行：

- 保持你的 Mac 插电
- 在系统设置 → 节能中禁用睡眠
- 如需要使用 caffeinate

对于真正的永久在线，考虑专用 Mac mini 或小型 VPS。参见 VPS 托管。

故障排除

| 问题 | 解决方案 |

| ----- | ----- |

| 无法 SSH 进入 VM | 检查 VM 的系统设置中是否启用了"远程登录" |

| VM IP 未显示 | 等待 VM 完全启动，再次运行 lume get openclaw |

| 找不到 Lume 命令 | 将 ~/.local/bin 添加到你的 PATH |

| WhatsApp 二维码扫描失败 | 确保运行 openclaw channels login 时你是登录到 VM（而不是主机） |

相关文档

- VPS 托管
- 节点
- Gateway 网关远程
- BlueBubbles 渠道
- Lume 快速入门
- Lume CLI 参考
- 无人值守 VM 设置（高级）
- Docker 沙箱隔离（替代隔离方案）

12. 将 OpenClaw 迁移到新机器

将 OpenClaw 迁移到新机器

本指南将 OpenClaw Gateway 网关从一台机器迁移到另一台，无需重新进行新手引导。

迁移在概念上很简单：

- 复制状态目录（`$OPENCLAW_STATE_DIR`，默认：`~/.openclaw/`）——这包括配置、认证、会话和渠道状态。
- 复制你的工作区（默认 `~/.openclaw/workspace/`）——这包括你的智能体文件（记忆、提示等）。

但在配置文件、权限和部分复制方面有常见的陷阱。

开始之前（你要迁移什么）

1) 确定你的状态目录

大多数安装使用默认值：

- 状态目录：`~/.openclaw/`

但如果你使用以下方式，可能会不同：

- `--profile <name>`（通常变成 `~/.openclaw-<profile>/`）
- `OPENCLAW_STATE_DIR=/some/path`

如果你不确定，在旧机器上运行：

代码：`openclaw status`

在输出中查找 `OPENCLAW_STATE_DIR` / `profile` 的提及。如果你运行多个 Gateway 网关，对每个配置文件重复此操作。

2) 确定你的工作区

常见默认值：

- `~/.openclaw/workspace/`（推荐的工作区）
- 你创建的自定义文件夹

你的工作区是 `MEMORY.md`、`USER.md` 和 `memory/.md` 等文件所在的位置。

3) 了解你将保留什么

如果你复制两者——状态目录和工作区，你将保留：

- Gateway 网关配置 (openclaw.json)
- 认证配置文件 / API 密钥 / OAuth 令牌
- 会话历史 + 智能体状态
- 渠道状态 (例如 WhatsApp 登录/会话)
- 你的工作区文件 (记忆、Skills 笔记等)

如果你只复制工作区 (例如通过 Git)，你不会保留：

- 会话
- 凭证
- 渠道登录

这些存储在 \$OPENCLAW_STATE_DIR 下。

迁移步骤 (推荐)

步骤 0 — 备份 (旧机器)

在旧机器上，首先停止 Gateway 网关，这样文件不会在复制过程中发生变化：

代码：openclaw gateway stop

(可选但推荐) 归档状态目录和工作区：

代码：# 如果你使用配置文件或自定义位置，请调整路径

代码：cd ~

代码：tar -czf openclaw-state.tgz .openclaw

代码：tar -czf openclaw-workspace.tgz .openclaw/workspace

如果你有多个配置文件/状态目录 (例如

~/openclaw-main、~/openclaw-work)，分别归档每个。

步骤 1 — 在新机器上安装 OpenClaw

在新机器上，安装 CLI (如果需要还有 Node)：

- 参见：安装

在这个阶段，如果新手引导创建了一个新的 `~/.openclaw/` 也没关系 — 你将在下一步覆盖它。

步骤 2 — 将状态目录 + 工作区复制到新机器

复制两者：

- `$OPENCLAW_STATE_DIR`（默认 `~/.openclaw/`）
- 你的工作区（默认 `~/.openclaw/workspace/`）

常见方法：

- scp 压缩包并解压
- 通过 SSH 使用 `rsync -a`
- 外部驱动器

复制后，确保：

- 包含了隐藏目录（例如 `.openclaw/`）
- 文件所有权对于运行 Gateway 网关的用户是正确的

步骤 3 — 运行 Doctor（迁移 + 服务修复）

在新机器上：

代码：`openclaw doctor`

Doctor 是"安全可靠"的命令。它修复服务、应用配置迁移，并警告不匹配问题。

然后：

代码：`openclaw gateway restart`

代码：`openclaw status`

常见陷阱（以及如何避免）

陷阱：配置文件/状态目录不匹配

如果你在旧 Gateway 网关上使用了配置文件（或 `OPENCLAW_STATE_DIR`），而新 Gateway 网关使用了不同的配置，你会看到如下症状：

- 配置更改不生效
- 渠道丢失/已登出
- 会话历史为空

修复：使用你迁移的相同配置文件/状态目录运行 Gateway 网关/服务，然后重新运行：

代码：openclaw doctor

陷阱：只复制 `openclaw.json`

openclaw.json 是不够的。许多提供商在以下位置存储状态：

- \$OPENCLAW_STATE_DIR/credentials/
- \$OPENCLAW_STATE_DIR/agents/<agentId>/...

始终迁移整个 \$OPENCLAW_STATE_DIR 文件夹。

陷阱：权限/所有权

如果你以 root 身份复制或更改了用户，Gateway 网关可能无法读取凭证/会话。

修复：确保状态目录 + 工作区由运行 Gateway 网关的用户拥有。

陷阱：在远程/本地模式之间迁移

- 如果你的 UI (WebUI/TUI) 指向远程 Gateway 网关，远程主机拥有会话存储 + 工作区。
- 迁移你的笔记本电脑不会移动远程 Gateway 网关的状态。

如果你处于远程模式，请迁移 Gateway 网关主机。

陷阱：备份中的密钥

\$OPENCLAW_STATE_DIR 包含密钥 (API 密钥、OAuth 令牌、WhatsApp 凭证)。将备份视为生产密钥：

- 加密存储
- 避免通过不安全的渠道共享
- 如果怀疑泄露，轮换密钥

验证检查清单

在新机器上，确认：

- `openclaw status` 显示 Gateway 网关正在运行
- 你的渠道仍然连接（例如 WhatsApp 不需要重新配对）
- 仪表板打开并显示现有会话
- 你的工作区文件（记忆、配置）存在

相关内容

- Doctor
- Gateway 网关故障排除
- OpenClaw 在哪里存储数据？

13. Nix 安装

Nix 安装

使用 Nix 运行 OpenClaw 的推荐方式是通过 `nix-openclaw` — 一个开箱即用的 Home Manager 模块。

快速开始

将此粘贴给你的 AI 智能体（Claude、Cursor 等）：

代码：I want to set up nix-openclaw on my Mac.

代码：Repository: `github:openclaw/nix-openclaw`

代码：What I need you to do:

代码：1. Check if Determinate Nix is installed (if not, install it)

代码：2. Create a local flake at `~/code/openclaw-local` using `templates/agent-first/flake.nix`

代码：3. Help me create a Telegram bot (@BotFather) and get my chat ID (@userinfobot)

代码：4. Set up secrets (bot token, Anthropic key) - plain files at `~/secrets/` is fine

代码：5. Fill in the template placeholders and run `home-manager switch`

代码：6. Verify: `launchd` running, bot responds to messages

代码：Reference the `nix-openclaw` README for module options.

完整指南：github.com/openclaw/nix-openclaw

nix-openclaw 仓库是 Nix 安装的权威来源。本页只是一个快速概述。

你将获得

- Gateway 网关 + macOS 应用 + 工具 (whisper、spotify、cameras) — 全部固定版本
- 重启后仍能运行的 Launchd 服务
- 带有声明式配置的插件系统
- 即时回滚：home-manager switch --rollback

Nix 模式运行时行为

当设置 OPENCLAW_NIX_MODE=1 时 (nix-openclaw 会自动设置)：

OpenClaw 支持 Nix 模式，使配置确定性并禁用自动安装流程。

通过导出以下环境变量启用：

代码：OPENCLAW_NIX_MODE=1

在 macOS 上，GUI 应用不会自动继承 shell 环境变量。你也可以通过 defaults 启用 Nix 模式：

代码：defaults write bot.molt.mac openclaw.nixMode -bool true

配置 + 状态路径

OpenClaw 从 OPENCLAW_CONFIG_PATH 读取 JSON5 配置，并将可变数据存储在 OPENCLAW_STATE_DIR 中。

- OPENCLAW_STATE_DIR (默认：~/openclaw)
- OPENCLAW_CONFIG_PATH (默认：\$OPENCLAW_STATE_DIR/openclaw.json)

在 Nix 下运行时，将这些显式设置为 Nix 管理的位置，以便运行时状态和配置不会进入不可变存储。

Nix 模式下的运行时行为

- 自动安装和自我修改流程被禁用
- 缺失的依赖会显示 Nix 特定的修复消息
- 存在时 UI 会显示只读 Nix 模式横幅

打包注意事项（macOS）

macOS 打包流程期望在以下位置有一个稳定的 Info.plist 模板：

代码：`apps/macos/Sources/OpenClaw/Resources/Info.plist`

`scripts/package-mac-app.sh` 将此模板复制到应用包中并修补动态字段（bundle ID、版本/构建号、Git SHA、Sparkle 密钥）。这使 plist 对于 SwiftPM 打包和 Nix 构建保持确定性（它们不依赖完整的 Xcode 工具链）。

相关内容

- [nix-openclaw](#) — 完整设置指南
- [向导](#) — 非 Nix CLI 设置
- [Docker](#) — 容器化设置

14. Node.js

Node.js

该页面是英文文档的中文占位版本，完整内容请先参考英文版：[Node.js](#)。

15. northflank

通过一键模板在 Northflank 上部署 OpenClaw，然后在浏览器中完成设置。

这是最简单的"无需在服务器上使用终端"的方式：Northflank 为你运行 Gateway 网关，你通过 `/setup` 网页向导配置一切。

如何开始

- 点击 [Deploy OpenClaw](#) 打开模板。
- 如果你还没有账户，请创建一个 [Northflank](#) 账户。
- 点击 [Deploy OpenClaw now](#)。
- 设置必需的环境变量：`SETUP_PASSWORD`。
- 点击 [Deploy stack](#) 构建并运行 OpenClaw 模板。
- 等待部署完成，然后点击 [View resources](#)。
- 打开 OpenClaw 服务。
- 打开公开的 OpenClaw URL，在 `/setup` 完成设置。
- 在 `/openclaw` 打开控制面板 UI。

你将获得

- 托管的 OpenClaw Gateway 网关 + 控制面板 UI
- /setup 处的网页设置向导（无需终端命令）
- 通过 Northflank Volume (/data) 实现持久化存储，配置/凭据/工作区在重新部署后不会丢失

设置流程

- 访问 并输入你的 SETUP_PASSWORD`。
- 选择模型/认证提供商并粘贴你的密钥。
- （可选）添加 Telegram/Discord/Slack 令牌。
- 点击 Run setup。
- 在` 打开控制面板 UI。

如果 Telegram 私信设置为配对模式，设置向导可以审批配对码。

获取聊天令牌

Telegram 机器人令牌

- 在 Telegram 中给 @BotFather 发消息
- 运行 /newbot
- 复制令牌（格式如 123456789:AA...）
- 将其粘贴到 /setup 中

Discord 机器人令牌

- 前往
- New Application → 选择一个名称
- Bot → Add Bot
- 在 Bot → Privileged Gateway Intents 下启用 MESSAGE CONTENT INTENT（必需，否则机器人启动时会崩溃）
- 复制 Bot Token 并粘贴到 /setup 中
- 邀请机器人加入你的服务器（OAuth2 URL Generator；权限范围：bot、applications.commands）

16. railway

通过一键模板在 Railway 上部署 OpenClaw，并在浏览器中完成设置。

这是最简单的"无需在服务器上使用终端"的方式：Railway 为你运行 Gateway网关，你只需通过 /setup 网页向导完成所有配置。

快速检查清单（新用户）

- 点击下方的 Deploy on Railway。
- 添加一个挂载到 /data 的 Volume。
- 设置必需的变量（至少需要 SETUP_PASSWORD）。
- 在端口 8080 上启用 HTTP Proxy。
- 打开 ` 并完成向导。

一键部署

[Deploy on Railway](#)

部署完成后，在 Railway → 你的服务 → Settings → Domains 中找到你的公开 URL。

Railway 会：

- 为你生成一个域名（通常是 `
- 使用你绑定的自定义域名。

然后打开：

- ` — 设置向导（需密码保护）
- ` — 控制面板 UI

你将获得

- 托管的 OpenClaw Gateway网关 + 控制面板 UI
- /setup 网页设置向导（无需终端命令）
- 通过 Railway Volume (/data) 实现持久化存储，配置/凭证/工作区在重新部署后不会丢失
- 在 /setup/export 导出备份，方便日后从 Railway 迁移

必需的 Railway 设置

公共网络

为服务启用 HTTP Proxy。

- 端口：8080

Volume（必需）

挂载一个 Volume 到：

- /data

变量

在服务上设置以下变量：

- SETUP_PASSWORD（必需）
- PORT=8080（必需 — 必须与公共网络中的端口一致）
- OPENCLAW_STATE_DIR=/data/.openclaw（推荐）
- OPENCLAW_WORKSPACE_DIR=/data/workspace（推荐）
- OPENCLAW_GATEWAY_TOKEN（推荐；请视为管理员密钥）

设置流程

- 访问 并输入你的 SETUP_PASSWORD`。
- 选择模型/认证提供商并粘贴你的密钥。
- （可选）添加 Telegram/Discord/Slack 令牌。
- 点击 Run setup。

如果 Telegram 私信设置为配对模式，设置向导可以批准配对码。

获取聊天令牌

Telegram 机器人令牌

- 在 Telegram 中给 @BotFather 发消息
- 执行 /newbot
- 复制令牌（格式类似 123456789:AA...）

- 将其粘贴到 /setup 中

Discord 机器人令牌

- 前往
- New Application → 选择一个名称
- Bot → Add Bot
- 在 Bot → Privileged Gateway Intents 下启用 MESSAGE CONTENT INTENT（必需，否则机器人启动时会崩溃）
- 复制 Bot Token 并粘贴到 /setup 中
- 邀请机器人加入你的服务器（OAuth2 URL Generator ; scopes : bot 、 applications.commands）

备份与迁移

在以下地址下载备份：

- 、

这会导出你的 OpenClaw 状态和工作区，方便你迁移到其他主机而不丢失配置或记忆。

17. render

使用基础设施即代码方式在 Render 上部署 OpenClaw。内置的 render.yaml Blueprint 以声明式方式定义了你的整个技术栈——服务、磁盘、环境变量，让你只需一键即可完成部署，并将基础设施与代码一同进行版本管理。

前提条件

- 一个 Render 账户（提供免费套餐）
- 来自你首选模型提供商的 API 密钥

使用 Render Blueprint 部署

<a

href="

target="_blank"

rel="noreferrer"

部署到 Render

点击此链接将会：

- 根据本仓库根目录下的 render.yaml Blueprint 创建一个新的 Render 服务。
- 提示你设置 SETUP_PASSWORD
- 构建 Docker 镜像并部署

部署完成后，你的服务 URL 格式为`

了解 Blueprint

Render Blueprint 是定义基础设施的 YAML 文件。本仓库中的 render.yaml 配置了运行 OpenClaw 所需的一切：

代码：services:

代码：- type: web

代码：name: openclaw

代码：runtime: docker

代码：plan: starter

代码：healthCheckPath: /health

代码：envVars:

代码：- key: PORT

代码：value: "8080"

代码：- key: SETUP_PASSWORD

代码：sync: false # prompts during deploy

代码：- key: OPENCLAW_STATE_DIR

代码：value: /data/.openclaw

代码：- key: OPENCLAW_WORKSPACE_DIR

代码：value: /data/workspace

代码：- key: OPENCLAW_GATEWAY_TOKEN

代码：generateValue: true # auto-generates a secure token

代码：disk:

代码：name: openclaw-data

代码：mountPath: /data

代码：sizeGB: 1

使用的关键 Blueprint 功能：

功能	用途
runtime: docker	从仓库的 Dockerfile 进行构建
healthCheckPath	Render 监控 /health 并重启不健康的实例
sync: false	在部署时提示输入值（用于密钥）
generateValue: true	自动生成加密安全的值
disk	持久化存储，在重新部署后数据仍然保留

选择套餐

套餐	休眠机制	磁盘	适用场景
Free	空闲 15 分钟后休眠	不可用	测试、演示
Starter	永不休眠	1GB+	个人使用、小团队
Standard+	永不休眠	1GB+	生产环境、多渠道

Blueprint 默认使用 starter。如需使用免费套餐，请在你 fork 的 render.yaml 中将 plan: free（但请注意：没有持久化磁盘意味着每次部署后配置都会重置）。

部署完成后

完成设置向导

- 访问`
- 输入你的 SETUP_PASSWORD
- 选择模型提供商并粘贴你的 API 密钥
- 可选配置消息渠道（Telegram、Discord、Slack）
- 点击 Run setup

访问控制面板

Web 管理面板位于`

Render 仪表盘功能

日志

在 Dashboard → 你的服务 → Logs 中查看实时日志。可按以下类型筛选：

- 构建日志（Docker 镜像创建）
- 部署日志（服务启动）
- 运行时日志（应用输出）

Shell 访问

如需调试，可通过 Dashboard → 你的服务 → Shell 打开 shell 会话。持久化磁盘挂载在 /data。

环境变量

在 Dashboard → 你的服务 → Environment 中修改变量。更改会触发自动重新部署。

自动部署

如果你使用的是原始 OpenClaw 仓库，Render 不会自动部署你的 OpenClaw。要更新它，请在仪表盘中手动执行 Blueprint 同步。

自定义域名

- 前往 Dashboard → 你的服务 → Settings → Custom Domains
- 添加你的域名
- 按照指引配置 DNS（CNAME 指向 .onrender.com）
- Render 会自动配置 TLS 证书

扩展

Render 支持水平和垂直扩展：

- 垂直扩展：更改套餐以获取更多 CPU/内存
- 水平扩展：增加实例数量（Standard 套餐及以上）

对于 OpenClaw，垂直扩展通常就足够了。水平扩展需要粘性会话或外部状态管理。

备份与迁移

随时导出你的配置和工作区：

代码：<https://<your-service>.onrender.com/setup/export>

这将下载一个可移植的备份文件，你可以在任何 OpenClaw 主机上恢复。

故障排除

服务无法启动

在 Render 仪表盘中检查部署日志。常见问题：

- 缺少 SETUP_PASSWORD — Blueprint 会提示输入此值，但请确认已设置
- 端口不匹配 — 确保 PORT=8080 与 Dockerfile 暴露的端口一致

冷启动缓慢（免费套餐）

免费套餐的服务在 15 分钟无活动后会休眠。休眠后的首次请求需要几秒钟等待容器启动。升级到 Starter 套餐可实现始终在线。

重新部署后数据丢失

这发生在免费套餐上（无持久化磁盘）。升级到付费套餐，或通过 `/setup/export` 定期导出你的配置。

健康检查失败

Render 期望在 30 秒内从 `/health` 获得 200

响应。如果构建成功但部署失败，可能是服务启动耗时过长。请检查：

- 构建日志中是否有错误
- 容器是否能够通过 `docker build && docker run` 在本地正常运行

18. 卸载

卸载

两种方式：

- 如果 openclaw 仍已安装，使用简单方式。
- 如果 CLI 已删除但服务仍在运行，使用手动服务移除。

简单方式（CLI 仍已安装）

推荐：使用内置卸载程序：

代码：openclaw uninstall

非交互式（自动化 / npx）：

代码：openclaw uninstall --all --yes --non-interactive

代码：npx -y openclaw uninstall --all --yes --non-interactive

手动步骤（效果相同）：

- 停止 Gateway 网关服务：

代码：openclaw gateway stop

- 卸载 Gateway 网关服务（launchd/systemd/schtasks）：

代码：openclaw gateway uninstall

- 删除状态 + 配置：

代码：rm -rf "\${OPENCLAW_STATE_DIR:-\$HOME/.openclaw}"

如果你将 OPENCLAW_CONFIG_PATH 设置为状态目录外的自定义位置，也请删除该文件。

- 删除你的工作区（可选，移除智能体文件）：

代码：rm -rf ~/.openclaw/workspace

- 移除 CLI 安装（选择你使用的那个）：

代码：npm rm -g openclaw

代码：pnpm remove -g openclaw

代码：bun remove -g openclaw

- 如果你安装了 macOS 应用：

代码：rm -rf /Applications/OpenClaw.app

注意事项：

- 如果你使用了配置文件 (`--profile / OPENCLAW_PROFILE`)，对每个状态目录重复步骤 3 (默认为 `~/openclaw-<profile>`)。
- 在远程模式下，状态目录位于 Gateway 网关主机上，因此也需要在那里运行步骤 1-4。

手动服务移除 (CLI 未安装)

如果 Gateway 网关服务持续运行但 openclaw 缺失，请使用此方法。

macOS (launchd)

默认标签是 `bot.molt.gateway` (或 `bot.molt.<profile>`；旧版 `com.openclaw` 可能仍然存在)：

代码：`launchctl bootout gui/$UID/bot.molt.gateway`

代码：`rm -f ~/Library/LaunchAgents/bot.molt.gateway.plist`

如果你使用了配置文件，请将标签和 plist 名称替换为 `bot.molt.<profile>`。如果存在任何旧版 `com.openclaw.plist`，请将其移除。

Linux (systemd 用户单元)

默认单元名称是 `openclaw-gateway.service` (或 `openclaw-gateway-<profile>.service`)：

代码：`systemctl --user disable --now openclaw-gateway.service`

代码：`rm -f ~/.config/systemd/user/openclaw-gateway.service`

代码：`systemctl --user daemon-reload`

Windows (计划任务)

默认任务名称是 OpenClaw Gateway (或 OpenClaw Gateway (<profile>))。

任务脚本位于你的状态目录下。

代码：`schtasks /Delete /F /TN "OpenClaw Gateway"`

代码：`Remove-Item -Force "$env:USERPROFILE\openclaw\gateway.cmd"`

如果你使用了配置文件，请删除匹配的任务名称和 `~\openclaw-<profile>\gateway.cmd`。

普通安装 vs 源码检出

普通安装 (`install.sh / npm / pnpm / bun`)

如果你使用了 `install.ps1`，CLI 是通过 `npm install -g openclaw@latest`` 安装的。

使用 `npm rm -g openclaw` 移除（或 `pnpm remove -g` / `bun remove -g`，如果你是用那种方式安装的）。

源码检出（git clone）

如果你从仓库检出运行（`git clone + openclaw ... / bun run openclaw ...`）：

- 在删除仓库之前卸载 Gateway 网关服务（使用上面的简单方式或手动服务移除）。
- 删除仓库目录。
- 按上述方式移除状态 + 工作区。

19. 更新

更新

OpenClaw 发展迅速（尚未到"1.0"）。将更新视为发布基础设施：更新 → 运行检查 → 重启（或使用会重启的 `openclaw update`）→ 验证。

推荐：重新运行网站安装程序（原地升级）

首选的更新路径是重新运行网站上的安装程序。它会检测现有安装、原地升级，并在需要时运行 `openclaw doctor`。

代码：`curl -fsSL https://openclaw.ai/install.sh | bash`

说明：

- 如果你不想再次运行新手引导向导，添加 `--no-onboard`。
- 对于源码安装，使用：

代码：`curl -fsSL https://openclaw.ai/install.sh | bash -s -- --install-method git --no-onboard`

安装程序仅在仓库干净时才会执行 `git pull --rebase`。

- 对于全局安装，脚本底层使用 `npm install -g openclaw@latest`。
- 旧版说明：`clawdbot` 仍可作为兼容性垫片使用。

更新之前

- 了解你的安装方式：全局（`npm/pnpm`）还是源码（`git clone`）。
- 了解你的 Gateway 网关运行方式：前台终端还是受管理服务（`launchd/systemd`）。

- 快照你的定制内容：
- 配置：`~/.openclaw/openclaw.json`
- 凭证：`~/.openclaw/credentials/`
- 工作区：`~/.openclaw/workspace`

更新（全局安装）

全局安装（选择一个）：

代码：`npm i -g openclaw@latest`

代码：`pnpm add -g openclaw@latest`

我们不推荐将 Bun 用于 Gateway 网关运行时（WhatsApp/Telegram 有 bug）。

切换更新渠道（git + npm 安装）：

代码：`openclaw update --channel beta`

代码：`openclaw update --channel dev`

代码：`openclaw update --channel stable`

使用 `--tag <dist-tag|version>` 进行一次性安装指定标签/版本。

渠道语义和发布说明参见开发渠道。

注意：在 npm 安装上，Gateway 网关在启动时会记录更新提示（检查当前渠道标签）。通过 `update.checkOnStart: false` 禁用。

然后：

代码：`openclaw doctor`

代码：`openclaw gateway restart`

代码：`openclaw health`

说明：

- 如果你的 Gateway 网关作为服务运行，`openclaw gateway restart` 优于杀死 PID。
- 如果你固定在特定版本，参见下面的"回滚/固定"。

更新（``openclaw update``）

对于源码安装（git checkout），首选：

代码：openclaw update

它运行一个相对安全的更新流程：

- 需要干净的工作树。
- 切换到选定的渠道（标签或分支）。
- 获取并 rebase 到配置的上游（dev 渠道）。
- 安装依赖、构建、构建控制 UI，并运行 openclaw doctor。
- 默认重启 Gateway 网关（使用 --no-restart 跳过）。

如果你通过 npm/pnpm 安装（没有 git 元数据），openclaw update 将尝试通过你的包管理器更新。如果无法检测到安装，请改用"更新（全局安装）"。

更新（控制 UI / RPC）

控制 UI 有更新并重启（RPC：update.run）。它：

- 运行与 openclaw update 相同的源码更新流程（仅限 git checkout）。
- 写入带有结构化报告（stdout/stderr 尾部）的重启哨兵。
- 重启 Gateway 网关并向最后活跃的会话 ping 报告。

如果 rebase 失败，Gateway 网关会中止并在不应用更新的情况下重启。

更新（从源码）

从仓库 checkout：

首选：

代码：openclaw update

手动（大致等效）：

代码：git pull

代码：pnpm install

代码：pnpm build

代码：pnpm ui:build # 首次运行时自动安装 UI 依赖

代码：openclaw doctor

代码：openclaw health

说明：

- 当你运行打包的 openclaw 二进制文件（openclaw.mjs）或使用 Node 运行 dist/ 时，pnpm build 很重要。
- 如果你从仓库 checkout 运行而没有全局安装，CLI 命令使用 pnpm openclaw ...。
- 如果你直接从 TypeScript 运行（pnpm openclaw ...），通常不需要重新构建，但配置迁移仍然适用 → 运行 doctor。
- 在全局和 git 安装之间切换很容易：安装另一种方式，然后运行 openclaw doctor 以便将 Gateway 网关服务入口点重写为当前安装。

始终运行：`openclaw doctor`

Doctor 是"安全更新"命令。它故意很无聊：修复 + 迁移 + 警告。

注意：如果你是源码安装（git checkout），openclaw doctor 会提供先运行 openclaw update。

它通常做的事情：

- 迁移已弃用的配置键/旧版配置文件位置。
- 审计私信策略并对有风险的"开放"设置发出警告。
- 检查 Gateway 网关健康状况，可以提供重启。
- 检测并将旧版 Gateway 网关服务（launchd/systemd；旧版 schtasks）迁移到当前 OpenClaw 服务。
- 在 Linux 上，确保 systemd 用户 lingering（这样 Gateway 网关在登出后仍能存活）。

详情：Doctor

启动/停止/重启 Gateway 网关

CLI（无论操作系统都适用）：

代码：openclaw gateway status

代码：openclaw gateway stop

代码：openclaw gateway restart

代码：openclaw gateway --port 18789

代码：openclaw logs --follow

如果你使用受管理服务：

- macOS launchd（应用捆绑的 LaunchAgent）：launchctl kickstart -k gui/\$UID/bot.molt.gateway（使用 bot.molt.<profile>；旧版 com.openclaw. 仍然有效）
- Linux systemd 用户服务：systemctl --user restart openclaw-gateway[-<profile>].service
- Windows（WSL2）：systemctl --user restart openclaw-gateway[-<profile>].service
- launchctl/systemctl 仅在服务已安装时有效；否则运行 openclaw gateway install。

运行手册 + 确切的服务标签：Gateway 网关运行手册

回滚/固定（当出问题时）

固定（全局安装）

安装已知良好的版本（将 <version> 替换为最后可用的版本）：

代码：npm i -g openclaw@<version>

代码：pnpm add -g openclaw@<version>

提示：要查看当前发布的版本，运行 npm view openclaw version。

然后重启 + 重新运行 doctor：

代码：openclaw doctor

代码：openclaw gateway restart

按日期固定（源码）

选择某个日期的提交（示例："2026-01-01 时 main 的状态"）：

代码：git fetch origin

代码：git checkout "\$(git rev-list -n 1 --before="\2026-01-01\" origin/main)"

然后重新安装依赖 + 重启：

代码：pnpm install

代码：pnpm build

代码：openclaw gateway restart

如果你之后想回到最新版本：

代码：git checkout main

代码：git pull

如果你卡住了

- 再次运行 openclaw doctor 并仔细阅读输出（它通常会告诉你修复方法）。
- 查看：故障排除
- 在 Discord 上提问：

第三章：消息渠道接入

1. BlueBubbles (macOS REST)

BlueBubbles (macOS REST)

状态：内置插件，通过 HTTP 与 BlueBubbles macOS 服务器通信。由于其更丰富的 API 和更简便的设置，推荐用于 iMessage 集成，优于旧版 imsg 渠道。

概述

- 通过 BlueBubbles 辅助应用在 macOS 上运行 (bluebubbles.app)。
- 推荐/已测试版本：macOS Sequoia (15)。macOS Tahoe (26) 可用；但在 Tahoe 上编辑功能目前不可用，群组图标更新可能显示成功但实际未同步。
- OpenClaw 通过其 REST API 与之通信 (GET /api/v1/ping、POST /message/text、POST /chat/:id/)。
- 传入消息通过 webhook 到达；发出的回复、输入指示器、已读回执和 tapback 均为 REST 调用。
- 附件和贴纸作为进站媒体被接收（并在可能时呈现给智能体）。
- 配对/白名单的工作方式与其他渠道相同 (/channels/pairing 等)，使用 channels.bluebubbles.allowFrom + 配对码。
- 回应作为系统事件呈现，与 Slack/Telegram 类似，智能体可以在回复前"提及"它们。
- 高级功能：编辑、撤回、回复线程、消息效果、群组管理。

快速开始

- 在你的 Mac 上安装 BlueBubbles 服务器（按照 bluebubbles.app/install 的说明操作）。
- 在 BlueBubbles 配置中，启用 web API 并设置密码。
- 运行 openclaw onboard 并选择 BlueBubbles，或手动配置：

代码：{

代码： channels: {

代码： bluebubbles: {

代码： enabled: true,

代码： serverUrl: "http://192.168.1.100:1234",

代码： password: "example-password",

代码： webhookPath: "/bluebubbles-webhook",

代码：},

代码：},

代码：}

- 将 BlueBubbles webhook 指向你的 Gateway 网关（示例：`
- 启动 Gateway 网关；它将注册 webhook 处理程序并开始配对。

新手引导

BlueBubbles 可在交互式设置向导中使用：

代码：openclaw onboard

向导会提示输入：

- 服务器 URL（必填）：BlueBubbles 服务器地址（例如 `
- 密码（必填）：来自 BlueBubbles 服务器设置的 API 密码
- Webhook 路径（可选）：默认为 /bluebubbles-webhook
- 私信策略：配对、白名单、开放或禁用
- 白名单：电话号码、电子邮件或聊天目标

你也可以通过 CLI 添加 BlueBubbles：

代码：openclaw channels add bluebubbles --http-url http://192.168.1.100:1234
--password <password>

访问控制（私信 + 群组）

私信：

- 默认：channels.bluebubbles.dmPolicy = "pairing"。
- 未知发送者会收到配对码；在批准之前消息会被忽略（配对码 1 小时后过期）。
- 批准方式：
- openclaw pairing list bluebubbles
- openclaw pairing approve bluebubbles <CODE>
- 配对是默认的令牌交换方式。详情：配对

群组：

- `channels.bluebubbles.groupPolicy = open | allowlist | disabled`（默认：`allowlist`）。
- 当设置为 `allowlist` 时，`channels.bluebubbles.groupAllowFrom` 控制谁可以在群组中触发。

提及门控（群组）

BlueBubbles 支持群聊的提及门控，与 iMessage/WhatsApp 行为一致：

- 使用 `agents.list[].groupChat.mentionPatterns`（或 `messages.groupChat.mentionPatterns`）检测提及。
- 当群组启用 `requireMention` 时，智能体仅在被提及时响应。
- 来自授权发送者的控制命令会绕过提及门控。

单群组配置：

代码：{

代码： channels: {

代码： bluebubbles: {

代码： groupPolicy: "allowlist",

代码： groupAllowFrom: ["+15555550123"],

代码： groups: {

代码： "": { requireMention: true }, // 所有群组的默认设置

代码： "iMessage;-;chat123": { requireMention: false }, // 特定群组的覆盖设置

代码： },

代码： },

代码： },

代码： }

命令门控

- 控制命令（例如 `/config`、`/model`）需要授权。
- 使用 `allowFrom` 和 `groupAllowFrom` 确定命令授权。
- 授权发送者即使在群组中未被提及也可以运行控制命令。

输入状态 + 已读回执

- 输入指示器：在响应生成前和生成期间自动发送。

- 已读回执：由 channels.bluebubbles.sendReadReceipts 控制（默认：true）。
- 输入指示器：OpenClaw 发送输入开始事件；BlueBubbles 在发送或超时自动清除输入状态（通过 DELETE 手动停止不可靠）。

代码：{

代码： channels: {

代码： bluebubbles: {

代码： sendReadReceipts: false, // 禁用已读回执

代码： },

代码： },

代码： }

高级操作

BlueBubbles 在配置中启用时支持高级消息操作：

代码：{

代码： channels: {

代码： bluebubbles: {

代码： actions: {

代码： reactions: true, // tapback（默认：true）

代码： edit: true, // 编辑已发送消息（macOS 13+，在 macOS 26 Tahoe 上不可用）

代码： unsend: true, // 撤回消息（macOS 13+）

代码： reply: true, // 通过消息 GUID 进行回复线程

代码： sendWithEffect: true, // 消息效果（slam、loud 等）

代码： renameGroup: true, // 重命名群聊

代码： setGroupIcon: true, // 设置群聊图标/照片（在 macOS 26 Tahoe 上不稳定）

代码： addParticipant: true, // 将参与者添加到群组

代码： removeParticipant: true, // 从群组移除参与者

代码： leaveGroup: true, // 离开群聊

代码： sendAttachment: true, // 发送附件/媒体

代码： },

代码： },

代码：},

代码：}

可用操作：

- react：添加/移除 tapback 回应 (messageId、emoji、remove)
- edit：编辑已发送的消息 (messageId、text)
- unsend：撤回消息 (messageId)
- reply：回复特定消息 (messageId、text、to)
- sendWithEffect：带 iMessage 效果发送 (text、to、effectId)
- renameGroup：重命名群聊 (chatGuid、displayName)
- setGroupIcon：设置群聊图标/照片 (chatGuid、media) — 在 macOS 26 Tahoe 上不稳定 (API 可能返回成功但图标未同步)。
- addParticipant：将某人添加到群组 (chatGuid、address)
- removeParticipant：将某人从群组移除 (chatGuid、address)
- leaveGroup：离开群聊 (chatGuid)
- sendAttachment：发送媒体/文件 (to、buffer、filename、asVoice)
- 语音备忘录：将 asVoice: true 与 MP3 或 CAF 音频一起设置，以 iMessage 语音消息形式发送。BlueBubbles 在发送语音备忘录时会把 MP3 转换为 CAF。

消息 ID（短格式 vs 完整格式）

OpenClaw 可能会显示短消息 ID（例如 1、2）以节省 token。

- MessageSid / ReplyTold 可以是短 ID。
- MessageSidFull / ReplyToldFull 包含提供商的完整 ID。
- 短 ID 存储在内存中；它们可能在重启或缓存清除后过期。
- 操作接受短或完整的 messageId，但如果短 ID 不再可用将会报错。

对于持久化自动化和存储，请使用完整 ID：

- 模板：{{MessageSidFull}}、{{ReplyToldFull}}
- 上下文：入站负载中的 MessageSidFull / ReplyToldFull

参见配置了解模板变量。

分块流式传输

控制响应是作为单条消息发送还是分块流式传输：

代码：{

代码： channels: {

代码： bluebubbles: {

代码： blockStreaming: true, // 启用分块流式传输（默认关闭）

代码： },

代码： },

代码： }

媒体 + 限制

- 入站附件会被下载并存储在媒体缓存中。
- 媒体上限通过 channels.bluebubbles.mediaMaxMb 设置（默认：8 MB）。
- 出站文本按 channels.bluebubbles.textChunkLimit 分块（默认：4000 字符）。

配置参考

完整配置：配置

提供商选项：

- channels.bluebubbles.enabled：启用/禁用渠道。
- channels.bluebubbles.serverUrl：BlueBubbles REST API 基础 URL。
- channels.bluebubbles.password：API 密码。
- channels.bluebubbles.webhookPath：Webhook 端点路径（默认：/bluebubbles-webhook）。
- channels.bluebubbles.dmPolicy：pairing | allowlist | open | disabled（默认：pairing）。
- channels.bluebubbles.allowFrom：私信白名单（句柄、电子邮件、E.164 号码、chat_id:、chat_guid:）。
- channels.bluebubbles.groupPolicy：open | allowlist | disabled（默认：allowlist）。
- channels.bluebubbles.groupAllowFrom：群组发送者白名单。
- channels.bluebubbles.groups：单群组配置（requireMention 等）。
- channels.bluebubbles.sendReadReceipts：发送已读回执（默认：true）。

-
- channels.bluebubbles.blockStreaming：启用分块流式传输（默认：false；流式回复必需）。
- channels.bluebubbles.textChunkLimit：出站分块大小（字符）（默认：4000）。
- channels.bluebubbles.chunkMode：length（默认）仅在超过 textChunkLimit 时分割；newline 在长度分块前先按空行（段落边界）分割。
- channels.bluebubbles.mediaMaxMb：进站媒体上限（MB）（默认：8）。
- channels.bluebubbles.historyLimit：上下文的最大群组消息数（0 表示禁用）。
- channels.bluebubbles.dmHistoryLimit：私信历史限制。
- channels.bluebubbles.actions：启用/禁用特定操作。
- channels.bluebubbles.accounts：多账户配置。

相关全局选项：

- agents.list[].groupChat.mentionPatterns（或 messages.groupChat.mentionPatterns）。
- messages.responsePrefix。

地址 / 投递目标

优先使用 chat_guid 以获得稳定的路由：

- chat_guid:iMessage;-;+15555550123（群组推荐）
- chat_id:123
- chat_identifier:...
- 直接句柄：+15555550123、user@example.com
- 如果直接句柄没有现有的私信聊天，OpenClaw 将通过 POST /api/v1/chat/new 创建一个。这需要启用 BlueBubbles Private API。

安全性

- Webhook 请求通过比较 guid/password 查询参数或头部与 channels.bluebubbles.password 进行身份验证。来自 localhost 的请求也会被接受。
- 保持 API 密码和 webhook 端点的机密性（将它们视为凭证）。
- localhost 信任意味着同主机的反向代理可能无意中绕过密码验证。如果你使用代理 Gateway 网关，请在代理处要求身份验证并配置 gateway.trustedProxies。参见 Gateway 网关安全性。
- 如果将 BlueBubbles 服务器暴露在局域网之外，请启用 HTTPS + 防火墙规则。

故障排除

- 如果输入/已读事件停止工作，请检查 BlueBubbles webhook 日志并验证 Gateway 网关路径是否与 `channels.bluebubbles.webhookPath` 匹配。
- 配对码在一小时后过期；使用 `openclaw pairing list bluebubbles` 和 `openclaw pairing approve bluebubbles <code>`。
- 回应需要 BlueBubbles private API (POST `/api/v1/message/react`) ；确保服务器版本支持它。
- 编辑/撤回需要 macOS 13+ 和兼容的 BlueBubbles 服务器版本。在 macOS 26 (Tahoe) 上，由于 private API 变更，编辑功能目前不可用。
- 在 macOS 26 (Tahoe) 上群组图标更新可能不稳定：API 可能返回成功但新图标未同步。
- OpenClaw 会根据 BlueBubbles 服务器的 macOS 版本自动隐藏已知不可用的操作。如果在 macOS 26 (Tahoe) 上编辑仍然显示，请使用 `channels.bluebubbles.actions.edit=false` 手动禁用。
- 查看状态/健康信息：`openclaw status --all` 或 `openclaw status --deep`。

有关通用渠道工作流参考，请参阅渠道和插件指南。

2. 广播群组

广播群组

状态：实验性功能

版本：于 2026.1.9 版本新增

概述

广播群组允许多个智能体同时处理并响应同一条消息。这使你能够在单个 WhatsApp 群组或私信中创建协同工作的专业智能体团队——全部使用同一个手机号码。

当前范围：仅限 WhatsApp (web 渠道)。

广播群组在渠道白名单和群组激活规则之后进行评估。在 WhatsApp 群组中，这意味着广播会在 OpenClaw 正常回复时发生（例如：被提及时，具体取决于你的群组设置）。

使用场景

1. 专业智能体团队

部署多个具有原子化、专注职责的智能体：

代码：`Group: "Development Team"`

代码 : Agents:

代码 : - CodeReviewer (reviews code snippets)

代码 : - DocumentationBot (generates docs)

代码 : - SecurityAuditor (checks for vulnerabilities)

代码 : - TestGenerator (suggests test cases)

每个智能体处理相同的消息并提供其专业视角。

2. 多语言支持

代码 : Group: "International Support"

代码 : Agents:

代码 : - Agent_EN (responds in English)

代码 : - Agent_DE (responds in German)

代码 : - Agent_ES (responds in Spanish)

3. 质量保证 workflow

代码 : Group: "Customer Support"

代码 : Agents:

代码 : - SupportAgent (provides answer)

代码 : - QAAgent (reviews quality, only responds if issues found)

4. 任务自动化

代码 : Group: "Project Management"

代码 : Agents:

代码 : - TaskTracker (updates task database)

代码 : - TimeLogger (logs time spent)

代码 : - ReportGenerator (creates summaries)

配置

基本设置

添加一个顶层 broadcast 部分（与 bindings 同级）。键为 WhatsApp peer id :

- 群聊：群组 JID（例如 120363403215116621@g.us）
- 私信：E.164 格式的电话号码（例如 +15551234567）

代码：{

代码："broadcast": {

代码："120363403215116621@g.us": ["alfred", "baerbel", "assistant3"]

代码：}

代码：}

结果：当 OpenClaw 在此聊天中回复时，将运行所有三个智能体。

处理策略

控制智能体如何处理消息：

并行（默认）

所有智能体同时处理：

代码：{

代码："broadcast": {

代码："strategy": "parallel",

代码："120363403215116621@g.us": ["alfred", "baerbel"]

代码：}

代码：}

顺序

智能体按顺序处理（后一个等待前一个完成）：

代码：{

代码："broadcast": {

代码："strategy": "sequential",

代码："120363403215116621@g.us": ["alfred", "baerbel"]

代码：}

代码：}

完整示例

```
代码：{
代码： "agents": {
代码： "list": [
代码： {
代码： "id": "code-reviewer",
代码： "name": "Code Reviewer",
代码： "workspace": "/path/to/code-reviewer",
代码： "sandbox": { "mode": "all" }
代码： },
代码： {
代码： "id": "security-auditor",
代码： "name": "Security Auditor",
代码： "workspace": "/path/to/security-auditor",
代码： "sandbox": { "mode": "all" }
代码： },
代码： {
代码： "id": "docs-generator",
代码： "name": "Documentation Generator",
代码： "workspace": "/path/to/docs-generator",
代码： "sandbox": { "mode": "all" }
代码： }
代码： ]
代码： },
代码： "broadcast": {
代码： "strategy": "parallel",
代码： "120363403215116621@g.us": ["code-reviewer", "security-auditor",
"docs-generator"],
代码： "120363424282127706@g.us": ["support-en", "support-de"],
代码： "+15555550123": ["assistant", "logger"]
代码： }
```


代码：}

工作原理

消息流程

- 接收消息 到达 WhatsApp 群组
- 广播检查：系统检查 peer ID 是否在 broadcast 中
- 如果在广播列表中：
- 所有列出的智能体处理该消息
- 每个智能体有自己的会话键和隔离的上下文
- 智能体并行处理（默认）或顺序处理
- 如果不在广播列表中：
- 应用正常路由（第一个匹配的绑定）

注意：广播群组不会绕过渠道白名单或群组激活规则（提及/命令等）。它们只改变消息符合处理条件时运行哪些智能体。

会话隔离

广播群组中的每个智能体完全独立维护：

- 会话键（agent:alfred:whatsapp:group:120363... vs agent:baerbel:whatsapp:group:120363...）
- 对话历史（智能体看不到其他智能体的消息）
- 工作空间（如果配置了则使用独立的沙箱）
- 工具访问权限（不同的允许/拒绝列表）
- 记忆/上下文（独立的 IDENTITY.md、SOUL.md 等）
- 群组上下文缓冲区（用于上下文的最近群组消息）按 peer 共享，因此所有广播智能体在被触发时看到相同的上下文

这允许每个智能体拥有：

- 不同的个性
- 不同的工具访问权限（例如只读 vs 读写）
- 不同的模型（例如 opus vs sonnet）
- 不同的已安装 Skills

示例：隔离的会话

在群组 120363403215116621@g.us 中，智能体为 ["alfred", "baerbel"]：

Alfred 的上下文：

代码：Session: agent:alfred:whatsapp:group:120363403215116621@g.us

代码：History: [user message, alfred's previous responses]

代码：Workspace: /Users/pascal/openclaw-alfred/

代码：Tools: read, write, exec

Bärbel 的上下文：

代码：Session: agent:baerbel:whatsapp:group:120363403215116621@g.us

代码：History: [user message, baerbel's previous responses]

代码：Workspace: /Users/pascal/openclaw-baerbel/

代码：Tools: read only

最佳实践

1. 保持智能体专注

将每个智能体设计为具有单一、明确的职责：

代码：{

代码： "broadcast": {

代码： "DEV_GROUP": ["formatter", "linter", "tester"]

代码： }

代码： }

好的做法： 每个智能体只有一个任务

不好的做法： 一个通用的"dev-helper"智能体

2. 使用描述性名称

明确每个智能体的功能：

代码：{

代码： "agents": {

代码： "security-scanner": { "name": "Security Scanner" },

代码： "code-formatter": { "name": "Code Formatter" },

代码： "test-generator": { "name": "Test Generator" }

代码： }

代码： }

3. 配置不同的工具访问权限

只给智能体提供它们需要的工具：

代码： {

代码： "agents": {

代码： "reviewer": {

代码： "tools": { "allow": ["read", "exec"] } // Read-only

代码： },

代码： "fixer": {

代码： "tools": { "allow": ["read", "write", "edit", "exec"] } // Read-write

代码： }

代码： }

代码： }

4. 监控性能

当有多个智能体时，请考虑：

- 使用 "strategy": "parallel"（默认）以提高速度
- 将广播群组限制在 5-10 个智能体
- 为较简单的智能体使用较快的模型

5. 优雅地处理失败

智能体独立失败。一个智能体的错误不会阻塞其他智能体：

代码： Message → [Agent A , Agent B error, Agent C]

代码： Result: Agent A and C respond, Agent B logs error

兼容性

提供商

广播群组目前支持：

- WhatsApp（已实现）
- Telegram（计划中）
- Discord（计划中）
- Slack（计划中）

路由

广播群组与现有路由一起工作：

代码：{

代码："bindings": [

代码：{

代码："match": {"channel": "whatsapp", "peer": {"kind": "group", "id": "GROUP_A"}},

代码："agentId": "alfred"

代码：}

代码：],

代码："broadcast": {

代码："GROUP_B": ["agent1", "agent2"]

代码：}

代码：}

- GROUP_A：只有 alfred 响应（正常路由）
- GROUP_B：agent1 和 agent2 都响应（广播）

优先级：broadcast 优先于 bindings。

故障排除

智能体不响应

检查：

- 智能体 ID 存在于 agents.list 中
- Peer ID 格式正确（例如 120363403215116621@g.us）
- 智能体不在拒绝列表中

调试：

代码：tail -f ~/.openclaw/logs/gateway.log | grep broadcast

只有一个智能体响应

原因：Peer ID 可能在 bindings 中但不在 broadcast 中。

修复：添加到广播配置或从绑定中移除。

性能问题

如果智能体较多时速度较慢：

- 减少每个群组的智能体数量
- 使用较轻的模型（sonnet 而非 opus）
- 检查沙箱启动时间

示例

示例 1：代码审查团队

代码：{

代码："broadcast": {

代码："strategy": "parallel",

代码："120363403215116621@g.us": [

代码："code-formatter",

代码："security-scanner",

代码："test-coverage",

代码："docs-checker"

代码：]

代码：},

代码："agents": {

```
代码： "list": [  
代码： {  
代码： "id": "code-formatter",  
代码： "workspace": "~/agents/formatter",  
代码： "tools": { "allow": ["read", "write"] }  
代码： },  
代码： {  
代码： "id": "security-scanner",  
代码： "workspace": "~/agents/security",  
代码： "tools": { "allow": ["read", "exec"] }  
代码： },  
代码： {  
代码： "id": "test-coverage",  
代码： "workspace": "~/agents/testing",  
代码： "tools": { "allow": ["read", "exec"] }  
代码： },  
代码： { "id": "docs-checker", "workspace": "~/agents/docs", "tools": { "allow": ["read"] } }  
代码： ]  
代码： }  
代码： }
```

用户发送： 代码片段

响应：

- code-formatter : "修复了缩进并添加了类型提示"
- security-scanner : "第 12 行存在 SQL 注入漏洞"
- test-coverage : "覆盖率为 45%，缺少错误情况的测试"
- docs-checker : "函数 process_data 缺少文档字符串"

示例 2：多语言支持

```
代码： {  
代码： "broadcast": {
```

代码： "strategy": "sequential",

代码： "+15555550123": ["detect-language", "translator-en", "translator-de"]

代码： },

代码： "agents": {

代码： "list": [

代码： { "id": "detect-language", "workspace": "~/agents/lang-detect" },

代码： { "id": "translator-en", "workspace": "~/agents/translate-en" },

代码： { "id": "translator-de", "workspace": "~/agents/translate-de" }

代码：]

代码： }

代码： }

API 参考

配置模式

代码： interface OpenClawConfig {

代码： broadcast?: {

代码： strategy?: "parallel" | "sequential";

代码： [peerId: string]: string[];

代码： };

代码： }

字段

- strategy（可选）：如何处理智能体
- "parallel"（默认）：所有智能体同时处理
- "sequential"：智能体按数组顺序处理
- [peerId]：WhatsApp 群组 JID、E.164 号码或其他 peer ID
- 值：应处理消息的智能体 ID 数组

限制

- 最大智能体数：无硬性限制，但 10 个以上智能体可能会较慢
- 共享上下文：智能体看不到彼此的响应（设计如此）

- 消息顺序：并行响应可能以任意顺序到达
- 速率限制：所有智能体都计入 WhatsApp 速率限制

未来增强

计划中的功能：

- [] 共享上下文模式（智能体可以看到彼此的响应）
- [] 智能体协调（智能体可以相互发信号）
- [] 动态智能体选择（根据消息内容选择智能体）
- [] 智能体优先级（某些智能体先于其他智能体响应）

另请参阅

- 多智能体配置
- 路由配置
- 会话管理

3. 渠道与路由

渠道与路由

OpenClaw

将回复路由回消息来源的渠道。模型不会选择渠道；路由是确定性的，由主机配置控制。

关键术语

- 渠道：whatsapp、telegram、discord、slack、signal、imessage、webchat。
- AccountId：每个渠道的账户实例（在支持的情况下）。
- AgentId：隔离的工作区 + 会话存储（"大脑"）。
- SessionKey：用于存储上下文和控制并发的桶键。

会话键格式（示例）

私信会折叠到智能体的主会话：

- agent:<agentId>:<mainKey>（默认：agent:main:main）

群组和渠道按渠道隔离：

- 群组：agent:<agentId>:<channel>:group:<id>
- 渠道/房间：agent:<agentId>:<channel>:channel:<id>

线程：

- Slack/Discord 线程会在基础键后追加 :thread:<threadId>。
- Telegram 论坛主题在群组键中嵌入 :topic:<topicId>。

示例：

- agent:main:telegram:group:-1001234567890:topic:42
- agent:main:discord:channel:123456:thread:987654

路由规则（如何选择智能体）

路由为每条入站消息选择一个智能体：

- 精确对端匹配（bindings 中的 peer.kind + peer.id）。
- Guild 匹配（Discord）通过 guildId。
- Team 匹配（Slack）通过 teamId。
- 账户匹配（渠道上的 accountId）。
- 渠道匹配（该渠道上的任意账户）。
- 默认智能体（agents.list[].default，否则取列表第一项，兜底为 main）。

匹配到的智能体决定使用哪个工作区和会话存储。

广播组（运行多个智能体）

广播组允许你为同一对端运行多个智能体，在 OpenClaw 正常回复时触发（例如：在 WhatsApp 群组中，经过提及/激活门控之后）。

配置：

代码：{

代码：broadcast: {

代码：strategy: "parallel",

代码："120363403215116621@g.us": ["alfred", "baerbel"],

代码："+15555550123": ["support", "logger"],

代码：},

代码：}

参见：广播组。

配置概览

- agents.list：命名的智能体定义（工作区、模型等）。
- bindings：将进站渠道/账户/对端映射到智能体。

示例：

代码：{

代码：agents: {

代码：list: [{ id: "support", name: "Support", workspace: "~/.openclaw/workspace-support" }],

代码：},

代码：bindings: [

代码：{ match: { channel: "slack", teamId: "T123" }, agentId: "support" },

代码：{ match: { channel: "telegram", peer: { kind: "group", id: "-100123" } }, agentId: "support" },

代码：],

代码：}

会话存储

会话存储位于状态目录下（默认 ~/.openclaw）：

- ~/.openclaw/agents/<agentId>/sessions/sessions.json
- JSONL 记录文件与存储位于同一目录

你可以通过 session.store 和 {agentId} 模板来覆盖存储路径。

WebChat 行为

WebChat 连接到所选智能体，并默认使用该智能体的主会话。因此，WebChat 让你可以在一个地方查看该智能体的跨渠道上下文。

回复上下文

入站回复包含：

- ReplyTold、ReplyToBody 和 ReplyToSender（在可用时）。
- 引用的上下文会以 [Replying to ...] 块的形式追加到 Body 中。

这在所有渠道中保持一致。

4. Discord (Bot API)

Discord (Bot API)

状态：已支持通过官方 Discord 机器人网关进行私信和服务器文字频道通信。

快速设置（新手）

- 创建 Discord 机器人并复制机器人令牌。
- 在 Discord 应用设置中启用 Message Content Intent（如果你计划使用允许列表或名称查找，还需启用 Server Members Intent）。
- 为 OpenClaw 设置令牌：
- 环境变量：DISCORD_BOT_TOKEN=...
- 或配置：channels.discord.token: "...".
- 如果两者都设置，配置优先（环境变量回退仅适用于默认账户）。
- 使用消息权限邀请机器人到你的服务器（如果你只想使用私信，可以创建一个私人服务器）。
- 启动 Gateway 网关。
- 私信访问默认采用配对模式；首次联系时需批准配对码。

最小配置：

代码：{

代码： channels: {

代码： discord: {

代码： enabled: true,

代码： token: "YOUR_BOT_TOKEN",

代码： },

代码： },

代码： }

目标

- 通过 Discord 私信或服务器频道与 OpenClaw 对话。
- 直接聊天会合并到智能体的主会话（默认 agent:main:main）；服务器频道保持隔离为 agent:<agentId>:discord:channel:<channelId>（显示名称使用 discord:<guildSlug>#<channelSlug>）。
- 群组私信默认被忽略；通过 channels.discord.dm.groupEnabled 启用，并可选择通过 channels.discord.dm.groupChannels 进行限制。
- 保持路由确定性：回复始终返回到消息来源的渠道。

工作原理

- 创建 Discord 应用程序 → Bot，启用你需要的意图（私信 + 服务器消息 + 消息内容），并获取机器人令牌。
- 使用所需权限邀请机器人到你的服务器，以便在你想使用的地方读取/发送消息。
- 使用 channels.discord.token 配置 OpenClaw（或使用 DISCORD_BOT_TOKEN 作为回退）。
- 运行 Gateway 网关；当令牌可用（配置优先，环境变量回退）且 channels.discord.enabled 不为 false 时，它会自动启动 Discord 渠道。
- 如果你更喜欢使用环境变量，设置 DISCORD_BOT_TOKEN（配置块是可选的）。
- 直接聊天：发送时使用 user:<id>（或 <@id> 提及）；所有对话都进入共享的 main 会话。纯数字 ID 是模糊的，会被拒绝。
- 服务器频道：发送时使用 channel:<channelId>。默认需要提及，可以按服务器或按频道设置。
- 直接聊天：默认通过 channels.discord.dm.policy 进行安全保护（默认："pairing"）。未知发送者会收到配对码（1 小时后过期）；通过 openclaw pairing approve discord <code> 批准。
- 要保持旧的"对任何人开放"行为：设置 channels.discord.dm.policy="open" 和 channels.discord.dm.allowFrom=[""]。
- 要使用硬编码允许列表：设置 channels.discord.dm.policy="allowlist" 并在 channels.discord.dm.allowFrom 中列出发送者。
- 要忽略所有私信：设置 channels.discord.dm.enabled=false 或 channels.discord.dm.policy="disabled"。
- 群组私信默认被忽略；通过 channels.discord.dm.groupEnabled 启用，并可选择通过 channels.discord.dm.groupChannels 进行限制。

- 可选服务器规则：设置 `channels.discord.guilds`，以服务器 ID（首选）或 slug 为键，并包含每个频道的规则。
- 可选原生命令：`commands.native` 默认为 "auto"（Discord/Telegram 开启，Slack 关闭）。使用 `channels.discord.commands.native: true|false|"auto"` 覆盖；false 会清除之前注册的命令。文本命令由 `commands.text` 控制，必须作为独立的 /... 消息发送。使用 `commands.useAccessGroups: false` 可跳过命令的访问组检查。
- 完整命令列表 + 配置：斜杠命令
- 可选服务器上下文历史：设置 `channels.discord.historyLimit`（默认 20，回退到 `messages.groupChat.historyLimit`）以在回复提及包含最近 N 条服务器消息作为上下文。设置 0 禁用。
- 表情反应：智能体可以通过 discord 工具触发表情反应（受 `channels.discord.actions` 控制）。
- 表情反应移除语义：参见 `/tools/reactions`。
- discord 工具仅在当前渠道是 Discord 时暴露。
- 原生命令使用隔离的会话键（`agent:<agentId>:discord:slash:<userId>`）而不是共享的 main 会话。

注意：名称 → ID 解析使用服务器成员搜索，需要 Server Members Intent；如果机器人无法搜索成员，请使用 ID 或 `<@id>` 提及。

注意：Slug 为小写，空格替换为 -。频道名称的 slug 不包含前导 #。

注意：服务器上下文 `[from:]` 行包含 `author.tag + id`，便于进行可提及的回复。

配置写入

默认情况下，允许 Discord 写入由 `/config set|unset` 触发的配置更新（需要 `commands.config: true`）。

禁用方式：

代码：{

代码： `channels: { discord: { configWrites: false } },`

代码：}

如何创建自己的机器人

这是在服务器（guild）频道（如 #help）中运行 OpenClaw 的"Discord 开发者门户"设置。

1) 创建 Discord 应用 + 机器人用户

- Discord 开发者门户 → Applications → New Application
- 在你的应用中：
- Bot → Add Bot
- 复制 Bot Token（这是你放入 DISCORD_BOT_TOKEN 的内容）

2) 启用 OpenClaw 需要的网关意图

Discord 会阻止"特权意图"，除非你明确启用它们。

在 Bot → Privileged Gateway Intents 中启用：

- Message Content Intent（在大多数服务器中读取消息文本所必需；没有它你会看到"Used disallowed intents"或机器人会连接但不响应消息）
- Server Members Intent（推荐；服务器中的某些成员/用户查找和允许列表匹配需要）

你通常不需要 Presence Intent。

3) 生成邀请 URL (OAuth2 URL Generator)

在你的应用中：OAuth2 → URL Generator

Scopes

- bot
- applications.commands（原生命令所需）

Bot Permissions（最小基线）

- View Channels
- Send Messages
- Read Message History
- Embed Links
- Attach Files
- Add Reactions（可选但推荐）
- Use External Emojis / Stickers（可选；仅当你需要时）

除非你在调试并完全信任机器人，否则避免使用 Administrator。

复制生成的 URL，打开它，选择你的服务器，然后安装机器人。

4) 获取 ID (服务器/用户/频道)

Discord 到处使用数字 ID ; OpenClaw 配置优先使用 ID 。

- Discord (桌面/网页) → 用户设置 → 高级 → 启用 开发者模式
- 右键点击 :
- 服务器名称 → 复制服务器 ID (服务器 ID)
- 频道 (例如 #help) → 复制频道 ID
- 你的用户 → 复制用户 ID

5) 配置 OpenClaw

令牌

通过环境变量设置机器人令牌 (服务器上推荐) :

- DISCORD_BOT_TOKEN=...

或通过配置 :

代码 : {

代码 : channels: {

代码 : discord: {

代码 : enabled: true,

代码 : token: "YOUR_BOT_TOKEN",

代码 : },

代码 : },

代码 : }

多账户支持 : 使用 channels.discord.accounts , 每个账户有自己的令牌和可选的 name 。参见 gateway/configuration 了解通用模式 。

允许列表 + 频道路由

示例"单服务器 , 只允许我 , 只允许 #help" :

代码 : {

代码 : channels: {

代码： discord: {
代码： enabled: true,
代码： dm: { enabled: false },
代码： guilds: {
代码： YOUR_GUILD_ID: {
代码： users: ["YOUR_USER_ID"],
代码： requireMention: true,
代码： channels: {
代码： help: { allow: true, requireMention: true },
代码： },
代码： },
代码： },
代码： retry: {
代码： attempts: 3,
代码： minDelayMs: 500,
代码： maxDelayMs: 30000,
代码： jitter: 0.1,
代码： },
代码： },
代码： },
代码： }

注意：

- requireMention: true 意味着机器人只在被提及时回复（推荐用于共享频道）。
- agents.list[].groupChat.mentionPatterns（或 messages.groupChat.mentionPatterns）对于服务器消息也算作提及。
- 多智能体覆盖：在 agents.list[].groupChat.mentionPatterns 上设置每个智能体的模式。
- 如果存在 channels，任何未列出的频道默认被拒绝。
- 使用 "" 频道条目在所有频道应用默认值；显式频道条目覆盖通配符。
- 话题继承父频道配置（允许列表、requireMention、Skills、提示词等），除非你显式添加话题频道 ID。

- 机器人发送的消息默认被忽略；设置 `channels.discord.allowBots=true` 允许它们（自己的消息仍被过滤）。
- 警告：如果你允许回复其他机器人（`channels.discord.allowBots=true`），请使用 `requireMention`、`channels.discord.guilds..channels.<id>.users` 允许列表和/或在 `AGENTS.md` 和 `SOUL.md` 中设置明确的防护措施来防止机器人之间的回复循环。

6) 验证是否工作

- 启动 Gateway 网关。
- 在你的服务器频道中发送：@Krill hello（或你的机器人名称）。
- 如果没有反应：查看下面的故障排除。

故障排除

- 首先：运行 `openclaw doctor` 和 `openclaw channels status --probe`（可操作的警告 + 快速审计）。
- "Used disallowed intents"：在开发者门户中启用 Message Content Intent（可能还需要 Server Members Intent），然后重启 Gateway 网关。
- 机器人连接但从不在服务器频道回复：
- 缺少 Message Content Intent，或
- 机器人缺少频道权限（View/Send/Read History），或
- 你的配置需要提及但你没有提及它，或
- 你的服务器/频道允许列表拒绝了该频道/用户。
- `requireMention: false` 但仍然没有回复：
- `channels.discord.groupPolicy` 默认为 `allowlist`；将其设置为 `"open"` 或在 `channels.discord.guilds` 下添加服务器条目（可选择在 `channels.discord.guilds.<id>.channels` 下列出频道以进行限制）。
- 如果你只设置了 `DISCORD_BOT_TOKEN` 而从未创建 `channels.discord` 部分，运行时会将 `groupPolicy` 默认为 `open`。添加 `channels.discord.groupPolicy`、`channels.defaults.groupPolicy` 或服务器/频道允许列表来锁定它。
- `requireMention` 必须位于 `channels.discord.guilds`（或特定频道）下。顶层的 `channels.discord.requireMention` 会被忽略。
- 权限审计（`channels status --probe`）只检查数字频道 ID。如果你使用 slug/名称作为 `channels.discord.guilds..channels` 键，审计无法验证权限。

- 私信不工作：channels.discord.dm.enabled=false、channels.discord.dm.policy="disabled"，或者你尚未被批准（channels.discord.dm.policy="pairing"）。
- Discord 中的执行审批：Discord 支持私信中执行审批的按钮 UI（允许一次 / 始终允许 / 拒绝）。/approve <id> ... 仅用于转发的审批，不会解析 Discord 的按钮提示。如果你看到 Failed to submit approval: Error: unknown approval id 或 UI 从未出现，请检查：
- 你的配置中有 channels.discord.execApprovals.enabled: true。
- 你的 Discord 用户 ID 在 channels.discord.execApprovals.approvers 中列出（UI 仅发送给审批者）。
- 使用私信提示中的按钮（Allow once、Always allow、Deny）。
- 参见执行审批和斜杠命令了解更广泛的审批和命令流程。

功能和限制

- 支持私信和服务器文字频道（话题被视为独立频道；不支持语音）。
- 打字指示器尽力发送；消息分块使用 channels.discord.textChunkLimit（默认 2000），并按行数分割长回复（channels.discord.maxLinesPerMessage，默认 17）。
- 可选换行分块：设置 channels.discord.chunkMode="newline" 以在空行（段落边界）处分割，然后再进行长度分块。
- 支持文件上传，最大 channels.discord.mediaMaxMb（默认 8 MB）。
- 默认服务器回复需要提及，以避免嘈杂的机器人。
- 当消息引用另一条消息时，会注入回复上下文（引用内容 + ID）。
- 原生回复线程默认关闭；使用 channels.discord.replyToMode 和回复标签启用。

重试策略

出站 Discord API 调用在速率限制（429）时使用 Discord retry_after（如果可用）进行重试，采用指数退避和抖动。通过 channels.discord.retry 配置。参见重试策略。

配置

代码：{

代码： channels: {

代码： discord: {

代码： enabled: true,

代码： token: "abc.123",

代码 : groupPolicy: "allowlist",

代码 : guilds: {

代码 : "": {

代码 : channels: {

代码 : general: { allow: true },

代码 : },

代码 : },

代码 : },

代码 : mediaMaxMb: 8,

代码 : actions: {

代码 : reactions: true,

代码 : stickers: true,

代码 : emojiUploads: true,

代码 : stickerUploads: true,

代码 : polls: true,

代码 : permissions: true,

代码 : messages: true,

代码 : threads: true,

代码 : pins: true,

代码 : search: true,

代码 : memberInfo: true,

代码 : roleInfo: true,

代码 : roles: false,

代码 : channelInfo: true,

代码 : channels: true,

代码 : voiceStatus: true,

代码 : events: true,

代码 : moderation: false,

代码 : },

代码 : replyToMode: "off",

代码 : dm: {
代码 : enabled: true,
代码 : policy: "pairing", // pairing | allowlist | open | disabled
代码 : allowFrom: ["123456789012345678", "steipete"],
代码 : groupEnabled: false,
代码 : groupChannels: ["openclaw-dm"],
代码 : },
代码 : guilds: {
代码 : "": { requireMention: true },
代码 : "123456789012345678": {
代码 : slug: "friends-of-openclaw",
代码 : requireMention: false,
代码 : reactionNotifications: "own",
代码 : users: ["987654321098765432", "steipete"],
代码 : channels: {
代码 : general: { allow: true },
代码 : help: {
代码 : allow: true,
代码 : requireMention: true,
代码 : users: ["987654321098765432"],
代码 : skills: ["search", "docs"],
代码 : systemPrompt: "Keep answers short.",
代码 : },
代码 : },
代码 : },
代码 : },
代码 : },
代码 : },
代码 : },
代码 : }

确认表情反应通过 `messages.ackReaction + messages.ackReactionScope` 全局控制。使用 `messages.removeAckAfterReply` 在机器人回复后清除确认表情反应。

- `dm.enabled` : 设置 `false` 忽略所有私信 (默认 `true`) 。
- `dm.policy` : 私信访问控制 (推荐 `pairing`) 。“`open`” 需要 `dm.allowFrom=[""]` 。
- `dm.allowFrom` : 私信允许列表 (用户 ID 或名称) 。用于 `dm.policy="allowlist"` 和 `dm.policy="open"` 验证。向导接受用户名, 并在机器人可以搜索成员时将其解析为 ID 。
- `dm.groupEnabled` : 启用群组私信 (默认 `false`) 。
- `dm.groupChannels` : 群组私信频道 ID 或 slug 的可选允许列表 。
- `groupPolicy` : 控制服务器频道处理 (`open|disabled|allowlist`) ; `allowlist` 需要频道允许列表 。
- `guilds` : 按服务器规则, 以服务器 ID (首选) 或 slug 为键 。
- `guilds.""` : 当没有显式条目时应用的默认每服务器设置 。
- `guilds.<id>.slug` : 用于显示名称的可选友好 slug 。
- `guilds.<id>.users` : 可选的每服务器用户允许列表 (ID 或名称) 。
- `guilds.<id>.tools` : 可选的每服务器工具策略覆盖 (`allow/deny/alsoAllow`) , 在频道覆盖缺失时使用 。
- `guilds.<id>.toolsBySender` : 服务器级别的可选每发送者工具策略覆盖 (在频道覆盖缺失时应用 ; 支持 `""` 通配符) 。
- `guilds.<id>.channels.<channel>.allow` : 当 `groupPolicy="allowlist"` 时允许/拒绝频道 。
- `guilds.<id>.channels.<channel>.requireMention` : 频道的提及限制 。
- `guilds.<id>.channels.<channel>.tools` : 可选的每频道工具策略覆盖 (`allow/deny/alsoAllow`) 。
- `guilds.<id>.channels.<channel>.toolsBySender` : 频道内的可选每发送者工具策略覆盖 (支持 `""` 通配符) 。
- `guilds.<id>.channels.<channel>.users` : 可选的每频道用户允许列表 。
- `guilds.<id>.channels.<channel>.skills` : Skills 过滤器 (省略 = 所有 Skills, 空 = 无) 。
- `guilds.<id>.channels.<channel>.systemPrompt` : 频道的额外系统提示词 (与频道主题组合) 。
- `guilds.<id>.channels.<channel>.enabled` : 设置 `false` 禁用频道 。
- `guilds.<id>.channels` : 频道规则 (键为频道 slug 或 ID) 。
- `guilds.<id>.requireMention` : 每服务器提及要求 (可按频道覆盖) 。

- `guilds.<id>.reactionNotifications`：表情反应系统事件模式（off、own、all、allowlist）。
- `textChunkLimit`：出站文本块大小（字符）。默认：2000。
- `chunkMode`：length（默认）仅在超过 `textChunkLimit` 时分割；newline 在空行（段落边界）处分割，然后再进行长度分块。
- `maxLinesPerMessage`：每条消息的软最大行数。默认：17。
- `mediaMaxMb`：限制保存到磁盘的入站媒体大小。
- `historyLimit`：回复提及时作为上下文包含的最近服务器消息数量（默认 20；回退到 `messages.groupChat.historyLimit`；0 禁用）。
- `dmHistoryLimit`：私信历史限制（用户轮次）。每用户覆盖：`dms["<user_id>"].historyLimit`。
- `retry`：出站 Discord API 调用的重试策略（`attempts`、`minDelayMs`、`maxDelayMs`、`jitter`）。
- `pluralkit`：解析 PluralKit 代理消息，使系统成员显示为不同的发送者。
- `actions`：每操作工具门控；省略允许所有（设置 false 禁用）。
- `reactions`（涵盖表情反应 + 读取表情反应）
- `stickers`、`emojiUploads`、`stickerUploads`、`polls`、`permissions`、`messages`、`threads`、`pins`、`search`
- `memberInfo`、`roleInfo`、`channelInfo`、`voiceStatus`、`events`
- `channels`（创建/编辑/删除频道 + 类别 + 权限）
- `roles`（角色添加/移除，默认 false）
- `moderation`（超时/踢出/封禁，默认 false）
- `execApprovals`：Discord 专用执行审批私信（按钮 UI）。支持 `enabled`、`approvers`、`agentFilter`、`sessionFilter`。

表情反应通知使用 `guilds.<id>.reactionNotifications`：

- `off`：无表情反应事件。
- `own`：机器人自己消息上的表情反应（默认）。
- `all`：所有消息上的所有表情反应。
- `allowlist`：来自 `guilds.<id>.users` 的用户在所有消息上的表情反应（空列表禁用）。

PluralKit (PK) 支持

启用 PK 查找，以便代理消息解析到底层系统 + 成员。启用后，OpenClaw 使用成员身份进行允许列表匹配，并将发送者标记为 Member (PK:System) 以避免意外的 Discord 提及。

```

代码：{
代码： channels: {
代码： discord: {
代码： pluralkit: {
代码： enabled: true,
代码： token: "pk_live_...", // 可选；私有系统需要
代码： },
代码： },
代码： },
代码： }

```

允许列表注意事项（启用 PK 时）：

- 在 dm.allowFrom 、 guilds.<id>.users 或每频道 users 中使用 pk:<memberId> 。
- 成员显示名称也按名称/slug 匹配。
- 查找使用原始 Discord 消息 ID（代理前的消息），因此 PK API 只在其 30 分钟窗口内解析它。
- 如果 PK

查找失败（例如，没有令牌的私有系统），代理消息会被视为机器人消息并被丢弃，除非 channels.discord.allowBots=true 。

工具操作默认值

操作组	默认	说明
reactions	启用	表情反应 + 列出表情反应 + emojiList
stickers	启用	发送贴纸
emojiUploads	启用	上传表情
stickerUploads	启用	上传贴纸
polls	启用	创建投票
permissions	启用	频道权限快照
messages	启用	读取/发送/编辑/删除
threads	启用	创建/列出/回复
pins	启用	置顶/取消置顶/列出

| search | 启用 | 消息搜索（预览功能） |

| memberInfo | 启用 | 成员信息 |

| roleInfo | 启用 | 角色列表 |

| channelInfo | 启用 | 频道信息 + 列表 |

| channels | 启用 | 频道/类别管理 |

| voiceStatus | 启用 | 语音状态查询 |

| events | 启用 | 列出/创建预定事件 |

| roles | 禁用 | 角色添加/移除 |

| moderation | 禁用 | 超时/踢出/封禁 |

- replyToMode : off（默认）、first 或 all。仅在模型包含回复标签时适用。

回复标签

要请求线程回复，模型可以在其输出中包含一个标签：

- [[reply_to_current]] — 回复触发的 Discord 消息。
- [[reply_to:<id>]] — 回复上下文/历史中的特定消息 ID。当前消息 ID 作为 [message_id: ...] 附加到提示词；历史条目已包含 ID。

行为由 channels.discord.replyToMode 控制：

- off：忽略标签。
- first：只有第一个出站块/附件是回复。
- all：每个出站块/附件都是回复。

允许列表匹配注意事项：

- allowFrom/users/groupChannels 接受 ID、名称、标签或像 <@id> 这样的提及。
- 支持 discord:/user:（用户）和 channel:（群组私信）等前缀。
- 使用 允许任何发送者/频道。
- 当存在 guilds.<id>.channels 时，未列出的频道默认被拒绝。
- 当省略 guilds.<id>.channels 时，允许列表中服务器的所有频道都被允许。
- 要不允许任何频道，设置 channels.discord.groupPolicy: "disabled"（或保持空允许列表）。
- 配置向导接受 Guild/Channel 名称（公开 + 私有）并在可能时将其解析为 ID。

- 启动时，OpenClaw 将允许列表中的频道/用户名称解析为 ID（当机器人可以搜索成员时）并记录映射；未解析的条目保持原样。

原生命令注意事项：

- 注册的命令镜像 OpenClaw 的聊天命令。
- 原生命令遵循与私信/服务器消息相同的允许列表（channels.discord.dm.allowFrom、channels.discord.guilds、每频道规则）。
- 斜杠命令可能在 Discord UI 中对未在允许列表中的用户仍然可见；OpenClaw 在执行时强制执行允许列表并回复"未授权"。

工具操作

智能体可以使用以下操作调用 discord：

- react / reactions（添加或列出表情反应）
- sticker、poll、permissions
- readMessages、sendMessage、editMessage、deleteMessage
- 读取/搜索/置顶工具负载包含规范化的 timestampMs（UTC 纪元毫秒）和 timestampUtc 以及原始 Discord timestamp。
- threadCreate、threadList、threadReply
- pinMessage、unpinMessage、listPins
- searchMessages、memberInfo、roleInfo、roleAdd、roleRemove、emojiList
- channelInfo、channelList、voiceStatus、eventList、eventCreate
- timeout、kick、ban

Discord 消息 ID 在注入的上下文中显示（[discord message id: ...] 和历史行），以便智能体可以定位它们。

表情可以是 unicode（例如）或自定义表情语法如 <:party_blob:1234567890>。

安全与运维

- 像对待密码一样对待机器人令牌；在受监督的主机上优先使用 DISCORD_BOT_TOKEN 环境变量，或锁定配置文件权限。
- 只授予机器人所需的权限（通常是读取/发送消息）。
- 如果机器人卡住或受到速率限制，在确认没有其他进程拥有 Discord 会话后重启 Gateway 网关（openclaw gateway --force）。

5. 飞书机器人

飞书机器人

状态：生产就绪，支持机器人私聊和群组。使用 WebSocket 长连接模式接收消息。

需要插件

安装 Feishu 插件：

代码：openclaw plugins install @openclaw/feishu

本地 checkout（在 git 仓库内运行）：

代码：openclaw plugins install ./extensions/feishu

快速开始

添加飞书渠道有两种方式：

方式一：通过安装向导添加（推荐）

如果您刚安装完 OpenClaw，可以直接运行向导，根据提示添加飞书：

代码：openclaw onboard

向导会引导您完成：

- 创建飞书应用并获取凭证
- 配置应用凭证
- 启动网关

完成配置后，您可以使用以下命令检查网关状态：

- openclaw gateway status - 查看网关运行状态
- openclaw logs --follow - 查看实时日志

方式二：通过命令行添加

如果您已经完成了初始安装，可以用以下命令添加飞书渠道：

代码：openclaw channels add

然后根据交互式提示选择 Feishu，输入 App ID 和 App Secret 即可。

完成配置后，您可以使用以下命令管理网关：

- openclaw gateway status - 查看网关运行状态
- openclaw gateway restart - 重启网关以应用新配置
- openclaw logs --follow - 查看实时日志

第一步：创建飞书应用

1. 打开飞书开放平台

访问 飞书开放平台，使用飞书账号登录。

Lark（国际版）请使用 domain: "lark"。

2. 创建应用

- 点击 创建企业自建应用
- 填写应用名称和描述
- 选择应用图标

创建企业自建应用

3. 获取应用凭证

在应用的 凭证与基础信息 页面，复制：

- App ID（格式如 cli_xxx）
- App Secret

重要：请妥善保管 App Secret，不要分享给他人。

获取应用凭证

4. 配置应用权限

在 权限管理 页面，点击 批量导入 按钮，粘贴以下 JSON 配置一键导入所需权限：

```
代码：{
代码： "scopes": {
代码： "tenant": [
代码： "aily:file:read",
代码： "aily:file:write",
代码： "application:application.app_message_stats.overview:readonly",
代码： "application:application:self_manage",
代码： "application:bot.menu:write",
代码： "cardkit:card:write",
代码： "contact:user.employee_id:readonly",
代码： "corehr:file:download",
代码： "docs:document.content:read",
代码： "event:ip_list",
代码： "im:chat",
代码： "im:chat.access_event.bot_p2p_chat:read",
代码： "im:chat.members:bot_access",
代码： "im:message",
代码： "im:message.group_at_msg:readonly",
代码： "im:message.group_msg",
代码： "im:message.p2p_msg:readonly",
代码： "im:message:readonly",
代码： "im:message:send_as_bot",
代码： "im:resource",
代码： "sheets:spreadsheet",
代码： "wiki:wiki:readonly"
代码： ],
代码： "user": ["aily:file:read", "aily:file:write", "im:chat.access_event.bot_p2p_chat:read"]
代码： }
代码： }
```

配置应用权限

5. 启用机器人能力

在 应用能力 > 机器人 页面：

- 开启机器人能力
- 配置机器人名称

启用机器人能力

6. 配置事件订阅

重要提醒：在配置事件订阅前，请务必确保已完成以下步骤：

- 运行 `openclaw channels add` 添加了 Feishu 渠道
- 网关处于启动状态（可通过 `openclaw gateway status` 检查状态）

在 事件订阅 页面：

- 选择 使用长连接接收事件（WebSocket 模式）
- 添加事件：`im.message.receive_v1`（接收消息）

注意：如果网关未启动或渠道未添加，长连接设置将保存失败。

配置事件订阅

7. 发布应用

- 在 版本管理与发布 页面创建版本
- 提交审核并发布
- 等待管理员审批（企业自建应用通常自动通过）

第二步：配置 OpenClaw

通过向导配置（推荐）

运行以下命令，根据提示粘贴 App ID 和 App Secret：

代码：`openclaw channels add`

选择 Feishu，然后输入您在第一步获取的凭证即可。

通过配置文件配置

编辑 ~/.openclaw/openclaw.json :

```
代码 : {  
代码 : channels: {  
代码 : feishu: {  
代码 : enabled: true,  
代码 : dmPolicy: "pairing",  
代码 : accounts: {  
代码 : main: {  
代码 : appId: "cli_xxx",  
代码 : appSecret: "xxx",  
代码 : botName: "我的AI助手",  
代码 : },  
代码 : },  
代码 : },  
代码 : },  
代码 : }
```

通过环境变量配置

Lark（国际版）域名

如果您的租户在 Lark（国际版），请设置域名为 lark（或完整域名），可配置 channels.feishu.domain 或 channels.feishu.accounts.<id>.domain :

```
代码 : {  
代码 : channels: {  
代码 : feishu: {  
代码 : domain: "lark",  
代码 : accounts: {  
代码 : main: {  
代码 : appId: "cli_xxx",
```

代码：appSecret: "xxx",

代码：},

代码：},

代码：},

代码：},

代码：}

第三步：启动并测试

1. 启动网关

代码：openclaw gateway

2. 发送测试消息

在飞书中找到您创建的机器人，发送一条消息。

3. 配对授权

默认情况下，机器人会回复一个 配对码。您需要批准此代码：

代码：openclaw pairing approve feishu <配对码>

批准后即可正常对话。

介绍

- 飞书机器人渠道：由网关管理的飞书机器人
- 确定性路由：回复始终返回飞书，模型不会选择渠道
- 会话隔离：私聊共享主会话；群组独立隔离
- WebSocket 连接：使用飞书 SDK 的长连接模式，无需公网 URL

访问控制

私聊访问

- 默认：dmPolicy: "pairing"，陌生用户会收到配对码
- 批准配对：

代码：openclaw pairing list feishu # 查看待审批列表

代码：openclaw pairing approve feishu <CODE> # 批准

- 白名单模式：通过 channels.feishu.allowFrom 配置允许的用户 Open ID

群组访问

1. 群组策略（channels.feishu.groupPolicy）：

- "open" = 允许群组中所有人（默认）
- "allowlist" = 仅允许 groupAllowFrom 中的用户
- "disabled" = 禁用群组消息

2. @提及要求（channels.feishu.groups.<chat_id>.requireMention）：

- true = 需要 @机器人才响应（默认）
- false = 无需 @也响应

群组配置示例

允许所有群组，需要 @提及（默认行为）

代码：{

代码：channels: {

代码：feishu: {

代码：groupPolicy: "open",

代码：// 默认 requireMention: true

代码：},

代码：},

代码：}

允许所有群组，无需 @提及

需要为特定群组配置：

```
代码：{  
代码： channels: {  
代码： feishu: {  
代码： groups: {  
代码： oc_xxx: { requireMention: false },  
代码： },  
代码： },  
代码： },  
代码： }
```

仅允许特定用户在群组中使用

```
代码：{  
代码： channels: {  
代码： feishu: {  
代码： groupPolicy: "allowlist",  
代码： groupAllowFrom: ["ou_xxx", "ou_yyy"],  
代码： },  
代码： },  
代码： }
```

获取群组/用户 ID

获取群组 ID (chat_id)

群组 ID 格式为 oc_xxx，可以通过以下方式获取：

方法一（推荐）：

- 启动网关并在群组中 @机器人发消息
- 运行 `openclaw logs --follow` 查看日志中的 chat_id

方法二：

使用飞书 API 调试工具获取机器人所在群组列表。

获取用户 ID (open_id)

用户 ID 格式为 ou_xxx，可以通过以下方式获取：

方法一（推荐）：

- 启动网关并给机器人发消息
- 运行 `openclaw logs --follow` 查看日志中的 `open_id`

方法二：

查看配对请求列表，其中包含用户的 Open ID：

代码：`openclaw pairing list feishu`

常用命令

命令	说明
-----	-----
/status	查看机器人状态
/reset	重置对话会话
/model	查看/切换模型

注意：飞书目前不支持原生命令菜单，命令需要以文本形式发送。

网关管理命令

在配置和使用飞书渠道时，您可能需要使用以下网关管理命令：

命令	说明
-----	-----
openclaw gateway status	查看网关运行状态
openclaw gateway install	安装/启动网关服务
openclaw gateway stop	停止网关服务
openclaw gateway restart	重启网关服务
openclaw logs --follow	实时查看日志输出

故障排除

机器人在群组中不响应

- 检查机器人是否已添加到群组
- 检查是否 @了机器人（默认需要 @提及）
- 检查 groupPolicy 是否为 "disabled"
- 查看日志：openclaw logs --follow

机器人收不到消息

- 检查应用是否已发布并审批通过
- 检查事件订阅是否配置正确（im.message.receive_v1）
- 检查是否选择了 长连接 模式
- 检查应用权限是否完整
- 检查网关是否正在运行：openclaw gateway status
- 查看实时日志：openclaw logs --follow

App Secret 泄露怎么办

- 在飞书开放平台重置 App Secret
- 更新配置文件中的 App Secret
- 重启网关

发送消息失败

- 检查应用是否有 im:message:send_as_bot 权限
- 检查应用是否已发布
- 查看日志获取详细错误信息

高级配置

多账号配置

如果需要管理多个飞书机器人：

```
代码：{  
代码： channels: {  
代码： feishu: {  
代码： accounts: {  
代码： main: {  
代码： appld: "cli_xxx",  
代码： appSecret: "xxx",  
代码： botName: "主机器人",  
代码： },  
代码： backup: {  
代码： appld: "cli_yyy",  
代码： appSecret: "yyy",  
代码： botName: "备用机器人",  
代码： enabled: false, // 暂时禁用  
代码： },  
代码： },  
代码： },  
代码： },  
代码： },  
代码： }
```

消息限制

- textChunkLimit：出站文本分块大小（默认 2000 字符）
- mediaMaxMb：媒体上传/下载限制（默认 30MB）

流式输出

飞书支持通过交互式卡片实现流式输出，机器人会实时更新卡片内容显示生成进度。默认配置：

```
代码：{  
代码： channels: {  
代码： feishu: {
```

```
代码： streaming: true, // 启用流式卡片输出（默认 true）
代码： blockStreaming: true, // 启用块级流式（默认 true）
代码： },
代码： },
代码： }
```

如需禁用流式输出（等待完整回复后一次性发送），可设置 streaming: false。

消息引用

在群聊中，机器人的回复可以引用用户发送的原始消息，让对话上下文更加清晰。

配置选项：

```
代码： {
代码：   channels: {
代码：     feishu: {
代码：       // 账户级别配置（默认 "all"）
代码：       replyToMode: "all",
代码：     groups: {
代码：       oc_xxx: {
代码：         // 特定群组可以覆盖
代码：         replyToMode: "first",
代码：       },
代码：     },
代码：   },
代码： },
代码： }
```

replyToMode 值说明：

值	行为
-----	-----
"off"	不引用原消息（私聊默认值）
"first"	仅在第一条回复时引用原消息

| "all" | 所有回复都引用原消息（群聊默认值） |

注意：消息引用功能与流式卡片输出（streaming: true）不能同时使用。当启用流式输出时，回复会以卡片形式呈现，不会显示引用。

多 Agent 路由

通过 bindings 配置，您可以用一个飞书机器人对接多个不同功能或性格的 Agent。系统会根据用户 ID 或群组 ID 自动将对话分发到对应的 Agent。

配置示例：

```
代码：{
代码： agents: {
代码： list: [
代码： { id: "main" },
代码： {
代码： id: "clawd-fan",
代码： workspace: "/home/user/clawd-fan",
代码： agentDir: "/home/user/.openclaw/agents/clawd-fan/agent",
代码： },
代码： {
代码： id: "clawd-xi",
代码： workspace: "/home/user/clawd-xi",
代码： agentDir: "/home/user/.openclaw/agents/clawd-xi/agent",
代码： },
代码： ],
代码： },
代码： bindings: [
代码： {
代码： // 用户 A 的私聊 → main agent
代码： agentId: "main",
代码： match: {
代码： channel: "feishu",
```

```
代码： peer: { kind: "dm", id: "ou_28b31a88..." },
代码： },
代码： },
代码： {
代码： // 用户 B 的私聊 → clawd-fan agent
代码： agentId: "clawd-fan",
代码： match: {
代码： channel: "feishu",
代码： peer: { kind: "dm", id: "ou_0fe6b1c9..." },
代码： },
代码： },
代码： {
代码： // 某个群组 → clawd-xi agent
代码： agentId: "clawd-xi",
代码： match: {
代码： channel: "feishu",
代码： peer: { kind: "group", id: "oc_xxx..." },
代码： },
代码： },
代码： ],
代码： }
```

匹配规则说明：

字段	说明
agentId	目标 Agent 的 ID，需要在 agents.list 中定义
match.channel	渠道类型，这里固定为 "feishu"
match.peer.kind	对话类型："dm"（私聊）或 "group"（群组）
match.peer.id	用户 Open ID（ou_xxx）或群组 ID（oc_xxx）

获取 ID 的方法：参见上文 获取群组/用户 ID 章节。

配置参考

完整配置请参考：网关配置

主要选项：

配置项	说明	默认值
-----	-----	-----
channels.feishu.enabled	启用/禁用渠道	true
channels.feishu.domain	API 域名（feishu 或 lark）	feishu
channels.feishu.accounts.<id>.appId	应用 App ID	-
channels.feishu.accounts.<id>.appSecret	应用 App Secret	-
channels.feishu.accounts.<id>.domain	单账号 API 域名覆盖	feishu
channels.feishu.dmPolicy	私聊策略	pairing
channels.feishu.allowFrom	私聊白名单（open_id 列表）	-
channels.feishu.groupPolicy	群组策略	open
channels.feishu.groupAllowFrom	群组白名单	-
channels.feishu.groups.<chat_id>.requireMention	是否需要 @提及	true
channels.feishu.groups.<chat_id>.enabled	是否启用该群组	true
channels.feishu.textChunkLimit	消息分块大小	2000
channels.feishu.mediaMaxMb	媒体大小限制	30
channels.feishu.streaming	启用流式卡片输出	true
channels.feishu.blockStreaming	启用块级流式	true

dmPolicy 策略说明

值	行为
-----	-----
"pairing"	默认。未知用户收到配对码，管理员批准后才能对话
"allowlist"	仅 allowFrom 列表中的用户可对话，其他静默忽略
"open"	允许所有人对话（需在 allowFrom 中加 ""）

| "disabled" | 完全禁止私聊 |

支持的消息类型

接收

- 文本消息
- 图片
- 文件
- 音频
- 视频
- 表情包

发送

- 文本消息
- 图片
- 文件
- 音频
- 富文本（部分支持）

6. Google Chat (Chat API)

Google Chat (Chat API)

状态：已支持通过 Google Chat API webhooks（仅 HTTP）使用私信和空间。

快速设置（新手）

- 创建一个 Google Cloud 项目并启用 Google Chat API。
- 前往：Google Chat API Credentials
- 如果 API 尚未启用，请启用它。
- 创建一个服务账号：
- 点击 Create Credentials > Service Account。
- 随意命名（例如 openclaw-chat）。

- 权限留空（点击 Continue）。
- 有访问权限的主账号留空（点击 Done）。
- 创建并下载 JSON 密钥：
- 在服务账号列表中，点击刚刚创建的账号。
- 前往 Keys 标签页。
- 点击 Add Key > Create new key。
- 选择 JSON 并点击 Create。
- 将下载的 JSON 文件存储在 Gateway 网关主机上（例如 `~/.openclaw/googlechat-service-account.json`）。
- 在 Google Cloud Console Chat Configuration 中创建一个 Google Chat 应用：
- 填写 Application info：
- App name：（例如 OpenClaw）
- Avatar URL：（例如 `
- Description：（例如 Personal AI Assistant）
- 启用 Interactive features。
- 在 Functionality 下，勾选 Join spaces and group conversations。
- 在 Connection settings 下，选择 HTTP endpoint URL。
- 在 Triggers 下，选择 Use a common HTTP endpoint URL for all triggers 并将其设置为你的 Gateway 网关公网 URL 后加 `/googlechat`。
- _提示：运行 `openclaw status` 查看你的 Gateway 网关公网 URL。_
- 在 Visibility 下，勾选 Make this Chat app available to specific people and groups in <Your Domain>。
- 在文本框中输入你的邮箱地址（例如 `user@example.com`）。
- 点击底部的 Save。
- 启用应用状态：
- 保存后，刷新页面。
- 找到 App status 部分（通常在保存后位于顶部或底部附近）。
- 将状态更改为 Live - available to users。
- 再次点击 Save。
- 使用服务账号路径和 webhook audience 配置 OpenClaw：
- 环境变量：`GOOGLE_CHAT_SERVICE_ACCOUNT_FILE=/path/to/service-account.json`

- 或配置：channels.googlechat.serviceAccountFile: "/path/to/service-account.json"。
- 设置 webhook audience 类型和值（与你的 Chat 应用配置匹配）。
- 启动 Gateway 网关。Google Chat 将向你的 webhook 路径发送 POST 请求。

添加到 Google Chat

Gateway 网关运行后，且你的邮箱已添加到可见性列表中：

- 前往 Google Chat。
- 点击 Direct Messages 旁边的 +（加号）图标。
- 在搜索栏（通常用于添加联系人的位置）中，输入你在 Google Cloud Console 中配置的 App name。
-

注意：该机器人不会出现在"Marketplace"浏览列表中，因为它是私有应用。你必须按名称搜索。

- 从结果中选择你的机器人。
- 点击 Add 或 Chat 开始一对一对话。
- 发送"Hello"来触发助手！

公网 URL（仅 Webhook）

Google Chat webhooks 需要一个公网 HTTPS 端点。为了安全起见，只将 /googlechat 路径暴露到互联网。将 OpenClaw 仪表板和其他敏感端点保留在你的私有网络上。

方案 A：Tailscale Funnel（推荐）

使用 Tailscale Serve 提供私有仪表板，使用 Funnel 提供公网 webhook 路径。这样可以保持 / 私有，同时只暴露 /googlechat。

- 检查你的 Gateway 网关绑定的地址：

代码：ss -tlnp | grep 18789

记下 IP 地址（例如 127.0.0.1、0.0.0.0 或你的 Tailscale IP 如 100.x.x.x）。

- 仅将仪表板暴露给 tailnet（端口 8443）：

代码：# 如果绑定到 localhost（127.0.0.1 或 0.0.0.0）：

代码：tailscale serve --bg --https 8443 http://127.0.0.1:18789

代码：# 如果仅绑定到 Tailscale IP（例如 100.106.161.80）：

代码： `tailscale serve --bg --https 8443 http://100.106.161.80:18789`

- 仅公开暴露 webhook 路径：

代码： # 如果绑定到 localhost (127.0.0.1 或 0.0.0.0)：

代码： `tailscale funnel --bg --set-path /googlechat http://127.0.0.1:18789/googlechat`

代码： # 如果仅绑定到 Tailscale IP (例如 100.106.161.80)：

代码： `tailscale funnel --bg --set-path /googlechat`

`http://100.106.161.80:18789/googlechat`

- 授权节点访问 Funnel：

如果出现提示，请访问输出中显示的授权 URL，以在你的 tailnet 策略中为此节点启用 Funnel。

- 验证配置：

代码： `tailscale serve status`

代码： `tailscale funnel status`

你的公网 webhook URL 将是：

、

你的私有仪表板仅限 tailnet 访问：

、

在 Google Chat 应用配置中使用公网 URL (不带 :8443)。

注意：此配置在重启后会保留。如需稍后移除，请运行 `tailscale funnel reset` 和 `tailscale serve reset`。

方案 B：反向代理 (Caddy)

如果你使用像 Caddy 这样的反向代理，只代理特定路径：

代码： `your-domain.com {`

代码： `reverse_proxy /googlechat* localhost:18789`

代码： `}`

使用此配置，任何发往 `your-domain.com/` 的请求将被忽略或返回 404，而 `your-domain.com/googlechat` 会安全地路由到 OpenClaw。

方案 C : Cloudflare Tunnel

配置你的隧道入口规则，只路由 webhook 路径：

- 路径：/googlechat -> `
- 默认规则：HTTP 404（未找到）

工作原理

- Google Chat 向 Gateway 网关发送 webhook POST 请求。每个请求都包含一个 Authorization: Bearer <token> 头。
- OpenClaw 根据配置的 audienceType + audience 验证令牌：
- audienceType: "app-url" → audience 是你的 HTTPS webhook URL。
- audienceType: "project-number" → audience 是 Cloud 项目编号。
- 消息按空间路由：
- 私信使用会话键 agent:<agentId>:googlechat:dm:<spaceId>。
- 空间使用会话键 agent:<agentId>:googlechat:group:<spaceId>。
- 私信访问默认为配对模式。未知发送者会收到配对码；使用以下命令批准：
- openclaw pairing approve googlechat <code>
- 群组空间默认需要 @提及。如果提及检测需要应用的用户名，请使用 botUser。

目标标识符

使用这些标识符进行消息投递和允许列表：

- 私信：users/<userId> 或 users/<email>（接受邮箱地址）。
- 空间：spaces/<spaceId>。

配置要点

代码：{

代码： channels: {

代码： googlechat: {

代码： enabled: true,

代码： serviceAccountFile: "/path/to/service-account.json",

代码： audienceType: "app-url",

代码：audience: "https://gateway.example.com/googlechat",

代码：webhookPath: "/googlechat",

代码：botUser: "users/1234567890", // 可选；帮助提及检测

代码：dm: {

代码：policy: "pairing",

代码：allowFrom: ["users/1234567890", "name@example.com"],

代码：},

代码：groupPolicy: "allowlist",

代码：groups: {

代码："spaces/AAAA": {

代码：allow: true,

代码：requireMention: true,

代码：users: ["users/1234567890"],

代码：systemPrompt: "Short answers only.",

代码：},

代码：},

代码：actions: { reactions: true },

代码：typingIndicator: "message",

代码：mediaMaxMb: 20,

代码：},

代码：},

代码：}

注意事项：

- 服务账号凭证也可以通过 serviceAccount (JSON 字符串) 内联传递。
- 如果未设置 webhookPath，默认 webhook 路径为 /googlechat。
- 当 actions.reactions 启用时，可通过 reactions 工具和 channels action 使用表情回应。
- typingIndicator 支持 none、message（默认）和 reaction（reaction 需要用户 OAuth）。
- 附件通过 Chat API 下载并存储在媒体管道中（大小受 mediaMaxMb 限制）。

故障排除

405 Method Not Allowed

如果 Google Cloud Logs Explorer 显示如下错误：

代码：status code: 405, reason phrase: HTTP error response: HTTP/1.1 405 Method Not Allowed

这意味着 webhook 处理程序未注册。常见原因：

- 渠道未配置：配置中缺少 channels.googlechat 部分。使用以下命令验证：

代码：openclaw config get channels.googlechat

如果返回"Config path not found"，请添加配置（参见配置要点）。

- 插件未启用：检查插件状态：

代码：openclaw plugins list | grep googlechat

如果显示"disabled"，请在配置中添加 plugins.entries.googlechat.enabled: true。

- Gateway 网关未重启：添加配置后，重启 Gateway 网关：

代码：openclaw gateway restart

验证渠道是否正在运行：

代码：openclaw channels status

代码：# 应显示：Google Chat default: enabled, configured, ...

其他问题

- 检查 openclaw channels status --probe 以查看认证错误或缺少 audience 配置。
- 如果没有收到消息，请确认 Chat 应用的 webhook URL 和事件订阅。
- 如果提及门控阻止了回复，请将 botUser 设置为应用的用户资源名称并验证 requireMention。
- 在发送测试消息时使用 openclaw logs --follow 查看请求是否到达 Gateway 网关。

相关文档：

- Gateway 网关配置
- 安全
- 表情回应

7. grammY 集成 (Telegram Bot API)

grammY 集成 (Telegram Bot API)

为什么选择 grammY

- 以 TS 为核心的 Bot API 客户端，内置长轮询 + webhook 辅助工具、中间件、错误处理和速率限制器。
- 媒体处理辅助工具比手动编写 fetch + FormData 更简洁；支持所有 Bot API 方法。
- 可扩展：通过自定义 fetch 支持代理，可选的会话中间件，类型安全的上下文。

我们发布的内容

- 单一客户端路径：移除了基于 fetch 的实现；grammY 现在是唯一的 Telegram 客户端（发送 + Gateway 网关），默认启用 grammY throttler。
- Gateway 网关：monitorTelegramProvider 构建 grammY Bot，接入 mention/allowlist 网关控制，通过 getFile/download 下载媒体，并使用 sendMessage/sendPhoto/sendVideo/sendAudio/sendDocument 发送回复。通过 webhookCallback 支持长轮询或 webhook。
- 代理：可选的 channels.telegram.proxy 通过 grammY 的 client.baseFetch 使用 undici.ProxyAgent。
- Webhook 支持：webhook-set.ts 封装了 setWebhook/deleteWebhook；webhook.ts 托管回调，支持健康检查和优雅关闭。当设置了 channels.telegram.webhookUrl + channels.telegram.webhookSecret 时，Gateway 网关启用 webhook 模式（否则使用长轮询）。
- 会话：私聊折叠到智能体主会话 (agent:<agentId>:<mainKey>)；群组使用 agent:<agentId>:telegram:group:<chatId>；回复路由回同一渠道。
- 配置选项：channels.telegram.botToken、channels.telegram.dmPolicy、channels.telegram.groups (allowlist + mention 默认值)、channels.telegram.allowFrom、channels.telegram.groupAllowFrom、channels.telegram.groupPolicy、channels.telegram.mediaMax Mb、channels.telegram.linkPreview、channels.telegram.proxy、channels.telegram.webhookSecret、channels.telegram.webhookUrl。
- 草稿流式传输：可选的 channels.telegram.streamMode 在私有话题聊天中使用 sendMessageDraft (Bot API 9.3+)。这与渠道分块流式传输是分开的。
- 测试：grammY mock 覆盖了私信 + 群组 mention 网关控制和出站发送；欢迎添加更多媒体/webhook 测试用例。

待解决问题

- 如果遇到 Bot API 429 错误，考虑使用可选的 grammY 插件 (throttler) 。
- 添加更多结构化媒体测试 (贴纸、语音消息) 。
- 使 webhook 监听端口可配置 (目前固定为 8787，除非通过 Gateway 网关配置) 。

8. 群组消息 (WhatsApp 网页渠道)

群组消息 (WhatsApp 网页渠道)

目标：让 Clawd 留在 WhatsApp

群组中，仅在被提及时唤醒，并将该对话线程与个人私信会话分开。

注意：agents.list[].groupChat.mentionPatterns 现在也被

Telegram/Discord/Slack/iMessage 使用；本文档重点介绍 WhatsApp

特定的行为。对于多智能体设置，为每个智能体设置

agents.list[].groupChat.mentionPatterns (或使用 messages.groupChat.mentionPatterns 作为全局回退) 。

已实现的功能 (2025-12-03)

- 激活模式：mention (默认) 或 always。mention 需要被提及 (通过 mentionedJids 的真实 WhatsApp @提及、正则表达式模式，或文本中任意位置的机器人 E.164 号码) 。always 会在每条消息时唤醒智能体，但它应该只在能提供有意义价值时才回复；否则返回静默令牌 NO_REPLY。默认值可在配置中设置 (channels.whatsapp.groups)，并可通过 /activation 为每个群组单独覆盖。当设置了 channels.whatsapp.groups 时，它同时充当群组允许列表 (包含 "" 以允许所有群组) 。

- 群组策略：channels.whatsapp.groupPolicy

控制是否接受群组消息 (open|disabled|allowlist) 。allowlist 使用

channels.whatsapp.groupAllowFrom (回退：显式的

channels.whatsapp.allowFrom) 。默认为 allowlist (在你添加发送者之前被阻止) 。

- 独立群组会话：会话键格式为 agent:<agentId>:whatsapp:group:<jid>，因此 /verbose on 或 /think high (作为独立消息发送) 等命令仅作用于该群组；个人私信状态不受影响。群组线程会跳过心跳。

- 上下文注入：仅待处理的群组消息 (默认 50 条)，即未触发运行的消息，会以 [Chat messages since your last reply - for context] 为前缀注入，触发行在 [Current message - respond to this] 下。已在会话中的消息不会重复注入。

- 发送者显示：每个群组批次现在以 [from: Sender Name (+E164)] 结尾，让 Pi 知道是谁在说话。

-

阅后即焚/一次性查看：我们在提取文本/提及之前会先解包这些消息，因此其中的提及仍会触发。

- 群组系统提示：在群组会话的第一轮（以及每当 /activation 更改模式时），我们会向系统提示注入一段简短说明，如 You are replying inside the WhatsApp group "<subject>". Group members: Alice (+44...), Bob (+43...), ... Activation: trigger-only ... Address the specific sender noted in the message context. 如果元数据不可用，我们仍会告知智能体这是一个群聊。

配置示例 (WhatsApp)

在 ~/.openclaw/openclaw.json 中添加 groupChat 块，以便在 WhatsApp 剥离文本正文中的可视 @ 时，显示名称提及仍能正常工作：

```
代码：{
代码：  channels: {
代码：    whatsapp: {
代码：      groups: {
代码：        "": { requireMention: true },
代码：      },
代码：    },
代码：  },
代码：  agents: {
代码：    list: [
代码：      {
代码：        id: "main",
代码：        groupChat: {
代码：          historyLimit: 50,
代码：          mentionPatterns: ["@?openclaw", "\\+?15555550123"],
代码：        },
代码：      },
代码：    ],
代码：  },
代码：}
```

注意：

- 正则表达式不区分大小写；它们涵盖了像 @openclaw 这样的显示名称提及，以及带或不带 +/ 空格的原始号码。
- 当有人点击联系人时，WhatsApp 仍会通过 mentionedJids 发送规范的提及，因此号码回退很少需要，但作为安全网很有用。

激活命令（仅所有者）

使用群聊命令：

- /activation mention
- /activation always

只有所有者号码（来自 channels.whatsapp.allowFrom，或未设置时使用机器人自己的 E.164）可以更改此设置。在群组中发送 /status 作为独立消息以查看当前激活模式。

使用方法

- 将你的 WhatsApp 账号（运行 OpenClaw 的账号）添加到群组。
- 说 @openclaw ...（或包含号码）。只有允许列表中的发送者才能触发，除非你设置 groupPolicy: "open"。
- 智能体提示将包含最近的群组上下文以及尾部的 [from: ...] 标记，以便它能够回应正确的人。
- 会话级指令（/verbose on、/think high、/new 或 /reset、/compact）仅适用于该群组的会话；将它们作为独立消息发送以使其生效。你的个人私信会话保持独立。

测试/验证

- 手动冒烟测试：
- 在群组中发送 @openclaw 提及，确认收到引用发送者名称的回复。
- 发送第二次提及，验证历史记录块被包含，然后在下一轮清除。
- 检查 Gateway 网关日志（使用 --verbose 运行）以查看 inbound web message 条目，显示 from: <groupId> 和 [from: ...] 后缀。

已知注意事项

- 群组有意跳过心跳以避免嘈杂的广播。
- 回声抑制使用组合的批次字符串；如果你发送两次相同的文本但没有提及，只有第一次会得到响应。
- 会话存储条目将在会话存储中显示为 agent:<agentId>:whatsapp:group:<jid>（默认为 ~/.openclaw/agents/<agentId>/sessions/sessions.json）；缺失条目只是意味着该群组尚未触发运

行。

- 群组中的输入指示器遵循 `agents.defaults.typingMode`（默认：未被提及时为 `message`）。

9. 群组

群组

OpenClaw 在各平台上统一处理群聊：WhatsApp、Telegram、Discord、Slack、Signal、iMessage、Microsoft Teams。

新手入门（2 分钟）

OpenClaw"运行"在你自己的消息账户上。没有单独的 WhatsApp 机器人用户。如果你在一个群组中，OpenClaw 就可以看到该群组并在其中回复。

默认行为：

- 群组受限 (`groupPolicy: "allowlist"`)。
- 除非你明确禁用提及限制，否则回复需要 @ 提及。

解释：允许列表中的发送者可以通过提及来触发 OpenClaw。

简而言之

- 私信访问由 `.allowFrom` 控制。
- 群组访问由 `.groupPolicy` + 允许列表 (`.groups`、`.groupAllowFrom`) 控制。
- 回复触发由提及限制 (`requireMention`、`/activation`) 控制。

快速流程（群消息会发生什么）：

代码：`groupPolicy? disabled` -> 丢弃

代码：`groupPolicy? allowlist` -> 群组允许? 否 -> 丢弃

代码：`requireMention? 是` -> 被提及? 否 -> 仅存储为上下文

代码：否则 -> 回复

群消息流程

如果你想...

| 目标 | 设置什么 |

|-----|-----|

| 允许所有群组但仅在 @ 提及时回复 | groups: { '': { requireMention: true } } |

| 禁用所有群组回复 | groupPolicy: "disabled" |

| 仅特定群组 | groups: { "<group-id>": { ... } } (无 "" 键) |

| 仅你可以在群组中触发 | groupPolicy: "allowlist" 、 groupAllowFrom: ["+1555..."] |

会话键

- 群组会话使用 agent:<agentId>:<channel>:group:<id> 会话键（房间/频道使用 agent:<agentId>:<channel>:channel:<id>）。
- Telegram 论坛话题在群组 ID 后添加 :topic:<threadId>，因此每个话题都有自己的会话。
- 私聊使用主会话（或按发送者配置时使用各自的会话）。
- 群组会话跳过心跳。

模式：个人私信 + 公开群组（单智能体）

是的——如果你的"个人"流量是私信而"公开"流量是群组，这种方式效果很好。

原因：在单智能体模式下，私信通常落在主会话键（agent:main:main）中，而群组始终使用非主会话键（agent:main:<channel>:group:<id>）。如果你启用 mode: "non-main" 的沙箱隔离，这些群组会话在 Docker 中运行，而你的主私信会话保持在主机上。

这给你一个智能体"大脑"（共享工作区 + 记忆），但两种执行姿态：

- 私信：完整工具（主机）
- 群组：沙箱 + 受限工具（Docker）

如果你需要真正独立的工作区/角色（"个人"和"公开"绝不能混合），请使用第二个智能体 + 绑定。参见多智能体路由。

示例（私信在主机上，群组沙箱隔离 + 仅消息工具）：

代码：{

代码： agents: {

代码： defaults: {

代码： sandbox: {

代码： mode: "non-main", // 群组/频道是非主 -> 沙箱隔离

代码： scope: "session", // 最强隔离（每个群组/频道一个容器）

代码： workspaceAccess: "none",

代码： },

代码： },

代码： },

代码： tools: {

代码： sandbox: {

代码： tools: {

代码： // 如果 allow 非空，其他所有工具都被阻止（deny 仍然优先）。

代码： allow: ["group:messaging", "group:sessions"],

代码： deny: ["group:runtime", "group:fs", "group:ui", "nodes", "cron", "gateway"],

代码： },

代码： },

代码： },

代码： }

想要"群组只能看到文件夹 X"而不是"无主机访问"？保持 workspaceAccess: "none" 并仅将允许的路径挂载到沙箱中：

代码： {

代码： agents: {

代码： defaults: {

代码： sandbox: {

代码： mode: "non-main",

代码： scope: "session",

代码： workspaceAccess: "none",

代码： docker: {

代码： binds: [

代码： // hostPath:containerPath:mode

代码： "~/FriendsShared:/data:ro",

代码：],

代码： },

代码： },

代码： },

代码：},

代码：}

相关：

- 配置键和默认值：Gateway 网关配置
- 调试为什么工具被阻止：沙箱 vs 工具策略 vs 提权
- 绑定挂载详情：沙箱隔离

显示标签

- UI 标签在可用时使用 displayName，格式为 <channel>:<token>。
- #room 保留用于房间/频道；群聊使用 g-<slug>（小写，空格 -> -，保留 #@+._-）。

群组策略

控制每个渠道如何处理群组/房间消息：

代码：{

代码： channels: {

代码： whatsapp: {

代码： groupPolicy: "disabled", // "open" | "disabled" | "allowlist"

代码： groupAllowFrom: ["+15551234567"],

代码： },

代码： telegram: {

代码： groupPolicy: "disabled",

代码： groupAllowFrom: ["123456789", "@username"],

代码： },

代码： signal: {

代码： groupPolicy: "disabled",

代码： groupAllowFrom: ["+15551234567"],

代码： },

代码： imessage: {

代码： groupPolicy: "disabled",

代码： groupAllowFrom: ["chat_id:123"],

```

代码： },
代码： msteams: {
代码：   groupPolicy: "disabled",
代码：   groupAllowFrom: ["user@org.com"],
代码： },
代码： discord: {
代码：   groupPolicy: "allowlist",
代码：   guilds: {
代码：     GUILD_ID: { channels: { help: { allow: true } } },
代码：   },
代码： },
代码： slack: {
代码：   groupPolicy: "allowlist",
代码：   channels: { "#general": { allow: true } },
代码： },
代码： matrix: {
代码：   groupPolicy: "allowlist",
代码：   groupAllowFrom: ["@owner:example.org"],
代码：   groups: {
代码：     "!roomId:example.org": { allow: true },
代码：     "#alias:example.org": { allow: true },
代码：   },
代码： },
代码： },
代码： },
代码： }

```

| 策略 | 行为 |

| ----- | ----- |

| "open" | 群组绕过允许列表；提及限制仍然适用。 |

| "disabled" | 完全阻止所有群组消息。 |

| "allowlist" | 仅允许与配置的允许列表匹配的群组/房间。 |

注意事项：

- groupPolicy 与提及限制（需要 @ 提及）是分开的。
- WhatsApp/Telegram/Signal/iMessage/Microsoft Teams：使用 groupAllowFrom（回退：显式 allowFrom）。
- Discord：允许列表使用 channels.discord.guilds.<id>.channels。
- Slack：允许列表使用 channels.slack.channels。
- Matrix：允许列表使用 channels.matrix.groups（房间 ID、别名或名称）。使用 channels.matrix.groupAllowFrom 限制发送者；也支持每个房间的 users 允许列表。
- 群组私信单独控制（channels.discord.dm.、channels.slack.dm.）。
- Telegram 允许列表可以匹配用户 ID（"123456789"、"telegram:123456789"、"tg:123456789"）或用户名（"@alice" 或 "alice"）；前缀不区分大小写。
- 默认为 groupPolicy: "allowlist"；如果你的群组允许列表为空，群组消息将被阻止。

快速心智模型（群组消息的评估顺序）：

- groupPolicy（open/disabled/allowlist）
- 群组允许列表（.groups、.groupAllowFrom、渠道特定允许列表）
- 提及限制（requireMention、/activation）

提及限制（默认）

群组消息需要提及，除非按群组覆盖。默认值位于 .groups."" 下的每个子系统中。

回复机器人消息被视为隐式提及（当渠道支持回复元数据时）。这适用于 Telegram、WhatsApp、Slack、Discord 和 Microsoft Teams。

代码：{

代码： channels: {

代码： whatsapp: {

代码： groups: {

代码： "" : { requireMention: true },

代码： "123@g.us": { requireMention: false },

代码： },

代码： },

代码： telegram: {
代码： groups: {
代码： "": { requireMention: true },
代码： "123456789": { requireMention: false },
代码： },
代码： },
代码： imessage: {
代码： groups: {
代码： "": { requireMention: true },
代码： "123": { requireMention: false },
代码： },
代码： },
代码： },
代码： agents: {
代码： list: [
代码： {
代码： id: "main",
代码： groupChat: {
代码： mentionPatterns: ["@openclaw", "openclaw", "\\+15555550123"],
代码： historyLimit: 50,
代码： },
代码： },
代码：],
代码： },
代码： }

注意事项：

- mentionPatterns 是不区分大小写的正则表达式。
- 提供显式提及的平台仍然通过；模式是回退。
- 每个智能体覆盖：agents.list[].groupChat.mentionPatterns（当多个智能体共享一个群组时
有用）。

- 提及限制仅在提及检测可行时强制执行（原生提及或 mentionPatterns 已配置）。
- Discord 默认值位于 channels.discord.guilds.（可按服务器/频道覆盖）。
- 群组历史上下文在渠道间统一包装，并且是仅待处理（由于提及限制而跳过的消息）；使用 messages.groupChat.historyLimit 作为全局默认值，使用 channels.<channel>.historyLimit（或 channels.<channel>.accounts..historyLimit）进行覆盖。设置 0 以禁用。

群组/频道工具限制（可选）

某些渠道配置支持限制特定群组/房间/频道内可用的工具。

- tools：为整个群组允许/拒绝工具。
- toolsBySender：群组内的按发送者覆盖（键是发送者 ID/用户名/邮箱/电话号码，取决于渠道）。使用 "" 作为通配符。

解析顺序（最具体的优先）：

- 群组/频道 toolsBySender 匹配
- 群组/频道 tools
- 默认 (") toolsBySender 匹配
- 默认 (") tools

示例 (Telegram)：

代码：{

代码： channels: {

代码： telegram: {

代码： groups: {

代码： "": { tools: { deny: ["exec"] } },

代码： "-1001234567890": {

代码： tools: { deny: ["exec", "read", "write"] },

代码： toolsBySender: {

代码： "123456789": { alsoAllow: ["exec"] },

代码： },

代码： },

代码： },

代码：},

代码：},

代码：}

注意事项：

- 群组/频道工具限制在全局/智能体工具策略之外额外应用（deny 仍然优先）。
- 某些渠道对房间/频道使用不同的嵌套结构（例如，Discord guilds..channels.、Slack channels.、MS Teams teams..channels.）。

群组允许列表

当配置了 channels.whatsapp.groups、channels.telegram.groups 或 channels.imessage.groups 时，键作为群组允许列表。使用 "" 允许所有群组，同时仍设置默认提及行为。

常见意图（复制/粘贴）：

- 禁用所有群组回复

代码：{

代码： channels: { whatsapp: { groupPolicy: "disabled" } },

代码：}

- 仅允许特定群组（WhatsApp）

代码：{

代码： channels: {

代码： whatsapp: {

代码： groups: {

代码： "123@g.us": { requireMention: true },

代码： "456@g.us": { requireMention: false },

代码： },

代码： },

代码： },

代码：}

- 允许所有群组但需要提及（显式）

```
代码：{
代码： channels: {
代码： whatsapp: {
代码： groups: { "*": { requireMention: true } },
代码： },
代码： },
代码： }
```

- 仅所有者可以在群组中触发（WhatsApp）

```
代码：{
代码： channels: {
代码： whatsapp: {
代码： groupPolicy: "allowlist",
代码： groupAllowFrom: ["+15551234567"],
代码： groups: { "*": { requireMention: true } },
代码： },
代码： },
代码： }
```

激活（仅所有者）

群组所有者可以切换每个群组的激活状态：

- /activation mention
- /activation always

所有者由 channels.whatsapp.allowFrom 确定（未设置时为机器人自身的 E.164）。将命令作为独立消息发送。其他平台目前忽略 /activation。

上下文字段

群组入站负载设置：

- ChatType=group
- GroupSubject（如果已知）

- GroupMembers（如果已知）
- WasMentioned（提及限制结果）
- Telegram 论坛话题还包括 MessageThreadId 和 IsForum。

智能体系统提示在新群组会话的第一轮包含群组介绍。它提醒模型像人类一样回复，避免 Markdown 表格，避免输入字面量 \n 序列。

iMessage 特定内容

- 路由或允许列表时优先使用 chat_id:<id>。
- 列出聊天：imsg chats --limit 20。
- 群组回复始终返回到相同的 chat_id。

WhatsApp 特定内容

参见群消息了解 WhatsApp 专有行为（历史注入、提及处理详情）。

10. iMessage (imsg)

iMessage (imsg)

状态：外部 CLI 集成。Gateway 网关生成 imsg rpc（基于 stdio 的 JSON-RPC）。

快速设置（新手）

- 确保在此 Mac 上已登录"信息"。
- 安装 imsg：
- brew install steipete/tap/imsg
- 配置 OpenClaw 的 channels.imessage.cliPath 和 channels.imessage.dbPath。
- 启动 Gateway 网关并批准所有 macOS 提示（自动化 + 完全磁盘访问权限）。

最小配置：

代码：{

代码： channels: {

代码： imessage: {

代码： enabled: true,

代码： cliPath: "/usr/local/bin/imsg",

代码： dbPath: "/Users/<you>/Library/Messages/chat.db",

代码： },

代码： },

代码： }

简介

- 基于 macOS 上 imsg 的 iMessage 渠道。
- 确定性路由：回复始终返回到 iMessage。
-

私信共享智能体的主会话；群组是隔离的（agent:<agentId>:imessage:group:<chat_id>）。

- 如果多参与者会话以 is_group=false 到达，你仍可使用 channels.imessage.groups 按 chat_id 隔离（参见下方"类群组会话"）。

配置写入

默认情况下，iMessage 允许写入由 /config set|unset 触发的配置更新（需要 commands.config: true）。

禁用方式：

代码： {

代码： channels: { imessage: { configWrites: false } },

代码： }

要求

- 已登录"信息"的 macOS。
- OpenClaw + imsg 的完全磁盘访问权限（访问"信息"数据库）。
- 发送时需要自动化权限。
- channels.imessage.cliPath 可以指向任何代理 stdin/stdout 的命令（例如，通过 SSH 连接到另一台 Mac 并运行 imsg rpc 的包装脚本）。

设置（快速路径）

- 确保在此 Mac 上已登录"信息"。
- 配置 iMessage 并启动 Gateway 网关。

专用机器人 macOS 用户（用于隔离身份）

如果你希望机器人从独立的 iMessage 身份发送（并保持你的个人"信息"整洁），请使用专用 Apple ID + 专用 macOS 用户。

- 创建专用 Apple ID（例如：my-cool-bot@icloud.com）。
- Apple 可能需要电话号码进行验证 / 2FA。
- 创建 macOS 用户（例如：openclawhome）并登录。
- 在该 macOS 用户中打开"信息"并使用机器人 Apple ID 登录 iMessage。
- 启用远程登录（系统设置 → 通用 → 共享 → 远程登录）。
- 安装 imsg：
 - brew install steipete/tap/imshow
- 设置 SSH 使 ssh <bot-macos-user>@localhost true 无需密码即可工作。
- 将 channels.imessage.accounts.bot.cliPath 指向以机器人用户身份运行 imshow 的 SSH 包装脚本。

首次运行注意事项：发送/接收可能需要在机器人 macOS 用户中进行 GUI 批准（自动化 + 完全磁盘访问权限）。如果 imshow rpc

看起来卡住或退出，请登录该用户（屏幕共享很有帮助），运行一次 imshow chats --limit 1 / imshow send ...，批准提示，然后重试。

示例包装脚本（chmod +x）。将 <bot-macos-user> 替换为你的实际 macOS 用户名：

代码：#!/usr/bin/env bash

代码：set -euo pipefail

代码：# Run an interactive SSH once first to accept host keys:

代码：# ssh <bot-macos-user>@localhost true

代码：exec /usr/bin/ssh -o BatchMode=yes -o ConnectTimeout=5 -T
<bot-macos-user>@localhost \

代码："/usr/local/bin/imshow" "\$@"

示例配置：

代码：{

代码：channels: {

代码：imessage: {

代码：enabled: true,

代码：accounts: {

代码：bot: {

代码：name: "Bot",

代码：enabled: true,

代码：cliPath: "/path/to/imshow-bot",

代码：dbPath: "/Users/<bot-macos-user>/Library/Messages/chat.db",

代码：},

代码：},

代码：},

代码：},

代码：}

对于单账户设置，使用扁平选项（channels.imessage.cliPath、channels.imessage.dbPath）而不是 accounts 映射。

远程/SSH 变体（可选）

如果你想在另一台 Mac 上使用 iMessage，请将 channels.imessage.cliPath 设置为通过 SSH 在远程 macOS 主机上运行 imshow 的包装脚本。OpenClaw 只需要 stdio。

示例包装脚本：

代码：#!/usr/bin/env bash

代码：exec ssh -T gateway-host imshow "\$@"

远程附件：当 cliPath 通过 SSH

指向远程主机时，“信息”数据库中的附件路径引用的是远程机器上的文件。OpenClaw 可以通过设置 channels.imessage.remoteHost 自动通过 SCP 获取这些文件：

代码：{

代码：channels: {

代码：imessage: {

代码：cliPath: "~/imshow-ssh", // SSH wrapper to remote Mac

代码：remoteHost: "user@gateway-host", // for SCP file transfer

代码：includeAttachments: true,

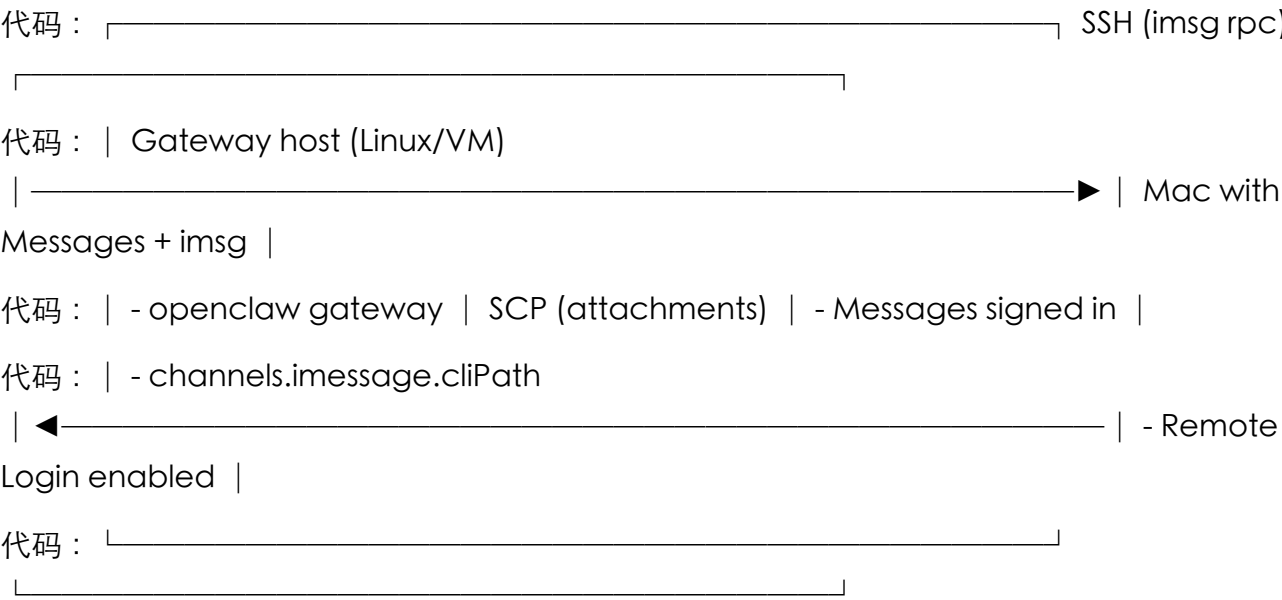
```
代码： },
代码： },
代码： }
```

如果未设置 remoteHost，OpenClaw 会尝试通过解析包装脚本中的 SSH 命令自动检测。建议显式配置以提高可靠性。

通过 Tailscale 连接远程 Mac（示例）

如果 Gateway 网关运行在 Linux 主机/虚拟机上但 iMessage 必须运行在 Mac 上，Tailscale 是最简单的桥接方式：Gateway 网关通过 tailnet 与 Mac 通信，通过 SSH 运行 imsg，并通过 SCP 获取附件。

架构：



```
代码： ▲
代码： | Tailscale tailnet (hostname or 100.x.y.z)
代码： ▼
代码： user@gateway-host
```

具体配置示例（Tailscale 主机名）：

```
代码： {
代码：   channels: {
代码：     imessage: {
代码：       enabled: true,
```

代码：cliPath: "~/openclaw/scripts/imshow-ssh",
代码：remoteHost: "bot@mac-mini.tailnet-1234.ts.net",
代码：includeAttachments: true,
代码：dbPath: "/Users/bot/Library/Messages/chat.db",
代码：},
代码：},
代码：}

示例包装脚本（~/openclaw/scripts/imshow-ssh）：

代码：#!/usr/bin/env bash
代码：exec ssh -T bot@mac-mini.tailnet-1234.ts.net imshow "\$@"

注意事项：

- 确保 Mac 已登录"信息"，并已启用远程登录。
- 使用 SSH 密钥使 ssh bot@mac-mini.tailnet-1234.ts.net 无需提示即可工作。
- remoteHost 应与 SSH 目标匹配，以便 SCP 可以获取附件。

多账户支持：使用 channels.imessage.accounts 配置每个账户及可选的 name。参见 gateway/configuration 了解共享模式。不要提交 ~/openclaw/openclaw.json（它通常包含令牌）。

访问控制（私信 + 群组）

私信：

- 默认：channels.imessage.dmPolicy = "pairing"。
- 未知发送者会收到配对码；消息在批准前会被忽略（配对码在 1 小时后过期）。
- 批准方式：
- openclaw pairing list imessage
- openclaw pairing approve imessage <CODE>
- 配对是 iMessage 私信的默认令牌交换方式。详情：配对

群组：

- channels.imessage.groupPolicy = open | allowlist | disabled。

- 设置 allowlist 时，channels.imessage.groupAllowFrom 控制谁可以在群组中触发。
- 提及检测使用 agents.list[].groupChat.mentionPatterns（或 messages.groupChat.mentionPatterns），因为 iMessage 没有原生提及元数据。
- 多智能体覆盖：在 agents.list[].groupChat.mentionPatterns 上设置每个智能体的模式。

工作原理（行为）

- imsg 流式传输消息事件；Gateway 网关将它们规范化为共享渠道信封。
- 回复始终路由回相同的 chat id 或 handle。

类群组会话（`is\_group=false`）

某些 iMessage 会话可能有多个参与者，但根据"信息"存储聊天标识符的方式，仍以 is_group=false 到达。

如果你在 channels.imessage.groups 下显式配置了 chat_id，OpenClaw 会将该会话视为"群组"用于：

- 会话隔离（独立的 agent:<agentId>:imessage:group:<chat_id> 会话键）
- 群组允许列表 / 提及检测行为

示例：

代码：{

代码： channels: {

代码： imessage: {

代码： groupPolicy: "allowlist",

代码： groupAllowFrom: ["+15555550123"],

代码： groups: {

代码： "42": { requireMention: false },

代码： },

代码： },

代码： },

代码： }

当你想为特定会话使用隔离的个性/模型时这很有用（参见多智能体路由）。关于文件系统隔离，参见沙箱隔离。

媒体 + 限制

- 通过 `channels.imessage.includeAttachments` 可选附件摄取。
- 通过 `channels.imessage.mediaMaxMb` 设置媒体上限。

限制

- 出站文本按 `channels.imessage.textChunkLimit` 分块（默认 4000）。
- 可选换行分块：设置 `channels.imessage.chunkMode="newline"` 在长度分块前按空行（段落边界）分割。
- 媒体上传受 `channels.imessage.mediaMaxMb` 限制（默认 16）。

寻址 / 投递目标

优先使用 `chat_id` 进行稳定路由：

- `chat_id:123`（推荐）
- `chat_guid:...`
- `chat_identifier:...`
- 直接 handle：`imessage:+1555 / sms:+1555 / user@example.com`

列出聊天：

代码：`imsg chats --limit 20`

配置参考 (iMessage)

完整配置：配置

提供商选项：

- `channels.imessage.enabled`：启用/禁用渠道启动。
- `channels.imessage.cliPath`：imsg 路径。
- `channels.imessage.dbPath`："信息"数据库路径。
- `channels.imessage.remoteHost`：当 `cliPath` 指向远程 Mac 时用于 SCP 附件传输的 SSH 主机（例如 `user@gateway-host`）。如未设置则从 SSH 包装脚本自动检测。
- `channels.imessage.service`：`imessage` | `sms` | `auto`。
- `channels.imessage.region`：短信区域。
- `channels.imessage.dmPolicy`：`pairing` | `allowlist` | `open` | `disabled`（默认：`pairing`）。

- `channels.imessage.allowFrom` : 私信允许列表 (`handle`、邮箱、E.164 号码或 `chat_id`) 。 `open` 需要 "" 。 `iMessage` 没有用户名；使用 `handle` 或聊天目标。
- `channels.imessage.groupPolicy` : `open` | `allowlist` | `disabled` (默认: `allowlist`) 。
- `channels.imessage.groupAllowFrom` : 群组发送者允许列表。
- `channels.imessage.historyLimit` / `channels.imessage.accounts..historyLimit` : 作为上下文包含的最大群组消息数 (0 禁用) 。
- `channels.imessage.dmHistoryLimit` : 私信历史限制 (用户轮次) 。 每用户覆盖: `channels.imessage.dms["<handle>"].historyLimit` 。
- `channels.imessage.groups` : 每群组默认值 + 允许列表 (使用 "" 作为全局默认值) 。
- `channels.imessage.includeAttachments` : 将附件摄取到上下文。
- `channels.imessage.mediaMaxMb` : 入站/出站媒体上限 (MB) 。
- `channels.imessage.textChunkLimit` : 出站分块大小 (字符) 。
- `channels.imessage.chunkMode` : `length` (默认) 或 `newline` 在长度分块前按空行 (段落边界) 分割。

相关全局选项：

- `agents.list[].groupChat.mentionPatterns` (或 `messages.groupChat.mentionPatterns`) 。
- `messages.responsePrefix` 。

11. 聊天渠道

聊天渠道

OpenClaw 可以在你已经使用的任何聊天应用上与你交流。每个渠道通过 Gateway 网关连接。所有渠道都支持文本；媒体和表情回应的支持因渠道而异。

支持的渠道

- WhatsApp — 最受欢迎；使用 Baileys，需要二维码配对。
- Telegram — 通过 grammY 使用 Bot API；支持群组。
- Discord — Discord Bot API + Gateway；支持服务器、频道和私信。
- Slack — Bolt SDK；工作区应用。
- 飞书 — 飞书 (Lark) 机器人 (插件，需单独安装) 。
- Google Chat — 通过 HTTP webhook 的 Google Chat API 应用。
- Mattermost — Bot API + WebSocket；频道、群组、私信 (插件，需单独安装) 。

- Signal — signal-cli；注重隐私。
- BlueBubbles — 推荐用于 iMessage；使用 BlueBubbles macOS 服务器 REST API，功能完整（编辑、撤回、特效、回应、群组管理——编辑功能在 macOS 26 Tahoe 上目前不可用）。
- iMessage（旧版） — 通过 imsg CLI 的旧版 macOS 集成（已弃用，新设置请使用 BlueBubbles）。
- Microsoft Teams — Bot Framework；企业支持（插件，需单独安装）。
- LINE — LINE Messaging API 机器人（插件，需单独安装）。
- Nextcloud Talk — 通过 Nextcloud Talk 的自托管聊天（插件，需单独安装）。
- Matrix — Matrix 协议（插件，需单独安装）。
- Nostr — 通过 NIP-04 的去中心化私信（插件，需单独安装）。
- Tlon — 基于 Urbit 的消息应用（插件，需单独安装）。
- Twitch — 通过 IRC 连接的 Twitch 聊天（插件，需单独安装）。
- Zalo — Zalo Bot API；越南流行的消息应用（插件，需单独安装）。
- Zalo Personal — 通过二维码登录的 Zalo 个人账号（插件，需单独安装）。
- WebChat — 基于 WebSocket 的 Gateway 网关 WebChat 界面。

注意事项

- 渠道可以同时运行；配置多个渠道后，OpenClaw 会按聊天进行路由。
- 最快的设置方式通常是 Telegram（简单的机器人令牌）。WhatsApp 需要二维码配对，并在磁盘上存储更多状态。
- 群组行为因渠道而异；参见群组。
- 为安全起见，私信配对和允许列表会被强制执行；参见安全。
- Telegram 内部机制：grammY 说明。
- 故障排除：渠道故障排除。
- 模型提供商单独记录；参见模型提供商。

12. LINE（插件）

LINE（插件）

LINE 通过 LINE Messaging API 连接到 OpenClaw。该插件作为 webhook 接收器在 Gateway 网关上运行，使用你的 channel access token + channel secret 进行身份验证。

状态：通过插件支持。支持私信、群聊、媒体、位置、Flex 消息、模板消息和快捷回复。不支持表情回应和话题回复。

需要安装插件

安装 LINE 插件：

代码：openclaw plugins install @openclaw/line

本地检出（从 git 仓库运行时）：

代码：openclaw plugins install ./extensions/line

配置步骤

- 创建 LINE Developers 账户并打开控制台：
- 创建（或选择）一个 Provider 并添加 Messaging API 渠道。
- 从渠道设置中复制 Channel access token 和 Channel secret。
- 在 Messaging API 设置中启用 Use webhook。
- 将 webhook URL 设置为你的 Gateway 网关端点（必须使用 HTTPS）：

代码：<https://gateway-host/line/webhook>

Gateway 网关会响应 LINE 的 webhook

验证（GET）和入站事件（POST）。如果你需要自定义路径，请设置 `channels.line.webhookPath` 或 `channels.line.accounts.<id>.webhookPath` 并相应更新 URL。

配置

最小配置：

代码：

```
{
```

代码：

```
  channels: {
```

代码：

```
    line: {
```

代码：

```
      enabled: true,
```

代码：

```
      channelAccessToken: "LINE_CHANNEL_ACCESS_TOKEN",
```

代码：

```
      channelSecret: "LINE_CHANNEL_SECRET",
```

代码：

```
      dmPolicy: "pairing",
```


代码： },

代码： },

代码： }

环境变量（仅限默认账户）：

- LINE_CHANNEL_ACCESS_TOKEN

- LINE_CHANNEL_SECRET

Token/secret 文件：

代码： {

代码： channels: {

代码： line: {

代码： tokenFile: "/path/to/line-token.txt",

代码： secretFile: "/path/to/line-secret.txt",

代码： },

代码： },

代码： }

多账户配置：

代码： {

代码： channels: {

代码： line: {

代码： accounts: {

代码： marketing: {

代码： channelAccessToken: "...",

代码： channelSecret: "...",

代码： webhookPath: "/line/marketing",

代码： },

代码： },

代码： },

代码： },

代码：}

访问控制

私信默认使用配对模式。未知发送者会收到配对码，其消息在获得批准前会被忽略。

代码：openclaw pairing list line

代码：openclaw pairing approve line <CODE>

允许列表和策略：

- channels.line.dmPolicy：pairing | allowlist | open | disabled
- channels.line.allowFrom：私信的允许列表 LINE 用户 ID
- channels.line.groupPolicy：allowlist | open | disabled
- channels.line.groupAllowFrom：群组的允许列表 LINE 用户 ID
- 单群组覆盖：channels.line.groups.<groupId>.allowFrom

LINE ID 区分大小写。有效 ID 格式如下：

- 用户：U + 32 位十六进制字符
- 群组：C + 32 位十六进制字符
- 房间：R + 32 位十六进制字符

消息行为

- 文本按 5000 字符分块。
- Markdown 格式会被移除；代码块和表格会尽可能转换为 Flex 卡片。
- 流式响应会被缓冲；智能体处理时，LINE 会收到完整分块并显示加载动画。
- 媒体下载受 channels.line.mediaMaxMb 限制（默认 10）。

渠道数据（富消息）

使用 channelData.line 发送快捷回复、位置、Flex 卡片或模板消息。

代码：{

代码：text: "Here you go",

代码：channelData: {

代码：line: {

代码： quickReplies: ["Status", "Help"],

代码： location: {

代码： title: "Office",

代码： address: "123 Main St",

代码： latitude: 35.681236,

代码： longitude: 139.767125,

代码： },

代码： flexMessage: {

代码： altText: "Status card",

代码： contents: {

代码： /* Flex payload */

代码： },

代码： },

代码： templateMessage: {

代码： type: "confirm",

代码： text: "Proceed?",

代码： confirmLabel: "Yes",

代码： confirmData: "yes",

代码： cancelLabel: "No",

代码： cancelData: "no",

代码： },

代码： },

代码： },

代码： }

LINE 插件还提供 /card 命令用于 Flex 消息预设：

代码： /card info "Welcome" "Thanks for joining!"

故障排除

- Webhook 验证失败： 确保 webhook URL 使用 HTTPS 且 channelSecret 与 LINE 控制台的一致。

- 没有进站事件：确认 webhook 路径与 channels.line.webhookPath 匹配，且 Gateway 网关可从 LINE 访问。
- 媒体下载错误：如果媒体超过默认限制，请提高 channels.line.mediaMaxMb。

13. 渠道位置解析

渠道位置解析

OpenClaw 将聊天渠道中分享的位置标准化为：

- 附加到进站消息体的可读文本，以及
- 自动回复上下文负载中的结构化字段。

目前支持：

- Telegram（位置图钉 + 地点 + 实时位置）
- WhatsApp（locationMessage + liveLocationMessage）
- Matrix（m.location 配合 geo_uri）

文本格式

位置以友好的行格式呈现，不带括号：

- 图钉：
- 48.858844, 2.294351 ±12m
- 命名地点：
- Eiffel Tower — Champ de Mars, Paris (48.858844, 2.294351 ±12m)
- 实时分享：
- Live location: 48.858844, 2.294351 ±12m

如果渠道包含标题/评论，会附加在下一行：

代码：48.858844, 2.294351 ±12m

代码：Meet here

上下文字段

当存在位置信息时，以下字段会被添加到 ctx 中：

- LocationLat（数字）

- LocationLon（数字）
- LocationAccuracy（数字，米；可选）
- LocationName（字符串；可选）
- LocationAddress（字符串；可选）
- LocationSource（pin | place | live）
- LocationIsLive（布尔值）

渠道说明

- Telegram：地点映射到 LocationName/LocationAddress；实时位置使用 live_period。
- WhatsApp：locationMessage.comment 和 liveLocationMessage.caption 作为标题行附加。
- Matrix：geo_uri 解析为图钉位置；忽略海拔高度，LocationIsLive 始终为 false。

14. Matrix（插件）

Matrix（插件）

Matrix 是一个开放的去中心化消息协议。OpenClaw 以 Matrix 用户身份连接到任意主服务器，因此你需要为机器人创建一个 Matrix 账户。登录后，你可以直接私信机器人或邀请它加入房间（Matrix"群组"）。Beeper 也是一个有效的客户端选项，但它需要启用 E2EE。

状态：通过插件（@vector-im/matrix-bot-sdk）支持。支持私信、房间、话题、媒体、表情回应、投票（发送 + poll-start 作为文本）、位置和 E2EE（需要加密支持）。

需要插件

Matrix 作为插件提供，不包含在核心安装中。

通过 CLI 安装（npm 仓库）：

代码：openclaw plugins install @openclaw/matrix

本地检出（从 git 仓库运行时）：

代码：openclaw plugins install ./extensions/matrix

如果你在配置/新手引导期间选择 Matrix 并检测到 git 检出，OpenClaw 将自动提供本地安装路径。

详情：插件

设置

- 安装 Matrix 插件：
- 从 npm：openclaw plugins install @openclaw/matrix
- 从本地检出：openclaw plugins install ./extensions/matrix
- 在主服务器上创建 Matrix 账户：
- 在 浏览托管选项
- 或自行托管。
- 获取机器人账户的访问令牌：
- 在你的主服务器上使用 curl 调用 Matrix 登录 API：

代码：curl --request POST \

代码：--url https://matrix.example.org/_matrix/client/v3/login \

代码：--header 'Content-Type: application/json' \

代码：--data '{

代码："type": "m.login.password",

代码："identifier": {

代码："type": "m.id.user",

代码："user": "your-user-name"

代码：},

代码："password": "your-password"

代码：}'

- 将 matrix.example.org 替换为你的主服务器 URL。
- 或设置 channels.matrix.userId + channels.matrix.password：OpenClaw 会调用相同的登录端点，将访问令牌存储在 ~/.openclaw/credentials/matrix/credentials.json，并在下次启动时重用。

- 配置凭证：
- 环境变量：MATRIX_HOMESERVER、MATRIX_ACCESS_TOKEN（或 MATRIX_USER_ID + MATRIX_PASSWORD）
- 或配置：channels.matrix.

- 如果两者都设置，配置优先。
- 使用访问令牌时：用户 ID 通过 /whoami 自动获取。
- 设置时，channels.matrix.userId 应为完整的 Matrix ID（示例：@bot:example.org）。
- 重启 Gateway 网关（或完成新手引导）。
- 从任何 Matrix 客户端（Element、Beeper 等；参见 需要 E2EE，因此请设置 channels.matrix.encryption: true 并验证设备）。

最小配置（访问令牌，用户 ID 自动获取）：

```
代码：{
代码： channels: {
代码： matrix: {
代码： enabled: true,
代码： homeserver: "https://matrix.example.org",
代码： accessToken: "syt_***",
代码： dm: { policy: "pairing" },
代码： },
代码： },
代码： }
```

E2EE 配置（启用端到端加密）：

```
代码：{
代码： channels: {
代码： matrix: {
代码： enabled: true,
代码： homeserver: "https://matrix.example.org",
代码： accessToken: "syt_***",
代码： encryption: true,
代码： dm: { policy: "pairing" },
代码： },
代码： },
代码： }
```

加密 (E2EE)

通过 Rust 加密 SDK 支持端到端加密。

使用 `channels.matrix.encryption: true` 启用：

- 如果加密模块加载成功，加密房间会自动解密。
- 发送到加密房间时，出站媒体会被加密。
- 首次连接时，OpenClaw 会向你的其他会话请求设备验证。
- 在另一个 Matrix 客户端 (Element 等) 中验证设备以启用密钥共享。
- 如果无法加载加密模块，E2EE 将被禁用，加密房间将无法解密；OpenClaw 会记录警告。
- 如果你看到缺少加密模块的错误 (例如 @matrix-org/matrix-sdk-crypto-nodejs-)，请允许 @matrix-org/matrix-sdk-crypto-nodejs 的构建脚本并运行 `pnpm rebuild @matrix-org/matrix-sdk-crypto-nodejs`，或使用 `node node_modules/@matrix-org/matrix-sdk-crypto-nodejs/download-lib.js` 获取二进制文件。

加密状态按账户 + 访问令牌存储在 `~/.openclaw/matrix/accounts/<account>/<homeserver><user>/<token-hash>/crypto/` (SQLite 数据库)。同步状态存储在同目录的 `bot-storage.json`

中。如果访问令牌 (设备) 更改，将创建新的存储，机器人必须重新验证才能访问加密房间。

设备验证：

启用 E2EE 时，机器人将在启动时向你的其他会话请求验证。打开 Element (或其他客户端) 并批准验证请求以建立信任。验证后，机器人可以解密加密房间中的消息。

路由模型

- 回复始终返回到 Matrix。
- 私信共享智能体的主会话；房间映射到群组会话。

访问控制 (私信)

- 默认：`channels.matrix.dm.policy = "pairing"`。未知发送者会收到配对码。
- 通过以下方式批准：
- `openclaw pairing list matrix`
- `openclaw pairing approve matrix <CODE>`
- 公开私信：`channels.matrix.dm.policy="open"` 加上 `channels.matrix.dm.allowFrom=[""]`。

- `channels.matrix.dm.allowFrom` 仅接受完整 Matrix 用户 ID（例如 `@user:server`）。向导仅在目录搜索得到唯一精确匹配时将显示名称解析为用户 ID。

房间（群组）

- 默认：`channels.matrix.groupPolicy = "allowlist"`（提及门控）。使用 `channels.defaults.groupPolicy` 在未设置时覆盖默认值。
- 使用 `channels.matrix.groups` 配置房间允许列表（房间 ID 或别名；名称仅在目录搜索得到唯一精确匹配时解析为 ID）：

代码：{

代码：channels: {

代码：matrix: {

代码：groupPolicy: "allowlist",

代码：groups: {

代码："!roomId:example.org": { allow: true },

代码："#alias:example.org": { allow: true },

代码：},

代码：groupAllowFrom: ["@owner:example.org"],

代码：},

代码：},

代码：}

- `requireMention: false` 启用该房间的自动回复。
- `groups. ""` 可以设置跨房间的提及门控默认值。
- `groupAllowFrom` 限制哪些发送者可以在房间中触发机器人（需完整 Matrix 用户 ID）。
- 每个房间的 `users` 允许列表可以进一步限制特定房间内的发送者（需完整 Matrix 用户 ID）。
- 配置向导会提示输入房间允许列表（房间 ID、别名或名称），仅在精确且唯一匹配时解析名称。
- 启动时，OpenClaw 将允许列表中的房间/用户名称解析为 ID 并记录映射；未解析的条目不会参与允许列表匹配。
- 默认自动加入邀请；使用 `channels.matrix.autoJoin` 和 `channels.matrix.autoJoinAllowlist` 控制。
- 要禁止所有房间，设置 `channels.matrix.groupPolicy: "disabled"`（或保持空的允许列表）。
- 旧版键名：`channels.matrix.rooms`（与 `groups` 相同的结构）。

话题

- 支持回复话题。
- `channels.matrix.threadReplies` 控制回复是否保持在话题中：
 - `off`、`inbound`（默认）、`always`
- `channels.matrix.replyToMode` 控制不在话题中回复时的 `reply-to` 元数据：
 - `off`（默认）、`first`、`all`

功能

| 功能 | 状态 |

| ----- | ----- |

| 私信 | 支持 |

| 房间 | 支持 |

| 话题 | 支持 |

| 媒体 | 支持 |

| E2EE | 支持（需要加密模块） |

| 表情回应 | 支持（通过工具发送/读取） |

| 投票 | 支持发送；入站投票开始转换为文本（响应/结束被忽略） |

| 位置 | 支持（geo URI；忽略海拔） |

| 原生命令 | 支持 |

配置参考（Matrix）

完整配置：配置

提供商选项：

- `channels.matrix.enabled`：启用/禁用渠道启动。
- `channels.matrix.homeserver`：主服务器 URL。
- `channels.matrix.userId`：Matrix 用户 ID（使用访问令牌时可选）。
- `channels.matrix.accessToken`：访问令牌。
- `channels.matrix.password`：登录密码（令牌会被存储）。
- `channels.matrix.deviceName`：设备显示名称。
- `channels.matrix.encryption`：启用 E2EE（默认：`false`）。

- `channels.matrix.initialSyncLimit` : 初始同步限制。
- `channels.matrix.threadReplies` : `off` | `inbound` | `always` (默认 : `inbound`) 。
- `channels.matrix.textChunkLimit` : 出站文本分块大小 (字符) 。
- `channels.matrix.chunkMode` : `length` (默认) 或 `newline`
在长度分块前按空行 (段落边界) 分割。
- `channels.matrix.dm.policy` : `pairing` | `allowlist` | `open` | `disabled` (默认 : `pairing`) 。
- `channels.matrix.dm.allowFrom` : 私信允许列表 (需完整 Matrix 用户 ID) 。
- `open` 需要 "" 。
- 向导在可能时将名称解析为 ID 。
- `channels.matrix.groupPolicy` : `allowlist` | `open` | `disabled` (默认 : `allowlist`) 。
- `channels.matrix.groupAllowFrom` : 群组消息的允许发送者列表 (需完整 Matrix 用户 ID) 。
- `channels.matrix.allowlistOnly` : 强制私信 + 房间使用允许列表规则 。
- `channels.matrix.groups` : 群组允许列表 + 每个房间的设置映射 。
- `channels.matrix.rooms` : 旧版群组允许列表/配置 。
- `channels.matrix.replyToMode` : 话题/标签的 `reply-to` 模式 。
- `channels.matrix.mediaMaxMb` : 入站/出站媒体上限 (MB) 。
- `channels.matrix.autoJoin` : 邀请处理 (`always` | `allowlist` | `off` , 默认 : `always`) 。
- `channels.matrix.autoJoinAllowlist` : 自动加入的允许房间 ID/别名 。
- `channels.matrix.actions` : 每个操作的工具限制 (`reactions/messages/pins/memberInfo/channelInfo`) 。

15. Mattermost (插件)

Mattermost (插件)

状态 : 通过插件支持 (bot token + WebSocket 事件) 。

Mattermost 是一个可自托管的团队消息平台 ; 有关产品详情和下载 , 请访问官方网站 mattermost.com 。

需要插件

Mattermost 以插件形式提供 , 不包含在核心安装中 。

通过 CLI 安装 (npm 注册表) :

代码 : `openclaw plugins install @openclaw/mattermost`

本地检出（从 git 仓库运行时）：

代码：openclaw plugins install ./extensions/mattermost

如果你在配置/新手引导期间选择 Mattermost 并检测到 git 检出，OpenClaw 会自动提供本地安装路径。

详情：插件

快速设置

- 安装 Mattermost 插件。
- 创建 Mattermost bot 账户并复制 bot token。
- 复制 Mattermost 基础 URL（例如 `
- 配置 OpenClaw 并启动 Gateway 网关。

最小配置：

代码：{

代码： channels: {

代码： mattermost: {

代码： enabled: true,

代码： botToken: "mm-token",

代码： baseUrl: "https://chat.example.com",

代码： dmPolicy: "pairing",

代码： },

代码： },

代码： }

环境变量（默认账户）

如果你偏好使用环境变量，请在 Gateway 网关主机上设置：

- MATTERMOST_BOT_TOKEN=...
- `MATTERMOST_URL=

环境变量仅适用于默认账户（default）。其他账户必须使用配置值。

聊天模式

Mattermost 自动响应私信。频道行为由 chatmode 控制：

- oncall（默认）：仅在频道中被 @提及时响应。
- onmessage：响应每条频道消息。
- onchar：当消息以触发前缀开头时响应。

配置示例：

代码：{

代码： channels: {

代码： mattermost: {

代码： chatmode: "onchar",

代码： oncharPrefixes: [">", "!"],

代码： },

代码： },

代码： }

注意事项：

- onchar 仍会响应显式 @提及。
- channels.mattermost.requireMention 对旧配置仍然有效，但推荐使用 chatmode。

访问控制（私信）

- 默认：channels.mattermost.dmPolicy = "pairing"（未知发送者会收到配对码）。
- 通过以下方式批准：
- openclaw pairing list mattermost
- openclaw pairing approve mattermost <CODE>
- 公开私信：channels.mattermost.dmPolicy="open" 加上 channels.mattermost.allowFrom=[""]。

频道（群组）

- 默认：channels.mattermost.groupPolicy = "allowlist"（提及限制）。

- 使用 `channels.mattermost.groupAllowFrom` 将发送者加入允许列表（用户 ID 或 `@username`）。
- 开放频道：`channels.mattermost.groupPolicy="open"`（提及限制）。

出站投递目标

在 `openclaw message send` 或 `cron/webhooks` 中使用这些目标格式：

- `channel:<id>` 用于频道
- `user:<id>` 用于私信
- `@username` 用于私信（通过 Mattermost API 解析）

裸 ID 被视为频道。

多账户

Mattermost 支持在 `channels.mattermost.accounts` 下配置多个账户：

代码：{

代码：channels: {

代码：mattermost: {

代码：accounts: {

代码：default: { name: "Primary", botToken: "mm-token", baseUrl: "https://chat.example.com" },

代码：alerts: { name: "Alerts", botToken: "mm-token-2", baseUrl: "https://alerts.example.com" },

代码：},

代码：},

代码：},

代码：}

故障排除

- 频道中无回复：确保 bot 在频道中并提及它（`oncall`），使用触发前缀（`onchar`），或设置 `chatmode: "onmessage"`。
- 认证错误：检查 bot token、基础 URL 以及账户是否已启用。
- 多账户问题：环境变量仅适用于 default 账户。

16. Microsoft Teams（插件）

Microsoft Teams（插件）

"进入此地者，放弃一切希望。"

更新时间：2026-01-21

状态：支持文本 + 私信附件；频道/群组文件发送需要 sharePointSiteId + Graph 权限（参见在群聊中发送文件）。投票通过 Adaptive Cards 发送。

需要插件

Microsoft Teams 作为插件提供，不包含在核心安装中。

破坏性变更（2026.1.15）：MS Teams 已从核心移出。如果你使用它，必须安装插件。

原因说明：保持核心安装更轻量，并让 MS Teams 依赖项可以独立更新。

通过 CLI 安装（npm 注册表）：

代码：openclaw plugins install @openclaw/msteams

本地检出（从 git 仓库运行时）：

代码：openclaw plugins install ./extensions/msteams

如果你在配置/新手引导过程中选择 Teams 并检测到 git 检出，OpenClaw 将自动提供本地安装路径。

详情：插件

快速设置（初学者）

- 安装 Microsoft Teams 插件。
- 创建一个 Azure Bot（App ID + 客户端密钥 + 租户 ID）。
- 使用这些凭证配置 OpenClaw。
- 通过公共 URL 或隧道暴露 /api/messages（默认端口 3978）。
- 安装 Teams 应用包并启动 Gateway 网关。

最小配置：

```
代码：{  
代码： channels: {  
代码： msteams: {  
代码： enabled: true,  
代码： appId: "<APP_ID>",  
代码： appPassword: "<APP_PASSWORD>",  
代码： tenantId: "<TENANT_ID>",  
代码： webhook: { port: 3978, path: "/api/messages" },  
代码： },  
代码： },  
代码： }
```

注意：群聊默认被阻止（channels.msteams.groupPolicy: "allowlist"）。要允许群组回复，请设置 channels.msteams.groupAllowFrom（或使用 groupPolicy: "open" 允许任何成员，需要提及才能触发）。

目标

- 通过 Teams 私信、群聊或频道与 OpenClaw 交流。
- 保持路由确定性：回复始终返回到消息到达的渠道。
- 默认使用安全的渠道行为（除非另有配置，否则需要提及）。

配置写入

默认情况下，Microsoft Teams 允许通过 /config set|unset 触发的配置更新写入（需要 commands.config: true）。

禁用方式：

```
代码：{  
代码： channels: { msteams: { configWrites: false } },  
代码： }
```

访问控制（私信 + 群组）

私信访问

- 默认：channels.msteams.dmPolicy = "pairing"。未知发送者在获得批准之前将被忽略。
- channels.msteams.allowFrom 接受 AAD 对象 ID、UPN 或显示名称。当凭证允许时，向导会通过 Microsoft Graph 将名称解析为 ID。

群组访问

- 默认：channels.msteams.groupPolicy = "allowlist"（除非添加 groupAllowFrom，否则被阻止）。使用 channels.defaults.groupPolicy 在未设置时覆盖默认值。
- channels.msteams.groupAllowFrom 控制哪些发送者可以在群聊/频道中触发（回退到 channels.msteams.allowFrom）。
- 设置 groupPolicy: "open" 允许任何成员（默认仍需提及才能触发）。
- 要不允许任何频道，设置 channels.msteams.groupPolicy: "disabled"。

示例：

```
代码：{
代码： channels: {
代码： msteams: {
代码： groupPolicy: "allowlist",
代码： groupAllowFrom: ["user@org.com"],
代码： },
代码： },
代码： }
```

团队 + 频道允许列表

- 通过在 channels.msteams.teams 下列出团队和频道来限定群组/频道回复的范围。
- 键可以是团队 ID 或名称；频道键可以是会话 ID 或名称。
- 当 groupPolicy="allowlist" 且存在团队允许列表时，仅接受列出的团队/频道（需要提及才能触发）。
- 配置向导接受 Team/Channel 条目并为你存储。
- 启动时，OpenClaw 将团队/频道和用户允许列表名称解析为 ID（当 Graph 权限允许时）并记录映射；未解析的条目保持原样。

示例：

```
代码 : {
代码 : channels: {
代码 : msteams: {
代码 : groupPolicy: "allowlist",
代码 : teams: {
代码 : "My Team": {
代码 : channels: {
代码 : General: { requireMention: true },
代码 : },
代码 : },
代码 : },
代码 : },
代码 : },
代码 : }
```

工作原理

- 安装 Microsoft Teams 插件。
- 创建一个 Azure Bot（App ID + 密钥 + 租户 ID）。
- 构建一个引用机器人并包含以下 RSC 权限的 Teams 应用包。
- 将 Teams 应用上传/安装到团队中（或用于私信的个人范围）。
- 在 ~/.openclaw/openclaw.json（或环境变量）中配置 msteams 并启动 Gateway 网关。
- Gateway 网关默认在 /api/messages 上监听 Bot Framework webhook 流量。

Azure Bot 设置（前提条件）

在配置 OpenClaw 之前，你需要创建一个 Azure Bot 资源。

步骤 1：创建 Azure Bot

- 前往创建 Azure Bot
- 填写基本信息选项卡：

字段	值
-----	-----

| Bot handle | 你的机器人名称，例如 openclaw-msteams（必须唯一） |

| Subscription | 选择你的 Azure 订阅 |

| Resource group | 新建或使用现有 |

| Pricing tier | Free 用于开发/测试 |

| Type of App | Single Tenant（推荐 - 见下方说明） |

| Creation type | Create new Microsoft App ID |

弃用通知：2025-07-31 之后已弃用创建新的多租户机器人。新机器人请使用 Single Tenant。

- 点击 Review + create → Create（等待约 1-2 分钟）

步骤 2：获取凭证

- 前往你的 Azure Bot 资源 → Configuration
- 复制 Microsoft App ID → 这是你的 appId
- 点击 Manage Password → 前往应用注册
- 在 Certificates & secrets → New client secret → 复制 Value → 这是你的 appPassword
- 前往 Overview → 复制 Directory (tenant) ID → 这是你的 tenantId

步骤 3：配置消息端点

- 在 Azure Bot → Configuration
- 将 Messaging endpoint 设置为你的 webhook URL：
- 生产环境：`
- 本地开发：使用隧道（见下方本地开发）

步骤 4：启用 Teams 渠道

- 在 Azure Bot → Channels
- 点击 Microsoft Teams → Configure → Save
- 接受服务条款

本地开发（隧道）

Teams 无法访问 localhost。本地开发请使用隧道：

选项 A：ngrok

代码：ngrok http 3978

代码：# 复制 https URL，例如 https://abc123.ngrok.io

代码：# 将消息端点设置为：https://abc123.ngrok.io/api/messages

选项 B：Tailscale Funnel

代码：tailscale funnel 3978

代码：# 使用你的 Tailscale funnel URL 作为消息端点

Teams 开发者门户（替代方案）

除了手动创建清单 ZIP，你可以使用 Teams 开发者门户：

- 点击 + New app
- 填写基本信息（名称、描述、开发者信息）
- 前往 App features → Bot
- 选择 Enter a bot ID manually 并粘贴你的 Azure Bot App ID
- 勾选范围：Personal、Team、Group Chat
- 点击 Distribute → Download app package
- 在 Teams 中：Apps → Manage your apps → Upload a custom app → 选择 ZIP

这通常比手动编辑 JSON 清单更容易。

测试机器人

选项 A：Azure Web Chat（先验证 webhook）

- 在 Azure 门户 → 你的 Azure Bot 资源 → Test in Web Chat
- 发送一条消息 - 你应该看到响应
- 这确认你的 webhook 端点在 Teams 设置之前正常工作

选项 B：Teams（应用安装后）

- 安装 Teams 应用（侧载或组织目录）
- 在 Teams 中找到机器人并发送私信
- 检查 Gateway 网关日志中的传入活动

设置（最小纯文本）

- 安装 Microsoft Teams 插件
- 从 npm : openclaw plugins install @openclaw/msteams
- 从本地检出 : openclaw plugins install ./extensions/msteams
- 机器人注册
- 创建一个 Azure Bot（见上文）并记录：
- App ID
- 客户端密钥（App password）
- 租户 ID（单租户）
- Teams 应用清单
- 包含一个 bot 条目，其中 botId = <App ID>。
- 范围：personal、team、groupChat。
- supportsFiles: true（个人范围文件处理所需）。
- 添加 RSC 权限（见下文）。
- 创建图标：outline.png（32x32）和 color.png（192x192）。
- 将三个文件一起打包：manifest.json、outline.png、color.png。
- 配置 OpenClaw

代码：{

代码： "msteams": {

代码： "enabled": true,

代码： "appld": "<APP_ID>",

代码： "appPassword": "<APP_PASSWORD>",

代码： "tenantId": "<TENANT_ID>",

代码： "webhook": { "port": 3978, "path": "/api/messages" }

代码： }

代码： }

你也可以使用环境变量代替配置键：

- MSTEAMS_APP_ID
- MSTEAMS_APP_PASSWORD
- MSTEAMS_TENANT_ID

- 机器人端点
- 将 Azure Bot Messaging Endpoint 设置为：
- 、
- 运行 Gateway 网关
- 当插件已安装且 msteams 配置存在并有凭证时，Teams 渠道会自动启动。

历史上下文

- channels.msteams.historyLimit 控制将多少条最近的频道/群组消息包含到提示中。
- 回退到 messages.groupChat.historyLimit。设置 0 禁用（默认 50）。
- 私信历史可以通过 channels.msteams.dmHistoryLimit（用户轮次）限制。每用户覆盖：channels.msteams.dms["<user_id>"].historyLimit。

当前 Teams RSC 权限（清单）

这些是我们 Teams 应用清单中现有的 resourceSpecific 权限。它们仅适用于安装了应用的团队/聊天内部。

对于频道（团队范围）：

- ChannelMessage.Read.Group (Application) - 无需 @提及即可接收所有频道消息
- ChannelMessage.Send.Group (Application)
- Member.Read.Group (Application)
- Owner.Read.Group (Application)
- ChannelSettings.Read.Group (Application)
- TeamMember.Read.Group (Application)
- TeamSettings.Read.Group (Application)

对于群聊：

- ChatMessage.Read.Chat (Application) - 无需 @提及即可接收所有群聊消息

Teams 清单示例（已脱敏）

包含必需字段的最小有效示例。请替换 ID 和 URL。

代码：{

代码 : "\$schema": "https://developer.microsoft.com/en-us/json-schemas/teams/v1.23/MicrosoftTeams.schema.json",

代码 : "manifestVersion": "1.23",

代码 : "version": "1.0.0",

代码 : "id": "00000000-0000-0000-0000-000000000000",

代码 : "name": { "short": "OpenClaw" },

代码 : "developer": {

代码 : "name": "Your Org",

代码 : "websiteUrl": "https://example.com",

代码 : "privacyUrl": "https://example.com/privacy",

代码 : "termsOfUseUrl": "https://example.com/terms"

代码 : },

代码 : "description": { "short": "OpenClaw in Teams", "full": "OpenClaw in Teams" },

代码 : "icons": { "outline": "outline.png", "color": "color.png" },

代码 : "accentColor": "#5B6DEF",

代码 : "bots": [

代码 : {

代码 : "botId": "11111111-1111-1111-1111-111111111111",

代码 : "scopes": ["personal", "team", "groupChat"],

代码 : "isNotificationOnly": false,

代码 : "supportsCalling": false,

代码 : "supportsVideo": false,

代码 : "supportsFiles": true

代码 : }

代码 :],

代码 : "webApplicationInfo": {

代码 : "id": "11111111-1111-1111-1111-111111111111"

代码 : },

代码 : "authorization": {

代码 : "permissions": {

代码 : "resourceSpecific": [

代码：{"name": "ChannelMessage.Read.Group", "type": "Application"},

代码：{"name": "ChannelMessage.Send.Group", "type": "Application"},

代码：{"name": "Member.Read.Group", "type": "Application"},

代码：{"name": "Owner.Read.Group", "type": "Application"},

代码：{"name": "ChannelSettings.Read.Group", "type": "Application"},

代码：{"name": "TeamMember.Read.Group", "type": "Application"},

代码：{"name": "TeamSettings.Read.Group", "type": "Application"},

代码：{"name": "ChatMessage.Read.Chat", "type": "Application" }

代码：]

代码：}

代码：}

代码：}

清单注意事项（必填字段）

- bots[].botId 必须与 Azure Bot App ID 匹配。
- webApplicationInfo.id 必须与 Azure Bot App ID 匹配。
- bots[].scopes 必须包含你计划使用的界面（personal、team、groupChat）。
- bots[].supportsFiles: true 是个人范围文件处理所需的。
- authorization.permissions.resourceSpecific

如果你需要频道流量，必须包含频道读取/发送权限。

更新现有应用

要更新已安装的 Teams 应用（例如，添加 RSC 权限）：

- 使用新设置更新你的 manifest.json
- 增加 version 字段（例如，1.0.0 → 1.1.0）
- 重新打包清单和图标（manifest.json、outline.png、color.png）
- 上传新的 zip：
- 选项 A（Teams 管理中心）：Teams 管理中心 → Teams apps → Manage apps → 找到你的应用 → Upload new version
- 选项 B（侧载）：在 Teams 中 → Apps → Manage your apps → Upload a custom app
- 对于团队频道：在每个团队中重新安装应用以使新权限生效

- 完全退出并重新启动 Teams（不仅仅是关闭窗口）以清除缓存的应用元数据

功能：仅 RSC 与 Graph

仅使用 ****Teams RSC****（应用已安装，无 Graph API 权限）

可用：

- 读取频道消息文本内容。
- 发送频道消息文本内容。
- 接收个人（私信）文件附件。

不可用：

- 频道/群组图片或文件内容（负载仅包含 HTML 存根）。
- 下载存储在 SharePoint/OneDrive 中的附件。
- 读取消息历史（超出实时 webhook 事件）。

使用 ****Teams RSC + Microsoft Graph Application 权限****

增加：

- 下载托管内容（粘贴到消息中的图片）。
- 下载存储在 SharePoint/OneDrive 中的文件附件。
- 通过 Graph 读取频道/聊天消息历史。

RSC 与 Graph API 对比

功能	RSC 权限	Graph API
实时消息	是（通过 webhook）	否（仅轮询）
历史消息	否	是（可查询历史）
设置复杂度	仅应用清单	需要管理员同意 + 令牌流程
离线工作	否（必须运行）	是（随时查询）

结论：RSC 用于实时监听；Graph API 用于历史访问。要在离线时补上错过的消息，你需要带有 ChannelMessage.Read.All 的 Graph API（需要管理员同意）。

启用 Graph 的媒体 + 历史（频道所需）

如果你需要频道中的图片/文件或想要获取消息历史，你必须启用 Microsoft Graph 权限并授予管理员同意。

- 在 Entra ID (Azure AD) App Registration 中，添加 Microsoft Graph Application 权限：
- ChannelMessage.Read.All (频道附件 + 历史)
- Chat.Read.All 或 ChatMessage.Read.All (群聊)
- 为租户授予管理员同意。
- 提升 Teams 应用清单版本，重新上传，并在 Teams 中重新安装应用。
- 完全退出并重新启动 Teams 以清除缓存的应用元数据。

已知限制

Webhook 超时

Teams 通过 HTTP webhook 传递消息。如果处理时间过长（例如，LLM 响应缓慢），你可能会看到：

- Gateway 网关超时
- Teams 重试消息（导致重复）
- 丢失的回复

OpenClaw 通过快速返回并主动发送回复来处理这个问题，但非常慢的响应仍可能导致问题。

格式化

Teams markdown 比 Slack 或 Discord 更有限：

- 基本格式化有效：粗体、斜体、代码、链接
- 复杂的 markdown（表格、嵌套列表）可能无法正确渲染
- 支持 Adaptive Cards 用于投票和任意卡片发送（见下文）

配置

关键设置（共享渠道模式见 /gateway/configuration）：

- channels.msteams.enabled：启用/禁用渠道。
- channels.msteams.appld、channels.msteams.appPassword、channels.msteams.tenantId：机器人凭证。

- channels.msteams.webhook.port (默认 3978)
- channels.msteams.webhook.path (默认 /api/messages)
- channels.msteams.dmPolicy : pairing | allowlist | open | disabled (默认 : pairing)
- channels.msteams.allowFrom : 私信允许列表 (AAD 对象 ID、UPN 或显示名称) 。当 Graph 访问可用时, 向导在设置期间将名称解析为 ID 。
- channels.msteams.textChunkLimit : 出站文本分块大小。
- channels.msteams.chunkMode : length (默认) 或 newline
在长度分块之前按空行 (段落边界) 分割。
- channels.msteams.mediaAllowHosts : 进站附件主机允许列表 (默认为 Microsoft/Teams 域名) 。
- channels.msteams.mediaAuthAllowHosts : 在媒体重试时附加 Authorization 头的允许列表 (默认为 Graph + Bot Framework 主机) 。
- channels.msteams.requireMention : 在频道/群组中需要 @提及 (默认 true) 。
- channels.msteams.replyStyle : thread | top-level (见回复样式) 。
- channels.msteams.teams.<teamId>.replyStyle : 每团队覆盖。
- channels.msteams.teams.<teamId>.requireMention : 每团队覆盖。
- channels.msteams.teams.<teamId>.tools : 当缺少频道覆盖时使用的默认每团队工具策略覆盖 (allow/deny/alsoAllow) 。
- channels.msteams.teams.<teamId>.toolsBySender : 默认每团队每发送者工具策略覆盖 (支持 "" 通配符) 。
- channels.msteams.teams.<teamId>.channels.<conversationId>.replyStyle : 每频道覆盖。
- channels.msteams.teams.<teamId>.channels.<conversationId>.requireMention : 每频道覆盖。
- channels.msteams.teams.<teamId>.channels.<conversationId>.tools : 每频道工具策略覆盖 (allow/deny/alsoAllow) 。
- channels.msteams.teams.<teamId>.channels.<conversationId>.toolsBySender : 每频道每发送者工具策略覆盖 (支持 "" 通配符) 。
- channels.msteams.sharePointSiteId : 用于群聊/频道文件上传的 SharePoint 站点 ID (见在群聊中发送文件) 。

路由和会话

- 会话键遵循标准智能体格式 (见 /concepts/session) :

- 私信共享主会话（agent:<agentId>:<mainKey>）。
- 频道/群组消息使用会话 ID：
- agent:<agentId>msteams:channel:<conversationId>
- agent:<agentId>msteams:group:<conversationId>

回复样式：话题 vs 帖子

Teams 最近在相同的底层数据模型上引入了两种频道 UI 样式：

样式	描述	推荐的 replyStyle
Posts（经典）	消息显示为卡片，下方有话题回复	thread（默认）
Threads（类 Slack）	消息线性流动，更像 Slack	top-level

问题：Teams API 不暴露频道使用的 UI 样式。如果你使用错误的 replyStyle：

- 在 Threads 样式频道中使用 thread → 回复嵌套显示很别扭
- 在 Posts 样式频道中使用 top-level → 回复显示为单独的顶级帖子而不是在话题中

解决方案：根据频道的设置方式为每个频道配置 replyStyle：

代码：{

代码："msteams":{

代码："replyStyle":"thread",

代码："teams":{

代码："19:abc...@thread.tacv2":{

代码："channels":{

代码："19:xyz...@thread.tacv2":{

代码："replyStyle":"top-level"

代码：}

代码：}

代码：}

代码：}

代码：}

代码：}

附件和图片

当前限制：

- 私信：图片和文件附件通过 Teams bot file API 工作。
- 频道/群组：附件存储在 M365 存储（SharePoint/OneDrive）中。webhook 负载仅包含 HTML 存根，而非实际文件字节。需要 Graph API 权限才能下载频道附件。

没有 Graph 权限，带图片的频道消息将作为纯文本接收（机器人无法访问图片内容）。

默认情况下，OpenClaw 仅从 Microsoft/Teams 主机名下载媒体。使用 channels.msteams.mediaAllowHosts 覆盖（使用 [""] 允许任何主机）。

Authorization 头仅附加到 channels.msteams.mediaAuthAllowHosts 中的主机（默认为 Graph + Bot Framework 主机）。保持此列表严格（避免多租户后缀）。

在群聊中发送文件

机器人可以使用 FileConsentCard

流程在私信中发送文件（内置）。但是，在群聊/频道中发送文件需要额外设置：

上下文	文件发送方式	所需设置
私信	FileConsentCard → 用户接受 → 机器人上传	开箱即用
群聊/频道	上传到 SharePoint → 共享链接	需要 sharePointSiteId + Graph 权限
图片（任何上下文）	Base64 编码内联	开箱即用

为什么群聊需要 SharePoint

机器人没有个人 OneDrive 驱动器（/me/drive Graph API 端点对应用程序身份不起作用）。要在群聊/频道中发送文件，机器人上传到 SharePoint 站点并创建共享链接。

设置

- 在 Entra ID（Azure AD）→ App Registration 中添加 Graph API 权限：
- Sites.ReadWrite.All（Application）- 上传文件到 SharePoint
- Chat.Read.All（Application）- 可选，启用每用户共享链接
- 为租户授予管理员同意。

- 获取你的 SharePoint 站点 ID :

代码 : # 通过 Graph Explorer 或带有效令牌的 curl :

代码 : curl -H "Authorization: Bearer \$TOKEN" \

代码 : "https://graph.microsoft.com/v1.0/sites/{hostname}/{site-path}"

代码 : # 示例 : 对于 "contoso.sharepoint.com/sites/BotFiles" 的站点

代码 : curl -H "Authorization: Bearer \$TOKEN" \

代码 : "https://graph.microsoft.com/v1.0/sites/contoso.sharepoint.com:/sites/BotFiles"

代码 : # 响应包含 : "id": "contoso.sharepoint.com,guid1,guid2"

- 配置 OpenClaw :

代码 : {

代码 : channels: {

代码 : msteams: {

代码 : // ... 其他配置 ...

代码 : sharePointSiteId: "contoso.sharepoint.com,guid1,guid2",

代码 : },

代码 : },

代码 : }

共享行为

| 权限 | 共享行为 |

| ----- | ----- |

| 仅 Sites.ReadWrite.All | 组织范围共享链接（组织中任何人都可以访问） |

| Sites.ReadWrite.All + Chat.Read.All | 每用户共享链接（仅聊天成员可以访问） |

每用户共享更安全，因为只有聊天参与者才能访问文件。如果缺少 Chat.Read.All 权限，机器人回退到组织范围共享。

回退行为

| 场景 | 结果 |

| ----- | ----- |

群聊 + 文件 + 已配置 sharePointSiteId	上传到 SharePoint，发送共享链接
群聊 + 文件 + 无 sharePointSiteId	尝试 OneDrive 上传（可能失败），仅发送文本
个人聊天 + 文件	FileConsentCard 流程（无需 SharePoint 即可工作）
任何上下文 + 图片	Base64 编码内联（无需 SharePoint 即可工作）

文件存储位置

上传的文件存储在配置的 SharePoint 站点默认文档库中的 /OpenClawShared/ 文件夹中。

投票 (Adaptive Cards)

OpenClaw 将 Teams 投票作为 Adaptive Cards 发送（没有原生 Teams 投票 API）。

- CLI : `openclaw message poll --channel msteams --target conversation:<id> ...`
- 投票由 Gateway 网关记录在 `~/.openclaw/msteams-polls.json` 中。
- Gateway 网关必须保持在线才能记录投票。
- 投票尚不自动发布结果摘要（如需要请检查存储文件）。

Adaptive Cards (任意)

使用 message 工具或 CLI 向 Teams 用户或会话发送任意 Adaptive Card JSON。

card 参数接受 Adaptive Card JSON 对象。当提供 card 时，消息文本是可选的。

智能体工具：

```
代码：{  
代码： "action": "send",  
代码： "channel": "msteams",  
代码： "target": "user:<id>",  
代码： "card": {  
代码： "type": "AdaptiveCard",  
代码： "version": "1.5",  
代码： "body": [{ "type": "TextBlock", "text": "Hello!" }]  
代码： }  
代码： }
```

CLI :

代码 : openclaw message send --channel msteams \

代码 : --target "conversation:19:abc...@thread.tacv2" \

代码 : --card

'{"type":"AdaptiveCard","version":"1.5","body":[{"type":"TextBlock","text":"Hello!"}]}'

参见 Adaptive Cards 文档了解卡片模式和示例。目标格式详情见下方目标格式。

目标格式

MSTeams 目标使用前缀来区分用户和会话：

| 目标类型 | 格式 | 示例 |

| ----- | ----- | ----- |

| 用户 (按 ID) | user:<aad-object-id> | user:40a1a0ed-4ff2-4164-a219-55518990c197 |

| 用户 (按名称) | user:<display-name> | user:John Smith (需要 Graph API) |

| 群组/频道 | conversation:<conversation-id> | conversation:19:abc123...@thread.tacv2 |

| 群组/频道 (原始) | <conversation-id> | 19:abc123...@thread.tacv2 (如果包含 @thread) |

CLI 示例：

代码 : # 按 ID 发送给用户

代码 : openclaw message send --channel msteams --target "user:40a1a0ed-..." --message "Hello"

代码 : # 按显示名称发送给用户 (触发 Graph API 查找)

代码 : openclaw message send --channel msteams --target "user:John Smith" --message "Hello"

代码 : # 发送到群聊或频道

代码 : openclaw message send --channel msteams --target "conversation:19:abc...@thread.tacv2" --message "Hello"

代码 : # 向会话发送 Adaptive Card

代码 : openclaw message send --channel msteams --target "conversation:19:abc...@thread.tacv2" \

代码：--card

```
{ "type": "AdaptiveCard", "version": "1.5", "body": [ { "type": "TextBlock", "text": "Hello" } ] }
```

智能体工具示例：

代码：{

代码： "action": "send",

代码： "channel": "msteams",

代码： "target": "user:John Smith",

代码： "message": "Hello!"

代码：}

代码：{

代码： "action": "send",

代码： "channel": "msteams",

代码： "target": "conversation:19:abc...@thread.tacv2",

代码： "card": {

代码： "type": "AdaptiveCard",

代码： "version": "1.5",

代码： "body": [{ "type": "TextBlock", "text": "Hello" }]

代码： }

代码：}

注意：没有 user: 前缀时，名称默认解析为群组/团队。按显示名称定位人员时始终使用 user:。

主动消息

- 主动消息仅在用户交互之后才可能，因为我们在那时存储会话引用。
- 有关 dmPolicy 和允许列表控制，请参见 </gateway/configuration>。

团队和频道 ID（常见陷阱）

Teams URL 中的 groupId 查询参数不是用于配置的团队 ID。请从 URL 路径中提取 ID：

团队 URL：

代码：https://teams.microsoft.com/l/team/19%3ABk4j...%40thread.tacv2/conversations?groupId=...

代码： _____

代码： 团队 ID（URL 解码此部分）

频道 URL：

代码：https://teams.microsoft.com/l/channel/19%3A15bc...%40thread.tacv2/ChannelName?groupId=...

代码： _____

代码： 频道 ID（URL 解码此部分）

用于配置：

- 团队 ID = /team/ 后的路径段（URL 解码，例如 19:Bk4j...@thread.tacv2）
- 频道 ID = /channel/ 后的路径段（URL 解码）
- 忽略 groupId 查询参数

私有频道

机器人在私有频道中的支持有限：

功能	标准频道	私有频道
-----	-----	-----
机器人安装	是	有限
实时消息（webhook）	是	可能不工作
RSC 权限	是	行为可能不同
@提及	是	如果机器人可访问
Graph API 历史	是	是（有权限）

如果私有频道不工作的变通方法：

- 使用标准频道进行机器人交互
- 使用私信 - 用户始终可以直接给机器人发消息
- 使用 Graph API 进行历史访问（需要 ChannelMessage.Read.All）

故障排除

常见问题

- 频道中图片不显示：缺少 Graph 权限或管理员同意。重新安装 Teams 应用并完全退出/重新打开 Teams。
- 频道中无响应：默认需要提及；设置 `channels.msteams.requireMention=false` 或按团队/频道配置。
- 版本不匹配（Teams 仍显示旧清单）：移除 + 重新添加应用并完全退出 Teams 以刷新。
- 来自 webhook 的 401 Unauthorized：在没有 Azure JWT 的情况下手动测试时属于预期情况 - 意味着端点可达但认证失败。使用 Azure Web Chat 正确测试。

清单上传错误

- "Icon file cannot be empty"：清单引用的图标文件为 0 字节。创建有效的 PNG 图标（outline.png 为 32x32，color.png 为 192x192）。
- "webApplicationInfo.Id already in use"：应用仍安装在另一个团队/聊天中。先找到并卸载它，或等待 5-10 分钟让其传播。
- 上传时"Something went wrong"：改为通过 上传，打开浏览器 DevTools (F12) → Network 选项卡，检查响应正文中的实际错误。
- 侧载失败：尝试"Upload an app to your org's app catalog"而不是"Upload a custom app" - 这通常可以绕过侧载限制。

RSC 权限不工作

- 验证 `webApplicationInfo.id` 与你的机器人 App ID 完全匹配
- 重新上传应用并在团队/聊天中重新安装
- 检查你的组织管理员是否阻止了 RSC 权限
- 确认你使用的是正确的范围：团队使用 `ChannelMessage.Read.Group`，群聊使用 `ChatMessage.Read.Chat`

参考资料

- 创建 Azure Bot - Azure Bot 设置指南
- Teams 开发者门户 - 创建/管理 Teams 应用
- Teams 应用清单模式
- 使用 RSC 接收频道消息
- RSC 权限参考
- Teams 机器人文件处理（频道/群组需要 Graph）

- 主动消息

17. Nextcloud Talk（插件）

Nextcloud Talk（插件）

状态：通过插件支持（webhook 机器人）。支持私信、房间、表情回应和 Markdown 消息。

需要插件

Nextcloud Talk 以插件形式提供，不包含在核心安装包中。

通过 CLI 安装（npm 仓库）：

代码：openclaw plugins install @openclaw/nextcloud-talk

本地检出安装（从 git 仓库运行时）：

代码：openclaw plugins install ./extensions/nextcloud-talk

如果你在配置/新手引导过程中选择了 Nextcloud Talk，并且检测到 git 检出，OpenClaw 将自动提供本地安装路径。

详情：插件

快速设置（新手）

- 安装 Nextcloud Talk 插件。
- 在你的 Nextcloud 服务器上创建机器人：
代码：./occ talk:bot:install "OpenClaw" "<shared-secret>" "<webhook-url>" --feature reaction
- 在目标房间设置中启用机器人。
- 配置 OpenClaw：
 - 配置项：channels.nextcloud-talk.baseUrl + channels.nextcloud-talk.botSecret
 - 或环境变量：NEXTCLOUD_TALK_BOT_SECRET（仅默认账户）
- 重启 Gateway 网关（或完成新手引导）。

最小配置：

代码：{

```
代码： channels: {  
代码： "nextcloud-talk": {  
代码： enabled: true,  
代码： baseUrl: "https://cloud.example.com",  
代码： botSecret: "shared-secret",  
代码： dmPolicy: "pairing",  
代码： },  
代码： },  
代码： }
```

注意事项

- 机器人无法主动发起私信。用户必须先向机器人发送消息。
- Webhook URL 必须可被 Gateway 网关访问；如果在代理后面，请设置 webhookPublicUrl。
- 机器人 API 不支持媒体上传；媒体以 URL 形式发送。
- Webhook 载荷无法区分私信和房间；设置 apiUser + apiPassword 以启用房间类型查询（否则私信将被视为房间）。

访问控制（私信）

- 默认：channels.nextcloud-talk.dmPolicy = "pairing"。未知发送者将收到配对码。
- 批准方式：
 - openclaw pairing list nextcloud-talk
 - openclaw pairing approve nextcloud-talk <CODE>
- 公开私信：channels.nextcloud-talk.dmPolicy="open" 加上 channels.nextcloud-talk.allowFrom=[""]。

房间（群组）

- 默认：channels.nextcloud-talk.groupPolicy = "allowlist"（需要提及触发）。
- 使用 channels.nextcloud-talk.rooms 设置房间白名单：

```
代码： {  
代码： channels: {  
代码： "nextcloud-talk": {
```

代码：rooms: {

代码： "room-token": { requireMention: true },

代码： },

代码： },

代码： },

代码： }

- 如需禁止所有房间，保持白名单为空或设置
channels.nextcloud-talk.groupPolicy="disabled"。

功能支持

| 功能 | 状态 |

| ----- | ----- |

| 私信 | 支持 |

| 房间 | 支持 |

| 话题 | 不支持 |

| 媒体 | 仅 URL |

| 表情回应 | 支持 |

| 原生命令 | 不支持 |

配置参考（Nextcloud Talk）

完整配置：配置

提供商选项：

- channels.nextcloud-talk.enabled：启用/禁用渠道启动。
- channels.nextcloud-talk.baseUrl：Nextcloud 实例 URL。
- channels.nextcloud-talk.botSecret：机器人共享密钥。
- channels.nextcloud-talk.botSecretFile：密钥文件路径。
- channels.nextcloud-talk.apiUser：用于房间查询的 API 用户（私信检测）。
- channels.nextcloud-talk.apiPassword：用于房间查询的 API/应用密码。
- channels.nextcloud-talk.apiPasswordFile：API 密码文件路径。
- channels.nextcloud-talk.webhookPort：webhook 监听端口（默认：8788）。

- channels.nextcloud-talk.webhookHost : webhook 主机 (默认 : 0.0.0.0) 。
- channels.nextcloud-talk.webhookPath : webhook 路径 (默认 : /nextcloud-talk-webhook) 。
- channels.nextcloud-talk.webhookPublicUrl : 外部可达的 webhook URL 。
- channels.nextcloud-talk.dmPolicy : pairing | allowlist | open | disabled 。
- channels.nextcloud-talk.allowFrom : 私信白名单 (用户 ID) 。
- channels.nextcloud-talk.groupPolicy : allowlist | open | disabled 。
- channels.nextcloud-talk.groupAllowFrom : 群组白名单 (用户 ID) 。
- channels.nextcloud-talk.rooms : 每个房间的设置和白名单 。
- channels.nextcloud-talk.historyLimit : 群组历史记录限制 (0 表示禁用) 。
- channels.nextcloud-talk.dmHistoryLimit : 私信历史记录限制 (0 表示禁用) 。
- channels.nextcloud-talk.dms : 每个私信的覆盖设置 (historyLimit) 。
- channels.nextcloud-talk.textChunkLimit : 出站文本分块大小 (字符数) 。
- channels.nextcloud-talk.chunkMode : length (默认) 或 newline , 在长度分块前按空行 (段落边界) 分割 。
- channels.nextcloud-talk.blockStreaming : 禁用此渠道的分块流式传输 。
- channels.nextcloud-talk.blockStreamingCoalesce : 分块流式传输合并调优 。
- channels.nextcloud-talk.mediaMaxMb : 入站媒体大小上限 (MB) 。

18. Nostr

Nostr

状态 : 可选插件 (默认禁用) 。

Nostr 是一个去中心化的社交网络协议。此渠道使 OpenClaw 能够通过 NIP-04 接收和回复加密私信 (DMs) 。

安装 (按需)

新手引导 (推荐)

- 新手引导向导 (openclaw onboard) 和 openclaw channels add 会列出可选的渠道插件 。
- 选择 Nostr 会提示你按需安装插件 。

安装默认值 :

- Dev 渠道 + git checkout 可用：使用本地插件路径。
- Stable/Beta：从 npm 下载。

你可以随时在提示中覆盖选择。

手动安装

代码：openclaw plugins install @openclaw/nostr

使用本地 checkout（开发工作流）：

代码：openclaw plugins install --link <path-to-openclaw>/extensions/nostr

安装或启用插件后重启 Gateway 网关。

快速设置

- 生成 Nostr 密钥对（如需要）：

代码：# 使用 nak

代码：nak key generate

- 添加到配置：

代码：{

代码： "channels": {

代码： "nostr": {

代码： "privateKey": "\${NOSTR_PRIVATE_KEY}"

代码： }

代码： }

代码： }

- 导出密钥：

- 重启 Gateway 网关。

配置参考

键	类型	默认值	描述
-----	-----	-----	-----

privateKey	string	必填	nsec 或十六进制格式的私钥
relays	string[]	['wss://relay.damus.io', 'wss://nos.lol']	中继 URL (WebSocket)
dmPolicy	string	pairing	私信访问策略
allowFrom	string[]	[]	允许的发送者公钥
enabled	boolean	true	启用/禁用渠道
name	string	-	显示名称
profile	object	-	NIP-01 个人资料元数据

个人资料元数据

个人资料数据作为 NIP-01 kind:0 事件发布。你可以从控制界面 (Channels -> Nostr -> Profile) 管理它，或直接在配置中设置。

示例：

```
代码：{  
  代码："channels": {  
    代码："nostr": {  
      代码："privateKey": "${NOSTR_PRIVATE_KEY}",  
      代码："profile": {  
        代码："name": "openclaw",  
        代码："displayName": "OpenClaw",  
        代码："about": "Personal assistant DM bot",  
        代码："picture": "https://example.com/avatar.png",  
        代码："banner": "https://example.com/banner.png",  
        代码："website": "https://example.com",  
        代码："nip05": "openclaw@example.com",  
        代码："lud16": "openclaw@example.com"  
      }  
    }  
  }  
}
```

注意事项：

- 个人资料 URL 必须使用 `
- 从中继导入会合并字段并保留本地覆盖。

访问控制

私信策略

- pairing（默认）：未知发送者会收到配对码。
- allowlist：只有 allowFrom 中的公钥可以发送私信。
- open：公开接收私信（需要 allowFrom: [""]）。
- disabled：忽略接收的私信。

允许列表示例

```
代码：{
代码： "channels": {
代码： "nostr": {
代码： "privateKey": "${NOSTR_PRIVATE_KEY}",
代码： "dmPolicy": "allowlist",
代码： "allowFrom": ["npub1abc...", "npub1xyz..."]
代码： }
代码： }
代码： }
```

密钥格式

接受的格式：

- 私钥：nsec... 或 64 字符十六进制
- 公钥（allowFrom）：npub... 或十六进制

中继

默认值：relay.damus.io 和 nos.lol。

```
代码：{
代码： "channels": {
```

代码： "nostr": {

代码： "privateKey": "\${NOSTR_PRIVATE_KEY}",

代码： "relays": ["wss://relay.damus.io", "wss://relay.primal.net", "wss://nostr.wine"]

代码： }

代码： }

代码： }

提示：

- 使用 2-3 个中继以实现冗余。
- 避免使用过多中继（延迟、重复）。
- 付费中继可以提高可靠性。
- 本地中继适合测试（ws://localhost:7777）。

协议支持

| NIP | 状态 | 描述 |

| ----- | ----- | ----- |

| NIP-01 | 已支持 | 基本事件格式 + 个人资料元数据 |

| NIP-04 | 已支持 | 加密私信 (kind:4) |

| NIP-17 | 计划中 | 礼物包装私信 |

| NIP-44 | 计划中 | 版本化加密 |

测试

本地中继

代码： # 启动 strfry

代码： docker run -p 7777:7777 ghcr.io/hoytech/strfry

代码： {

代码： "channels": {

代码： "nostr": {

代码： "privateKey": "\${NOSTR_PRIVATE_KEY}",

代码： "relays": ["ws://localhost:7777"]

代码：}

代码：}

代码：}

手动测试

- 从日志中记下机器人公钥（npub）。
- 打开 Nostr 客户端（Damus、Amethyst 等）。
- 向机器人公钥发送私信。
- 验证响应。

故障排除

未收到消息

- 验证私钥是否有效。
- 确保中继 URL 可访问并使用 wss://（本地使用 ws://）。
- 确认 enabled 不是 false。
- 检查 Gateway 网关日志中的中继连接错误。

未发送响应

- 检查中继是否接受写入。
- 验证出站连接。
- 注意中继速率限制。

重复响应

- 使用多个中继时属于正常现象。
- 消息按事件 ID 去重；只有首次投递会触发响应。

安全

- 切勿提交私钥。
- 使用环境变量存储密钥。
- 生产环境机器人考虑使用 allowlist。

限制（MVP）

- 仅支持私信（不支持群聊）。
- 不支持媒体附件。
- 仅支持 NIP-04（计划支持 NIP-17 礼物包装）。

19. 配对

配对

"配对"是 OpenClaw 的显式所有者批准步骤。它用于两个地方：

- 私信配对（谁被允许与机器人对话）
- 节点配对（哪些设备/节点被允许加入 Gateway 网关网络）

安全上下文：安全

1) 私信配对（进站聊天访问）

当渠道配置为私信策略 pairing

时，未知发送者会收到一个短代码，他们的消息不会被处理，直到你批准。

默认私信策略记录在：安全

配对代码：

- 8 个字符，大写，无歧义字符（0011）。
- 1 小时后过期。机器人仅在创建新请求时发送配对消息（大约每个发送者每小时一次）。
- 待处理的私信配对请求默认上限为每个渠道 3 个；在一个过期或被批准之前，额外的请求将被忽略。

批准发送者

代码：openclaw pairing list telegram

代码：openclaw pairing approve telegram <CODE>

支持的渠道：telegram、whatsapp、signal、imessage、discord、slack。

状态存储位置

存储在 ~/.openclaw/credentials/ 下：

- 待处理请求：<channel>-pairing.json

- 已批准允许列表存储：`<channel>-allowFrom.json`

将这些视为敏感信息（它们控制对你助手的访问）。

2) 节点设备配对 (iOS/Android/macOS/无头节点)

节点作为 `role: node` 的设备连接到 Gateway 网关。Gateway 网关创建一个必须被批准的设备配对请求。

批准节点设备

代码：`openclaw devices list`

代码：`openclaw devices approve <requestId>`

代码：`openclaw devices reject <requestId>`

状态存储位置

存储在 `~/.openclaw/devices/` 下：

- `pending.json`（短期；待处理请求会过期）
- `paired.json`（已配对设备 + 令牌）

说明

- 旧版 `node.pair` API (CLI：`openclaw nodes pending/approve`) 是一个单独的 Gateway 网关拥有的配对存储。WS 节点仍然需要设备配对。

相关文档

- 安全模型 + 提示注入：安全
- 安全更新（运行 `doctor`）：更新
- 渠道配置：
- Telegram：Telegram
- WhatsApp：WhatsApp
- Signal：Signal
- iMessage：iMessage
- Discord：Discord
- Slack：Slack

20. Signal (signal-cli)

Signal (signal-cli)

状态：外部 CLI 集成。Gateway 网关通过 HTTP JSON-RPC + SSE 与 signal-cli 通信。

快速设置（初学者）

- 为 bot 使用单独的 Signal 号码（推荐）。
- 安装 signal-cli（需要 Java）。
- 链接 bot 设备并启动守护进程：
- `signal-cli link -n "OpenClaw"`
- 配置 OpenClaw 并启动 Gateway 网关。

最小配置：

代码：{

代码： channels: {

代码： signal: {

代码： enabled: true,

代码： account: "+15551234567",

代码： cliPath: "signal-cli",

代码： dmPolicy: "pairing",

代码： allowFrom: ["+15557654321"],

代码： },

代码： },

代码： }

它是什么

- 通过 signal-cli 的 Signal 渠道（非嵌入式 libsignal）。
- 确定性路由：回复始终返回到 Signal。
- 私信共享智能体的主会话；群组是隔离的（agent:<agentId>:signal:group:<groupId>）。

配置写入

默认情况下，Signal 允许写入由 /config set|unset 触发的配置更新（需要 commands.config: true）。

禁用方式：

代码：{

代码： channels: { signal: { configWrites: false } },

代码：}

号码模型（重要）

- Gateway 网关连接到一个 Signal 设备（signal-cli 账户）。
- 如果你在个人 Signal 账户上运行 bot，它会忽略你自己的消息（循环保护）。
- 要实现"我发消息给 bot 然后它回复"，请使用单独的 bot 号码。

设置（快速路径）

- 安装 signal-cli（需要 Java）。
- 链接 bot 账户：
- signal-cli link -n "OpenClaw" 然后在 Signal 中扫描二维码。
- 配置 Signal 并启动 Gateway 网关。

示例：

代码：{

代码： channels: {

代码： signal: {

代码： enabled: true,

代码： account: "+15551234567",

代码： cliPath: "signal-cli",

代码： dmPolicy: "pairing",

代码： allowFrom: ["+15557654321"],

代码： },

代码： },

代码：}

多账户支持：使用 `channels.signal.accounts` 配置每个账户及可选的 `name`。共享模式请参见 `gateway/configuration`。

外部守护进程模式（`httpUrl`）

如果你想自己管理 `signal-cli`（JVM 冷启动慢、容器初始化或共享 CPU），请单独运行守护进程并将 `OpenClaw` 指向它：

```
代码：{  
代码： channels: {  
代码： signal: {  
代码： httpUrl: "http://127.0.0.1:8080",  
代码： autoStart: false,  
代码： },  
代码： },  
代码： }
```

这会跳过自动启动和 `OpenClaw` 内部的启动等待。对于自动启动时的慢启动，请设置 `channels.signal.startupTimeoutMs`。

访问控制（私信 + 群组）

私信：

- 默认：`channels.signal.dmPolicy = "pairing"`。
- 未知发送者会收到配对码；消息在批准前会被忽略（配对码 1 小时后过期）。
- 通过以下方式批准：
 - `openclaw pairing list signal`
 - `openclaw pairing approve signal <CODE>`
- 配对是 Signal 私信的默认令牌交换方式。详情：配对
- 仅有 UUID 的发送者（来自 `sourceUuid`）在 `channels.signal.allowFrom` 中存储为 `uuid:<id>`。

群组：

- `channels.signal.groupPolicy = open | allowlist | disabled`。
- 当设置为 `allowlist` 时，`channels.signal.groupAllowFrom` 控制谁可以在群组中触发。

工作原理（行为）

- signal-cli 作为守护进程运行；Gateway 网关通过 SSE 读取事件。
- 入站消息被规范化为共享渠道信封。
- 回复始终路由回同一号码或群组。

媒体 + 限制

- 出站文本按 `channels.signal.textChunkLimit` 分块（默认 4000）。
- 可选换行分块：设置 `channels.signal.chunkMode="newline"` 在长度分块前按空行（段落边界）分割。
- 支持附件（从 signal-cli 获取 base64）。
- 默认媒体上限：`channels.signal.mediaMaxMb`（默认 8）。
- 使用 `channels.signal.ignoreAttachments` 跳过下载媒体。
- 群组历史上下文使用 `channels.signal.historyLimit`（或 `channels.signal.accounts..historyLimit`），回退到 `messages.groupChat.historyLimit`。设置 0 禁用（默认 50）。

输入指示器 + 已读回执

- 输入指示器：OpenClaw 通过 signal-cli `sendTyping` 发送输入信号，并在回复运行时刷新它们。
- 已读回执：当 `channels.signal.sendReadReceipts` 为 `true` 时，OpenClaw 为允许的私信转发已读回执。
- Signal-cli 不暴露群组的已读回执。

表情回应（message 工具）

- 使用 `message action=react` 配合 `channel=signal`。
- 目标：发送者 E.164 或 UUID（使用配对输出中的 `uuid:<id>`；裸 UUID 也可以）。
- `messageId` 是你要回应的消息的 Signal 时间戳。
- 群组表情回应需要 `targetAuthor` 或 `targetAuthorUuid`。

示例：

代码：`message action=react channel=signal
target=uuid:123e4567-e89b-12d3-a456-426614174000 messageId=1737630212345
emoji=`

代码：message action=react channel=signal target=+15551234567

messageId=1737630212345 emoji= remove=true

代码：message action=react channel=signal target=signal:group:<groupId>

targetAuthor=uuid:<sender-uuid> messageId=1737630212345 emoji=

配置：

- channels.signal.actions.reactions：启用/禁用表情回应操作（默认 true）。
- channels.signal.reactionLevel：off | ack | minimal | extensive。
- off/ack 禁用智能体表情回应（message 工具 react 会报错）。
- minimal/extensive 启用智能体表情回应并设置指导级别。
- 每账户覆盖：channels.signal.accounts.<id>.actions.reactions、channels.signal.accounts.<id>.reactionLevel。

投递目标（CLI/cron）

- 私信：signal:+15551234567（或纯 E.164）。
- UUID 私信：uuid:<id>（或裸 UUID）。
- 群组：signal:group:<groupId>。
- 用户名：username:<name>（如果你的 Signal 账户支持）。

配置参考（Signal）

完整配置：配置

提供商选项：

- channels.signal.enabled：启用/禁用渠道启动。
- channels.signal.account：bot 账户的 E.164。
- channels.signal.cliPath：signal-cli 的路径。
- channels.signal.httpUrl：完整守护进程 URL（覆盖 host/port）。
- channels.signal.httpHost、channels.signal.httpPort：守护进程绑定（默认 127.0.0.1:8080）。
- channels.signal.autoStart：自动启动守护进程（如果未设置 httpUrl 则默认 true）。
- channels.signal.startupTimeoutMs：启动等待超时（毫秒）（上限 120000）。
- channels.signal.receiveMode：on-start | manual。

- `channels.signal.ignoreAttachments` : 跳过附件下载。
- `channels.signal.ignoreStories` : 忽略来自守护进程的动态。
- `channels.signal.sendReadReceipts` : 转发已读回执。
- `channels.signal.dmPolicy` : `pairing` | `allowlist` | `open` | `disabled` (默认 : `pairing`) 。
- `channels.signal.allowFrom` : 私信允许列表 (E.164 或 `uuid:<id>`) 。 `open` 需要 "" 。 Signal 没有用户名 ; 使用电话/UUID id 。
- `channels.signal.groupPolicy` : `open` | `allowlist` | `disabled` (默认 : `allowlist`) 。
- `channels.signal.groupAllowFrom` : 群组发送者允许列表。
- `channels.signal.historyLimit` : 作为上下文包含的最大群组消息数 (0 禁用) 。
- `channels.signal.dmHistoryLimit` : 私信历史限制 (用户轮次) 。 每用户覆盖 : `channels.signal.dms["<phone_or_uuid>"].historyLimit` 。
- `channels.signal.textChunkLimit` : 出站分块大小 (字符) 。
- `channels.signal.chunkMode` : `length` (默认) 或 `newline` 在长度分块前按空行 (段落边界) 分割。
- `channels.signal.mediaMaxMb` : 入站/出站媒体上限 (MB) 。

相关全局选项 :

- `agents.list[].groupChat.mentionPatterns` (Signal 不支持原生提及) 。
- `messages.groupChat.mentionPatterns` (全局回退) 。
- `messages.responsePrefix` 。

21. Slack

Slack

Socket 模式 (默认)

快速设置 (新手)

- 创建一个 Slack 应用并启用 Socket Mode 。
- 创建一个 App Token (`xapp-...`) 和 Bot Token (`xoxb-...`) 。
- 为 OpenClaw 设置令牌并启动 Gateway 网关。

最小配置 :

代码 : {

代码： channels: {

代码： slack: {

代码： enabled: true,

代码： appToken: "xapp-...",

代码： botToken: "xoxb-...",

代码： },

代码： },

代码： }

设置

- 在 创建一个 Slack 应用（从头开始）。
- Socket Mode → 开启。然后前往 Basic Information → App-Level Tokens → Generate Token and Scopes，添加 connections:write 权限范围。复制 App Token（xapp-...）。
- OAuth & Permissions → 添加 bot token 权限范围（使用下面的 manifest）。点击 Install to Workspace。复制 Bot User OAuth Token（xoxb-...）。
- 可选：OAuth & Permissions → 添加 User Token Scopes（参见下面的只读列表）。重新安装应用并复制 User OAuth Token（xoxp-...）。
- Event Subscriptions → 启用事件并订阅：
 - message.（包括编辑/删除/线程广播）
 - app_mention
 - reaction_added、reaction_removed
 - member_joined_channel、member_left_channel
 - channel_rename
 - pin_added、pin_removed
- 邀请机器人加入你希望它读取的频道。
- Slash Commands → 如果你使用 channels.slack.slashCommand，创建 /openclaw。如果启用原生命令，为每个内置命令添加一个斜杠命令（名称与 /help 相同）。除非你设置 channels.slack.commands.native: true，否则 Slack 默认关闭原生命令（全局 commands.native 是 "auto"，对 Slack 保持关闭）。
- App Home → 启用 Messages Tab 以便用户可以私信机器人。

使用下面的 manifest 以保持权限范围和事件同步。

多账户支持：使用 `channels.slack.accounts` 配置每个账户的令牌和可选的 `name`。参见 `gateway/configuration` 了解共享模式。

OpenClaw 配置（最小）

通过环境变量设置令牌（推荐）：

- `SLACK_APP_TOKEN=xapp-...`
- `SLACK_BOT_TOKEN=xoxb-...`

或通过配置：

代码：{

代码： channels: {

代码： slack: {

代码： enabled: true,

代码： appToken: "xapp-...",

代码： botToken: "xoxb-...",

代码： },

代码： },

代码： }

用户令牌（可选）

OpenClaw 可以使用 Slack 用户令牌（`xoxp-...`）进行读取操作（历史记录、置顶、表情回应、表情符号、成员信息）。默认情况下保持只读：当存在用户令牌时，读取优先使用用户令牌，而写入仍然使用 bot 令牌，除非你明确选择加入。即使设置了 `userTokenReadOnly: false`，当 bot 令牌可用时，写入仍然优先使用 bot 令牌。

用户令牌在配置文件中配置（不支持环境变量）。对于多账户，设置 `channels.slack.accounts.<id>.userToken`。

包含 bot + app + 用户令牌的示例：

代码：{

代码： channels: {

代码： slack: {

代码： enabled: true,

代码： appToken: "xapp-...",

代码： botToken: "xoxb-...",

代码： userToken: "xoxp-...",

代码： },

代码： },

代码： }

明确设置 userTokenReadOnly 的示例（允许用户令牌写入）：

代码： {

代码： channels: {

代码： slack: {

代码： enabled: true,

代码： appToken: "xapp-...",

代码： botToken: "xoxb-...",

代码： userToken: "xoxp-...",

代码： userTokenReadOnly: false,

代码： },

代码： },

代码： }

令牌使用

- 读取操作（历史记录、表情回应列表、置顶列表、表情符号列表、成员信息、搜索）在配置了用户令牌时优先使用用户令牌，否则使用 bot 令牌。
- 写入操作（发送/编辑/删除消息、添加/移除表情回应、置顶/取消置顶、文件上传）默认使用 bot 令牌。如果 userTokenReadOnly: false 且没有可用的 bot 令牌，OpenClaw 会回退到用户令牌。

历史上下文

- channels.slack.historyLimit（或 channels.slack.accounts..historyLimit）控制将多少条最近的频道/群组消息包含到提示中。
- 回退到 messages.groupChat.historyLimit。设置为 0 以禁用（默认 50）。

HTTP 模式（Events API）

当你的 Gateway 网关可以通过 HTTPS 被 Slack 访问时（服务器部署的典型情况），使用 HTTP webhook 模式。

HTTP 模式使用 Events API + Interactivity + Slash Commands，共享一个请求 URL。

设置

- 创建一个 Slack 应用并禁用 Socket Mode（如果你只使用 HTTP 则可选）。
- Basic Information → 复制 Signing Secret。
- OAuth & Permissions → 安装应用并复制 Bot User OAuth Token (xoxb-...)。
- Event Subscriptions → 启用事件并将 Request URL 设置为你的 Gateway 网关 webhook 路径（默认 /slack/events）。
- Interactivity & Shortcuts → 启用并设置相同的 Request URL。
- Slash Commands → 为你的命令设置相同的 Request URL。

示例请求 URL：

、

OpenClaw 配置（最小）

代码：{

代码： channels: {

代码： slack: {

代码： enabled: true,

代码： mode: "http",

代码： botToken: "xoxb-...",

代码： signingSecret: "your-signing-secret",

代码： webhookPath: "/slack/events",

代码： },

代码： },

代码： }

多账户 HTTP 模式：设置 channels.slack.accounts.<id>.mode = "http"

并为每个账户提供唯一的 webhookPath，以便每个 Slack 应用可以指向自己的 URL。

Manifest（可选）

使用此 Slack 应用 manifest 快速创建应用（如果需要可以调整名称/命令）。如果你计划配置用户令牌，请包含用户权限范围。

代码：{

代码： "display_information": {

代码： "name": "OpenClaw",

代码： "description": "Slack connector for OpenClaw"

代码： },

代码： "features": {

代码： "bot_user": {

代码： "display_name": "OpenClaw",

代码： "always_online": false

代码： },

代码： "app_home": {

代码： "messages_tab_enabled": true,

代码： "messages_tab_read_only_enabled": false

代码： },

代码： "slash_commands": [

代码： {

代码： "command": "/openclaw",

代码： "description": "Send a message to OpenClaw",

代码： "should_escape": false

代码： }

代码：]

代码： },

代码： "oauth_config": {

代码： "scopes": {

代码： "bot": [

代码： "chat:write",

代码： "channels:history",

代码： "channels:read",

代码 : "groups:history",
代码 : "groups:read",
代码 : "groups:write",
代码 : "im:history",
代码 : "im:read",
代码 : "im:write",
代码 : "mpim:history",
代码 : "mpim:read",
代码 : "mpim:write",
代码 : "users:read",
代码 : "app_mentions:read",
代码 : "reactions:read",
代码 : "reactions:write",
代码 : "pins:read",
代码 : "pins:write",
代码 : "emoji:read",
代码 : "commands",
代码 : "files:read",
代码 : "files:write"
代码 :],
代码 : "user": [
代码 : "channels:history",
代码 : "channels:read",
代码 : "groups:history",
代码 : "groups:read",
代码 : "im:history",
代码 : "im:read",
代码 : "mpim:history",
代码 : "mpim:read",
代码 : "users:read",

```
代码 : "reactions:read",
代码 : "pins:read",
代码 : "emoji:read",
代码 : "search:read"
代码 : ]
代码 : }
代码 : },
代码 : "settings": {
代码 : "socket_mode_enabled": true,
代码 : "event_subscriptions": {
代码 : "bot_events": [
代码 : "app_mention",
代码 : "message.channels",
代码 : "message.groups",
代码 : "message.im",
代码 : "message.mpim",
代码 : "reaction_added",
代码 : "reaction_removed",
代码 : "member_joined_channel",
代码 : "member_left_channel",
代码 : "channel_rename",
代码 : "pin_added",
代码 : "pin_removed"
代码 : ]
代码 : }
代码 : }
代码 : }
```

如果启用原生命令，为每个要公开的命令添加一个 slash_commands 条目（与 /help 列表匹配）。使用 channels.slack.commands.native 覆盖。

权限范围（当前 vs 可选）

Slack 的 Conversations API 是按类型区分的：你只需要你实际接触的会话类型（channels、groups、im、mpim）的权限范围。概述参见

Bot 令牌权限范围（必需）

- chat:write（通过 chat.postMessage 发送/更新/删除消息）
- im:write（通过 conversations.open 打开私信用于用户私信）
- channels:history、groups:history、im:history、mpim:history
- channels:read、groups:read、im:read、mpim:read
- users:read（用户查询）
- reactions:read、reactions:write（reactions.get / reactions.add）
- pins:read、pins:write（pins.list / pins.add / pins.remove）
- emoji:read（emoji.list）
- files:write（通过 files.uploadV2 上传）

用户令牌权限范围（可选，默认只读）

如果你配置了 channels.slack.userToken，在 User Token Scopes 下添加这些。

- channels:history、groups:history、im:history、mpim:history
- channels:read、groups:read、im:read、mpim:read
- users:read
- reactions:read
- pins:read
- emoji:read
- search:read

目前不需要（但未来可能需要）

- mpim:write（仅当我们添加群组私信打开/私信启动时通过 conversations.open）
- groups:write（仅当我们添加私有频道管理时：创建/重命名/邀请/归档）
- chat:write.public（仅当我们想发布到机器人未加入的频道时）
- users:read.email（仅当我们需要从 users.info 获取邮箱字段时）
- files:read（仅当我们开始列出/读取文件元数据时）

配置

Slack 仅使用 Socket Mode（无 HTTP webhook 服务器）。提供两个令牌：

代码：{

代码： "slack": {

代码： "enabled": true,

代码： "botToken": "xoxb-...",

代码： "appToken": "xapp-...",

代码： "groupPolicy": "allowlist",

代码： "dm": {

代码： "enabled": true,

代码： "policy": "pairing",

代码： "allowFrom": ["U123", "U456", "*"],

代码： "groupEnabled": false,

代码： "groupChannels": ["G123"],

代码： "replyToMode": "all"

代码： },

代码： "channels": {

代码： "C123": { "allow": true, "requireMention": true },

代码： "#general": {

代码： "allow": true,

代码： "requireMention": true,

代码： "users": ["U123"],

代码： "skills": ["search", "docs"],

代码： "systemPrompt": "Keep answers short."

代码： }

代码： },

代码： "reactionNotifications": "own",

代码： "reactionAllowlist": ["U123"],

代码： "replyToMode": "off",

代码： "actions": {

代码： "reactions": true,
代码： "messages": true,
代码： "pins": true,
代码： "memberInfo": true,
代码： "emojiList": true
代码： },
代码： "slashCommand": {
代码： "enabled": true,
代码： "name": "openclaw",
代码： "sessionPrefix": "slack:slash",
代码： "ephemeral": true
代码： },
代码： "textChunkLimit": 4000,
代码： "mediaMaxMb": 20
代码： }
代码： }

令牌也可以通过环境变量提供：

- SLACK_BOT_TOKEN
- SLACK_APP_TOKEN

确认表情回应通过 `messages.ackReaction + messages.ackReactionScope` 全局控制。使用 `messages.removeAckAfterReply` 在机器人回复后清除确认表情回应。

限制

- 出站文本按 `channels.slack.textChunkLimit` 分块（默认 4000）。
- 可选的换行分块：设置 `channels.slack.chunkMode="newline"` 以在长度分块之前按空行（段落边界）分割。
- 媒体上传受 `channels.slack.mediaMaxMb` 限制（默认 20）。

回复线程

默认情况下，OpenClaw 在主频道回复。使用 `channels.slack.replyToMode` 控制自动线程：

| 模式 | 行为 |

| ----- | ----- |

| off | 默认。在主频道回复。仅当触发消息已在线程中时才使用线程。 |

| first | 第一条回复进入线程（在触发消息下），后续回复进入主频道。适合保持上下文可见同时避免线程混乱。 |

| all | 所有回复都进入线程。保持对话集中但可能降低可见性。 |

该模式适用于自动回复和智能体工具调用（slack sendMessage）。

按聊天类型的线程

你可以通过设置 channels.slack.replyToModeByChatType 为每种聊天类型配置不同的线程行为：

代码：{

代码： channels: {

代码： slack: {

代码： replyToMode: "off", // 频道的默认值

代码： replyToModeByChatType: {

代码： direct: "all", // 私信始终使用线程

代码： group: "first", // 群组私信/MPIM 第一条回复使用线程

代码： },

代码： },

代码： },

代码： }

支持的聊天类型：

- direct：一对一私信（Slack im）
- group：群组私信 / MPIM（Slack mpim）
- channel：标准频道（公开/私有）

优先级：

- replyToModeByChatType.<chatType>
- replyToMode

- 提供商默认值 (off)

当未设置聊天类型覆盖时，旧版 `channels.slack.dm.replyToMode` 仍可作为 `direct` 的回退。

示例：

仅对私信使用线程：

```
代码：{  
代码： channels: {  
代码： slack: {  
代码： replyToMode: "off",  
代码： replyToModeByChatType: { direct: "all" },  
代码： },  
代码： },  
代码： }
```

对群组私信使用线程但保持频道在根级别：

```
代码：{  
代码： channels: {  
代码： slack: {  
代码： replyToMode: "off",  
代码： replyToModeByChatType: { group: "first" },  
代码： },  
代码： },  
代码： }
```

让频道使用线程，保持私信在根级别：

```
代码：{  
代码： channels: {  
代码： slack: {  
代码： replyToMode: "first",  
代码： replyToModeByChatType: { direct: "off", group: "off" },  
代码： },  
代码： }
```


代码：},

代码：}

手动线程标签

对于细粒度控制，在智能体响应中使用这些标签：

- `[[reply_to_current]]` — 回复触发消息（开始/继续线程）。
- `[[reply_to:<id>]]` — 回复特定的消息 id。

会话 + 路由

- 私信共享 main 会话（与 WhatsApp/Telegram 相同）。
- 频道映射到 `agent:<agentId>:slack:channel:<channelId>` 会话。
- 斜杠命令使用 `agent:<agentId>:slack:slash:<userId>` 会话（前缀可通过 `channels.slack.slashCommand.sessionPrefix` 配置）。
- 如果 Slack 未提供 `channel_type`，OpenClaw 会从频道 ID 前缀（D、C、G）推断并默认为 `channel` 以保持会话键稳定。
- 原生命令注册使用 `commands.native`（全局默认 "auto" → Slack 关闭），可以使用 `channels.slack.commands.native` 按工作空间覆盖。文本命令需要独立的 /... 消息，可以使用 `commands.text: false` 禁用。Slack 斜杠命令在 Slack 应用中管理，不会自动移除。使用 `commands.useAccessGroups: false` 绕过命令的访问组检查。
- 完整命令列表 + 配置：斜杠命令

私信安全（配对）

- 默认：`channels.slack.dm.policy="pairing"` — 未知的私信发送者会收到配对码（1 小时后过期）。
- 通过以下方式批准：`openclaw pairing approve slack <code>`。
- 要允许任何人：设置 `channels.slack.dm.policy="open"` 和 `channels.slack.dm.allowFrom=[""]`。
- `channels.slack.dm.allowFrom` 接受用户 ID、@用户名或邮箱（在令牌允许时启动时解析）。向导在设置期间接受用户名，并在令牌允许时将其解析为 ID。

群组策略

- `channels.slack.groupPolicy` 控制频道处理（`open|disabled|allowlist`）。
- `allowlist` 要求频道列在 `channels.slack.channels` 中。

- 如果你只设置了 SLACK_BOT_TOKEN/SLACK_APP_TOKEN 而从未创建 channels.slack 部分，运行时默认将 groupPolicy 设为 open。添加 channels.slack.groupPolicy、channels.defaults.groupPolicy 或频道白名单来锁定它。
- 配置向导接受 #channel 名称，并在可能时（公开 + 私有）将其解析为 ID；如果存在多个匹配，它优先选择活跃的频道。
- 启动时，OpenClaw 将白名单中的频道/用户名解析为 ID（在令牌允许时）并记录映射；未解析的条目按原样保留。
- 要不允许任何频道，设置 channels.slack.groupPolicy: "disabled"（或保留空白名单）。

频道选项（channels.slack.channels.<id> 或 channels.slack.channels.<name>）：

- allow：当 groupPolicy="allowlist" 时允许/拒绝频道。
- requireMention：频道的提及门控。
- tools：可选的每频道工具策略覆盖（allow/deny/alsoAllow）。
- toolsBySender：频道内可选的每发送者工具策略覆盖（键为发送者 id/@用户名/邮箱；支持 "" 通配符）。
- allowBots：允许此频道中机器人发送的消息（默认：false）。
- users：可选的每频道用户白名单。
- skills：Skills 过滤器（省略 = 所有 Skills，空 = 无）。
- systemPrompt：频道的额外系统提示（与主题/目的组合）。
- enabled：设置为 false 以禁用频道。

投递目标

与 cron/CLI 发送一起使用：

- user:<id> 用于私信
- channel:<id> 用于频道

工具操作

Slack 工具操作可以通过 channels.slack.actions. 进行门控：

操作组	默认	说明
reactions	已启用	表情回应 + 列出表情回应
messages	已启用	读取/发送/编辑/删除

| pins | 已启用 | 置顶/取消置顶/列表 |

| memberInfo | 已启用 | 成员信息 |

| emojiList | 已启用 | 自定义表情符号列表 |

安全说明

- 写入默认使用 bot 令牌，因此状态更改操作保持在应用的机器人权限和身份范围内。
- 设置 `userTokenReadOnly: false` 允许在 bot 令牌不可用时使用用户令牌进行写入操作，这意味着操作以安装用户的访问权限运行。将用户令牌视为高权限，并保持操作门控和白名单严格。
- 如果你启用用户令牌写入，请确保用户令牌包含你期望的写入权限范围（`chat:write`、`reaction:write`、`pins:write`、`files:write`），否则这些操作将失败。

说明

- 提及门控通过 `channels.slack.channels` 控制（将 `requireMention` 设置为 `true`）；`agents.list[].groupChat.mentionPatterns`（或 `messages.groupChat.mentionPatterns`）也算作提及。
- 多智能体覆盖：在 `agents.list[].groupChat.mentionPatterns` 上设置每智能体的模式。
- 表情回应通知遵循 `channels.slack.reactionNotifications`（在 `allowlist` 模式下使用 `reactionAllowlist`）。
- 默认忽略机器人发送的消息；通过 `channels.slack.allowBots` 或 `channels.slack.channels.<id>.allowBots` 启用。
- 警告：如果你允许回复其他机器人（`channels.slack.allowBots=true` 或 `channels.slack.channels.<id>.allowBots=true`），请使用 `requireMention`、`channels.slack.channels.<id>.users` 白名单和/或在 `AGENTS.md` 和 `SOUL.md` 中设置明确的防护措施来防止机器人之间的回复循环。
- 对于 Slack 工具，表情回应移除语义见 `/tools/reactions`。
- 附件在允许且在大小限制内时会下载到媒体存储。

22. Telegram (Bot API)

Telegram (Bot API)

状态：通过 `grammY` 支持机器人私信和群组，已可用于生产环境。默认使用长轮询；webhook 可选。

快速设置（入门）

- 通过 @BotFather（直达链接）创建机器人。确认用户名确实是 @BotFather，然后复制 token。
- 设置 token：
- 环境变量：TELEGRAM_BOT_TOKEN=...
- 或配置：channels.telegram.botToken: "...".
- 如果两者都设置了，配置优先（环境变量回退仅适用于默认账户）。
- 启动 Gateway 网关。
- 私信访问默认使用配对模式；首次联系时需要批准配对码。

最小配置：

```
代码：{
代码： channels: {
代码： telegram: {
代码： enabled: true,
代码： botToken: "123:abc",
代码： dmPolicy: "pairing",
代码： },
代码： },
代码： }
```

这是什么

- 一个由 Gateway 网关拥有的 Telegram Bot API 渠道。
- 确定性路由：回复返回到 Telegram；模型不会选择渠道。
- 私信共享智能体的主会话；群组保持隔离（agent:<agentId>:telegram:group:<chatId>）。

设置（快速路径）

1) 创建机器人 token (BotFather)

- 打开 Telegram 并与 @BotFather（直达链接）对话。确认用户名确实是 @BotFather。
- 运行 /newbot，然后按照提示操作（名称 + 以 bot 结尾的用户名）。
- 复制 token 并安全保存。

可选的 BotFather 设置：

- /setjoiningroups — 允许/拒绝将机器人添加到群组。
- /setprivacy — 控制机器人是否可以查看所有群组消息。

2) 配置 token (环境变量或配置文件)

示例：

代码：{

代码： channels: {

代码： telegram: {

代码： enabled: true,

代码： botToken: "123:abc",

代码： dmPolicy: "pairing",

代码： groups: { "*" : { requireMention: true } },

代码： },

代码： },

代码： }

环境变量选项：TELEGRAM_BOT_TOKEN=... (适用于默认账户)。

如果环境变量和配置都设置了，配置优先。

多账户支持：使用 channels.telegram.accounts，每个账户有独立的 token 和可选的 name。参见 gateway/configuration 了解共享模式。

- 启动 Gateway 网关。当 token 解析成功时 Telegram 启动 (配置优先，环境变量回退)。
- 私信访问默认为配对模式。机器人首次被联系时批准配对码。
- 对于群组：添加机器人，决定隐私/管理员行为 (见下文)，然后设置 channels.telegram.groups 来控制提及门控和允许列表。

Token + 隐私 + 权限 (Telegram 端)

Token 创建 (BotFather)

- /newbot 创建机器人并返回 token (请保密)。
- 如果 token 泄露，通过 @BotFather 撤销/重新生成，并更新你的配置。

群组消息可见性 (隐私模式)

Telegram 机器人默认启用隐私模式，这会限制它们接收哪些群组消息。

如果你的机器人必须看到所有群组消息，有两个选项：

- 使用 `/setprivacy` 禁用隐私模式或
- 将机器人添加为群组管理员（管理员机器人可以接收所有消息）。

注意：当你切换隐私模式时，Telegram

要求将机器人从每个群组中移除并重新添加，更改才能生效。

群组权限（管理员权限）

管理员状态在群组内设置（Telegram

UI）。管理员机器人始终接收所有群组消息，因此如果需要完全可见性，请使用管理员身份。

工作原理（行为）

- 入站消息被规范化为共享渠道信封，包含回复上下文和媒体占位符。
- 群组回复默认需要提及（原生 @提及或 `agents.list[].groupChat.mentionPatterns / messages.groupChat.mentionPatterns`）。
- 多智能体覆盖：在 `agents.list[].groupChat.mentionPatterns` 上设置每个智能体的模式。
- 回复始终路由回同一个 Telegram 聊天。
- 长轮询使用 `grammy` runner，每个聊天按顺序处理；总体并发受 `agents.defaults.maxConcurrent` 限制。
- Telegram Bot API 不支持已读回执；没有 `sendReadReceipts` 选项。

草稿流式传输

OpenClaw 可以在 Telegram 私信中使用 `sendMessageDraft` 流式传输部分回复。

要求：

- 在 `@BotFather` 中为机器人启用线程模式（论坛话题模式）。
- 仅限私聊线程（Telegram 在入站消息中包含 `message_thread_id`）。
- `channels.telegram.streamMode` 未设置为 "off"（默认："partial"，"block" 启用分块草稿更新）。

草稿流式传输仅限私信；Telegram 在群组或频道中不支持此功能。

格式化（Telegram HTML）

- 出站 Telegram 文本使用 `parse_mode: "HTML"` (Telegram 支持的标签子集) 。
- 类 Markdown 输入被渲染为 Telegram 安全 HTML (粗体/斜体/删除线/代码/链接) ；块级元素被扁平化为带换行/项目符号的文本。
- 来自模型的原始 HTML 会被转义，以避免 Telegram 解析错误。
- 如果 Telegram 拒绝 HTML 负载，OpenClaw 会以纯文本重试相同的消息。

命令 (原生 + 自定义)

OpenClaw 在启动时向 Telegram 的机器人菜单注册原生命令 (如 `/status`、`/reset`、`/model`) 。

你可以通过配置向菜单添加自定义命令：

代码：{

代码： channels: {

代码： telegram: {

代码： customCommands: [

代码： { command: "backup", description: "Git 备份"},

代码： { command: "generate", description: "创建图片"},

代码：],

代码： },

代码： },

代码： }

故障排除

- 日志中出现 `setMyCommands failed` 通常意味着到 `api.telegram.org` 的出站 HTTPS/DNS 被阻止。
- 如果你看到 `sendMessage` 或 `sendChatAction` 失败，检查 IPv6 路由和 DNS。

更多帮助：渠道故障排除。

注意：

- 自定义命令仅是菜单条目；除非你在其他地方处理它们，否则 OpenClaw 不会实现它们。
- 命令名称会被规范化 (去除前导 /，转为小写)，必须匹配 `a-z`、`0-9`、`_` (1-32 个字符) 。
- 自定义命令不能覆盖原生命令。冲突会被忽略并记录日志。

- 如果禁用了 `commands.native`，则只注册自定义命令（如果没有则清空）。

限制

- 出站文本按 `channels.telegram.textChunkLimit` 分块（默认 4000）。
- 可选的换行分块：设置 `channels.telegram.chunkMode="newline"` 在长度分块之前按空行（段落边界）分割。
- 媒体下载/上传受 `channels.telegram.mediaMaxMb` 限制（默认 5）。
- Telegram Bot API 请求在 `channels.telegram.timeoutSeconds` 后超时（通过 `grammY` 默认 500）。设置较低的值以避免长时间挂起。
- 群组历史上下文使用 `channels.telegram.historyLimit`（或 `channels.telegram.accounts..historyLimit`），回退到 `messages.groupChat.historyLimit`。设置 0 禁用（默认 50）。
- 私信历史可以用 `channels.telegram.dmHistoryLimit`（用户轮次）限制。每用户覆盖：`channels.telegram.dms["<user_id>"].historyLimit`。

群组激活模式

默认情况下，机器人只响应群组中的提及（@botname 或 `agents.list[].groupChat.mentionPatterns` 中的模式）。要更改此行为：

通过配置（推荐）

代码：{

代码： channels: {

代码： telegram: {

代码： groups: {

代码： "-1001234567890": { requireMention: false }, // 在此群组中始终响应

代码： },

代码： },

代码： },

代码： }

重要： 设置 `channels.telegram.groups` 会创建一个允许列表 - 只有列出的群组（或 ""）会被接受。

论坛话题继承其父群组配置（`allowFrom`、`requireMention`、`skills`、`prompts`），除非你在 `channels.telegram.groups.<groupId>.topics.<topicId>` 下添加每话题覆盖。

要允许所有群组并始终响应：

```
代码：{  
代码： channels: {  
代码： telegram: {  
代码： groups: {  
代码： "": { requireMention: false }, // 所有群组，始终响应  
代码： },  
代码： },  
代码： },  
代码： }
```

要保持所有群组仅提及响应（默认行为）：

```
代码：{  
代码： channels: {  
代码： telegram: {  
代码： groups: {  
代码： "": { requireMention: true }, // 或完全省略 groups  
代码： },  
代码： },  
代码： },  
代码： }
```

通过命令（会话级别）

在群组中发送：

- /activation always - 响应所有消息
- /activation mention - 需要提及（默认）

注意：命令只更新会话状态。要在重启后保持持久行为，请使用配置。

获取群组聊天 ID

将群组中的任何消息转发给 Telegram 上的 @userinfobot 或 @getidsbot 以查看聊天 ID（负数，如 -1001234567890）。

提示：要获取你自己的用户 ID，私信机器人，它会回复你的用户 ID（配对消息），或者在命令启用后使用 /whoami。

隐私注意：@userinfobot

是第三方机器人。如果你更倾向于其他方式，将机器人添加到群组，发送一条消息，然后使用 `openclaw logs --follow` 读取 `chat.id`，或使用 Bot API `getUpdates`。

配置写入

默认情况下，Telegram 允许写入由渠道事件或 `/config set|unset` 触发的配置更新。

这发生在以下情况：

- 群组升级为超级群组，Telegram 发出 `migrate_to_chat_id`（聊天 ID 更改）。OpenClaw 可以自动迁移 `channels.telegram.groups`。
- 你在 Telegram 聊天中运行 `/config set` 或 `/config unset`（需要 `commands.config: true`）。

禁用方式：

代码：{

代码： `channels: { telegram: { configWrites: false } },`

代码：}

话题（论坛超级群组）

Telegram 论坛话题在每条消息中包含 `message_thread_id`。OpenClaw：

- 将 `:topic:<threadId>` 附加到 Telegram 群组会话键，使每个话题隔离。
- 发送输入指示器和回复时带上 `message_thread_id`，使响应保持在话题内。
- 通用话题（线程 id 1）是特殊的：消息发送省略 `message_thread_id`（Telegram 会拒绝），但输入指示器仍然包含它。
- 在模板上下文中暴露 `MessageThreadId + IsForum` 用于路由/模板。
- 话题特定配置可在 `channels.telegram.groups.<chatId>.topics.<threadId>` 下设置（skills、允许列表、自动回复、系统提示、禁用）。
- 话题配置继承群组设置（`requireMention`、允许列表、skills、提示、enabled），除非每话题覆盖。

私聊在某些边缘情况下可能包含 message_thread_id。OpenClaw 保持私信会话键不变，但在存在线程 id 时仍将其用于回复/草稿流式传输。

内联按钮

Telegram 支持带回调按钮的内联键盘。

```
代码：{
代码： channels: {
代码： telegram: {
代码： capabilities: {
代码： inlineButtons: "allowlist",
代码： },
代码： },
代码： },
代码： }
```

对于每账户配置：

```
代码：{
代码： channels: {
代码： telegram: {
代码： accounts: {
代码： main: {
代码： capabilities: {
代码： inlineButtons: "allowlist",
代码： },
代码： },
代码： },
代码： },
代码： },
代码： }
```

作用域：

- off — 禁用内联按钮
- dm — 仅私信（群组目标被阻止）
- group — 仅群组（私信目标被阻止）
- all — 私信 + 群组
- allowlist — 私信 + 群组，但仅限 allowFrom/groupAllowFrom 允许的发送者（与控制命令规则相同）

默认：allowlist。

旧版：capabilities: ["inlineButtons"] = inlineButtons: "all"。

发送按钮

使用带 buttons 参数的消息工具：

代码：{

代码：action: "send",

代码：channel: "telegram",

代码：to: "123456789",

代码：message: "选择一个选项："，

代码：buttons: [

代码：[

代码：{ text: "是", callback_data: "yes" },

代码：{ text: "否", callback_data: "no" },

代码：],

代码：[{ text: "取消", callback_data: "cancel" }],

代码：],

代码：}

当用户点击按钮时，回调数据会以以下格式作为消息发送回智能体：

callback_data: value

配置选项

Telegram 功能可以在两个级别配置（上面显示的对象形式；旧版字符串数组仍然支持）：

- `channels.telegram.capabilities` : 应用于所有 Telegram

账户的全局默认功能配置，除非被覆盖。

- `channels.telegram.accounts.<account>.capabilities` : 每账户功能，覆盖该特定账户的全局默认值。

当所有 Telegram 机器人/账户应具有相同行为时使用全局设置。当不同机器人需要不同行为时使用每账户配置（例如，一个账户只处理私信，而另一个允许在群组中使用）。

访问控制（私信 + 群组）

私信访问

- 默认：`channels.telegram.dmPolicy = "pairing"`。未知发送者收到配对码；在批准之前消息被忽略（配对码 1 小时后过期）。
- 批准方式：
 - `openclaw pairing list telegram`
 - `openclaw pairing approve telegram <CODE>`
- 配对是 Telegram 私信使用的默认 token 交换。详情：配对
- `channels.telegram.allowFrom` 接受数字用户 ID（推荐）或 `@username` 条目。这不是机器人用户名；使用人类发送者的 ID。向导接受 `@username` 并在可能时将其解析为数字 ID。

查找你的 Telegram 用户 ID

更安全（无第三方机器人）：

- 启动 Gateway 网关并私信你的机器人。
- 运行 `openclaw logs --follow` 并查找 `from.id`。

备选（官方 Bot API）：

- 私信你的机器人。
- 使用你的机器人 token 获取更新并读取 `message.from.id`：

代码：`curl "https://api.telegram.org/bot<bot_token>/getUpdates"`

第三方（隐私性较低）：

- 私信 `@userinfobot` 或 `@getidsbot` 并使用返回的用户 id。

群组访问

两个独立的控制：

1. 允许哪些群组（通过 `channels.telegram.groups` 的群组允许列表）：

- 无 `groups` 配置 = 允许所有群组
- 有 `groups` 配置 = 只允许列出的群组或 ""
- 示例：`"groups": { "-1001234567890": {}, "": {} }` 允许所有群组

2. 允许哪些发送者（通过 `channels.telegram.groupPolicy` 的发送者过滤）：

- `"open"` = 允许群组中的所有发送者发消息
- `"allowlist"` = 只有 `channels.telegram.groupAllowFrom` 中的发送者可以发消息
- `"disabled"` = 不接受任何群组消息

默认是 `groupPolicy: "allowlist"`（除非添加 `groupAllowFrom` 否则被阻止）。

大多数用户需要：`groupPolicy: "allowlist" + groupAllowFrom + 在 channels.telegram.groups 中列出特定群组`

长轮询 vs webhook

- 默认：长轮询（不需要公共 URL）。
- Webhook 模式：设置 `channels.telegram.webhookUrl` 和 `channels.telegram.webhookSecret`（可选 `channels.telegram.webhookPath`）。
- 本地监听器绑定到 `0.0.0.0:8787`，默认服务于 `POST /telegram-webhook`。
- 如果你的公共 URL 不同，使用反向代理并将 `channels.telegram.webhookUrl` 指向公共端点。

回复线程

Telegram 通过标签支持可选的线程回复：

- `[[reply_to_current]]` -- 回复触发消息。
- `[[reply_to:<id>]]` -- 回复特定消息 id。

通过 `channels.telegram.replyToMode` 控制：

- `first`（默认）、`all`、`off`。

音频消息（语音 vs 文件）

Telegram 区分语音备忘录（圆形气泡）和音频文件（元数据卡片）。

OpenClaw 默认使用音频文件以保持向后兼容性。

要在智能体回复中强制使用语音备忘录气泡，在回复中的任何位置包含此标签：

- `[[audio_as_voice]]` — 将音频作为语音备忘录而不是文件发送。

该标签会从发送的文本中去除。其他渠道会忽略此标签。

对于消息工具发送，设置 `asVoice: true` 并配合兼容语音的音频 media URL（当存在 media 时 message 是可选的）：

```
代码：{  
代码： action: "send",  
代码： channel: "telegram",  
代码： to: "123456789",  
代码： media: "https://example.com/voice.ogg",  
代码： asVoice: true,  
代码： }
```

贴纸

OpenClaw 支持接收和发送 Telegram 贴纸，并具有智能缓存功能。

接收贴纸

当用户发送贴纸时，OpenClaw 根据贴纸类型处理：

- 静态贴纸（WEBP）：下载并通过视觉处理。贴纸在消息内容中显示为 `<media:sticker>` 占位符。
- 动画贴纸（TGS）：跳过（Lottie 格式不支持处理）。
- 视频贴纸（WEBM）：跳过（视频格式不支持处理）。

接收贴纸时可用的模板上下文字段：

- `Sticker` — 包含以下属性的对象：
- `emoji` — 与贴纸关联的表情符号
- `setName` — 贴纸集名称
- `fileId` — Telegram 文件 ID（用于发送相同贴纸）
- `fileUniqueId` — 用于缓存查找的稳定 ID

- cachedDescription — 可用时的缓存视觉描述

贴纸缓存

贴纸通过 AI 的视觉功能处理以生成描述。由于相同的贴纸经常重复发送，OpenClaw 缓存这些描述以避免冗余的 API 调用。

工作原理：

- 首次遇到：贴纸图像被发送给 AI 进行视觉分析。AI 生成描述（例如"一只卡通猫热情地挥手"）。
- 缓存存储：描述与贴纸的文件 ID、表情符号和集合名称一起保存。
- 后续遇到：当再次看到相同贴纸时，直接使用缓存的描述。图像不会发送给 AI。

缓存位置：~/.openclaw/telegram/sticker-cache.json

缓存条目格式：

```
代码：{  
代码： "fileId": "CAACAgIAAxkBAAl...",  
代码： "fileUniqueId": "AgADBAADb6cxG2Y",  
代码： "emoji": "",  
代码： "setName": "CoolCats",  
代码： "description": "一只卡通猫热情地挥手",  
代码： "cachedAt": "2026-01-15T10:30:00.000Z"  
代码：}
```

优点：

- 通过避免对相同贴纸重复调用视觉 API 来降低 API 成本
- 缓存贴纸响应更快（无视觉处理延迟）
- 基于缓存描述启用贴纸搜索功能

缓存在接收贴纸时自动填充。无需手动缓存管理。

发送贴纸

智能体可以使用 sticker 和 sticker-search

动作发送和搜索贴纸。这些默认禁用，必须在配置中启用：


```
代码：{  
代码： channels: {  
代码： telegram: {  
代码： actions: {  
代码： sticker: true,  
代码： },  
代码： },  
代码： },  
代码： }
```

发送贴纸：

```
代码：{  
代码： action: "sticker",  
代码： channel: "telegram",  
代码： to: "123456789",  
代码： fileId: "CAACAgIAAxkBAAI...",  
代码： }
```

参数：

- fileId（必需）— 贴纸的 Telegram 文件 ID。从接收贴纸时的 Sticker.fileId 获取，或从 sticker-search 结果获取。
- replyTo（可选）— 要回复的消息 ID。
- threadId（可选）— 论坛话题的消息线程 ID。

搜索贴纸：

智能体可以按描述、表情符号或集合名称搜索缓存的贴纸：

```
代码：{  
代码： action: "sticker-search",  
代码： channel: "telegram",  
代码： query: "猫 挥手",  
代码： limit: 5,
```

代码：}

返回缓存中匹配的贴纸：

代码：{

代码： ok: true,

代码： count: 2,

代码： stickers: [

代码： {

代码： fileId: "CAACAgIAAxkBAAI...",

代码： emoji: "",

代码： description: "一只卡通猫热情地挥手",

代码： setName: "CoolCats",

代码： },

代码：],

代码：}

搜索在描述文本、表情符号字符和集合名称之间使用模糊匹配。

带线程的示例：

代码：{

代码： action: "sticker",

代码： channel: "telegram",

代码： to: "-1001234567890",

代码： fileId: "CAACAgIAAxkBAAI...",

代码： replyTo: 42,

代码： threadId: 123,

代码：}

流式传输（草稿）

Telegram 可以在智能体生成响应时流式传输草稿气泡。

OpenClaw 使用 Bot API

sendMessageDraft（不是真实消息），然后将最终回复作为普通消息发送。

要求 (Telegram Bot API 9.3+) :

- 启用话题的私聊 (机器人的论坛话题模式) 。
- 入站消息必须包含 `message_thread_id` (私有话题线程) 。
- 群组/超级群组/频道的流式传输被忽略 。

配置 :

- `channels.telegram.streamMode: "off" | "partial" | "block"` (默认 : `partial`)
- `partial` : 用最新的流式文本更新草稿气泡 。
- `block` : 以较大块 (分块) 更新草稿气泡 。
- `off` : 禁用草稿流式传输 。
- 可选 (仅用于 `streamMode: "block"`) :
- `channels.telegram.draftChunk: { minChars?, maxChars?, breakPreference? }`
- 默认值 : `minChars: 200` 、 `maxChars: 800` 、 `breakPreference: "paragraph"` (限制在 `channels.telegram.textChunkLimit` 内) 。

注意 : 草稿流式传输与分块流式传输 (渠道消息) 不同 。

分块流式传输默认关闭, 如果你想要早期 Telegram 消息而不是草稿更新, 需要 `channels.telegram.blockStreaming: true` 。

推理流 (仅限 Telegram) :

- `/reasoning stream`

在回复生成时将推理流式传输到草稿气泡中, 然后发送不带推理的最终答案 。

- 如果 `channels.telegram.streamMode` 为 `off`, 推理流被禁用 。

更多上下文 : 流式传输 + 分块 。

重试策略

出站 Telegram API 调用在遇到临时网络/429 错误时会以指数退避和抖动进行重试。通过 `channels.telegram.retry` 配置。参见重试策略 。

智能体工具 (消息 + 反应)

- 工具 : `telegram`, 使用 `sendMessage` 动作 (`to` 、 `content`, 可选 `mediaUrl` 、 `replyToMessageId` 、 `messageThreadId`) 。
- 工具 : `telegram`, 使用 `react` 动作 (`chatId` 、 `messageId` 、 `emoji`) 。

- 工具：telegram，使用 deleteMessage 动作（chatId、messageId）。
- 反应移除语义：参见 /tools/reactions。
- 工具门控：channels.telegram.actions.reactions、channels.telegram.actions.sendMessage、channels.telegram.actions.deleteMessage（默认：启用），以及 channels.telegram.actions.sticker（默认：禁用）。

反应通知

反应工作原理：

Telegram 反应作为单独的 message_reaction

事件到达，而不是消息负载中的属性。当用户添加反应时，OpenClaw：

- 从 Telegram API 接收 message_reaction 更新
- 将其转换为系统事件，格式为："Telegram reaction added: {emoji} by {user} on msg {id}"
- 使用与常规消息相同的会话键将系统事件加入队列
- 当该对话中的下一条消息到达时，系统事件被排出并前置到智能体的上下文中

智能体将反应视为对话历史中的系统通知，而不是消息元数据。

配置：

- channels.telegram.reactionNotifications：控制哪些反应触发通知
- "off" — 忽略所有反应
- "own" — 当用户对机器人消息做出反应时通知（尽力而为；内存中）（默认）
- "all" — 通知所有反应
- channels.telegram.reactionLevel：控制智能体的反应能力
- "off" — 智能体不能对消息做出反应
- "ack" — 机器人发送确认反应（处理时显示）（默认）
- "minimal" — 智能体可以少量反应（指导：每 5-10 次交换 1 次）
- "extensive" — 智能体可以在适当时自由反应

论坛群组：论坛群组中的反应包含 message_thread_id，使用类似 agent:main:telegram:group:{chatId}:topic:{threadId} 的会话键。这确保同一话题中的反应和消息保持在一起。

示例配置：

代码：{

代码： channels: {

代码： telegram: {

代码： reactionNotifications: "all", // 查看所有反应

代码： reactionLevel: "minimal", // 智能体可以少量反应

代码： },

代码： },

代码： }

要求：

- Telegram 机器人必须在 allowed_updates 中明确请求 message_reaction（由 OpenClaw 自动配置）
- 对于 webhook 模式，反应包含在 webhook allowed_updates 中
- 对于轮询模式，反应包含在 getUpdates allowed_updates 中

投递目标（CLI/cron）

- 使用聊天 id（123456789）或用户名（@name）作为目标。
- 示例：openclaw message send --channel telegram --target 123456789 --message "hi"。

故障排除

机器人不响应群组中的非提及消息：

- 如果你设置了 channels.telegram.groups..requireMention=false，Telegram 的 Bot API 隐私模式必须禁用。
- BotFather：/setprivacy → Disable（然后从群组中移除并重新添加机器人）
- openclaw channels status 在配置期望未提及群组消息时显示警告。
- openclaw channels status --probe 可以额外检查显式数字群组 ID 的成员资格（它无法审计通配符 "" 规则）。
- 快速测试：/activation always（仅会话级别；使用配置以持久化）

机器人完全看不到群组消息：

- 如果设置了 channels.telegram.groups，群组必须被列出或使用 ""
- 在 @BotFather 中检查隐私设置 → "Group Privacy" 应为 OFF

- 验证机器人确实是成员（不仅仅是没有读取权限的管理员）
- 检查 Gateway 网关日志：openclaw logs --follow（查找"skipping group message"）

机器人响应提及但不响应 /activation always：

- /activation 命令更新会话状态但不持久化到配置
- 要持久化行为，将群组添加到 channels.telegram.groups 并设置 requireMention: false

像 /status 这样的命令不起作用：

- 确保你的 Telegram 用户 ID 已授权（通过配对或 channels.telegram.allowFrom）
- 即使在 groupPolicy: "open" 的群组中，命令也需要授权

长轮询在 Node 22+ 上立即中止（通常与代理/自定义 fetch 有关）：

- Node 22+ 对 AbortSignal 实例更严格；外部信号可以立即中止 fetch 调用。
- 升级到规范化中止信号的 OpenClaw 构建版本，或在可以升级之前在 Node 20 上运行 Gateway 网关。

机器人启动后静默停止响应（或日志显示 HttpError: Network request ... failed）：

- 某些主机首先将 api.telegram.org 解析为 IPv6。如果你的服务器没有可用的 IPv6 出口，grammyY 可能会卡在仅 IPv6 的请求上。
- 通过启用 IPv6 出口或强制 api.telegram.org 使用 IPv4 解析来修复（例如，使用 IPv4 A 记录添加 /etc/hosts 条目，或在你的 OS DNS 堆栈中优先使用 IPv4），然后重启 Gateway 网关。
- 快速检查：dig +short api.telegram.org A 和 dig +short api.telegram.org AAAA 确认 DNS 返回的内容。

配置参考 (Telegram)

完整配置：配置

提供商选项：

- channels.telegram.enabled：启用/禁用渠道启动。
- channels.telegram.botToken：机器人 token (BotFather)。
- channels.telegram.tokenFile：从文件路径读取 token。
- channels.telegram.dmPolicy：pairing | allowlist | open | disabled（默认：pairing）。
- channels.telegram.allowFrom：私信允许列表 (id/用户名)。open 需要 ""。

- channels.telegram.groupPolicy : open | allowlist | disabled (默认 : allowlist) 。
- channels.telegram.groupAllowFrom : 群组发送者允许列表 (id/用户名) 。
- channels.telegram.groups : 每群组默认值 + 允许列表 (使用 "" 作为全局默认值) 。
- channels.telegram.groups.<id>.requireMention : 提及门控默认值 。
- channels.telegram.groups.<id>.skills : skill 过滤器 (省略 = 所有 skills , 空 = 无) 。
- channels.telegram.groups.<id>.allowFrom : 每群组发送者允许列表覆盖 。
- channels.telegram.groups.<id>.systemPrompt : 群组的额外系统提示 。
- channels.telegram.groups.<id>.enabled : 为 false 时禁用群组 。
- channels.telegram.groups.<id>.topics.<threadId>. : 每话题覆盖 (与群组相同的字段) 。
- channels.telegram.groups.<id>.topics.<threadId>.requireMention : 每话题提及门控覆盖 。
- channels.telegram.capabilities.inlineButtons : off | dm | group | all | allowlist (默认 : allowlist) 。
- channels.telegram.accounts.<account>.capabilities.inlineButtons : 每账户覆盖 。
- channels.telegram.replyToMode : off | first | all (默认 : off) 。
- channels.telegram.textChunkLimit : 出站分块大小 (字符) 。
- channels.telegram.chunkMode : length (默认) 或 newline 在长度分块之前按空行 (段落边界) 分割 。
- channels.telegram.linkPreview : 切换出站消息的链接预览 (默认 : true) 。
- channels.telegram.streamMode : off | partial | block (草稿流式传输) 。
- channels.telegram.mediaMaxMb : 入站/出站媒体上限 (MB) 。
- channels.telegram.retry : 出站 Telegram API 调用的重试策略 (attempts 、 minDelayMs 、 maxDelayMs 、 jitter) 。
- channels.telegram.network.autoSelectFamily : 覆盖 Node autoSelectFamily (true=启用 , false=禁用) 。在 Node 22 上默认禁用以避免 Happy Eyeballs 超时 。
- channels.telegram.proxy : Bot API 调用的代理 URL (SOCKS/HTTP) 。
- channels.telegram.webhookUrl : 启用 webhook 模式 (需要 channels.telegram.webhookSecret) 。
- channels.telegram.webhookSecret : webhook 密钥 (设置 webhookUrl 时必需) 。
- channels.telegram.webhookPath : 本地 webhook 路径 (默认 /telegram-webhook) 。
- channels.telegram.actions.reactions : 门控 Telegram 工具反应 。

- `channels.telegram.actions.sendMessage` : 门控 Telegram 工具消息发送。
- `channels.telegram.actions.deleteMessage` : 门控 Telegram 工具消息删除。
- `channels.telegram.actions.sticker` : 门控 Telegram 贴纸动作 — 发送和搜索（默认：`false`）。
- `channels.telegram.reactionNotifications` : `off` | `own` | `all` — 控制哪些反应触发系统事件（未设置时默认：`own`）。
- `channels.telegram.reactionLevel` : `off` | `ack` | `minimal` | `extensive` — 控制智能体的反应能力（未设置时默认：`minimal`）。

相关全局选项：

- `agents.list[].groupChat.mentionPatterns`（提及门控模式）。
- `messages.groupChat.mentionPatterns`（全局回退）。
- `commands.native`（默认为 `"auto"` → Telegram/Discord 开启，Slack 关闭）、`commands.text`、`commands.useAccessGroups`（命令行为）。使用 `channels.telegram.commands.native` 覆盖。
- `messages.responsePrefix`、`messages.ackReaction`、`messages.ackReactionScope`、`messages.removeAckAfterReply`。

23. Tlon（插件）

Tlon（插件）

Tlon 是一个基于 Urbit 构建的去中心化即时通讯工具。OpenClaw 连接到你的 Urbit ship，可以响应私信和群聊消息。群组回复默认需要 @ 提及，并可通过允许列表进一步限制。

状态：通过插件支持。支持私信、群组提及、话题回复和纯文本媒体回退（URL 附加到说明文字）。不支持表情回应、投票和原生媒体上传。

需要插件

Tlon 作为插件提供，不包含在核心安装中。

通过 CLI 安装（npm 仓库）：

代码：`openclaw plugins install @openclaw/tlon`

本地检出（从 git 仓库运行时）：

代码：`openclaw plugins install ./extensions/tlon`

详情：插件

设置

- 安装 Tlon 插件。
- 获取你的 ship URL 和登录代码。
- 配置 channels.tlon。
- 重启 Gateway 网关。
- 私信机器人或在群组频道中提及它。

最小配置（单账户）：

代码：{

代码： channels: {

代码： tlon: {

代码： enabled: true,

代码： ship: "~sampel-palnet",

代码： url: "https://your-ship-host",

代码： code: "lidlut-tabwed-pillex-ridrup",

代码： },

代码： },

代码： }

群组频道

默认启用自动发现。你也可以手动固定频道：

代码：{

代码： channels: {

代码： tlon: {

代码： groupChannels: ["chat/~host-ship/general", "chat/~host-ship/support"],

代码： },

代码： },

代码： }

禁用自动发现：

代码：{

代码： channels: {

代码： tlon: {

代码： autoDiscoverChannels: false,

代码： },

代码： },

代码： }

访问控制

私信允许列表（空 = 允许全部）：

代码：{

代码： channels: {

代码： tlon: {

代码： dmAllowlist: ["~zod", "~nec"],

代码： },

代码： },

代码： }

群组授权（默认受限）：

代码：{

代码： channels: {

代码： tlon: {

代码： defaultAuthorizedShips: ["~zod"],

代码： authorization: {

代码： channelRules: {

代码： "chat/~host-ship/general": {

代码： mode: "restricted",

代码： allowedShips: ["~zod", "~nec"],

代码： },

代码： "chat/~host-ship/announcements": {

代码： mode: "open",

代码： },

代码： },

代码： },

代码： },

代码： },

代码： }

投递目标 (CLI/cron)

与 openclaw message send 或 cron 投递一起使用：

- 私信：~sampel-palnet 或 dm/~sampel-palnet
- 群组：chat/~host-ship/channel 或 group:~host-ship/channel

注意事项

- 群组回复需要提及（例如 ~your-bot-ship）才能响应。
- 话题回复：如果入站消息在话题中，OpenClaw 会在话题内回复。
- 媒体：sendMedia 回退为文本 + URL（无原生上传）。

24. 渠道故障排除

渠道故障排除

首先运行：

代码： openclaw doctor

代码： openclaw channels status --probe

channels status --probe

会在检测到常见渠道配置错误时输出警告，并包含小型实时检查（凭据、部分权限/成员资格）。

渠道

- Discord： /channels/discord#troubleshooting
- Telegram： /channels/telegram#troubleshooting

- WhatsApp : [/channels/whatsapp#troubleshooting-quick](#)

Telegram 快速修复

- 日志显示 `HttpError: Network request for 'sendMessage' failed` 或 `sendChatAction` → 检查 IPv6 DNS。如果 `api.telegram.org` 优先解析为 IPv6 而主机缺少 IPv6 出站连接，请强制使用 IPv4 或启用 IPv6。参见 [/channels/telegram#troubleshooting](#)。
- 日志显示 `setMyCommands failed` → 检查到 `api.telegram.org` 的出站 HTTPS 和 DNS 可达性（常见于限制严格的 VPS 或代理环境）。

25. Twitch（插件）

Twitch（插件）

通过 IRC 连接支持 Twitch 聊天。OpenClaw 以 Twitch 用户（机器人账户）身份连接，在频道中接收和发送消息。

需要插件

Twitch 作为插件发布，未与核心安装捆绑。

通过 CLI 安装（npm 注册表）：

代码：`openclaw plugins install @openclaw/twitch`

本地检出（从 git 仓库运行时）：

代码：`openclaw plugins install ./extensions/twitch`

详情：插件

快速设置（新手）

- 为机器人创建一个专用的 Twitch 账户（或使用现有账户）。
- 生成凭证：Twitch Token Generator
- 选择 Bot Token
- 确认已选择 `chat:read` 和 `chat:write` 权限范围
- 复制 Client ID 和 Access Token
- 查找你的 Twitch 用户 ID：
- 配置令牌：

- 环境变量：OPENCLAW_TWITCH_ACCESS_TOKEN=...（仅限默认账户）
- 或配置：channels.twitch.accessToken
- 如果两者都设置，配置优先（环境变量回退仅适用于默认账户）。
- 启动 Gateway 网关。

重要：添加访问控制（allowFrom 或 allowedRoles）以防止未授权用户触发机器人。requireMention 默认为 true。

最小配置：

代码：{

代码： channels: {

代码： twitch: {

代码： enabled: true,

代码： username: "openclaw", // 机器人的 Twitch 账户

代码： accessToken: "oauth:abc123...", // OAuth Access Token（或使用 OPENCLAW_TWITCH_ACCESS_TOKEN 环境变量）

代码： clientId: "xyz789...", // Token Generator 中的 Client ID

代码： channel: "vevisk", // 要加入的 Twitch 频道聊天（必填）

代码： allowFrom: ["123456789"], // （推荐）仅限你的 Twitch 用户 ID - 从 <https://www.streamweasels.com/tools/convert-twitch-username-to-user-id/> 获取

代码： },

代码： },

代码： }

它是什么

- 由 Gateway 网关拥有的 Twitch 渠道。
- 确定性路由：回复总是返回到 Twitch。
- 每个账户映射到一个隔离的会话键 agent:<agentId>:twitch:<accountName>。
- username 是机器人账户（进行身份验证的账户），channel 是要加入的聊天室。

设置（详细）

生成凭证

使用 Twitch Token Generator :

- 选择 Bot Token
- 确认已选择 chat:read 和 chat:write 权限范围
- 复制 Client ID 和 Access Token

无需手动注册应用。令牌在几小时后过期。

配置机器人

环境变量（仅限默认账户）：

代码：OPENCLAW_TWITCH_ACCESS_TOKEN=oauth:abc123...

或配置：

代码：{

代码： channels: {

代码： twitch: {

代码： enabled: true,

代码： username: "openclaw",

代码： accessToken: "oauth:abc123...",

代码： clientId: "xyz789...",

代码： channel: "vevisk",

代码： },

代码： },

代码： }

如果环境变量和配置都设置了，配置优先。

访问控制（推荐）

代码：{

代码： channels: {

代码： twitch: {

代码： allowFrom: ["123456789"], // （推荐）仅限你的 Twitch 用户 ID

代码： },

代码：},

代码：}

优先使用 `allowFrom` 作为硬性允许列表。如果你想要基于角色的访问控制，请改用 `allowedRoles`。

可用角色："moderator"、"owner"、"vip"、"subscriber"、"all"。

为什么用用户 ID？用户名可以更改，允许冒充。用户 ID 是永久的。

查找你的 Twitch 用户 ID：Twitch 用户名转换为 ID)

令牌刷新（可选）

来自 Twitch Token Generator 的令牌无法自动刷新 - 过期时需要重新生成。

要实现自动令牌刷新，请在 Twitch Developer Console 创建你自己的 Twitch 应用并添加到配置中：

代码：{

代码：channels: {

代码：twitch: {

代码：clientSecret: "your_client_secret",

代码：refreshToken: "your_refresh_token",

代码：},

代码：},

代码：}

机器人会在令牌过期前自动刷新，并记录刷新事件。

多账户支持

使用 `channels.twitch.accounts` 配置每个账户的令牌。参阅 `gateway/configuration` 了解共享模式。

示例（一个机器人账户在两个频道中）：

代码：{

代码：channels: {

代码：twitch: {

```
代码：accounts: {
代码：channel1: {
代码：username: "openclaw",
代码：accessToken: "oauth:abc123...",
代码：clientId: "xyz789...",
代码：channel: "vevisk",
代码：},
代码：channel2: {
代码：username: "openclaw",
代码：accessToken: "oauth:def456...",
代码：clientId: "uvw012...",
代码：channel: "secondchannel",
代码：},
代码：},
代码：},
代码：},
代码：},
代码：}
```

注意：每个账户需要自己的令牌（每个频道一个令牌）。

访问控制

基于角色的限制

```
代码：{
代码：channels: {
代码：twitch: {
代码：accounts: {
代码：default: {
代码：allowedRoles: ["moderator", "vip"],
代码：},
代码：},
代码：},
代码：},
```


代码： },

代码： }

按用户 ID 允许列表（最安全）

代码： {

代码： channels: {

代码： twitch: {

代码： accounts: {

代码： default: {

代码： allowFrom: ["123456789", "987654321"],

代码： },

代码： },

代码： },

代码： },

代码： }

基于角色的访问（替代方案）

allowFrom 是硬性允许列表。设置后，只允许这些用户 ID。

如果你想要基于角色的访问，请不设置 allowFrom，改为配置 allowedRoles：

代码： {

代码： channels: {

代码： twitch: {

代码： accounts: {

代码： default: {

代码： allowedRoles: ["moderator"],

代码： },

代码： },

代码： },

代码： },

代码： }

禁用 @提及要求

默认情况下，requireMention 为 true。要禁用并响应所有消息：

代码：{

代码： channels: {

代码： twitch: {

代码： accounts: {

代码： default: {

代码： requireMention: false,

代码： },

代码： },

代码： },

代码： },

代码： }

故障排除

首先，运行诊断命令：

代码： openclaw doctor

代码： openclaw channels status --probe

机器人不响应消息

检查访问控制： 确保你的用户 ID 在 allowFrom 中，或临时移除 allowFrom 并设置 allowedRoles: ["all"] 来测试。

检查机器人是否在频道中： 机器人必须加入 channel 中指定的频道。

令牌问题

"Failed to connect"或身份验证错误：

- 验证 accessToken 是 OAuth 访问令牌值（通常以 oauth: 前缀开头）
- 检查令牌具有 chat:read 和 chat:write 权限范围
- 如果使用令牌刷新，验证 clientSecret 和 refreshToken 已设置

令牌刷新不工作

检查日志中的刷新事件：

代码：Using env token source for mybot

代码：Access token refreshed for user 123456 (expires in 14400s)

如果你看到"token refresh disabled (no refresh token)"：

- 确保提供了 clientSecret
- 确保提供了 refreshToken

配置

账户配置：

- username - 机器人用户名
- accessToken - 具有 chat:read 和 chat:write 权限的 OAuth 访问令牌
- clientId - Twitch Client ID（来自 Token Generator 或你的应用）
- channel - 要加入的频道（必填）
- enabled - 启用此账户（默认：true）
- clientSecret - 可选：用于自动令牌刷新
- refreshToken - 可选：用于自动令牌刷新
- expiresIn - 令牌过期时间（秒）
- obtainmentTimestamp - 令牌获取时间戳
- allowFrom - 用户 ID 允许列表
- allowedRoles - 基于角色的访问控制 ("moderator" | "owner" | "vip" | "subscriber" | "all")
- requireMention - 需要 @提及（默认：true）

提供商选项：

- channels.twitch.enabled - 启用/禁用渠道启动
- channels.twitch.username - 机器人用户名（简化的单账户配置）
- channels.twitch.accessToken - OAuth 访问令牌（简化的单账户配置）
- channels.twitch.clientId - Twitch Client ID（简化的单账户配置）
- channels.twitch.channel - 要加入的频道（简化的单账户配置）

- channels.twitch.accounts.<accountName> - 多账户配置（以上所有账户字段）

完整示例：

```
代码：{
代码： channels: {
代码： twitch: {
代码： enabled: true,
代码： username: "openclaw",
代码： accessToken: "oauth:abc123...",
代码： clientId: "xyz789...",
代码： channel: "vevisk",
代码： clientSecret: "secret123...",
代码： refreshToken: "refresh456...",
代码： allowFrom: ["123456789"],
代码： allowedRoles: ["moderator", "vip"],
代码： accounts: {
代码： default: {
代码： username: "mybot",
代码： accessToken: "oauth:abc123...",
代码： clientId: "xyz789...",
代码： channel: "your_channel",
代码： enabled: true,
代码： clientSecret: "secret123...",
代码： refreshToken: "refresh456...",
代码： expiresIn: 14400,
代码： obtainmentTimestamp: 1706092800000,
代码： allowFrom: ["123456789", "987654321"],
代码： allowedRoles: ["moderator"],
代码： },
代码： },
代码： },
代码： }
```

代码： },

代码： }

工具操作

智能体可以调用 twitch 执行以下操作：

- send - 向频道发送消息

示例：

代码： {

代码： action: "twitch",

代码： params: {

代码： message: "Hello Twitch!",

代码： to: "#mychannel",

代码： },

代码： }

安全与运维

- 将令牌视为密码 - 永远不要将令牌提交到 git
- 使用自动令牌刷新 用于长时间运行的机器人
- 使用用户 ID 允许列表 而不是用户名进行访问控制
- 监控日志 查看令牌刷新事件和连接状态
- 最小化令牌权限范围 - 只请求 chat:read 和 chat:write
- 如果卡住：在确认没有其他进程拥有会话后重启 Gateway 网关

限制

- 每条消息 500 个字符（在单词边界自动分块）
- 分块前会去除 Markdown
- 无速率限制（使用 Twitch 内置的速率限制）

26. WhatsApp（网页渠道）

WhatsApp（网页渠道）

状态：仅支持通过 Baileys 的 WhatsApp Web。Gateway 网关拥有会话。

快速设置（新手）

- 如果可能，使用单独的手机号码（推荐）。
- 在 `~/.openclaw/openclaw.json` 中配置 WhatsApp。
- 运行 `openclaw channels login` 扫描二维码（关联设备）。
- 启动 Gateway 网关。

最小配置：

```
代码：{  
代码： channels: {  
代码： whatsapp: {  
代码： dmPolicy: "allowlist",  
代码： allowFrom: ["+15551234567"],  
代码： },  
代码： },  
代码： }
```

目标

- 在一个 Gateway 网关进程中支持多个 WhatsApp 账户（多账户）。
- 确定性路由：回复返回到 WhatsApp，无模型路由。
- 模型能看到足够的上下文来理解引用回复。

配置写入

默认情况下，WhatsApp 允许写入由 `/config set|unset` 触发的配置更新（需要 `commands.config: true`）。

禁用方式：

```
代码：{  
代码： channels: { whatsapp: { configWrites: false } },  
代码： }
```

架构（谁拥有什么）

- Gateway 网关拥有 Baileys socket 和收件箱循环。
- CLI / macOS 应用与 Gateway 网关通信；不直接使用 Baileys。
- 发送出站消息需要活跃的监听器；否则发送会快速失败。

获取手机号码（两种模式）

WhatsApp 需要真实手机号码进行验证。VoIP 和虚拟号码通常会被封锁。在 WhatsApp 上运行 OpenClaw 有两种支持的方式：

专用号码（推荐）

为 OpenClaw

使用单独的手机号码。最佳用户体验，清晰的路由，无自聊天怪异问题。理想设置：备用/旧 Android 手机 + eSIM。保持 Wi-Fi 和电源连接，通过二维码关联。

WhatsApp Business：你可以在同一设备上使用不同号码的 WhatsApp Business。非常适合将个人 WhatsApp 分开——安装 WhatsApp Business 并在那里注册 OpenClaw 号码。

示例配置（专用号码，单用户允许列表）：

```
代码：{
代码： channels: {
代码： whatsapp: {
代码： dmPolicy: "allowlist",
代码： allowFrom: ["+15551234567"],
代码： },
代码： },
代码： }
```

配对模式（可选）：

如果你想使用配对而不是允许列表，请将 channels.whatsapp.dmPolicy 设置为 pairing。未知发送者会收到配对码；使用以下命令批准：

```
openclaw pairing approve whatsapp <code>
```

个人号码（备选方案）

快速备选方案：在你自己的号码上运行 OpenClaw。给自己发消息（WhatsApp"给自己发消息"）进行测试，这样就不会打扰联系人。在设置和实验期间需要在主手机上阅读验证码。必须启用自聊天模式。

当向导询问你的个人 WhatsApp

号码时，输入你将用于发送消息的手机（所有者/发送者），而不是助手号码。

示例配置（个人号码，自聊天）：

代码：{

代码："whatsapp": {

代码："selfChatMode": true,

代码："dmPolicy": "allowlist",

代码："allowFrom": ["+15551234567"]

代码：}

代码：}

当设置了 identity.name 时，自聊天回复默认为 [{identity.name}]（否则为 [openclaw]），前提是 messages.responsePrefix 未设置。明确设置它可以自定义或禁用前缀（使用 "" 来移除）。

号码获取提示

- 本地 eSIM 来自你所在国家的移动运营商（最可靠）
- 奥地利：hot.at
- 英国：giffgaff — 免费 SIM 卡，无合约
- 预付费 SIM 卡 — 便宜，只需接收一条验证短信

避免：TextNow、Google Voice、大多数"免费短信"服务——WhatsApp 会积极封锁这些。

提示：该号码只需要接收一条验证短信。之后，WhatsApp Web 会话通过 creds.json 持久化。

为什么不用 Twilio？

- 早期 OpenClaw 版本支持 Twilio 的 WhatsApp Business 集成。
- WhatsApp Business 号码不适合个人助手。
- Meta 强制执行 24 小时回复窗口；如果你在过去 24 小时内没有回复，商业号码无法发起新消息。

- 高频或"频繁"使用会触发激进的封锁，因为商业账户不适合发送大量个人助手消息。
- 结果：投递不可靠且频繁被封锁，因此该支持已被移除。

登录 + 凭证

- 登录命令：`openclaw channels login`（通过关联设备扫描二维码）。
- 多账户登录：`openclaw channels login --account <id>`（`<id> = accountId`）。
- 默认账户（省略 `--account` 时）：如果存在则为 `default`，否则为第一个配置的账户 `id`（排序后）。
- 凭证存储在 `~/.openclaw/credentials/whatsapp/<accountId>/creds.json`。
- 备份副本在 `creds.json.bak`（损坏时恢复）。
- 旧版兼容性：较旧的安装将 `Baileys` 文件直接存储在 `~/.openclaw/credentials/` 中。
- 登出：`openclaw channels logout`（或 `--account <id>`）删除 WhatsApp 认证状态（但保留共享的 `oauth.json`）。
- 已登出的 `socket =>` 错误提示重新关联。

进站流程（私信 + 群组）

- WhatsApp 事件来自 `messages.upsert`（`Baileys`）。
- 收件箱监听器在关闭时分离，以避免在测试/重启时累积事件处理器。
- 状态/广播聊天被忽略。
- 直接聊天使用 `E.164`；群组使用群组 `JID`。
- 私信策略：`channels.whatsapp.dmPolicy` 控制直接聊天访问（默认：`pairing`）。
- 配对：未知发送者会收到配对码（通过 `openclaw pairing approve whatsapp <code>` 批准；码在 1 小时后过期）。
- 开放：需要 `channels.whatsapp.allowFrom` 包含 `""`。
- 你关联的 WhatsApp 号码是隐式信任的，因此自身消息会跳过 `channels.whatsapp.dmPolicy` 和 `channels.whatsapp.allowFrom` 检查。

个人号码模式（备选方案）

如果你在个人 WhatsApp 号码上运行 OpenClaw，请启用 `channels.whatsapp.selfChatMode`（见上面的示例）。

行为：

- 出站私信永远不会触发配对回复（防止打扰联系人）。

- 入站未知发送者仍遵循 channels.whatsapp.dmPolicy 。
- 自聊天模式（allowFrom 包含你的号码）避免自动已读回执并忽略提及 JID 。
- 非自聊天私信会发送已读回执 。

已读回执

默认情况下，Gateway 网关在接受入站 WhatsApp 消息后将其标记为已读（蓝色勾号）。

全局禁用：

代码：{

代码： channels: { whatsapp: { sendReadReceipts: false } },

代码：}

按账户禁用：

代码：{

代码： channels: {

代码： whatsapp: {

代码： accounts: {

代码： personal: { sendReadReceipts: false },

代码： },

代码： },

代码： },

代码：}

注意事项：

- 自聊天模式始终跳过已读回执 。

WhatsApp 常见问题：发送消息 + 配对

当我关联 WhatsApp 时，OpenClaw 会给随机联系人发消息吗？

不会。默认私信策略是配对，因此未知发送者只会收到配对码，他们的消息不会被处理。OpenClaw 只会回复它收到的聊天，或你明确触发的发送（智能体/CLI）。

WhatsApp 上的配对是如何工作的？

配对是未知发送者的私信门控：

- 来自新发送者的第一条私信返回一个短码（消息不会被处理）。
- 使用以下命令批准：`openclaw pairing approve whatsapp <code>`（使用 `openclaw pairing list whatsapp` 列出）。
- 码在 1 小时后过期；每个渠道的待处理请求上限为 3 个。

多个人可以在一个 WhatsApp 号码上使用不同的 OpenClaw 实例吗？

可以，通过 bindings 将每个发送者路由到不同的智能体（peer kind: "dm"，发送者 E.164 如 +15551234567）。回复仍然来自同一个 WhatsApp 账户，直接聊天会折叠到每个智能体的主会话，因此每人使用一个智能体。私信访问控制（dmPolicy/allowFrom）是每个 WhatsApp 账户全局的。参见多智能体路由。

为什么向导会询问我的手机号码？

向导使用它来设置你的允许列表/所有者，以便允许你自己的私信。它不会用于自动发送。如果你在个人 WhatsApp 号码上运行，请使用相同的号码并启用 `channels.whatsapp.selfChatMode`。

消息规范化（模型看到的内容）

- Body 是带有信封的当前消息正文。
- 引用回复上下文始终附加：

代码：`[Replying to +1555 id:ABC123]`

代码：`<quoted text or <media:...>>`

代码：`[/Replying]`

- 回复元数据也会设置：
- ReplyTold = stanzald
- ReplyToBody = 引用正文或媒体占位符
- ReplyToSender = 已知时为 E.164
- 纯媒体入站消息使用占位符：
- `<media:image|video|audio|document|sticker>`

群组

- 群组映射到 `agent:<agentId>:whatsapp:group:<jid>` 会话。
- 群组策略：`channels.whatsapp.groupPolicy = open|disabled|allowlist`（默认 allowlist）。
- 激活模式：

- mention（默认）：需要 @提及或正则匹配。
- always：始终触发。
- /activation mention|always 仅限所有者，必须作为独立消息发送。
- 所有者 = channels.whatsapp.allowFrom（如果未设置则为自身 E.164）。
- 历史注入（仅待处理）：
- 最近未处理的消息（默认 50 条）插入在：

[Chat messages since your last reply - for context]（已在会话中的消息不会重新注入）

- 当前消息在：

[Current message - respond to this]

- 附加发送者后缀：[from: Name (+E164)]
- 群组元数据缓存 5 分钟（主题 + 参与者）。

回复投递（线程）

- WhatsApp Web 发送标准消息（当前 Gateway 网关无引用回复线程）。
- 此渠道忽略回复标签。

确认表情（收到时自动回应）

WhatsApp 可以在收到传入消息时立即自动发送表情回应，在机器人生成回复之前。这为用户提供即时反馈，表明他们的消息已收到。

配置：

代码：{

代码： "whatsapp": {

代码： "ackReaction": {

代码： "emoji": "",

代码： "direct": true,

代码： "group": "mentions"

代码： }

代码： }

代码： }

选项：

- emoji（字符串）：用于确认的表情（例如""、""、""）。为空或省略 = 功能禁用。
- direct（布尔值，默认：true）：在直接/私信聊天中发送表情回应。
- group（字符串，默认："mentions"）：群聊行为：
- "always"：对所有群消息做出回应（即使没有 @提及）
- "mentions"：仅在机器人被 @提及时做出回应
- "never"：从不在群组中做出回应

按账户覆盖：

代码：{

代码："whatsapp": {

代码："accounts": {

代码："work": {

代码："ackReaction": {

代码："emoji": "",

代码："direct": false,

代码："group": "always"

代码：}

代码：}

代码：}

代码：}

代码：}

行为说明：

- 表情回应应在消息收到时立即发送，在输入指示器或机器人回复之前。
- 在 requireMention: false（激活：always）的群组中，group: "mentions" 会对所有消息做出回应（不仅仅是 @提及）。
- 即发即忘：表情回应失败会被记录但不会阻止机器人回复。
- 群组表情回应会自动包含参与者 JID。
- WhatsApp 忽略 messages.ackReaction；请改用 channels.whatsapp.ackReaction。

智能体工具（表情回应）

- 工具：whatsapp，带有 react 动作（chatJid、messageld、emoji，可选 remove）。
- 可选：participant（群组发送者）、fromMe（对自己的消息做出回应）、accountId（多账户）。
- 表情移除语义：参见 /tools/reactions。
- 工具门控：channels.whatsapp.actions.reactions（默认：启用）。

限制

- 出站文本按 channels.whatsapp.textChunkLimit 分块（默认 4000）。
- 可选换行分块：设置 channels.whatsapp.chunkMode="newline" 在长度分块之前按空行（段落边界）分割。
- 进站媒体保存受 channels.whatsapp.mediaMaxMb 限制（默认 50 MB）。
- 出站媒体项受 agents.defaults.mediaMaxMb 限制（默认 5 MB）。

出站发送（文本 + 媒体）

- 使用活跃的网页监听器；如果 Gateway 网关未运行则报错。
- 文本分块：每条消息最大 4k（可通过 channels.whatsapp.textChunkLimit 配置，可选 channels.whatsapp.chunkMode）。
- 媒体：
- 支持图片/视频/音频/文档。
- 音频作为 PTT 发送；audio/ogg => audio/ogg; codecs=opus。
- 仅在第一个媒体项上添加标题。
- 媒体获取支持 HTTP(S) 和本地路径。
- 动画 GIF：WhatsApp 期望带有 gifPlayback: true 的 MP4 以实现内联循环。
- CLI：openclaw message send --media <mp4> --gif-playback
- Gateway 网关：send 参数包含 gifPlayback: true

语音消息（PTT 音频）

WhatsApp 将音频作为语音消息（PTT 气泡）发送。

- 最佳效果：OGG/Opus。OpenClaw 将 audio/ogg 重写为 audio/ogg; codecs=opus。
- WhatsApp 忽略 [[audio_as_voice]]（音频已作为语音消息发送）。

媒体限制 + 优化

- 默认出站上限：5 MB（每个媒体项）。
- 覆盖：agents.defaults.mediaMaxMb。
- 图片自动优化为上限以下的 JPEG（调整大小 + 质量扫描）。
- 超大媒体 => 错误；媒体回复降级为文本警告。

心跳

- Gateway 网关心跳记录连接健康状态（web.heartbeatSeconds，默认 60 秒）。
- 智能体心跳可以按智能体配置（agents.list[].heartbeat）或通过 agents.defaults.heartbeat 全局配置（当没有设置按智能体条目时的降级）。
- 使用配置的心跳提示词（默认：Read HEARTBEAT.md if it exists (workspace context). Follow it strictly. Do not infer or repeat old tasks from prior chats. If nothing needs attention, reply HEARTBEAT_OK.）+ HEARTBEAT_OK 跳过行为。
- 投递默认为最后使用的渠道（或配置的目标）。

重连行为

- 退避策略：web.reconnect：
- initialMs、maxMs、factor、jitter、maxAttempts。
- 如果达到 maxAttempts，网页监控停止（降级）。
- 已登出 => 停止并要求重新关联。

配置快速映射

- channels.whatsapp.dmPolicy（私信策略：pairing/allowlist/open/disabled）。
- channels.whatsapp.selfChatMode（同手机设置；机器人使用你的个人 WhatsApp 号码）。
- channels.whatsapp.allowFrom（私信允许列表）。WhatsApp 使用 E.164 手机号码（无用户名）。
- channels.whatsapp.mediaMaxMb（入站媒体保存上限）。
- channels.whatsapp.ackReaction（消息收到时的自动回应：{emoji, direct, group}）。
- channels.whatsapp.accounts.<accountId>。（按账户设置 + 可选 authDir）。
- channels.whatsapp.accounts.<accountId>.mediaMaxMb（按账户入站媒体上限）。
- channels.whatsapp.accounts.<accountId>.ackReaction（按账户确认回应覆盖）。
- channels.whatsapp.groupAllowFrom（群组发送者允许列表）。

- channels.whatsapp.groupPolicy（群组策略）。
- channels.whatsapp.historyLimit / channels.whatsapp.accounts.<accountId>.historyLimit（群组历史上下文；0 禁用）。
- channels.whatsapp.dmHistoryLimit（私信历史限制，按用户轮次）。按用户覆盖：channels.whatsapp.dms["<phone>"].historyLimit。
- channels.whatsapp.groups（群组允许列表 + 提及门控默认值；使用 "" 允许全部）
- channels.whatsapp.actions.reactions（门控 WhatsApp 工具表情回应）。
- agents.list[].groupChat.mentionPatterns（或 messages.groupChat.mentionPatterns）
- messages.groupChat.historyLimit
- channels.whatsapp.messagePrefix（入站前缀；按账户：channels.whatsapp.accounts.<accountId>.messagePrefix；已弃用：messages.messagePrefix）
- messages.responsePrefix（出站前缀）
- agents.defaults.mediaMaxMb
- agents.defaults.heartbeat.every
- agents.defaults.heartbeat.model（可选覆盖）
- agents.defaults.heartbeat.target
- agents.defaults.heartbeat.to
- agents.defaults.heartbeat.session
- agents.list[].heartbeat.（按智能体覆盖）
- session.（scope、idle、store、mainKey）
- web.enabled（为 false 时禁用渠道启动）
- web.heartbeatSeconds
- web.reconnect.

日志 + 故障排除

- 子系统：whatsapp/inbound、whatsapp/outbound、web-heartbeat、web-reconnect。
- 日志文件：/tmp/openclaw/openclaw-YYYY-MM-DD.log（可配置）。
- 故障排除指南：Gateway 网关故障排除。

故障排除（快速）

未关联 / 需要二维码登录

- 症状：channels status 显示 linked: false 或警告"Not linked"。
- 修复：在 Gateway 网关主机上运行 openclaw channels login 并扫描二维码（WhatsApp → 设置 → 关联设备）。

已关联但断开连接 / 重连循环

- 症状：channels status 显示 running, disconnected 或警告"Linked but disconnected"。
- 修复：openclaw doctor（或重启 Gateway 网关）。如果问题持续，通过 channels login 重新关联并检查 openclaw logs --follow。

Bun 运行时

- 不推荐 Bun。WhatsApp（Baileys）和 Telegram 在 Bun 上不可靠。
- 请使用 Node 运行 Gateway 网关。（参见入门指南运行时说明。）

27. Zalo (Bot API)

Zalo (Bot API)

状态：实验性。仅支持私信；根据 Zalo 文档，群组即将推出。

需要插件

Zalo 以插件形式提供，不包含在核心安装中。

- 通过 CLI 安装：openclaw plugins install @openclaw/zalo
- 或在新手引导期间选择 Zalo 并确认安装提示
- 详情：插件

快速设置（初学者）

- 安装 Zalo 插件：
- 从源代码检出：openclaw plugins install ./extensions/zalo
- 从 npm（如果已发布）：openclaw plugins install @openclaw/zalo
- 或在新手引导中选择 Zalo 并确认安装提示
- 设置 token：
- 环境变量：ZALO_BOT_TOKEN=...
- 或配置：channels.zalo.botToken: "..."。

- 重启 Gateway 网关（或完成新手引导）。
- 私信访问默认为配对模式；首次联系时批准配对码。

最小配置：

代码：{

代码： channels: {

代码： zalo: {

代码： enabled: true,

代码： botToken: "12345689:abc-xyz",

代码： dmPolicy: "pairing",

代码： },

代码： },

代码： }

它是什么

Zalo 是一款专注于越南市场的即时通讯应用；其 Bot API 让 Gateway 网关可以运行一个用于一对一对话的 bot。

它非常适合需要确定性路由回 Zalo 的支持或通知场景。

- 由 Gateway 网关拥有的 Zalo Bot API 渠道。
- 确定性路由：回复返回到 Zalo；模型不会选择渠道。
- 私信共享智能体的主会话。
- 群组尚不支持（Zalo 文档标注"即将推出"）。

设置（快速路径）

1) 创建 bot token（Zalo Bot 平台）

- 前往 并登录。
- 创建新 bot 并配置其设置。
- 复制 bot token（格式：12345689:abc-xyz）。

2) 配置 token（环境变量或配置）

示例：

代码：{

代码： channels: {

代码： zalo: {

代码： enabled: true,

代码： botToken: "12345689:abc-xyz",

代码： dmPolicy: "pairing",

代码： },

代码： },

代码： }

环境变量选项：ZALO_BOT_TOKEN=...（仅适用于默认账户）。

多账户支持：使用 channels.zalo.accounts 配置每账户 token 和可选的 name。

- 重启 Gateway 网关。当 token 被解析（环境变量或配置）时，Zalo 启动。
- 私信访问默认为配对模式。当 bot 首次被联系时批准配对码。

工作原理（行为）

- 进站消息被规范化为带有媒体占位符的共享渠道信封。
- 回复始终路由回同一 Zalo 聊天。
- 默认使用长轮询；可通过 channels.zalo.webhookUrl 启用 webhook 模式。

限制

- 出站文本按 2000 字符分块（Zalo API 限制）。
- 媒体下载/上传受 channels.zalo.mediaMaxMb 限制（默认 5）。
- 由于 2000 字符限制使流式传输效果不佳，默认阻止流式传输。

访问控制（私信）

私信访问

- 默认：channels.zalo.dmPolicy = "pairing"。未知发送者会收到配对码；消息在批准前会被忽略（配对码 1 小时后过期）。
- 通过以下方式批准：
- openclaw pairing list zalo

- openclaw pairing approve zalo <CODE>
- 配对是默认的令牌交换方式。详情：配对
- channels.zalo.allowFrom 接受数字用户 ID（无用户名查找功能）。

长轮询与 webhook

- 默认：长轮询（不需要公共 URL）。
- Webhook 模式：设置 channels.zalo.webhookUrl 和 channels.zalo.webhookSecret。
- Webhook secret 必须为 8-256 个字符。
- Webhook URL 必须使用 HTTPS。
- Zalo 发送事件时带有 X-Bot-API-Secret-Token 头用于验证。
- Gateway 网关 HTTP 在 channels.zalo.webhookPath 处理 webhook 请求（默认为 webhook URL 路径）。

注意：根据 Zalo API 文档，getUpdates（轮询）和 webhook 是互斥的。

支持的消息类型

- 文本消息：完全支持，2000 字符分块。
- 图片消息：下载和处理入站图片；通过 sendPhoto 发送图片。
- 贴纸：已记录但未完全处理（无智能体响应）。
- 不支持的类型：已记录（例如来自受保护用户的消息）。

功能

功能 状态
----- -----
私信 支持
群组 即将推出（根据 Zalo 文档）
媒体（图片） 支持
表情回应 不支持
主题 不支持
投票 不支持
原生命令 不支持
流式传输 已阻止（2000 字符限制）

投递目标 (CLI/cron)

- 使用聊天 id 作为目标。
- 示例：openclaw message send --channel zalo --target 123456789 --message "hi"。

故障排除

Bot 不响应：

- 检查 token 是否有效：openclaw channels status --probe
- 验证发送者已被批准（配对或 allowFrom）
- 检查 Gateway 网关日志：openclaw logs --follow

Webhook 未收到事件：

- 确保 webhook URL 使用 HTTPS
- 验证 secret token 为 8-256 个字符
- 确认 Gateway 网关 HTTP 端点在配置的路径上可访问
- 检查 getUpdates 轮询未在运行（它们是互斥的）

配置参考 (Zalo)

完整配置：配置

提供商选项：

- channels.zalo.enabled：启用/禁用渠道启动。
- channels.zalo.botToken：来自 Zalo Bot 平台的 bot token。
- channels.zalo.tokenFile：从文件路径读取 token。
- channels.zalo.dmPolicy：pairing | allowlist | open | disabled（默认：pairing）。
- channels.zalo.allowFrom：私信允许列表（用户 ID）。open 需要 ""。向导会询问数字 ID。
- channels.zalo.mediaMaxMb：入站/出站媒体上限（MB，默认 5）。
- channels.zalo.webhookUrl：启用 webhook 模式（需要 HTTPS）。
- channels.zalo.webhookSecret：webhook secret（8-256 字符）。
- channels.zalo.webhookPath：Gateway 网关 HTTP 服务器上的 webhook 路径。
- channels.zalo.proxy：API 请求的代理 URL。

多账户选项：

- `channels.zalo.accounts.<id>.botToken`：每账户 token。
- `channels.zalo.accounts.<id>.tokenFile`：每账户 token 文件。
- `channels.zalo.accounts.<id>.name`：显示名称。
- `channels.zalo.accounts.<id>.enabled`：启用/禁用账户。
- `channels.zalo.accounts.<id>.dmPolicy`：每账户私信策略。
- `channels.zalo.accounts.<id>.allowFrom`：每账户允许列表。
- `channels.zalo.accounts.<id>.webhookUrl`：每账户 webhook URL。
- `channels.zalo.accounts.<id>.webhookSecret`：每账户 webhook secret。
- `channels.zalo.accounts.<id>.webhookPath`：每账户 webhook 路径。
- `channels.zalo.accounts.<id>.proxy`：每账户代理 URL。

28. Zalo Personal（非官方）

Zalo Personal（非官方）

状态：实验性。此集成通过 `zca-cli` 自动化个人 Zalo 账户。

警告：这是一个非官方集成，可能导致账户被暂停/封禁。使用风险自负。

需要插件

Zalo Personal 作为插件提供，不包含在核心安装中。

- 通过 CLI 安装：`openclaw plugins install @openclaw/zalouser`
- 或从源码检出安装：`openclaw plugins install ./extensions/zalouser`
- 详情：插件

前置条件：zca-cli

Gateway 网关机器必须在 PATH 中有可用的 `zca` 二进制文件。

- 验证：`zca --version`
- 如果缺失，请安装 `zca-cli`（参见 `extensions/zalouser/README.md` 或上游 `zca-cli` 文档）。

快速设置（新手）

- 安装插件（见上文）。

- 登录（QR，在 Gateway 网关机器上）：
- `openclaw channels login --channel zalouser`
- 用 Zalo 手机应用扫描终端中的二维码。
- 启用渠道：

代码：{

代码： channels: {

代码： zalouser: {

代码： enabled: true,

代码： dmPolicy: "pairing",

代码： },

代码： },

代码： }

- 重启 Gateway 网关（或完成新手引导）。
- 私信访问默认为配对模式；首次联系时批准配对码。

这是什么

- 使用 `zca listen` 接收入站消息。
- 使用 `zca msg ...` 发送回复（文本/媒体/链接）。
- 专为"个人账户"使用场景设计，适用于 Zalo Bot API 不可用的情况。

命名

渠道 ID 为 `zalouser`，以明确表示这是自动化个人 Zalo 用户账户（非官方）。我们保留 `zalo` 用于未来可能的官方 Zalo API 集成。

查找 ID（目录）

使用目录 CLI 发现联系人/群组及其 ID：

代码： `openclaw directory self --channel zalouser`

代码： `openclaw directory peers list --channel zalouser --query "name"`

代码： `openclaw directory groups list --channel zalouser --query "work"`

限制

- 出站文本分块为约 2000 字符（Zalo 客户端限制）。
- 默认阻止流式传输。

访问控制（私信）

channels.zalouser.dmPolicy 支持：pairing | allowlist | open | disabled（默认：pairing）。

channels.zalouser.allowFrom 接受用户 ID 或名称。向导会在可用时通过 zca friend find 将名称解析为 ID。

通过以下方式批准：

- openclaw pairing list zalouser
- openclaw pairing approve zalouser <code>

群组访问（可选）

• 默认：channels.zalouser.groupPolicy = "open"（允许群组）。使用 channels.defaults.groupPolicy 在未设置时覆盖默认值。

• 通过以下方式限制为允许列表：

- channels.zalouser.groupPolicy = "allowlist"
- channels.zalouser.groups（键为群组 ID 或名称）
- 阻止所有群组：channels.zalouser.groupPolicy = "disabled"。
- 配置向导可以提示输入群组允许列表。
- 启动时，OpenClaw 将允许列表中的群组/用户名称解析为 ID 并记录映射；未解析的条目保持原样。

示例：

代码：{

代码： channels: {

代码： zalouser: {

代码： groupPolicy: "allowlist",

代码： groups: {

代码： "123456789": { allow: true },

代码： "Work Chat": { allow: true },

代码： },

代码： },

代码： },

代码： }

多账户

账户映射到 zca 配置文件。示例：

代码： {

代码： channels: {

代码： zalouser: {

代码： enabled: true,

代码： defaultAccount: "default",

代码： accounts: {

代码： work: { enabled: true, profile: "work" },

代码： },

代码： },

代码： },

代码： }

故障排除

找不到 zca：

- 安装 zca-cli 并确保它在 Gateway 网关进程的 PATH 中。

登录不保持：

- openclaw channels status --probe
- 重新登录：openclaw channels logout --channel zalouser && openclaw channels login --channel zalouser

第四章：模型与提供商配置

1. Anthropic (Claude)

Anthropic (Claude)

Anthropic 构建了 Claude 模型系列，并通过 API 提供访问。

在 OpenClaw 中，你可以使用 API 密钥或 setup-token 进行认证。

选项 A：Anthropic API 密钥

适用于：标准 API 访问和按用量计费。

在 Anthropic Console 中创建你的 API 密钥。

CLI 设置

代码：openclaw onboard

代码：# 选择：Anthropic API key

代码：# 或非交互式

代码：openclaw onboard --anthropic-api-key "\$ANTHROPIC_API_KEY"

配置片段

代码：{

代码：env: { ANTHROPIC_API_KEY: "sk-ant-..." },

代码：agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },

代码：}

提示缓存 (Anthropic API)

OpenClaw 支持 Anthropic 的提示缓存功能。这是仅限 API；订阅认证不支持缓存设置。

配置

在模型配置中使用 cacheRetention 参数：

值	缓存时长	描述
-----	-----	-----
none	无缓存	禁用提示缓存

| short | 5 分钟 | API 密钥认证的默认值 |

| long | 1 小时 | 扩展缓存（需要 beta 标志） |

代码：{

代码：agents: {

代码：defaults: {

代码：models: {

代码："anthropic/claude-opus-4-5": {

代码：params: { cacheRetention: "long" },

代码：},

代码：},

代码：},

代码：},

代码：}

默认值

使用 Anthropic API 密钥认证时，OpenClaw 会自动为所有 Anthropic 模型应用 `cacheRetention: "short"`（5 分钟缓存）。你可以通过在配置中显式设置 `cacheRetention` 来覆盖此设置。

旧版参数

为了向后兼容，仍支持旧版 `cacheControlTtl` 参数：

- "5m" 映射到 short
- "1h" 映射到 long

我们建议迁移到新的 `cacheRetention` 参数。

OpenClaw 在 Anthropic API 请求中包含 `extended-cache-ttl-2025-04-11` beta 标志；如果你覆盖提供商头信息，请保留它（参见 `/gateway/configuration`）。

选项 B：Claude setup-token

适用于：使用你的 Claude 订阅。

在哪里获取 setup-token

setup-token 由 Claude Code CLI 创建，而不是 Anthropic Console。你可以在任何机器上运行：

代码：claude setup-token

将令牌粘贴到 OpenClaw（向导：Anthropic token (paste setup-token)），或在 Gateway 网关主机上运行：

代码：openclaw models auth setup-token --provider anthropic

如果你在不同的机器上生成了令牌，请粘贴它：

代码：openclaw models auth paste-token --provider anthropic

CLI 设置

代码：# 在新手引导期间粘贴 setup-token

代码：openclaw onboard --auth-choice setup-token

配置片段

代码：{

代码：agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },

代码：}

注意事项

- 使用 claude setup-token 生成 setup-token 并粘贴，或在 Gateway 网关主机上运行 openclaw models auth setup-token。
- 如果你在 Claude 订阅上看到"OAuth token refresh failed ..."，请使用 setup-token 重新认证。参见 </gateway/troubleshooting#oauth-token-refresh-failed-anthropic-claude-subscription>。
- 认证详情 + 重用规则在 </concepts/oauth>。

故障排除

401 错误/令牌突然失效

- Claude 订阅认证可能过期或被撤销。重新运行 claude setup-token 并将其粘贴到 Gateway 网关主机。

- 如果 Claude CLI 登录在不同的机器上，在 Gateway 网关主机上使用 `openclaw models auth paste-token --provider anthropic`。

No API key found for provider "anthropic"

- 认证是按智能体的。新智能体不会继承主智能体的密钥。
- 为该智能体重新运行新手引导，或在 Gateway 网关主机上粘贴 `setup-token` / API 密钥，然后使用 `openclaw models status` 验证。

No credentials found for profile anthropic:default

- 运行 `openclaw models status` 查看哪个认证配置文件处于活动状态。
- 重新运行新手引导，或为该配置文件粘贴 `setup-token` / API 密钥。

No available auth profile (all in cooldown/unavailable)

- 检查 `openclaw models status --json` 中的 `auth.unusableProfiles`。
- 添加另一个 Anthropic 配置文件或等待冷却期结束。

更多信息：`/gateway/troubleshooting` 和 `/help/faq`。

2. Amazon Bedrock

Amazon Bedrock

OpenClaw 可以通过 `pi.ai` 的 Bedrock Converse 流式提供商使用 Amazon Bedrock 模型。Bedrock 认证使用 AWS SDK 默认凭证链，而非 API 密钥。

`pi.ai` 支持的功能

- 提供商：`amazon-bedrock`
- API：`bedrock-converse-stream`
- 认证：AWS 凭证（环境变量、共享配置或实例角色）
- 区域：`AWS_REGION` 或 `AWS_DEFAULT_REGION`（默认：`us-east-1`）

自动模型发现

如果检测到 AWS 凭证，OpenClaw 可以自动发现支持流式传输和文本输出的 Bedrock 模型。发现功能使用 `bedrock:ListFoundationModels`，并会被缓存（默认：1 小时）。

配置选项位于 `models.bedrockDiscovery` 下：

```
代码：{  
代码： models: {  
代码： bedrockDiscovery: {  
代码： enabled: true,  
代码： region: "us-east-1",  
代码： providerFilter: ["anthropic", "amazon"],  
代码： refreshInterval: 3600,  
代码： defaultContextWindow: 32000,  
代码： defaultMaxTokens: 4096,  
代码： },  
代码： },  
代码： }
```

注意事项：

- enabled 在存在 AWS 凭证时默认为 true。
- region 默认为 AWS_REGION 或 AWS_DEFAULT_REGION，然后是 us-east-1。
- providerFilter 匹配 Bedrock 提供商名称（例如 anthropic）。
- refreshInterval 单位为秒；设置为 0 可禁用缓存。
- defaultContextWindow（默认：32000）和 defaultMaxTokens（默认：4096）用于已发现的模型（如果你知道模型限制，可以覆盖这些值）。

设置（手动）

- 确保 AWS 凭证在 Gateway 网关主机上可用：

代码：# 可选：

代码：# 可选（Bedrock API 密钥/Bearer 令牌）：

- 在配置中添加 Bedrock 提供商和模型（无需 apiKey）：

```
代码：{  
代码： models: {  
代码： providers: {  
代码： "amazon-bedrock": {
```

```
代码： baseUrl: "https://bedrock-runtime.us-east-1.amazonaws.com",
代码： api: "bedrock-converse-stream",
代码： auth: "aws-sdk",
代码： models: [
代码： {
代码： id: "anthropic.claude-opus-4-5-20251101-v1:0",
代码： name: "Claude Opus 4.5 (Bedrock)",
代码： reasoning: true,
代码： input: ["text", "image"],
代码： cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码： contextWindow: 200000,
代码： maxTokens: 8192,
代码： },
代码： ],
代码： },
代码： },
代码： },
代码： },
代码： agents: {
代码： defaults: {
代码： model: { primary: "amazon-bedrock/anthropic.claude-opus-4-5-20251101-v1:0" },
代码： },
代码： },
代码： }
```

EC2 实例角色

当在附加了 IAM 角色的 EC2 实例上运行 OpenClaw 时，AWS SDK 会自动使用实例元数据服务（IMDS）进行认证。但是，OpenClaw 的凭证检测目前只检查环境变量，不检查 IMDS 凭证。

解决方法：设置 `AWS_PROFILE=default` 以表明 AWS 凭证可用。实际认证仍然通过 IMDS 使用实例角色。

代码：# 添加到 `~/.bashrc` 或你的 shell 配置文件

EC2 实例角色所需的 IAM 权限：

- bedrock:InvokeModel
- bedrock:InvokeModelWithResponseStream
- bedrock:ListFoundationModels（用于自动发现）

或者附加托管策略 AmazonBedrockFullAccess。

快速设置：

代码：# 1. 创建 IAM 角色和实例配置文件

代码：aws iam create-role --role-name EC2-Bedrock-Access \

代码：--assume-role-policy-document '{

代码："Version": "2012-10-17",

代码："Statement": [{

代码："Effect": "Allow",

代码："Principal": {"Service": "ec2.amazonaws.com"},

代码："Action": "sts:AssumeRole"

代码：}]

代码：}'

代码：aws iam attach-role-policy --role-name EC2-Bedrock-Access \

代码：--policy-arn arn:aws:iam::aws:policy/AmazonBedrockFullAccess

代码：aws iam create-instance-profile --instance-profile-name EC2-Bedrock-Access

代码：aws iam add-role-to-instance-profile \

代码：--instance-profile-name EC2-Bedrock-Access \

代码：--role-name EC2-Bedrock-Access

代码：# 2. 附加到你的 EC2 实例

代码：aws ec2 associate-iam-instance-profile \

代码：--instance-id i-xxxxx \

代码：--iam-instance-profile Name=EC2-Bedrock-Access

代码：# 3. 在 EC2 实例上启用发现功能


```
代码：openclaw config set models.bedrockDiscovery.enabled true
代码：openclaw config set models.bedrockDiscovery.region us-east-1

代码：# 4. 设置解决方法所需的环境变量
代码：echo 'export AWS_PROFILE=default' >> ~/.bashrc
代码：echo 'export AWS_REGION=us-east-1' >> ~/.bashrc
代码：source ~/.bashrc

代码：# 5. 验证模型已被发现
代码：openclaw models list
```

注意事项

- Bedrock 需要在你的 AWS 账户/区域中启用模型访问。
- 自动发现需要 bedrock:ListFoundationModels 权限。
- 如果你使用配置文件，请在 Gateway 网关主机上设置 AWS_PROFILE。
- OpenClaw 按以下顺序获取凭证来源：AWS_BEARER_TOKEN_BEDROCK，然后是 AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY，然后是 AWS_PROFILE，最后是默认的 AWS SDK 链。
- 推理支持取决于模型；请查看 Bedrock 模型卡了解当前功能。
- 如果你更喜欢托管密钥流程，也可以在 Bedrock 前面放置一个 OpenAI 兼容的代理，并将其配置为 OpenAI 提供商。

3. Claude Max API 代理

Claude Max API 代理

claude-max-api-proxy 是一个社区工具，将你的 Claude Max/Pro 订阅暴露为 OpenAI 兼容的 API 端点。这使你可以将订阅与任何支持 OpenAI API 格式的工具配合使用。

为什么使用它？

方式	费用	适用场景
Anthropic API	按 token 计费（Opus 约 \$15/M 输入，\$75/M 输出）	生产应用、高流量场景
Claude Max 订阅	每月固定 \$200	个人使用、开发、无限用量

如果你有 Claude Max 订阅并希望与 OpenAI 兼容工具配合使用，这个代理可以帮你节省大量费用。

工作原理

代码：你的应用 → claude-max-api-proxy → Claude Code CLI → Anthropic（通过订阅）

代码：（OpenAI 格式）（转换格式）（使用你的登录凭据）

该代理：

- 在`接受 OpenAI 格式的请求
- 将其转换为 Claude Code CLI 命令
- 以 OpenAI 格式返回响应（支持流式传输）

安装

代码：# 需要 Node.js 20+ 和 Claude Code CLI

代码：npm install -g claude-max-api-proxy

代码：# 验证 Claude CLI 已认证

代码：claude --version

使用方法

启动服务器

代码：claude-max-api

代码：# 服务器运行在 http://localhost:3456

测试

代码：# 健康检查

代码：curl http://localhost:3456/health

代码：# 列出模型

代码：curl http://localhost:3456/v1/models

代码：# 聊天补全

代码：curl http://localhost:3456/v1/chat/completions \

代码：-H "Content-Type: application/json" \

代码：-d '{

代码："model": "claude-opus-4",

代码："messages": [{"role": "user", "content": "Hello!"}]

代码：}'

与 OpenClaw 配合使用

你可以将 OpenClaw 指向该代理作为自定义 OpenAI 兼容端点：

代码：{

代码：env: {

代码：OPENAI_API_KEY: "not-needed",

代码：OPENAI_BASE_URL: "http://localhost:3456/v1",

代码：},

代码：agents: {

代码：defaults: {

代码：model: { primary: "openai/claude-opus-4" },

代码：},

代码：},

代码：}

可用模型

| 模型 ID | 对应模型 |

| ----- | ----- |

| claude-opus-4 | Claude Opus 4 |

| claude-sonnet-4 | Claude Sonnet 4 |

| claude-haiku-4 | Claude Haiku 4 |

macOS 自动启动

创建 LaunchAgent 以自动运行代理：

代码：cat > ~/Library/LaunchAgents/com.claude-max-api.plist << 'EOF'

代码：<?xml version="1.0" encoding="UTF-8"?>

代码：<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">

代码：<plist version="1.0">

代码：<dict>

代码：<key>Label</key>

代码：<string>com.claude-max-api</string>

代码：<key>RunAtLoad</key>

代码：<true/>

代码：<key>KeepAlive</key>

代码：<true/>

代码：<key>ProgramArguments</key>

代码：<array>

代码：<string>/usr/local/bin/node</string>

代码：<string>/usr/local/lib/node_modules/claude-max-api-proxy/dist/server/standalone.js</string>

代码：</array>

代码：<key>EnvironmentVariables</key>

代码：<dict>

代码：<key>PATH</key>

代码：<string>/usr/local/bin:/opt/homebrew/bin:~/.local/bin:/usr/bin:/bin</string>

代码：</dict>

代码：</dict>

代码：</plist>

代码：EOF

代码：launchctl bootstrap gui/\${id -u} ~/Library/LaunchAgents/com.claude-max-api.plist

链接

- npm:
- GitHub:
- Issues:

注意事项

- 这是一个社区工具，并非由 Anthropic 或 OpenClaw 官方支持
- 需要有效的 Claude Max/Pro 订阅并已认证 Claude Code CLI
- 代理在本地运行，不会将数据发送到任何第三方服务器
- 完全支持流式响应

另请参阅

- Anthropic 提供商 - OpenClaw 与 Claude 的原生集成，使用 setup-token 或 API 密钥
- OpenAI 提供商 - 适用于 OpenAI/Codex 订阅

4. Deepgram（音频转录）

Deepgram（音频转录）

Deepgram 是一个语音转文字 API。在 OpenClaw 中，它通过 `tools.media.audio` 用于接收音频/语音消息的转录。

启用后，OpenClaw 会将音频文件上传到 Deepgram，并将转录文本注入回复管道（`{{Transcript}}` + `[Audio]` 块）。这不是流式处理；它使用的是预录音转录端点。

网站：

文档：

快速开始

- 设置你的 API 密钥：

代码：`DEEPGRAM_API_KEY=dg_...`

- 启用提供商：

代码：{

代码： `tools: {`

代码： `media: {`

代码： `audio: {`

代码： `enabled: true,`

代码： models: [{ provider: "deepgram", model: "nova-3" }],

代码： },

代码： },

代码： },

代码： }

选项

- model : Deepgram 模型 ID (默认 : nova-3)

- language : 语言提示 (可选)

-

tools.media.audio.providerOptions.deepgram.detect_language : 启用语言检测 (可选)

- tools.media.audio.providerOptions.deepgram.punctuate : 启用标点符号 (可选)

- tools.media.audio.providerOptions.deepgram.smart_format : 启用智能格式化 (可选)

带语言参数的示例：

代码： {

代码： tools: {

代码： media: {

代码： audio: {

代码： enabled: true,

代码： models: [{ provider: "deepgram", model: "nova-3", language: "en" }],

代码： },

代码： },

代码： },

代码： }

带 Deepgram 选项的示例：

代码： {

代码： tools: {

代码： media: {

代码： audio: {

代码：enabled: true,
代码：providerOptions: {
代码：deepgram: {
代码：detect_language: true,
代码：punctuate: true,
代码：smart_format: true,
代码：},
代码：},
代码：models: [{ provider: "deepgram", model: "nova-3" }],
代码：},
代码：},
代码：},
代码：}

注意事项

- 认证遵循标准提供商认证顺序；DEEPGRAM_API_KEY 是最简单的方式。
- 使用代理时，可通过 tools.media.audio.baseUrl 和 tools.media.audio.headers 覆盖端点或请求头。
- 输出遵循与其他提供商相同的音频规则（大小限制、超时、转录文本注入）。

5. GitHub Copilot

GitHub Copilot

什么是 GitHub Copilot？

GitHub Copilot 是 GitHub 的 AI 编程助手。它为你的 GitHub 账户和订阅计划提供 Copilot 模型的访问权限。OpenClaw 可以通过两种不同的方式将 Copilot 用作模型提供商。

在 OpenClaw 中使用 Copilot 的两种方式

1) 内置 GitHub Copilot 提供商（`github-copilot`）

使用原生设备登录流程获取 GitHub 令牌，然后在 OpenClaw 运行时将其兑换为 Copilot API 令牌。这是默认且最简单的方式，因为它不需要 VS Code。

2) Copilot Proxy 插件（`copilot-proxy`）

使用 Copilot Proxy VS Code 扩展作为本地桥接。OpenClaw 与代理的 /v1 端点通信，并使用你在其中配置的模型列表。当你已经在 VS Code 中运行 Copilot Proxy 或需要通过它进行路由时，选择此方式。你必须启用该插件并保持 VS Code 扩展运行。

使用 GitHub Copilot 作为模型提供商（github-copilot）。登录命令运行 GitHub 设备流程，保存认证配置文件，并更新你的配置以使用该配置文件。

CLI 设置

代码：`openclaw models auth login-github-copilot`

系统会提示你访问一个 URL 并输入一次性代码。请保持终端打开直到流程完成。

可选参数

代码：`openclaw models auth login-github-copilot --profile-id github-copilot:work`

代码：`openclaw models auth login-github-copilot --yes`

设置默认模型

代码：`openclaw models set github-copilot/gpt-4o`

配置片段

代码：{

代码：`agents: { defaults: { model: { primary: "github-copilot/gpt-4o" } } },`

代码：}

注意事项

- 需要交互式 TTY；请直接在终端中运行。
- Copilot 模型的可用性取决于你的订阅计划；如果某个模型被拒绝，请尝试其他 ID（例如 github-copilot/gpt-4.1）。
- 登录会将 GitHub 令牌存储在认证配置文件中，并在 OpenClaw 运行时将其兑换为 Copilot API 令牌。

6. GLM 模型

GLM 模型

GLM 是一个模型系列（而非公司），通过 Z.AI 平台提供。在 OpenClaw 中，GLM 模型通过 zai 提供商访问，模型 ID 格式如 zai/glm-4.7。

CLI 设置

代码：openclaw onboard --auth-choice zai-api-key

配置片段

代码：{

代码：env: { ZAI_API_KEY: "sk-..." },

代码：agents: { defaults: { model: { primary: "zai/glm-4.7" } } },

代码：}

注意事项

- GLM 版本和可用性可能会变化；请查阅 Z.AI 的文档获取最新信息。
- 示例模型 ID 包括 glm-4.7 和 glm-4.6。
- 有关提供商的详细信息，请参阅 /providers/zai。

7. 模型提供商

模型提供商

OpenClaw 可以使用许多 LLM 提供商。选择一个提供商，进行认证，然后将默认模型设置为 provider/model。

正在寻找聊天渠道文档（WhatsApp/Telegram/Discord/Slack/Mattermost（插件）等）？参见渠道。

亮点：Venice（Venice AI）

Venice 是我们推荐的 Venice AI 设置，用于隐私优先的推理，并可选择使用 Opus 处理困难任务。

- 默认：venice/llama-3.3-70b
- 最佳综合：venice/claude-opus-45（Opus 仍然是最强的）

参见 Venice AI。

快速开始

- 与提供商进行认证（通常通过 openclaw onboard）。
- 设置默认模型：

代码：{

代码： agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },

代码：}

提供商文档

- OpenAI (API + Codex)
- Anthropic (API + Claude Code CLI)
- Qwen (OAuth)
- OpenRouter
- Vercel AI Gateway
- Moonshot AI (Kimi + Kimi Coding)
- OpenCode Zen
- Amazon Bedrock
- Z.AI
- Xiaomi
- GLM 模型
- MiniMax
- Venice (Venice AI，注重隐私)
- Ollama (本地模型)

转录提供商

- Deepgram (音频转录)

社区工具

- Claude Max API Proxy - 将 Claude Max/Pro 订阅作为 OpenAI 兼容的 API 端点使用

有关完整的提供商目录（xAI、Groq、Mistral 等）和高级配置，

参见模型提供商。

8. MiniMax

MiniMax

MiniMax 是一家构建 M2/M2.1 模型系列的 AI 公司。当前面向编程的版本是 MiniMax M2.1（2025 年 12 月 23 日），专为现实世界的复杂任务而构建。

来源：MiniMax M2.1 发布说明

模型概述（M2.1）

MiniMax 强调 M2.1 的以下改进：

- 更强的多语言编程能力（Rust、Java、Go、C++、Kotlin、Objective-C、TS/JS）。
- 更好的 Web/应用开发和美观输出质量（包括原生移动端）。
- 改进的复合指令处理，适用于办公风格的工作流程，基于交错思考和集成约束执行。
- 更简洁的响应，更低的 token 使用量和更快的迭代循环。
- 更强的工具/智能体框架兼容性和上下文管理（Claude Code、Droid/Factory AI、Cline、Kilo Code、Roo Code、BlackBox）。
- 更高质量的对话和技术写作输出。

MiniMax M2.1 vs MiniMax M2.1 Lightning

- 速度：Lightning 是 MiniMax 定价文档中的"快速"变体。
- 成本：定价显示相同的输入成本，但 Lightning 的输出成本更高。
- 编程计划路由：Lightning 后端在 MiniMax 编程计划中不能直接使用。MiniMax 自动将大多数请求路由到 Lightning，但在流量高峰期会回退到常规 M2.1 后端。

选择设置方式

MiniMax OAuth（编程计划）— 推荐

适用于：通过 OAuth 快速设置 MiniMax 编程计划，无需 API 密钥。

启用内置 OAuth 插件并进行认证：

代码：openclaw plugins enable minimax-portal-auth # 如果已加载则跳过

代码：openclaw gateway restart # 如果 Gateway 网关已在运行则重启

代码：openclaw onboard --auth-choice minimax-portal

系统会提示你选择端点：

- Global - 国际用户 (api.minimax.io)
- CN - 中国用户 (api.minimaxi.com)

详情参见 MiniMax OAuth 插件 README。

MiniMax M2.1 (API 密钥)

适用于：使用 Anthropic 兼容 API 的托管 MiniMax。

通过 CLI 配置：

- 运行 openclaw configure
- 选择 Model/auth
- 选择 MiniMax M2.1

代码：{

代码：env: { MINIMAX_API_KEY: "sk-..." },

代码：agents: { defaults: { model: { primary: "minimax/MiniMax-M2.1" } } },

代码：models: {

代码：mode: "merge",

代码：providers: {

代码：minimax: {

代码：baseUrl: "https://api.minimax.io/anthropic",

代码：apiKey: "\${MINIMAX_API_KEY}",

代码：api: "anthropic-messages",

代码：models: [

代码：{

代码：id: "MiniMax-M2.1",

代码：name: "MiniMax M2.1",

代码：reasoning: false,

代码：input: ["text"],

代码：cost: { input: 15, output: 60, cacheRead: 2, cacheWrite: 10 },

代码：contextWindow: 200000,

代码：maxTokens: 8192,

代码：},

代码：],

代码：},

代码：},

代码：},

代码：}

MiniMax M2.1 作为备用（Opus 为主）

适用于：保持 Opus 4.5 为主模型，故障时切换到 MiniMax M2.1。

代码：{

代码：env: { MINIMAX_API_KEY: "sk-..." },

代码：agents: {

代码：defaults: {

代码：models: {

代码："anthropic/claude-opus-4-5": { alias: "opus" },

代码："minimax/MiniMax-M2.1": { alias: "minimax" },

代码：},

代码：model: {

代码：primary: "anthropic/claude-opus-4-5",

代码：fallbacks: ["minimax/MiniMax-M2.1"],

代码：},

代码：},

代码：},

代码：}

可选：通过 LM Studio 本地运行（手动）

适用于：使用 LM Studio 进行本地推理。

我们在强大硬件（例如台式机/服务器）上使用 LM Studio 的本地服务器运行 MiniMax M2.1 时看到了出色的效果。

通过 openclaw.json 手动配置：

```
代码：{
代码： agents: {
代码： defaults: {
代码： model: { primary: "lmstudio/minimax-m2.1-gs32" },
代码： models: { "lmstudio/minimax-m2.1-gs32": { alias: "Minimax" } },
代码： },
代码： },
代码： models: {
代码： mode: "merge",
代码： providers: {
代码： lmstudio: {
代码： baseUrl: "http://127.0.0.1:1234/v1",
代码： apiKey: "lmstudio",
代码： api: "openai-responses",
代码： models: [
代码： {
代码： id: "minimax-m2.1-gs32",
代码： name: "MiniMax M2.1 GS32",
代码： reasoning: false,
代码： input: ["text"],
代码： cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码： contextWindow: 196608,
代码： maxTokens: 8192,
代码： },
代码： ],
代码： },
代码： },
代码： },
代码： }
```

通过 `openclaw configure` 配置

使用交互式配置向导设置 MiniMax，无需编辑 JSON：

- 运行 `openclaw configure`。
- 选择 `Model/auth`。
- 选择 `MiniMax M2.1`。
- 在提示时选择你的默认模型。

配置选项

- `models.providers.minimax.baseUrl`：推荐使用 `兼容`；可选用于 `OpenAI 兼容` 的负载。
- `models.providers.minimax.api`：推荐使用 `anthropic-messages`；`openai-completions` 可选用于 `OpenAI 兼容` 的负载。
- `models.providers.minimax.apiKey`：MiniMax API 密钥（`MINIMAX_API_KEY`）。
- `models.providers.minimax.models`：定义 `id`、`name`、`reasoning`、`contextWindow`、`maxTokens`、`cost`。
- `agents.defaults.models`：为你想要在允许列表中的模型设置别名。
- `models.mode`：如果你想将 MiniMax 与内置模型一起添加，保持 `merge`。

注意事项

- 模型引用格式为 `minimax/<model>`。
- 编程计划使用量 API：`
- 如果需要精确的成本跟踪，请更新 `models.json` 中的定价值。
- MiniMax 编程计划推荐链接（9 折优惠）：
- 参见 `/concepts/model-providers` 了解提供商规则。
- 使用 `openclaw models list` 和 `openclaw models set minimax/MiniMax-M2.1` 切换模型。

故障排除

"Unknown model: minimax/MiniMax-M2.1"

这通常意味着 MiniMax 提供商未配置（没有提供商条目，也没有找到 MiniMax 认证配置文件/环境变量密钥）。此检测的修复在 2026.1.12 中（撰写本文时尚未发布）。修复方法：

- 升级到 2026.1.12（或从源码 `main` 分支运行），然后重启 Gateway 网关。

- 运行 `openclaw configure` 并选择 MiniMax M2.1，或
- 手动添加 `models.providers.minimax` 块，或
- 设置 `MINIMAX_API_KEY`（或 MiniMax 认证配置文件）以便注入提供商。

确保模型 id 区分大小写：

- `minimax/MiniMax-M2.1`
- `minimax/MiniMax-M2.1-lightning`

然后重新检查：

代码：`openclaw models list`

9. 模型提供商

模型提供商

OpenClaw 可以使用许多 LLM 提供商。选择一个，进行认证，然后将默认模型设置为 `provider/model`。

推荐：Venice（Venice AI）

Venice 是我们推荐的 Venice AI 设置，用于隐私优先的推理，并可选择使用 Opus 处理最困难的任务。

- 默认：`venice/llama-3.3-70b`
- 最佳综合：`venice/claude-opus-45`（Opus 仍然是最强的）

参见 Venice AI。

快速开始（两个步骤）

- 与提供商认证（通常通过 `openclaw onboard`）。
- 设置默认模型：

代码：{

代码：`agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },`

代码：}

支持的提供商（入门集）

- OpenAI (API + Codex)
- Anthropic (API + Claude Code CLI)
- OpenRouter
- Vercel AI Gateway
- Moonshot AI (Kimi + Kimi Coding)
- Synthetic
- OpenCode Zen
- Z.AI
- GLM 模型
- MiniMax
- Venice (Venice AI)
- Amazon Bedrock

有关完整的提供商目录（xAI、Groq、Mistral 等）和高级配置，请参阅模型提供商。

10. Moonshot AI (Kimi)

Moonshot AI (Kimi)

Moonshot 提供兼容 OpenAI 端点的 Kimi API。配置提供商并将默认模型设置为 moonshot/kimi-k2.5，或使用 Kimi Coding 的 kimi-coding/k2p5。

当前 Kimi K2 模型 ID：

```
{/_ moonshot-kimi-k2-ids:start _/}
```

- kimi-k2.5
- kimi-k2-0905-preview
- kimi-k2-turbo-preview
- kimi-k2-thinking
- kimi-k2-thinking-turbo

```
{/_ moonshot-kimi-k2-ids:end _/}
```

代码：openclaw onboard --auth-choice moonshot-api-key

Kimi Coding：

代码：openclaw onboard --auth-choice kimi-code-api-key

注意：Moonshot 和 Kimi Coding

是独立的提供商。密钥不可互换，端点不同，模型引用也不同（Moonshot 使用 moonshot/...，Kimi Coding 使用 kimi-coding/...）。

配置片段（Moonshot API）

代码：{

代码：env: { MOONSHOT_API_KEY: "sk-..." },

代码：agents: {

代码：defaults: {

代码：model: { primary: "moonshot/kimi-k2.5" },

代码：models: {

代码：// moonshot-kimi-k2-aliases:start

代码："moonshot/kimi-k2.5": { alias: "Kimi K2.5" },

代码："moonshot/kimi-k2-0905-preview": { alias: "Kimi K2" },

代码："moonshot/kimi-k2-turbo-preview": { alias: "Kimi K2 Turbo" },

代码："moonshot/kimi-k2-thinking": { alias: "Kimi K2 Thinking" },

代码："moonshot/kimi-k2-thinking-turbo": { alias: "Kimi K2 Thinking Turbo" },

代码：// moonshot-kimi-k2-aliases:end

代码：},

代码：},

代码：},

代码：models: {

代码：mode: "merge",

代码：providers: {

代码：moonshot: {

代码：baseUrl: "https://api.moonshot.ai/v1",

代码：apiKey: "\${MOONSHOT_API_KEY}",

代码：api: "openai-completions",

代码：models: [

代码：// moonshot-kimi-k2-models:start

代码：{

代码 : id: "kimi-k2.5",
代码 : name: "Kimi K2.5",
代码 : reasoning: false,
代码 : input: ["text"],
代码 : cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码 : contextWindow: 256000,
代码 : maxTokens: 8192,
代码 : },
代码 : {
代码 : id: "kimi-k2-0905-preview",
代码 : name: "Kimi K2 0905 Preview",
代码 : reasoning: false,
代码 : input: ["text"],
代码 : cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码 : contextWindow: 256000,
代码 : maxTokens: 8192,
代码 : },
代码 : {
代码 : id: "kimi-k2-turbo-preview",
代码 : name: "Kimi K2 Turbo",
代码 : reasoning: false,
代码 : input: ["text"],
代码 : cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码 : contextWindow: 256000,
代码 : maxTokens: 8192,
代码 : },
代码 : {
代码 : id: "kimi-k2-thinking",
代码 : name: "Kimi K2 Thinking",
代码 : reasoning: true,

```
代码 : input: ["text"],
代码 : cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码 : contextWindow: 256000,
代码 : maxTokens: 8192,
代码 : },
代码 : {
代码 : id: "kimi-k2-thinking-turbo",
代码 : name: "Kimi K2 Thinking Turbo",
代码 : reasoning: true,
代码 : input: ["text"],
代码 : cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码 : contextWindow: 256000,
代码 : maxTokens: 8192,
代码 : },
代码 : // moonshot-kimi-k2-models:end
代码 : ],
代码 : },
代码 : },
代码 : },
代码 : }
```

Kimi Coding

```
代码 : {
代码 : env: { KIMI_API_KEY: "sk-..." },
代码 : agents: {
代码 : defaults: {
代码 : model: { primary: "kimi-coding/k2p5" },
代码 : models: {
代码 : "kimi-coding/k2p5": { alias: "Kimi K2.5" },
代码 : },
代码 : },
```

代码：},

代码：}

注意事项

- Moonshot 模型引用使用 moonshot/<modelId>。Kimi Coding 模型引用使用 kimi-coding/<modelId>。
- 如有需要，可在 models.providers 中覆盖定价和上下文元数据。
- 如果 Moonshot 发布了某个模型的不同上下文限制，请相应调整 contextWindow。
- 如需使用中国端点，请使用`

11. Ollama

Ollama

Ollama 是一个本地 LLM 运行时，可以轻松在你的机器上运行开源模型。OpenClaw 通过 Ollama 的 OpenAI 兼容 API 进行集成，并且当你通过 OLLAMA_API_KEY（或认证配置）启用且未定义显式的 models.providers.ollama 条目时，可以自动发现支持工具调用的模型。

快速开始

- 安装 Ollama：
- 拉取模型：

代码：ollama pull llama3.3

代码：# 或

代码：ollama pull qwen2.5-coder:32b

代码：# 或

代码：ollama pull deepseek-r1:32b

- 为 OpenClaw 启用 Ollama（任意值即可；Ollama 不需要真实密钥）：

代码：# 设置环境变量

代码：# 或在配置文件中设置

代码：openclaw config set models.providers.ollama.apiKey "ollama-local"

- 使用 Ollama 模型：

```
代码：{  
代码： agents: {  
代码： defaults: {  
代码： model: { primary: "ollama/llama3.3" },  
代码： },  
代码： },  
代码： }
```

模型发现（隐式提供商）

当你设置了 OLLAMA_API_KEY（或认证配置）且未定义 `models.providers.ollama` 时，OpenClaw 会从本地 Ollama 实例发现模型：

- 查询 `/api/tags` 和 `/api/show`
- 仅保留报告了 tools 能力的模型
- 当模型报告 thinking 时标记为 reasoning
- 在可用时从 `model_info["<arch>.context_length"]` 读取 `contextWindow`
- 将 `maxTokens` 设置为上下文窗口的 10 倍
- 所有费用设置为 0

这样无需手动配置模型条目，同时保持目录与 Ollama 的能力对齐。

查看可用模型：

```
代码：ollama list
```

```
代码：openclaw models list
```

要添加新模型，只需通过 Ollama 拉取：

```
代码：ollama pull mistral
```

新模型将被自动发现并可供使用。

如果你显式设置了

`models.providers.ollama`，自动发现将被跳过，你必须手动定义模型（见下文）。

配置

基本设置（隐式发现）

启用 Ollama 最简单的方式是通过环境变量：

显式设置（手动模型）

在以下情况使用显式配置：

- Ollama 运行在其他主机/端口上。
- 你想强制指定上下文窗口或模型列表。
- 你想包含未报告工具支持的模型。

代码：{

代码： models: {

代码： providers: {

代码： ollama: {

代码： // 使用包含 /v1 的主机地址以兼容 OpenAI API

代码： baseUrl: "http://ollama-host:11434/v1",

代码： apiKey: "ollama-local",

代码： api: "openai-completions",

代码： models: [

代码： {

代码： id: "llama3.3",

代码： name: "Llama 3.3",

代码： reasoning: false,

代码： input: ["text"],

代码： cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },

代码： contextWindow: 8192,

代码： maxTokens: 8192 * 10

代码： }

代码：]

代码： }

代码： }

代码： }

代码：}

如果设置了 OLLAMA_API_KEY，你可以在提供商条目中省略 apiKey，OpenClaw 会自动填充以进行可用性检查。

自定义基础 URL（显式配置）

如果 Ollama 运行在不同的主机或端口上（显式配置会禁用自动发现，因此需要手动定义模型）：

代码：{

代码：models: {

代码：providers: {

代码：ollama: {

代码：apiKey: "ollama-local",

代码：baseUrl: "http://ollama-host:11434/v1",

代码：},

代码：},

代码：},

代码：}

模型选择

配置完成后，所有 Ollama 模型即可使用：

代码：{

代码：agents: {

代码：defaults: {

代码：model: {

代码：primary: "ollama/llama3.3",

代码：fallbacks: ["ollama/qwen2.5-coder:32b"],

代码：},

代码：},

代码：},

代码：}

高级用法

推理模型

当 Ollama 在 `/api/show` 中报告 `thinking` 时，OpenClaw 会将模型标记为具有推理能力：

代码：`ollama pull deepseek-r1:32b`

模型费用

Ollama 免费且在本地运行，因此所有模型费用均设置为 \$0。

上下文窗口

对于自动发现的模型，OpenClaw 会使用 Ollama 报告的上下文窗口（如果可用），否则默认为 8192。你可以在显式提供商配置中覆盖 `contextWindow` 和 `maxTokens`。

故障排除

Ollama 未被检测到

确保 Ollama 正在运行，且你已设置 `OLLAMA_API_KEY`（或认证配置），并且未定义显式的 `models.providers.ollama` 条目：

代码：`ollama serve`

同时确认 API 可访问：

代码：`curl http://localhost:11434/api/tags`

没有可用模型

OpenClaw 仅自动发现报告了工具支持的模型。如果你的模型未列出，可以：

- 拉取一个支持工具调用的模型，或
- 在 `models.providers.ollama` 中显式定义该模型。

添加模型：

代码：`ollama list` # 查看已安装的模型

代码：`ollama pull llama3.3` # 拉取模型

连接被拒绝

检查 Ollama 是否在正确的端口上运行：

代码：# 检查 Ollama 是否在运行

代码：ps aux | grep ollama

代码：# 或重启 Ollama

代码：ollama serve

另请参阅

- 模型提供商 - 所有提供商概览
- 模型选择 - 如何选择模型
- 配置 - 完整配置参考

12. OpenAI

OpenAI

OpenAI 提供 GPT 模型的开发者 API。Codex 支持 ChatGPT 登录进行订阅访问，或 API 密钥登录进行按量计费访问。Codex 云端需要 ChatGPT 登录。

方式 A：OpenAI API 密钥（OpenAI Platform）

适用于：直接 API 访问和按量计费。

从 OpenAI 控制台获取你的 API 密钥。

CLI 设置

代码：openclaw onboard --auth-choice openai-api-key

代码：# 或非交互式

代码：openclaw onboard --openai-api-key "\$OPENAI_API_KEY"

配置片段

代码：{

代码：env: { OPENAI_API_KEY: "sk-...",

代码：agents: { defaults: { model: { primary: "openai/gpt-5.2" } } },

代码：}

方式 B：OpenAI Code（Codex）订阅

适用于：使用 ChatGPT/Codex 订阅访问而非 API 密钥。

Codex 云端需要 ChatGPT 登录，而 Codex CLI 支持 ChatGPT 或 API 密钥登录。

CLI 设置

代码：# 在向导中运行 Codex OAuth

代码：openclaw onboard --auth-choice openai-codex

代码：# 或直接运行 OAuth

代码：openclaw models auth login --provider openai-codex

配置片段

代码：{

代码：agents: { defaults: { model: { primary: "openai-codex/gpt-5.2" } } },

代码：}

注意事项

- 模型引用始终使用 provider/model 格式（参见 /concepts/models）。
- 认证详情和复用规则请参阅 /concepts/oauth。

13. OpenCode Zen

OpenCode Zen

OpenCode Zen 是由 OpenCode

团队推荐的一组精选模型列表，适用于编程智能体。它是一个可选的托管模型访问路径，使用 API 密钥和 opencode 提供商。Zen 目前处于测试阶段。

CLI 设置

代码：openclaw onboard --auth-choice opencode-zen

代码：# 或非交互式

代码：openclaw onboard --opencode-zen-api-key "\$OPENCODE_API_KEY"

配置片段

代码：{

代码： env: { OPENCODE_API_KEY: "sk-..." },

代码： agents: { defaults: { model: { primary: "opencode/claude-opus-4-5" } } },

代码：}

注意事项

- 也支持 OPENCODE_ZEN_API_KEY。
- 你需要登录 Zen，添加账单信息，然后复制你的 API 密钥。
- OpenCode Zen 按请求计费；详情请查看 OpenCode 控制台。

14. OpenRouter

OpenRouter

OpenRouter 提供了一个统一 API，通过单一端点和 API 密钥将请求路由到多种模型。它兼容 OpenAI，因此大多数 OpenAI SDK 只需切换 base URL 即可使用。

CLI 设置

代码： openclaw onboard --auth-choice apiKey --token-provider openrouter --token "\$OPENROUTER_API_KEY"

配置片段

代码：{

代码： env: { OPENROUTER_API_KEY: "sk-or-..." },

代码： agents: {

代码： defaults: {

代码： model: { primary: "openrouter/anthropic/claude-sonnet-4-5" },

代码： },

代码： },

代码：}

注意事项

- 模型引用格式为 openrouter/<provider>/<model>。
- 更多模型/提供商选项，请参阅模型提供商。

- OpenRouter 底层使用 Bearer 令牌和你的 API 密钥进行认证。

15. 千帆 (Qianfan)

千帆 (Qianfan)

该页面是英文文档的中文占位版本，完整内容请先参考英文版：Qianfan。

16. Qwen

Qwen

Qwen 为 Qwen Coder 和 Qwen Vision 模型提供免费层 OAuth 流程（每天 2,000 次请求，受 Qwen 速率限制约束）。

启用插件

代码：openclaw plugins enable qwen-portal-auth

启用后重启 Gateway 网关。

认证

代码：openclaw models auth login --provider qwen-portal --set-default

这会运行 Qwen 设备码 OAuth 流程并将提供商条目写入你的 models.json（加上一个 qwen 别名以便快速切换）。

模型 ID

- qwen-portal/coder-model
- qwen-portal/vision-model

切换模型：

代码：openclaw models set qwen-portal/coder-model

复用 Qwen Code CLI 登录

如果你已经使用 Qwen Code CLI 登录，OpenClaw 会在加载认证存储时从 `~/.qwen/oauth_creds.json` 同步凭证。你仍然需要一个 `models.providers.qwen-portal` 条目（使用上面的登录命令创建一个）。

注意

- 令牌自动刷新；如果刷新失败或访问被撤销，请重新运行登录命令。
- 默认基础 URL：Qwen 提供不同的端点，使用 `models.providers.qwen-portal.baseUrl`` 覆盖）。
- 参阅模型提供商了解提供商级别的规则。

17. Synthetic

Synthetic

Synthetic 提供兼容 Anthropic 的端点。OpenClaw 将其注册为 synthetic 提供商，并使用 Anthropic Messages API。

快速设置

- 设置 `SYNTHETIC_API_KEY`（或运行以下向导）。
- 运行新手引导：

代码： `openclaw onboard --auth-choice synthetic-api-key`

默认模型设置为：

代码： `synthetic/hf:MiniMaxAI/MiniMax-M2.1`

配置示例

代码：{

代码： `env: { SYNTHETIC_API_KEY: "sk-..." },`

代码： `agents: {`

代码： `defaults: {`

代码： `model: { primary: "synthetic/hf:MiniMaxAI/MiniMax-M2.1" },`

代码： `models: { "synthetic/hf:MiniMaxAI/MiniMax-M2.1": { alias: "MiniMax M2.1" } },`

代码： `},`

代码： `},`

代码： `models: {`

代码： `mode: "merge",`

代码： `providers: {`

```
代码 : synthetic: {
代码 : baseUrl: "https://api.synthetic.new/anthropic",
代码 : apiKey: "${SYNTHETIC_API_KEY}",
代码 : api: "anthropic-messages",
代码 : models: [
代码 : {
代码 : id: "hf:MiniMaxAI/MiniMax-M2.1",
代码 : name: "MiniMax M2.1",
代码 : reasoning: false,
代码 : input: ["text"],
代码 : cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码 : contextWindow: 192000,
代码 : maxTokens: 65536,
代码 : },
代码 : ],
代码 : },
代码 : },
代码 : },
代码 : }
```

注意：OpenClaw 的 Anthropic 客户端会自动在 base URL 后追加 /v1，因此请使用 /anthropic/v1）。如果 Synthetic 更改了其 base URL，请覆盖 models.providers.synthetic.baseUrl`。

模型目录

以下所有模型的费用均为 0（输入/输出/缓存）。

模型 ID	上下文窗口	最大令牌数	推理	输入
hf:MiniMaxAI/MiniMax-M2.1	192000	65536	false	text
hf:moonshotai/Kimi-K2-Thinking	256000	8192	true	text
hf:zai-org/GLM-4.7	198000	128000	false	text

hf:deepseek-ai/DeepSeek-R1-0528	128000	8192	false	text
hf:deepseek-ai/DeepSeek-V3-0324	128000	8192	false	text
hf:deepseek-ai/DeepSeek-V3.1	128000	8192	false	text
hf:deepseek-ai/DeepSeek-V3.1-Terminus	128000	8192	false	text
hf:deepseek-ai/DeepSeek-V3.2	159000	8192	false	text
hf:meta-llama/Llama-3.3-70B-Instruct	128000	8192	false	text
hf:meta-llama/Llama-4-Maverick-17B-128E-Instruct-FP8	524000	8192	false	text
hf:moonshotai/Kimi-K2-Instruct-0905	256000	8192	false	text
hf:openai/gpt-oss-120b	128000	8192	false	text
hf:Qwen/Qwen3-235B-A22B-Instruct-2507	256000	8192	false	text
hf:Qwen/Qwen3-Coder-480B-A35B-Instruct	256000	8192	false	text
hf:Qwen/Qwen3-VL-235B-A22B-Instruct	250000	8192	false	text + image
hf:zai-org/GLM-4.5	128000	128000	false	text
hf:zai-org/GLM-4.6	198000	128000	false	text
hf:deepseek-ai/DeepSeek-V3	128000	8192	false	text
hf:Qwen/Qwen3-235B-A22B-Thinking-2507	256000	8192	true	text

注意事项

- 模型引用格式为 `synthetic/<modelId>`。
- 如果启用了模型允许列表 (`agents.defaults.models`)，请添加你计划使用的所有模型。
- 参阅模型提供商了解提供商规则。

18. Venice AI (Venice 精选)

Venice AI (Venice 精选)

Venice 是我们精选的 Venice 隐私优先推理配置，支持可选的匿名化访问专有模型。

Venice AI 提供注重隐私的 AI 推理服务，支持无审查模型，并可通过其匿名代理访问主流专有模型。所有推理默认私密——不会用你的数据训练，不会记录日志。

为什么在 OpenClaw 中使用 Venice

- 私密推理，适用于开源模型（无日志记录）。
- 需要时可使用无审查模型。

- 在质量重要时，可匿名访问专有模型（Opus/GPT/Gemini）。
- 兼容 OpenAI 的 /v1 端点。

隐私模式

Venice 提供两种隐私级别——理解这一点是选择模型的关键：

模式	描述	模型
私密	完全私密。提示词/回复从不存储或记录。临时性处理。	Llama、Qwen、DeepSeek、Venice Uncensored 等
匿名化	通过 Venice 代理转发并剥离元数据。底层提供商（OpenAI、Anthropic）收到的是匿名化请求。	Claude、GPT、Gemini、Grok、Kimi、MiniMax

功能特性

- 注重隐私：可选择"私密"（完全私密）和"匿名化"（代理转发）模式
- 无审查模型：访问无内容限制的模型
- 主流模型访问：通过 Venice 匿名代理使用 Claude、GPT-5.2、Gemini、Grok
- 兼容 OpenAI API：标准 /v1 端点，易于集成
- 流式输出：所有模型均支持
- 函数调用：部分模型支持（请检查模型能力）
- 视觉：具有视觉能力的模型支持
- 无硬性速率限制：极端使用情况下可能触发公平使用限流

设置

1. 获取 API 密钥

- 在 [venice.ai](#) 注册
- 前往 Settings → API Keys → Create new key
- 复制你的 API 密钥（格式：vapi_xxxxxxxxxxxxx）

2. 配置 OpenClaw

方案 A：环境变量

方案 B：交互式设置（推荐）

代码：openclaw onboard --auth-choice venice-api-key

这将：

- 提示输入你的 API 密钥（或使用已有的 VENICE_API_KEY）
- 显示所有可用的 Venice 模型
- 让你选择默认模型
- 自动配置提供商

方案 C：非交互式

代码：openclaw onboard --non-interactive \

代码：--auth-choice venice-api-key \

代码：--venice-api-key "vapi_xxxxxxxxxxx"

3. 验证设置

代码：openclaw chat --model venice/llama-3.3-70b "Hello, are you working?"

模型选择

设置完成后，OpenClaw 会显示所有可用的 Venice 模型。根据你的需求选择：

- 默认（我们的推荐）：venice/llama-3.3-70b，私密且性能均衡。
- 最佳整体质量：venice/claude-opus-45，适合复杂任务（Opus 仍然是最强的）。
- 隐私：选择"私密"模型以获得完全私密的推理。
- 能力：选择"匿名化"模型以通过 Venice 代理访问 Claude、GPT、Gemini。

随时更改默认模型：

代码：openclaw models set venice/claude-opus-45

代码：openclaw models set venice/llama-3.3-70b

列出所有可用模型：

代码：openclaw models list | grep venice

通过`openclaw configure` 配置

- 运行 openclaw configure
- 选择 Model/auth
- 选择 Venice AI

应该使用哪个模型？

使用场景	推荐模型	原因
通用对话	llama-3.3-70b	综合表现好，完全私密
最佳整体质量	claude-opus-45	Opus 在复杂任务上仍然最强
隐私 + Claude 品质	claude-opus-45	通过匿名代理获得最佳推理能力
编程	qwen3-coder-480b-a35b-instruct	代码优化，262k 上下文
视觉任务	qwen3-vl-235b-a22b	最佳私密视觉模型
无审查	venice-uncensored	无内容限制
快速 + 低成本	qwen3-4b	轻量级，仍有不错能力
复杂推理	deepseek-v3.2	推理能力强，私密

可用模型（共 25 个）

私密模型（15 个） — 完全私密，无日志记录

模型 ID	名称	上下文 (token)	特性
llama-3.3-70b	Llama 3.3 70B	131k	通用
llama-3.2-3b	Llama 3.2 3B	131k	快速，轻量
hermes-3-llama-3.1-405b	Hermes 3 Llama 3.1 405B	131k	复杂任务
qwen3-235b-a22b-thinking-2507	Qwen3 235B Thinking	131k	推理
qwen3-235b-a22b-instruct-2507	Qwen3 235B Instruct	131k	通用
qwen3-coder-480b-a35b-instruct	Qwen3 Coder 480B	262k	编程
qwen3-next-80b	Qwen3 Next 80B	262k	通用
qwen3-vl-235b-a22b	Qwen3 VL 235B	262k	视觉
qwen3-4b	Venice Small (Qwen3 4B)	32k	快速，推理

	deepseek-v3.2		DeepSeek V3.2		163k		推理	
	venice-uncensored		Venice Uncensored		32k		无审查	
	mistral-31-24b		Venice Medium (Mistral)		131k		视觉	
	google-gemma-3-27b-it		Gemma 3 27B Instruct		202k		视觉	
	openai-gpt-oss-120b		OpenAI GPT OSS 120B		131k		通用	
	zai-org-glm-4.7		GLM 4.7		202k		推理，多语言	

匿名化模型（10 个） — 通过 Venice 代理

	模型 ID		原始模型		上下文 (token)		特性	
	-----		-----		-----		-----	
	claude-opus-45		Claude Opus 4.5		202k		推理，视觉	
	claude-sonnet-45		Claude Sonnet 4.5		202k		推理，视觉	
	openai-gpt-52		GPT-5.2		262k		推理	
	openai-gpt-52-codex		GPT-5.2 Codex		262k		推理，视觉	
	gemini-3-pro-preview		Gemini 3 Pro		202k		推理，视觉	
	gemini-3-flash-preview		Gemini 3 Flash		262k		推理，视觉	
	grok-41-fast		Grok 4.1 Fast		262k		推理，视觉	
	grok-code-fast-1		Grok Code Fast 1		262k		推理，编程	
	kimi-k2-thinking		Kimi K2 Thinking		262k		推理	
	minimax-m21		MiniMax M2.1		202k		推理	

模型发现

当设置了 VENICE_API_KEY 时，OpenClaw 会自动从 Venice API 发现模型。如果 API 不可达，则回退到静态目录。

/models 端点是公开的（列出模型无需认证），但推理需要有效的 API 密钥。

流式输出与工具支持

	功能		支持情况	
	-----		-----	
	流式输出		所有模型	
	函数调用		大多数模型（请检查 API 中的 supportsFunctionCalling）	

- | 视觉/图像 | 标记为"视觉"特性的模型 |
- | JSON 模式 | 通过 response_format 支持 |

定价

Venice 使用积分制。请查看 venice.ai/pricing 了解当前费率：

- 私密模型：通常成本较低
- 匿名化模型：与直接 API 定价相近 + 少量 Venice 费用

对比：Venice 与直接 API

方面	Venice（匿名化）	直接 API
-----	-----	-----
隐私	剥离元数据，匿名化	关联你的账户
延迟	+10-50ms（代理）	直连
功能	支持大部分功能	完整功能
计费	Venice 积分	提供商计费

使用示例

```
代码：# 使用默认私密模型
代码：openclaw chat --model venice/llama-3.3-70b

代码：# 通过 Venice 使用 Claude（匿名化）
代码：openclaw chat --model venice/claude-opus-45

代码：# 使用无审查模型
代码：openclaw chat --model venice/venice-uncensored

代码：# 使用视觉模型处理图像
代码：openclaw chat --model venice/qwen3-vl-235b-a22b

代码：# 使用编程模型
代码：openclaw chat --model venice/qwen3-coder-480b-a35b-instruct
```

故障排除

API 密钥无法识别

代码：echo \$VENICE_API_KEY

代码：openclaw models list | grep venice

确保密钥以 vapi_ 开头。

模型不可用

Venice 模型目录会动态更新。运行 openclaw models list 查看当前可用的模型。部分模型可能暂时离线。

连接问题

Venice API 地址为 `HTTPS` 连接。

配置文件示例

代码：{

代码：env: { VENICE_API_KEY: "vapi_..." },

代码：agents: { defaults: { model: { primary: "venice/llama-3.3-70b" } } },

代码：models: {

代码：mode: "merge",

代码：providers: {

代码：venice: {

代码：baseUrl: "https://api.venice.ai/api/v1",

代码：apiKey: "\${VENICE_API_KEY}",

代码：api: "openai-completions",

代码：models: [

代码：{

代码：id: "llama-3.3-70b",

代码：name: "Llama 3.3 70B",

代码：reasoning: false,

代码：input: ["text"],

代码：cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },

代码：contextWindow: 131072,

代码：maxTokens: 8192,

代码： },

代码：],

代码： },

代码： },

代码： },

代码： }

链接

- Venice AI
- API 文档
- 定价
- 状态页

19. Vercel AI Gateway

Vercel AI Gateway

Vercel AI Gateway 提供了一个统一的 API，通过单一端点访问数百个模型。

- 提供商：vercel-ai-gateway
- 认证：AI_GATEWAY_API_KEY
- API：兼容 Anthropic Messages

快速开始

- 设置 API 密钥（推荐：为 Gateway 网关存储它）：

代码： `openclaw onboard --auth-choice ai-gateway-api-key`

- 设置默认模型：

代码： {

代码： agents: {

代码： defaults: {

代码： model: { primary: "vercel-ai-gateway/anthropic/claude-opus-4.5" },

代码： },

代码： },

代码： }

非交互式示例

代码： `openclaw onboard --non-interactive \`

代码： `--mode local \`

代码： `--auth-choice ai-gateway-api-key \`

代码： `--ai-gateway-api-key "$AI_GATEWAY_API_KEY"`

环境变量说明

如果 Gateway 网关作为守护进程运行（`launchd/systemd`），请确保 `AI_GATEWAY_API_KEY` 对该进程可用（例如，在 `~/.openclaw/.env` 中或通过 `env.shellEnv`）。

20. Xiaomi MiMo

Xiaomi MiMo

Xiaomi MiMo 是 MiMo 模型的 API 平台。它提供与 OpenAI 和 Anthropic 格式兼容的 REST API，并使用 API 密钥进行身份验证。请在 [Xiaomi MiMo 控制台](#) 中创建你的 API 密钥。OpenClaw 使用 xiaomi 提供商配合 Xiaomi MiMo API 密钥。

模型概览

- `mimo-v2-flash`：262144 token 上下文窗口，兼容 Anthropic Messages API。
- 基础 URL：`
- 授权方式：Bearer `$XIAOMI_API_KEY`

CLI 设置

代码： `openclaw onboard --auth-choice xiaomi-api-key`

代码： `# 或非交互式`

代码： `openclaw onboard --auth-choice xiaomi-api-key --xiaomi-api-key "$XIAOMI_API_KEY"`

配置片段


```
代码：{
代码： env: { XIAOMI_API_KEY: "your-key" },
代码： agents: { defaults: { model: { primary: "xiaomi/mimo-v2-flash" } } },
代码： models: {
代码： mode: "merge",
代码： providers: {
代码： xiaomi: {
代码： baseUrl: "https://api.xiaomimimo.com/anthropic",
代码： api: "anthropic-messages",
代码： apiKey: "XIAOMI_API_KEY",
代码： models: [
代码： {
代码： id: "mimo-v2-flash",
代码： name: "Xiaomi MiMo V2 Flash",
代码： reasoning: false,
代码： input: ["text"],
代码： cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
代码： contextWindow: 262144,
代码： maxTokens: 8192,
代码： },
代码： ],
代码： },
代码： },
代码： },
代码： },
代码： }
```

备注

- 模型引用：xiaomi/mimo-v2-flash。
- 当设置了 XIAOMI_API_KEY（或存在身份验证配置文件）时，该提供商会自动注入。
- 有关提供商规则，请参阅 /concepts/model-providers。

21. Z.AI

Z.AI

Z.AI 是 GLM 模型的 API 平台。它为 GLM 提供 REST API，并使用 API 密钥进行身份验证。请在 Z.AI 控制台中创建你的 API 密钥。OpenClaw 通过 zai 提供商配合 Z.AI API 密钥使用。

CLI 设置

代码：openclaw onboard --auth-choice zai-api-key

代码：# 或非交互式

代码：openclaw onboard --zai-api-key "\$ZAI_API_KEY"

配置片段

代码：{

代码：env: { ZAI_API_KEY: "sk-..." },

代码：agents: { defaults: { model: { primary: "zai/glm-4.7" } } },

代码：}

注意事项

- GLM 模型以 zai/<model> 的形式提供（例如：zai/glm-4.7）。
- 参阅 /providers/glm 了解模型系列概览。
- Z.AI 使用 Bearer 认证方式配合你的 API 密钥。

第五章：工具体系与调用

1. `openclaw agent`（直接智能体运行）

`openclaw agent`（直接智能体运行）

openclaw agent 运行单个智能体回合，无需入站聊天消息。

默认情况下它通过 Gateway 网关运行；添加 --local 以强制在当前机器上使用嵌入式运行时。

行为

- 必需：--message <text>
- 会话选择：
 - --to <dest> 派生会话键（群组/频道目标保持隔离；直接聊天折叠到 main），或
 - --session-id <id> 通过 ID 重用现有会话，或
 - --agent <id> 直接定位已配置的智能体（使用该智能体的 main 会话键）
- 运行与正常入站回复相同的嵌入式智能体运行时。
- 思考/详细标志持久化到会话存储中。
- 输出：
 - 默认：打印回复文本（加上 MEDIA:<url> 行）
 - --json：打印结构化负载 + 元数据
- 可选使用 --deliver + --channel 将回复投递回渠道（目标格式与 openclaw message --target 匹配）。
- 使用 --reply-channel/--reply-to/--reply-account 覆盖投递而不更改会话。

如果 Gateway 网关不可达，CLI 会回退到嵌入式本地运行。

示例

代码：openclaw agent --to +15555550123 --message "status update"

代码：openclaw agent --agent ops --message "Summarize logs"

代码：openclaw agent --session-id 1234 --message "Summarize inbox" --thinking medium

代码：openclaw agent --to +15555550123 --message "Trace logs" --verbose on --json

代码：openclaw agent --to +15555550123 --message "Summon reply" --deliver

代码：openclaw agent --agent ops --message "Generate report" --deliver --reply-channel slack --reply-to "#reports"

标志

- --local : 本地运行 (需要你的 shell 中有模型提供商 API 密钥)
- --deliver : 将回复发送到所选渠道
- --channel : 投递渠道 (whatsapp|telegram|discord|googlechat|slack|signal|imessage, 默认: whatsapp)
- --reply-to : 投递目标覆盖
- --reply-channel : 投递渠道覆盖
- --reply-account : 投递账户 ID 覆盖
- --thinking <off|minimal|low|medium|high|xhigh> : 持久化思考级别 (仅限 GPT-5.2 + Codex 模型)
- --verbose <on|full|off> : 持久化详细级别
- --timeout <seconds> : 覆盖智能体超时
- --json : 输出结构化 JSON

2. apply_patch 工具

apply_patch 工具

使用结构化补丁格式应用文件更改。这非常适合多文件或
多段编辑, 在这些场景下单次 edit 调用会很脆弱。

该工具接受一个 input 字符串, 其中包含一个或多个文件操作:

代码: *** Begin Patch

代码: *** Add File: path/to/file.txt

代码: +line 1

代码: +line 2

代码: *** Update File: src/app.ts

代码: @@

代码: -old line

代码: +new line

代码: *** Delete File: obsolete.txt

代码: *** End Patch

参数

- input（必需）：完整的补丁内容，包括 Begin Patch 和 End Patch。

说明

- 路径相对于工作区根目录解析。
- 在 Update File: 段中使用 Move to: 可重命名文件。
- 需要时使用 End of File 标记仅在文件末尾的插入。
- 实验性功能，默认禁用。通过 tools.exec.applyPatch.enabled 启用。
- 仅限 OpenAI（包括 OpenAI Codex）。可选通过 tools.exec.applyPatch.allowModels 按模型进行限制。
- 配置仅在 tools.exec 下。

示例

代码：{

代码："tool": "apply_patch",

代码："input": "**** Begin Patch\n**** Update File: src/index.ts\n@@\n-const foo = 1\n+const foo = 2\n**** End Patch"

代码：}

3. 浏览器故障排除（Linux）

浏览器故障排除（Linux）

问题："Failed to start Chrome CDP on port 18800"

OpenClaw 的浏览器控制服务器无法启动 Chrome/Brave/Edge/Chromium，出现以下错误：

代码：{"error": "Error: Failed to start Chrome CDP on port 18800 for profile \"openclaw\"."}

根本原因

在 Ubuntu（和许多 Linux 发行版）上，默认的 Chromium 安装是 snap 包。Snap 的 AppArmor 限制会干扰 OpenClaw 启动和监控浏览器进程的方式。

apt install chromium 命令安装的是一个重定向到 snap 的存根包：

代码：Note, selecting 'chromium-browser' instead of 'chromium'

代码：chromium-browser is already the newest version (2:1snap1-0ubuntu2).

这不是真正的浏览器——它只是一个包装器。

解决方案 1：安装 Google Chrome（推荐）

安装官方 Google Chrome .deb 包，它不受 snap 沙箱限制：

代码：wget

https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb

代码：sudo dpkg -i google-chrome-stable_current_amd64.deb

代码：sudo apt --fix-broken install -y # if there are dependency errors

然后更新你的 OpenClaw 配置（~/.openclaw/openclaw.json）：

代码：{

代码："browser": {

代码："enabled": true,

代码："executablePath": "/usr/bin/google-chrome-stable",

代码："headless": true,

代码："noSandbox": true

代码：}

代码：}

解决方案 2：使用 Snap Chromium 的仅附加模式

如果你必须使用 snap Chromium，配置 OpenClaw 附加到手动启动的浏览器：

- 更新配置：

代码：{

代码："browser": {

代码："enabled": true,

代码："attachOnly": true,

代码："headless": true,

代码："noSandbox": true

代码：}

代码：}

- 手动启动 Chromium：

代码：chromium-browser --headless --no-sandbox --disable-gpu \

代码：--remote-debugging-port=18800 \

代码：--user-data-dir=\$HOME/.openclaw/browser/openclaw/user-data \

代码：about:blank &

- 可选创建 systemd 用户服务以自动启动 Chrome：

代码：# ~/.config/systemd/user/openclaw-browser.service

代码：[Unit]

代码：Description=OpenClaw Browser (Chrome CDP)

代码：After=network.target

代码：[Service]

代码：ExecStart=/snap/bin/chromium --headless --no-sandbox --disable-gpu

--remote-debugging-port=18800

--user-data-dir=%h/.openclaw/browser/openclaw/user-data about:blank

代码：Restart=on-failure

代码：RestartSec=5

代码：[Install]

代码：WantedBy=default.target

启用：systemctl --user enable --now openclaw-browser.service

验证浏览器是否工作

检查状态：

代码：curl -s http://127.0.0.1:18791/ | jq '{running, pid, chosenBrowser}'

测试浏览：

代码：curl -s -X POST http://127.0.0.1:18791/start

代码：curl -s http://127.0.0.1:18791/tabs

配置参考

选项	描述	默认值
-----	-----	-----
browser.enabled	启用浏览器控制	true
browser.executablePath	Chromium 系浏览器二进制文件路径（Chrome/Brave/Edge/Chromium）	自动检测（当默认浏览器是 Chromium 系时优先使用）
browser.headless	无 GUI 运行	false
browser.noSandbox	添加 --no-sandbox 标志（某些 Linux 设置需要）	false
browser.attachOnly	不启动浏览器，仅附加到现有浏览器	false
browser.cdpPort	Chrome DevTools Protocol 端口	18800

问题："Chrome extension relay is running, but no tab is connected"

你正在使用 chrome 配置文件（扩展中继）。它期望 OpenClaw 浏览器扩展附加到一个活动标签页。

修复选项：

- 使用托管浏览器：openclaw browser start --browser-profile openclaw（或设置 browser.defaultProfile: "openclaw"）。
- 使用扩展中继：安装扩展，打开一个标签页，然后点击 OpenClaw 扩展图标来附加它。

注意事项：

- chrome 配置文件在可能时使用你的系统默认 Chromium 浏览器。
- 本地 openclaw 配置文件自动分配 cdpPort/cdpUrl；仅为远程 CDP 设置这些。

4. 浏览器登录 + X/Twitter 发帖

浏览器登录 + X/Twitter 发帖

手动登录（推荐）

当网站需要登录时，请在主机浏览器配置文件（openclaw 浏览器）中手动登录。

不要将你的凭证提供给模型。自动登录通常会触发反机器人防御并可能锁定账户。

返回主浏览器文档：浏览器。

使用哪个 Chrome 配置文件？

OpenClaw 控制一个专用的 Chrome 配置文件（名为 openclaw，橙色调 UI）。这与你的日常浏览器配置文件是分开的。

两种简单的访问方式：

- 让智能体打开浏览器，然后你自己登录。
- 通过 CLI 打开：

代码：openclaw browser start

代码：openclaw browser open https://x.com

如果你有多个配置文件，传入 --browser-profile <name>（默认是 openclaw）。

X/Twitter：推荐流程

- 阅读/搜索/话题：使用 bird CLI Skills（无浏览器，稳定）。
- 仓库：
- 发布更新：使用主机浏览器（手动登录）。

沙箱隔离 + 主机浏览器访问

沙箱隔离的浏览器会话更容易触发机器人检测。对于 X/Twitter（和其他严格的网站），优先使用主机浏览器。

如果智能体在沙箱中，浏览器工具默认使用沙箱。要允许主机控制：

代码：{

代码：agents: {

代码：defaults: {

代码：sandbox: {

代码：mode: "non-main",

代码：browser: {

代码：allowHostControl: true,

代码：},

代码：},

代码：},

代码：},

代码：}

然后定位主机浏览器：

代码：openclaw browser open https://x.com --browser-profile openclaw --target host

或者为发布更新的智能体禁用沙箱隔离。

5. 浏览器（openclaw 托管）

浏览器（openclaw 托管）

OpenClaw 可以运行一个由智能体控制的专用 Chrome/Brave/Edge/Chromium 配置文件。

它与你的个人浏览器隔离，通过 Gateway 网关内部的小型本地控制服务进行管理（仅限 loopback）。

新手视角：

- 把它想象成一个独立的、仅供智能体使用的浏览器。
- openclaw 配置文件不会触及你的个人浏览器配置文件。
- 智能体可以在安全的通道中打开标签页、读取页面、点击和输入。
- 默认的 chrome 配置文件通过扩展中继使用系统默认的 Chromium 浏览器；切换到 openclaw 可使用隔离的托管浏览器。

功能概览

- 一个名为 openclaw 的独立浏览器配置文件（默认橙色主题）。
- 确定性标签页控制（列出/打开/聚焦/关闭）。
- 智能体操作（点击/输入/拖动/选择）、快照、截图、PDF。
- 可选的多配置文件支持（openclaw、work、remote 等）。

此浏览器不是你的日常浏览器。它是一个安全、隔离的界面，用于智能体自动化和验证。

快速开始

代码：openclaw browser --browser-profile openclaw status

代码：openclaw browser --browser-profile openclaw start

代码：openclaw browser --browser-profile openclaw open https://example.com

代码：openclaw browser --browser-profile openclaw snapshot

如果出现"Browser disabled"，请在配置中启用它（见下文）并重启 Gateway 网关。

配置文件：`openclaw` 与 `chrome`

- openclaw：托管的隔离浏览器（无需扩展）。
- chrome：到你系统浏览器的扩展中继（需要将 OpenClaw 扩展附加到标签页）。

如果你希望默认使用托管模式，请设置 browser.defaultProfile: "openclaw"。

配置

浏览器设置位于 ~/.openclaw/openclaw.json。

代码：{

代码： browser: {

代码： enabled: true, // default: true

代码： // cdpUrl: "http://127.0.0.1:18792", // legacy single-profile override

代码： remoteCdpTimeoutMs: 1500, // remote CDP HTTP timeout (ms)

代码： remoteCdpHandshakeTimeoutMs: 3000, // remote CDP WebSocket handshake timeout (ms)

代码： defaultProfile: "chrome",

代码： color: "#FF4500",

代码： headless: false,

代码： noSandbox: false,

代码： attachOnly: false,

代码： executablePath: "/Applications/Brave Browser.app/Contents/MacOS/Brave Browser",

代码： profiles: {

代码： openclaw: { cdpPort: 18800, color: "#FF4500" },

代码： work: { cdpPort: 18801, color: "#0066CC" },

代码： remote: { cdpUrl: "http://10.0.0.42:9222", color: "#00AA00" },

代码： },

代码：},

代码：}

注意事项：

- 浏览器控制服务绑定到 loopback 上的端口，该端口从 gateway.port 派生（默认：18791，即 gateway + 2）。中继使用下一个端口（18792）。
- 如果你覆盖了 Gateway 网关端口（gateway.port 或 OPENCLAW_GATEWAY_PORT），派生的浏览器端口会相应调整以保持在同一"系列"中。
- 未设置时，cdpUrl 默认为中继端口。
- remoteCdpTimeoutMs 适用于远程（非 loopback）CDP 可达性检查。
- remoteCdpHandshakeTimeoutMs 适用于远程 CDP WebSocket 可达性检查。
- attachOnly: true 表示"永不启动本地浏览器；仅在浏览器已运行时附加"。
- color + 每个配置文件的 color 为浏览器 UI 着色，以便你能看到哪个配置文件处于活动状态。
- 默认配置文件是 chrome（扩展中继）。使用 defaultProfile: "openclaw" 来使用托管浏览器。
- 自动检测顺序：如果系统默认浏览器是基于 Chromium 的则使用它；否则 Chrome → Brave → Edge → Chromium → Chrome Canary。
- 本地 openclaw 配置文件会自动分配 cdpPort/cdpUrl — 仅为远程 CDP 设置这些。

使用 Brave（或其他基于 Chromium 的浏览器）

如果你的系统默认浏览器是基于 Chromium 的（Chrome/Brave/Edge 等），OpenClaw 会自动使用它。设置 browser.executablePath 可覆盖自动检测：

CLI 示例：

代码：openclaw config set browser.executablePath "/usr/bin/google-chrome"

代码：// macOS

代码：{

代码：browser: {

代码：executablePath: "/Applications/Brave Browser.app/Contents/MacOS/Brave Browser"

代码：}

代码：}

代码：// Windows

代码：{

代码： browser: {

代码： executablePath: "C:\\Program

Files\\BraveSoftware\\Brave-Browser\\Application\\brave.exe"

代码： }

代码： }

代码： // Linux

代码： {

代码： browser: {

代码： executablePath: "/usr/bin/brave-browser"

代码： }

代码： }

本地控制与远程控制

- 本地控制（默认）： Gateway 网关启动 loopback 控制服务，可以启动本地浏览器。
- 远程控制（节点主机）： 在有浏览器的机器上运行节点主机； Gateway 网关将浏览器操作代理到该节点。
- 远程 CDP： 设置 browser.profiles.<name>.cdpUrl（或 browser.cdpUrl）以附加到远程的基于 Chromium 的浏览器。在这种情况下，OpenClaw 不会启动本地浏览器。

远程 CDP URL 可以包含认证信息：

- 查询令牌（例如`
- HTTP Basic 认证（例如`

OpenClaw 在调用 /json/ 端点和连接 CDP WebSocket

时会保留认证信息。建议使用环境变量或密钥管理器存储令牌，而不是将其提交到配置文件中。

节点浏览器代理（零配置默认）

如果你在有浏览器的机器上运行节点主机，OpenClaw

可以自动将浏览器工具调用路由到该节点，无需任何额外的浏览器配置。这是远程 Gateway 网关的默认路径。

注意事项：

- 节点主机通过代理命令暴露其本地浏览器控制服务器。
- 配置文件来自节点自己的 browser.profiles 配置（与本地相同）。
- 如果不需要可以禁用：
- 在节点上：nodeHost.browserProxy.enabled=false
- 在 Gateway 网关上：gateway.nodes.browser.mode="off"

Browserless（托管远程 CDP）

Browserless 是一个托管的 Chromium 服务，通过 HTTPS 暴露 CDP 端点。你可以将 OpenClaw 浏览器配置文件指向 Browserless 区域端点，并使用你的 API 密钥进行认证。

示例：

代码：{

代码： browser: {

代码： enabled: true,

代码： defaultProfile: "browserless",

代码： remoteCdpTimeoutMs: 2000,

代码： remoteCdpHandshakeTimeoutMs: 4000,

代码： profiles: {

代码： browserless: {

代码： cdpUrl: "https://production-sfo.browserless.io?token=<BROWSERLESS_API_KEY>",

代码： color: "#00AA00",

代码： },

代码： },

代码： },

代码： }

注意事项：

- 将 <BROWSERLESS_API_KEY> 替换为你真实的 Browserless 令牌。
- 选择与你的 Browserless 账户匹配的区域端点（请参阅其文档）。

安全性

核心理念：

- 浏览器控制仅限 loopback；访问通过 Gateway 网关的认证或节点配对进行。
- 将 Gateway 网关和任何节点主机保持在私有网络上（Tailscale）；避免公开暴露。
- 将远程 CDP URL/令牌视为机密；优先使用环境变量或密钥管理器。

远程 CDP 提示：

- 尽可能使用 HTTPS 端点和短期令牌。
- 避免在配置文件中直接嵌入长期令牌。

配置文件（多浏览器）

OpenClaw 支持多个命名配置文件（路由配置）。配置文件可以是：

- openclaw 托管：具有独立用户数据目录和 CDP 端口的专用基于 Chromium 的浏览器实例
- 远程：显式 CDP URL（在其他地方运行的基于 Chromium 的浏览器）
- 扩展中继：通过本地中继 + Chrome 扩展访问你现有的 Chrome 标签页

默认值：

- 如果缺少 openclaw 配置文件，会自动创建。
- chrome 配置文件是内置的，用于 Chrome 扩展中继（默认指向 `
- 本地 CDP 端口默认从 18800–18899 分配。
- 删除配置文件会将其本地数据目录移至回收站。

所有控制端点接受 ?profile=<name>；CLI 使用 --browser-profile。

Chrome 扩展中继（使用你现有的 Chrome）

OpenClaw 还可以通过本地 CDP 中继 + Chrome 扩展驱动你现有的 Chrome 标签页（无需单独的"openclaw"Chrome 实例）。

完整指南：Chrome 扩展

流程：

- Gateway 网关在本地运行（同一台机器）或节点主机在浏览器所在机器上运行。
- 本地中继服务器在 loopback 的 cdpUrl 上监听（默认：`
- 你点击标签页上的 OpenClaw Browser Relay 扩展图标来附加（它不会自动附加）。
- 智能体通过选择正确的配置文件，使用普通的 browser 工具控制该标签页。

如果 Gateway 网关在其他地方运行，请在浏览器所在机器上运行节点主机，以便 Gateway 网关可以代理浏览器操作。

沙箱会话

如果智能体会话是沙箱隔离的，browser 工具可能默认为 target="sandbox"（沙箱浏览器）。

Chrome 扩展中继接管需要主机浏览器控制，因此要么：

- 在非沙箱模式下运行会话，或者
- 设置 agents.defaults.sandbox.browser.allowHostControl: true 并在调用工具时使用 target="host"。

设置

- 加载扩展（开发/未打包）：

代码：openclaw browser extension install

- Chrome → chrome://extensions → 启用"开发者模式"
- "加载已解压的扩展程序" → 选择 openclaw browser extension path 打印的目录
- 固定扩展，然后在你想要控制的标签页上点击它（徽章显示 ON）。
- 使用它：
- CLI：openclaw browser --browser-profile chrome tabs
- 智能体工具：browser 配合 profile="chrome"

可选：如果你想要不同的名称或中继端口，创建你自己的配置文件：

代码：openclaw browser create-profile \

代码：--name my-chrome \

代码：--driver extension \

代码：--cdp-url http://127.0.0.1:18792 \

代码：--color "#00AA00"

注意事项：

- 此模式依赖 Playwright-on-CDP 进行大多数操作（截图/快照/操作）。
- 再次点击扩展图标可分离。

隔离保证

- 专用用户数据目录：永不触及你的个人浏览器配置文件。
- 专用端口：避免使用 9222 以防止与开发工作流冲突。
- 确定性标签页控制：通过 `targetId` 定位标签页，而非"最后一个标签页"。

浏览器选择

本地启动时，OpenClaw 选择第一个可用的：

- Chrome
- Brave
- Edge
- Chromium
- Chrome Canary

你可以使用 `browser.executablePath` 覆盖。

平台：

- macOS：检查 `/Applications` 和 `~/Applications`。
- Linux：查找 `google-chrome`、`brave`、`microsoft-edge`、`chromium` 等。
- Windows：检查常见安装位置。

控制 API（可选）

仅用于本地集成，Gateway 网关暴露一个小型的 loopback HTTP API：

- 状态/启动/停止：GET `/`、POST `/start`、POST `/stop`
- 标签页：GET `/tabs`、POST `/tabs/open`、POST `/tabs/focus`、DELETE `/tabs/:targetId`
- 快照/截图：GET `/snapshot`、POST `/screenshot`
- 操作：POST `/navigate`、POST `/act`
- 钩子：POST `/hooks/file-chooser`、POST `/hooks/dialog`
- 下载：POST `/download`、POST `/wait/download`
- 调试：GET `/console`、POST `/pdf`
- 调试：GET `/errors`、GET `/requests`、POST `/trace/start`、POST `/trace/stop`、POST `/highlight`
- 网络：POST `/response/body`

- 状态：GET /cookies、POST /cookies/set、POST /cookies/clear
- 状态：GET /storage/:kind、POST /storage/:kind/set、POST /storage/:kind/clear
- 设置：POST /set/offline、POST /set/headers、POST /set/credentials、POST /set/geolocation、POST /set/media、POST /set/timezone、POST /set/locale、POST /set/device

所有端点接受 ?profile=<name>。

Playwright 要求

某些功能（navigate/act/AI 快照/角色快照、元素截图、PDF）需要 Playwright。如果未安装 Playwright，这些端点会返回明确的 501 错误。ARIA 快照和基本截图对于 openclaw 托管的 Chrome 仍然有效。对于 Chrome 扩展中继驱动程序，ARIA 快照和截图需要 Playwright。

如果你看到 Playwright is not available in this gateway build，请安装完整的 Playwright 包（不是 playwright-core）并重启 Gateway 网关，或者重新安装带浏览器支持的 OpenClaw。

Docker Playwright 安装

如果你的 Gateway 网关在 Docker 中运行，避免使用 npx playwright（npm 覆盖冲突）。改用捆绑的 CLI：

代码：docker compose run --rm openclaw-cli \

代码：node /app/node_modules/playwright-core/cli.js install chromium

要持久化浏览器下载，设置 PLAYWRIGHT_BROWSERS_PATH（例如 /home/node/.cache/ms-playwright）并确保 /home/node 通过 OPENCLAW_HOME_VOLUME 或绑定挂载持久化。参见 Docker。

工作原理（内部）

高层流程：

- 一个小型控制服务器接受 HTTP 请求。
- 它通过 CDP 连接到基于 Chromium 的浏览器（Chrome/Brave/Edge/Chromium）。
- 对于高级操作（点击/输入/快照/PDF），它在 CDP 之上使用 Playwright。
- 当缺少 Playwright 时，仅非 Playwright 操作可用。

这种设计使智能体保持在稳定、确定性的接口上，同时让你可以切换本地/远程浏览器和配置文件。

CLI 快速参考

所有命令接受 `--browser-profile <name>` 以定位特定配置文件。

所有命令也接受 `--json` 以获得机器可读的输出（稳定的负载）。

基础操作：

- `openclaw browser status`
- `openclaw browser start`
- `openclaw browser stop`
- `openclaw browser tabs`
- `openclaw browser tab`
- `openclaw browser tab new`
- `openclaw browser tab select 2`
- `openclaw browser tab close 2`
- ``openclaw browser open`
- `openclaw browser focus abcd1234`
- `openclaw browser close abcd1234`

检查：

- `openclaw browser screenshot`
- `openclaw browser screenshot --full-page`
- `openclaw browser screenshot --ref 12`
- `openclaw browser screenshot --ref e12`
- `openclaw browser snapshot`
- `openclaw browser snapshot --format aria --limit 200`
- `openclaw browser snapshot --interactive --compact --depth 6`
- `openclaw browser snapshot --efficient`
- `openclaw browser snapshot --labels`
- `openclaw browser snapshot --selector "#main" --interactive`
- `openclaw browser snapshot --frame "iframe#main" --interactive`
- `openclaw browser console --level error`

- `openclaw browser errors --clear`
- `openclaw browser requests --filter api --clear`
- `openclaw browser pdf`
- `openclaw browser responsebody "/api" --max-chars 5000`

操作：

- ``openclaw browser navigate`
- `openclaw browser resize 1280 720`
- `openclaw browser click 12 --double`
- `openclaw browser click e12 --double`
- `openclaw browser type 23 "hello" --submit`
- `openclaw browser press Enter`
- `openclaw browser hover 44`
- `openclaw browser scrollintoview e12`
- `openclaw browser drag 10 11`
- `openclaw browser select 9 OptionA OptionB`
- `openclaw browser download e12 /tmp/report.pdf`
- `openclaw browser waitfordownload /tmp/report.pdf`
- `openclaw browser upload /tmp/file.pdf`
- `openclaw browser fill --fields '[{"ref":"1","type":"text","value":"Ada"}]'`
- `openclaw browser dialog --accept`
- `openclaw browser wait --text "Done"`
- `openclaw browser wait "#main" --url "/dash" --load networkidle --fn "window.ready===true"`
- `openclaw browser evaluate --fn '(el) => el.textContent' --ref 7`
- `openclaw browser highlight e12`
- `openclaw browser trace start`
- `openclaw browser trace stop`

状态：

- `openclaw browser cookies`

- ``openclaw browser cookies set session abc123 --url "`
- `openclaw browser cookies clear`
- `openclaw browser storage local get`
- `openclaw browser storage local set theme dark`
- `openclaw browser storage session clear`
- `openclaw browser set offline on`
- `openclaw browser set headers --json '{"X-Debug":"1"}'`
- `openclaw browser set credentials user pass`
- `openclaw browser set credentials --clear`
- ``openclaw browser set geo 37.7749 -122.4194 --origin "`
- `openclaw browser set geo --clear`
- `openclaw browser set media dark`
- `openclaw browser set timezone America/New_York`
- `openclaw browser set locale en-US`
- `openclaw browser set device "iPhone 14"`

注意事项：

- `upload` 和 `dialog` 是预备调用；在触发选择器/对话框的点击/按键之前运行它们。
- `upload` 也可以通过 `--input-ref` 或 `--element` 直接设置文件输入。
- `snapshot`：
 - `--format ai`（安装 Playwright 时的默认值）：返回带有数字 `ref` 的 AI 快照（`aria-ref="<n>"`）。
 - `--format aria`：返回无障碍树（无 `ref`；仅供检查）。
 - `--efficient`（或 `--mode efficient`）：紧凑角色快照预设（`interactive + compact + depth +` 较低的 `maxChars`）。
 - 配置默认值（仅限工具/CLI）：设置 `browser.snapshotDefaults.mode: "efficient"` 以在调用者未传递模式时使用高效快照（参见 Gateway 网关配置）。
- 角色快照选项（`--interactive`、`--compact`、`--depth`、`--selector`）强制使用带有 `ref=e12` 等 `ref` 的基于角色的快照。
- `--frame "<iframe selector>"` 将角色快照范围限定到 `iframe`（与 `e12` 等角色 `ref` 配合使用）。
- `--interactive` 输出一个扁平的、易于选择的交互元素列表（最适合驱动操作）。
- `--labels` 添加一个带有叠加 `ref` 标签的视口截图（打印 `MEDIA:<path>`）。

- click/type 等需要来自 snapshot 的 ref（数字 12 或角色 ref e12）。

操作故意不支持 CSS 选择器。

快照和 ref

OpenClaw 支持两种"快照"风格：

- AI 快照（数字 ref）：openclaw browser snapshot（默认；--format ai）
- 输出：包含数字 ref 的文本快照。
- 操作：openclaw browser click 12、openclaw browser type 23 "hello"。
- 内部通过 Playwright 的 aria-ref 解析 ref。
- 角色快照（角色 ref 如 e12）：openclaw browser snapshot --interactive（或 --compact、--depth、--selector、--frame）
- 输出：带有 [ref=e12]（和可选的 [nth=1]）的基于角色的列表/树。
- 操作：openclaw browser click e12、openclaw browser highlight e12。
- 内部通过 getByRole(...)（加上重复项的 nth()）解析 ref。
- 添加 --labels 可包含带有叠加 e12 标签的视口截图。

ref 行为：

- ref 在导航之间不稳定；如果出错，重新运行 snapshot 并使用新的 ref。
- 如果角色快照是使用 --frame 拍摄的，角色 ref 将限定在该 iframe 内，直到下一次角色快照。

等待增强功能

你可以等待的不仅仅是时间/文本：

- 等待 URL（Playwright 支持通配符）：
- openclaw browser wait --url "/dash"
- 等待加载状态：
- openclaw browser wait --load networkidle
- 等待 JS 断言：
- openclaw browser wait --fn "window.ready===true"
- 等待选择器变得可见：
- openclaw browser wait "#main"

这些可以组合使用：

代码：openclaw browser wait "#main" \

代码：--url "**/dash" \

代码：--load networkidle \

代码：--fn "window.ready===true" \

代码：--timeout-ms 15000

调试工作流

当操作失败时（例如"not visible"、"strict mode violation"、"covered"）：

- openclaw browser snapshot --interactive
- 使用 click <ref> / type <ref>（在交互模式下优先使用角色 ref）
- 如果仍然失败：openclaw browser highlight <ref> 查看 Playwright 定位的目标
- 如果页面行为异常：
- openclaw browser errors --clear
- openclaw browser requests --filter api --clear
- 深度调试：录制 trace：
- openclaw browser trace start
- 重现问题
- openclaw browser trace stop（打印 TRACE:<path>）

JSON 输出

--json 用于脚本和结构化工具。

示例：

代码：openclaw browser status --json

代码：openclaw browser snapshot --interactive --json

代码：openclaw browser requests --filter api --json

代码：openclaw browser cookies --json

JSON 格式的角色快照包含 refs 加上一个小的 stats

块（lines/chars/refs/interactive），以便工具可以推断负载大小和密度。

状态和环境开关

这些对于"让网站表现得像 X"的工作流很有用：

- Cookies：cookies、cookies set、cookies clear
- 存储：storage local|session get|set|clear
- 离线：set offline on|off
- 请求头：set headers --json '{"X-Debug":"1"}'（或 --clear）
- HTTP basic 认证：set credentials user pass（或 --clear）
- 地理位置：set geo <lat> <lon> --origin " --clear`）
- 媒体：set media dark|light|no-preference|none
- 时区/语言环境：set timezone ...、set locale ...
- 设备/视口：
- set device "iPhone 14"（Playwright 设备预设）
- set viewport 1280 720

安全与隐私

- openclaw 浏览器配置文件可能包含已登录的会话；请将其视为敏感信息。
- browser act kind=evaluate / openclaw browser evaluate 和 wait --fn
在页面上下文中执行任意 JavaScript。提示注入可能会操纵它。如果不需要，请使用 browser.evaluateEnabled=false 禁用它。
- 有关登录和反机器人注意事项（X/Twitter 等），请参阅 浏览器登录 + X/Twitter 发帖。
- 保持 Gateway 网关/节点主机私有（仅限 loopback 或 tailnet）。
- 远程 CDP 端点功能强大；请通过隧道保护它们。

故障排除

有关 Linux 特定问题（特别是 snap Chromium），请参阅浏览器故障排除。

智能体工具 + 控制工作原理

智能体获得一个工具用于浏览器自动化：

- browser — status/start/stop/tabs/open/focus/close/snapshot/screenshot/navigate/act

映射方式：

- browser snapshot 返回稳定的 UI 树 (AI 或 ARIA) 。
- browser act 使用快照 ref ID 来点击/输入/拖动/选择。
- browser screenshot 捕获像素 (整页或元素) 。
- browser 接受：
- profile 来选择命名的浏览器配置文件 (openclaw、chrome 或远程 CDP) 。
- target (sandbox | host | node) 来选择浏览器所在位置。
- 在沙箱会话中，target: "host" 需要 `agents.defaults.sandbox.browser.allowHostControl=true` 。
- 如果省略 target：沙箱会话默认为 sandbox，非沙箱会话默认为 host。
- 如果连接了具有浏览器能力的节点，工具可能会自动路由到该节点，除非你指定 target="host" 或 target="node"。

这使智能体保持确定性并避免脆弱的选择器。

6. Chrome 扩展 (浏览器中继)

Chrome 扩展 (浏览器中继)

OpenClaw Chrome 扩展让智能体控制你现有的 Chrome 标签页 (你的正常 Chrome 窗口)，而不是启动一个单独的 openclaw 管理的 Chrome 配置文件。

附加/分离通过一个单独的 Chrome 工具栏按钮实现。

它是什么 (概念)

有三个部分：

- 浏览器控制服务 (Gateway 网关或节点)：智能体/工具调用的 API (通过 Gateway 网关)
- 本地中继服务器 (loopback CDP)：在控制服务器和扩展之间桥接 (默认)
- Chrome MV3 扩展：使用 chrome.debugger 附加到活动标签页，并将 CDP 消息传送到中继

然后 OpenClaw 通过正常的 browser 工具界面控制附加的标签页 (选择正确的配置文件)。

安装/加载 (未打包)

- 将扩展安装到稳定的本地路径：

代码：openclaw browser extension install

- 打印已安装扩展的目录路径：

代码：openclaw browser extension path

- Chrome → chrome://extensions
- 启用"开发者模式"
- "加载已解压的扩展程序" → 选择上面打印的目录
- 固定扩展。

更新（无构建步骤）

扩展作为静态文件包含在 OpenClaw 发布版（npm 包）中。没有单独的"构建"步骤。

升级 OpenClaw 后：

- 重新运行 openclaw browser extension install 以刷新 OpenClaw 状态目录下的已安装文件。
- Chrome → chrome://extensions → 点击扩展上的"重新加载"。

使用它（无需额外配置）

OpenClaw 附带一个名为 chrome 的内置浏览器配置文件，它指向默认端口上的扩展中继。

使用它：

- CLI：openclaw browser --browser-profile chrome tabs
- 智能体工具：browser 配合 profile="chrome"

如果你想要不同的名称或不同的中继端口，创建你自己的配置文件：

代码：openclaw browser create-profile \

代码：--name my-chrome \

代码：--driver extension \

代码：--cdp-url http://127.0.0.1:18792 \

代码：--color "#00AA00"

附加/分离（工具栏按钮）

- 打开你希望 OpenClaw 控制的标签页。
- 点击扩展图标。

- 附加时徽章显示 ON。
- 再次点击以分离。

它控制哪个标签页？

- 它不会自动控制"你正在查看的任何标签页"。
- 它仅控制你通过点击工具栏按钮明确附加的标签页。
- 要切换：打开另一个标签页并在那里点击扩展图标。

徽章 + 常见错误

- ON：已附加；OpenClaw 可以驱动该标签页。
- ...：正在连接到本地中继。
- !：中继不可达（最常见：浏览器中继服务器未在此机器上运行）。

如果你看到！：

- 确保 Gateway 网关在本地运行（默认设置），或者如果 Gateway 网关在其他地方运行，在此机器上运行一个节点主机。
- 打开扩展选项页面；它会显示中继是否可达。

远程 Gateway 网关（使用节点主机）

本地 Gateway 网关（与 Chrome 在同一台机器上）——通常**无需额外步骤**

如果 Gateway 网关运行在与 Chrome 相同的机器上，它会在 loopback 上启动浏览器控制服务并自动启动中继服务器。扩展与本地中继通信；CLI/工具调用发送到 Gateway 网关。

远程 Gateway 网关（Gateway 网关运行在其他地方）——**运行节点主机**

如果你的 Gateway 网关运行在另一台机器上，在运行 Chrome 的机器上启动一个节点主机。Gateway 网关将把浏览器操作代理到该节点；扩展 + 中继保持在浏览器机器本地。

如果连接了多个节点，使用 `gateway.nodes.browser.node` 固定一个或设置 `gateway.nodes.browser.mode`。

沙箱隔离（工具容器）

如果你的智能体会话在沙箱中（`agents.defaults.sandbox.mode != "off"`），browser 工具可能受到限制：

- 默认情况下，沙箱隔离的会话通常指向沙箱浏览器（`target="sandbox"`），而不是你的主机 Chrome。
- Chrome 扩展中继接管需要控制主机浏览器控制服务器。

选项：

- 最简单：从非沙箱隔离的会话/智能体使用扩展。
- 或者为沙箱隔离的会话允许主机浏览器控制：

代码：{

代码：agents: {

代码：defaults: {

代码：sandbox: {

代码：browser: {

代码：allowHostControl: true,

代码：},

代码：},

代码：},

代码：},

代码：}

然后确保工具未被工具策略拒绝，并（如果需要）以 `target="host"` 调用 browser。

调试：`openclaw sandbox explain`

远程访问提示

- 将 Gateway 网关和节点主机保持在同一个 tailnet 上；避免将中继端口暴露到 LAN 或公共 Internet。
- 有意配对节点；如果你不想要远程控制，禁用浏览器代理路由（`gateway.nodes.browser.mode="off"`）。

"extension path"的工作原理

`openclaw browser extension path` 打印包含扩展文件的已安装磁盘目录。

CLI 有意不打印 node_modules 路径。始终先运行 openclaw browser extension install 将扩展复制到 OpenClaw 状态目录下的稳定位置。

如果你移动或删除该安装目录，Chrome 将把扩展标记为损坏，直到你从有效路径重新加载它。

安全影响（请阅读此内容）

这是强大且有风险的。将其视为给模型"在你的浏览器上动手"。

- 扩展使用 Chrome 的调试器 API (chrome.debugger)。附加时，模型可以：
- 在该标签页中点击/输入/导航
- 读取页面内容
- 访问标签页已登录会话可以访问的任何内容
- 这不像专用的 openclaw 管理配置文件那样隔离。
- 如果你附加到你的日常使用配置文件/标签页，你就是在授予对该账户状态的访问权限。

建议：

- 对于扩展中继使用，优先使用专用的 Chrome 配置文件（与你的个人浏览分开）。
- 将 Gateway 网关和任何节点主机保持在仅限 tailnet；依赖 Gateway 网关身份验证 + 节点配对。
- 避免通过 LAN (0.0.0.0) 暴露中继端口，避免使用 Funnel（公开）。
- 中继阻止非扩展来源，并要求 CDP 客户端提供内部身份验证令牌。

相关：

- 浏览器工具概述：浏览器
- 安全审计：安全
- Tailscale 设置：Tailscale

7. ClawHub

ClawHub

ClawHub 是 OpenClaw 的公共 Skills 注册中心。它是一项免费服务：所有 Skills 都是公开的、开放的，所有人可以查看、共享和复用。Skills 就是一个包含 SKILL.md 文件（以及辅助文本文件）的文件夹。你可以在网页应用中浏览 Skills，也可以使用 CLI 来搜索、安装、更新和发布 Skills。

网站：clawhub.com

适用人群（新手友好）

如果你想为 OpenClaw 智能体添加新功能，ClawHub 是查找和安装 Skills 的最简单方式。你不需要了解后端的工作原理。你可以：

- 使用自然语言搜索 Skills。
- 将 Skills 安装到你的工作区。
- 之后使用一条命令更新 Skills。
- 通过发布 Skills 来备份你自己的 Skills。

快速入门（非技术人员）

- 安装 CLI（参见下一节）。
- 搜索你需要的内容：
- `clawhub search "calendar"`
- 安装一个 Skills：
- `clawhub install <skill-slug>`
- 启动一个新的 OpenClaw 会话，以加载新 Skills。

安装 CLI

任选其一：

代码：`npm i -g clawhub`

代码：`pnpm add -g clawhub`

在 OpenClaw 中的定位

默认情况下，CLI 会将 Skills 安装到当前工作目录下的 `./skills`。如果已配置 OpenClaw 工作区，clawhub 会回退到该工作区，除非你通过 `--workdir`（或 `CLAWHUB_WORKDIR`）进行覆盖。OpenClaw 从 `<workspace>/skills` 加载工作区 Skills，并会在下一个会话中生效。如果你已经在使用 `~/openclaw/skills` 或内置 Skills，工作区 Skills 优先级更高。

有关 Skills 加载、共享和权限控制的更多详情，请参阅 Skills。

服务功能

- 公开浏览Skills 及其 SKILL.md 内容。
- 基于嵌入向量（向量搜索）的搜索，而不仅仅是关键词匹配。
- 支持语义化版本号、变更日志和标签（包括 latest）的版本管理。
- 每个版本以 zip 格式下载。
- 星标和评论，支持社区反馈。
- 审核钩子，用于审批和审计。
- CLI 友好的 API，支持自动化和脚本编写。

CLI 命令和参数

全局选项（适用于所有命令）：

- `--workdir <dir>`：工作目录（默认：当前目录；回退到 OpenClaw 工作区）。
- `--dir <dir>`：Skills 目录，相对于工作目录（默认：skills）。
- `--site <url>`：网站基础 URL（浏览器登录）。
- `--registry <url>`：注册中心 API 基础 URL。
- `--no-input`：禁用提示（非交互模式）。
- `-V, --cli-version`：打印 CLI 版本。

认证：

- `clawhub login`（浏览器流程）或 `clawhub login --token <token>`
- `clawhub logout`
- `clawhub whoami`

选项：

- `--token <token>`：粘贴 API 令牌。
- `--label <label>`：为浏览器登录令牌存储的标签（默认：CLI token）。
- `--no-browser`：不打开浏览器（需要 `--token`）。

搜索：

- `clawhub search "query"`
- `--limit <n>`：最大结果数。

安装：

- `clawhub install <slug>`
- `--version <version>`：安装指定版本。
- `--force`：如果文件夹已存在则覆盖。

更新：

- `clawhub update <slug>`
- `clawhub update --all`
- `--version <version>`：更新到指定版本（仅限单个 slug）。
- `--force`：当本地文件与任何已发布版本不匹配时强制覆盖。

列表：

- `clawhub list`（读取 `.clawhub/lock.json`）

发布：

- `clawhub publish <path>`
- `--slug <slug>`：Skills 标识符。
- `--name <name>`：显示名称。
- `--version <version>`：语义化版本号。
- `--changelog <text>`：变更日志文本（可以为空）。
- `--tags <tags>`：逗号分隔的标签（默认：latest）。

删除/恢复（仅所有者/管理员）：

- `clawhub delete <slug> --yes`
- `clawhub undelete <slug> --yes`

同步（扫描本地 Skills + 发布新增/更新的 Skills）：

- `clawhub sync`
- `--root <dir...>`：额外的扫描根目录。
- `--all`：无提示上传所有内容。
- `--dry-run`：显示将要上传的内容。
- `--bump <type>`：更新的版本号递增类型 `patch|minor|major`（默认：patch）。

- `--changelog <text>`：非交互更新的变更日志。
- `--tags <tags>`：逗号分隔的标签（默认：latest）。
- `--concurrency <n>`：注册中心检查并发数（默认：4）。

智能体常用 workflow

搜索 Skills

代码：`clawhub search "postgres backups"`

下载新 Skills

代码：`clawhub install my-skill-pack`

更新已安装的 Skills

代码：`clawhub update --all`

备份你的 Skills（发布或同步）

对于单个 Skills 文件夹：

代码：`clawhub publish ./my-skill --slug my-skill --name "My Skill" --version 1.0.0 --tags latest`

一次扫描并备份多个 Skills：

代码：`clawhub sync --all`

高级详情（技术性）

版本管理和标签

- 每次发布都会创建一个新的语义化版本 `SkillVersion`。
- 标签（如 `latest`）指向某个版本；移动标签可以实现回滚。
- 变更日志附加在每个版本上，在同步或发布更新时可以为空。

本地更改与注册中心版本

更新时会使用内容哈希将本地 Skills

内容与注册中心版本进行比较。如果本地文件与任何已发布版本不匹配，CLI 会在覆盖前询问确认（或在非交互模式下需要 `--force`）。

同步扫描和回退根目录

clawhub sync 首先扫描当前工作目录。如果未找到 Skills，它会回退到已知的旧版位置（例如 `~/openclaw/skills` 和 `~/.openclaw/skills`）。这样设计是为了在不需要额外标志的情况下找到旧版 Skills 安装。

存储和锁文件

- 已安装的 Skills 记录在工作目录下的 `.clawhub/lock.json` 中。
- 认证令牌存储在 ClawHub CLI 配置文件中（可通过 `CLAWHUB_CONFIG_PATH` 覆盖）。

遥测（安装计数）

当你在登录状态下运行 `clawhub sync` 时，CLI 会发送一个最小快照用于计算安装次数。你可以完全禁用此功能：

环境变量

- `CLAWHUB_SITE`：覆盖网站 URL。
- `CLAWHUB_REGISTRY`：覆盖注册中心 API URL。
- `CLAWHUB_CONFIG_PATH`：覆盖 CLI 存储令牌/配置的位置。
- `CLAWHUB_WORKDIR`：覆盖默认工作目录。
- `CLAWHUB_DISABLE_TELEMETRY=1`：禁用 sync 的遥测功能。

8. 创建自定义 Skills

创建自定义 Skills

OpenClaw 被设计为易于扩展。"Skills"是为你的助手添加新功能的主要方式。

什么是 Skill？

Skill 是一个包含 `SKILL.md` 文件（为 LLM 提供指令和工具定义）的目录，可选包含一些脚本或资源。

分步指南：你的第一个 Skill

1. 创建目录

Skills 位于你的工作区中，通常是 `~/.openclaw/workspace/skills/`。为你的 Skill 创建一个新文件夹：

代码：`mkdir -p ~/.openclaw/workspace/skills/hello-world`

2. 定义 `SKILL.md`

在该目录中创建一个 SKILL.md 文件。此文件使用 YAML frontmatter 作为元数据，使用 Markdown 作为指令。

代码：---

代码：name: hello_world

代码：description: A simple skill that says hello.

代码：---

代码：# Hello World Skill

代码：When the user asks for a greeting, use the `echo` tool to say "Hello from your custom skill!".

3. 添加工具（可选）

你可以在 frontmatter 中定义自定义工具，或指示智能体使用现有的系统工具（如 bash 或 browser）。

4. 刷新 OpenClaw

让你的智能体"刷新 skills"或重启 Gateway 网关。OpenClaw 将发现新目录并索引 SKILL.md。

最佳实践

- 简洁明了：指示模型做什么，而不是如何成为一个 AI。
- 安全第一：如果你的 Skill 使用 bash，确保提示词不允许来自不受信任用户输入的任意命令注入。
- 本地测试：使用 `openclaw agent --message "use my new skill"` 进行测试。

共享 Skills

你也可以在 ClawHub 上浏览和贡献 Skills。

9. 提升模式（/elevated 指令）

提升模式（/elevated 指令）

功能说明

- `/elevated on` 在 Gateway 网关主机上运行并保留 `exec` 审批（与 `/elevated ask` 相同）。
- `/elevated full` 在 Gateway 网关主机上运行并自动批准 `exec`（跳过 `exec` 审批）。
- `/elevated ask` 在 Gateway 网关主机上运行但保留 `exec` 审批（与 `/elevated on` 相同）。
- `on/ask` 不会强制 `exec.security=full`；配置的安全/询问策略仍然适用。
- 仅在智能体被沙箱隔离时改变行为（否则 `exec` 已经在主机上运行）。
- 指令形式：`/elevated on|off|ask|full`、`/elev on|off|ask|full`。
- 仅接受 `on|off|ask|full`；其他任何内容返回提示且不改变状态。

它控制什么（以及不控制什么）

- 可用性门控：`tools.elevated` 是全局基线。`agents.list[].tools.elevated` 可以进一步限制每个智能体的提升（两者都必须允许）。
- 每会话状态：`/elevated on|off|ask|full` 为当前会话键设置提升级别。
- 内联指令：消息内的 `/elevated on|ask|full` 仅适用于该消息。
- 群组：在群聊中，仅当智能体被提及时才遵守提升指令。绕过提及要求的纯命令消息被视为已提及。
- 主机执行：`elevated` 强制 `exec` 到 Gateway 网关主机；`full` 还设置 `security=full`。
- 审批：`full` 跳过 `exec` 审批；`on/ask` 在允许列表/询问规则要求时遵守审批。
- 非沙箱隔离智能体：对位置无影响；仅影响门控、日志和状态。
- 工具策略仍然适用：如果 `exec` 被工具策略拒绝，则无法使用 `elevated`。
- 与 `/exec` 分开：`/exec` 为授权发送者调整每会话默认值，不需要 `elevated`。

解析顺序

- 消息上的内联指令（仅适用于该消息）。
- 会话覆盖（通过发送仅含指令的消息设置）。
- 全局默认值（配置中的 `agents.defaults.elevatedDefault`）。

设置会话默认值

- 发送一条仅包含指令的消息（允许空白），例如 `/elevated full`。
- 发送确认回复（Elevated mode set to full... / Elevated mode disabled.）。

- 如果 `elevated`

访问被禁用或发送者不在批准的允许列表中，指令会回复一个可操作的错误且不改变会话状态。

- 发送不带参数的 `/elevated`（或 `/elevated:`）以查看当前的 `elevated` 级别。

可用性 + 允许列表

- 功能门控：`tools.elevated.enabled`（即使代码支持，也可以通过配置将默认值设为关闭）。
- 发送者允许列表：`tools.elevated.allowFrom`，带有每提供商允许列表（例如 `discord`、`whatsapp`）。
- 每智能体门控：`agents.list[].tools.elevated.enabled`（可选；只能进一步限制）。
- 每智能体允许列表：`agents.list[].tools.elevated.allowFrom`（可选；设置时，发送者必须同时匹配全局 + 每智能体允许列表）。
- Discord 回退：如果省略 `tools.elevated.allowFrom.discord`，则使用 `channels.discord.dm.allowFrom` 列表作为回退。设置 `tools.elevated.allowFrom.discord`（即使是 `[]`）以覆盖。每智能体允许列表不使用回退。
- 所有门控都必须通过；否则 `elevated` 被视为不可用。

日志 + 状态

- `Elevated exec` 调用以 `info` 级别记录。
- 会话状态包括 `elevated` 模式（例如 `elevated=ask`、`elevated=full`）。

10. 执行审批

执行审批

执行审批是配套应用/节点主机的安全护栏，用于允许沙箱隔离的智能体在真实主机（`gateway` 或 `node`）上运行命令。可以将其理解为安全联锁：只有当策略 + 允许列表 + （可选的）用户审批都同意时，命令才会被允许执行。

执行审批是附加于工具策略和提权门控之上的（除非 `elevated` 设置为 `full`，这会跳过审批）。

生效策略取 `tools.exec.` 和审批默认值中更严格的一方；如果审批字段被省略，则使用 `tools.exec` 的值。

如果配套应用 UI 不可用，任何需要提示的请求都将由 `ask fallback`（默认：`deny`）决定。

适用范围

执行审批在执行主机上本地强制执行：

- gateway 主机 → gateway 机器上的 openclaw 进程
- node 主机 → 节点运行器 (macOS 配套应用或无头节点主机)

macOS 分工：

- node 主机服务通过本地 IPC 将 system.run 转发给 macOS 应用。
- macOS 应用执行审批并在 UI 上下文中执行命令。

设置和存储

审批信息存储在执行主机上的本地 JSON 文件中：

~/openclaw/exec-approvals.json

示例结构：

```
代码：{
代码： "version": 1,
代码： "socket": {
代码： "path": "~/openclaw/exec-approvals.sock",
代码： "token": "base64url-token"
代码： },
代码： "defaults": {
代码： "security": "deny",
代码： "ask": "on-miss",
代码： "askFallback": "deny",
代码： "autoAllowSkills": false
代码： },
代码： "agents": {
代码： "main": {
代码： "security": "allowlist",
代码： "ask": "on-miss",
代码： "askFallback": "deny",
代码： "autoAllowSkills": true,
代码： "allowlist": [
```

代码：{

代码："id": "B0C8C0B3-2C2D-4F8A-9A3C-5A4B3C2D1E0F",

代码："pattern": "~/Projects/**/bin/rg",

代码："lastUsedAt": 1737150000000,

代码："lastUsedCommand": "rg -n TODO",

代码："lastResolvedPath": "/Users/user/Projects/.../bin/rg"

代码：}

代码：]

代码：}

代码：}

代码：}

策略选项

Security (`exec.security`)

- deny：阻止所有主机执行请求。
- allowlist：仅允许在允许列表中的命令。
- full：允许所有命令（等同于提权模式）。

Ask (`exec.ask`)

- off：从不提示。
- on-miss：仅在允许列表未匹配时提示。
- always：每次命令都提示。

Ask fallback (`askFallback`)

如果需要提示但无法访问 UI，fallback 决定：

- deny：阻止。
- allowlist：仅在允许列表匹配时允许。
- full：允许。

允许列表（按智能体）

允许列表是按智能体配置的。如果存在多个智能体，请在 macOS 应用中切换要编辑的智能体。模式匹配不区分大小写。

模式应解析为二进制路径（仅包含基本名称的条目会被忽略）。

旧版 `agents.default` 条目在加载时会迁移到 `agents.main`。

示例：

- `~/Projects//bin/bird`
- `~/local/bin/`
- `/opt/homebrew/bin/rg`

每个允许列表条目会跟踪：

- `id` 用于 UI 标识的稳定 UUID（可选）
- `last used` 时间戳
- `last used command`
- `last resolved path`

自动允许 skill CLI

启用 Auto-allow skill CLIs 后，已知 Skills 引用的可执行文件在节点（macOS 节点或无头节点主机）上被视为已列入允许列表。这通过 Gateway RPC 的 `skills.bins` 获取 skill 二进制列表。如果你想要严格的手动允许列表，请禁用此选项。

安全二进制（仅限标准输入）

`tools.exec.safeBins` 定义了一小组仅限标准输入的二进制文件（例如 `jq`），这些文件可以在允许列表模式下运行，无需显式的允许列表条目。安全二进制会拒绝位置文件参数和类路径标记，因此它们只能操作传入的流。

在允许列表模式下，shell 链式命令和重定向不会被自动允许。

当每个顶级段都满足允许列表（包括安全二进制或 skill 自动允许）时，允许 shell 链式命令（`&&`、`||`、`;`）。重定向在允许列表模式下仍不受支持。

命令替换（`$()` / 反引号）在允许列表解析期间会被拒绝，包括在双引号内；如果你需要字面的 `$()` 文本，请使用单引号。

默认安全二进制：`jq`、`grep`、`cut`、`sort`、`uniq`、`head`、`tail`、`tr`、`wc`。

Control UI 编辑

使用 Control UI → Nodes → Exec approvals

卡片来编辑默认值、按智能体的覆盖设置和允许列表。选择一个作用域（Defaults 或某个智能体），调整策略，添加/删除允许列表模式，然后点击 Save。UI 会显示每个模式的 last used 元数据，以便你保持列表整洁。

目标选择器可选择 Gateway（本地审批）或 Node。节点必须通告 `system.execApprovals.get/set`（macOS 应用或无头节点主机）。

如果节点尚未通告执行审批，请直接编辑其本地的 `~/.openclaw/exec-approvals.json`。

CLI：openclaw approvals 支持 gateway 或 node 编辑（参见 Approvals CLI）。

审批流程

当需要提示时，gateway 向操作员客户端广播 `exec.approval.requested`。

Control UI 和 macOS 应用通过 `exec.approval.resolve` 进行处理，然后 gateway 将已批准的请求转发给节点主机。

当需要审批时，exec 工具会立即返回一个审批 id。使用该 id 来关联后续的系统事件（Exec finished / Exec

denied）。如果在超时前没有收到决定，请求将被视为审批超时，并作为拒绝原因显示。

确认对话框包括：

- 命令 + 参数
- cwd
- 智能体 id
- 解析后的可执行文件路径
- 主机 + 策略元数据

操作：

- Allow once → 立即运行
- Always allow → 添加到允许列表 + 运行
- Deny → 阻止

审批转发到聊天渠道

你可以将执行审批提示转发到任何聊天渠道（包括插件渠道），并使用 `/approve` 进行批准。这使用正常的出站投递管道。

配置：

代码：{

代码： approvals: {

代码： exec: {

代码： enabled: true,

代码： mode: "session", // "session" | "targets" | "both"

代码： agentFilter: ["main"],

代码： sessionFilter: ["discord"], // substring or regex

代码： targets: [

代码： { channel: "slack", to: "U12345678" },

代码： { channel: "telegram", to: "123456789" },

代码：],

代码： },

代码： },

代码： }

在聊天中回复：

代码： /approve <id> allow-once

代码： /approve <id> allow-always

代码： /approve <id> deny

macOS IPC 流程

代码： Gateway -> Node Service (WS)

代码： | IPC (UDS + token + HMAC + TTL)

代码： v

代码： Mac App (UI + approvals + system.run)

安全注意事项：

- Unix socket 模式 0600，token 存储在 exec-approvals.json 中。
- 同 UID 对端检查。
- 挑战/响应 (nonce + HMAC token + 请求哈希) + 短 TTL。

系统事件

执行生命周期以系统消息的形式呈现：

- Exec running（仅当命令超过运行通知阈值时）
- Exec finished
- Exec denied

这些消息在节点报告事件后发布到智能体的会话中。

Gateway

主机执行审批在命令完成时（以及可选地在运行时间超过阈值时）发出相同的生命周期事件。

经过审批门控的执行会复用审批 id 作为这些消息中的 runId，以便于关联。

影响

- full 权限很大；尽可能优先使用允许列表。
- ask 让你保持知情，同时仍允许快速审批。
- 按智能体的允许列表可防止一个智能体的审批泄漏到其他智能体。
- 审批仅适用于来自授权发送者的主机执行请求。未授权的发送者无法发出 /exec。
- /exec security=full 是为授权操作员提供的会话级便利功能，设计上会跳过审批。

要完全阻止主机执行，请将审批 security 设置为 deny，或通过工具策略拒绝 exec 工具。

相关内容：

- Exec 工具
- 提权模式
- Skills

11. Exec 工具

Exec 工具

在工作区中运行 shell 命令。通过 process 支持前台和后台执行。

如果 process 被禁用，exec 将同步运行并忽略 yieldMs/background。

后台会话按智能体隔离；process 只能看到同一智能体的会话。

参数

- command (必填)
- workdir (默认为当前工作目录)
- env (键值对覆盖)
- yieldMs (默认 10000) : 延迟后自动转入后台
- background (布尔值) : 立即转入后台
- timeout (秒, 默认 1800) : 超时后终止
- pty (布尔值) : 在可用时使用伪终端运行 (仅限 TTY 的 CLI、编程智能体、终端 UI)
- host (sandbox | gateway | node) : 执行位置
- security (deny | allowlist | full) : gateway/node 的执行策略
- ask (off | on-miss | always) : gateway/node 的审批提示
- node (字符串) : host=node 时的节点 id/名称
- elevated (布尔值) : 请求提升模式 (gateway 主机) ; 仅当 elevated 解析为 full 时才强制 security=full

注意事项 :

- host 默认为 sandbox 。
- 当沙箱隔离关闭时, elevated 会被忽略 (exec 已在主机上运行) 。
- gateway/node 审批由 ~/.openclaw/exec-approvals.json 控制 。
- node 需要已配对的节点 (配套应用或无头节点主机) 。
- 如果有多个可用节点, 设置 exec.node 或 tools.exec.node 来选择一个 。
- 在非 Windows 主机上, exec 会使用已设置的 SHELL ; 如果 SHELL 是 fish, 它会优先从 PATH 中选择 bash (或 sh) 以避免 fish 不兼容的脚本, 如果两者都不存在则回退到 SHELL 。
- 主机执行 (gateway/node) 会拒绝 env.PATH 和加载器覆盖 (LD_/DYLD_) , 以防止二进制劫持或代码注入 。
- 重要提示 : 沙箱隔离默认关闭。如果沙箱隔离关闭, host=sandbox 将直接在 Gateway 网关主机上运行 (无容器) 且不需要审批。如需审批, 请使用 host=gateway 运行并配置 exec 审批 (或启用沙箱隔离) 。

配置

- tools.exec.notifyOnExit (默认 : true) : 为 true 时, 后台 exec 会话在退出时会入队系统事件并请求心跳 。
- tools.exec.approvalRunningNoticeMs (默认 : 10000) : 当需要审批的 exec 运行时间超过此值时发出单次"运行中"通知 (0 表示禁用) 。

- tools.exec.host (默认 : sandbox)
- tools.exec.security (默认 : sandbox 为 deny , gateway + node 未设置时为 allowlist)
- tools.exec.ask (默认 : on-miss)
- tools.exec.node (默认 : 未设置)
- tools.exec.pathPrepend : exec 运行时添加到 PATH 前面的目录列表。
- tools.exec.safeBins : 仅限 stdin 的安全二进制文件 , 无需显式白名单条目即可运行。

示例 :

代码 : {

代码 : tools: {

代码 : exec: {

代码 : pathPrepend: ["~/bin", "/opt/oss/bin"],

代码 : },

代码 : },

代码 : }

PATH 处理

- host=gateway : 将你的登录 shell PATH 合并到 exec 环境中。主机执行时会拒绝 env.PATH 覆盖。守护进程本身仍使用最小 PATH 运行 :
- macOS : /opt/homebrew/bin 、 /usr/local/bin 、 /usr/bin 、 /bin
- Linux : /usr/local/bin 、 /usr/bin 、 /bin
- host=sandbox : 在容器内运行 sh -lc (登录 shell) , 因此 /etc/profile 可能会重置 PATH。OpenClaw 在 profile 加载后通过内部环境变量将 env.PATH 添加到前面 (无 shell 插值) ; tools.exec.pathPrepend 在此也适用。
- host=node : 只有你传递的未被阻止的 env 覆盖会发送到节点。主机执行时会拒绝 env.PATH 覆盖。无头节点主机仅在 PATH 添加到节点主机 PATH 前面时才接受 (不允许替换) 。 macOS 节点完全丢弃 PATH 覆盖。

按智能体绑定节点 (在配置中使用智能体列表索引) :

代码 : openclaw config get agents.list

代码 : openclaw config set agents.list[0].tools.exec.node "node-id-or-name"

控制 UI : Nodes 标签页包含一个小的 "Exec 节点绑定" 面板用于相同的设置。

会话覆盖（`/exec`）

使用 `/exec` 为 `host`、`security`、`ask` 和 `node` 设置每会话默认值。

不带参数发送 `/exec` 可显示当前值。

示例：

代码：`/exec host=gateway security=allowlist ask=on-miss node=mac-1`

授权模型

`/exec` 仅对已授权发送者（渠道白名单/配对加 `commands.useAccessGroups`）生效。

它仅更新会话状态，不写入配置。要彻底禁用 `exec`，请通过工具策略拒绝它（`tools.deny: ["exec"]` 或按智能体配置）。除非你显式设置 `security=full` 和 `ask=off`，否则主机审批仍然适用。

Exec 审批（配套应用/节点主机）

沙箱隔离的智能体可以要求在 `exec` 于 Gateway 网关或节点主机上运行前进行逐请求审批。

参阅 Exec 审批 了解策略、白名单和 UI 流程。

当需要审批时，`exec` 工具会立即返回 `status: "approval-pending"` 和审批 `id`。一旦被批准（或拒绝/超时），Gateway 网关会发出系统事件（`Exec finished` / `Exec denied`）。如果命令在 `tools.exec.approvalRunningNoticeMs` 之后仍在运行，会发出单次 `Exec running` 通知。

白名单 + 安全二进制文件

白名单执行仅匹配解析后的二进制路径（不匹配基本名称）。当 `security=allowlist` 时，仅当每个管道段都在白名单中或是安全二进制文件时，`shell` 命令才会自动允许。在白名单模式下，链式命令（`;`、`&&`、`||`）和重定向会被拒绝。

示例

前台：

代码：`{ "tool": "exec", "command": "ls -la" }`

后台 + 轮询：

代码：`{ "tool": "exec", "command": "npm run build", "yieldMs": 1000 }`

代码：`{ "tool": "process", "action": "poll", "sessionId": "<id>" }`

发送按键（tmux 风格）：

代码：`{"tool": "process", "action": "send-keys", "sessionId": "<id>", "keys": ["Enter"]}`

代码：`{"tool": "process", "action": "send-keys", "sessionId": "<id>", "keys": ["C-c"]}`

代码：`{"tool": "process", "action": "send-keys", "sessionId": "<id>", "keys": ["Up", "Up", "Enter"]}`

提交（仅发送 CR）：

代码：`{"tool": "process", "action": "submit", "sessionId": "<id>"}`

粘贴（默认带括号）：

代码：`{"tool": "process", "action": "paste", "sessionId": "<id>", "text": "line1\nline2\n"}`

apply_patch（实验性）

apply_patch 是 exec 的子工具，用于结构化多文件编辑。

需显式启用：

代码：`{`

代码：`tools: {`

代码：`exec: {`

代码：`applyPatch: { enabled: true, allowModels: ["gpt-5.2"] },`

代码：`},`

代码：`},`

代码：`}`

注意事项：

- 仅适用于 OpenAI/OpenAI Codex 模型。
- 工具策略仍然适用；allow: ["exec"] 隐式允许 apply_patch。
- 配置位于 tools.exec.applyPatch 下。

12. Firecrawl

Firecrawl

OpenClaw 可以使用 Firecrawl 作为 web_fetch 的回退提取器。它是一个托管的内容提取服务，支持机器人规避和缓存，有助于处理

JS 密集型网站或阻止普通 HTTP 请求的页面。

获取 API 密钥

- 创建 Firecrawl 账户并生成 API 密钥。
- 将其存储在配置中或在 Gateway 网关环境中设置 FIRECRAWL_API_KEY。

配置 Firecrawl

代码：{

代码： tools: {

代码： web: {

代码： fetch: {

代码： firecrawl: {

代码： apiKey: "FIRECRAWL_API_KEY_HERE",

代码： baseUrl: "https://api.firecrawl.dev",

代码： onlyMainContent: true,

代码： maxAgeMs: 172800000,

代码： timeoutSeconds: 60,

代码： },

代码： },

代码： },

代码： },

代码： }

注意事项：

- 当存在 API 密钥时，firecrawl.enabled 默认为 true。
- maxAgeMs 控制缓存结果可以保留多久（毫秒）。默认为 2 天。

隐身 / 机器人规避

Firecrawl 提供了一个用于机器人规避的代理模式参数（basic、stealth 或 auto）。

OpenClaw 对 Firecrawl 请求始终使用 proxy: "auto" 加 storeInCache: true。

如果省略 proxy，Firecrawl 默认使用 auto。auto 在基本尝试失败时会使用隐身代理重试，这可能比

仅使用基本抓取消耗更多积分。

`web\_fetch` 如何使用 Firecrawl

web_fetch 提取顺序：

- Readability（本地）
- Firecrawl（如果已配置）
- 基本 HTML 清理（最后回退）

参见 Web 工具 了解完整的 Web 工具设置。

13. 工具 (OpenClaw)

工具 (OpenClaw)

OpenClaw 为 browser、canvas、nodes 和 cron 暴露一流的智能体工具。

这些工具取代了旧的 openclaw- Skills：工具是类型化的，无需调用 shell，

智能体应该直接依赖它们。

禁用工具

你可以通过 openclaw.json 中的 tools.allow / tools.deny 全局允许/拒绝工具（deny 优先）。这会阻止不允许的工具被发送到模型提供商。

代码：{

代码：tools: { deny: ["browser"] },

代码：}

注意：

- 匹配不区分大小写。
- 支持 通配符（"*" 表示所有工具）。
- 如果 tools.allow 仅引用未知或未加载的插件工具名称，OpenClaw 会记录警告并忽略允许列表，以确保核心工具保持可用。

工具配置文件（基础允许列表）

tools.profile 在 tools.allow/tools.deny 之前设置基础工具允许列表。

按智能体覆盖：agents.list[].tools.profile。

配置文件：

- minimal：仅 session_status
- coding：group:fs、group:runtime、group:sessions、group:memory、image
- messaging：group:messaging、sessions_list、sessions_history、sessions_send、session_status
- full：无限制（与未设置相同）

示例（默认仅消息，同时允许 Slack + Discord 工具）：

代码：{

代码：tools: {

代码：profile: "messaging",

代码：allow: ["slack", "discord"],

代码：},

代码：}

示例（coding 配置文件，但在所有地方拒绝 exec/process）：

代码：{

代码：tools: {

代码：profile: "coding",

代码：deny: ["group:runtime"],

代码：},

代码：}

示例（全局 coding 配置文件，仅消息的支持智能体）：

代码：{

代码：tools: { profile: "coding" },

代码：agents: {

代码：list: [

代码：{

代码：id: "support",

代码：tools: { profile: "messaging", allow: ["slack"] },

代码： },

代码：],

代码： },

代码： }

特定提供商的工具策略

使用 `tools.byProvider` 为特定提供商（或单个 `provider/model`）进一步限制工具，而不更改你的全局默认值。

按智能体覆盖：`agents.list[].tools.byProvider`。

这在基础工具配置文件之后和允许/拒绝列表之前应用，因此它只能缩小工具集。

提供商键接受 `provider`（例如 `google-antigravity`）或 `provider/model`（例如 `openai/gpt-5.2`）。

示例（保持全局 `coding` 配置文件，但 Google Antigravity 使用最小工具）：

代码： {

代码： tools: {

代码： profile: "coding",

代码： byProvider: {

代码： "google-antigravity": { profile: "minimal" },

代码： },

代码： },

代码： }

示例（针对不稳定端点的 `provider/model` 特定允许列表）：

代码： {

代码： tools: {

代码： allow: ["group:fs", "group:runtime", "sessions_list"],

代码： byProvider: {

代码： "openai/gpt-5.2": { allow: ["group:fs", "sessions_list"] },

代码： },

代码： },

代码： }

示例（针对单个提供商的智能体特定覆盖）：

代码： {

代码： agents: {

代码： list: [

代码： {

代码： id: "support",

代码： tools: {

代码： byProvider: {

代码： "google-antigravity": { allow: ["message", "sessions_list"] },

代码： },

代码： },

代码： },

代码：],

代码： },

代码： }

工具组（简写）

工具策略（全局、智能体、沙箱）支持 group: 条目，它们会展开为多个工具。

在 tools.allow / tools.deny 中使用这些。

可用的组：

- group:runtime : exec 、 bash 、 process
- group:fs : read 、 write 、 edit 、 apply_patch
- group:sessions : sessions_list 、 sessions_history 、 sessions_send 、 sessions_spawn 、 session_status
- group:memory : memory_search 、 memory_get
- group:web : web_search 、 web_fetch
- group:ui : browser 、 canvas

- group:automation : cron 、 gateway
- group:messaging : message
- group:nodes : nodes
- group:openclaw : 所有内置 OpenClaw 工具（不包括提供商插件）

示例（仅允许文件工具 + browser）：

代码：{

代码： tools: {

代码： allow: ["group:fs", "browser"],

代码： },

代码： }

插件 + 工具

插件可以在核心集之外注册额外的工具（和 CLI 命令）。

参见插件了解安装 + 配置，以及 Skills 了解

工具使用指导如何被注入到提示中。一些插件随工具一起提供自己的 Skills

（例如，voice-call 插件）。

可选的插件工具：

- Lobster：带有可恢复审批的类型化工作流运行时（需要 Gateway 网关主机上的 Lobster CLI）。
- LLM Task：用于结构化工作流输出的 JSON-only LLM 步骤（可选 schema 验证）。

工具清单

`apply\_patch`

跨一个或多个文件应用结构化补丁。用于多块编辑。

实验性：通过 tools.exec.applyPatch.enabled 启用（仅 OpenAI 模型）。

`exec`

在工作区中运行 shell 命令。

核心参数：

- command (必需)
- yieldMs (超时后自动后台运行，默认 10000)
- background (立即后台运行)
- timeout (秒；超过则终止进程，默认 1800)
- elevated (布尔值；如果启用/允许提升模式，则在主机上运行；仅在智能体被沙箱隔离时改变行为)
- host (sandbox | gateway | node)
- security (deny | allowlist | full)
- ask (off | on-miss | always)
- node (host=node 时的节点 id/名称)
- 需要真正的 TTY？设置 pty: true。

注意：

- 后台运行时返回带有 sessionId 的 status: "running"。
- 使用 process 来轮询/日志/写入/终止/清除后台会话。
- 如果不允许 process，exec 会同步运行并忽略 yieldMs/background。
- elevated 受 tools.elevated 加上任何 agents.list[].tools.elevated 覆盖的门控（两者都必须允许），是 host=gateway + security=full 的别名。
- elevated 仅在智能体被沙箱隔离时改变行为（否则是空操作）。
- host=node 可以针对 macOS 配套应用或无头节点主机（openclaw node run）。
- Gateway 网关/节点审批和允许列表：执行审批。

`process`

管理后台 exec 会话。

核心操作：

- list、poll、log、write、kill、clear、remove

注意：

- poll 返回新输出，完成时返回退出状态。
- log 支持基于行的 offset/limit（省略 offset 以获取最后 N 行）。
- process 按智能体作用域；来自其他智能体的会话不可见。

`web\_search`

使用 Brave Search API 搜索网络。

核心参数：

- query (必需)
- count (1-10；默认来自 tools.web.search.maxResults)

注意：

- 需要 Brave API 密钥（推荐：openclaw configure --section web，或设置 BRAVE_API_KEY）。
- 通过 tools.web.search.enabled 启用。
- 响应被缓存（默认 15 分钟）。
- 参见 Web 工具 了解设置。

`web\_fetch`

从 URL 获取并提取可读内容（HTML → markdown/text）。

核心参数：

- url (必需)
- extractMode (markdown | text)
- maxChars (截断长页面)

注意：

- 通过 tools.web.fetch.enabled 启用。
- 响应被缓存（默认 15 分钟）。
- 对于 JS 密集型网站，优先使用 browser 工具。
- 参见 Web 工具 了解设置。
- 参见 Firecrawl 了解可选的反机器人回退。

`browser`

控制专用的 OpenClaw 管理的浏览器。

核心操作：

- status 、 start 、 stop 、 tabs 、 open 、 focus 、 close
- snapshot (aria/ai)
- screenshot (返回图像块 + MEDIA:<path>)
- act (UI 操作 : click/type/press/hover/drag/select/fill/resize/wait/evaluate)
- navigate 、 console 、 pdf 、 upload 、 dialog

配置文件管理 :

- profiles — 列出所有浏览器配置文件及其状态
- create-profile — 使用自动分配的端口 (或 cdpUrl) 创建新配置文件
- delete-profile — 停止浏览器, 删除用户数据, 从配置中移除 (仅本地)
- reset-profile — 终止配置文件端口上的孤儿进程 (仅本地)

常用参数 :

- profile (可选 ; 默认为 browser.defaultProfile)
- target (sandbox | host | node)
- node (可选 ; 选择特定的节点 id/名称)

注意 :

- 需要 browser.enabled=true (默认为 true ; 设置为 false 以禁用) 。
- 所有操作接受可选的 profile 参数以支持多实例。
- 当省略 profile 时, 使用 browser.defaultProfile (默认为"chrome") 。
- 配置文件名称 : 仅小写字母数字 + 连字符 (最多 64 字符) 。
- 端口范围 : 18800-18899 (最多约 100 个配置文件) 。
- 远程配置文件仅支持附加 (无 start/stop/reset) 。
- 如果连接了支持浏览器的节点, 工具可能会自动路由到它 (除非你固定了 target) 。
- 安装 Playwright 时 snapshot 默认为 ai ; 使用 aria 获取无障碍树。
- snapshot 还支持角色快照选项 (interactive 、 compact 、 depth 、 selector) , 返回像 e12 这样的引用。
- act 需要来自 snapshot 的 ref (AI 快照中的数字 12 , 或角色快照中的 e12) ; 对于罕见的 CSS 选择器需求使用 evaluate 。
- 默认避免 act → wait ; 仅在特殊情况下使用 (没有可靠的 UI 状态可等待) 。
- upload 可以选择性地传递 ref 以在准备后自动点击。

- upload 还支持 inputRef (aria 引用) 或 element (CSS 选择器) 以直接设置 <input type="file">。

`canvas`

驱动节点 Canvas (present、eval、snapshot、A2UI)。

核心操作：

- present、hide、navigate、eval
- snapshot (返回图像块 + MEDIA:<path>)
- a2ui_push、a2ui_reset

注意：

- 底层使用 Gateway 网关 node.invoke。
- 如果未提供 node，工具会选择默认值 (单个连接的节点或本地 mac 节点)。
- A2UI 仅限 v0.8 (无 createSurface)；CLI 会拒绝 v0.9 JSONL 并显示行错误。
- 快速冒烟测试：openclaw nodes canvas a2ui push --node <id> --text "Hello from A2UI"。

`nodes`

发现和定位配对的节点；发送通知；捕获摄像头/屏幕。

核心操作：

- status、describe
- pending、approve、reject (配对)
- notify (macOS system.notify)
- run (macOS system.run)
- camera_snap、camera_clip、screen_record
- location_get

注意：

- 摄像头/屏幕命令需要节点应用在前台。
- 图像返回图像块 + MEDIA:<path>。
- 视频返回 FILE:<path> (mp4)。

- 位置返回 JSON 负载 (lat/lon/accuracy/timestamp) 。
- run 参数：command argv 数组；可选的 cwd、env (KEY=VAL)、commandTimeoutMs、invokeTimeoutMs、needsScreenRecording。

示例 (run)：

代码：{

代码："action": "run",

代码："node": "office-mac",

代码："command": ["echo", "Hello"],

代码："env": ["FOO=bar"],

代码："commandTimeoutMs": 12000,

代码："invokeTimeoutMs": 45000,

代码："needsScreenRecording": false

代码：}

`image`

使用配置的图像模型分析图像。

核心参数：

- image (必需的路径或 URL)
- prompt (可选；默认为"Describe the image.")
- model (可选覆盖)
- maxBytesMb (可选大小上限)

注意：

- 仅在配置了 agents.defaults.imageModel (主要或回退) 时可用，或者当可以从你的默认模型 + 配置的认证推断出隐式图像模型时 (尽力配对)。
- 直接使用图像模型 (独立于主聊天模型)。

`message`

跨 Discord/Google Chat/Slack/Telegram/WhatsApp/Signal/iMessage/MS Teams 发送消息和渠道操作。

核心操作：

- send (文本 + 可选媒体；MS Teams 还支持用于 Adaptive Cards 的 card)
- poll (WhatsApp/Discord/MS Teams 投票)
- react / reactions / read / edit / delete
- pin / unpin / list-pins
- permissions
- thread-create / thread-list / thread-reply
- search
- sticker
- member-info / role-info
- emoji-list / emoji-upload / sticker-upload
- role-add / role-remove
- channel-info / channel-list
- voice-status
- event-list / event-create
- timeout / kick / ban

注意：

- send 通过 Gateway 网关路由 WhatsApp；其他渠道直接发送。
- poll 对 WhatsApp 和 MS Teams 使用 Gateway 网关；Discord 投票直接发送。
- 当消息工具调用绑定到活动聊天会话时，发送被限制到该会话的目标以避免跨上下文泄露。

`cron`

管理 Gateway 网关定时任务和唤醒。

核心操作：

- status 、 list
- add 、 update 、 remove 、 run 、 runs
- wake (入队系统事件 + 可选的立即心跳)

注意：

- add 期望完整的定时任务对象（与 cron.add RPC 相同的 schema）。
- update 使用 { id, patch }。

`gateway`

重启或对运行中的 Gateway 网关进程应用更新（就地）。

核心操作：

- restart（授权 + 发送 SIGUSR1 进行进程内重启；openclaw gateway 就地重启）
- config.get / config.schema
- config.apply（验证 + 写入配置 + 重启 + 唤醒）
- config.patch（合并部分更新 + 重启 + 唤醒）
- update.run（运行更新 + 重启 + 唤醒）

注意：

- 使用 delayMs（默认 2000）以避免中断进行中的回复。
- restart 默认禁用；使用 commands.restart: true 启用。

`sessions\_list` / `sessions\_history` / `sessions\_send` / `sessions\_spawn` / `session\_status`

列出会话，检查转录历史，或发送到另一个会话。

核心参数：

- sessions_list：kinds?、limit?、activeMinutes?、messageLimit?（0 = 无）
- sessions_history：sessionKey（或 sessionId） 、limit?、includeTools?
- sessions_send：sessionKey（或 sessionId） 、message、timeoutSeconds?（0 = fire-and-forget）
- sessions_spawn：task、label?、agentId?、model?、runTimeoutSeconds?、cleanup?
- session_status：sessionKey?（默认当前；接受 sessionId） 、model?（default 清除覆盖）

注意：

- main 是规范的私聊键；global/unknown 是隐藏的。
- messageLimit > 0 获取每个会话的最后 N 条消息（工具消息被过滤）。
- 当 timeoutSeconds > 0 时，sessions_send 等待最终完成。

- 递送/宣告发生在完成后，是尽力而为的；status: "ok" 确认智能体运行完成，而不是宣告已递送。
- sessions_spawn 启动子智能体运行并将宣告回复发送回请求者聊天。
- sessions_spawn 是非阻塞的，立即返回 status: "accepted"。
- sessions_send 运行回复往返乒乓（回复 REPLY_SKIP 以停止；最大轮次通过 session.agentToAgent.maxPingPongTurns，0-5）。
- 乒乓之后，目标智能体运行一个宣告步骤；回复 ANNOUNCE_SKIP 以抑制宣告。

`agents\_list`

列出当前会话可以用 sessions_spawn 定位的智能体 id。

注意：

- 结果受每智能体允许列表限制（agents.list[].subagents.allowAgents）。
- 当配置为 [""] 时，工具包含所有已配置的智能体并标记 allowAny: true。

参数（通用）

Gateway 网关支持的工具（canvas、nodes、cron）：

- gatewayUrl（默认 ws://127.0.0.1:18789）
- gatewayToken（如果启用了认证）
- timeoutMs

Browser 工具：

- profile（可选；默认为 browser.defaultProfile）
- target（sandbox | host | node）
- node（可选；固定特定的节点 id/名称）

推荐的智能体流程

浏览器自动化：

- browser → status / start
- snapshot（ai 或 aria）
- act（click/type/press）
- screenshot 如果你需要视觉确认

Canvas 渲染：

- canvas → present
- a2ui_push（可选）
- snapshot

节点定位：

- nodes → status
- 在选定的节点上 describe
- notify / run / camera_snap / screen_record

安全性

- 避免直接 system.run；仅在用户明确同意时使用 nodes → run。
- 尊重用户对摄像头/屏幕捕获的同意。
- 在调用媒体命令前使用 status/describe 确保权限。

工具如何呈现给智能体

工具通过两个并行渠道暴露：

- 系统提示文本：人类可读的列表 + 指导。
- 工具 schema：发送到模型 API 的结构化函数定义。

这意味着智能体同时看到"存在哪些工具"和"如何调用它们"。如果工具没有出现在系统提示或 schema 中，模型就无法调用它。

14. LLM 任务

LLM 任务

llm-task 是一个可选插件工具，用于运行纯 JSON 的 LLM 任务并返回结构化输出（可选择根据 JSON Schema 进行验证）。

这非常适合像 Lobster 这样的工作流引擎：你可以添加单个 LLM 步骤，而无需为每个工作流编写自定义 OpenClaw 代码。

启用插件

- 启用插件：

```
代码：{  
代码： "plugins": {  
代码： "entries": {  
代码： "llm-task": { "enabled": true }  
代码： }  
代码： }  
代码： }
```

- 将工具加入允许列表（它以 optional: true 注册）：

```
代码：{  
代码： "agents": {  
代码： "list": [  
代码： {  
代码： "id": "main",  
代码： "tools": { "allow": ["llm-task"] }  
代码： }  
代码： ]  
代码： }  
代码： }
```

配置（可选）

```
代码：{  
代码： "plugins": {  
代码： "entries": {  
代码： "llm-task": {  
代码： "enabled": true,  
代码： "config": {  
代码： "defaultProvider": "openai-codex",  
代码： "defaultModel": "gpt-5.2",  
代码： "defaultAuthProfileId": "main",
```

代码： "allowedModels": ["openai-codex/gpt-5.2"],

代码： "maxTokens": 800,

代码： "timeoutMs": 30000

代码： }

代码： }

代码： }

代码： }

代码： }

allowedModels 是 provider/model

字符串的允许列表。如果设置了该项，任何不在列表中的请求都会被拒绝。

工具参数

- prompt（字符串，必填）
- input（任意类型，可选）
- schema（对象，可选 JSON Schema）
- provider（字符串，可选）
- model（字符串，可选）
- authProfileId（字符串，可选）
- temperature（数字，可选）
- maxTokens（数字，可选）
- timeoutMs（数字，可选）

输出

返回 details.json，包含解析后的 JSON（如果提供了 schema，则会进行验证）。

示例：Lobster 工作流程步骤

代码： openclaw.invoke --tool llm-task --action json --args-json '{

代码： "prompt": "Given the input email, return intent and draft.",

代码： "input": {

代码： "subject": "Hello",

代码： "body": "Can you help?"


```
代码： },
代码： "schema": {
代码： "type": "object",
代码： "properties": {
代码： "intent": { "type": "string" },
代码： "draft": { "type": "string" }
代码： },
代码： "required": ["intent", "draft"],
代码： "additionalProperties": false
代码： }
代码： }
```

安全注意事项

- 该工具为纯 JSON 模式，指示模型仅输出 JSON（无代码围栏、无注释说明）。
- 此次运行不会向模型暴露任何工具。
- 除非使用 schema 进行验证，否则应将输出视为不可信。
- 在任何有副作用的步骤（发送、发布、执行）之前设置审批流程。

15. Lobster

Lobster

Lobster 是一个工作流外壳，让 OpenClaw 能够将多步骤工具序列作为单个确定性操作运行，并带有显式审批检查点。

亮点

你的助手可以构建管理自身的工具。请求一个工作流，30 分钟后你就有了一个 CLI 和作为单次调用运行的管道。Lobster 是缺失的那一块：确定性管道、显式审批和可恢复状态。

为什么

如今，复杂的工作流需要多次来回的工具调用。每次调用都消耗 token，LLM 必须编排每一步。Lobster 将这种编排移入类型化运行时：

- 一次调用代替多次：OpenClaw 运行一次 Lobster 工具调用并获得结构化结果。

- 内置审批：副作用（发送邮件、发布评论）会暂停工作流，直到明确批准。
- 可恢复：暂停的工作流返回一个令牌；批准并恢复而无需重新运行所有内容。

为什么用 DSL 而不是普通程序？

Lobster 故意很小。目标不是"一种新语言"，而是一个可预测的、AI 友好的管道规范，具有一流的审批和恢复令牌。

- 内置批准/恢复：普通程序可以提示人类，但它无法暂停和恢复并带有持久令牌，除非你自己发明那个运行时。
- 确定性 + 可审计性：管道是数据，所以它们易于记录、比较、重放和审查。
- AI 的受限表面：微小的语法 + JSON 管道减少了"创造性"代码路径，使验证变得现实可行。
- 内置安全策略：超时、输出上限、沙箱检查和白名单由运行时强制执行，而不是每个脚本。
- 仍然可编程：每个步骤都可以调用任何 CLI 或脚本。如果你想要 JS/TS，可以从代码生成 .lobster 文件。

工作原理

OpenClaw 以工具模式启动本地 lobster CLI，并从 stdout 解析 JSON 信封。

如果管道暂停等待审批，工具会返回一个 resumeToken，以便你稍后继续。

模式：小型 CLI + JSON 管道 + 审批

构建输出 JSON 的小命令，然后将它们链接成单个 Lobster 调用。（下面是示例命令名称——替换成你自己的。）

代码：inbox list --json

代码：inbox categorize --json

代码：inbox apply --json

代码：{

代码："action": "run",

代码："pipeline": "exec --json --shell 'inbox list --json' | exec --stdin json --shell 'inbox categorize --json' | exec --stdin json --shell 'inbox apply --json' | approve --preview-from-stdin --limit 5 --prompt 'Apply changes?',"

代码："timeoutMs": 30000

代码：}

如果管道请求审批，使用令牌恢复：

代码：{

代码： "action": "resume",

代码： "token": "<resumeToken>",

代码： "approve": true

代码：}

AI 触发工作流；Lobster 执行步骤。审批关卡使副作用显式且可审计。

示例：将输入项映射到工具调用：

代码：gog.gmail.search --query 'newer_than:1d' \

代码： | openclaw.invoke --tool message --action send --each --item-key message
--args-json '{"provider":"telegram","to":"..."}'

纯 JSON 的 LLM 步骤 (llm-task)

对于需要结构化 LLM 步骤的工作流，启用可选的

llm-task 插件工具并从 Lobster 调用它。这保持了工作流的

确定性，同时仍然允许你使用模型进行分类/摘要/起草。

启用工具：

代码：{

代码： "plugins": {

代码： "entries": {

代码： "llm-task": { "enabled": true }

代码： }

代码： },

代码： "agents": {

代码： "list": [

代码： {

代码： "id": "main",

代码： "tools": { "allow": ["llm-task"] }

代码： }

代码：]

代码： }

代码： }

在管道中使用它：

代码： openclaw.invoke --tool llm-task --action json --args-json '{

代码： "prompt": "Given the input email, return intent and draft.",

代码： "input": { "subject": "Hello", "body": "Can you help?" },

代码： "schema": {

代码： "type": "object",

代码： "properties": {

代码： "intent": { "type": "string" },

代码： "draft": { "type": "string" }

代码： },

代码： "required": ["intent", "draft"],

代码： "additionalProperties": false

代码： }

代码： }'

参见 LLM Task 了解详情和配置选项。

工作流文件 (.lobster)

Lobster 可以运行包含 name、args、steps、env、condition 和 approval 字段的 YAML/JSON 工作流文件。在 OpenClaw 工具调用中，将 pipeline 设置为文件路径。

代码： name: inbox-triage

代码： args:

代码： tag:

代码： default: "family"

代码： steps:

代码： - id: collect

代码： command: inbox list --json

代码：-id: categorize

代码：command: inbox categorize --json

代码：stdin: \$collect.stdout

代码：-id: approve

代码：command: inbox apply --approve

代码：stdin: \$categorize.stdout

代码：approval: required

代码：-id: execute

代码：command: inbox apply --execute

代码：stdin: \$categorize.stdout

代码：condition: \$approve.approved

注意事项：

- stdin: \$step.stdout 和 stdin: \$step.json 传递前一步骤的输出。
- condition（或 when）可以根据 \$step.approved 控制步骤。

安装 Lobster

在运行 OpenClaw Gateway 网关的同一主机上安装 Lobster CLI（参见 Lobster 仓库），并确保 lobster 在 PATH 中。

如果你想使用自定义二进制位置，在工具调用中传递绝对路径 lobsterPath。

启用工具

Lobster 是一个可选的插件工具（默认未启用）。

推荐（附加，安全）：

代码：{

代码："tools": {

代码："alsoAllow": ["lobster"]

代码：}

代码：}

或每个智能体：

```
代码：{
代码： "agents": {
代码： "list": [
代码： {
代码： "id": "main",
代码： "tools": {
代码： "alsoAllow": ["lobster"]
代码： }
代码： }
代码： ]
代码： }
代码： }
```

避免使用 `tools.allow: ["lobster"]`，除非你打算在限制性白名单模式下运行。

注意：白名单对于可选插件是自愿加入的。如果你的白名单只包含插件工具（如 lobster），OpenClaw 会保持核心工具启用。要限制核心工具，也要在白名单中包含你想要的核心工具或组。

示例：邮件分类

不使用 Lobster：

```
代码：用户："检查我的邮件并起草回复"
代码：→ openclaw 调用 gmail.list
代码：→ LLM 总结
代码：→ 用户："给 #2 和 #5 起草回复"
代码：→ LLM 起草
代码：→ 用户："发送 #2"
代码：→ openclaw 调用 gmail.send
代码：（每天重复，不记得已分类的内容）
```

使用 Lobster：

```
代码：{
```

代码： "action": "run",

代码： "pipeline": "email.triage --limit 20",

代码： "timeoutMs": 30000

代码： }

返回一个 JSON 信封（已截断）：

代码： {

代码： "ok": true,

代码： "status": "needs_approval",

代码： "output": [{ "summary": "5 need replies, 2 need action" }],

代码： "requiresApproval": {

代码： "type": "approval_request",

代码： "prompt": "Send 2 draft replies?",

代码： "items": [],

代码： "resumeToken": "..."

代码： }

代码： }

用户批准 → 恢复：

代码： {

代码： "action": "resume",

代码： "token": "<resumeToken>",

代码： "approve": true

代码： }

一个工作流。确定性。安全。

工具参数

`run`

以工具模式运行管道。

代码： {

代码： "action": "run",

代码： "pipeline": "gog.gmail.search --query 'newer_than:1d' | email.triage",

代码： "cwd": "/path/to/workspace",

代码： "timeoutMs": 30000,

代码： "maxStdoutBytes": 512000

代码： }

使用参数运行 workflow 文件：

代码： {

代码： "action": "run",

代码： "pipeline": "/path/to/inbox-triage.lobster",

代码： "argsJson": "{ \"tag\": \"family\" }"

代码： }

`resume`

在审批后继续暂停的工作流。

代码： {

代码： "action": "resume",

代码： "token": "<resumeToken>",

代码： "approve": true

代码： }

可选输入

- lobsterPath：Lobster 二进制文件的绝对路径（省略则使用 PATH）。
- cwd：管道的工作目录（默认为当前进程工作目录）。
- timeoutMs：如果子进程超过此持续时间则终止（默认：20000）。
- maxStdoutBytes：如果 stdout 超过此大小则终止子进程（默认：512000）。
- argsJson：传递给 lobster run --args-json 的 JSON 字符串（仅限 workflow 文件）。

输出信封

Lobster 返回一个具有三种状态之一的 JSON 信封：

- ok → 成功完成
- needs_approval → 已暂停；需要 requiresApproval.resumeToken 才能恢复
- cancelled → 明确拒绝或取消

工具在 content（格式化 JSON）和 details（原始对象）中都显示信封。

审批

如果存在 requiresApproval，检查提示并决定：

- approve: true → 恢复并继续副作用
- approve: false → 取消并终结 workflow

使用 `approve --preview-from-stdin --limit N` 将 JSON 预览附加到审批请求，无需自定义 jq/heredoc 粘合代码。恢复令牌现在很紧凑：Lobster 在其状态目录下存储 workflow 恢复状态，并返回一个小令牌键。

OpenProse

OpenProse 与 Lobster 配合良好：使用 /prose 编排多智能体准备，然后运行 Lobster 管道进行确定性审批。如果 Prose 程序需要 Lobster，通过 `tools.subagents.tools` 为子智能体允许 lobster 工具。参见 OpenProse。

安全

- 仅限本地子进程 — 插件本身不进行网络调用。
- 无密钥 — Lobster 不管理 OAuth；它调用管理 OAuth 的 OpenClaw 工具。
- 沙箱感知 — 当工具上下文处于沙箱隔离状态时禁用。
- 加固 — 如果指定，lobsterPath 必须是绝对路径；强制执行超时和输出上限。

故障排除

- lobster subprocess timed out → 增加 timeoutMs，或拆分长管道。
- lobster output exceeded maxStdoutBytes → 提高 maxStdoutBytes 或减少输出大小。
- lobster returned invalid JSON → 确保管道以工具模式运行并只打印 JSON。
- lobster failed (code ...) → 在终端中运行相同的管道以检查 stderr。

了解更多

- 插件
- 插件工具开发

案例研究：社区工作流

一个公开示例：一个"第二大脑" CLI + Lobster 管道，管理三个 Markdown 库（个人、伴侣、共享）。CLI 为统计、收件箱列表和过时扫描输出 JSON；Lobster 将这些命令链接成 weekly-review、inbox-triage、memory-consolidation 和 shared-task-sync 等工作流，每个都有审批关卡。AI 在可用时处理判断（分类），不可用时回退到确定性规则。

- 帖子：
- 仓库：

16. 多智能体沙箱与工具配置

多智能体沙箱与工具配置

概述

多智能体设置中的每个智能体现在可以拥有自己的：

- 沙箱配置（`agents.list[].sandbox` 覆盖 `agents.defaults.sandbox`）
- 工具限制（`tools.allow` / `tools.deny`，以及 `agents.list[].tools`）

这允许你运行具有不同安全配置文件的多个智能体：

- 具有完全访问权限的个人助手
- 具有受限工具的家庭/工作智能体
- 在沙箱中运行的面向公众的智能体

`setupCommand` 属于 `sandbox.docker` 下（全局或按智能体），在容器创建时运行一次。

认证是按智能体的：每个智能体从其自己的 `agentDir` 认证存储读取：

代码：`~/.openclaw/agents/<agentId>/agent/auth-profiles.json`

凭证不会在智能体之间共享。切勿在智能体之间重用 `agentDir`。

如果你想共享凭证，请将 `auth-profiles.json` 复制到其他智能体的 `agentDir` 中。

有关沙箱隔离在运行时的行为，请参见沙箱隔离。

有关调试"为什么这被阻止了？"，请参见沙箱 vs 工具策略 vs 提权 和 openclaw sandbox explain。

配置示例

示例 1：个人 + 受限家庭智能体

代码：{

代码： "agents": {

代码： "list": [

代码： {

代码： "id": "main",

代码： "default": true,

代码： "name": "Personal Assistant",

代码： "workspace": "~/.openclaw/workspace",

代码： "sandbox": { "mode": "off" }

代码： },

代码： {

代码： "id": "family",

代码： "name": "Family Bot",

代码： "workspace": "~/.openclaw/workspace-family",

代码： "sandbox": {

代码： "mode": "all",

代码： "scope": "agent"

代码： },

代码： "tools": {

代码： "allow": ["read"],

代码： "deny": ["exec", "write", "edit", "apply_patch", "process", "browser"]

代码： }

代码： }

代码：]

```
代码： },
代码： "bindings": [
代码： {
代码： "agentId": "family",
代码： "match": {
代码： "provider": "whatsapp",
代码： "accountId": "*",
代码： "peer": {
代码： "kind": "group",
代码： "id": "120363424282127706@g.us"
代码： }
代码： }
代码： }
代码： ]
代码： }
```

结果：

- main 智能体：在主机上运行，完全工具访问
- family 智能体：在 Docker 中运行（每个智能体一个容器），仅有 read 工具

示例 2：具有共享沙箱的工作智能体

```
代码： {
代码： "agents": {
代码： "list": [
代码： {
代码： "id": "personal",
代码： "workspace": "~/openclaw/workspace-personal",
代码： "sandbox": { "mode": "off" }
代码： },
代码： {
```

```
代码 : "id": "work",
代码 : "workspace": "~/.openclaw/workspace-work",
代码 : "sandbox": {
代码 : "mode": "all",
代码 : "scope": "shared",
代码 : "workspaceRoot": "/tmp/work-sandboxes"
代码 : },
代码 : "tools": {
代码 : "allow": ["read", "write", "apply_patch", "exec"],
代码 : "deny": ["browser", "gateway", "discord"]
代码 : }
代码 : }
代码 : ]
代码 : }
代码 : }
```

示例 2b：全局编码配置文件 + 仅消息智能体

```
代码 : {
代码 : "tools": { "profile": "coding" },
代码 : "agents": {
代码 : "list": [
代码 : {
代码 : "id": "support",
代码 : "tools": { "profile": "messaging", "allow": ["slack"] }
代码 : }
代码 : ]
代码 : }
代码 : }
```

结果：

- 默认智能体获得编码工具
- support 智能体仅用于消息 (+ Slack 工具)

示例 3：每个智能体不同的沙箱模式

```
代码：{
代码： "agents": {
代码： "defaults": {
代码： "sandbox": {
代码： "mode": "non-main", // 全局默认
代码： "scope": "session"
代码： }
代码： },
代码： "list": [
代码： {
代码： "id": "main",
代码： "workspace": "~/.openclaw/workspace",
代码： "sandbox": {
代码： "mode": "off" // 覆盖：main 永不沙箱隔离
代码： }
代码： },
代码： {
代码： "id": "public",
代码： "workspace": "~/.openclaw/workspace-public",
代码： "sandbox": {
代码： "mode": "all", // 覆盖：public 始终沙箱隔离
代码： "scope": "agent"
代码： },
代码： "tools": {
代码： "allow": ["read"],
```

代码： "deny": ["exec", "write", "edit", "apply_patch"]

代码： }

代码： }

代码：]

代码： }

代码： }

配置优先级

当全局（agents.defaults.）和智能体特定（agents.list[.]）配置同时存在时：

沙箱配置

智能体特定设置覆盖全局：

代码： agents.list[.].sandbox.mode > agents.defaults.sandbox.mode

代码： agents.list[.].sandbox.scope > agents.defaults.sandbox.scope

代码： agents.list[.].sandbox.workspaceRoot > agents.defaults.sandbox.workspaceRoot

代码： agents.list[.].sandbox.workspaceAccess >
agents.defaults.sandbox.workspaceAccess

代码： agents.list[.].sandbox.docker.* > agents.defaults.sandbox.docker.*

代码： agents.list[.].sandbox.browser.* > agents.defaults.sandbox.browser.*

代码： agents.list[.].sandbox.prune.* > agents.defaults.sandbox.prune.*

注意事项：

- agents.list[.].sandbox.{docker,browser,prune}. 为该智能体覆盖 agents.defaults.sandbox.{docker,browser,prune}. （当沙箱 scope 解析为 "shared" 时忽略）。

工具限制

过滤顺序是：

- 工具配置文件（tools.profile 或 agents.list[.].tools.profile）

- 提供商工具配置文件 (`tools.byProvider[provider].profile` 或 `agents.list[].tools.byProvider[provider].profile`)
- 全局工具策略 (`tools.allow / tools.deny`)
- 提供商工具策略 (`tools.byProvider[provider].allow/deny`)
- 智能体特定工具策略 (`agents.list[].tools.allow/deny`)
- 智能体提供商策略 (`agents.list[].tools.byProvider[provider].allow/deny`)
- 沙箱工具策略 (`tools.sandbox.tools` 或 `agents.list[].tools.sandbox.tools`)
- 子智能体工具策略 (`tools.subagents.tools` , 如适用)

每个级别可以进一步限制工具，但不能恢复之前级别拒绝的工具。

如果设置了 `agents.list[].tools.sandbox.tools`，它将替换该智能体的 `tools.sandbox.tools`。

如果设置了 `agents.list[].tools.profile`，它将覆盖该智能体的 `tools.profile`。

提供商工具键接受 `provider` (例如 `google-antigravity`) 或 `provider/model` (例如 `openai/gpt-5.2`)。

工具组 (简写)

工具策略 (全局、智能体、沙箱) 支持 `group:` 条目，可扩展为多个具体工具：

- `group:runtime` : `exec`、`bash`、`process`
- `group:fs` : `read`、`write`、`edit`、`apply_patch`
- `group:sessions` : `sessions_list`、`sessions_history`、`sessions_send`、`sessions_spawn`、`session_status`
- `group:memory` : `memory_search`、`memory_get`
- `group:ui` : `browser`、`canvas`
- `group:automation` : `cron`、`gateway`
- `group:messaging` : `message`
- `group:nodes` : `nodes`
- `group:openclaw` : 所有内置 OpenClaw 工具 (不包括提供商插件)

提权模式

`tools.elevated` 是全局基线 (基于发送者的允许列表)。`agents.list[].tools.elevated` 可以为特定智能体进一步限制提权 (两者都必须允许)。

缓解模式：

- 为不受信任的智能体拒绝 exec (agents.list[].tools.deny: ["exec"])
- 避免将发送者加入允许列表后路由到受限智能体
- 如果你只想要沙箱隔离执行，全局禁用提权 (tools.elevated.enabled: false)
- 为敏感配置文件按智能体禁用提权 (agents.list[].tools.elevated.enabled: false)

从单智能体迁移

之前（单智能体）：

```
代码：{
代码： "agents": {
代码： "defaults": {
代码： "workspace": "~/.openclaw/workspace",
代码： "sandbox": {
代码： "mode": "non-main"
代码： }
代码： }
代码： },
代码： "tools": {
代码： "sandbox": {
代码： "tools": {
代码： "allow": ["read", "write", "apply_patch", "exec"],
代码： "deny": []
代码： }
代码： }
代码： }
代码： }
```

之后（具有不同配置文件的多智能体）：

```
代码：{
代码： "agents": {
```

```
代码： "list": [  
代码： {  
代码： "id": "main",  
代码： "default": true,  
代码： "workspace": "~/.openclaw/workspace",  
代码： "sandbox": { "mode": "off" }  
代码： }  
代码： ]  
代码： }  
代码： }
```

旧版 agent. 配置由 openclaw doctor 迁移；今后请优先使用 agents.defaults + agents.list。

工具限制示例

只读智能体

```
代码： {  
代码： "tools": {  
代码： "allow": ["read"],  
代码： "deny": ["exec", "write", "edit", "apply_patch", "process"]  
代码： }  
代码： }
```

安全执行智能体（无文件修改）

```
代码： {  
代码： "tools": {  
代码： "allow": ["read", "exec", "process"],  
代码： "deny": ["write", "edit", "apply_patch", "browser", "gateway"]  
代码： }  
代码： }
```

仅通信智能体

代码：{

代码： "tools": {

代码： "allow": ["sessions_list", "sessions_send", "sessions_history", "session_status"],

代码： "deny": ["exec", "write", "edit", "apply_patch", "read", "browser"]

代码： }

代码： }

常见陷阱："non-main"

agents.defaults.sandbox.mode: "non-main" 基于 session.mainKey（默认 "main"），

而不是智能体 id。群组/渠道会话始终获得自己的键，因此它们

被视为非 main 并将被沙箱隔离。如果你希望智能体永不

沙箱隔离，请设置 agents.list[].sandbox.mode: "off"。

测试

配置多智能体沙箱和工具后：

- 检查智能体解析：

代码： openclaw agents list --bindings

- 验证沙箱容器：

代码： docker ps --filter "name=openclaw-sbx-"

- 测试工具限制：
- 发送需要受限工具的消息
- 验证智能体无法使用被拒绝的工具
- 监控日志：

代码： tail -f "\${OPENCLAW_STATE_DIR:-\$HOME/.openclaw}/logs/gateway.log" | grep -E "routing|sandbox|tools"

故障排除

尽管设置了 `mode: "all"` 但智能体未被沙箱隔离

- 检查是否有全局 `agents.defaults.sandbox.mode` 覆盖它
- 智能体特定配置优先，因此设置 `agents.list[].sandbox.mode: "all"`

尽管有拒绝列表但工具仍然可用

- 检查工具过滤顺序：全局 → 智能体 → 沙箱 → 子智能体
- 每个级别只能进一步限制，不能恢复
- 通过日志验证：`[tools] filtering tools for agent:${agentId}`

容器未按智能体隔离

- 在智能体特定沙箱配置中设置 `scope: "agent"`
- 默认是 "session"，每个会话创建一个容器

另请参阅

- 多智能体路由
- 沙箱配置
- 会话管理

17. 插件（扩展）

插件（扩展）

快速开始（插件新手？）

插件只是一个小型代码模块，用额外功能（命令、工具和 Gateway 网关 RPC）扩展 OpenClaw。

大多数时候，当你想要一个尚未内置到核心 OpenClaw 的功能（或你想将可选功能排除在主安装之外）时，你会使用插件。

快速路径：

- 查看已加载的内容：

代码：openclaw plugins list

- 安装官方插件（例如：Voice Call）：

代码：openclaw plugins install @openclaw/voice-call

- 重启 Gateway 网关，然后在 plugins.entries.<id>.config 下配置。

参见 Voice Call 了解具体的插件示例。

可用插件（官方）

- 从 2026.1.15 起 Microsoft Teams 仅作为插件提供；如果使用 Teams，请安装 @openclaw/msteams。
- Memory (Core) — 捆绑的记忆搜索插件（通过 plugins.slots.memory 默认启用）
- Memory (LanceDB) — 捆绑的长期记忆插件（自动召回/捕获；设置 plugins.slots.memory = "memory-lancedb"）
- Voice Call — @openclaw/voice-call
- Zalo Personal — @openclaw/zalouser
- Matrix — @openclaw/matrix
- Nostr — @openclaw/nostr
- Zalo — @openclaw/zalo
- Microsoft Teams — @openclaw/msteams
- Google Antigravity OAuth（提供商认证）— 作为 google-antigravity-auth 捆绑（默认禁用）
- Gemini CLI OAuth（提供商认证）— 作为 google-gemini-cli-auth 捆绑（默认禁用）
- Qwen OAuth（提供商认证）— 作为 qwen-portal-auth 捆绑（默认禁用）
- Copilot Proxy（提供商认证）— 本地 VS Code Copilot Proxy 桥接；与内置 github-copilot 设备登录不同（捆绑，默认禁用）

OpenClaw 插件是通过 jiti 在运行时加载的 TypeScript

模块。配置验证不会执行插件代码；它使用插件清单和 JSON Schema。参见 插件清单。

插件可以注册：

- Gateway 网关 RPC 方法

- Gateway 网关 HTTP 处理程序
- 智能体工具
- CLI 命令
- 后台服务
- 可选的配置验证
- Skills（通过在插件清单中列出 skills 目录）
- 自动回复命令（不调用 AI 智能体即可执行）

插件与 Gateway 网关在同一进程中运行，因此将它们视为受信任的代码。

工具编写指南：插件智能体工具。

运行时辅助工具

插件可以通过 `api.runtime` 访问选定的核心辅助工具。对于电话 TTS：

代码：`const result = await api.runtime.tts.textToSpeechTelephony({`

代码：`text: "Hello from OpenClaw",`

代码：`cfg: api.config,`

代码：`});`

注意事项：

- 使用核心 `messages.tts` 配置（OpenAI 或 ElevenLabs）。
- 返回 PCM 音频缓冲区 + 采样率。插件必须为提供商重新采样/编码。
- Edge TTS 不支持电话。

发现和优先级

OpenClaw 按顺序扫描：

- 配置路径
- `plugins.load.paths`（文件或目录）
- 工作区扩展
- `<workspace>/openclaw/extensions/.ts`
- `<workspace>/openclaw/extensions//index.ts`

- 全局扩展
- `~/openclaw/extensions/.ts`
- `~/openclaw/extensions//index.ts`
- 捆绑扩展（随 OpenClaw 一起发布，默认禁用）
- `<openclaw>/extensions/`

捆绑插件必须通过 `plugins.entries.<id>.enabled` 或 `openclaw plugins enable <id>` 显式启用。已安装的插件默认启用，但可以用相同方式禁用。

每个插件必须在其根目录中包含 `openclaw.plugin.json` 文件。如果路径指向文件，则插件根目录是文件的目录，必须包含清单。

如果多个插件解析到相同的 id，上述顺序中的第一个匹配项获胜，较低优先级的副本被忽略。

包集合

插件目录可以包含带有 `openclaw.extensions` 的 `package.json`：

```
代码：{
代码： "name": "my-pack",
代码： "openclaw": {
代码： "extensions": [".src/safety.ts", ".src/tools.ts"]
代码： }
代码： }
```

每个条目成为一个插件。如果包列出多个扩展，插件 id 变为 `name/<fileBase>`。

如果你的插件导入 npm 依赖，请在该目录中安装它们以便 `node_modules` 可用（`npm install / pnpm install`）。

渠道目录元数据

渠道插件可以通过 `openclaw.channel` 广播新手引导元数据，通过 `openclaw.install` 广播安装提示。这使核心目录保持无数据。

示例：

```
代码：{
```

```

代码： "name": "@openclaw/nextcloud-talk",
代码： "openclaw": {
代码： "extensions": ["/index.ts"],
代码： "channel": {
代码： "id": "nextcloud-talk",
代码： "label": "Nextcloud Talk",
代码： "selectionLabel": "Nextcloud Talk (self-hosted)",
代码： "docsPath": "/channels/nextcloud-talk",
代码： "docsLabel": "nextcloud-talk",
代码： "blurb": "Self-hosted chat via Nextcloud Talk webhook bots.",
代码： "order": 65,
代码： "aliases": ["nc-talk", "nc"]
代码： },
代码： "install": {
代码： "npmSpec": "@openclaw/nextcloud-talk",
代码： "localPath": "extensions/nextcloud-talk",
代码： "defaultChoice": "npm"
代码： }
代码： }
代码： }

```

OpenClaw 还可以合并外部渠道目录（例如，MPM 注册表导出）。将 JSON 文件放在以下位置之一：

- ~/.openclaw/mpm/plugins.json
- ~/.openclaw/mpm/catalog.json
- ~/.openclaw/plugins/catalog.json

或将 OPENCLAW_PLUGIN_CATALOG_PATHS（或 OPENCLAW_MPM_CATALOG_PATHS）指向一个或多个 JSON 文件（逗号/分号/PATH 分隔）。每个文件应包含 { "entries": [{ "name": "@scope/pkg", "openclaw": { "channel": {...}, "install": {...} } }] }。

插件 ID

默认插件 id :

- 包集合 : package.json 的 name
- 独立文件 : 文件基本名称 (~../voice-call.ts → voice-call)

如果插件导出 id , OpenClaw 会使用它 , 但当它与配置的 id 不匹配时会发出警告。

配置

代码 : {

代码 : plugins: {

代码 : enabled: true,

代码 : allow: ["voice-call"],

代码 : deny: ["untrusted-plugin"],

代码 : load: { paths: ["~/Projects/oss/voice-call-extension"] },

代码 : entries: {

代码 : "voice-call": { enabled: true, config: { provider: "twilio" } },

代码 : },

代码 : },

代码 : }

字段 :

- enabled : 主开关 (默认 : true)
- allow : 允许列表 (可选)
- deny : 拒绝列表 (可选 ; deny 优先)
- load.paths : 额外的插件文件/目录
- entries.<id> : 每个插件的开关 + 配置

配置更改需要重启 Gateway 网关。

验证规则 (严格) :

- entries 、 allow 、 deny 或 slots 中的未知插件 id 是错误。
- 未知的 channels.<id> 键是错误 , 除非插件清单声明了渠道 id 。
- 插件配置使用嵌入在 openclaw.plugin.json (configSchema) 中的 JSON Schema 进行验证。

- 如果插件被禁用，其配置会保留并发出警告。

插件槽位（独占类别）

某些插件类别是独占的（一次只有一个活跃）。使用 `plugins.slots` 选择哪个插件拥有该槽位：

代码：{

代码： plugins: {

代码： slots: {

代码： memory: "memory-core", // or "none" to disable memory plugins

代码： },

代码： },

代码： }

如果多个插件声明 `kind: "memory"`，只有选定的那个加载。其他的被禁用并带有诊断信息。

控制界面（schema + 标签）

控制界面使用 `config.schema`（JSON Schema + `uiHints`）来渲染更好的表单。

OpenClaw 在运行时根据发现的插件增强 `uiHints`：

- 为 `plugins.entries.<id> / .enabled / .config` 添加每插件标签
- 在以下位置合并可选的插件提供的配置字段提示：

`plugins.entries.<id>.config.<field>`

如果你希望插件配置字段显示良好的标签/占位符（并将密钥标记为敏感），请在插件清单中提供 `uiHints` 和 JSON Schema。

示例：

代码：{

代码： "id": "my-plugin",

代码： "configSchema": {

代码： "type": "object",

代码： "additionalProperties": false,

代码： "properties": {

代码： "apiKey": { "type": "string" },

代码： "region": { "type": "string" }

代码： }

代码： },

代码： "uiHints": {

代码： "apiKey": { "label": "API Key", "sensitive": true },

代码： "region": { "label": "Region", "placeholder": "us-east-1" }

代码： }

代码： }

CLI

代码： openclaw plugins list

代码： openclaw plugins info <id>

代码： openclaw plugins install <path> # copy a local file/dir into
~/.openclaw/extensions/<id>

代码： openclaw plugins install ./extensions/voice-call # relative path ok

代码： openclaw plugins install ./plugin.tgz # install from a local tarball

代码： openclaw plugins install ./plugin.zip # install from a local zip

代码： openclaw plugins install -l ./extensions/voice-call # link (no copy) for dev

代码： openclaw plugins install @openclaw/voice-call # install from npm

代码： openclaw plugins update <id>

代码： openclaw plugins update --all

代码： openclaw plugins enable <id>

代码： openclaw plugins disable <id>

代码： openclaw plugins doctor

plugins update 仅适用于在 plugins.installs 下跟踪的 npm 安装。

插件也可以注册自己的顶级命令（例如：openclaw voicecall）。

插件 API（概述）

插件导出以下之一：

- 函数：(api) => { ... }

- 对象：{ id, name, configSchema, register(api) { ... } }

插件钩子

插件可以附带钩子并在运行时注册它们。这让插件可以捆绑事件驱动的自动化，而无需单独安装钩子包。

示例

代码：registerPluginHooksFromDir(api, "./hooks");

代码：}

注意事项：

- 钩子目录遵循正常的钩子结构（HOOK.md + handler.ts）。
- 钩子资格规则仍然适用（操作系统/二进制文件/环境/配置要求）。
- 插件管理的钩子在 openclaw hooks list 中显示为 plugin:<id>。
- 你 cannot 通过 openclaw hooks 启用/禁用插件管理的钩子；而是启用/禁用插件。

提供商插件（模型认证）

插件可以注册模型提供商认证流程，以使用户可以在 OpenClaw 内运行 OAuth 或 API 密钥设置（无需外部脚本）。

通过 api.registerProvider(...) 注册提供商。每个提供商暴露一个或多个认证方法（OAuth、API 密钥、设备码等）。这些方法驱动：

- openclaw models auth login --provider <id> [--method <id>]

示例：

代码：api.registerProvider({

代码：id: "acme",

代码：label: "AcmeAI",

代码：auth: [

代码：{

代码：id: "oauth",

代码：label: "OAuth",

代码：kind: "oauth",

```

代码： run: async (ctx) => {
代码： // Run OAuth flow and return auth profiles.
代码： return {
代码： profiles: [
代码： {
代码： profileId: "acme:default",
代码： credential: {
代码： type: "oauth",
代码： provider: "acme",
代码： access: "...",
代码： refresh: "...",
代码： expires: Date.now() + 3600 * 1000,
代码： },
代码： },
代码： ],
代码： defaultModel: "acme/opus-1",
代码： };
代码： },
代码： },
代码： ],
代码： });

```

注意事项：

- run 接收带有 prompter、runtime、openUrl 和 oauth.createVpsAwareHandlers 辅助工具的 ProviderAuthContext。
- 当需要添加默认模型或提供商配置时返回 configPatch。
- 返回 defaultModel 以便 --set-default 可以更新智能体默认值。

注册消息渠道

插件可以注册渠道插件，其行为类似于内置渠道（WhatsApp、Telegram 等）。渠道配置位于 channels.<id> 下，由你的渠道插件代码验证。

```

代码：const myChannel = {
代码： id: "acmechat",
代码： meta: {
代码： id: "acmechat",
代码： label: "AcmeChat",
代码： selectionLabel: "AcmeChat (API)",
代码： docsPath: "/channels/acmechat",
代码： blurb: "demo channel plugin.",
代码： aliases: ["acme"],
代码： },
代码： capabilities: { chatTypes: ["direct"] },
代码： config: {
代码： listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
代码： resolveAccount: (cfg, accountId) =>
代码： cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
代码： accountId,
代码： },
代码： },
代码： outbound: {
代码： deliveryMode: "direct",
代码： sendText: async () => ({ ok: true }),
代码： },
代码： };

代码： api.registerChannel({ plugin: myChannel });
代码： }

```

注意事项：

- 将配置放在 channels.<id> 下（而不是 plugins.entries）。
- meta.label 用于 CLI/UI 列表中的标签。
- meta.aliases 添加用于规范化和 CLI 输入的备用 id。

- meta.preferOver 列出当两者都配置时要跳过自动启用的渠道 id。
- meta.detailLabel 和 meta.systemImage 让 UI 显示更丰富的渠道标签/图标。

编写新的消息渠道（分步指南）

当你想要一个新的聊天界面（“消息渠道”）而不是模型提供商时使用此方法。

模型提供商文档位于 /providers/ 下。

- 选择 id + 配置结构
- 所有渠道配置位于 channels.<id> 下。
- 对于多账户设置，优先使用 channels.<id>.accounts.<accountId>。
- 定义渠道元数据
- meta.label、meta.selectionLabel、meta.docsPath、meta.blurb 控制 CLI/UI 列表。
- meta.docsPath 应指向像 /channels/<id> 这样的文档页面。
- meta.preferOver 让插件替换另一个渠道（自动启用优先选择它）。
- meta.detailLabel 和 meta.systemImage 被 UI 用于详细文本/图标。
- 实现必需的适配器
- config.listAccountIds + config.resolveAccount
- capabilities（聊天类型、媒体、线程等）
- outbound.deliveryMode + outbound.sendText（用于基本发送）
- 根据需要添加可选适配器
- setup（向导）、security（私信策略）、status（健康/诊断）
- gateway（启动/停止/登录）、mentions、threading、streaming
- actions（消息操作）、commands（原生命令行为）
- 在插件中注册渠道
- api.registerChannel({ plugin })

最小配置示例：

代码：{

代码： channels: {

```
代码 : acmechat: {  
代码 : accounts: {  
代码 : default: { token: "ACME_TOKEN", enabled: true },  
代码 : },  
代码 : },  
代码 : },  
代码 : }
```

最小渠道插件（仅出站）：

```
代码 : const plugin = {  
代码 : id: "acmechat",  
代码 : meta: {  
代码 : id: "acmechat",  
代码 : label: "AcmeChat",  
代码 : selectionLabel: "AcmeChat (API)",  
代码 : docsPath: "/channels/acmechat",  
代码 : blurb: "AcmeChat messaging channel.",  
代码 : aliases: ["acme"],  
代码 : },  
代码 : capabilities: { chatTypes: ["direct"] },  
代码 : config: {  
代码 : listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),  
代码 : resolveAccount: (cfg, accountId) =>  
代码 : cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {  
代码 : accountId,  
代码 : },  
代码 : },  
代码 : outbound: {  
代码 : deliveryMode: "direct",  
代码 : sendText: async ({ text }) => {  
代码 : // deliver `text` to your channel here
```


代码： return { ok: true };

代码： },

代码： },

代码： };

代码： api.registerChannel({ plugin });

代码： }

加载插件（扩展目录或 plugins.load.paths），重启 Gateway 网关，然后在配置中配置 channels.<id>。

智能体工具

参见专门指南：插件智能体工具。

注册 Gateway 网关 RPC 方法

代码： api.registerGatewayMethod("myplugin.status", ({ respond }) => {

代码： respond(true, { ok: true });

代码： });

代码： }

注册 CLI 命令

代码： api.registerCli(

代码： ({ program }) => {

代码： program.command("mycmd").action(() => {

代码： console.log("Hello");

代码： });

代码： },

代码： { commands: ["mycmd"] },

代码：);

代码： }

注册自动回复命令

插件可以注册自定义斜杠命令，无需调用 AI

智能体即可执行。这对于切换命令、状态检查或不需要 LLM 处理的快速操作很有用。

```
代码： api.registerCommand({  
代码： name: "mystatus",  
代码： description: "Show plugin status",  
代码： handler: (ctx) => {  
代码： text: `Plugin is running! Channel: ${ctx.channel}`,  
代码： }},  
代码： });  
代码： }
```

命令处理程序上下文：

- senderId：发送者的 ID（如可用）
- channel：发送命令的渠道
- isAuthorizedSender：发送者是否是授权用户
- args：命令后传递的参数（如果 acceptsArgs: true）
- commandBody：完整的命令文本
- config：当前 OpenClaw 配置

命令选项：

- name：命令名称（不带前导 /）
- description：命令列表中显示的帮助文本
- acceptsArgs：命令是否接受参数（默认：false）。如果为 false 且提供了参数，命令不会匹配，消息会传递给其他处理程序
- requireAuth：是否需要授权发送者（默认：true）
- handler：返回 { text: string } 的函数（可以是异步的）

带授权和参数的示例：

```
代码： api.registerCommand({  
代码： name: "setmode",  
代码： description: "Set plugin mode",  
代码： acceptsArgs: true,
```

代码：requireAuth: true,

代码：handler: async (ctx) => {

代码：const mode = ctx.args?.trim() || "default";

代码：await saveMode(mode);

代码：return { text: `Mode set to: \${mode}` };

代码：},

代码：});

注意事项：

- 插件命令在内置命令和 AI 智能体之前处理
- 命令全局注册，适用于所有渠道
- 命令名称不区分大小写（/MyStatus 匹配 /mystatus）
- 命令名称必须以字母开头，只能包含字母、数字、连字符和下划线
- 保留的命令名称（如 help、status、reset 等）不能被插件覆盖
- 跨插件的重复命令注册将失败并显示诊断错误

注册后台服务

代码：api.registerService({

代码：id: "my-service",

代码：start: () => api.logger.info("ready"),

代码：stop: () => api.logger.info("bye"),

代码：});

代码：}

命名约定

- Gateway 网关方法：pluginId.action（例如：voicecall.status）
- 工具：snake_case（例如：voice_call）
- CLI 命令：kebab 或 camel，但避免与核心命令冲突

Skills

插件可以在仓库中附带 Skills（skills/<name>/SKILL.md）。

使用

`plugins.entries.<id>.enabled`（或其他配置门控）启用它，并确保它存在于你的工作区/托管 Skills 位置。

分发 (npm)

推荐的打包方式：

- 主包：openclaw（本仓库）
- 插件：@openclaw/ 下的独立 npm 包（例如：@openclaw/voice-call）

发布契约：

- 插件 `package.json` 必须包含带有一个或多个入口文件的 `openclaw.extensions`。
- 入口文件可以是 `.js` 或 `.ts`（`jiti` 在运行时加载 TS）。
- `openclaw plugins install <npm-spec>` 使用 `npm pack`，提取到 `~/.openclaw/extensions/<id>/`，并在配置中启用它。
- 配置键稳定性：作用域包被规范化为 `plugins.entries.` 的无作用域 `id`。

示例插件：Voice Call

本仓库包含一个语音通话插件（Twilio 或 log 回退）：

- 源码：extensions/voice-call
- Skills：skills/voice-call
- CLI：openclaw voicecall start|status
- 工具：voice_call
- RPC：voicecall.start、voicecall.status
- 配置（twilio）：provider: "twilio" + twilio.accountSid/authToken/from（可选 statusCallbackUrl、twimlUrl）
- 配置（dev）：provider: "log"（无网络）

参见 Voice Call 和 extensions/voice-call/README.md 了解设置和用法。

安全注意事项

插件与 Gateway 网关在同一进程中运行。将它们视为受信任的代码：

- 只安装你信任的插件。

- 优先使用 `plugins.allow` 允许列表。
- 更改后重启 Gateway 网关。

测试插件

插件可以（也应该）附带测试：

- 仓库内插件可以在 `src/` 下保留 Vitest 测试（例如：`src/plugins/voice-call.plugin.test.ts`）。
- 单独发布的插件应运行自己的 CI（lint/构建/测试）并验证 `openclaw.extensions` 指向构建的入口点（`dist/index.js`）。

18. 表情回应工具

表情回应工具

跨渠道共享的表情回应语义：

- 添加表情回应时，`emoji` 为必填项。
- `emoji=""` 在支持的情况下移除机器人的表情回应。
- `remove: true` 在支持的情况下移除指定的表情（需要提供 `emoji`）。

渠道说明：

- Discord/Slack：空 `emoji` 移除机器人在该消息上的所有表情回应；`remove: true` 仅移除指定的表情。
- Google Chat：空 `emoji` 移除应用在该消息上的表情回应；`remove: true` 仅移除指定的表情。
- Telegram：空 `emoji` 移除机器人的表情回应；`remove: true` 同样移除表情回应，但工具验证仍要求 `emoji` 为非空值。
- WhatsApp：空 `emoji` 移除机器人的表情回应；`remove: true` 映射为空 `emoji`（仍需提供 `emoji`）。
- Signal：当启用 `channels.signal.reactionNotifications` 时，收到的表情回应通知会触发系统事件。

19. Skills 配置

Skills 配置

所有 Skills 相关配置都位于 `~/.openclaw/openclaw.json` 中的 `skills` 下。

代码：{

```
代码： skills: {
代码： allowBundled: ["gemini", "peekaboo"],
代码： load: {
代码： extraDirs: ["~/Projects/agent-scripts/skills", "~/Projects/oss/some-skill-pack/skills"],
代码： watch: true,
代码： watchDebounceMs: 250,
代码： },
代码： install: {
代码： preferBrew: true,
代码： nodeManager: "npm", // npm | pnpm | yarn | bun (Gateway 网关运行时仍为
Node ; 不推荐 bun)
代码： },
代码： entries: {
代码： "nano-banana-pro": {
代码： enabled: true,
代码： apiKey: "GEMINI_KEY_HERE",
代码： env: {
代码： GEMINI_API_KEY: "GEMINI_KEY_HERE",
代码： },
代码： },
代码： peekaboo: { enabled: true },
代码： sag: { enabled: false },
代码： },
代码： },
代码： }
```

字段

- allowBundled：可选的仅用于内置 Skills 的白名单。设置后，只有列表中的内置 Skills 才有资格（托管/工作区 Skills 不受影响）。
- load.extraDirs：要扫描的附加 Skills 目录（最低优先级）。
- load.watch：监视 Skills 文件夹并刷新 Skills 快照（默认：true）。

- `load.watchDebounceMs` : Skills 监视器事件的防抖时间（毫秒）（默认：250）。
- `install.preferBrew` : 在可用时优先使用 brew 安装器（默认：true）。
- `install.nodeManager` : node 安装器偏好（npm | pnpm | yarn | bun，默认：npm）。这仅影响 Skills 安装；Gateway 网关运行时仍为 Node（不推荐 Bun 用于 WhatsApp/Telegram）。
- `entries.<skillKey>` : 单 Skills 覆盖。

单 Skills 字段：

- `enabled` : 设置为 false 以禁用某个 Skills，即使它是内置/已安装的。
- `env` : 为智能体运行注入的环境变量（仅在尚未设置时）。
- `apiKey` : 可选的便捷字段，用于声明主环境变量的 Skills。

注意事项

- `entries` 下的键默认映射到 Skills 名称。如果 Skills 定义了 `metadata.openclaw.skillKey`，则使用该键。
- 启用监视器后，Skills 的更改会在下一个智能体轮次被获取。

沙箱隔离的 Skills + 环境变量

当会话处于沙箱隔离状态时，Skills 进程在 Docker 内运行。沙箱不会继承宿主机的 `process.env`。

使用以下方式之一：

- `agents.defaults.sandbox.docker.env`（或单智能体的 `agents.list[].sandbox.docker.env`）
- 将环境变量烘焙到你的自定义沙箱镜像中

全局 `env` 和 `skills.entries.<skill>.env/apiKey` 仅适用于宿主机运行。

20. Skills (OpenClaw)

Skills (OpenClaw)

OpenClaw 使用兼容 AgentSkills 的 Skills 文件夹来教智能体如何使用工具。每个 Skills 是一个包含带有 YAML frontmatter 和说明的 SKILL.md 的目录。OpenClaw 加载内置 Skills 以及可选的本地覆盖，并在加载时根据环境、配置和二进制文件存在情况进行过滤。

位置和优先级

Skills 从三个位置加载：

- 内置 Skills：随安装包一起发布（npm 包或 OpenClaw.app）
- 托管/本地 Skills：~/.openclaw/skills
- 工作区 Skills：<workspace>/skills

如果 Skills 名称冲突，优先级为：

<workspace>/skills（最高）→ ~/.openclaw/skills → 内置 Skills（最低）

此外，你可以通过 ~/.openclaw/openclaw.json 中的 skills.load.extraDirs 配置额外的 Skills 文件夹（最低优先级）。

单智能体 vs 共享 Skills

在多智能体设置中，每个智能体有自己的工作区。这意味着：

- 单智能体 Skills 位于 <workspace>/skills 中，仅供该智能体使用。
- 共享 Skills 位于 ~/.openclaw/skills（托管/本地），对同一机器上的所有智能体可见。
- 如果你想要多个智能体使用一个通用的 Skills 包，也可以通过 skills.load.extraDirs（最低优先级）添加共享文件夹。

如果同一个 Skills

名称存在于多个位置，将应用通常的优先级规则：工作区优先，然后是托管/本地，最后是内置。

插件 + Skills

插件可以通过在 openclaw.plugin.json 中列出 skills

目录（相对于插件根目录的路径）来发布自己的 Skills。插件 Skills

在插件启用时加载，并参与正常的 Skills 优先级规则。你可以通过插件配置条目上的

metadata.openclaw.requires.config

对它们进行门控。参见插件了解发现/配置，以及工具了解这些 Skills 所教授的工具接口。

ClawHub（安装 + 同步）

ClawHub 是 OpenClaw 的公共 Skills 注册表。浏览 Skills。完整指南：ClawHub。

常见流程：

- 将 Skills 安装到你的工作区：
- clawhub install <skill-slug>

- 更新所有已安装的 Skills :
- `clawhub update --all`
- 同步（扫描 + 发布更新） :
- `clawhub sync --all`

默认情况下，clawhub 安装到当前工作目录下的 `./skills`（或回退到配置的 OpenClaw 工作区）。OpenClaw 在下一个会话中将其识别为 `<workspace>/skills`。

安全注意事项

- 将第三方 Skills 视为不受信任的代码。启用前请阅读它们。
- 对于不受信任的输入和高风险工具，优先使用沙箱隔离运行。参见沙箱隔离。
- `skills.entries..env` 和 `skills.entries..apiKey`
为该智能体轮次将秘密注入到宿主机进程中（而非沙箱）。将秘密保持在提示词和日志之外。
- 有关更广泛的威胁模型和检查清单，参见安全性。

格式（AgentSkills + Pi 兼容）

SKILL.md 必须至少包含：

代码：---

代码：name: nano-banana-pro

代码：description: Generate or edit images via Gemini 3 Pro Image

代码：---

注意事项：

- 我们遵循 AgentSkills 规范的布局/意图。
- 内嵌智能体使用的解析器仅支持单行 frontmatter 键。
- metadata 应该是单行 JSON 对象。
- 在说明中使用 `{baseDir}` 来引用 Skills 文件夹路径。
- 可选的 frontmatter 键：
- homepage — 在 macOS Skills UI 中显示为"Website"的 URL（也支持通过 `metadata.openclaw.homepage`）。
- user-invokable — `true|false`（默认：true）。当为 true 时，Skills 作为用户斜杠命令暴露。

- `disable-model-invocation` — `true|false`（默认：`false`）。当为 `true` 时，Skills 从模型提示词中排除（仍可通过用户调用使用）。
- `command-dispatch` — `tool`（可选）。当设置为 `tool` 时，斜杠命令绕过模型直接调度到工具。
- `command-tool` — 当设置 `command-dispatch: tool` 时要调用的工具名称。
- `command-arg-mode` — `raw`（默认）。对于工具调度，将原始参数字符串转发到工具（无核心解析）。

工具使用以下参数调用：

```
{ command: "<raw args>", commandName: "<slash command>", skillName: "<skill name>"
}
```

门控（加载时过滤）

OpenClaw 使用 `metadata`（单行 JSON）在加载时过滤 Skills：

代码：---

代码：`name: nano-banana-pro`

代码：`description: Generate or edit images via Gemini 3 Pro Image`

代码：`metadata:`

代码：{

代码：`"openclaw":`

代码：{

代码：`"requires": { "bins": ["uv"], "env": ["GEMINI_API_KEY"], "config": ["browser.enabled"] },`

代码：`"primaryEnv": "GEMINI_API_KEY",`

代码：`},`

代码：`}`

代码：---

`metadata.openclaw` 下的字段：

- `always: true` — 始终包含该 Skills（跳过其他门控）。
- `emoji` — macOS Skills UI 使用的可选表情符号。
- `homepage` — 在 macOS Skills UI 中显示为“Website”的可选 URL。

- os — 可选的平台列表（darwin、linux、win32）。如果设置，该 Skills 仅在这些操作系统上有资格。
- requires.bins — 列表；每个都必须存在于 PATH 中。
- requires.anyBins — 列表；至少一个必须存在于 PATH 中。
- requires.env — 列表；环境变量必须存在或在配置中提供。
- requires.config — openclaw.json 路径列表，必须为真值。
- primaryEnv — 与 skills.entries.<name>.apiKey 关联的环境变量名称。
- install — macOS Skills UI 使用的可选安装器规格数组（brew/node/go/uv/download）。

沙箱隔离注意事项：

- requires.bins 在 Skills 加载时在宿主机上检查。
- 如果智能体处于沙箱隔离状态，二进制文件也必须存在于容器内部。通过 agents.defaults.sandbox.docker.setupCommand（或自定义镜像）安装它。setupCommand 在容器创建后运行一次。包安装还需要网络出口、可写的根文件系统和沙箱中的 root 用户。示例：summarize Skills（skills/summarize/SKILL.md）需要 summarize CLI 在沙箱容器中才能运行。

安装器示例：

代码：---

代码：name: gemini

代码：description: Use Gemini CLI for coding assistance and Google search lookups.

代码：metadata:

代码：{

代码："openclaw":

代码：{

代码："emoji": "",

代码："requires": {"bins": ["gemini"] },

代码："install":

代码：[

代码：{

代码："id": "brew",

代码："kind": "brew",

代码： "formula": "gemini-cli",

代码： "bins": ["gemini"],

代码： "label": "Install Gemini CLI (brew)",

代码： },

代码：],

代码： },

代码： }

代码： ---

注意事项：

- 如果列出了多个安装器，Gateway 网关会选择单个首选选项（可用时选择 brew，否则选择 node）。
- 如果所有安装器都是 download，OpenClaw 会列出每个条目，以便你查看可用的构件。
- 安装器规格可以包含 os: ["darwin"|"linux"|"win32"] 按平台过滤选项。
- Node 安装遵循 openclaw.json 中的 skills.install.nodeManager（默认：npm；选项：npm/pnpm/yarn/bun）。这仅影响 Skills 安装；Gateway 网关运行时仍应为 Node（不推荐 Bun 用于 WhatsApp/Telegram）。
- Go 安装：如果缺少 go 且 brew 可用，Gateway 网关会首先通过 Homebrew 安装 Go，并在可能时将 GOBIN 设置为 Homebrew 的 bin。
- Download 安装：url（必填）、archive（tar.gz | tar.bz2 | zip）、extract（默认：检测到归档时自动）、stripComponents、targetDir（默认：~/.openclaw/tools/<skillKey>）。

如果没有 metadata.openclaw，该 Skills 始终有资格（除非在配置中禁用或被 skills.allowBundled 阻止用于内置 Skills）。

配置覆盖（`~/.openclaw/openclaw.json`）

内置/托管 Skills 可以被切换并提供环境变量值：

代码： {

代码： skills: {

代码： entries: {

代码： "nano-banana-pro": {

代码： enabled: true,

代码： apiKey: "GEMINI_KEY_HERE",

```
代码： env: {  
代码： GEMINI_API_KEY: "GEMINI_KEY_HERE",  
代码： },  
代码： config: {  
代码： endpoint: "https://example.invalid",  
代码： model: "nano-pro",  
代码： },  
代码： },  
代码： peekaboo: { enabled: true },  
代码： sag: { enabled: false },  
代码： },  
代码： },  
代码： }
```

注意：如果 Skills 名称包含连字符，请用引号括起键名（JSON5 允许带引号的键名）。

配置键默认匹配 Skills 名称。如果 Skills 定义了 metadata.openclaw.skillKey，请在 skills.entries 下使用该键。

规则：

- enabled: false 禁用该 Skills，即使它是内置/已安装的。
- env：仅在变量在进程中尚未设置时注入。
- apiKey：为声明 metadata.openclaw.primaryEnv 的 Skills 提供的便捷字段。
- config：用于自定义单 Skills 字段的可选容器；自定义键必须放在这里。
- allowBundled：可选的仅用于内置 Skills 的白名单。如果设置，只有列表中的内置 Skills 才有资格（托管/工作区 Skills 不受影响）。

环境变量注入（每次智能体运行）

当智能体运行开始时，OpenClaw：

- 读取 Skills 元数据。
- 将任何 skills.entries.<key>.env 或 skills.entries.<key>.apiKey 应用到 process.env。
- 使用有资格的 Skills 构建系统提示词。

- 在运行结束后恢复原始环境。

这是限定于智能体运行范围内的，不是全局 shell 环境。

会话快照（性能）

OpenClaw 在会话开始时有资格的 Skills 进行快照，并在同一会话的后续轮次中重用该列表。对 Skills 或配置的更改在下一个新会话中生效。

当 Skills 监视器启用或出现新的有资格的远程节点时，Skills 也可以在会话中刷新（见下文）。将此视为热重载：刷新后的列表会在下一个智能体轮次被获取。

远程 macOS 节点（Linux Gateway 网关）

如果 Gateway 网关运行在 Linux 上但连接了一个允许 system.run 的 macOS 节点（Exec 批准安全设置未设为 deny），当所需的二进制文件存在于该节点上时，OpenClaw 可以将仅限 macOS 的 Skills 视为有资格。智能体应通过 nodes 工具（通常是 nodes.run）执行这些 Skills。

这依赖于节点报告其命令支持以及通过 system.run 进行的二进制文件探测。如果 macOS 节点稍后离线，Skills 仍然可见；调用可能会失败，直到节点重新连接。

Skills 监视器（自动刷新）

默认情况下，OpenClaw 监视 Skills 文件夹，并在 SKILL.md 文件更改时更新 Skills 快照。在 skills.load 下配置：

```
代码：{  
代码： skills: {  
代码： load: {  
代码： watch: true,  
代码： watchDebounceMs: 250,  
代码： },  
代码： },  
代码： }
```

Token 影响（Skills 列表）

当 Skills 有资格时，OpenClaw 将可用 Skills 的紧凑 XML 列表注入到系统提示词中（通过 pi-coding-agent 中的 formatSkillsForPrompt）。成本是确定性的：

- 基础开销（仅当 ≥ 1 个 Skills 时）：195 字符。
- 每个 Skills：97 字符 + XML 转义的 `<name>`、`<description>` 和 `<location>` 值的长度。

公式（字符）：

代码：`total = 195 + \sum (97 + \text{len}(\text{name\_escaped}) + \text{len}(\text{description\_escaped}) + \text{len}(\text{location\_escaped}))`

注意事项：

- XML 转义将 `& < > ' "` 扩展为实体（`&`、`<` 等），增加长度。
- Token 数量因模型分词器而异。粗略的 OpenAI 风格估计是 ~ 4 字符/token，所以每个 Skills 97 字符 ≈ 24 token 加上你的实际字段长度。

托管 Skills 生命周期

OpenClaw 作为安装的一部分（npm 包或 OpenClaw.app）发布一组基线 Skills 作为内置 Skills。`~/.openclaw/skills` 用于本地覆盖（例如，在不更改内置副本的情况下固定/修补 Skills）。工作区 Skills 由用户拥有，在名称冲突时覆盖两者。

配置参考

参见 Skills 配置了解完整的配置 schema。

寻找更多 Skills？

浏览

21. 斜杠命令

斜杠命令

命令由 Gateway 网关处理。大多数命令必须作为以 `/` 开头的独立消息发送。

仅主机的 bash 聊天命令使用 `!<cmd>`（`/bash <cmd>` 是别名）。

有两个相关系统：

- 命令：独立的 `/...` 消息。
- 指令：`/think`、`/verbose`、`/reasoning`、`/elevated`、`/exec`、`/model`、`/queue`。
- 指令在模型看到消息之前被剥离。

- 在普通聊天消息中（不是仅指令消息），它们被视为"内联提示"，不会持久化会话设置。
 - 在仅指令消息中（消息只包含指令），它们会持久化到会话并回复确认。
 - 指令仅对授权发送者生效（渠道白名单/配对加上 `commands.useAccessGroups`）。
- 未授权发送者的指令被视为纯文本。

还有一些内联快捷方式（仅限白名单/授权发送者）：`/help`、`/commands`、`/status`、`/whoami (/id)`。

它们立即运行，在模型看到消息之前被剥离，剩余文本继续通过正常流程。

配置

代码：{

代码： `commands: {`

代码： `native: "auto",`

代码： `nativeSkills: "auto",`

代码： `text: true,`

代码： `bash: false,`

代码： `bashForegroundMs: 2000,`

代码： `config: false,`

代码： `debug: false,`

代码： `restart: false,`

代码： `useAccessGroups: true,`

代码： `},`

代码： `}`

- `commands.text`（默认 `true`）启用解析聊天消息中的 `/...`。
- 在没有原生命令的平台上（WhatsApp/WebChat/Signal/iMessage/Google Chat/MS Teams），即使你将此设置为 `false`，文本命令仍然有效。
- `commands.native`（默认 `"auto"`）注册原生命令。
- Auto：在 Discord/Telegram 上启用；在 Slack 上禁用（直到你添加斜杠命令）；在不支持原生命令的提供商上忽略。
- 设置 `channels.discord.commands.native`、`channels.telegram.commands.native` 或 `channels.slack.commands.native` 以按提供商覆盖（布尔值或 `"auto"`）。

- false 在启动时清除 Discord/Telegram 上之前注册的命令。Slack 命令在 Slack 应用中管理，不会自动删除。
- commands.nativeSkills (默认 "auto") 在支持时原生注册 Skill 命令。
- Auto : 在 Discord/Telegram 上启用；在 Slack 上禁用 (Slack 需要为每个 Skill 创建一个斜杠命令)。
- 设置
channels.discord.commands.nativeSkills、channels.telegram.commands.nativeSkills 或 channels.slack.commands.nativeSkills 以按提供商覆盖 (布尔值或 "auto")。
- commands.bash (默认 false) 启用 !<cmd> 来运行主机 shell 命令 (/bash <cmd> 是别名；需要 tools.elevated 白名单)。
- commands.bashForegroundMs (默认 2000) 控制 bash 切换到后台模式之前等待多长时间 (0 立即后台运行)。
- commands.config (默认 false) 启用 /config (读写 openclaw.json)。
- commands.debug (默认 false) 启用 /debug (仅运行时覆盖)。
- commands.useAccessGroups (默认 true) 对命令强制执行白名单/策略。

命令列表

文本 + 原生 (启用时) :

- /help
- /commands
- /skill <name> [input] (按名称运行 Skill)
- /status (显示当前状态；在可用时包含当前模型提供商的提供商使用量/配额)
- /allowlist (列出/添加/删除白名单条目)
- /approve <id> allow-once|allow-always|deny (解决 exec 审批提示)
- /context [list|detail|json] (解释"上下文"；detail 显示每个文件 + 每个工具 + 每个 Skill + 系统提示词大小)
- /whoami (显示你的发送者 ID；别名：/id)
- /subagents
list|kill|log|info|send|steer|spawn (检查、控制或创建当前会话的子智能体运行)
- /config show|get|set|unset (将配置持久化到磁盘，仅所有者；需要 commands.config: true)
- /debug show|set|unset|reset (运行时覆盖，仅所有者；需要 commands.debug: true)

- /usage off|tokens|full|cost (每响应使用量页脚或本地成本摘要)
- /tts off|always|inbound|tagged|status|provider|limit|summary|audio (控制 TTS ; 参见 /tts)
- Discord : 原生命令是 /voice (Discord 保留了 /tts) ; 文本 /tts 仍然有效。
- /stop
- /restart
- /dock-telegram (别名 : /dock_telegram) (将回复切换到 Telegram)
- /dock-discord (别名 : /dock_discord) (将回复切换到 Discord)
- /dock-slack (别名 : /dock_slack) (将回复切换到 Slack)
- /activation mention|always (仅限群组)
- /send on|off|inherit (仅所有者)
- /reset 或 /new [model] (可选模型提示 ; 其余部分传递)
- /think
<off|minimal|low|medium|high|xhigh> (按模型/提供商动态选择 ; 别名 : /thinking 、 /t)
- /verbose on|full|off (别名 : /v)
- /reasoning on|off|stream (别名 : /reason ; 启用时 , 发送带有 Reasoning: 前缀的单独消息 ; stream = 仅 Telegram 草稿)
- /elevated on|off|ask|full (别名 : /elev ; full 跳过 exec 审批)
- /exec host=<sandbox|gateway|node> security=<deny|allowlist|full>
ask=<off|on-miss|always> node=<id> (发送 /exec 显示当前设置)
- /model <name> (别名 : /models ; 或 agents.defaults.models..alias 中的 /<alias>)
- /queue <mode> (加上选项如 debounce:2s cap:25 drop:summarize ; 发送 /queue 查看当前设置)
- /bash <command> (仅主机 ; !<command> 的别名 ; 需要 commands.bash: true + tools.elevated 白名单)

仅文本 :

- /compact [instructions] (参见 /concepts/compaction)
- !<command> (仅主机 ; 一次一个 ; 对长时间运行的任务使用 !poll + !stop)
- !poll (检查输出/状态 ; 接受可选的 sessionId ; /bash poll 也可用)
- !stop (停止正在运行的 bash 任务 ; 接受可选的 sessionId ; /bash stop 也可用)

注意事项 :

- 命令接受命令和参数之间的可选：（例如 `/think: high`、`/send: on`、`/help:`）。
- `/new <model>` 接受模型别名、`provider/model` 或提供商名称（模糊匹配）；如果没有匹配，文本被视为消息正文。
- 要获取完整的提供商使用量分解，使用 `openclaw status --usage`。
- `/allowlist add|remove` 需要 `commands.config=true` 并遵循渠道 `configWrites`。
- `/usage` 控制每响应使用量页脚；`/usage cost` 从 OpenClaw 会话日志打印本地成本摘要。
- `/restart` 默认禁用；设置 `commands.restart: true` 启用它。
- `/verbose` 用于调试和额外可见性；在正常使用中保持关闭。
- `/reasoning`（和 `/verbose`）在群组设置中有风险：它们可能会暴露你不打算公开的推理或工具输出。最好保持关闭，尤其是在群聊中。
- 快速路径：来自白名单发送者的仅命令消息会立即处理（绕过队列 + 模型）。
- 群组提及门控：来自白名单发送者的仅命令消息绕过提及要求。
- 内联快捷方式（仅限白名单发送者）：
某些命令在嵌入普通消息时也能工作，并在模型看到剩余文本之前被剥离。
- 示例：`hey /status` 触发状态回复，剩余文本继续通过正常流程。
- 目前：`/help`、`/commands`、`/status`、`/whoami (/id)`。
- 未授权的仅命令消息被静默忽略，内联 `/...` 令牌被视为纯文本。
- Skill 命令：`user-invocable Skills` 作为斜杠命令公开。名称被清理为 `a-z0-9_`（最多 32 个字符）；冲突获得数字后缀（例如 `_2`）。
- `/skill <name> [input]` 按名称运行 Skill（当原生命令限制阻止每个 Skill 命令时有用）。
- 默认情况下，Skill 命令作为普通请求转发给模型。
- Skills 可以选择声明 `command-dispatch: tool` 将命令直接路由到工具（确定性，无模型）。
- 示例：`/prose`（OpenProse 插件）— 参见 OpenProse。
- 原生命令参数：Discord 使用自动完成进行动态选项（以及当你省略必需参数时的按钮菜单）。当命令支持选择且你省略参数时，Telegram 和 Slack 显示按钮菜单。

使用量显示（什么显示在哪里）

- 提供商使用量/配额（示例：`"Claude 80% left"`）在启用使用量跟踪时显示在 `/status` 中，针对当前模型提供商。
- 每响应令牌/成本由 `/usage off|tokens|full` 控制（附加到普通回复）。
- `/model status` 是关于模型/认证/端点的，不是使用量。

模型选择（`/model`）

`/model` 作为指令实现。

示例：

代码：`/model`

代码：`/model list`

代码：`/model 3`

代码：`/model openai/gpt-5.2`

代码：`/model opus@anthropic:default`

代码：`/model status`

注意事项：

- `/model` 和 `/model list` 显示紧凑的编号选择器（模型系列 + 可用提供商）。
- `/model <#>` 从该选择器中选择（并在可能时优先选择当前提供商）。
- `/model status` 显示详细视图，包括在可用时配置的提供商端点（`baseUrl`）和 API 模式（`api`）。

调试覆盖

`/debug` 让你设置仅运行时的配置覆盖（内存，不写磁盘）。仅所有者。默认禁用；使用 `commands.debug: true` 启用。

示例：

代码：`/debug show`

代码：`/debug set messages.responsePrefix="[openclaw]"`

代码：`/debug set channels.whatsapp.allowFrom=["+1555","+4477"]`

代码：`/debug unset messages.responsePrefix`

代码：`/debug reset`

注意事项：

- 覆盖立即应用于新的配置读取，但不会写入 `openclaw.json`。
- 使用 `/debug reset` 清除所有覆盖并返回到磁盘上的配置。

配置更新

/config 写入你的磁盘配置 (openclaw.json) 。仅所有者。默认禁用；使用 commands.config: true 启用。

示例：

代码：/config show

代码：/config show messages.responsePrefix

代码：/config get messages.responsePrefix

代码：/config set messages.responsePrefix="[openclaw]"

代码：/config unset messages.responsePrefix

注意事项：

- 配置在写入前会验证；无效更改会被拒绝。
- /config 更新在重启后持久化。

平台注意事项

- 文本命令在普通聊天会话中运行（私信共享 main，群组有自己的会话）。
- 原生命令使用隔离的会话：
- Discord：agent:<agentId>:discord:slash:<userId>
- Slack：agent:<agentId>:slack:slash:<userId>（前缀可通过 channels.slack.slashCommand.sessionPrefix 配置）
- Telegram：telegram:slash:<userId>（通过 CommandTargetSessionKey 定向到聊天会话）
- /stop 定向到活动聊天会话，因此可以中止当前运行。
- Slack：channels.slack.slashCommand 仍然支持单个 /openclaw 风格的命令。如果你启用 commands.native，你必须为每个内置命令创建一个 Slack 斜杠命令（与 /help 相同的名称）。Slack 的命令参数菜单以临时 Block Kit 按钮形式发送。

22. 子智能体

子智能体

子智能体是从现有智能体运行中生成的后台智能体运行。它们在自己的会话中运行（agent:<agentId>:subagent:<uuid>），完成后将结果通告回请求者的聊天渠道。

斜杠命令

使用 `/subagents` 检查或控制当前会话的子智能体运行：

- `/subagents list`
- `/subagents kill <id| #|all>`
- `/subagents log <id| #> [limit] [tools]`
- `/subagents info <id| #>`
- `/subagents send <id| #> <message>`
- `/subagents steer <id| #> <message>`
- `/subagents spawn <agentId> <task> [--model <model>] [--thinking <level>]`

`/subagents info` 显示运行元数据（状态、时间戳、会话 id、转录路径、清理）。

启动行为

`/subagents spawn`

以用户命令方式启动后台子智能体，任务完成后会向请求者聊天频道回发一条最终完成消息。

- 该命令非阻塞，先返回 `runId`。
- 完成后，子智能体会将汇总/结果消息发布到请求者聊天渠道。
- `--model` 与 `--thinking` 可仅对本次运行做覆盖设置。
- 可在完成后通过 `info/log` 查看详细信息和输出。

主要目标：

- 并行化"研究 / 长任务 / 慢工具"工作，而不阻塞主运行。
- 默认保持子智能体隔离（会话分离 + 可选沙箱隔离）。
- 保持工具接口难以滥用：子智能体默认不获得会话工具。
- 避免嵌套扇出：子智能体不能生成子智能体。

成本说明：每个子智能体都有自己的上下文和 token 使用量。对于繁重或重复的任务，为子智能体设置更便宜的模型，而让主智能体使用更高质量的模型。你可以通过 `agents.defaults.subagents.model` 或每智能体覆盖来配置。

工具

使用 `sessions_spawn`：

- 启动子智能体运行（`deliver: false`，全局队列：`subagent`）

- 然后运行通告步骤，并将通告回复发布到请求者的聊天渠道
- 默认模型：继承调用者，除非你设置了 `agents.defaults.subagents.model`（或每智能体的 `agents.list[].subagents.model`）；显式的 `sessions_spawn.model` 仍然优先。
- 默认思考：继承调用者，除非你设置了 `agents.defaults.subagents.thinking`（或每智能体的 `agents.list[].subagents.thinking`）；显式的 `sessions_spawn.thinking` 仍然优先。

工具参数：

- `task`（必需）
- `label?`（可选）
- `agentId?`（可选；如果允许，在另一个智能体 id 下生成）
- `model?`（可选；覆盖子智能体模型；无效值会被跳过，子智能体将使用默认模型运行并在工具结果中显示警告）
- `thinking?`（可选；覆盖子智能体运行的思考级别）
- `runTimeoutSeconds?`（默认 0；设置后，子智能体运行在 N 秒后中止）
- `cleanup?`（`delete|keep`，默认 `keep`）

允许列表：

- `agents.list[].subagents.allowAgents`：可以通过 `agentId` 指定的智能体 id 列表（`[]` 允许任意）。默认：仅限请求者智能体。

发现：

- 使用 `agents_list` 查看当前允许用于 `sessions_spawn` 的智能体 id。

自动归档：

- 子智能体会话在 `agents.defaults.subagents.archiveAfterMinutes` 后自动归档（默认：60）。
- 归档使用 `sessions.delete` 并将转录重命名为 `.deleted.<timestamp>`（同一文件夹）。
- `cleanup: "delete"` 在通告后立即归档（仍通过重命名保留转录）。
- 自动归档是尽力而为的；如果 Gateway 网关重启，待处理的定时器会丢失。
- `runTimeoutSeconds` 不会自动归档；它只停止运行。会话会保留直到自动归档。

认证

子智能体认证按智能体 id 解析，而不是按会话类型：

- 子智能体会话键是 agent:<agentId>:subagent:<uuid>。
- 认证存储从该智能体的 agentDir 加载。
- 主智能体的认证配置文件作为回退合并；智能体配置文件在冲突时覆盖主配置文件。

注意：合并是累加的，所以主配置文件始终可用作回退。目前尚不支持每智能体完全隔离的认证。

通告

子智能体通过通告步骤报告：

- 通告步骤在子智能体会话中运行（不是请求者会话）。
- 如果子智能体精确回复 ANNOUNCE_SKIP，则不发布任何内容。
- 否则，通告回复通过后续的 agent 调用（deliver=true）发布到请求者的聊天渠道。
- 通告回复在可用时保留线程/话题路由（Slack 线程、Telegram 话题、Matrix 线程）。
- 通告消息被规范化为稳定模板：
- Status: 从运行结果派生（success、error、timeout 或 unknown）。
- Result: 通告步骤的摘要内容（如果缺失则为 (not available)）。
- Notes: 错误详情和其他有用的上下文。
- Status 不是从模型输出推断的；它来自运行时结果信号。

通告负载在末尾包含统计行（即使被包装）：

- 运行时间（例如 runtime 5m12s）
- Token 使用量（输入/输出/总计）
- 配置模型定价时的估计成本（models.providers..models[].cost）
- sessionKey、sessionId 和转录路径（以便主智能体可以通过 sessions_history 获取历史记录或检查磁盘上的文件）

工具策略（子智能体工具）

默认情况下，子智能体获得除会话工具外的所有工具：

- sessions_list
- sessions_history
- sessions_send
- sessions_spawn

通过配置覆盖：

代码：{

代码： agents: {

代码： defaults: {

代码： subagents: {

代码： maxConcurrent: 1,

代码： },

代码： },

代码： },

代码： tools: {

代码： subagents: {

代码： tools: {

代码： // deny 优先

代码： deny: ["gateway", "cron"],

代码： // 如果设置了 allow，则变为仅允许模式（deny 仍然优先）

代码： // allow: ["read", "exec", "process"]

代码： },

代码： },

代码： },

代码： }

并发

子智能体使用专用的进程内队列通道：

- 通道名称：subagent
- 并发数：agents.defaults.subagents.maxConcurrent（默认 8）

停止

- 在请求者聊天中发送 /stop 会中止请求者会话并停止从中生成的任何活动子智能体运行。

限制

- 子智能体通告是尽力而为的。如果 Gateway 网关重启，待处理的"通告回复"工作会丢失。
- 子智能体仍然共享相同的 Gateway 网关进程资源；将 maxConcurrent 视为安全阀。
- sessions_spawn 始终是非阻塞的：它立即返回 { status: "accepted", runId, childSessionKey }。
- 子智能体上下文仅注入 AGENTS.md + TOOLS.md（无 SOUL.md、IDENTITY.md、USER.md、HEARTBEAT.md 或 BOOTSTRAP.md）。

23. 思考级别 (/think 指令)

思考级别 (/think 指令)

功能说明

- 在任何入站消息正文中使用内联指令：/t <level>、/think:<level> 或 /thinking <level>。
- 级别（别名）：off | minimal | low | medium | high | xhigh（仅 GPT-5.2 + Codex 模型）
- minimal → "think"
- low → "think hard"
- medium → "think harder"
- high → "ultrathink"（最大预算）
- xhigh → "ultrathink+"（仅 GPT-5.2 + Codex 模型）
- highest、max 映射为 high。
- 提供商说明：
- Z.AI (zai/) 仅支持二元思考 (on/off)。任何非 off 级别均视为 on（映射为 low）。

解析优先顺序

- 消息上的内联指令（仅适用于该条消息）。
- 会话覆盖（通过发送仅包含指令的消息设置）。
- 全局默认值（配置中的 agents.defaults.thinkingDefault）。
- 回退：具备推理能力的模型为 low；否则为 off。

设置会话默认值

- 发送一条仅包含指令的消息（允许空白），例如 /think:medium 或 /t high。
- 该设置在当前会话中持续生效（默认按发送者）；通过 /think:off 或会话空闲重置来清除。

- 会发送确认回复 (Thinking level set to high. / Thinking disabled.) 。如果级别无效 (例如 /thinking big) ，命令将被拒绝并给出提示，会话状态保持不变。
- 不带参数发送 /think (或 /think:) 可查看当前思考级别。

按智能体应用

- 内嵌 Pi：解析后的级别传递给进程内的 Pi 智能体运行时。

详细模式指令 (/verbose 或 /v)

- 级别：on (最小) | full | off (默认) 。
- 仅包含指令的消息切换会话详细模式并回复 Verbose logging enabled. / Verbose logging disabled.；无效级别返回提示且不改变状态。
- /verbose off 存储一个显式的会话覆盖；通过会话 UI 选择 inherit 来清除。
- 内联指令仅影响该条消息；否则应用会话/全局默认值。
- 不带参数发送 /verbose (或 /verbose:) 可查看当前详细模式级别。
- 启用详细模式后，发出结构化工具结果的智能体 (Pi 及其他 JSON 智能体) 会将每个工具调用作为独立的元数据消息发回，可用时以 <emoji> <tool-name>: <arg> 为前缀 (路径/命令)。这些工具摘要在每个工具启动时立即发送 (独立气泡)，而非作为流式增量。
- 当详细模式为 full 时，工具输出也会在完成后转发 (独立气泡，截断至安全长度)。如果在运行过程中切换 /verbose on|full|off，后续的工具气泡会遵循新设置。

推理可见性 (/reasoning)

- 级别：on|off|stream 。
- 仅包含指令的消息切换回复中是否显示思考块。
- 启用时，推理内容作为独立消息发送，以 Reasoning: 为前缀。
- stream (仅 Telegram)：在回复生成期间将推理内容流式输出到 Telegram 草稿气泡中，然后发送不包含推理的最终回答。
- 别名：/reason 。
- 不带参数发送 /reasoning (或 /reasoning:) 可查看当前推理级别。

相关内容

- 提权模式文档位于提权模式。

心跳

- 心跳探测正文为配置的心跳提示词（默认：Read HEARTBEAT.md if it exists (workspace context). Follow it strictly. Do not infer or repeat old tasks from prior chats. If nothing needs attention, reply HEARTBEAT_OK.）。心跳消息中的内联指令照常生效（但避免从心跳中更改会话默认值）。
- 心跳投递默认仅包含最终负载。要同时发送单独的 Reasoning: 消息（如果可用），请设置 `agents.defaults.heartbeat.includeReasoning: true` 或按智能体 `agents.list[].heartbeat.includeReasoning: true`。

Web 聊天 UI

- Web 聊天的思考选择器在页面加载时从入站会话存储/配置中读取并反映会话的已存储级别。
- 选择另一个级别仅应用于下一条消息（`thinkingOnce`）；发送后，选择器会回到已存储的会话级别。
- 要更改会话默认值，请发送 `/think:<level>` 指令（和之前一样）；选择器将在下次刷新后反映该设置。

24. Web 工具

Web 工具

OpenClaw 提供两个轻量级 Web 工具：

- `web_search` — 通过 Brave Search API（默认）或 Perplexity Sonar（直连或通过 OpenRouter）搜索网络。
- `web_fetch` — HTTP 获取 + 可读性提取（HTML → markdown/文本）。

这些不是浏览器自动化。对于 JS 密集型网站或需要登录的情况，请使用浏览器工具。

工作原理

- `web_search` 调用你配置的提供商并返回结果。
- Brave（默认）：返回结构化结果（标题、URL、摘要）。
- Perplexity：返回带有实时网络搜索引用的 AI 综合答案。
- 结果按查询缓存 15 分钟（可配置）。
- `web_fetch` 执行普通 HTTP GET 并提取可读内容（HTML → markdown/文本）。它不执行 JavaScript。
- `web_fetch` 默认启用（除非显式禁用）。

选择搜索提供商

提供商	优点	缺点	API 密钥
-----	-----	-----	-----
Brave（默认）	快速、结构化结果、免费层	传统搜索结果	BRAVE_API_KEY
Perplexity	AI 综合答案、引用、实时	需要 Perplexity 或 OpenRouter 访问	OPENROUTER_API_KEY 或 PERPLEXITY_API_KEY

参见 Brave Search 设置 和 Perplexity Sonar 了解提供商特定详情。

在配置中设置提供商：

```
代码：{
代码：  tools: {
代码：    web: {
代码：      search: {
代码：        provider: "brave", // 或 "perplexity"
代码：      },
代码：    },
代码：  },
代码：}
```

示例：切换到 Perplexity Sonar（直连 API）：

```
代码：{
代码：  tools: {
代码：    web: {
代码：      search: {
代码：        provider: "perplexity",
代码：        perplexity: {
代码：          apiKey: "pplx-...",
代码：          baseUrl: "https://api.perplexity.ai",
代码：          model: "perplexity/sonar-pro",
代码：        },
代码：      },
代码：    },
代码：  },
代码：}
```

代码： },

代码： },

代码： },

代码： }

获取 Brave API 密钥

- 在 创建 Brave Search API 账户
- 在控制面板中，选择 Data for Search 计划（不是 "Data for AI"）并生成 API 密钥。
- 运行 `openclaw configure --section web` 将密钥存储在配置中（推荐），或在环境中设置 `BRAVE_API_KEY`。

Brave 提供免费层和付费计划；查看 Brave API 门户了解当前限制和定价。

在哪里设置密钥（推荐）

推荐： 运行 `openclaw configure --section web`。它将密钥存储在 `~/.openclaw/openclaw.json` 的 `tools.web.search.apiKey` 下。

环境变量替代方案： 在 Gateway 网关进程环境中设置 `BRAVE_API_KEY`。对于 Gateway 网关安装，将其放在 `~/.openclaw/.env`（或你的服务环境）中。参见环境变量。

使用 Perplexity（直连或通过 OpenRouter）

Perplexity Sonar 模型具有内置的网络搜索功能，并返回带有引用的 AI 综合答案。你可以通过 OpenRouter 使用它们（无需信用卡 - 支持加密货币/预付费）。

获取 OpenRouter API 密钥

- 在 创建账户
- 添加额度（支持加密货币、预付费或信用卡）
- 在账户设置中生成 API 密钥

设置 Perplexity 搜索

代码： {

代码： tools: {

代码： web: {

代码： search: {

```
代码 : enabled: true,
代码 : provider: "perplexity",
代码 : perplexity: {
代码 : // API 密钥 (如果设置了 OPENROUTER_API_KEY 或 PERPLEXITY_API_KEY 则可选)
代码 : apiKey: "sk-or-v1-...",
代码 : // 基础 URL (如果省略则根据密钥感知默认值)
代码 : baseUrl: "https://openrouter.ai/api/v1",
代码 : // 模型 (默认为 perplexity/sonar-pro)
代码 : model: "perplexity/sonar-pro",
代码 : },
代码 : },
代码 : },
代码 : },
代码 : }
```

环境变量替代方案：在 Gateway 网关环境中设置 OPENROUTER_API_KEY 或 PERPLEXITY_API_KEY。对于 Gateway 网关安装，将其放在 ~/.openclaw/.env 中。

如果未设置基础 URL，OpenClaw 会根据 API 密钥来源选择默认值：

- PERPLEXITY_API_KEY 或 pplx-... → `
- OPENROUTER_API_KEY 或 sk-or-... → `
- 未知密钥格式 → OpenRouter (安全回退)

可用的 Perplexity 模型

模型	描述	最适合
perplexity/sonar	带网络搜索的快速问答	快速查询
perplexity/sonar-pro (默认)	带网络搜索的多步推理	复杂问题
perplexity/sonar-reasoning-pro	思维链分析	深度研究

web_search

使用配置的提供商搜索网络。

要求

- tools.web.search.enabled 不能为 false（默认：启用）
- 所选提供商的 API 密钥：
- Brave：BRAVE_API_KEY 或 tools.web.search.apiKey
- Perplexity：OPENROUTER_API_KEY、PERPLEXITY_API_KEY 或 tools.web.search.perplexity.apiKey

配置

代码：{

代码：tools: {

代码：web: {

代码：search: {

代码：enabled: true,

代码：apiKey: "BRAVE_API_KEY_HERE", // 如果设置了 BRAVE_API_KEY 则可选

代码：maxResults: 5,

代码：timeoutSeconds: 30,

代码：cacheTtlMinutes: 15,

代码：},

代码：},

代码：},

代码：}

工具参数

- query（必需）
- count（1-10；默认来自配置）
- country（可选）：用于特定地区结果的 2 字母国家代码（例如"DE"、"US"、"ALL"）。如果省略，Brave 选择其默认地区。
- search_lang（可选）：搜索结果的 ISO 语言代码（例如"de"、"en"、"fr"）
- ui_lang（可选）：UI 元素的 ISO 语言代码
- freshness（可选，仅限 Brave）：按发现时间过滤（pd、pw、pm、py 或 YYYY-MM-DDtoYYYY-MM-DD）

示例：

代码：// 德国特定搜索

代码：await web_search({

代码： query: "TV online schauen",

代码： count: 10,

代码： country: "DE",

代码： search_lang: "de",

代码：});

代码：// 带法语 UI 的法语搜索

代码：await web_search({

代码： query: "actualités",

代码： country: "FR",

代码： search_lang: "fr",

代码： ui_lang: "fr",

代码：});

代码：// 最近结果（过去一周）

代码：await web_search({

代码： query: "TMBG interview",

代码： freshness: "pw",

代码：});

web_fetch

获取 URL 并提取可读内容。

要求

- tools.web.fetch.enabled 不能为 false（默认：启用）
- 可选的 Firecrawl 回退：设置 tools.web.fetch.firecrawl.apiKey 或 FIRECRAWL_API_KEY。

配置

代码：{

代码：tools: {
代码：web: {
代码：fetch: {
代码：enabled: true,
代码：maxChars: 50000,
代码：timeoutSeconds: 30,
代码：cacheTtlMinutes: 15,
代码：maxRedirects: 3,
代码：userAgent: "Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36",
代码：readability: true,
代码：firecrawl: {
代码：enabled: true,
代码：apiKey: "FIRECRAWL_API_KEY_HERE", // 如果设置了 FIRECRAWL_API_KEY 则可选
代码：baseUrl: "https://api.firecrawl.dev",
代码：onlyMainContent: true,
代码：maxAgeMs: 86400000, // 毫秒 (1 天)
代码：timeoutSeconds: 60,
代码：},
代码：},
代码：},
代码：},
代码：},
代码：}

工具参数

- url (必需，仅限 http/https)
- extractMode (markdown | text)
- maxChars (截断长页面)

注意：

- `web_fetch` 首先使用 Readability（主要内容提取），然后使用 Firecrawl（如果已配置）。如果两者都失败，工具返回错误。
- Firecrawl 请求使用机器人规避模式并默认缓存结果。
- `web_fetch` 默认发送类 Chrome 的 User-Agent 和 Accept-Language；如需要可覆盖 userAgent。
- `web_fetch` 阻止私有/内部主机名并重新检查重定向（用 `maxRedirects` 限制）。
- `web_fetch` 是尽力提取；某些网站需要浏览器工具。
- 参见 Firecrawl 了解密钥设置和服务详情。
- 响应会被缓存（默认 15 分钟）以减少重复获取。
- 如果你使用工具配置文件/允许列表，添加 `web_search/web_fetch` 或 `group:web`。
- 如果缺少 Brave 密钥，`web_search` 返回一个简短的设置提示和文档链接。

第六章：命令行实战

1. acp

acp

运行与 OpenClaw Gateway 网关通信的 ACP（Agent Client Protocol）桥接器。

此命令通过 stdio 使用 ACP 协议与 IDE 通信，并通过 WebSocket 将提示转发到 Gateway 网关。它将 ACP 会话映射到 Gateway 网关会话键。

用法

代码：openclaw acp

代码：# Remote Gateway

代码：openclaw acp --url wss://gateway-host:18789 --token <token>

代码：# Attach to an existing session key

代码：openclaw acp --session agent:main:main

代码：# Attach by label (must already exist)

代码：openclaw acp --session-label "support inbox"

代码：# Reset the session key before the first prompt

代码：openclaw acp --session agent:main:main --reset-session

ACP 客户端（调试）

使用内置 ACP 客户端在没有 IDE 的情况下检查桥接器的安装完整性。

它会启动 ACP 桥接器并让你交互式输入提示。

代码：openclaw acp client

代码：# Point the spawned bridge at a remote Gateway

代码：openclaw acp client --server-args --url wss://gateway-host:18789 --token <token>

代码：# Override the server command (default: openclaw)

代码：openclaw acp client --server "node" --server-args openclaw.mjs acp --url ws://127.0.0.1:19001

如何使用

当 IDE（或其他客户端）使用 Agent Client Protocol 并且你希望它驱动 OpenClaw Gateway 网关会话时，请使用 ACP。

- 确保 Gateway 网关正在运行（本地或远程）。
- 配置 Gateway 网关目标（配置或标志）。
- 将你的 IDE 配置为通过 stdio 运行 openclaw acp。

示例配置（持久化）：

代码：openclaw config set gateway.remote.url wss://gateway-host:18789

代码：openclaw config set gateway.remote.token <token>

示例直接运行（不写入配置）：

代码：openclaw acp --url wss://gateway-host:18789 --token <token>

选择智能体

ACP 不直接选择智能体。它通过 Gateway 网关会话键进行路由。

使用智能体作用域的会话键来定位特定智能体：

代码：openclaw acp --session agent:main:main

代码：openclaw acp --session agent:design:main

代码：openclaw acp --session agent:qa:bug-123

每个 ACP 会话映射到单个 Gateway

网关会话键。一个智能体可以有多个会话；除非你覆盖键或标签，否则 ACP 默认使用隔离的 acp:<uuid> 会话。

Zed 编辑器设置

在 ~/.config/zed/settings.json 中添加自定义 ACP 智能体（或使用 Zed 的设置界面）：

代码：{

代码： "agent_servers": {

代码： "OpenClaw ACP": {

代码： "type": "custom",

代码： "command": "openclaw",

代码： "args": ["acp"],

代码： "env": {}

代码： }

代码： }

代码： }

要定位特定的 Gateway 网关或智能体：

代码： {

代码： "agent_servers": {

代码： "OpenClaw ACP": {

代码： "type": "custom",

代码： "command": "openclaw",

代码： "args": [

代码： "acp",

代码： "--url",

代码： "wss://gateway-host:18789",

代码： "--token",

代码： "<token>",

代码： "--session",

代码： "agent:design:main"

代码：],

代码： "env": {}

代码： }

代码： }

代码： }

在 Zed 中，打开 Agent 面板并选择"OpenClaw ACP"来开始一个会话。

会话映射

默认情况下，ACP 会话获得一个带有 acp: 前缀的隔离 Gateway 网关会话键。

要重用已知会话，请传递会话键或标签：

- `--session <key>` : 使用特定的 Gateway 网关会话键。
- `--session-label <label>` : 通过标签解析现有会话。
- `--reset-session` : 为该键生成新的会话 ID（相同键，新对话记录）。

如果你的 ACP 客户端支持元数据，你可以按会话覆盖：

代码：{

代码： `"_meta": {`

代码： `"sessionKey": "agent:main:main",`

代码： `"sessionLabel": "support inbox",`

代码： `"resetSession": true`

代码： `}`

代码： `}`

在 `/concepts/session` 了解更多关于会话键的信息。

选项

- `--url <url>` : Gateway 网关 WebSocket URL（配置后默认为 `gateway.remote.url`）。
- `--token <token>` : Gateway 网关认证令牌。
- `--password <password>` : Gateway 网关认证密码。
- `--session <key>` : 默认会话键。
- `--session-label <label>` : 要解析的默认会话标签。
- `--require-existing` : 如果会话键/标签不存在则失败。
- `--reset-session` : 在首次使用前重置会话键。
- `--no-prefix-cwd` : 不在提示前添加工作目录前缀。
- `--verbose, -v` : 向 `stderr` 输出详细日志。

`acp client` 选项

- `--cwd <dir>` : ACP 会话的工作目录。
- `--server <command>` : ACP 服务器命令（默认：`openclaw`）。
- `--server-args <args...>` : 传递给 ACP 服务器的额外参数。
- `--server-verbose` : 启用 ACP 服务器的详细日志。
- `--verbose, -v` : 详细客户端日志。

2. `openclaw agent`

`openclaw agent`

通过 Gateway 网关运行智能体回合（使用 --local 进行嵌入式运行）。使用 --agent <id> 直接指定已配置的智能体。

相关内容：

- 智能体发送工具：Agent send

示例

代码：openclaw agent --to +15555550123 --message "status update" --deliver

代码：openclaw agent --agent ops --message "Summarize logs"

代码：openclaw agent --session-id 1234 --message "Summarize inbox" --thinking medium

代码：openclaw agent --agent ops --message "Generate report" --deliver --reply-channel slack --reply-to "#reports"

3. `openclaw agents`

`openclaw agents`

管理隔离的智能体（工作区 + 认证 + 路由）。

相关内容：

- 多智能体路由：多智能体路由
- 智能体工作区：智能体工作区

示例

代码：openclaw agents list

代码：openclaw agents add work --workspace ~/.openclaw/workspace-work

代码：openclaw agents set-identity --workspace ~/.openclaw/workspace --from-identity

代码：openclaw agents set-identity --agent main --avatar avatars/openclaw.png

代码：openclaw agents delete work

身份文件

每个智能体工作区可以在工作区根目录包含一个 IDENTITY.md :

- 示例路径 : `~/.openclaw/workspace/IDENTITY.md`
- `set-identity --from-identity` 从工作区根目录读取 (或从显式指定的 `--identity-file` 读取)

头像路径相对于工作区根目录解析。

设置身份

`set-identity` 将字段写入 `agents.list[].identity` :

- `name`
- `theme`
- `emoji`
- `avatar` (工作区相对路径、`http(s)` URL 或 `data URI`)

从 IDENTITY.md 加载 :

代码 : `openclaw agents set-identity --workspace ~/.openclaw/workspace --from-identity`

显式覆盖字段 :

代码 : `openclaw agents set-identity --agent main --name "OpenClaw" --emoji ""
--avatar avatars/openclaw.png`

配置示例 :

代码 : {

代码 : agents: {

代码 : list: [

代码 : {

代码 : id: "main",

代码 : identity: {

代码 : name: "OpenClaw",

代码 : theme: "space lobster",

代码 : emoji: "",

代码 : avatar: "avatars/openclaw.png",

代码 : },

代码： },

代码：],

代码： },

代码： }

4. `openclaw approvals`

`openclaw approvals`

管理本地主机、Gateway 网关主机或节点主机的执行审批。

默认情况下，命令针对磁盘上的本地审批文件。使用 --gateway 可针对 Gateway 网关，使用 --node 可针对特定节点。

相关内容：

- 执行审批：执行审批
- 节点：节点

常用命令

代码： openclaw approvals get

代码： openclaw approvals get --node <id|name|ip>

代码： openclaw approvals get --gateway

从文件替换审批

代码： openclaw approvals set --file ./exec-approvals.json

代码： openclaw approvals set --node <id|name|ip> --file ./exec-approvals.json

代码： openclaw approvals set --gateway --file ./exec-approvals.json

允许列表辅助命令

代码： openclaw approvals allowlist add "~/Projects/**/bin/rg"

代码： openclaw approvals allowlist add --agent main --node <id|name|ip>
"/usr/bin/uptime"

代码： openclaw approvals allowlist add --agent "*" "/usr/bin/uname"

代码： openclaw approvals allowlist remove "~/Projects/**/bin/rg"

注意事项

- `--node` 使用与 `openclaw nodes` 相同的解析器 (`id`、`name`、`ip` 或 `id` 前缀)。
- `--agent` 默认为 `""`，表示适用于所有智能体。
- 节点主机必须公开 `system.execApprovals.get/set` (macOS 应用或无头节点主机)。
- 审批文件按主机存储在 `~/openclaw/exec-approvals.json`。

5. `openclaw browser`

`openclaw browser`

管理 OpenClaw

的浏览器控制服务器并运行浏览器操作 (标签页、快照、截图、导航、点击、输入)。

相关：

- 浏览器工具 + API：浏览器工具
- Chrome 扩展中继：Chrome 扩展

通用标志

- `--url <gatewayWsUrl>`：Gateway 网关 WebSocket URL (默认从配置获取)。
- `--token <token>`：Gateway 网关令牌 (如果需要)。
- `--timeout <ms>`：请求超时 (毫秒)。
- `--browser-profile <name>`：选择浏览器配置文件 (默认从配置获取)。
- `--json`：机器可读输出 (在支持的地方)。

快速开始 (本地)

代码：`openclaw browser --browser-profile chrome tabs`

代码：`openclaw browser --browser-profile openclaw start`

代码：`openclaw browser --browser-profile openclaw open https://example.com`

代码：`openclaw browser --browser-profile openclaw snapshot`

配置文件

配置文件是命名的浏览器路由配置。实际上：

- `openclaw`：启动/附加到专用的 OpenClaw 管理的 Chrome 实例 (隔离的用户数据目录)。

- chrome：通过 Chrome 扩展中继控制你现有的 Chrome 标签页。

代码：openclaw browser profiles

代码：openclaw browser create-profile --name work --color "#FF5A36"

代码：openclaw browser delete-profile --name work

使用特定配置文件：

代码：openclaw browser --browser-profile work tabs

标签页

代码：openclaw browser tabs

代码：openclaw browser open https://docs.openclaw.ai

代码：openclaw browser focus <targetId>

代码：openclaw browser close <targetId>

快照 / 截图 / 操作

快照：

代码：openclaw browser snapshot

截图：

代码：openclaw browser screenshot

导航/点击/输入（基于 ref 的 UI 自动化）：

代码：openclaw browser navigate https://example.com

代码：openclaw browser click <ref>

代码：openclaw browser type <ref> "hello"

Chrome 扩展中继（通过工具栏按钮附加）

此模式让智能体控制你手动附加的现有 Chrome 标签页（不会自动附加）。

将未打包的扩展安装到稳定路径：

代码：openclaw browser extension install

代码：openclaw browser extension path

然后 Chrome → chrome://extensions → 启用"开发者模式" → "加载已解压的扩展程序" → 选择打印的文件夹。

完整指南：Chrome 扩展

远程浏览器控制 (node host 代理)

如果 Gateway 网关与浏览器运行在不同的机器上，在有 Chrome/Brave/Edge/Chromium 的机器上运行 node host。Gateway 网关会将浏览器操作代理到该节点（无需单独的浏览器控制服务器）。

使用 gateway.nodes.browser.mode 控制自动路由，使用 gateway.nodes.browser.node 在连接多个节点时固定特定节点。

安全 + 远程设置：浏览器工具、远程访问、Tailscale、安全

6. `openclaw channels`

`openclaw channels`

管理 Gateway 网关上的聊天渠道账户及其运行时状态。

相关文档：

- 渠道指南：渠道
- Gateway 网关配置：配置

常用命令

代码：openclaw channels list

代码：openclaw channels status

代码：openclaw channels capabilities

代码：openclaw channels capabilities --channel discord --target channel:123

代码：openclaw channels resolve --channel slack "#general" "@jane"

代码：openclaw channels logs --channel all

添加/删除账户

代码：openclaw channels add --channel telegram --token <bot-token>

代码：openclaw channels remove --channel telegram --delete

提示：openclaw channels add --help 显示每个渠道的标志（token、app token、signal-cli 路径等）。

登录/登出（交互式）

代码：openclaw channels login --channel whatsapp

代码：openclaw channels logout --channel whatsapp

故障排除

- 运行 openclaw status --deep 进行全面探测。
- 使用 openclaw doctor 获取引导式修复。
- openclaw channels list 输出 Claude: HTTP 403 ... user:profile → 用量快照需要 user:profile 权限范围。使用 --no-usage，或提供 claude.ai 会话密钥（CLAUDE_WEB_SESSION_KEY / CLAUDE_WEB_COOKIE），或通过 Claude Code CLI 重新授权。

能力探测

获取提供商能力提示（可用的 intents/scopes）以及静态功能支持：

代码：openclaw channels capabilities

代码：openclaw channels capabilities --channel discord --target channel:123

说明：

- --channel 是可选的；省略它可列出所有渠道（包括扩展）。
- --target 接受 channel:<id> 或原始数字频道 id，仅适用于 Discord。
- 探测是特定于提供商的：Discord intents + 可选的频道权限；Slack bot + user scopes；Telegram bot 标志 + webhook；Signal daemon 版本；MS Teams app token + Graph roles/scopes（在已知处标注）。没有探测功能的渠道报告 Probe: unavailable。

解析名称为 ID

使用提供商目录将渠道/用户名称解析为 ID：

代码：openclaw channels resolve --channel slack "#general" "@jane"

代码：openclaw channels resolve --channel discord "My Server/#support" "@someone"

代码：openclaw channels resolve --channel matrix "Project Room"

说明：

- 使用 `--kind user|group|auto` 强制指定目标类型。
- 当多个条目共享相同名称时，解析优先选择活跃的匹配项。

7. `openclaw config`

`openclaw config`

配置辅助命令：通过路径获取/设置/取消设置值。不带子命令运行将打开配置向导（与 `openclaw configure` 相同）。

示例

代码：`openclaw config get browser.executablePath`

代码：`openclaw config set browser.executablePath "/usr/bin/google-chrome"`

代码：`openclaw config set agents.defaults.heartbeat.every "2h"`

代码：`openclaw config set agents.list[0].tools.exec.node "node-id-or-name"`

代码：`openclaw config unset tools.web.search.apiKey`

路径

路径使用点号或括号表示法：

代码：`openclaw config get agents.defaults.workspace`

代码：`openclaw config get agents.list[0].id`

使用智能体列表索引来定位特定智能体：

代码：`openclaw config get agents.list`

代码：`openclaw config set agents.list[1].tools.exec.node "node-id-or-name"`

值

值会尽可能解析为 JSON5；否则将被视为字符串。

使用 `--json` 强制要求 JSON5 解析。

代码：`openclaw config set agents.defaults.heartbeat.every "0m"`

代码：`openclaw config set gateway.port 19001 --json`

代码：`openclaw config set channels.whatsapp.groups ['*'] --json`

编辑后请重启 Gateway 网关。

8. `openclaw configure`

`openclaw configure`

用于设置凭证、设备和智能体默认值的交互式提示。

注意：模型部分现在包含一个用于 `agents.defaults.models` 允许列表的多选项（显示在 `/model` 和模型选择器中的内容）。

提示：不带子命令的 `openclaw config` 会打开相同的向导。使用 `openclaw config get|set|unset` 进行非交互式编辑。

相关内容：

- Gateway 网关配置参考：配置
- Config CLI：Config

注意事项：

- 选择 Gateway 网关运行位置始终会更新 `gateway.mode`。如果这是你唯一需要的，可以不选择其他部分直接选择"继续"。
- 面向渠道的服务（Slack/Discord/Matrix/Microsoft Teams）在设置期间会提示输入频道/房间允许列表。你可以输入名称或 ID；向导会尽可能将名称解析为 ID。

示例

代码：`openclaw configure`

代码：`openclaw configure --section models --section channels`

9. `openclaw cron`

`openclaw cron`

管理 Gateway 网关调度器的 cron 作业。

相关内容：

- Cron 作业：Cron 作业

提示：运行 `openclaw cron --help` 查看完整的命令集。

说明：隔离式 cron add 任务默认使用 --announce 投递摘要。使用 --no-deliver 仅内部运行。
--deliver 仍作为 --announce 的弃用别名保留。

说明：一次性 (--at) 任务成功后默认删除。使用 --keep-after-run 保留。

常见编辑

更新投递设置而不更改消息：

代码：openclaw cron edit <job-id> --announce --channel telegram --to "123456789"

为隔离的作业禁用投递：

代码：openclaw cron edit <job-id> --no-deliver

10. `openclaw dashboard`

`openclaw dashboard`

使用当前认证信息打开控制界面。

代码：openclaw dashboard

代码：openclaw dashboard --no-open

11. `openclaw devices`

`openclaw devices`

管理设备配对请求和设备范围的 token。

命令

`openclaw devices list`

列出待处理的配对请求和已配对的设备。

代码：openclaw devices list

代码：openclaw devices list --json

`openclaw devices approve <requestId>`

批准待处理的设备配对请求。

代码：openclaw devices approve <requestId>

`openclaw devices reject <requestId>`

拒绝待处理的设备配对请求。

代码：openclaw devices reject <requestId>

`openclaw devices rotate --device <id> --role <role> [--scope <scope...>]`

为特定角色轮换设备 token（可选更新 scope）。

代码：openclaw devices rotate --device <deviceId> --role operator --scope operator.read --scope operator.write

`openclaw devices revoke --device <id> --role <role>`

为特定角色撤销设备 token。

代码：openclaw devices revoke --device <deviceId> --role node

通用选项

- --url <url> : Gateway 网关 WebSocket URL（配置后默认使用 gateway.remote.url）。
- --token <token> : Gateway 网关 token（如需要）。
- --password <password> : Gateway 网关密码（密码认证）。
- --timeout <ms> : RPC 超时。
- --json : JSON 输出（推荐用于脚本）。

注意事项

- Token 轮换会返回新 token（敏感信息）。请像对待密钥一样对待它。
- 这些命令需要 operator.pairing（或 operator.admin）scope。

12. `openclaw directory`

`openclaw directory`

对支持目录功能的渠道进行查找（联系人/对等方、群组和我"）。

通用参数

- `--channel <name>` : 渠道 ID/别名 (配置了多个渠道时为必填; 仅配置一个渠道时自动选择)
- `--account <id>` : 账号 ID (默认: 渠道默认账号)
- `--json` : 输出 JSON 格式

说明

- `directory` 用于帮助你查找可粘贴到其他命令中的 ID (特别是 `openclaw message send --target ...`) 。
- 对于许多渠道, 结果来源于配置 (允许列表/已配置的群组), 而非实时的提供商目录。
- 默认输出为以制表符分隔的 id (有时包含 name) ; 脚本中请使用 `--json` 。

将结果用于 `message send`

代码: `openclaw directory peers list --channel slack --query "U0"`

代码: `openclaw message send --channel slack --target user:U012ABCDEF --message "hello"`

ID 格式 (按渠道)

- WhatsApp : `+15551234567` (私聊) , `1234567890-1234567890@g.us` (群组)
- Telegram : `@username` 或数字聊天 ID ; 群组为数字 ID
- Slack : `user:U...` 和 `channel:C...`
- Discord : `user:<id>` 和 `channel:<id>`
- Matrix (插件) : `user:@user:server` 、 `room:!roomId:server` 或 `#alias:server`
- Microsoft Teams (插件) : `user:<id>` 和 `conversation:<id>`
- Zalo (插件) : 用户 ID (Bot API)
- Zalo Personal / zalouser (插件) : 来自 `zca` 的会话 ID (私聊/群组) (`me` 、 `friend list` 、 `group list`)

Self ("我")

代码: `openclaw directory self --channel zalouser`

Peers (联系人/用户)

代码: `openclaw directory peers list --channel zalouser`

代码：openclaw directory peers list --channel zalouser --query "name"

代码：openclaw directory peers list --channel zalouser --limit 50

群组

代码：openclaw directory groups list --channel zalouser

代码：openclaw directory groups list --channel zalouser --query "work"

代码：openclaw directory groups members --channel zalouser --group-id <id>

13. `openclaw dns`

`openclaw dns`

用于广域设备发现（Tailscale + CoreDNS）的 DNS 辅助工具。目前专注于 macOS + Homebrew CoreDNS。

相关内容：

- Gateway 网关设备发现：设备发现
- 广域设备发现配置：配置

设置

代码：openclaw dns setup

代码：openclaw dns setup --apply

14. `openclaw docs`

`openclaw docs`

搜索实时文档索引。

代码：openclaw docs browser extension

代码：openclaw docs sandbox allowHostControl

15. `openclaw doctor`

`openclaw doctor`

Gateway 网关和渠道的健康检查 + 快速修复。

相关内容：

- 故障排除：故障排除
- 安全审计：安全

示例

代码：openclaw doctor

代码：openclaw doctor --repair

代码：openclaw doctor --deep

注意事项：

- 交互式提示（如钥匙串/OAuth 修复）仅在 stdin 是 TTY 且未设置 --non-interactive 时运行。无头运行（cron、Telegram、无终端）将跳过提示。
- --fix（--repair 的别名）会将备份写入
~/.openclaw/openclaw.json.bak，并删除未知的配置键，同时列出每个删除项。

macOS：`launchctl` 环境变量覆盖

如果你之前运行过 `launchctl setenv OPENCLAW_GATEWAY_TOKEN ...`（或 `...PASSWORD`），该值会覆盖你的配置文件，并可能导致持续的"未授权"错误。

代码：launchctl getenv OPENCLAW_GATEWAY_TOKEN

代码：launchctl getenv OPENCLAW_GATEWAY_PASSWORD

代码：launchctl unsetenv OPENCLAW_GATEWAY_TOKEN

代码：launchctl unsetenv OPENCLAW_GATEWAY_PASSWORD

16. Gateway 网关 CLI

Gateway 网关 CLI

Gateway 网关是 OpenClaw 的 WebSocket 服务器（渠道、节点、会话、hooks）。

本页中的子命令位于 `openclaw gateway ...` 下。

相关文档：

- /gateway/bonjour
- /gateway/discovery

- /gateway/configuration

运行 Gateway 网关

运行本地 Gateway 网关进程：

代码：openclaw gateway

前台运行别名：

代码：openclaw gateway run

注意事项：

- 默认情况下，除非在 ~/.openclaw/openclaw.json 中设置了 gateway.mode=local，否则 Gateway 网关将拒绝启动。使用 --allow-unconfigured 进行临时/开发运行。
- 在没有认证的情况下绑定到 loopback 之外的地址会被阻止（安全护栏）。
- SIGUSR1 在授权时触发进程内重启（启用 commands.restart 或使用 gateway 工具/config apply/update）。
- SIGINT/SIGTERM 处理程序会停止 Gateway 网关进程，但不会恢复任何自定义终端状态。如果你用 TUI 或 raw-mode 输入包装 CLI，请在退出前恢复终端。

选项

- --port <port>：WebSocket 端口（默认来自配置/环境变量；通常为 18789）。
- --bind <loopback|lan|tailnet|auto|custom>：监听器绑定模式。
- --auth <token|password>：认证模式覆盖。
- --token <token>：令牌覆盖（同时为进程设置 OPENCLAW_GATEWAY_TOKEN）。
- --password <password>：密码覆盖（同时为进程设置 OPENCLAW_GATEWAY_PASSWORD）。
- --tailscale <off|serve|funnel>：通过 Tailscale 暴露 Gateway 网关。
- --tailscale-reset-on-exit：关闭时重置 Tailscale serve/funnel 配置。
- --allow-unconfigured：允许在配置中没有 gateway.mode=local 的情况下启动 Gateway 网关。
- --dev：如果缺失则创建开发配置 + 工作区（跳过 BOOTSTRAP.md）。
- --reset：重置开发配置 + 凭证 + 会话 + 工作区（需要 --dev）。
- --force：启动前杀死所选端口上的任何现有监听器。

- `--verbose` : 详细日志。
- `--claude-cli-logs` : 仅在控制台显示 `claude-cli` 日志（并启用其 `stdout/stderr`）。
- `--ws-log <auto|full|compact>` : WebSocket 日志样式（默认 `auto`）。
- `--compact` : `--ws-log compact` 的别名。
- `--raw-stream` : 将原始模型流事件记录到 `jsonl`。
- `--raw-stream-path <path>` : 原始流 `jsonl` 路径。

查询运行中的 Gateway 网关

所有查询命令使用 WebSocket RPC。

输出模式：

- 默认：人类可读（TTY 中带颜色）。
- `--json` : 机器可读 JSON（无样式/进度指示器）。
- `--no-color`（或 `NO_COLOR=1`）：禁用 ANSI 但保持人类可读布局。

共享选项（在支持的地方）：

- `--url <url>` : Gateway 网关 WebSocket URL。
- `--token <token>` : Gateway 网关令牌。
- `--password <password>` : Gateway 网关密码。
- `--timeout <ms>` : 超时/预算（因命令而异）。
- `--expect-final` : 等待"最终"响应（智能体调用）。

`gateway health`

代码：`openclaw gateway health --url ws://127.0.0.1:18789`

`gateway status`

`gateway status` 显示 Gateway 网关服务（`launchd/systemd/schtasks`）以及可选的 RPC 探测。

代码：`openclaw gateway status`

代码：`openclaw gateway status --json`

选项：

- `--url <url>` : 覆盖探测 URL 。
- `--token <token>` : 探测的令牌认证 。
- `--password <password>` : 探测的密码认证 。
- `--timeout <ms>` : 探测超时（默认 10000） 。
- `--no-probe` : 跳过 RPC 探测（仅服务视图） 。
- `--deep` : 也扫描系统级服务 。

`gateway probe`

gateway probe 是"调试一切"命令。它始终探测：

- 你配置的远程 Gateway 网关（如果设置了），以及
- localhost (loopback) 即使配置了远程也会探测 。

如果多个 Gateway

网关可达，它会打印所有。当你使用隔离的配置文件/端口（例如救援机器人）时支持多个 Gateway 网关，但大多数安装仍然运行单个 Gateway 网关 。

代码：openclaw gateway probe

代码：openclaw gateway probe --json

通过 SSH 远程（Mac 应用对等）

macOS 应用的"通过 SSH 远程"模式使用本地端口转发，因此远程 Gateway 网关（可能仅绑定到 loopback）变得可以通过 `ws://127.0.0.1:<port>` 访问 。

CLI 等效命令：

代码：openclaw gateway probe --ssh user@gateway-host

选项：

- `--ssh <target>` : user@host 或 user@host:port（端口默认为 22） 。
- `--ssh-identity <path>` : 身份文件 。
- `--ssh-auto` : 选择第一个发现的 Gateway 网关主机作为 SSH 目标（仅限局域网/WAB） 。

配置（可选，用作默认值）：

- gateway.remote.sshTarget
- gateway.remote.sshIdentity

`gateway call <method>`

低级 RPC 辅助工具。

代码：openclaw gateway call status

代码：openclaw gateway call logs.tail --params '{"sinceMs": 60000}'

管理 Gateway 网关服务

代码：openclaw gateway install

代码：openclaw gateway start

代码：openclaw gateway stop

代码：openclaw gateway restart

代码：openclaw gateway uninstall

注意事项：

- gateway install 支持 --port、--runtime、--token、--force、--json。
- 生命周期命令接受 --json 用于脚本。

发现 Gateway 网关 (Bonjour)

gateway discover 扫描 Gateway 网关信标 (_openclaw-gw._tcp)。

- 组播 DNS-SD：local。
- 单播 DNS-SD（广域 Bonjour）：选择一个域（示例：openclaw.internal.）并设置分割 DNS + DNS 服务器；参见 /gateway/bonjour

只有启用了 Bonjour 发现（默认）的 Gateway 网关才会广播信标。

广域发现记录包括 (TXT)：

- role (Gateway 网关角色提示)
- transport (传输提示，例如 gateway)
- gatewayPort (WebSocket 端口，通常为 18789)
- sshPort (SSH 端口；如果不存在则默认为 22)
- tailnetDns (MagicDNS 主机名，如果可用)
- gatewayTls / gatewayTlsSha256 (TLS 启用 + 证书指纹)

- cliPath（远程安装的可选提示）

`gateway discover`

代码：openclaw gateway discover

选项：

- --timeout <ms>：每个命令的超时（浏览/解析）；默认 2000。
- --json：机器可读输出（同时禁用样式/进度指示器）。

示例：

代码：openclaw gateway discover --timeout 4000

代码：openclaw gateway discover --json | jq '.beacons[].wsUrl'

17. `openclaw health`

`openclaw health`

从运行中的 Gateway 网关获取健康状态。

代码：openclaw health

代码：openclaw health --json

代码：openclaw health --verbose

注意：

- --verbose 运行实时探测，并在配置了多个账户时打印每个账户的耗时。
- 当配置了多个智能体时，输出包括每个智能体的会话存储。

18. `openclaw hooks`

`openclaw hooks`

管理智能体钩子（针对 /new、/reset 等命令以及 Gateway 网关启动的事件驱动自动化）。

相关内容：

- 钩子：钩子
- 插件钩子：插件

列出所有钩子

代码：openclaw hooks list

列出从工作区、托管目录和内置目录中发现的所有钩子。

选项：

- --eligible：仅显示符合条件的钩子（满足要求）
- --json：以 JSON 格式输出
- -v, --verbose：显示详细信息，包括缺失的要求

示例输出：

代码：Hooks (3/3 ready)

代码：Ready:

代码：boot-md - Run BOOT.md on gateway startup

代码：command-logger - Log all command events to a centralized audit file

代码：session-memory - Save session context to memory when /new command is issued

示例（详细模式）：

代码：openclaw hooks list --verbose

显示不符合条件的钩子缺失的要求。

示例（JSON）：

代码：openclaw hooks list --json

返回结构化 JSON，供程序化使用。

获取钩子信息

代码：openclaw hooks info <name>

显示特定钩子的详细信息。

参数：

- <name>：钩子名称（例如 session-memory）

选项：

- --json：以 JSON 格式输出

示例：

代码：openclaw hooks info session-memory

输出：

代码：session-memory Ready

代码：Save session context to memory when /new command is issued

代码：Details:

代码：Source: openclaw-bundled

代码：Path: /path/to/openclaw/hooks/bundled/session-memory/HOOK.md

代码：Handler: /path/to/openclaw/hooks/bundled/session-memory/handler.ts

代码：Homepage: <https://docs.openclaw.ai/automation/hooks#session-memory>

代码：Events: command:new

代码：Requirements:

代码：Config: workspace.dir

检查钩子资格

代码：openclaw hooks check

显示钩子资格状态摘要（有多少已就绪，有多少未就绪）。

选项：

- --json：以 JSON 格式输出

示例输出：

代码：Hooks Status

代码：Total hooks: 4

代码：Ready: 4

代码：Not ready: 0

启用钩子

代码：openclaw hooks enable <name>

通过将特定钩子添加到配置（~/openclaw/config.json）来启用它。

注意：由插件管理的钩子在 openclaw hooks list 中显示 plugin:<id>，无法在此处启用/禁用。请改为启用/禁用该插件。

参数：

- <name>：钩子名称（例如 session-memory）

示例：

代码：openclaw hooks enable session-memory

输出：

代码：Enabled hook: session-memory

执行操作：

- 检查钩子是否存在且符合条件
- 在配置中更新 hooks.internal.entries.<name>.enabled = true
- 将配置保存到磁盘

启用后：

- 重启 Gateway 网关以重新加载钩子（macOS 上重启菜单栏应用，或在开发环境中重启 Gateway 网关进程）。

禁用钩子

代码：openclaw hooks disable <name>

通过更新配置来禁用特定钩子。

参数：

- <name>：钩子名称（例如 command-logger）

示例：

代码：openclaw hooks disable command-logger

输出：

代码：Disabled hook: command-logger

禁用后：

- 重启 Gateway 网关以重新加载钩子

安装钩子

代码：openclaw hooks install <path-or-spec>

从本地文件夹/压缩包或 npm 安装钩子包。

执行操作：

- 将钩子包复制到 ~/.openclaw/hooks/<id>
- 在 hooks.internal.entries. 中启用已安装的钩子
- 在 hooks.internal.installs 下记录安装信息

选项：

- -l, --link：链接本地目录而不是复制（将其添加到 hooks.internal.load.extraDirs）

支持的压缩包格式：.zip、.tgz、.tar.gz、.tar

示例：

代码：# 本地目录

代码：openclaw hooks install ./my-hook-pack

代码：# 本地压缩包

代码：openclaw hooks install ./my-hook-pack.zip

代码：# NPM 包

代码：openclaw hooks install @openclaw/my-hook-pack

代码：# 链接本地目录而不复制

代码：openclaw hooks install -l ./my-hook-pack

更新钩子

代码：openclaw hooks update <id>

代码：openclaw hooks update --all

更新已安装的钩子包（仅限 npm 安装）。

选项：

- --all：更新所有已跟踪的钩子包
- --dry-run：显示将要进行的更改，但不写入

内置钩子

session-memory

在你执行 /new 时将会话上下文保存到记忆中。

启用：

代码：openclaw hooks enable session-memory

输出：~/.openclaw/workspace/memory/YYYY-MM-DD-slug.md

参见：session-memory 文档

command-logger

将所有命令事件记录到集中的审计文件中。

启用：

代码：openclaw hooks enable command-logger

输出：~/.openclaw/logs/commands.log

查看日志：

代码：# 最近的命令

代码：tail -n 20 ~/.openclaw/logs/commands.log

代码：# 格式化输出

代码：cat ~/.openclaw/logs/commands.log | jq .

代码：# 按操作过滤

代码：grep "action":"new" ~/.openclaw/logs/commands.log | jq .

参见：command-logger 文档

boot-md

在 Gateway 网关启动时（渠道启动后）运行 BOOT.md。

事件：gateway:startup

启用：

代码：openclaw hooks enable boot-md

参见：boot-md 文档

19. CLI 参考

CLI 参考

本页描述当前的 CLI 行为。如果命令发生变化，请更新此文档。

命令页面

- setup
- onboard
- configure
- config
- doctor
- dashboard
- reset
- uninstall
- update
- message
- agent
- agents
- acp

- status
- health
- sessions
- gateway
- logs
- system
- models
- memory
- nodes
- devices
- node
- approvals
- sandbox
- tui
- browser
- cron
- dns
- docs
- hooks
- webhooks
- pairing
- plugins（插件命令）
- channels
- security
- skills
- voicecall（插件；如已安装）

全局标志

- --dev：将状态隔离到 ~/.openclaw-dev 下并调整默认端口。
- --profile <name>：将状态隔离到 ~/.openclaw-<name> 下。
- --no-color：禁用 ANSI 颜色。

- `--update` : `openclaw update` 的简写（仅限源码安装）。
- `-V`、`--version`、`-v` : 打印版本并退出。

输出样式

- ANSI 颜色和进度指示器仅在 TTY 会话中渲染。
- OSC-8 超链接在支持的终端中渲染为可点击链接；否则回退到纯 URL。
- `--json`（以及支持的地方使用 `--plain`）禁用样式以获得干净输出。
- `--no-color` 禁用 ANSI 样式；也支持 `NO_COLOR=1`。
- 长时间运行的命令显示进度指示器（支持时使用 OSC 9;4）。

颜色调色板

OpenClaw 在 CLI 输出中使用龙虾调色板。

- `accent` (`#FF5A2D`) : 标题、标签、主要高亮。
- `accentBright` (`#FF7A3D`) : 命令名称、强调。
- `accentDim` (`#D14A22`) : 次要高亮文本。
- `info` (`#FF8A5B`) : 信息性值。
- `success` (`#2FBF71`) : 成功状态。
- `warn` (`#FFB020`) : 警告、回退、注意。
- `error` (`#E23D2D`) : 错误、失败。
- `muted` (`#8B7F77`) : 弱化、元数据。

调色板权威来源：`src/terminal/palette.ts`（又名"lobster seam"）。

命令树

代码：`openclaw [--dev] [--profile <name>] <command>`

代码：`setup`

代码：`onboard`

代码：`configure`

代码：`config`

代码：`get`

代码：`set`

代码：unset

代码：doctor

代码：security

代码：audit

代码：reset

代码：uninstall

代码：update

代码：channels

代码：list

代码：status

代码：logs

代码：add

代码：remove

代码：login

代码：logout

代码：skills

代码：list

代码：info

代码：check

代码：plugins

代码：list

代码：info

代码：install

代码：enable

代码：disable

代码：doctor

代码：memory

代码：status

代码：index

代码：search

代码：message

代码：agent

代码：agents

代码：list

代码：add

代码：delete

代码：acp

代码：status

代码：health

代码：sessions

代码：gateway

代码：call

代码：health

代码：status

代码：probe

代码：discover

代码：install

代码：uninstall

代码：start

代码：stop

代码：restart

代码：run

代码：logs

代码：system

代码：event

代码：heartbeat last|enable|disable

代码：presence

代码：models

代码：list

代码：status

代码： set

代码： set-image

代码： aliases list|add|remove

代码： fallbacks list|add|remove|clear

代码： image-fallbacks list|add|remove|clear

代码： scan

代码： auth add|setup-token|paste-token

代码： auth order get|set|clear

代码： sandbox

代码： list

代码： recreate

代码： explain

代码： cron

代码： status

代码： list

代码： add

代码： edit

代码： rm

代码： enable

代码： disable

代码： runs

代码： run

代码： nodes

代码： devices

代码： node

代码： run

代码： status

代码： install

代码： uninstall

代码： start

代码：stop

代码：restart

代码：approvals

代码：get

代码：set

代码：allowlist add|remove

代码：browser

代码：status

代码：start

代码：stop

代码：reset-profile

代码：tabs

代码：open

代码：focus

代码：close

代码：profiles

代码：create-profile

代码：delete-profile

代码：screenshot

代码：snapshot

代码：navigate

代码：resize

代码：click

代码：type

代码：press

代码：hover

代码：drag

代码：select

代码：upload

代码：fill

代码：dialog

代码：wait

代码：evaluate

代码：console

代码：pdf

代码：hooks

代码：list

代码：info

代码：check

代码：enable

代码：disable

代码：install

代码：update

代码：webhooks

代码：gmail setup|run

代码：pairing

代码：list

代码：approve

代码：docs

代码：dns

代码：setup

代码：tui

注意：插件可以添加额外的顶级命令（例如 openclaw voicecall）。

安全

- openclaw security audit — 审计配置 + 本地状态中常见的安全隐患。
- openclaw security audit --deep — 尽力进行实时 Gateway 网关探测。
- openclaw security audit --fix — 收紧安全默认值并 chmod 状态/配置。

插件

管理扩展及其配置：

- `openclaw plugins list` — 发现插件（使用 `--json` 获取机器可读输出）。
- `openclaw plugins info <id>` — 显示插件详情。
- `openclaw plugins install <path|.tgz|npm-spec>` — 安装插件（或将插件路径添加到 `plugins.load.paths`）。
- `openclaw plugins enable <id> / disable <id>` — 切换 `plugins.entries.<id>.enabled`。
- `openclaw plugins doctor` — 报告插件加载错误。

大多数插件更改需要重启 Gateway 网关。参见 `/plugin`。

记忆

对 `MEMORY.md` + `memory/.md` 进行向量搜索：

- `openclaw memory status` — 显示索引统计。
- `openclaw memory index` — 重新索引记忆文件。
- `openclaw memory search "<query>"` — 对记忆进行语义搜索。

聊天斜杠命令

聊天消息支持 `/...` 命令（文本和原生）。参见 `/tools/slash-commands`。

亮点：

- `/status` 用于快速诊断。
- `/config` 用于持久化配置更改。
- `/debug` 用于仅运行时的配置覆盖（内存中，不写入磁盘；需要 `commands.debug: true`）。

设置 + 新手引导

``setup``

初始化配置 + 工作区。

选项：

- `--workspace <dir>`：智能体工作区路径（默认 `~/.openclaw/workspace`）。
- `--wizard`：运行新手引导向导。
- `--non-interactive`：无提示运行向导。

- --mode <local|remote> : 向导模式。
- --remote-url <url> : 远程 Gateway 网关 URL。
- --remote-token <token> : 远程 Gateway 网关令牌。

当存在任何向导标志（--non-interactive、--mode、--remote-url、--remote-token）时，向导自动运行。

`onboard`

交互式向导，用于设置 Gateway 网关、工作区和 Skills。

选项：

- --workspace <dir>
- --reset（在向导之前重置配置 + 凭证 + 会话 + 工作区）
- --non-interactive
- --mode <local|remote>
- --flow <quickstart|advanced|manual>（manual 是 advanced 的别名）
- --auth-choice <setup-token|token|chutes|openai-codex|openai-api-key|openrouter-api-key|ai-gateway-api-key|moonshot-api-key|kimi-code-api-key|synthetic-api-key|venice-api-key|gemini-api-key|zai-api-key|apiKey|minimax-api|minimax-api-lightning|open code-zen|skip>
- --token-provider <id>（非交互式；与 --auth-choice token 配合使用）
- --token <token>（非交互式；与 --auth-choice token 配合使用）
- --token-profile-id <id>（非交互式；默认：<provider>:manual）
- --token-expires-in <duration>（非交互式；例如 365d、12h）
- --anthropic-api-key <key>
- --openai-api-key <key>
- --openrouter-api-key <key>
- --ai-gateway-api-key <key>
- --moonshot-api-key <key>
- --kimi-code-api-key <key>
- --gemini-api-key <key>
- --zai-api-key <key>
- --minimax-api-key <key>

- `--opencode-zen-api-key <key>`
- `--gateway-port <port>`
- `--gateway-bind <loopback|lan|tailnet|auto|custom>`
- `--gateway-auth <token|password>`
- `--gateway-token <token>`
- `--gateway-password <password>`
- `--remote-url <url>`
- `--remote-token <token>`
- `--tailscale <off|serve|funnel>`
- `--tailscale-reset-on-exit`
- `--install-daemon`
- `--no-install-daemon`（别名：`--skip-daemon`）
- `--daemon-runtime <node|bun>`
- `--skip-channels`
- `--skip-skills`
- `--skip-health`
- `--skip-ui`
- `--node-manager <npm|pnpm|bun>`（推荐 pnpm；不建议将 bun 用于 Gateway 网关运行时）
- `--json`

`configure`

交互式配置向导（模型、渠道、Skills、Gateway 网关）。

`config`

非交互式配置辅助工具（get/set/unset）。不带子命令运行 `openclaw config` 会启动向导。

子命令：

- `config get <path>`：打印配置值（点/括号路径）。
- `config set <path> <value>`：设置值（JSON5 或原始字符串）。
- `config unset <path>`：删除值。

`doctor`

健康检查 + 快速修复（配置 + Gateway 网关 + 旧版服务）。

选项：

- --no-workspace-suggestions：禁用工作区记忆提示。
- --yes：无提示接受默认值（无头模式）。
- --non-interactive：跳过提示；仅应用安全迁移。
- --deep：扫描系统服务以查找额外的 Gateway 网关安装。

渠道辅助工具

`channels`

管理聊天渠道账户（WhatsApp/Telegram/Discord/Google Chat/Slack/Mattermost（插件）/Signal/iMessage/MS Teams）。

子命令：

- channels list：显示已配置的渠道和认证配置文件。
- channels status：检查 Gateway 网关可达性和渠道健康状况（--probe 运行额外检查；使用 openclaw health 或 openclaw status --deep 进行 Gateway 网关健康探测）。
- 提示：channels status 在检测到常见配置错误时会打印带有建议修复的警告（然后指向 openclaw doctor）。
- channels logs：显示 Gateway 网关日志文件中最近的渠道日志。
- channels add：不传标志时使用向导式设置；标志切换到非交互模式。
- channels remove：默认禁用；传 --delete 可无提示删除配置条目。
- channels login：交互式渠道登录（仅限 WhatsApp Web）。
- channels logout：登出渠道会话（如支持）。

通用选项：

- --channel <name>：whatsapp|telegram|discord|googlechat|slack|mattermost|signal|imessage|msteams
- --account <id>：渠道账户 id（默认 default）
- --name <label>：账户的显示名称

channels login 选项：

- --channel <channel> (默认 whatsapp；支持 whatsapp/web)
- --account <id>
- --verbose

channels logout 选项：

- --channel <channel> (默认 whatsapp)
- --account <id>

channels list 选项：

- --no-usage：跳过模型提供商用量/配额快照（仅限 OAuth/API 支持的）。
- --json：输出 JSON（除非设置 --no-usage，否则包含用量）。

channels logs 选项：

- --channel <name|all> (默认 all)
- --lines <n> (默认 200)
- --json

更多详情：</concepts/oauth>

示例：

代码：openclaw channels add --channel telegram --account alerts --name "Alerts Bot" --token \$TELEGRAM_BOT_TOKEN

代码：openclaw channels add --channel discord --account work --name "Work Bot" --token \$DISCORD_BOT_TOKEN

代码：openclaw channels remove --channel discord --account work --delete

代码：openclaw channels status --probe

代码：openclaw status --deep

`skills`

列出和检查可用的 Skills 及就绪信息。

子命令：

- skills list : 列出 Skills (无子命令时的默认行为) 。
- skills info <name> : 显示单个 Skill 的详情 。
- skills check : 就绪与缺失需求的摘要 。

选项 :

- --eligible : 仅显示就绪的 Skills 。
- --json : 输出 JSON (无样式) 。
- -v 、 --verbose : 包含缺失需求详情 。

提示 : 使用 npx clawhub 搜索、安装和同步 Skills 。

`pairing`

批准跨渠道的私信配对请求 。

子命令 :

- pairing list <channel> [--json]
- pairing approve <channel> <code> [--notify]

`webhooks gmail`

Gmail Pub/Sub 钩子设置 + 运行器 。

子命令 :

- webhooks gmail setup (需要 --account <email> ; 支持 --project 、 --topic 、 --subscription 、 --label 、 --hook-url 、 --hook-token 、 --push-token 、 --bind 、 --port 、 --path 、 --include-body 、 --max-bytes 、 --renew-minutes 、 --tailscale 、 --tailscale-path 、 --tailscale-target 、 --push-endpoint 、 --json)
- webhooks gmail run (相同标志的运行时覆盖)

`dns setup`

广域发现 DNS 辅助工具 (CoreDNS + Tailscale) 。

选项 :

- --apply : 安装/更新 CoreDNS 配置 (需要 sudo ; 仅限 macOS) 。

消息 + 智能体

`message`

统一的出站消息 + 渠道操作。

参见：[/cli/message](#)

子命令：

- `message send|poll|react|reactions|read|edit|delete|pin|unpin|pins|permissions|search|timeout|kick|ban`
- `message thread <create|list|reply>`
- `message emoji <list|upload>`
- `message sticker <send|upload>`
- `message role <info|add|remove>`
- `message channel <info|list>`
- `message member info`
- `message voice status`
- `message event <list|create>`

示例：

- `openclaw message send --target +15555550123 --message "Hi"`
- `openclaw message poll --channel discord --target channel:123 --poll-question "Snack?" --poll-option Pizza --poll-option Sushi`

`agent`

通过 Gateway 网关运行一个智能体回合（或使用 `--local` 嵌入式运行）。

必需：

- `--message <text>`

选项：

- `--to <dest>`（用于会话键和可选发送）
- `--session-id <id>`

- --thinking <off|minimal|low|medium|high|xhigh> (仅限 GPT-5.2 + Codex 模型)
- --verbose <on|full|off>
- --channel <whatsapp|telegram|discord|slack|mattermost|signal|imessage|msteams>
- --local
- --deliver
- --json
- --timeout <seconds>

`agents`

管理隔离的智能体（工作区 + 认证 + 路由）。

agents list

列出已配置的智能体。

选项：

- --json
- --bindings

agents add [name]

添加新的隔离智能体。除非传入标志（或 --non-interactive），否则运行引导向导；非交互模式下 --workspace 是必需的。

选项：

- --workspace <dir>
- --model <id>
- --agent-dir <dir>
- --bind <channel[:accountId]> (可重复)
- --non-interactive
- --json

绑定规范使用 channel[:accountId]。对于 WhatsApp，省略 accountId 时使用默认账户 id。

agents delete <id>

删除智能体并清理其工作区 + 状态。

选项：

- --force
- --json

``acp``

运行连接 IDE 到 Gateway 网关的 ACP 桥接。

完整选项和示例参见 `acp`。

``status``

显示关联会话健康状况和最近的收件人。

选项：

- --json
- --all（完整诊断；只读，可粘贴）
- --deep（探测渠道）
- --usage（显示模型提供商用量/配额）
- --timeout <ms>
- --verbose
- --debug（--verbose 的别名）

说明：

- 概览包含 Gateway 网关 + 节点主机服务状态（如可用）。

用量跟踪

当 OAuth/API 凭证可用时，OpenClaw 可以显示提供商用量/配额。

显示位置：

- /status（可用时添加简短的提供商用量行）
- `openclaw status --usage`（打印完整的提供商明细）
- macOS 菜单栏（上下文下的用量部分）

说明：

- 数据直接来自提供商用量端点（非估算）。
- 提供商：Anthropic、GitHub Copilot、OpenAI Codex OAuth，以及启用这些提供商插件时的 Gemini CLI/Antigravity。
- 如果没有匹配的凭证，用量会被隐藏。
- 详情：参见用量跟踪。

`health`

从运行中的 Gateway 网关获取健康状态。

选项：

- --json
- --timeout <ms>
- --verbose

`sessions`

列出存储的对话会话。

选项：

- --json
- --verbose
- --store <path>
- --active <minutes>

重置/卸载

`reset`

重置本地配置/状态（保留 CLI 安装）。

选项：

- --scope <config|config+creds+sessions|full>
- --yes
- --non-interactive

- --dry-run

说明：

- --non-interactive 需要 --scope 和 --yes。

`uninstall`

卸载 Gateway 网关服务 + 本地数据（CLI 保留）。

选项：

- --service
- --state
- --workspace
- --app
- --all
- --yes
- --non-interactive
- --dry-run

说明：

- --non-interactive 需要 --yes 和明确的范围（或 --all）。

Gateway 网关

`gateway`

运行 WebSocket Gateway 网关。

选项：

- --port <port>
- --bind <loopback|tailnet|lan|auto|custom>
- --token <token>
- --auth <token|password>
- --password <password>
- --tailscale <off|serve|funnel>

- --tailscale-reset-on-exit
- --allow-unconfigured
- --dev
- --reset (重置 dev 配置 + 凭证 + 会话 + 工作区)
- --force (终止端口上的现有监听器)
- --verbose
- --claude-cli-logs
- --ws-log <auto|full|compact>
- --compact (--ws-log compact 的别名)
- --raw-stream
- --raw-stream-path <path>

`gateway service`

管理 Gateway 网关服务 (launchd/systemd/schtasks) 。

子命令：

- gateway status (默认探测 Gateway 网关 RPC)
- gateway install (服务安装)
- gateway uninstall
- gateway start
- gateway stop
- gateway restart

说明：

- gateway status 默认使用服务解析的端口/配置探测 Gateway 网关 RPC (使用 --url/--token/--password 覆盖) 。
- gateway status 支持 --no-probe、--deep 和 --json 用于脚本化。
- gateway status 在检测到旧版或额外的 Gateway 网关服务时也会显示 (--deep 添加系统级扫描)。配置文件命名的 OpenClaw 服务被视为一等公民，不会被标记为"额外"。
- gateway status 打印 CLI

使用的配置路径与服务可能使用的配置 (服务环境)，以及解析的探测目标 URL。

- gateway install|uninstall|start|stop|restart 支持 --json 用于脚本化（默认输出保持人类友好）。
- gateway install 默认使用 Node 运行时；不建议使用 bun（WhatsApp/Telegram bug）。
- gateway install 选项：--port、--runtime、--token、--force、--json。

`logs`

通过 RPC 跟踪 Gateway 网关文件日志。

说明：

- TTY 会话渲染彩色、结构化视图；非 TTY 回退到纯文本。
- --json 输出行分隔的 JSON（每行一个日志事件）。

示例：

代码：openclaw logs --follow

代码：openclaw logs --limit 200

代码：openclaw logs --plain

代码：openclaw logs --json

代码：openclaw logs --no-color

`gateway <subcommand>`

Gateway 网关 CLI 辅助工具（RPC 子命令使用 --url、--token、--password、--timeout、--expect-final）。

子命令：

- gateway call <method> [--params <json>]
- gateway health
- gateway status
- gateway probe
- gateway discover
- gateway install|uninstall|start|stop|restart
- gateway run

常见 RPC：

- config.apply (验证 + 写入配置 + 重启 + 唤醒)
- config.patch (合并部分更新 + 重启 + 唤醒)
- update.run (运行更新 + 重启 + 唤醒)

提示：直接调用 config.set/config.apply/config.patch 时，如果配置已存在，请传入来自 config.get 的 baseHash。

模型

回退行为和扫描策略参见 /concepts/models。

首选 Anthropic 认证 (setup-token)：

代码：claude setup-token

代码：openclaw models auth setup-token --provider anthropic

代码：openclaw models status

`models` (根命令)

openclaw models 是 models status 的别名。

根选项：

- --status-json (models status --json 的别名)
- --status-plain (models status --plain 的别名)

`models list`

选项：

- --all
- --local
- --provider <name>
- --json
- --plain

`models status`

选项：

- --json
- --plain
- --check (退出码 1=过期/缺失, 2=即将过期)
- --probe (对已配置认证配置文件进行实时探测)
- --probe-provider <name>
- --probe-profile <id> (重复或逗号分隔)
- --probe-timeout <ms>
- --probe-concurrency <n>
- --probe-max-tokens <n>

始终包含认证概览和认证存储中配置文件的 OAuth 过期状态。--probe 运行实时请求 (可能消耗令牌并触发速率限制)。

``models set <model>``

设置 `agents.defaults.model.primary`。

``models set-image <model>``

设置 `agents.defaults.imageModel.primary`。

``models aliases list|add|remove``

选项：

- list : --json 、 --plain
- add <alias> <model>
- remove <alias>

``models fallbacks list|add|remove|clear``

选项：

- list : --json 、 --plain
- add <model>
- remove <model>
- clear

`models image-fallbacks list|add|remove|clear`

选项：

- list : --json 、 --plain
- add <model>
- remove <model>
- clear

`models scan`

选项：

- --min-params
- --max-age-days <days>
- --provider <name>
- --max-candidates <n>
- --timeout <ms>
- --concurrency <n>
- --no-probe
- --yes
- --no-input
- --set-default
- --set-image
- --json

`models auth add|setup-token|paste-token`

选项：

- add : 交互式认证辅助工具
- setup-token : --provider <name> (默认 anthropic) 、 --yes
- paste-token : --provider <name> 、 --profile-id <id> 、 --expires-in <duration>

`models auth order get|set|clear`

选项：

- get : --provider <name> 、 --agent <id> 、 --json
- set : --provider <name> 、 --agent <id> 、 <profileids...>
- clear : --provider <name> 、 --agent <id>

系统

`system event`

将系统事件加入队列并可选触发心跳（Gateway 网关 RPC）。

必需：

- --text <text>

选项：

- --mode <now|next-heartbeat>
- --json
- --url 、 --token 、 --timeout 、 --expect-final

`system heartbeat last|enable|disable`

心跳控制（Gateway 网关 RPC）。

选项：

- --json
- --url 、 --token 、 --timeout 、 --expect-final

`system presence`

列出系统存在条目（Gateway 网关 RPC）。

选项：

- --json
- --url 、 --token 、 --timeout 、 --expect-final

定时任务

管理计划任务（Gateway 网关 RPC）。参见 [/automation/cron-jobs](#)。

子命令：

- cron status [--json]
- cron list [--all] [--json]（默认表格输出；使用 --json 获取原始数据）
- cron add（别名：create；需要 --name 和 --at | --every | --cron 三选一，以及 --system-event | --message 负载二选一）
- cron edit <id>（补丁字段）
- cron rm <id>（别名：remove、delete）
- cron enable <id>
- cron disable <id>
- cron runs --id <id> [--limit <n>]
- cron run <id> [--force]

所有 cron 命令接受 --url、--token、--timeout、--expect-final。

节点主机

node 运行无头节点主机或将其作为后台服务管理。参见 `openclaw node`。

子命令：

- node run --host <gateway-host> --port 18789
- node status
- node install [--host <gateway-host>] [--port <port>] [--tls] [--tls-fingerprint <sha256>] [--node-id <id>] [--display-name <name>] [--runtime <node|bun>] [--force]
- node uninstall
- node stop
- node restart

节点

nodes 与 Gateway 网关通信并针对已配对的节点。参见 `/nodes`。

通用选项：

- --url、--token、--timeout、--json

子命令：

- nodes status [--connected] [--last-connected <duration>]
- nodes describe --node <id|name|ip>
- nodes list [--connected] [--last-connected <duration>]
- nodes pending
- nodes approve <requestId>
- nodes reject <requestId>
- nodes rename --node <id|name|ip> --name <displayName>
- nodes invoke --node <id|name|ip> --command <command> [--params <json>] [--invoke-timeout <ms>] [--idempotency-key <key>]
- nodes run --node <id|name|ip> [--cwd <path>] [--env KEY=VAL] [--command-timeout <ms>] [--needs-screen-recording] [--invoke-timeout <ms>] <command...> (mac 节点或无头节点主机)
- nodes notify --node <id|name|ip> [--title <text>] [--body <text>] [--sound <name>] [--priority <passive|active|timeSensitive>] [--delivery <system|overlay|auto>] [--invoke-timeout <ms>] (仅限 mac)

相机：

- nodes camera list --node <id|name|ip>
- nodes camera snap --node <id|name|ip> [--facing front|back|both] [--device-id <id>] [--max-width <px>] [--quality <0-1>] [--delay-ms <ms>] [--invoke-timeout <ms>]
- nodes camera clip --node <id|name|ip> [--facing front|back] [--device-id <id>] [--duration <ms|10s|1m>] [--no-audio] [--invoke-timeout <ms>]

画布 + 屏幕：

- nodes canvas snapshot --node <id|name|ip> [--format png|jpg|jpeg] [--max-width <px>] [--quality <0-1>] [--invoke-timeout <ms>]
- nodes canvas present --node <id|name|ip> [--target <urlOrPath>] [--x <px>] [--y <px>] [--width <px>] [--height <px>] [--invoke-timeout <ms>]
- nodes canvas hide --node <id|name|ip> [--invoke-timeout <ms>]
- nodes canvas navigate <url> --node <id|name|ip> [--invoke-timeout <ms>]
- nodes canvas eval [<js>] --node <id|name|ip> [--js <code>] [--invoke-timeout <ms>]
- nodes canvas a2ui push --node <id|name|ip> (--jsonl <path> | --text <text>) [--invoke-timeout <ms>]

- nodes canvas a2ui reset --node <id|name|ip> [--invoke-timeout <ms>]
- nodes screen record --node <id|name|ip> [--screen <index>] [--duration <ms|10s>] [--fps <n>] [--no-audio] [--out <path>] [--invoke-timeout <ms>]

位置：

- nodes location get --node <id|name|ip> [--max-age <ms>] [--accuracy <coarse|balanced|precise>] [--location-timeout <ms>] [--invoke-timeout <ms>]

浏览器

浏览器控制 CLI（专用 Chrome/Brave/Edge/Chromium）。参见 `openclaw browser` 和浏览器工具。

通用选项：

- --url、--token、--timeout、--json
- --browser-profile <name>

管理：

- browser status
- browser start
- browser stop
- browser reset-profile
- browser tabs
- browser open <url>
- browser focus <targetId>
- browser close [targetId]
- browser profiles
- browser create-profile --name <name> [--color <hex>] [--cdp-url <url>]
- browser delete-profile --name <name>

检查：

- browser screenshot [targetId] [--full-page] [--ref <ref>] [--element <selector>] [--type png|jpeg]

- browser snapshot [--format aria|ai] [--target-id <id>] [--limit <n>] [--interactive] [--compact] [--depth <n>] [--selector <sel>] [--out <path>]

操作：

- browser navigate <url> [--target-id <id>]
- browser resize <width> <height> [--target-id <id>]
- browser click <ref> [--double] [--button <left|right|middle>] [--modifiers <csv>] [--target-id <id>]
- browser type <ref> <text> [--submit] [--slowly] [--target-id <id>]
- browser press <key> [--target-id <id>]
- browser hover <ref> [--target-id <id>]
- browser drag <startRef> <endRef> [--target-id <id>]
- browser select <ref> <values...> [--target-id <id>]
- browser upload <paths...> [--ref <ref>] [--input-ref <ref>] [--element <selector>] [--target-id <id>] [--timeout-ms <ms>]
- browser fill [--fields <json>] [--fields-file <path>] [--target-id <id>]
- browser dialog --accept|--dismiss [--prompt <text>] [--target-id <id>] [--timeout-ms <ms>]
- browser wait [--time <ms>] [--text <value>] [--text-gone <value>] [--target-id <id>]
- browser evaluate --fn <code> [--ref <ref>] [--target-id <id>]
- browser console [--level <error|warn|info>] [--target-id <id>]
- browser pdf [--target-id <id>]

文档搜索

`docs [query...]

搜索在线文档索引。

TUI

`tui`

打开连接到 Gateway 网关的终端 UI。

选项：

- --url <url>
- --token <token>
- --password <password>
- --session <key>
- --deliver
- --thinking <level>
- --message <text>
- --timeout-ms <ms> (默认为 agents.defaults.timeoutSeconds)
- --history-limit <n>

20. `openclaw logs`

`openclaw logs`

通过 RPC 跟踪 Gateway 网关文件日志（在远程模式下可用）。

相关内容：

- 日志概述：日志

示例

代码：openclaw logs

代码：openclaw logs --follow

代码：openclaw logs --json

代码：openclaw logs --limit 500

21. `openclaw memory`

`openclaw memory`

管理语义记忆的索引和搜索。

由活跃的记忆插件提供（默认：memory-core；设置 plugins.slots.memory = "none" 可禁用）。

相关内容：

- 记忆概念：记忆

- 插件：插件

示例

代码：openclaw memory status

代码：openclaw memory status --deep

代码：openclaw memory status --deep --index

代码：openclaw memory status --deep --index --verbose

代码：openclaw memory index

代码：openclaw memory index --verbose

代码：openclaw memory search "release checklist"

代码：openclaw memory status --agent main

代码：openclaw memory index --agent main --verbose

选项

通用选项：

- --agent <id>：限定到单个智能体（默认：所有已配置的智能体）。
- --verbose：在探测和索引期间输出详细日志。

说明：

- memory status --deep 探测向量存储和嵌入模型的可用性。
- memory status --deep --index 在存储有未同步变更时运行重新索引。
- memory index --verbose 打印每个阶段的详细信息（提供商、模型、数据源、批处理活动）。
- memory status 包含通过 memorySearch.extraPaths 配置的所有额外路径。

22. `openclaw message`

`openclaw message`

用于发送消息和渠道操作的单一出站命令

(Discord/Google

Chat/Slack/Mattermost（插件）/Telegram/WhatsApp/Signal/iMessage/MS Teams）。

用法

代码：openclaw message <subcommand> [flags]

渠道选择：

- 如果配置了多个渠道，则必须指定 --channel。
- 如果只配置了一个渠道，则该渠道为默认值。
- 可选值：whatsapp|telegram|discord|googlechat|slack|mattermost|signal|imessage|msteams (Mattermost 需要插件)

目标格式 (--target)：

- WhatsApp：E.164 或群组 JID
- Telegram：聊天 ID 或 @username
- Discord：channel:<id> 或 user:<id>（或 <@id> 提及；纯数字 ID 被视为频道）
- Google Chat：spaces/<spaceId> 或 users/<userId>
- Slack：channel:<id> 或 user:<id>（接受纯频道 ID）
- Mattermost（插件）：channel:<id>、user:<id> 或 @username（纯 ID 被视为频道）
- Signal：+E.164、group:<id>、signal:+E.164、signal:group:<id> 或 username:<name>/u:<name>
- iMessage：句柄、chat_id:<id>、chat_guid:<guid> 或 chat_identifier:<id>
- MS Teams：会话 ID（19:...@thread.tacv2）或 conversation:<id> 或 user:<aad-object-id>

名称查找：

- 对于支持的提供商（Discord/Slack 等），如 Help 或 #help 之类的频道名称会通过目录缓存进行解析。
- 缓存未命中时，如果提供商支持，OpenClaw 将尝试实时目录查找。

通用标志

- --channel <name>
- --account <id>
- --target <dest>（用于 send/poll/read 等的目标渠道或用户）
- --targets <name>（可重复；仅限广播）
- --json
- --dry-run

- --verbose

操作

核心

- send
 - 渠道：WhatsApp/Telegram/Discord/Google Chat/Slack/Mattermost（插件）/Signal/iMessage/MS Teams
 - 必需：--target，以及 --message 或 --media
 - 可选：--media、--reply-to、--thread-id、--gif-playback
 - 仅限 Telegram：--buttons（需要 channels.telegram.capabilities.inlineButtons 以启用）
 - 仅限 Telegram：--thread-id（论坛主题 ID）
 - 仅限 Slack：--thread-id（线程时间戳；--reply-to 使用相同字段）
 - 仅限 WhatsApp：--gif-playback
- poll
 - 渠道：WhatsApp/Discord/MS Teams
 - 必需：--target、--poll-question、--poll-option（可重复）
 - 可选：--poll-multi
 - 仅限 Discord：--poll-duration-hours、--message
- react
 - 渠道：Discord/Google Chat/Slack/Telegram/WhatsApp/Signal
 - 必需：--message-id、--target
 - 可选：--emoji、--remove、--participant、--from-me、--target-author、--target-author-uid
 - 注意：--remove 需要 --emoji（省略 --emoji 可清除自己的表情回应（如果支持）；参见 /tools/reactions）
 - 仅限 WhatsApp：--participant、--from-me
 - Signal 群组表情回应：需要 --target-author 或 --target-author-uid
- reactions
 - 渠道：Discord/Google Chat/Slack
 - 必需：--message-id、--target

- 可选：--limit
- read
- 渠道：Discord/Slack
- 必需：--target
- 可选：--limit、--before、--after
- 仅限 Discord：--around
- edit
- 渠道：Discord/Slack
- 必需：--message-id、--message、--target
- delete
- 渠道：Discord/Slack/Telegram
- 必需：--message-id、--target
- pin / unpin
- 渠道：Discord/Slack
- 必需：--message-id、--target
- pins（列表）
- 渠道：Discord/Slack
- 必需：--target
- permissions
- 渠道：Discord
- 必需：--target
- search
- 渠道：Discord
- 必需：--guild-id、--query
-

可选：--channel-id、--channel-ids（可重复）、--author-id、--author-ids（可重复）、--limit

线程

- thread create

- 渠道：Discord
 - 必需：--thread-name 、--target (频道 ID)
 - 可选：--message-id 、--auto-archive-min
- thread list
 - 渠道：Discord
 - 必需：--guild-id
 - 可选：--channel-id 、--include-archived 、--before 、--limit
- thread reply
 - 渠道：Discord
 - 必需：--target (线程 ID) 、--message
 - 可选：--media 、--reply-to

表情符号

- emoji list
 - Discord：--guild-id
 - Slack：无需额外标志
- emoji upload
 - 渠道：Discord
 - 必需：--guild-id 、--emoji-name 、--media
 - 可选：--role-ids (可重复)

贴纸

- sticker send
 - 渠道：Discord
 - 必需：--target 、--sticker-id (可重复)
 - 可选：--message
- sticker upload
 - 渠道：Discord
 - 必需：--guild-id 、--sticker-name 、--sticker-desc 、--sticker-tags 、--media

角色 / 频道 / 成员 / 语音

- role info (Discord) : --guild-id
- role add / role remove (Discord) : --guild-id 、 --user-id 、 --role-id
- channel info (Discord) : --target
- channel list (Discord) : --guild-id
- member info (Discord/Slack) : --user-id (Discord 还需要 --guild-id)
- voice status (Discord) : --guild-id 、 --user-id

事件

- event list (Discord) : --guild-id
- event create (Discord) : --guild-id 、 --event-name 、 --start-time
- 可选 : --end-time 、 --desc 、 --channel-id 、 --location 、 --event-type

管理 (Discord)

- timeout : --guild-id 、 --user-id (可选 --duration-min 或 --until ; 两者都省略则清除超时)
- kick : --guild-id 、 --user-id (+ --reason)
- ban : --guild-id 、 --user-id (+ --delete-days 、 --reason)
- timeout 也支持 --reason

广播

- broadcast
- 渠道 : 任何已配置的渠道 ; 使用 --channel all 可针对所有提供商
- 必需 : --targets (可重复)
- 可选 : --message 、 --media 、 --dry-run

示例

发送 Discord 回复 :

代码 : openclaw message send --channel discord \

代码 : --target channel:123 --message "hi" --reply-to 456

创建 Discord 投票 :

代码：openclaw message poll --channel discord \

代码：--target channel:123 \

代码：--poll-question "Snack?" \

代码：--poll-option Pizza --poll-option Sushi \

代码：--poll-multi --poll-duration-hours 48

发送 Teams 主动消息：

代码：openclaw message send --channel msteams \

代码：--target conversation:19:abc@thread.tacv2 --message "hi"

创建 Teams 投票：

代码：openclaw message poll --channel msteams \

代码：--target conversation:19:abc@thread.tacv2 \

代码：--poll-question "Lunch?" \

代码：--poll-option Pizza --poll-option Sushi

在 Slack 中添加表情回应：

代码：openclaw message react --channel slack \

代码：--target C123 --message-id 456 --emoji ""

在 Signal 群组中添加表情回应：

代码：openclaw message react --channel signal \

代码：--target signal:group:abc123 --message-id 1737630212345 \

代码：--emoji "" --target-author-uuid 123e4567-e89b-12d3-a456-426614174000

发送 Telegram 内联按钮：

代码：openclaw message send --channel telegram --target @mychat --message "Choose:" \

代码：--buttons '[[{"text":"Yes","callback_data":"cmd:yes"}], [{"text":"No","callback_data":"cmd:no"}]]'

23. `openclaw models`

`openclaw models`

模型发现、扫描和配置（默认模型、回退、认证配置）。

相关内容：

- 提供商 + 模型：模型
- 提供商认证设置：快速开始

常用命令

代码：openclaw models status

代码：openclaw models list

代码：openclaw models set <model-or-alias>

代码：openclaw models scan

openclaw models status 显示已解析的默认模型/回退配置以及认证概览。

当提供商使用快照可用时，OAuth/令牌状态部分会包含提供商使用头信息。

添加 --probe 可对每个已配置的提供商配置运行实时认证探测。

探测会发送真实请求（可能消耗令牌并触发速率限制）。

使用 --agent <id> 可检查已配置智能体的模型/认证状态。省略时，

命令会使用 OPENCLAW_AGENT_DIR/PI_CODING_AGENT_DIR（如已设置），否则使用已配置的默认智能体。

注意事项：

- models set <model-or-alias> 接受 provider/model 或别名。
- 模型引用通过在第一个 / 处拆分来解析。如果模型 ID 包含 /（OpenRouter 风格），需包含提供商前缀（示例：openrouter/moonshotai/kimi-k2）。
- 如果省略提供商，OpenClaw 会将输入视为别名或默认提供商的模型（仅在模型 ID 不包含 / 时有效）。

`models status`

选项：

- --json
- --plain
- --check（退出码 1=已过期/缺失，2=即将过期）

- `--probe` (对已配置的认证配置进行实时探测)
- `--probe-provider <name>` (探测单个提供商)
- `--probe-profile <id>` (可重复或逗号分隔的配置 ID)
- `--probe-timeout <ms>`
- `--probe-concurrency <n>`
- `--probe-max-tokens <n>`
- `--agent <id>` (已配置的智能体 ID ; 覆盖 `OPENCLAW_AGENT_DIR/PI_CODING_AGENT_DIR`)

别名 + 回退

代码 : `openclaw models aliases list`

代码 : `openclaw models fallbacks list`

认证配置

代码 : `openclaw models auth add`

代码 : `openclaw models auth login --provider <id>`

代码 : `openclaw models auth setup-token`

代码 : `openclaw models auth paste-token`

`models auth login` 运行提供商插件的认证流程 (OAuth/API 密钥) 。使用 `openclaw plugins list` 查看已安装的提供商。

注意事项 :

- `setup-token` 会提示输入 `setup-token` 值 (在任意机器上使用 `claude setup-token` 生成) 。
- `paste-token` 接受在其他地方或通过自动化生成的令牌字符串。

24. `openclaw node`

``openclaw node``

运行一个无头节点主机，连接到 Gateway 网关 WebSocket 并在此机器上暴露 `system.run / system.which` 。

为什么使用节点主机？

当你希望智能体在网络中的其他机器上运行命令，而无需在那里安装完整的 macOS 配套应用时，请使用节点主机。

常见用例：

- 在远程 Linux/Windows 机器上运行命令（构建服务器、实验室机器、NAS）。
- 在 Gateway 网关上保持执行的沙箱隔离，但将批准的运行委托给其他主机。
- 为自动化或 CI 节点提供轻量级、无头的执行目标。

执行仍然受执行批准和节点主机上的每智能体允许列表保护，因此你可以保持命令访问的范围明确。

浏览器代理（零配置）

如果节点上的 `browser.enabled` 未被禁用，节点主机自动广播浏览器代理。这让智能体无需额外配置即可在该节点上使用浏览器自动化。

如需在节点上禁用：

代码：{

代码：nodeHost: {

代码：browserProxy: {

代码：enabled: false,

代码：},

代码：},

代码：}

运行（前台）

代码：openclaw node run --host <gateway-host> --port 18789

选项：

- `--host <host>`：Gateway 网关 WebSocket 主机（默认：127.0.0.1）
- `--port <port>`：Gateway 网关 WebSocket 端口（默认：18789）
- `--tls`：为 Gateway 网关连接使用 TLS
- `--tls-fingerprint <sha256>`：预期的 TLS 证书指纹（sha256）
- `--node-id <id>`：覆盖节点 id（清除配对 token）

- `--display-name <name>` : 覆盖节点显示名称

服务（后台）

将无头节点主机安装为用户服务。

代码：`openclaw node install --host <gateway-host> --port 18789`

选项：

- `--host <host>` : Gateway 网关 WebSocket 主机（默认：127.0.0.1）
- `--port <port>` : Gateway 网关 WebSocket 端口（默认：18789）
- `--tls` : 为 Gateway 网关连接使用 TLS
- `--tls-fingerprint <sha256>` : 预期的 TLS 证书指纹（sha256）
- `--node-id <id>` : 覆盖节点 id（清除配对 token）
- `--display-name <name>` : 覆盖节点显示名称
- `--runtime <runtime>` : 服务运行时（node 或 bun）
- `--force` : 如果已安装则重新安装/覆盖

管理服务：

代码：`openclaw node status`

代码：`openclaw node stop`

代码：`openclaw node restart`

代码：`openclaw node uninstall`

使用 `openclaw node run` 运行前台节点主机（无服务）。

服务命令接受 `--json` 以获取机器可读输出。

配对

首次连接会在 Gateway 网关上创建待处理的节点配对请求。

通过以下方式批准：

代码：`openclaw nodes pending`

代码：`openclaw nodes approve <requestId>`

节点主机将其节点 id、token、显示名称和 Gateway 网关连接信息存储在

~/openclaw/node.json 中。

执行批准

system.run 受本地执行批准限制：

- ~/openclaw/exec-approvals.json
- 执行批准
- openclaw approvals --node <id|name|ip> (从 Gateway 网关编辑)

25. `openclaw nodes`

`openclaw nodes`

管理已配对的节点（设备）并调用节点功能。

相关内容：

- 节点概述：节点
- 摄像头：摄像头节点
- 图像：图像节点

通用选项：

- --url 、--token 、--timeout 、--json

常用命令

代码：openclaw nodes list

代码：openclaw nodes list --connected

代码：openclaw nodes list --last-connected 24h

代码：openclaw nodes pending

代码：openclaw nodes approve <requestId>

代码：openclaw nodes status

代码：openclaw nodes status --connected

代码：openclaw nodes status --last-connected 24h

nodes list 打印待处理/已配对表格。已配对行包含最近连接时长（Last Connect）。

使用 --connected 仅显示当前已连接的节点。使用 --last-connected <duration>

筛选在指定时间段内连接过的节点（例如 24h、7d）。

调用 / 运行

代码：openclaw nodes invoke --node <id|name|ip> --command <command> --params <json>

代码：openclaw nodes run --node <id|name|ip> <command...>

代码：openclaw nodes run --raw "git status"

代码：openclaw nodes run --agent main --node <id|name|ip> --raw "git status"

调用标志：

- --params <json>：JSON 对象字符串（默认 {}）。
- --invoke-timeout <ms>：节点调用超时（默认 15000）。
- --idempotency-key <key>：可选的幂等键。

Exec 风格默认值

nodes run 与模型的 exec 行为一致（默认值 + 审批）：

- 读取 tools.exec.（以及 agents.list[].tools.exec. 覆盖）。
- 在调用 system.run 前使用 exec 审批（exec.approval.request）。
- 当设置了 tools.exec.node 时可省略 --node。
- 需要支持 system.run 的节点（macOS 配套应用或无头节点主机）。

标志：

- --cwd <path>：工作目录。
- --env <key=val>：环境变量覆盖（可重复）。
- --command-timeout <ms>：命令超时。
- --invoke-timeout <ms>：节点调用超时（默认 30000）。
- --needs-screen-recording：要求屏幕录制权限。
- --raw <command>：运行 shell 字符串（/bin/sh -lc 或 cmd.exe /c）。
- --agent <id>：智能体范围的审批/白名单（默认为已配置的智能体）。
- --ask <off|on-miss|always>、--security <deny|allowlist|full>：覆盖选项。

26. `openclaw onboard`

`openclaw onboard`

交互式新手引导向导（本地或远程 Gateway 网关设置）。

相关内容：

- 向导指南：新手引导

示例

代码：openclaw onboard

代码：openclaw onboard --flow quickstart

代码：openclaw onboard --flow manual

代码：openclaw onboard --mode remote --remote-url ws://gateway-host:18789

流程说明：

- quickstart：最少提示，自动生成 Gateway 网关令牌。
- manual：完整的端口/绑定/认证提示（advanced 的别名）。
- 最快开始聊天：openclaw dashboard（控制 UI，无需渠道设置）。

27. `openclaw pairing`

`openclaw pairing`

批准或检查私信配对请求（适用于支持配对的渠道）。

相关内容：

- 配对流程：配对

命令

代码：openclaw pairing list whatsapp

代码：openclaw pairing approve whatsapp <code> --notify

28. `openclaw plugins`

`openclaw plugins`

管理 Gateway 网关插件/扩展（进程内加载）。

相关内容：

- 插件系统：插件
- 插件清单 + 模式：插件清单
- 安全加固：安全

命令

代码：openclaw plugins list

代码：openclaw plugins info <id>

代码：openclaw plugins enable <id>

代码：openclaw plugins disable <id>

代码：openclaw plugins doctor

代码：openclaw plugins update <id>

代码：openclaw plugins update --all

内置插件随 OpenClaw 一起发布，但默认禁用。使用 plugins enable 来激活它们。

所有插件必须提供 openclaw.plugin.json 文件，其中包含内联 JSON Schema（configSchema，即使为空）。缺少或无效的清单或模式会阻止插件加载并导致配置验证失败。

安装

代码：openclaw plugins install <path-or-spec>

安全提示：将插件安装视为运行代码。优先使用固定版本。

支持的归档格式：.zip、.tgz、.tar.gz、.tar。

使用 --link 避免复制本地目录（添加到 plugins.load.paths）：

代码：openclaw plugins install -l ./my-plugin

更新

代码：openclaw plugins update <id>

代码：openclaw plugins update --all

代码：openclaw plugins update <id> --dry-run

更新仅适用于从 npm 安装的插件（在 plugins.installs 中跟踪）。

29. `openclaw reset`

`openclaw reset`

重置本地配置/状态（保留 CLI 安装）。

代码：openclaw reset

代码：openclaw reset --dry-run

代码：openclaw reset --scope config+creds+sessions --yes --non-interactive

30. 沙箱 CLI

沙箱 CLI

管理基于 Docker 的沙箱容器，用于隔离智能体执行。

概述

OpenClaw 可以在隔离的 Docker 容器中运行智能体以确保安全。sandbox 命令帮助你管理这些容器，特别是在更新或配置更改后。

命令

`openclaw sandbox explain`

检查生效的沙箱模式/作用域/工作区访问权限、沙箱工具策略和提权门控（附带修复配置的键路径）。

代码：openclaw sandbox explain

代码：openclaw sandbox explain --session agent:main:main

代码：openclaw sandbox explain --agent work

代码：openclaw sandbox explain --json

`openclaw sandbox list`

列出所有沙箱容器及其状态和配置。

代码：openclaw sandbox list

代码：openclaw sandbox list --browser # List only browser containers

代码：openclaw sandbox list --json # JSON output

输出包括：

- 容器名称和状态（运行中/已停止）
- Docker 镜像及其是否与配置匹配
- 创建时间
- 空闲时间（自上次使用以来的时间）
- 关联的会话/智能体

`openclaw sandbox recreate`

移除沙箱容器以强制使用更新的镜像/配置重新创建。

代码：openclaw sandbox recreate --all # Recreate all containers

代码：openclaw sandbox recreate --session main # Specific session

代码：openclaw sandbox recreate --agent mybot # Specific agent

代码：openclaw sandbox recreate --browser # Only browser containers

代码：openclaw sandbox recreate --all --force # Skip confirmation

选项：

- --all：重新创建所有沙箱容器
- --session <key>：重新创建特定会话的容器
- --agent <id>：重新创建特定智能体的容器
- --browser：仅重新创建浏览器容器
- --force：跳过确认提示

重要：容器会在智能体下次使用时自动重新创建。

使用场景

更新 Docker 镜像后

代码：# Pull new image

代码：docker pull openclaw-sandbox:latest

代码：docker tag openclaw-sandbox:latest openclaw-sandbox:bookworm-slim

代码：# Update config to use new image

代码：# Edit config: agents.defaults.sandbox.docker.image (or agents.list[].sandbox.docker.image)

代码：# Recreate containers

代码：openclaw sandbox recreate --all

更改沙箱配置后

代码：# Edit config: agents.defaults.sandbox.* (or agents.list[].sandbox.*)

代码：# Recreate to apply new config

代码：openclaw sandbox recreate --all

更改 setupCommand 后

代码：openclaw sandbox recreate --all

代码：# or just one agent:

代码：openclaw sandbox recreate --agent family

仅针对特定智能体

代码：# Update only one agent's containers

代码：openclaw sandbox recreate --agent alfred

为什么需要这个？

问题：当你更新沙箱 Docker 镜像或配置时：

- 现有容器继续使用旧设置运行
- 容器仅在空闲 24 小时后才被清理
- 经常使用的智能体会无限期保持旧容器运行

解决方案：使用 openclaw sandbox recreate

强制移除旧容器。它们会在下次需要时自动使用当前设置重新创建。

提示：优先使用 openclaw sandbox recreate 而不是手动 docker rm。它使用 Gateway 网关的容器命名规则，避免在作用域/会话键更改时出现不匹配。

配置

沙箱设置位于 ~/.openclaw/openclaw.json 的 agents.defaults.sandbox 下（每个智能体的覆盖设置在 agents.list[].sandbox 中）：

```
代码：{
代码： "agents": {
代码： "defaults": {
代码： "sandbox": {
代码： "mode": "all", // off, non-main, all
代码： "scope": "agent", // session, agent, shared
代码： "docker": {
代码： "image": "openclaw-sandbox:bookworm-slim",
代码： "containerPrefix": "openclaw-sbx-",
代码： // ... more Docker options
代码： },
代码： "prune": {
代码： "idleHours": 24, // Auto-prune after 24h idle
代码： "maxAgeDays": 7, // Auto-prune after 7 days
代码： },
代码： },
代码： },
代码： },
代码： }
```

另请参阅

- 沙箱文档
- 智能体配置
- Doctor 命令 - 检查沙箱设置

31. `openclaw security`

`openclaw security`

安全工具（审计 + 可选修复）。

相关：

- 安全指南：安全

审计

代码：openclaw security audit

代码：openclaw security audit --deep

代码：openclaw security audit --fix

当多个私信发送者共享主会话时，审计会发出警告，并建议对共享收件箱使用 `session.dmScope="per-channel-peer"`（或多账户渠道使用 `per-account-channel-peer`）。
当使用小模型（ $\leq 300B$ ）且未启用沙箱隔离但启用了 `web/browser` 工具时，它也会发出警告。

32. `openclaw sessions`

`openclaw sessions`

列出已存储的对话会话。

代码：openclaw sessions

代码：openclaw sessions --active 120

代码：openclaw sessions --json

33. `openclaw setup`

`openclaw setup`

初始化 `~/.openclaw/openclaw.json` 和智能体工作区。

相关内容：

- 快速开始：快速开始
- 向导：新手引导

示例

代码：openclaw setup

代码：openclaw setup --workspace ~/.openclaw/workspace

通过 `setup` 运行向导：

代码：openclaw setup --wizard

34. `openclaw skills`

`openclaw skills`

检查 Skills（内置 + 工作区 + 托管覆盖）并查看哪些符合条件，哪些缺少要求。

相关内容：

- Skills 系统：Skills
- Skills 配置：Skills 配置
- ClawHub 安装：ClawHub

命令

代码：openclaw skills list

代码：openclaw skills list --eligible

代码：openclaw skills info <name>

代码：openclaw skills check

35. `openclaw status`

`openclaw status`

渠道 + 会话的诊断。

代码：openclaw status

代码：openclaw status --all

代码：openclaw status --deep

代码：openclaw status --usage

注意事项：

- --deep 运行实时探测（WhatsApp Web + Telegram + Discord + Google Chat + Slack + Signal）。
- 当配置了多个智能体时，输出包含每个智能体的会话存储。
- 概览包含 Gateway 网关 + 节点主机服务安装/运行时状态（如果可用）。
- 概览包含更新渠道 + git SHA（用于源代码检出）。

- 更新信息显示在概览中；如果有可用更新，status 会打印提示运行 openclaw update（参见更新）。

36. `openclaw system`

`openclaw system`

Gateway 网关的系统级辅助工具：入队系统事件、控制心跳和查看在线状态。

常用命令

代码：openclaw system event --text "Check for urgent follow-ups" --mode now

代码：openclaw system heartbeat enable

代码：openclaw system heartbeat last

代码：openclaw system presence

`system event`

在主会话上入队系统事件。下一次心跳会将其作为 System: 行注入到提示中。使用 --mode now 立即触发心跳；next-heartbeat 等待下一个计划的心跳时刻。

标志：

- --text <text>：必填的系统事件文本。
- --mode <mode>：now 或 next-heartbeat（默认）。
- --json：机器可读输出。

`system heartbeat last|enable|disable`

心跳控制：

- last：显示最后一次心跳事件。
- enable：重新开启心跳（如果之前被禁用，使用此命令）。
- disable：暂停心跳。

标志：

- --json：机器可读输出。

`system presence`

列出 Gateway 网关已知的当前系统在线状态条目（节点、实例和类似状态行）。

标志：

- `--json`：机器可读输出。

注意

- 需要一个运行中的 Gateway 网关，可通过你当前的配置访问（本地或远程）。
- 系统事件是临时的，不会在重启后持久化。

37. ``openclaw tui``

``openclaw tui``

打开连接到 Gateway 网关的终端 UI。

相关：

- TUI 指南：TUI

示例

代码：`openclaw tui`

代码：`openclaw tui --url ws://127.0.0.1:18789 --token <token>`

代码：`openclaw tui --session main --deliver`

38. ``openclaw uninstall``

``openclaw uninstall``

卸载 Gateway 网关服务 + 本地数据（CLI 保留）。

代码：`openclaw uninstall`

代码：`openclaw uninstall --all --yes`

代码：`openclaw uninstall --dry-run`

39. ``openclaw update``

``openclaw update``

安全更新 OpenClaw 并在 `stable/beta/dev` 渠道之间切换。

如果你通过 npm/pnpm 安装（全局安装，无 git 元数据），更新通过 更新 中的包管理器流程进行。

用法

代码：openclaw update

代码：openclaw update status

代码：openclaw update wizard

代码：openclaw update --channel beta

代码：openclaw update --channel dev

代码：openclaw update --tag beta

代码：openclaw update --no-restart

代码：openclaw update --json

代码：openclaw --update

选项

- --no-restart：成功更新后跳过重启 Gateway 网关服务。
- --channel <stable|beta|dev>：设置更新渠道（git + npm；持久化到配置中）。
- --tag <dist-tag|version>：仅为本次更新覆盖 npm dist-tag 或版本。
- --json：打印机器可读的 UpdateRunResult JSON。
- --timeout <seconds>：每步超时时间（默认 1200 秒）。

注意：降级需要确认，因为旧版本可能会破坏配置。

`update status`

显示当前更新渠道 + git 标签/分支/SHA（对于源码检出），以及更新可用性。

代码：openclaw update status

代码：openclaw update status --json

代码：openclaw update status --timeout 10

选项：

- --json：打印机器可读的状态 JSON。
- --timeout <seconds>：检查超时时间（默认 3 秒）。

`update wizard`

交互式流程，用于选择更新渠道并确认是否在更新后重启 Gateway 网关（默认重启）。如果你选择 dev 但没有 git 检出，它会提供创建一个的选项。

工作原理

当你显式切换渠道（--channel ...）时，OpenClaw 也会保持安装方式一致：

- dev → 确保存在 git 检出（默认：~/openclaw，可通过 OPENCLAW_GIT_DIR 覆盖），更新它，并从该检出安装全局 CLI。
- stable/beta → 使用匹配的 dist-tag 从 npm 安装。

Git 检出流程

渠道：

- stable：检出最新的非 beta 标签，然后构建 + doctor。
- beta：检出最新的 -beta 标签，然后构建 + doctor。
- dev：检出 main，然后 fetch + rebase。

高层概述：

- 需要干净的工作树（无未提交的更改）。
- 切换到所选渠道（标签或分支）。
- 获取上游（仅 dev）。
- 仅 dev：在临时工作树中预检 lint + TypeScript 构建；如果最新提交失败，回退最多 10 个提交以找到最新的干净构建。
- Rebase 到所选提交（仅 dev）。
- 安装依赖（优先使用 pnpm；npm 作为备选）。
- 构建 + 构建控制界面。
- 运行 openclaw doctor 作为最终的"安全更新"检查。
- 将插件同步到当前渠道（dev 使用捆绑的扩展；stable/beta 使用 npm）并更新 npm 安装的插件。

`--update` 简写

openclaw --update 会重写为 openclaw update（便于 shell 和启动脚本使用）。

另请参阅

- [openclaw doctor](#)（在 git 检出上会提供先运行更新的选项）
- [开发渠道](#)
- [更新](#)
- [CLI 参考](#)

40. `openclaw voicecall`

`openclaw voicecall`

voicecall 是一个由插件提供的命令。只有在安装并启用了语音通话插件时才会出现。

主要文档：

- [语音通话插件：语音通话](#)

常用命令

代码：openclaw voicecall status --call-id <id>

代码：openclaw voicecall call --to "+15555550123" --message "Hello" --mode notify

代码：openclaw voicecall continue --call-id <id> --message "Any questions?"

代码：openclaw voicecall end --call-id <id>

暴露 Webhook (Tailscale)

代码：openclaw voicecall expose --mode serve

代码：openclaw voicecall expose --mode funnel

代码：openclaw voicecall unexpose

安全提示：仅将 webhook 端点暴露给你信任的网络。尽可能优先使用 Tailscale Serve 而非 Funnel。

41. `openclaw webhooks`

`openclaw webhooks`

Webhook 辅助工具和集成（Gmail Pub/Sub、Webhook 辅助工具）。

相关内容：

- Webhook : Webhook
- Gmail Pub/Sub : Gmail Pub/Sub

Gmail

代码 : `openclaw webhooks gmail setup --account you@example.com`

代码 : `openclaw webhooks gmail run`

详情请参阅 Gmail Pub/Sub 文档。

第七章：自动化与定时任务

1. 认证监控

认证监控

OpenClaw 通过 `openclaw models status` 提供 OAuth

过期健康状态。请使用该命令进行自动化和告警；脚本是为手机工作流程提供的可选附加功能。

推荐方式：CLI 检查（可移植）

代码：`openclaw models status --check`

退出码：

- 0：正常
- 1：凭证过期或缺失
- 2：即将过期（24 小时内）

此方式适用于 `cron/systemd`，无需额外脚本。

可选脚本（运维 / 手机工作流程）

这些脚本位于 `scripts/` 目录下，属于可选内容。它们假定你可以通过 SSH 访问 Gateway 网关主机，并针对 `systemd + Termux` 进行了调优。

- `scripts/claude-auth-status.sh` 现在使用 `openclaw models status --json` 作为数据来源（如果 CLI 不可用则回退到直接读取文件），因此请确保 `openclaw` 在定时器的 PATH 中。
- `scripts/auth-monitor.sh`：`cron/systemd` 定时器目标；发送告警（ntfy 或手机）。
- `scripts/systemd/openclaw-auth-monitor.{service,timer}`：systemd 用户定时器。
- `scripts/claude-auth-status.sh`：Claude Code + OpenClaw 认证检查器（完整/json/简洁模式）。
- `scripts/mobile-reauth.sh`：通过 SSH 引导的重新认证流程。
- `scripts/termux-quick-auth.sh`：一键小部件状态查看 + 打开认证 URL。
- `scripts/termux-auth-widget.sh`：完整的引导式小部件流程。
- `scripts/termux-sync-widget.sh`：同步 Claude Code 凭证 → OpenClaw。

如果你不需要手机自动化或 `systemd` 定时器，可以跳过这些脚本。

2. 定时任务（Gateway网关调度器）

定时任务（Gateway网关调度器）

定时任务还是心跳？请参阅定时任务与心跳对比了解何时使用哪种方式。

定时任务是 Gateway网关内置的调度器。它持久化任务、在合适的时间唤醒智能体，并可选择将输出发送回聊天。

如果你想要 _"每天早上运行"_ 或 _"20 分钟后提醒智能体"_，定时任务就是对应的机制。

简要概述

- 定时任务运行在 Gateway网关内部（而非模型内部）。
- 任务持久化存储在 ~/.openclaw/cron/ 下，因此重启不会丢失计划。
- 两种执行方式：
 - 主会话：入队一个系统事件，然后在下一次心跳时运行。
 - 隔离式：在 cron:<jobId> 中运行专用智能体轮次，可投递摘要（默认 announce）或不投递。
- 唤醒是一等功能：任务可以请求"立即唤醒"或"下次心跳时"。

快速开始（可操作）

创建一个一次性提醒，验证其存在，然后立即运行：

代码：openclaw cron add \

代码：--name "Reminder" \

代码：--at "2026-02-01T16:00:00Z" \

代码：--session main \

代码：--system-event "Reminder: check the cron docs draft" \

代码：--wake now \

代码：--delete-after-run

代码：openclaw cron list

代码：openclaw cron run <job-id> --force

代码：openclaw cron runs --id <job-id>

调度一个带投递功能的周期性隔离任务：

代码：openclaw cron add \

代码：--name "Morning brief" \

代码：--cron "0 7 * * *" \

代码：--tz "America/Los_Angeles" \

代码：--session isolated \

代码：--message "Summarize overnight updates." \

代码：--announce \

代码：--channel slack \

代码：--to "channel:C1234567890"

工具调用等价形式（Gateway网关定时任务工具）

有关规范的 JSON 结构和示例，请参阅工具调用的 JSON 模式。

定时任务的存储位置

定时任务默认持久化存储在 Gateway网关主机的 ~/.openclaw/cron/jobs.json 中。Gateway网关将文件加载到内存中，并在更改时写回，因此仅在 Gateway网关停止时手动编辑才是安全的。请优先使用 openclaw cron add/edit 或定时任务工具调用 API 进行更改。

新手友好概述

将定时任务理解为：何时运行 + 做什么。

- 选择调度计划
- 一次性提醒 → schedule.kind = "at" (CLI : --at)
- 重复任务 → schedule.kind = "every" 或 schedule.kind = "cron"
- 如果你的 ISO 时间戳省略了时区，将被视为 UTC。
- 选择运行位置
- sessionTarget: "main" → 在下一次心跳时使用主会话上下文运行。
- sessionTarget: "isolated" → 在 cron:<jobId> 中运行专用智能体轮次。
- 选择负载
- 主会话 → payload.kind = "systemEvent"
- 隔离会话 → payload.kind = "agentTurn"

可选：一次性任务 (schedule.kind = "at") 默认会在成功运行后删除。设置

`deleteAfterRun: false` 可保留它（成功后会禁用）。

概念

任务

定时任务是一条存储记录，包含：

- 一个调度计划（何时运行），
- 一个负载（做什么），
- 可选的投递（输出发送到哪里）。
- 可选的智能体绑定（`agentId`）：在指定智能体下运行任务；如果缺失或未知，Gateway网关会回退到默认智能体。

任务通过稳定的 `jobId` 标识（用于 CLI/Gateway 网关 API）。

在智能体工具调用中，`jobId` 是规范字段；旧版 `id` 仍可兼容使用。

一次性任务默认会在成功运行后自动删除；设置 `deleteAfterRun: false` 可保留它。

调度计划

定时任务支持三种调度类型：

- `at`：一次性时间戳（ISO 8601 字符串）。
- `every`：固定间隔（毫秒）。
- `cron`：5 字段 cron 表达式，可选 IANA 时区。

Cron 表达式使用 `croner`。如果省略时区，将使用 Gateway 网关主机的本地时区。

主会话与隔离式执行

主会话任务（系统事件）

主会话任务入队一个系统事件，并可选择唤醒心跳运行器。它们必须使用 `payload.kind = "systemEvent"`。

- `wakeMode: "next-heartbeat"`（默认）：事件等待下一次计划心跳。
- `wakeMode: "now"`：事件触发立即心跳运行。

当你需要正常的心跳提示 + 主会话上下文时，这是最佳选择。参见心跳。

隔离任务（专用定时会话）

隔离任务在会话 `cron:<jobId>` 中运行专用智能体轮次。

关键行为：

- 提示以 `[cron:<jobId> <任务名称>]` 为前缀，便于追踪。
- 每次运行都会启动一个全新的会话 ID（不继承之前的对话）。
- 如果未指定 `delivery`，隔离任务会默认以 “announce” 方式投递摘要。
- `delivery.mode` 可选 `announce`（投递摘要）或 `none`（内部运行）。

对于嘈杂、频繁或“后台杂务”类任务，使用隔离任务可以避免污染你的主聊天记录。

负载结构（运行内容）

支持两种负载类型：

- `systemEvent`：仅限主会话，通过心跳提示路由。
- `agentTurn`：仅限隔离会话，运行专用智能体轮次。

常用 `agentTurn` 字段：

- `message`：必填文本提示。
- `model / thinking`：可选覆盖（见下文）。
- `timeoutSeconds`：可选超时覆盖。

模型和思维覆盖

隔离任务（`agentTurn`）可以覆盖模型和思维级别：

- `model`：提供商/模型字符串（例如 `anthropic/claude-sonnet-4-20250514`）或别名（例如 `opus`）
- `thinking`：思维级别（`off`、`minimal`、`low`、`medium`、`high`、`xhigh`；仅限 GPT-5.2 + Codex 模型）

注意：你也可以在主会话任务上设置 `model`，但这会更改共享的主会话模型。我们建议仅对隔离任务使用模型覆盖，以避免意外的上下文切换。

优先级解析顺序：

- 任务负载覆盖（最高优先级）

- 钩子特定默认值（例如 `hooks.gmail.model`）
- 智能体配置默认值

投递（渠道 + 目标）

隔离任务可以通过顶层 delivery 配置投递输出：

- `delivery.mode` : `announce`（投递摘要）或 `none`
- `delivery.channel` : `whatsapp` / `telegram` / `discord` / `slack` / `mattermost`（插件） / `signal` / `imessage` / `last`
- `delivery.to` : 渠道特定的接收目标
- `delivery.bestEffort` : 投递失败时避免任务失败

当启用 announce 投递时，该轮次会抑制消息工具发送；请使用 `delivery.channel/delivery.to` 来指定目标。

如果省略 `delivery.channel` 或 `delivery.to`，定时任务会回退到主会话的“最后路由”（智能体最后回复的位置）。

目标格式提醒：

- Slack/Discord/Mattermost（插件）目标应使用明确前缀（例如 channel:<id>、user:<id>）以避免歧义。
- Telegram 主题应使用 :topic: 格式（见下文）。

Telegram 投递目标（主题/论坛帖子）

Telegram 通过 message_thread_id 支持论坛主题。对于定时任务投递，你可以将主题/帖子编码到 to 字段中：

- -1001234567890（仅聊天 ID）
- -1001234567890:topic:123（推荐：明确的主题标记）
- -1001234567890:123（简写：数字后缀）

帶前綴的目標如 telegram:... / telegram:group:... 也可接受：

- telegram:group:-1001234567890:topic:123

工具调用的 JSON 模式

直接调用 Gateway 网关 cron. 工具（智能体工具调用或 RPC）时使用这些结构。CLI 标志接受人类可读的时间格式如 20m，但工具调用应使用 ISO 8601 字符串作为 schedule.at，并使用毫秒作为 schedule.everyMs。

cron.add 参数

一次性主会话任务（系统事件）：

代码：{

代码： "name": "Reminder",

代码： "schedule": { "kind": "at", "at": "2026-02-01T16:00:00Z" },

代码： "sessionTarget": "main",

代码： "wakeMode": "now",

代码： "payload": { "kind": "systemEvent", "text": "Reminder text" },

代码： "deleteAfterRun": true

代码： }

带投递的周期性隔离任务：

代码：{

代码： "name": "Morning brief",

代码： "schedule": { "kind": "cron", "expr": "0 7 * * *", "tz": "America/Los_Angeles" },

代码： "sessionTarget": "isolated",

代码： "wakeMode": "next-heartbeat",

代码： "payload": {

代码： "kind": "agentTurn",

代码： "message": "Summarize overnight updates."

代码： },

代码： "delivery": {

代码： "mode": "announce",

代码： "channel": "slack",

代码： "to": "channel:C1234567890",

代码： "bestEffort": true

代码： }

代码：}

说明：

- schedule.kind：at (at) 、every (everyMs) 或 cron (expr，可选 tz) 。
- schedule.at 接受 ISO 8601（可省略时区；省略时按 UTC 处理）。
- everyMs 为毫秒数。
- sessionTarget 必须为 "main" 或 "isolated"，且必须与 payload.kind 匹配。
- 可选字段：agentId、description、enabled、deleteAfterRun、delivery。
- wakeMode 省略时默认为 "next-heartbeat"。

cron.update 参数

代码：{

代码："jobId": "job-123",

代码："patch": {

代码："enabled": false,

代码："schedule": { "kind": "every", "everyMs": 3600000 }

代码：}

代码：}

说明：

- jobId 是规范字段；id 可兼容使用。
- 在补丁中使用 agentId: null 可清除智能体绑定。

cron.run 和 cron.remove 参数

代码：{ "jobId": "job-123", "mode": "force" }

代码：{ "jobId": "job-123" }

存储与历史

- 任务存储：~/.openclaw/cron/jobs.json（Gateway网关管理的 JSON）。
- 运行历史：~/.openclaw/cron/runs/<jobId>.jsonl（JSONL，自动清理）。
- 覆盖存储路径：配置中的 cron.store。

配置

代码：{

代码： cron: {

代码： enabled: true, // 默认 true

代码： store: "~/.openclaw/cron/jobs.json",

代码： maxConcurrentRuns: 1, // 默认 1

代码： },

代码： }

完全禁用定时任务：

- cron.enabled: false（配置）
- OPENCLAW_SKIP_CRON=1（环境变量）

CLI 快速开始

一次性提醒（UTC ISO，成功后自动删除）：

代码： openclaw cron add \

代码： --name "Send reminder" \

代码： --at "2026-01-12T18:00:00Z" \

代码： --session main \

代码： --system-event "Reminder: submit expense report." \

代码： --wake now \

代码： --delete-after-run

一次性提醒（主会话，立即唤醒）：

代码： openclaw cron add \

代码： --name "Calendar check" \

代码： --at "20m" \

代码： --session main \

代码： --system-event "Next heartbeat: check calendar." \

代码： --wake now

周期性隔离任务（投递到 WhatsApp）：

代码：openclaw cron add \

代码：--name "Morning status" \

代码：--cron "0 7 * * *" \

代码：--tz "America/Los_Angeles" \

代码：--session isolated \

代码：--message "Summarize inbox + calendar for today." \

代码：--announce \

代码：--channel whatsapp \

代码：--to "+15551234567"

周期性隔离任务（投递到 Telegram 主题）：

代码：openclaw cron add \

代码：--name "Nightly summary (topic)" \

代码：--cron "0 22 * * *" \

代码：--tz "America/Los_Angeles" \

代码：--session isolated \

代码：--message "Summarize today; send to the nightly topic." \

代码：--announce \

代码：--channel telegram \

代码：--to "-1001234567890:topic:123"

带模型和思维覆盖的隔离任务：

代码：openclaw cron add \

代码：--name "Deep analysis" \

代码：--cron "0 6 * * 1" \

代码：--tz "America/Los_Angeles" \

代码：--session isolated \

代码：--message "Weekly deep analysis of project progress." \

代码：--model "opus" \

代码：--thinking high \

代码：--announce \

代码：--channel whatsapp \

代码：--to "+15551234567"

智能体选择（多智能体配置）：

代码：# 将任务绑定到智能体 "ops"（如果该智能体不存在则回退到默认智能体）

代码：openclaw cron add --name "Ops sweep" --cron "0 6 * * *" --session isolated
--message "Check ops queue" --agent ops

代码：# 切换或清除现有任务的智能体

代码：openclaw cron edit <jobld> --agent ops

代码：openclaw cron edit <jobld> --clear-agent

手动运行（调试）：

代码：openclaw cron run <jobld> --force

编辑现有任务（补丁字段）：

代码：openclaw cron edit <jobld> \

代码：--message "Updated prompt" \

代码：--model "opus" \

代码：--thinking low

运行历史：

代码：openclaw cron runs --id <jobld> --limit 50

不创建任务直接发送系统事件：

代码：openclaw system event --mode now --text "Next heartbeat: check battery."

Gateway网关 API 接口

- cron.list 、 cron.status 、 cron.add 、 cron.update 、 cron.remove
- cron.run（强制或到期）、cron.runs

如需不创建任务直接发送系统事件，请使用 openclaw system event。

故障排除

"没有任何任务运行"

- 检查定时任务是否已启用：cron.enabled 和 OPENCLAW_SKIP_CRON。
- 检查 Gateway网关是否持续运行（定时任务运行在 Gateway网关进程内部）。
- 对于 cron 调度：确认时区（--tz）与主机时区的关系。

Telegram 投递到了错误的位置

- 对于论坛主题，使用 -100...:topic:<id> 以确保明确无歧义。
- 如果你在日志或存储的"最后路由"目标中看到 telegram:... 前缀，这是正常的；定时任务投递接受这些前缀并仍能正确解析主题 ID。

3. 定时任务与心跳：何时使用哪种方式

定时任务与心跳：何时使用哪种方式

心跳和定时任务都可以按计划运行任务。本指南帮助你根据使用场景选择合适的机制。

快速决策指南

使用场景	推荐方式	原因
-----	-----	-----
每 30 分钟检查收件箱	心跳	可与其他检查批量处理，具备上下文感知能力
每天上午 9 点准时发送报告	定时任务（隔离式）	需要精确定时
监控日历中即将到来的事件	心跳	天然适合周期性感知
运行每周深度分析	定时任务（隔离式）	独立任务，可使用不同模型
20 分钟后提醒我	定时任务（主会话，--at）	精确定时的一次性任务
后台项目健康检查	心跳	搭载在现有周期上

心跳：周期性感知

心跳在主会话中以固定间隔运行（默认：30 分钟）。它的设计目的是让智能体检查各种事项并呈现重要信息。

何时使用心跳

- 多个周期性检查：与其设置 5 个独立的定时任务分别检查收件箱、日历、天气、通知和项目状态，不如用一次心跳批量处理所有内容。

- 上下文感知决策：智能体拥有完整的主会话上下文，因此可以智能判断哪些紧急、哪些可以等待。
- 对话连续性：心跳运行共享同一会话，因此智能体记得最近的对话，可以自然地进行后续跟进。
- 低开销监控：一次心跳替代多个小型轮询任务。

心跳优势

- 批量处理多项检查：一次智能体轮次可以同时审查收件箱、日历和通知。
- 减少 API 调用：一次心跳比 5 个隔离式定时任务更经济。
- 上下文感知：智能体了解你一直在做什么，可以据此排定优先级。
- 智能抑制：如果没有需要关注的事项，智能体回复 HEARTBEAT_OK，不会投递任何消息。
- 自然定时：会根据队列负载略有漂移，但对大多数监控来说没有问题。

心跳示例：HEARTBEAT.md 检查清单

代码：# Heartbeat checklist

代码：- Check email for urgent messages

代码：- Review calendar for events in next 2 hours

代码：- If a background task finished, summarize results

代码：- If idle for 8+ hours, send a brief check-in

智能体在每次心跳时读取此清单，并在一次轮次中处理所有项目。

配置心跳

代码：{

代码：agents: {

代码：defaults: {

代码：heartbeat: {

代码：every: "30m", // 间隔

代码：target: "last", // 告警投递目标

代码：activeHours: { start: "08:00", end: "22:00" }, // 可选

代码：},

代码：},

代码：},

代码：}

完整配置请参阅心跳。

定时任务：精确调度

定时任务在精确时间运行，可以在隔离会话中运行而不影响主会话上下文。

何时使用定时任务

- 需要精确定时："每周一上午 9:00 发送"（而不是"大约 9 点左右"）。
- 独立任务：不需要对话上下文的任务。
- 不同的模型/思维级别：需要更强大模型的深度分析。
- 一次性提醒：使用 --at 实现"20 分钟后提醒我"。
- 嘈杂/频繁的任务：会把主会话历史搞得杂乱的任务。
- 外部触发器：无论智能体是否处于活跃状态都应独立运行的任务。

定时任务优势

- 精确定时：支持带时区的 5 字段 cron 表达式。
- 会话隔离：在 cron:<jobId> 中运行，不会污染主会话历史。
- 模型覆盖：可按任务使用更便宜或更强大的模型。
- 投递控制：隔离任务默认以 announce 投递摘要，可选 none 仅内部运行。
- 无需智能体上下文：即使主会话空闲或已压缩，也能运行。
- 一次性支持：--at 用于精确的未来时间戳。

定时任务示例：每日早间简报

代码：openclaw cron add \

代码：--name "Morning briefing" \

代码：--cron "0 7 * * *" \

代码：--tz "America/New_York" \

代码：--session isolated \

代码：--message "Generate today's briefing: weather, calendar, top emails, news summary." \

代码：--model opus \

代码：--announce \

代码：--channel whatsapp \

代码：--to "+15551234567"

这会在纽约时间每天早上 7:00 准时运行，使用 Opus 保证质量，并直接投递到 WhatsApp。

定时任务示例：一次性提醒

代码：openclaw cron add \

代码：--name "Meeting reminder" \

代码：--at "20m" \

代码：--session main \

代码：--system-event "Reminder: standup meeting starts in 10 minutes." \

代码：--wake now \

代码：--delete-after-run

完整 CLI 参考请参阅定时任务。

决策流程图

代码：任务是否需要在精确时间运行？

代码：是 -> 使用定时任务

代码：否 -> 继续...

代码：任务是否需要与主会话隔离？

代码：是 -> 使用定时任务（隔离式）

代码：否 -> 继续...

代码：此任务能否与其他周期性检查批量处理？

代码：是 -> 使用心跳（添加到 HEARTBEAT.md）

代码：否 -> 使用定时任务

代码：这是一次性提醒吗？

代码：是 -> 使用定时任务配合 --at

代码：否 -> 继续...

代码：是否需要不同的模型或思维级别？

代码：是 -> 使用定时任务（隔离式）配合 `--model/--thinking`

代码：否 -> 使用心跳

组合使用

最高效的配置是两者结合：

- 心跳处理常规监控（收件箱、日历、通知），每 30 分钟批量处理一次。
- 定时任务处理精确调度（每日报告、每周回顾）和一次性提醒。

示例：高效自动化配置

HEARTBEAT.md（每 30 分钟检查一次）：

代码：# Heartbeat checklist

代码：- Scan inbox for urgent emails

代码：- Check calendar for events in next 2h

代码：- Review any pending tasks

代码：- Light check-in if quiet for 8+ hours

定时任务（精确定时）：

代码：# 每天早上 7 点的早间简报

代码：`openclaw cron add --name "Morning brief" --cron "0 7 * * *" --session isolated --message "..."` --announce

代码：# 每周一上午 9 点的项目回顾

代码：`openclaw cron add --name "Weekly review" --cron "0 9 * * 1" --session isolated --message "..."` --model opus

代码：# 一次性提醒

代码：`openclaw cron add --name "Call back" --at "2h" --session main --system-event "Call back the client" --wake now`

Lobster：带审批的确定性 workflow

Lobster 是用于多步骤工具管道的工作流运行时，适用于需要确定性执行和明确审批的场景。当任务不只是单次智能体轮次，且你需要可恢复的带人工检查点的工作流时，使用它。

何时适合使用 Lobster

- 多步骤自动化：你需要一个固定的工具调用管道，而不是一次性提示。
- 审批关卡：副作用应暂停直到你批准，然后继续执行。
- 可恢复运行：继续暂停的工作流而无需重新运行之前的步骤。

如何与心跳和定时任务配合

- 心跳/定时任务决定何时运行。
- Lobster 定义运行开始后执行哪些步骤。

对于计划性工作流，使用定时任务或心跳触发一次调用 Lobster 的智能体轮次。对于临时工作流，直接调用 Lobster。

操作说明（来自代码）

- Lobster 以本地子进程（lobster CLI）在工具模式下运行，并返回 JSON 信封。
- 如果工具返回 `needs_approval`，你需要使用 `resumeToken` 和 `approve` 标志来恢复。
- 该工具是可选插件；建议通过 `tools.alsoAllow: ["lobster"]` 附加启用。
- 如果传入 `lobsterPath`，必须是绝对路径。

完整用法和示例请参阅 Lobster。

主会话与隔离会话

心跳和定时任务都可以与主会话交互，但方式不同：

心跳 定时任务（主会话） 定时任务（隔离式）	
----- ----- ----- -----	
会话 主会话 主会话（通过系统事件） cron:<jobId>	
历史 共享 共享 每次运行全新	
上下文 完整 完整 无（从零开始）	
模型 主会话模型 主会话模型 可覆盖	
输出 非 HEARTBEAT_OK 时投递 心跳提示 + 事件 announce 摘要（默认）	

何时使用主会话定时任务

当你需要以下场景时，使用 `--session main` 配合 `--system-event`：

- 提醒/事件出现在主会话上下文中
- 智能体在下一次心跳时带着完整上下文处理它
- 不需要单独的隔离运行

代码：openclaw cron add \

代码：--name "Check project" \

代码：--every "4h" \

代码：--session main \

代码：--system-event "Time for a project health check" \

代码：--wake now

何时使用隔离式定时任务

当你需要以下场景时，使用 --session isolated：

- 无先前上下文的全新环境
- 不同的模型或思维设置
- 输出可通过 announce 直接投递摘要（或用 none 仅内部运行）
- 不会把主会话搞得杂乱的历史记录

代码：openclaw cron add \

代码：--name "Deep analysis" \

代码：--cron "0 6 * * 0" \

代码：--session isolated \

代码：--message "Weekly codebase analysis..." \

代码：--model opus \

代码：--thinking high \

代码：--announce

成本考量

| 机制 | 成本特征 |

| ----- | ----- |

| 心跳 | 每 N 分钟一次轮次；随 HEARTBEAT.md 大小扩展 |

| 定时任务（主会话） | 将事件添加到下一次心跳（无隔离轮次） |

| 定时任务（隔离式） | 每个任务一次完整智能体轮次；可使用更便宜的模型 |

建议：

- 保持 HEARTBEAT.md 精简以减少 token 开销。
- 将类似的检查批量放入心跳，而不是创建多个定时任务。
- 如果只需要内部处理，在心跳上使用 target: "none"。
- 对常规任务使用隔离式定时任务配合更便宜的模型。

相关内容

- 心跳 - 完整的心跳配置
- 定时任务 - 完整的定时任务 CLI 和 API 参考
- 系统 - 系统事件 + 心跳控制

4. Gmail Pub/Sub -> OpenClaw

Gmail Pub/Sub -> OpenClaw

目标：Gmail watch -> Pub/Sub 推送 -> gog gmail watch serve -> OpenClaw webhook。

前置条件

- 已安装并登录 gcloud（安装指南）。
- 已安装 gog (gogcli) 并为 Gmail 账户授权 (gogcli.sh)。
- 已启用 OpenClaw hooks（参见 Webhooks）。
- 已登录 tailscale (tailscale.com)。支持的设置使用 Tailscale Funnel 作为公共 HTTPS 端点。

其他隧道服务也可以使用，但需要自行配置/不受支持，需要手动接入。

目前，我们支持的是 Tailscale。

示例 hook 配置（启用 Gmail 预设映射）：

代码：{

代码： hooks: {

代码： enabled: true,

代码： token: "OPENCLAW_HOOK_TOKEN",

代码： path: "/hooks",

代码： presets: ["gmail"],

代码： },

代码： }

要将 Gmail 摘要投递到聊天界面，请用设置了 deliver 以及可选的 channel/to 的映射覆盖预设：

代码： {

代码： hooks: {

代码： enabled: true,

代码： token: "OPENCLAW_HOOK_TOKEN",

代码： presets: ["gmail"],

代码： mappings: [

代码： {

代码： match: { path: "gmail" },

代码： action: "agent",

代码： wakeMode: "now",

代码： name: "Gmail",

代码： sessionKey: "hook:gmail:{{messages[0].id}}",

代码： messageTemplate: "New email from {{messages[0].from}}\nSubject: {{messages[0].subject}}\n{{messages[0].snippet}}\n{{messages[0].body}}",

代码： model: "openai/gpt-5.2-mini",

代码： deliver: true,

代码： channel: "last",

代码： // to: "+15551234567"

代码： },

代码：],

代码： },

代码： }

如果你想使用固定渠道，请设置 channel + to。否则 channel: "last"

会使用上次的投递路由（默认回退到 WhatsApp）。

要为 Gmail 运行强制使用更便宜的模型，请在映射中设置 model（provider/model 或别名）。如果你强制启用了 agents.defaults.models，请将其包含在内。

要专门为 Gmail hooks 设置默认模型和思考级别，请在配置中添加 `hooks.gmail.model / hooks.gmail.thinking`：

代码：{

代码： hooks: {

代码： gmail: {

代码： model: "openrouter/meta-llama/llama-3.3-70b-instruct:free",

代码： thinking: "off",

代码： },

代码： },

代码： }

注意事项：

- 映射中的每个 hook 的 `model/thinking` 仍会覆盖这些默认值。
- 回退顺序：`hooks.gmail.model` → `agents.defaults.model.fallbacks` → 主模型（认证/速率限制/超时）。
- 如果设置了 `agents.defaults.models`，Gmail 模型必须在允许列表中。
- Gmail hook 内容默认使用外部内容安全边界包装。

要禁用（危险），请设置 `hooks.gmail.allowUnsafeExternalContent: true`。

要进一步自定义负载处理，请添加 `hooks.mappings` 或在 `hooks.transformsDir` 下添加 JS/TS 转换模块（参见 Webhooks）。

向导（推荐）

使用 OpenClaw 助手将所有内容接入在一起（在 macOS 上通过 brew 安装依赖）：

代码： `openclaw webhooks gmail setup \`

代码： `--account openclaw@gmail.com`

默认设置：

- 使用 Tailscale Funnel 作为公共推送端点。
- 为 `openclaw webhooks gmail run` 写入 `hooks.gmail` 配置。
- 启用 Gmail hook 预设（`hooks.presets: ["gmail"]`）。

路径说明：当启用 `tailscale.mode` 时，OpenClaw 会自动将 `hooks.gmail.serve.path` 设置为 `/`，并将公共路径保持在 `hooks.gmail.tailscale.path`（默认 `/gmail-pubsub`），因为 Tailscale 在代理之前会剥离设置的路径前缀。

如果你需要后端接收带前缀的路径，请将 `hooks.gmail.tailscale.target`（或 `--tailscale-target`）设置为完整 URL，如 `hooks.gmail.serve.path``。

想要自定义端点？使用 `--push-endpoint <url>` 或 `--tailscale off`。

平台说明：在 macOS 上，向导通过 Homebrew 安装 `gcloud`、`gogcli` 和 `tailscale`；在 Linux 上请先手动安装它们。

Gateway 网关自动启动（推荐）：

- 当 `hooks.enabled=true` 且设置了 `hooks.gmail.account` 时，Gateway 网关会在启动时运行 `gog gmail watch serve` 并自动续期 `watch`。
- 设置 `OPENCLAW_SKIP_GMAIL_WATCHER=1` 可退出（如果你自己运行守护进程则很有用）。
- 不要同时运行手动守护进程，否则会遇到 `listen tcp 127.0.0.1:8788: bind: address already in use`。

手动守护进程（启动 `gog gmail watch serve` + 自动续期）：

代码：`openclaw webhooks gmail run`

一次性设置

- 选择拥有 `gog` 使用的 OAuth 客户端的 GCP 项目。

代码：`gcloud auth login`

代码：`gcloud config set project <project-id>`

注意：Gmail watch 要求 Pub/Sub 主题与 OAuth 客户端位于同一项目中。

- 启用 API：

代码：`gcloud services enable gmail.googleapis.com pubsub.googleapis.com`

- 创建主题：

代码：`gcloud pubsub topics create gog-gmail-watch`

- 允许 Gmail push 发布：

代码：gcloud pubsub topics add-iam-policy-binding gog-gmail-watch \

代码：--member=serviceAccount:gmail-api-push@system.gserviceaccount.com \

代码：--role=roles/pubsub.publisher

启动 watch

代码：gog gmail watch start \

代码：--account openclaw@gmail.com \

代码：--label INBOX \

代码：--topic projects/<project-id>/topics/gog-gmail-watch

保存输出中的 history_id（用于调试）。

运行推送处理程序

本地示例（共享 token 认证）：

代码：gog gmail watch serve \

代码：--account openclaw@gmail.com \

代码：--bind 127.0.0.1 \

代码：--port 8788 \

代码：--path /gmail-pubsub \

代码：--token <shared> \

代码：--hook-url http://127.0.0.1:18789/hooks/gmail \

代码：--hook-token OPENCLAW_HOOK_TOKEN \

代码：--include-body \

代码：--max-bytes 20000

注意事项：

- --token 保护推送端点（x-gog-token 或 ?token=）。
- --hook-url 指向 OpenClaw /hooks/gmail（已映射；隔离运行 + 摘要发送到主线程）。
- --include-body 和 --max-bytes 控制发送到 OpenClaw 的正文片段。

推荐：openclaw webhooks gmail run 封装了相同的流程并自动续期 watch。

暴露处理程序（高级，不受支持）

如果你需要非 Tailscale 隧道，请手动接入并在推送订阅中使用公共 URL（不受支持，无保护措施）：

代码：cloudflared tunnel --url http://127.0.0.1:8788 --no-autoupdate

使用生成的 URL 作为推送端点：

代码：gcloud pubsub subscriptions create gog-gmail-watch-push \

代码：--topic gog-gmail-watch \

代码：--push-endpoint "https://<public-url>/gmail-pubsub?token=<shared>"

生产环境：使用稳定的 HTTPS 端点并配置 Pub/Sub OIDC JWT，然后运行：

代码：gog gmail watch serve --verify-oidc --oidc-email <svc@...>

测试

向被监视的收件箱发送一条消息：

代码：gog gmail send \

代码：--account openclaw@gmail.com \

代码：--to openclaw@gmail.com \

代码：--subject "watch test" \

代码：--body "ping"

检查 watch 状态和历史记录：

代码：gog gmail watch status --account openclaw@gmail.com

代码：gog gmail history --account openclaw@gmail.com --since <historyId>

故障排除

- Invalid topicName：项目不匹配（主题不在 OAuth 客户端项目中）。
- User not authorized：主题缺少 roles/pubsub.publisher。
- 空消息：Gmail push 仅提供 historyId；通过 gog gmail history 获取。

清理

代码：gog gmail watch stop --account openclaw@gmail.com

代码：gcloud pubsub subscriptions delete gog-gmail-watch-push

代码：gcloud pubsub topics delete gog-gmail-watch

5. Hooks

Hooks

Hooks 提供了一个可扩展的事件驱动系统，用于响应智能体命令和事件自动执行操作。Hooks 从目录中自动发现，可以通过 CLI 命令管理，类似于 OpenClaw 中 Skills 的工作方式。

入门指南

Hooks 是在事件发生时运行的小脚本。有两种类型：

- Hooks（本页）：当智能体事件触发时在 Gateway 网关内运行，如 /new、/reset、/stop 或生命周期事件。
- Webhooks：外部 HTTP webhooks，让其他系统触发 OpenClaw 中的工作。参见 Webhook Hooks 或使用 openclaw webhooks 获取 Gmail 助手命令。

Hooks 也可以捆绑在插件中；参见 插件。

常见用途：

- 重置会话时保存记忆快照
- 保留命令审计跟踪用于故障排除或合规
- 会话开始或结束时触发后续自动化
- 事件触发时向智能体工作区写入文件或调用外部 API

如果你能写一个小的 TypeScript 函数，你就能写一个 hook。Hooks 会自动发现，你可以通过 CLI 启用或禁用它们。

概述

hooks 系统允许你：

- 在发出 /new 时将会话上下文保存到记忆
- 记录所有命令以供审计
- 在智能体生命周期事件上触发自定义自动化
- 在不修改核心代码的情况下扩展 OpenClaw 的行为

入门

捆绑的 Hooks

OpenClaw 附带三个自动发现的捆绑 hooks：

- session-memory：当你发出 /new 时将会话上下文保存到智能体工作区（默认 ~/.openclaw/workspace/memory/）
- command-logger：将所有命令事件记录到 ~/.openclaw/logs/commands.log
- boot-md：当 Gateway 网关启动时运行 BOOT.md（需要启用内部 hooks）

列出可用的 hooks：

代码：openclaw hooks list

启用一个 hook：

代码：openclaw hooks enable session-memory

检查 hook 状态：

代码：openclaw hooks check

获取详细信息：

代码：openclaw hooks info session-memory

新手引导

在新手引导期间（openclaw onboard），你将被提示启用推荐的 hooks。向导会自动发现符合条件的 hooks 并呈现供选择。

Hook 发现

Hooks 从三个目录自动发现（按优先级顺序）：

- 工作区 hooks：<workspace>/hooks/（每智能体，最高优先级）
- 托管 hooks：~/.openclaw/hooks/（用户安装，跨工作区共享）
- 捆绑 hooks：<openclaw>/dist/hooks/bundled/（随 OpenClaw 附带）

托管 hook 目录可以是单个 hook 或 hook 包（包目录）。

每个 hook 是一个包含以下内容的目录：

代码：my-hook/

代码：|—— HOOK.md # 元数据 + 文档

代码：|—— handler.ts # 处理程序实现

Hook 包 (npm/archives)

Hook 包是标准的 npm 包，通过 package.json 中的 openclaw.hooks 导出一个或多个 hooks。使用以下命令安装：

代码：openclaw hooks install <path-or-spec>

示例 package.json：

代码：{

代码："name": "@acme/my-hooks",

代码："version": "0.1.0",

代码："openclaw": {

代码："hooks": ["/hooks/my-hook", "/hooks/other-hook"]

代码：}

代码：}

每个条目指向包含 HOOK.md 和 handler.ts（或 index.ts）的 hook 目录。

Hook 包可以附带依赖；它们将安装在 ~/.openclaw/hooks/<id> 下。

Hook 结构

HOOK.md 格式

HOOK.md 文件在 YAML frontmatter 中包含元数据，加上 Markdown 文档：

代码：---

代码：name: my-hook

代码：description: "Short description of what this hook does"

代码：homepage: https://docs.openclaw.ai/automation/hooks#my-hook

代码：metadata:

代码：{ "openclaw": { "emoji": "", "events": ["command:new"], "requires": { "bins": ["node"] } } }

代码：---

代码：# My Hook

代码：Detailed documentation goes here...

代码：## What It Does

代码：- Listens for `/new` commands

代码：- Performs some action

代码：- Logs the result

代码：## Requirements

代码：- Node.js must be installed

代码：## Configuration

代码：No configuration needed.

元数据字段

metadata.openclaw 对象支持：

- emoji：CLI 的显示表情符号（例如 ""）
- events：要监听的事件数组（例如 ["command:new", "command:reset"]）
- export：要使用的命名导出（默认为 "default"）
- homepage：文档 URL
- requires：可选要求
- bins：PATH 中需要的二进制文件（例如 ["git", "node"]）
- anyBins：这些二进制文件中至少有一个必须存在
- env：需要的环境变量
- config：需要的配置路径（例如 ["workspace.dir"]）
- os：需要的平台（例如 ["darwin", "linux"]）
- always：绕过资格检查（布尔值）
- install：安装方法（对于捆绑 hooks：[{ "id": "bundled", "kind": "bundled" }]

处理程序实现

handler.ts 文件导出一个 HookHandler 函数：

代码：const myHandler: HookHandler = async (event) => {

代码：// Only trigger on 'new' command

代码：if (event.type !== "command" || event.action !== "new") {

代码：return;

代码：}

代码：console.log(`[my-hook] New command triggered`);

代码：console.log(` Session: \${event.sessionKey}`);

代码：console.log(` Timestamp: \${event.timestamp.toISOString()}`);

代码：// Your custom logic here

代码：// Optionally send message to user

代码：event.messages.push(" My hook executed!");

代码：};

事件上下文

每个事件包含：

代码：{

代码：type: 'command' | 'session' | 'agent' | 'gateway',

代码：action: string, // e.g., 'new', 'reset', 'stop'

代码：sessionKey: string, // Session identifier

代码：timestamp: Date, // When the event occurred

代码：messages: string[], // Push messages here to send to user

代码：context: {

代码：sessionEntry?: SessionEntry,

代码：sessionId?: string,

代码：sessionFile?: string,

代码：commandSource?: string, // e.g., 'whatsapp', 'telegram'

代码：senderId?: string,

代码：workspaceDir?: string,

代码：bootstrapFiles?: WorkspaceBootstrapFile[],

代码：cfg?: OpenClawConfig

代码：}

代码：}

事件类型

命令事件

当发出智能体命令时触发：

- command：所有命令事件（通用监听器）
- command:new：当发出 /new 命令时
- command:reset：当发出 /reset 命令时
- command:stop：当发出 /stop 命令时

智能体事件

- agent:bootstrap：在注入工作区引导文件之前（hooks 可以修改 context.bootstrapFiles）

Gateway 网关事件

当 Gateway 网关启动时触发：

- gateway:startup：在渠道启动和 hooks 加载之后

工具结果 Hooks（插件 API）

这些 hooks 不是事件流监听器；它们让插件在 OpenClaw 持久化工具结果之前同步调整它们。

- tool_result_persist：在工具结果写入会话记录之前转换它们。必须是同步的；返回更新后的工具结果负载或 undefined 保持原样。参见 智能体循环。

未来事件

计划中的事件类型：

- session:start：当新会话开始时
- session:end：当会话结束时
- agent:error：当智能体遇到错误时

- message:sent : 当消息被发送时
- message:received : 当消息被接收时

创建自定义 Hooks

1. 选择位置

- 工作区 hooks (<workspace>/hooks/) : 每智能体，最高优先级
- 托管 hooks (~/.openclaw/hooks/) : 跨工作区共享

2. 创建目录结构

代码 : `mkdir -p ~/.openclaw/hooks/my-hook`

代码 : `cd ~/.openclaw/hooks/my-hook`

3. 创建 HOOK.md

代码 : ---

代码 : name: my-hook

代码 : description: "Does something useful"

代码 : metadata: { "openclaw": { "emoji": "", "events": ["command:new"] } }

代码 : ---

代码 : # My Custom Hook

代码 : This hook does something useful when you issue `/new`.

4. 创建 handler.ts

代码 : `const handler: HookHandler = async (event) => {`

代码 : `if (event.type !== "command" || event.action !== "new") {`

代码 : `return;`

代码 : `}`

代码 : `console.log("[my-hook] Running!");`

代码 : `// Your logic here`

代码 : `};`

5. 启用并测试

代码：# Verify hook is discovered

代码：openclaw hooks list

代码：# Enable it

代码：openclaw hooks enable my-hook

代码：# Restart your gateway process (menu bar app restart on macOS, or restart your dev process)

代码：# Trigger the event

代码：# Send /new via your messaging channel

配置

新配置格式（推荐）

代码：{

代码： "hooks": {

代码： "internal": {

代码： "enabled": true,

代码： "entries": {

代码： "session-memory": { "enabled": true },

代码： "command-logger": { "enabled": false }

代码： }

代码： }

代码： }

代码： }

每 Hook 配置

Hooks 可以有自定义配置：

代码：{

代码： "hooks": {

代码： "internal": {

代码： "enabled": true,

代码： "entries": {

代码： "my-hook": {

代码： "enabled": true,

代码： "env": {

代码： "MY_CUSTOM_VAR": "value"

代码： }

代码： }

代码： }

代码： }

代码： }

代码： }

额外目录

从额外目录加载 hooks：

代码： {

代码： "hooks": {

代码： "internal": {

代码： "enabled": true,

代码： "load": {

代码： "extraDirs": ["/path/to/more/hooks"]

代码： }

代码： }

代码： }

代码： }

遗留配置格式（仍然支持）

旧配置格式仍然有效以保持向后兼容：

代码： {

代码： "hooks": {

```
代码： "internal": {  
代码： "enabled": true,  
代码： "handlers": [  
代码： {  
代码： "event": "command:new",  
代码： "module": "./hooks/handlers/my-handler.ts",  
代码： "export": "default"  
代码： }  
代码： ]  
代码： }  
代码： }  
代码： }
```

迁移：对新 hooks 使用基于发现的新系统。遗留处理程序在基于目录的 hooks 之后加载。

CLI 命令

列出 Hooks

```
代码： # List all hooks  
代码： openclaw hooks list  
  
代码： # Show only eligible hooks  
代码： openclaw hooks list --eligible  
  
代码： # Verbose output (show missing requirements)  
代码： openclaw hooks list --verbose  
  
代码： # JSON output  
代码： openclaw hooks list --json
```

Hook 信息

```
代码： # Show detailed info about a hook  
代码： openclaw hooks info session-memory  
  
代码： # JSON output
```

代码：openclaw hooks info session-memory --json

检查资格

代码：# Show eligibility summary

代码：openclaw hooks check

代码：# JSON output

代码：openclaw hooks check --json

启用/禁用

代码：# Enable a hook

代码：openclaw hooks enable session-memory

代码：# Disable a hook

代码：openclaw hooks disable command-logger

捆绑的 Hooks

session-memory

当你发出 /new 时将会话上下文保存到记忆。

事件：command:new

要求：必须配置 workspace.dir

输出：<workspace>/memory/YYYY-MM-DD-slug.md（默认为 ~/.openclaw/workspace）

功能：

- 使用预重置会话条目定位正确的记录
- 提取最后 15 行对话
- 使用 LLM 生成描述性文件名 slug
- 将会话元数据保存到带日期的记忆文件

示例输出：

代码：# Session: 2026-01-16 14:30:00 UTC

代码：- ****Session Key****: agent:main:main

代码：- ****Session ID****: abc123def456

代码：- ****Source****: telegram

文件名示例：

- 2026-01-16-vendor-pitch.md
- 2026-01-16-api-design.md
- 2026-01-16-1430.md（如果 slug 生成失败则回退到时间戳）

启用：

代码：openclaw hooks enable session-memory

command-logger

将所有命令事件记录到集中审计文件。

事件：command

要求：无

输出：~/openclaw/logs/commands.log

功能：

- 捕获事件详情（命令操作、时间戳、会话键、发送者 ID、来源）
- 以 JSONL 格式追加到日志文件
- 在后台静默运行

示例日志条目：

代码：{"timestamp":"2026-01-16T14:30:00.000Z","action":"new","sessionKey":"agent:main:main","senderId":"+1234567890","source":"telegram"}

代码：{"timestamp":"2026-01-16T15:45:22.000Z","action":"stop","sessionKey":"agent:main:main","senderId":"user@example.com","source":"whatsapp"}

查看日志：

代码：# View recent commands

代码：tail -n 20 ~/openclaw/logs/commands.log

代码：# Pretty-print with jq

代码：cat ~/.openclaw/logs/commands.log | jq .

代码：# Filter by action

代码：grep '"action":"new"' ~/.openclaw/logs/commands.log | jq .

启用：

代码：openclaw hooks enable command-logger

boot-md

当 Gateway 网关启动时运行 BOOT.md（在渠道启动之后）。

必须启用内部 hooks 才能运行。

事件：gateway:startup

要求：必须配置 workspace.dir

功能：

- 从你的工作区读取 BOOT.md
- 通过智能体运行器运行指令
- 通过 message 工具发送任何请求的出站消息

启用：

代码：openclaw hooks enable boot-md

最佳实践

保持处理程序快速

Hooks 在命令处理期间运行。保持它们轻量：

代码：// Good - async work, returns immediately

代码：const handler: HookHandler = async (event) => {

代码：void processInBackground(event); // Fire and forget

代码：};

代码：// Bad - blocks command processing

代码：const handler: HookHandler = async (event) => {

代码：await slowDatabaseQuery(event);

代码：await evenSlowerAPICall(event);

代码：};

优雅处理错误

始终包装有风险的操作：

代码：const handler: HookHandler = async (event) => {

代码：try {

代码：await riskyOperation(event);

代码：} catch (err) {

代码：console.error("[my-handler] Failed:", err instanceof Error ? err.message : String(err));

代码：// Don't throw - let other handlers run

代码：}

代码：};

尽早过滤事件

如果事件不相关则尽早返回：

代码：const handler: HookHandler = async (event) => {

代码：// Only handle 'new' commands

代码：if (event.type !== "command" || event.action !== "new") {

代码：return;

代码：}

代码：// Your logic here

代码：};

使用特定事件键

尽可能在元数据中指定确切事件：

代码：metadata: { "openclaw": { "events": ["command:new"] } } # Specific

而不是：

代码：metadata: { "openclaw": { "events": ["command"] } } # General - more overhead

调试

启用 Hook 日志

Gateway 网关在启动时记录 hook 加载：

代码：Registered hook: session-memory -> command:new

代码：Registered hook: command-logger -> command

代码：Registered hook: boot-md -> gateway:startup

检查发现

列出所有发现的 hooks：

代码：openclaw hooks list --verbose

检查注册

在你的处理程序中，记录它被调用的时间：

代码：const handler: HookHandler = async (event) => {

代码： console.log("[my-handler] Triggered:", event.type, event.action);

代码： // Your logic

代码：};

验证资格

检查为什么 hook 不符合条件：

代码：openclaw hooks info my-hook

在输出中查找缺失的要求。

测试

Gateway 网关日志

监控 Gateway 网关日志以查看 hook 执行：

代码：# macOS

代码：./scripts/clawlog.sh -f

代码：# Other platforms

代码：tail -f ~/.openclaw/gateway.log

直接测试 Hooks

隔离测试你的处理程序：

代码：test("my handler works", async () => {

代码：const event = createHookEvent("command", "new", "test-session", {

代码：foo: "bar",

代码：});

代码：await myHandler(event);

代码：// Assert side effects

代码：});

架构

核心组件

- src/hooks/types.ts：类型定义
- src/hooks/workspace.ts：目录扫描和加载
- src/hooks/frontmatter.ts：HOOK.md 元数据解析
- src/hooks/config.ts：资格检查
- src/hooks/hooks-status.ts：状态报告
- src/hooks/loader.ts：动态模块加载器
- src/cli/hooks-cli.ts：CLI 命令
- src/gateway/server-startup.ts：在 Gateway 网关启动时加载 hooks
- src/auto-reply/reply/commands-core.ts：触发命令事件

发现流程

代码：Gateway 网关启动

代码： ↓

代码：扫描目录（工作区 → 托管 → 捆绑）

代码： ↓

代码：解析 HOOK.md 文件

代码： ↓

代码：检查资格（bins、env、config、os）

代码： ↓

代码：从符合条件的 hooks 加载处理程序

代码： ↓

代码：为事件注册处理程序

事件流程

代码：用户发送 /new

代码： ↓

代码：命令验证

代码： ↓

代码：创建 hook 事件

代码： ↓

代码：触发 hook（所有注册的处理程序）

代码： ↓

代码：命令处理继续

代码： ↓

代码：会话重置

故障排除

Hook 未被发现

- 检查目录结构：

代码：ls -la ~/.openclaw/hooks/my-hook/

代码：# Should show: HOOK.md, handler.ts

- 验证 HOOK.md 格式：

代码： `cat ~/.openclaw/hooks/my-hook/HOOK.md`

代码： `# Should have YAML frontmatter with name and metadata`

- 列出所有发现的 hooks：

代码： `openclaw hooks list`

Hook 不符合条件

检查要求：

代码： `openclaw hooks info my-hook`

查找缺失的：

- 二进制文件（检查 PATH）
- 环境变量
- 配置值
- 操作系统兼容性

Hook 未执行

- 验证 hook 已启用：

代码： `openclaw hooks list`

代码： `# Should show next to enabled hooks`

- 重启你的 Gateway 网关进程以重新加载 hooks。
- 检查 Gateway 网关日志中的错误：

代码： `./scripts/clawlog.sh | grep hook`

处理程序错误

检查 TypeScript/import 错误：

代码： `# Test import directly`

代码： `node -e "import('./path/to/handler.ts').then(console.log)"`

迁移指南

从遗留配置到发现

之前：

```
代码：{  
  代码："hooks": {  
    代码："internal": {  
      代码："enabled": true,  
      代码："handlers": [  
        代码：{  
          代码："event": "command:new",  
          代码："module": "./hooks/handlers/my-handler.ts"  
        }  
      ]  
    }  
  }  
}
```

之后：

- 创建 hook 目录：

```
代码：mkdir -p ~/.openclaw/hooks/my-hook  
代码：mv ./hooks/handlers/my-handler.ts ~/.openclaw/hooks/my-hook/handler.ts
```

- 创建 HOOK.md：

```
代码：---  
代码：name: my-hook  
代码：description: "My custom hook"  
代码：metadata: { "openclaw": { "emoji": "", "events": ["command:new"] } }  
代码：---  
  
代码：# My Hook  
  
代码：Does something useful.
```

- 更新配置：

代码：{

代码： "hooks": {

代码： "internal": {

代码： "enabled": true,

代码： "entries": {

代码： "my-hook": { "enabled": true }

代码： }

代码： }

代码： }

代码： }

- 验证并重启你的 Gateway 网关进程：

代码： openclaw hooks list

代码： # Should show: my-hook

迁移的好处：

- 自动发现
- CLI 管理
- 资格检查
- 更好的文档
- 一致的结构

另请参阅

- CLI 参考：hooks
- 捆绑 Hooks README
- Webhook Hooks
- 配置

6. 投票

投票

支持的渠道

- WhatsApp (Web 渠道)
- Discord
- MS Teams (Adaptive Cards)

CLI

代码：# WhatsApp

代码：openclaw message poll --target +15555550123 \

代码：--poll-question "Lunch today?" --poll-option "Yes" --poll-option "No" --poll-option "Maybe"

代码：openclaw message poll --target 123456789@g.us \

代码：--poll-question "Meeting time?" --poll-option "10am" --poll-option "2pm" --poll-option "4pm" --poll-multi

代码：# Discord

代码：openclaw message poll --channel discord --target channel:123456789 \

代码：--poll-question "Snack?" --poll-option "Pizza" --poll-option "Sushi"

代码：openclaw message poll --channel discord --target channel:123456789 \

代码：--poll-question "Plan?" --poll-option "A" --poll-option "B" --poll-duration-hours 48

代码：# MS Teams

代码：openclaw message poll --channel msteams --target conversation:19:abc@thread.tacv2 \

代码：--poll-question "Lunch?" --poll-option "Pizza" --poll-option "Sushi"

选项：

- --channel：whatsapp (默认)、discord 或 msteams
- --poll-multi：允许选择多个选项
- --poll-duration-hours：仅限 Discord (省略时默认为 24)

Gateway 网关 RPC

方法：poll

参数：

- to (字符串, 必需)
- question (字符串, 必需)
- options (字符串数组, 必需)
- maxSelections (数字, 可选)
- durationHours (数字, 可选)
- channel (字符串, 可选, 默认: whatsapp)
- idempotencyKey (字符串, 必需)

渠道差异

- WhatsApp: 2-12 个选项, maxSelections 必须在选项数量范围内, 忽略 durationHours。
- Discord: 2-10 个选项, durationHours 限制在 1-768 小时之间 (默认 24)。maxSelections > 1 启用多选; Discord 不支持严格的选择数量限制。
- MS Teams: Adaptive Card 投票 (由 OpenClaw 管理)。无原生投票 API; durationHours 被忽略。

智能体工具 (Message)

使用 message 工具的 poll 操作 (to、pollQuestion、pollOption, 可选 pollMulti、pollDurationHours、channel)。

注意: Discord 没有"恰好选择 N 个"模式; pollMulti 映射为多选。

Teams 投票以 Adaptive Cards 形式渲染, 需要 Gateway 网关保持在线以将投票记录到 ~/.openclaw/msteams-polls.json。

7. 自动化故障排查

自动化故障排查

该页面是英文文档的中文占位版本, 完整内容请先参考英文版: Automation Troubleshooting。

8. Webhooks

Webhooks

Gateway 网关可以暴露一个小型 HTTP webhook 端点用于外部触发。

启用

代码：{

代码： hooks: {

代码： enabled: true,

代码： token: "shared-secret",

代码： path: "/hooks",

代码： },

代码： }

注意事项：

- 当 hooks.enabled=true 时，hooks.token 为必填项。
- hooks.path 默认为 /hooks。

认证

每个请求必须包含 hook 令牌。推荐使用请求头：

- Authorization: Bearer <token>（推荐）
- x-openclaw-token: <token>
- ?token=<token>（已弃用；会记录警告日志，将在未来的主要版本中移除）

端点

`POST /hooks/wake`

请求体：

代码：{ "text": "System line", "mode": "now" }

- text 必填（字符串）：事件描述（例如"收到新邮件"）。
- mode 可选（now | next-heartbeat）：是否立即触发心跳（默认 now）或等待下一次定期检查。

效果：

- 为主会话加入一个系统事件队列
- 如果 mode=now，则立即触发心跳

`POST /hooks/agent`

请求体：

代码：{

代码： "message": "Run this",

代码： "name": "Email",

代码： "sessionKey": "hook:email:msg-123",

代码： "wakeMode": "now",

代码： "deliver": true,

代码： "channel": "last",

代码： "to": "+15551234567",

代码： "model": "openai/gpt-5.2-mini",

代码： "thinking": "low",

代码： "timeoutSeconds": 120

代码： }

- message 必填（字符串）：智能体要处理的提示或消息。
- name 可选（字符串）：hook 的可读名称（例如"GitHub"），用作会话摘要的前缀。
- sessionKey 可选（字符串）：用于标识智能体会话的键。默认为随机的 hook:<uuid>。使用一致的键可以在 hook 上下文中进行多轮对话。
- wakeMode 可选（now | next-heartbeat）：是否立即触发心跳（默认 now）或等待下一次定期检查。
- deliver 可选（布尔值）：如果为 true，智能体的响应将发送到消息渠道。默认为 true。仅为心跳确认的响应会自动跳过。
- channel 可选（字符串）：用于投递的消息渠道。可选值：last、whatsapp、telegram、discord、slack、mattermost（插件）、signal、imessage、msteams。默认为 last。
- to 可选（字符串）：渠道的接收者标识符（例如 WhatsApp/Signal 的电话号码、Telegram 的聊天 ID、Discord/Slack/Mattermost（插件）的频道 ID、MS Teams 的会话 ID）。默认为主会话中的最后一个接收者。
- model 可选（字符串）：模型覆盖（例如 anthropic/claude-3-5-sonnet 或别名）。如果有限制，必须在允许的模型列表中。
- thinking 可选（字符串）：思考级别覆盖（例如 low、medium、high）。
- timeoutSeconds 可选（数字）：智能体运行的最大持续时间（秒）。

效果：

- 运行一个隔离的智能体回合（独立的会话键）
- 始终在主会话中发布摘要
- 如果 wakeMode=now，则立即触发心跳

`POST /hooks/<name>`（映射）

自定义 hook 名称通过 hooks.mappings 解析（见配置）。映射可以将任意请求体转换为 wake 或 agent 操作，支持可选的模板或代码转换。

映射选项（摘要）：

- hooks.presets: ["gmail"] 启用内置的 Gmail 映射。
- hooks.mappings 允许你在配置中定义 match、action 和模板。
- hooks.transformsDir + transform.module 加载 JS/TS 模块用于自定义逻辑。
- 使用 match.source 保持通用的接收端点（基于请求体的路由）。
- TS 转换需要 TS 加载器（例如 bun 或 tsx）或运行时预编译的 .js。
- 在映射上设置 deliver: true + channel/to 可将回复路由到聊天界面（channel 默认为 last，回退到 WhatsApp）。
- allowUnsafeExternalContent: true 禁用该 hook 的外部内容安全包装（危险；仅用于受信任的内部来源）。
- openclaw webhooks gmail setup 为 openclaw webhooks gmail run 写入 hooks.gmail 配置。完整的 Gmail 监听流程请参阅 Gmail Pub/Sub。

响应

- 200 用于 /hooks/wake
- 202 用于 /hooks/agent（异步运行已启动）
- 401 认证失败
- 400 请求体无效
- 413 请求体过大

示例

代码：curl -X POST http://127.0.0.1:18789/hooks/wake \

代码：-H 'Authorization: Bearer SECRET' \

代码：-H 'Content-Type: application/json' \

代码：-d '{"text":"New email received","mode":"now"}'

代码：curl -X POST http://127.0.0.1:18789/hooks/agent \

代码：-H 'x-openclaw-token: SECRET' \

代码：-H 'Content-Type: application/json' \

代码：-d '{"message":"Summarize inbox","name":"Email","wakeMode":"next-heartbeat"}'

使用不同的模型

在智能体请求体（或映射）中添加 model 以覆盖该次运行的模型：

代码：curl -X POST http://127.0.0.1:18789/hooks/agent \

代码：-H 'x-openclaw-token: SECRET' \

代码：-H 'Content-Type: application/json' \

代码：-d '{"message":"Summarize inbox","name":"Email","model":"openai/gpt-5.2-mini"}'

如果你启用了 agents.defaults.models 限制，请确保覆盖的模型包含在其中。

代码：curl -X POST http://127.0.0.1:18789/hooks/gmail \

代码：-H 'Authorization: Bearer SECRET' \

代码：-H 'Content-Type: application/json' \

代码：-d '{"source":"gmail","messages":[{"from":"Ada","subject":"Hello","snippet":"Hi"}]}'

安全

- 将 hook 端点保持在 loopback、tailnet 或受信任的反向代理之后。
- 使用专用的 hook 令牌；不要复用 Gateway 网关认证令牌。
- 避免在 webhook 日志中包含敏感的原始请求体。
- Hook 请求体默认被视为不受信任并使用安全边界包装。如果你必须为特定 hook 禁用此功能，请在该 hook 的映射中设置 allowUnsafeExternalContent: true（危险）。

第八章：节点能力与远程设备

1. 音频 / 语音消息 — 2026-01-17

音频 / 语音消息 — 2026-01-17

已支持的功能

- 媒体理解（音频）：如果音频理解已启用（或自动检测），OpenClaw 会：
- 找到第一个音频附件（本地路径或 URL），如有需要则下载。
- 在发送给每个模型条目之前执行 maxBytes 限制。
- 按顺序运行第一个符合条件的模型条目（提供商或 CLI）。
- 如果失败或跳过（大小/超时），则尝试下一个条目。
- 成功后，将 Body 替换为 [Audio] 块并设置 {{Transcript}}。
- 命令解析：转录成功时，CommandBody/RawBody 会设置为转录文本，因此斜杠命令仍然有效。
- 详细日志：在 --verbose 模式下，我们会在转录运行和替换正文时记录日志。

自动检测（默认）

如果你未配置模型且 tools.media.audio.enabled 未设置为 false，OpenClaw 会按以下顺序自动检测，并在找到第一个可用选项时停止：

- 本地 CLI（如已安装）
- sherpa-onnx-offline（需要 SHERPA_ONNX_MODEL_DIR 包含 encoder/decoder/joiner/tokens）
- whisper-cli（来自 whisper-cpp；使用 WHISPER_CPP_MODEL 或内置的 tiny 模型）
- whisper（Python CLI；自动下载模型）
- Gemini CLI（gemini）使用 read_many_files
- 提供商密钥（OpenAI → Groq → Deepgram → Google）

要禁用自动检测，请设置 tools.media.audio.enabled: false。

要自定义，请设置 tools.media.audio.models。

注意：二进制检测在 macOS/Linux/Windows 上采用尽力而为的方式；请确保 CLI 在 PATH 中（我们会展开 ~），或通过完整命令路径设置显式 CLI 模型。

配置示例

提供商 + CLI 回退 (OpenAI + Whisper CLI)

```
代码 : {  
代码 : tools: {  
代码 : media: {  
代码 : audio: {  
代码 : enabled: true,  
代码 : maxBytes: 20971520,  
代码 : models: [  
代码 : { provider: "openai", model: "gpt-4o-mini-transcribe" },  
代码 : {  
代码 : type: "cli",  
代码 : command: "whisper",  
代码 : args: ["--model", "base", "{{MediaPath}}"],  
代码 : timeoutSeconds: 45,  
代码 : },  
代码 : ],  
代码 : },  
代码 : },  
代码 : },  
代码 : },  
代码 : }
```

仅提供商 + 作用域控制

```
代码 : {  
代码 : tools: {  
代码 : media: {  
代码 : audio: {  
代码 : enabled: true,  
代码 : scope: {  
代码 : default: "allow",  
代码 : rules: [{ action: "deny", match: { chatType: "group" } }],
```

代码： },

代码： models: [{ provider: "openai", model: "gpt-4o-mini-transcribe" }],

代码： },

代码： },

代码： },

代码： }

仅提供商（Deepgram）

代码： {

代码： tools: {

代码： media: {

代码： audio: {

代码： enabled: true,

代码： models: [{ provider: "deepgram", model: "nova-3" }],

代码： },

代码： },

代码： },

代码： }

注意事项与限制

- 提供商认证遵循标准的模型认证顺序（认证配置文件、环境变量、models.providers..apiKey）。
- 当使用 provider: "deepgram" 时，Deepgram 会读取 DEEPGRAM_API_KEY。
- Deepgram 设置详情：Deepgram（音频转录）。
- 音频提供商可以通过 tools.media.audio 覆盖 baseUrl、headers 和 providerOptions。
- 默认大小限制为 20MB（tools.media.audio.maxBytes）。超大音频会跳过该模型并尝试下一个条目。
- 音频的默认 maxChars 未设置（完整转录文本）。设置 tools.media.audio.maxChars 或每个条目的 maxChars 来裁剪输出。
- OpenAI 自动检测默认使用 gpt-4o-mini-transcribe；设置 model: "gpt-4o-transcribe" 可获得更高准确度。

- 使用 `tools.media.audio.attachments` 处理多条语音消息 (`mode: "all" + maxAttachments`) 。
- 转录文本可在模板中通过 `{{Transcript}}` 使用。
- CLI 标准输出有上限 (5MB) ；请保持 CLI 输出简洁。

常见陷阱

- 作用域规则采用首次匹配优先。`chatType` 会被规范化为 `direct`、`group` 或 `room`。
- 确保你的 CLI 以退出码 0 退出并输出纯文本；JSON 格式需要通过 `jq -r .text` 进行转换。
- 保持合理的超时时间 (`timeoutSeconds`，默认 60 秒)，以避免阻塞回复队列。

2. 相机捕获（智能体）

相机捕获（智能体）

OpenClaw 支持用于智能体工作流的相机捕获：

- iOS 节点（通过 Gateway 网关配对）：通过 `node.invoke` 捕获照片 (jpg) 或短视频片段 (mp4，可选音频)。
- Android 节点（通过 Gateway 网关配对）：通过 `node.invoke` 捕获照片 (jpg) 或短视频片段 (mp4，可选音频)。
- macOS 应用（通过 Gateway 网关的节点）：通过 `node.invoke` 捕获照片 (jpg) 或短视频片段 (mp4，可选音频)。

所有相机访问都受用户控制的设置限制。

iOS 节点

用户设置（默认开启）

- iOS 设置标签页 → 相机 → 允许相机 (`camera.enabled`)
- 默认：开启（缺少键时视为启用）。
- 关闭时：`camera.` 命令返回 `CAMERA_DISABLED`。

命令（通过 Gateway 网关 ``node.invoke``）

- `camera.list`
- 响应载荷：
- `devices`：{ `id`, `name`, `position`, `deviceType` } 数组

- camera.snap
- 参数：
- facing : front|back (默认 : front)
- maxWidth : 数字 (可选 ; iOS 节点默认 1600)
- quality : 0..1 (可选 ; 默认 0.9)
- format : 当前为 jpg
- delayMs : 数字 (可选 ; 默认 0)
- deviceId : 字符串 (可选 ; 来自 camera.list)
- 响应载荷：
- format: "jpg"
- base64: "<...>"
- width 、 height
- 载荷保护：照片会重新压缩以保持 base64 载荷小于 5 MB。

- camera.clip
- 参数：
- facing : front|back (默认 : front)
- durationMs : 数字 (默认 3000 , 上限 60000)
- includeAudio : 布尔值 (默认 true)
- format : 当前为 mp4
- deviceId : 字符串 (可选 ; 来自 camera.list)
- 响应载荷：
- format: "mp4"
- base64: "<...>"
- durationMs
- hasAudio

前台要求

与 canvas. 类似，iOS 节点仅允许在前台执行 camera. 命令。后台调用返回 NODE_BACKGROUND_UNAVAILABLE。

CLI 辅助工具 (临时文件 + MEDIA)

获取附件最简单的方法是通过 CLI 辅助工具，它将解码的媒体写入临时文件并打印 MEDIA:<path>。

示例：

代码：openclaw nodes camera snap --node <id> # default: both front + back (2 MEDIA lines)

代码：openclaw nodes camera snap --node <id> --facing front

代码：openclaw nodes camera clip --node <id> --duration 3000

代码：openclaw nodes camera clip --node <id> --no-audio

注意事项：

- nodes camera snap 默认拍摄两个方向以给智能体提供两个视角。
- 输出文件是临时的（在操作系统临时目录中），除非你构建自己的包装器。

Android 节点

用户设置（默认开启）

- Android 设置页 → 相机 → 允许相机（camera.enabled）
- 默认：开启（缺少键时视为启用）。
- 关闭时：camera. 命令返回 CAMERA_DISABLED。

权限

- Android 需要运行时权限：
- CAMERA 用于 camera.snap 和 camera.clip。
- RECORD_AUDIO 用于 includeAudio=true 时的 camera.clip。

如果缺少权限，应用会在可能时提示；如果被拒绝，camera. 请求会失败并返回 _PERMISSION_REQUIRED 错误。

前台要求

与 canvas. 类似，Android 节点仅允许在前台执行 camera. 命令。后台调用返回 NODE_BACKGROUND_UNAVAILABLE。

载荷保护

照片会重新压缩以保持 base64 载荷小于 5 MB。

macOS 应用

用户设置（默认关闭）

macOS 配套应用暴露一个复选框：

- 设置 → 通用 → 允许相机 (openclaw.cameraEnabled)
- 默认：关闭
- 关闭时：相机请求返回"用户已禁用相机"。

CLI 辅助工具（节点调用）

使用主 openclaw CLI 在 macOS 节点上调用相机命令。

示例：

代码：openclaw nodes camera list --node <id> # list camera ids

代码：openclaw nodes camera snap --node <id> # prints MEDIA:<path>

代码：openclaw nodes camera snap --node <id> --max-width 1280

代码：openclaw nodes camera snap --node <id> --delay-ms 2000

代码：openclaw nodes camera snap --node <id> --device-id <id>

代码：openclaw nodes camera clip --node <id> --duration 10s # prints MEDIA:<path>

代码：openclaw nodes camera clip --node <id> --duration-ms 3000 # prints
MEDIA:<path> (legacy flag)

代码：openclaw nodes camera clip --node <id> --device-id <id>

代码：openclaw nodes camera clip --node <id> --no-audio

注意事项：

- openclaw nodes camera snap 默认 maxWidht=1600，除非被覆盖。
- 在 macOS 上，camera.snap 在预热/曝光稳定后等待 delayMs（默认 2000ms）再捕获。
- 照片载荷会重新压缩以保持 base64 小于 5 MB。

安全性 + 实际限制

- 相机和麦克风访问会触发通常的操作系统权限提示（并需要 Info.plist 中的使用说明字符串）。

- 视频片段有上限（当前 $\leq 60s$ ）以避免过大的节点载荷（base64 开销 + 消息限制）。

macOS 屏幕视频（操作系统级别）

对于屏幕视频（非相机），使用 macOS 配套应用：

代码：openclaw nodes screen record --node <id> --duration 10s --fps 15 # prints
MEDIA:<path>

注意事项：

- 需要 macOS 屏幕录制权限（TCC）。

3. 图像与媒体支持 — 2025-12-05

图像与媒体支持 — 2025-12-05

WhatsApp 渠道通过 Baileys Web 运行。本文档记录了发送、Gateway 网关和智能体回复的当前媒体处理规则。

目标

- 通过 openclaw message send --media 发送带可选标题的媒体。
- 允许来自网页收件箱的自动回复在文本旁边包含媒体。
- 保持每种类型的限制合理且可预测。

CLI 接口

- openclaw message send --media <path-or-url> [--message <caption>]
- --media 可选；标题可以为空以进行纯媒体发送。
- --dry-run 打印解析后的负载；--json 输出 { channel, to, messageId, mediaUrl, caption }。

WhatsApp Web 渠道行为

- 输入：本地文件路径或 HTTP(S) URL。
- 流程：加载到 Buffer，检测媒体类型，并构建正确的负载：
- 图像：调整大小并重新压缩为 JPEG（最大边 2048px），目标为 agents.defaults.mediaMaxMb（默认 5 MB），上限 6 MB。
- 音频/语音/视频：直通最大 16 MB；音频作为语音消息发送（ptt: true）。
- 文档：其他任何内容，最大 100 MB，可用时保留文件名。

- WhatsApp GIF 风格播放：发送带 gifPlayback: true 的 MP4（CLI：--gif-playback），使移动客户端内联循环播放。
- MIME 检测优先使用魔数字节，然后是头信息，最后是文件扩展名。
- 标题来自 --message 或 reply.text；允许空标题。
- 日志：非详细模式显示 /；详细模式包含大小和源路径/URL。

自动回复管道

- getReplyFromConfig 返回 { text?, mediaUrl?, mediaUrls? }。
- 当存在媒体时，网页发送器使用与 openclaw message send 相同的管道解析本地路径或 URL。
- 如果提供多个媒体条目，则按顺序发送。

入站媒体到命令 (Pi)

- 当入站网页消息包含媒体时，OpenClaw 下载到临时文件并暴露模板变量：
- {{MediaUrl}} 入站媒体的伪 URL。
- {{MediaPath}} 运行命令前写入的本地临时路径。
- 当启用每会话 Docker 沙箱时，入站媒体被复制到沙箱工作区，MediaPath/MediaUrl 被重写为相对路径如 media/inbound/<filename>。
- 媒体理解（如果通过 tools.media. 或共享的 tools.media.models 配置）在模板化之前运行，可以将 [Image]、[Audio] 和 [Video] 块插入 Body。
- 音频设置 {{Transcript}} 并使用转录进行命令解析，因此斜杠命令仍然有效。
- 视频和图像描述保留任何标题文本用于命令解析。
- 默认情况下只处理第一个匹配的图像/音频/视频附件；设置 tools.media.<cap>.attachments 以处理多个附件。

限制与错误

出站发送上限（WhatsApp 网页发送）

- 图像：重新压缩后约 6 MB 上限。
- 音频/语音/视频：16 MB 上限；文档：100 MB 上限。
- 超大或无法读取的媒体 → 日志中有明确错误，回复被跳过。

媒体理解上限（转录/描述）

- 图像默认：10 MB (tools.media.image.maxBytes) 。
- 音频默认：20 MB (tools.media.audio.maxBytes) 。
- 视频默认：50 MB (tools.media.video.maxBytes) 。
- 超大媒体跳过理解，但回复仍然使用原始正文通过。

测试说明

- 覆盖图像/音频/文档情况的发送 + 回复流程。
- 验证图像的重新压缩（大小限制）和音频的语音消息标志。
- 确保多媒体回复作为顺序发送扇出。

4. 节点

节点

节点是一个配套设备 (macOS/iOS/Android/无头)，它以 `role: "node"` 连接到 Gateway 网关 WebSocket（与操作员相同的端口），并通过 `node.invoke` 暴露命令接口（例如 `canvas.`、`camera.`、`system.`）。协议详情：Gateway 网关协议。

旧版传输：Bridge 协议（TCP JSONL；当前节点已弃用/移除）。

macOS 也可以在节点模式下运行：菜单栏应用连接到 Gateway 网关的 WS 服务器，并将其本地 `canvas/camera` 命令作为节点暴露（因此 `openclaw nodes ...` 可以针对这台 Mac 工作）。

注意事项：

- 节点是外围设备，不是 Gateway 网关。它们不运行 Gateway 网关服务。
- Telegram/WhatsApp 等消息落在 Gateway 网关上，而不是节点上。

配对 + 状态

WS 节点使用设备配对。节点在 `connect` 期间呈现设备身份；Gateway 网关为 `role: node` 创建设备配对请求。通过设备 CLI（或 UI）批准。

快速 CLI：

代码：`openclaw devices list`

代码：`openclaw devices approve <requestId>`

代码：`openclaw devices reject <requestId>`

代码：openclaw nodes status

代码：openclaw nodes describe --node <idOrNameOrIp>

注意事项：

- 当节点的设备配对角色包含 node 时，nodes status 将节点标记为已配对。
- node.pair. (CLI：openclaw nodes pending/approve/reject) 是一个单独的 Gateway 网关拥有的

节点配对存储；它不会限制 WS connect 握手。

远程节点主机 (system.run)

当你的 Gateway 网关在一台机器上运行而你希望命令

在另一台机器上执行时，使用节点主机。模型仍然与 Gateway 网关通信；当选择 host=node 时，Gateway 网关

将 exec 调用转发到节点主机。

什么在哪里运行

- Gateway 网关主机：接收消息，运行模型，路由工具调用。
- 节点主机：在节点机器上执行 system.run/system.which。
- 批准：通过 ~/.openclaw/exec-approvals.json 在节点主机上执行。

启动节点主机（前台）

在节点机器上：

代码：openclaw node run --host <gateway-host> --port 18789 --display-name "Build Node"

通过 SSH 隧道访问远程 Gateway 网关（loopback 绑定）

如果 Gateway 网关绑定到 loopback (gateway.bind=loopback，本地模式下的默认值)，远程节点主机无法直接连接。创建 SSH 隧道并将节点主机指向隧道的本地端。

示例（节点主机 -> Gateway 网关主机）：

代码：# 终端 A（保持运行）：转发本地 18790 -> Gateway 网关 127.0.0.1:18789

代码：ssh -N -L 18790:127.0.0.1:18789 user@gateway-host

代码：# 终端 B：导出 Gateway 网关令牌并通过隧道连接

代码：openclaw node run --host 127.0.0.1 --port 18790 --display-name "Build Node"

注意事项：

- 令牌是 Gateway 网关配置中的 gateway.auth.token（Gateway 网关主机上的 ~/.openclaw/openclaw.json）。
- openclaw node run 读取 OPENCLAW_GATEWAY_TOKEN 进行认证。

启动节点主机（服务）

代码：openclaw node install --host <gateway-host> --port 18789 --display-name "Build Node"

代码：openclaw node restart

配对 + 命名

在 Gateway 网关主机上：

代码：openclaw nodes pending

代码：openclaw nodes approve <requestId>

代码：openclaw nodes list

命名选项：

- 在 openclaw node run / openclaw node install 上使用 --display-name（持久化在节点上的 ~/.openclaw/node.json 中）。
- openclaw nodes rename --node <id|name|ip> --name "Build Node"（Gateway 网关覆盖）。

将命令加入允许列表

Exec 批准是每个节点主机的。从 Gateway 网关添加允许列表条目：

代码：openclaw approvals allowlist add --node <id|name|ip> "/usr/bin/uname"

代码：openclaw approvals allowlist add --node <id|name|ip> "/usr/bin/sw_vers"

批准存储在节点主机的 ~/.openclaw/exec-approvals.json 中。

将 exec 指向节点

配置默认值（Gateway 网关配置）：

代码：openclaw config set tools.exec.host node

代码：openclaw config set tools.exec.security allowlist

代码：openclaw config set tools.exec.node "<id-or-name>"

或按会话：

代码：/exec host=node security=allowlist node=<id-or-name>

设置后，任何带有 host=node 的 exec 调用都会在节点主机上运行（受节点允许列表/批准约束）。

相关：

- 节点主机 CLI
- Exec 工具
- Exec 批准

调用命令

低级（原始 RPC）：

代码：openclaw nodes invoke --node <idOrNameOrIp> --command canvas.eval --params '{"javascript":"location.href"}'

对于常见的"给智能体一个 MEDIA 附件"工作流，存在更高级的辅助工具。

截图（canvas 快照）

如果节点正在显示 Canvas（WebView），canvas.snapshot 返回 { format, base64 }。

CLI 辅助工具（写入临时文件并打印 MEDIA:<path>）：

代码：openclaw nodes canvas snapshot --node <idOrNameOrIp> --format png

代码：openclaw nodes canvas snapshot --node <idOrNameOrIp> --format jpg --max-width 1200 --quality 0.9

Canvas 控制

代码：openclaw nodes canvas present --node <idOrNameOrlp> --target https://example.com

代码：openclaw nodes canvas hide --node <idOrNameOrlp>

代码：openclaw nodes canvas navigate https://example.com --node <idOrNameOrlp>

代码：openclaw nodes canvas eval --node <idOrNameOrlp> --js "document.title"

注意事项：

- canvas present 接受 URL 或本地文件路径（--target），以及可选的 --x/--y/--width/--height 用于定位。
- canvas eval 接受内联 JS（--js）或位置参数。

A2UI (Canvas)

代码：openclaw nodes canvas a2ui push --node <idOrNameOrlp> --text "Hello"

代码：openclaw nodes canvas a2ui push --node <idOrNameOrlp> --jsonl ./payload.jsonl

代码：openclaw nodes canvas a2ui reset --node <idOrNameOrlp>

注意事项：

- 仅支持 A2UI v0.8 JSONL（v0.9/createSurface 被拒绝）。

照片 + 视频（节点相机）

照片（jpg）：

代码：openclaw nodes camera list --node <idOrNameOrlp>

代码：openclaw nodes camera snap --node <idOrNameOrlp> # 默认：两个朝向（2 个 MEDIA 行）

代码：openclaw nodes camera snap --node <idOrNameOrlp> --facing front

视频片段（mp4）：

代码：openclaw nodes camera clip --node <idOrNameOrlp> --duration 10s

代码：openclaw nodes camera clip --node <idOrNameOrlp> --duration 3000 --no-audio

注意事项：

- 节点必须处于前台才能使用 canvas. 和 camera.（后台调用返回 NODE_BACKGROUND_UNAVAILABLE）。

- 片段时长被限制（当前 $\leq 60s$ ）以避免过大的 base64 负载。
- Android 会在可能时提示 CAMERA/RECORD_AUDIO 权限；权限被拒绝会以 _PERMISSION_REQUIRED 失败。

屏幕录制（节点）

节点暴露 screen.record (mp4) 。示例：

代码：openclaw nodes screen record --node <idOrNameOrIp> --duration 10s --fps 10

代码：openclaw nodes screen record --node <idOrNameOrIp> --duration 10s --fps 10
--no-audio

注意事项：

- screen.record 需要节点应用处于前台。
- Android 会在录制前显示系统屏幕捕获提示。
- 屏幕录制被限制为 $\leq 60s$ 。
- --no-audio 禁用麦克风捕获（iOS/Android 支持；macOS 使用系统捕获音频）。
- 当有多个屏幕可用时，使用 --screen <index> 选择显示器。

位置（节点）

当在设置中启用位置时，节点暴露 location.get 。

CLI 辅助工具：

代码：openclaw nodes location get --node <idOrNameOrIp>

代码：openclaw nodes location get --node <idOrNameOrIp> --accuracy precise
--max-age 15000 --location-timeout 10000

注意事项：

- 位置默认关闭。
- "始终"需要系统权限；后台获取是尽力而为的。
- 响应包括纬度/经度、精度（米）和时间戳。

短信（Android 节点）

当用户授予 SMS 权限且设备支持电话功能时，Android 节点可以暴露 sms.send 。

低级调用：

代码：openclaw nodes invoke --node <idOrNameOrIp> --command sms.send --params '{"to":"+15555550123","message":"Hello from OpenClaw"}'

注意事项：

- 在能力被广播之前，必须在 Android 设备上接受权限提示。
- 没有电话功能的纯 Wi-Fi 设备不会广播 sms.send。

系统命令（节点主机 / mac 节点）

macOS 节点暴露 system.run、system.notify 和 system.execApprovals.get/set。

无头节点主机暴露 system.run、system.which 和 system.execApprovals.get/set。

示例：

代码：openclaw nodes run --node <idOrNameOrIp> -- echo "Hello from mac node"

代码：openclaw nodes notify --node <idOrNameOrIp> --title "Ping" --body "Gateway ready"

注意事项：

- system.run 在负载中返回 stdout/stderr/退出码。
 - system.notify 遵守 macOS 应用上的通知权限状态。
 - system.run 支持 --cwd、--env KEY=VAL、--command-timeout 和 --needs-screen-recording。
 - system.notify 支持 --priority <passive|active|timeSensitive> 和 --delivery <system|overlay|auto>。
 - macOS 节点会丢弃 PATH 覆盖；无头节点主机仅在 PATH 前置到节点主机 PATH 时才接受它。
 - 在 macOS 节点模式下，system.run 受 macOS 应用中的 exec 批准限制（设置 → Exec 批准）。
- Ask/allowlist/full 的行为与无头节点主机相同；被拒绝的提示返回 SYSTEM_RUN_DENIED。
- 在无头节点主机上，system.run 受 exec 批准限制（~/openclaw/exec-approvals.json）。

Exec 节点绑定

当有多个节点可用时，你可以将 exec 绑定到特定节点。

这设置了 exec host=node 的默认节点（可以按智能体覆盖）。

全局默认：

代码：openclaw config set tools.exec.node "node-id-or-name"

按智能体覆盖：

代码：openclaw config get agents.list

代码：openclaw config set agents.list[0].tools.exec.node "node-id-or-name"

取消设置以允许任何节点：

代码：openclaw config unset tools.exec.node

代码：openclaw config unset agents.list[0].tools.exec.node

权限映射

节点可能在 node.list / node.describe 中包含 permissions 映射，按权限名称（例如 screenRecording、accessibility）键入，值为布尔值（true = 已授予）。

无头节点主机（跨平台）

OpenClaw 可以运行无头节点主机（无 UI），它连接到 Gateway 网关 WebSocket 并暴露 system.run / system.which。这在 Linux/Windows 上或在服务器旁运行最小节点时很有用。

启动它：

代码：openclaw node run --host <gateway-host> --port 18789

注意事项：

- 仍然需要配对（Gateway 网关会显示节点批准提示）。
- 节点主机将其节点 id、令牌、显示名称和 Gateway 网关连接信息存储在 ~/.openclaw/node.json 中。
- Exec 批准通过 ~/.openclaw/exec-approvals.json 在本地执行（参见 Exec 批准）。
- 在 macOS 上，当配套应用 exec 主机可达时，无头节点主机优先使用它，如果应用不可用则回退到本地执行。设置 OPENCLAW_NODE_EXEC_HOST=app 要求使用应用，或设置 OPENCLAW_NODE_EXEC_FALLBACK=0 禁用回退。

- 当 Gateway 网关 WS 使用 TLS 时，添加 `--tls / --tls-fingerprint`。

Mac 节点模式

- macOS 菜单栏应用作为节点连接到 Gateway 网关 WS 服务器（因此 `openclaw nodes` ... 可以针对这台 Mac 工作）。
- 在远程模式下，应用为 Gateway 网关端口打开 SSH 隧道并连接到 `localhost`。

5. 位置命令（节点）

位置命令（节点）

简要概述

- `location.get` 是一个节点命令（通过 `node.invoke`）。
- 默认关闭。
- 设置使用选择器：关闭 / 使用时 / 始终。
- 单独的开关：精确位置。

为什么用选择器（而不只是开关）

操作系统权限是多级的。我们可以在应用内暴露选择器，但操作系统仍然决定实际授权。

- iOS/macOS：用户可以在系统提示/设置中选择使用时或始终。应用可以请求升级，但操作系统可能要求进入设置。
- Android：后台位置是单独的权限；在 Android 10+ 上通常需要进入设置流程。
- 精确位置是单独的授权（iOS 14+ "精确"，Android "精细" vs "粗略"）。

UI 中的选择器驱动我们请求的模式；实际授权存在于操作系统设置中。

设置模型

每个节点设备：

- `location.enabledMode`：off | whileUsing | always
- `location.preciseEnabled`：bool

UI 行为：

- 选择 whileUsing 请求前台权限。

- 选择 always 首先确保 whileUsing，然后请求后台（或在需要时将用户引导到设置）。
- 如果操作系统拒绝请求的级别，回退到已授予的最高级别并显示状态。

权限映射 (node.permissions)

可选。macOS 节点通过权限映射报告 location；iOS/Android 可能省略它。

命令：``location.get``

通过 `node.invoke` 调用。

参数（建议）：

代码：{

代码： "timeoutMs": 10000,

代码： "maxAgeMs": 15000,

代码： "desiredAccuracy": "coarse|balanced|precise"

代码：}

响应负载：

代码：{

代码： "lat": 48.20849,

代码： "lon": 16.37208,

代码： "accuracyMeters": 12.5,

代码： "altitudeMeters": 182.0,

代码： "speedMps": 0.0,

代码： "headingDeg": 270.0,

代码： "timestamp": "2026-01-03T12:34:56.000Z",

代码： "isPrecise": true,

代码： "source": "gps|wifi|cell|unknown"

代码：}

错误（稳定代码）：

- `LOCATION_DISABLED`：选择器已关闭。
- `LOCATION_PERMISSION_REQUIRED`：缺少请求模式的权限。

- LOCATION_BACKGROUND_UNAVAILABLE：应用在后台但只允许使用时。
- LOCATION_TIMEOUT：在时间内没有定位。
- LOCATION_UNAVAILABLE：系统故障/没有提供商。

后台行为（未来）

目标：模型可以在节点处于后台时请求位置，但仅当：

- 用户选择了始终。
- 操作系统授予后台位置权限。
- 应用被允许在后台运行以获取位置（iOS 后台模式/Android 前台服务或特殊许可）。

推送触发流程（未来）：

- Gateway 网关向节点发送推送（静默推送或 FCM 数据）。
- 节点短暂唤醒并从设备请求位置。
- 节点将负载转发给 Gateway 网关。

说明：

- iOS：需要始终权限 + 后台位置模式。静默推送可能被限流；预期会有间歇性失败。
- Android：后台位置可能需要前台服务；否则预期会被拒绝。

模型/工具集成

- 工具接口：nodes 工具添加 location_get 操作（需要节点）。
- CLI：openclaw nodes location get --node <id>。
- 智能体指南：仅在用户启用位置并理解范围时调用。

UX 文案（建议）

- 关闭："位置共享已禁用。"
- 使用时："仅当 OpenClaw 打开时。"
- 始终："允许后台位置。需要系统权限。"
- 精确："使用精确 GPS 位置。关闭以共享大致位置。"

6. 媒体理解（入站） — 2026-01-17

媒体理解（入站） — 2026-01-17

OpenClaw 可以在回复流程运行之前摘要进站媒体（图片/音频/视频）。它会自动检测本地工具或提供商密钥是否可用，并且可以禁用或自定义。如果理解关闭，模型仍然会像往常一样接收原始文件/URL。

目标

- 可选：将进站媒体预先消化为短文本，以便更快路由 + 更好的命令解析。
- 保留原始媒体传递给模型（始终）。
- 支持提供商 API 和 CLI 回退。
- 允许多个模型并按顺序回退（错误/大小/超时）。

高层行为

- 收集进站附件（MediaPaths、MediaUrls、MediaTypes）。
- 对于每个启用的能力（图片/音频/视频），根据策略选择附件（默认：第一个）。
- 选择第一个符合条件的模型条目（大小 + 能力 + 认证）。
- 如果模型失败或媒体太大，回退到下一个条目。
- 成功时：
- Body 变为 [Image]、[Audio] 或 [Video] 块。
- 音频设置 {{Transcript}}；命令解析在有标题文本时使用标题文本，否则使用转录。
- 标题作为 User text: 保留在块内。

如果理解失败或被禁用，回复流程继续使用原始正文 + 附件。

配置概述

tools.media 支持共享模型加上每能力覆盖：

- tools.media.models：共享模型列表（使用 capabilities 来限定）。
- tools.media.image / tools.media.audio / tools.media.video：
- 默认值（prompt、maxChars、maxBytes、timeoutSeconds、language）
- 提供商覆盖（baseUrl、headers、providerOptions）
- 通过 tools.media.audio.providerOptions.deepgram 配置 Deepgram 音频选项
- 可选的每能力 models 列表（优先于共享模型）
- attachments 策略（mode、maxAttachments、prefer）

- scope（可选的按渠道/聊天类型/会话键限定）
- tools.media.concurrency：最大并发能力运行数（默认 2）。

```
代码：{  
代码： tools: {  
代码： media: {  
代码： models: [  
代码： /* 共享列表 */  
代码： ],  
代码： image: {  
代码： /* 可选覆盖 */  
代码： },  
代码： audio: {  
代码： /* 可选覆盖 */  
代码： },  
代码： video: {  
代码： /* 可选覆盖 */  
代码： },  
代码： },  
代码： },  
代码： },  
代码： }
```

模型条目

每个 models[] 条目可以是提供商或 CLI：

```
代码：{  
代码： type: "provider", // 省略时默认  
代码： provider: "openai",  
代码： model: "gpt-5.2",  
代码： prompt: "Describe the image in <= 500 chars.",  
代码： maxChars: 500,  
代码： maxBytes: 10485760,
```


代码： timeoutSeconds: 60,

代码： capabilities: ["image"], // 可选，用于多模态条目

代码： profile: "vision-profile",

代码： preferredProfile: "vision-fallback",

代码： }

代码： {

代码： type: "cli",

代码： command: "gemini",

代码： args: [

代码： "-m",

代码： "gemini-3-flash",

代码： "--allowed-tools",

代码： "read_file",

代码： "Read the media at {{MediaPath}} and describe it in <= {{MaxChars}} characters.",

代码：],

代码： maxChars: 500,

代码： maxBytes: 52428800,

代码： timeoutSeconds: 120,

代码： capabilities: ["video", "image"],

代码： }

CLI 模板还可以使用：

- {{MediaDir}}（包含媒体文件的目录）
- {{OutputDir}}（为本次运行创建的临时目录）
- {{OutputBase}}（临时文件基础路径，无扩展名）

默认值和限制

推荐默认值：

- maxChars：图片/视频为 500（简短，适合命令）
- maxChars：音频不设置（完整转录，除非你设置限制）

- maxBytes :
- 图片 : 10MB
- 音频 : 20MB
- 视频 : 50MB

规则 :

- 如果媒体超过 maxBytes , 该模型被跳过 , 尝试下一个模型。
- 如果模型返回超过 maxChars , 输出被截断。
- prompt 默认为简单的 "Describe the {media}." 加上 maxChars 指导 (仅图片/视频) 。
- 如果 <capability>.enabled: true 但未配置模型 , 当提供商支持该能力时 , OpenClaw 尝试活动的回复模型。

自动检测媒体理解 (默认)

如果 tools.media.<capability>.enabled 未设置为 false 且你没有配置模型 , OpenClaw 按以下顺序自动检测并在第一个可用选项处停止 :

- 本地 CLI (仅音频 ; 如果已安装)
- sherpa-onnx-offline (需要带有 encoder/decoder/joiner/tokens 的 SHERPA_ONNX_MODEL_DIR)
- whisper-cli (whisper-cpp ; 使用 WHISPER_CPP_MODEL 或捆绑的 tiny 模型)
- whisper (Python CLI ; 自动下载模型)
- Gemini CLI (gemini) 使用 read_many_files
- 提供商密钥
- 音频 : OpenAI → Groq → Deepgram → Google
- 图片 : OpenAI → Anthropic → Google → MiniMax
- 视频 : Google

要禁用自动检测 , 设置 :

代码 : {

代码 : tools: {

代码 : media: {

代码 : audio: {

代码 : enabled: false,

```
代码： },
代码： },
代码： },
代码： }
```

注意：二进制文件检测在 macOS/Linux/Windows 上是尽力而为的；确保 CLI 在 PATH 上（我们会展开 ~），或设置带有完整命令路径的显式 CLI 模型。

能力（可选）

如果你设置了 capabilities，该条目仅对这些媒体类型运行。对于共享列表，OpenClaw 可以推断默认值：

- openai、anthropic、minimax：图片
- google（Gemini API）：图片 + 音频 + 视频
- groq：音频
- deepgram：音频

对于 CLI 条目，显式设置 capabilities 以避免意外匹配。如果你省略 capabilities，该条目对它出现的列表都符合条件。

提供商支持矩阵（OpenClaw 集成）

能力	提供商集成	说明
图片	OpenAI / Anthropic / Google / 其他通过 pi-ai	注册表中任何支持图片的模型都可用。
音频	OpenAI、Groq、Deepgram、Google	提供商转录（Whisper/Deepgram/Gemini）。
视频	Google（Gemini API）	提供商视频理解。

推荐提供商

图片

- 如果支持图片，优先使用你的活动模型。
- 良好的默认值：openai/gpt-5.2、anthropic/claude-opus-4-5、google/gemini-3-pro-preview。

音频

- openai/gpt-4o-mini-transcribe 、 groq/whisper-large-v3-turbo 或 deepgram/nova-3 。
- CLI 回退：whisper-cli (whisper-cpp) 或 whisper 。
- Deepgram 设置：Deepgram (音频转录) 。

视频

- google/gemini-3-flash-preview (快速) 、 google/gemini-3-pro-preview (更丰富) 。
- CLI 回退：gemini CLI (支持对视频/音频使用 read_file) 。

附件策略

每能力的 attachments 控制处理哪些附件：

- mode：first (默认) 或 all
- maxAttachments：限制处理数量 (默认 1)
- prefer：first 、 last 、 path 、 url

当 mode: "all" 时，输出标记为 [Image 1/2] 、 [Audio 2/2] 等。

配置示例

1) 共享模型列表 + 覆盖

代码：{

代码：tools: {

代码：media: {

代码：models: [

代码：{ provider: "openai", model: "gpt-5.2", capabilities: ["image"] },

代码：{

代码：provider: "google",

代码：model: "gemini-3-flash-preview",

代码：capabilities: ["image", "audio", "video"],

代码：},

代码：{

代码：type: "cli",

代码： command: "gemini",
代码： args: [
代码： "-m",
代码： "gemini-3-flash",
代码： "--allowed-tools",
代码： "read_file",
代码： "Read the media at {{MediaPath}} and describe it in <= {{MaxChars}} characters.",
代码：],
代码： capabilities: ["image", "video"],
代码： },
代码：],
代码： audio: {
代码： attachments: { mode: "all", maxAttachments: 2 },
代码： },
代码： video: {
代码： maxChars: 500,
代码： },
代码： },
代码： },
代码： }

2) 仅音频 + 视频（图片关闭）

代码： {
代码： tools: {
代码： media: {
代码： audio: {
代码： enabled: true,
代码： models: [
代码： { provider: "openai", model: "gpt-4o-mini-transcribe" },
代码： {
代码： type: "cli",

```
代码： command: "whisper",
代码： args: ["--model", "base", "{{MediaPath}}"],
代码： },
代码： ],
代码： },
代码： video: {
代码： enabled: true,
代码： maxChars: 500,
代码： models: [
代码： { provider: "google", model: "gemini-3-flash-preview" },
代码： {
代码： type: "cli",
代码： command: "gemini",
代码： args: [
代码： "-m",
代码： "gemini-3-flash",
代码： "--allowed-tools",
代码： "read_file",
代码： "Read the media at {{MediaPath}} and describe it in <= {{MaxChars}} characters.",
代码： ],
代码： },
代码： ],
代码： },
代码： },
代码： },
代码： },
代码： }
```

3) 可选图片理解

```
代码： {
代码： tools: {
代码： media: {
```

```
代码 : image: {
代码 : enabled: true,
代码 : maxBytes: 10485760,
代码 : maxChars: 500,
代码 : models: [
代码 : { provider: "openai", model: "gpt-5.2" },
代码 : { provider: "anthropic", model: "claude-opus-4-5" },
代码 : {
代码 : type: "cli",
代码 : command: "gemini",
代码 : args: [
代码 : "-m",
代码 : "gemini-3-flash",
代码 : "--allowed-tools",
代码 : "read_file",
代码 : "Read the media at {{MediaPath}} and describe it in <= {{MaxChars}} characters.",
代码 : ],
代码 : },
代码 : ],
代码 : },
代码 : },
代码 : },
代码 : },
代码 : },
代码 : }
```

4) 多模态单条目（显式能力）

```
代码 : {
代码 : tools: {
代码 : media: {
代码 : image: {
代码 : models: [
代码 : {
```

```
代码： provider: "google",
代码： model: "gemini-3-pro-preview",
代码： capabilities: ["image", "video", "audio"],
代码： },
代码： ],
代码： },
代码： audio: {
代码： models: [
代码： {
代码： provider: "google",
代码： model: "gemini-3-pro-preview",
代码： capabilities: ["image", "video", "audio"],
代码： },
代码： ],
代码： },
代码： video: {
代码： models: [
代码： {
代码： provider: "google",
代码： model: "gemini-3-pro-preview",
代码： capabilities: ["image", "video", "audio"],
代码： },
代码： ],
代码： },
代码： },
代码： },
代码： }
```

状态输出

当媒体理解运行时，/status 包含一行简短摘要：

代码：Media: image ok (openai/gpt-5.2) · audio skipped (maxBytes)

这显示每能力的结果以及适用时选择的提供商/模型。

注意事项

- 理解是尽力而为的。错误不会阻止回复。
- 即使理解被禁用，附件仍然传递给模型。
- 使用 `scope` 限制理解运行的位置（例如仅私信）。

相关文档

- 配置
- 图片和媒体支持

7. Talk 模式

Talk 模式

Talk 模式是一个连续的语音对话循环：

- 监听语音
- 将转录文本发送到模型（main 会话，`chat.send`）
- 等待响应
- 通过 ElevenLabs 朗读（流式播放）

行为（macOS）

- Talk 模式启用时显示常驻悬浮窗。
- 监听 → 思考 → 朗读阶段转换。
- 短暂停顿（静音窗口）后，当前转录文本被发送。
- 回复被写入 WebChat（与打字相同）。
- 语音中断（默认开启）：如果用户在助手朗读时开始说话，我们会停止播放并记录中断时间戳供下一个提示使用。

回复中的语音指令

助手可以在回复前添加单行 JSON 来控制语音：

代码：{ "voice": "<voice-id>", "once": true }

规则：

- 仅适用于第一个非空行。
- 未知键会被忽略。
- once: true 仅适用于当前回复。
- 没有 once 时，该语音成为 Talk 模式的新默认值。
- JSON 行在 TTS 播放前会被移除。

支持的键：

- voice / voice_id / voiceId
- model / model_id / modelId
- speed 、 rate (WPM) 、 stability 、 similarity 、 style 、 speakerBoost
- seed 、 normalize 、 lang 、 output_format 、 latency_tier
- once

配置（`~/openclaw/openclaw.json`）

代码：{

代码： talk: {

代码： voiceId: "elevenlabs_voice_id",

代码： modelId: "eleven_v3",

代码： outputFormat: "mp3_44100_128",

代码： apiKey: "elevenlabs_api_key",

代码： interruptOnSpeech: true,

代码： },

代码： }

默认值：

- interruptOnSpeech : true
- voiceId : 回退到 ELEVENLABS_VOICE_ID / SAG_VOICE_ID（或当 API 密钥可用时使用第一个 ElevenLabs 语音）
- modelId : 未设置时默认为 eleven_v3

- apiKey：回退到 ELEVENLABS_API_KEY（或 Gateway 网关 shell profile（如果可用））
- outputFormat：macOS/iOS 上默认为 pcm_44100，Android 上默认为 pcm_24000（设置 mp3_ 以强制 MP3 流式传输）

macOS UI

- 菜单栏切换：Talk
- 配置标签页：Talk Mode 组（voice id + 中断开关）
- 悬浮窗：
- 监听：云朵随麦克风电平脉动
- 思考：下沉动画
- 朗读：辐射圆环
- 点击云朵：停止朗读
- 点击 X：退出 Talk 模式

注意事项

- 需要语音 + 麦克风权限。
- 使用 chat.send 针对会话键 main。
- TTS 使用带有 ELEVENLABS_API_KEY 的 ElevenLabs 流式 API，并在 macOS/iOS/Android 上进行增量播放以降低延迟。
- eleven_v3 的 stability 验证为 0.0、0.5 或 1.0；其他模型接受 0..1。
- 设置时 latency_tier 验证为 0..4。
- Android 支持 pcm_16000、pcm_22050、pcm_24000 和 pcm_44100 输出格式，用于低延迟 AudioTrack 流式传输。

8. 节点故障排查

节点故障排查

该页面是英文文档的中文占位版本，完整内容请先参考英文版：Node Troubleshooting。

9. 语音唤醒（全局唤醒词）

语音唤醒（全局唤醒词）

OpenClaw 将唤醒词作为单一全局列表，由 Gateway 网关拥有。

- 没有每节点的自定义唤醒词。
- 任何节点/应用 UI 都可以编辑列表；更改由 Gateway 网关持久化并广播给所有人。
- 每个设备仍保留自己的语音唤醒启用/禁用开关（本地用户体验 + 权限不同）。

存储（Gateway 网关主机）

唤醒词存储在 Gateway 网关机器上：

- ~/.openclaw/settings/voicewake.json

结构：

代码：`{ "triggers": ["openclaw", "claude", "computer"], "updatedAtMs": 1730000000000 }`

协议

方法

- voicewake.get → { triggers: string[] }
- voicewake.set，参数 { triggers: string[] } → { triggers: string[] }

注意事项：

- 触发词会被规范化（修剪空格、删除空值）。空列表回退到默认值。
- 为安全起见会强制执行限制（数量/长度上限）。

事件

- voicewake.changed 载荷 { triggers: string[] }

接收者：

- 所有 WebSocket 客户端（macOS 应用、WebChat 等）
- 所有已连接的节点（iOS/Android），以及节点连接时作为初始"当前状态"推送。

客户端行为

macOS 应用

- 使用全局列表来控制 VoiceWakeRuntime 触发器。
- 在语音唤醒设置中编辑"触发词"会调用 voicewake.set，然后依赖广播保持其他客户端同步。

iOS 节点

- 使用全局列表进行 VoiceWakeManager 触发检测。
- 在设置中编辑唤醒词会调用 voicewake.set（通过 Gateway 网关 WS），同时保持本地唤醒词检测的响应性。

Android 节点

- 在设置中暴露唤醒词编辑器。
- 通过 Gateway 网关 WS 调用 voicewake.set，使编辑在所有地方同步。

第九章：Web 与控制面板

1. 控制 UI（浏览器）

控制 UI（浏览器）

控制 UI 是一个由 Gateway 网关提供服务的小型 Vite + Lit 单页应用：

- 默认：`
- 可选前缀：设置 `gateway.controlUi.basePath`（例如 `/openclaw`）

它直接与同一端口上的 Gateway 网关 WebSocket 通信。

快速打开（本地）

如果 Gateway 网关在同一台计算机上运行，打开：

-

如果页面加载失败，请先启动 Gateway 网关：`openclaw gateway`。

认证在 WebSocket 握手期间通过以下方式提供：

- `connect.params.auth.token`
- `connect.params.auth.password`

仪表板设置面板允许你存储 token；密码不会被持久化。

新手引导向导默认生成一个 Gateway 网关 token，所以在首次连接时将其粘贴到这里。

设备配对（首次连接）

当你从新浏览器或设备连接到控制 UI 时，Gateway 网关需要一次性配对批准 — 即使你在同一个 Tailnet 上且 `gateway.auth.allowTailscale: true`。这是防止未经授权访问的安全措施。

你会看到：`"disconnected (1008): pairing required"`

批准设备：

代码：`# 列出待处理的请求`

代码：`openclaw devices list`

代码：`# 按请求 ID 批准`

代码：`openclaw devices approve <requestId>`

一旦批准，设备会被记住，除非你使用 `openclaw devices revoke --device <id> --role <role>` 撤销它，否则不需要重新批准。参见 Devices CLI 了解 token 轮换和撤销。

注意：

- 本地连接（127.0.0.1）会自动批准。
- 远程连接（LAN、Tailnet 等）需要显式批准。
- 每个浏览器配置文件生成唯一的设备 ID，因此切换浏览器或清除浏览器数据将需要重新配对。

目前可以做什么

- 通过 Gateway 网关 WS 与模型聊天（`chat.history`、`chat.send`、`chat.abort`、`chat.inject`）
- 在聊天中流式传输工具调用 + 实时工具输出卡片（智能体事件）
- 渠道：WhatsApp/Telegram/Discord/Slack + 插件渠道（Mattermost 等）状态 + QR 登录 + 每渠道配置（`channels.status`、`web.login.`、`config.patch`）
- 实例：在线列表 + 刷新（`system-presence`）
- 会话：列表 + 每会话思考/详细覆盖（`sessions.list`、`sessions.patch`）
- 定时任务：列出/添加/运行/启用/禁用 + 运行历史（`cron.`）
- Skills：状态、启用/禁用、安装、API 密钥更新（`skills.`）
- 节点：列表 + 能力（`node.list`）
- 执行批准：编辑 Gateway 网关或节点允许列表 + `exec host=gateway/node` 的询问策略（`exec.approvals.`）
- 配置：查看/编辑 `~/.openclaw/openclaw.json`（`config.get`、`config.set`）
- 配置：应用 + 带验证的重启（`config.apply`）并唤醒上次活动的会话
- 配置写入包含基础哈希保护，以防止覆盖并发编辑
- 配置 schema + 表单渲染（`config.schema`，包括插件 + 渠道 schema）；原始 JSON 编辑器仍然可用
- 调试：状态/健康/模型快照 + 事件日志 + 手动 RPC 调用（`status`、`health`、`models.list`）
- 日志：Gateway 网关文件日志的实时尾部跟踪，带过滤/导出（`logs.tail`）
- 更新：运行包/git 更新 + 重启（`update.run`）并显示重启报告

聊天行为

- `chat.send` 是非阻塞的：它立即以 `{ runId, status: "started" }` 确认，响应通过 chat 事件流式传输。

- 使用相同的 idempotencyKey 重新发送在运行时返回 { status: "in_flight" }，完成后返回 { status: "ok" }。
- chat.inject 将助手备注附加到会话转录，并为仅 UI 更新广播 chat 事件（无智能体运行，无渠道投递）。
- 停止：
- 点击停止（调用 chat.abort）
- 输入 /stop（或 stop|esc|abort|wait|exit|interrupt）以带外中止
- chat.abort 支持 { sessionKey }（无 runId）以中止该会话的所有活动运行

Tailnet 访问（推荐）

集成 Tailscale Serve（首选）

保持 Gateway 网关在 loopback 上，让 Tailscale Serve 用 HTTPS 代理它：

代码：openclaw gateway --tailscale serve

打开：

- gateway.controlUi.basePath`)

默认情况下，当 gateway.auth.allowTailscale 为 true 时，Serve 请求可以通过 Tailscale 身份头（tailscale-user-login）进行认证。OpenClaw 通过使用 tailscale whois 解析 x-forwarded-for 地址并与头匹配来验证身份，并且只在请求通过 Tailscale 的 x-forwarded- 头到达 loopback 时接受这些。如果你想即使对于 Serve 流量也要求 token/密码，请设置 gateway.auth.allowTailscale: false（或强制 gateway.auth.mode: "password"）。

绑定到 tailnet + token

代码：openclaw gateway --bind tailnet --token "\$(openssl rand -hex 32)"

然后打开：

- gateway.controlUi.basePath`)

将 token 粘贴到 UI 设置中（作为 connect.params.auth.token 发送）。

不安全的 HTTP

如果你通过普通 HTTP 打开仪表板（或 WebCrypto。默认情况下，OpenClaw 阻止没有设备身份的控制 UI 连接。

推荐修复：使用 HTTPS (Tailscale Serve) 或在本地打开 UI：

- 、
- ` Gateway 网关主机上)

降级示例（仅通过 HTTP 使用 token）：

代码：{

代码： gateway: {

代码： controlUi: { allowInsecureAuth: true },

代码： bind: "tailnet",

代码： auth: { mode: "token", token: "replace-me" },

代码： },

代码： }

这会为控制 UI 禁用设备身份 + 配对（即使在 HTTPS 上）。仅在你信任网络时使用。

参见 Tailscale 了解 HTTPS 设置指南。

构建 UI

Gateway 网关从 dist/control-ui 提供静态文件。使用以下命令构建：

代码：pnpm ui:build # 首次运行时自动安装 UI 依赖

可选的绝对基础路径（当你想要固定的资源 URL 时）：

代码：OPENCLAW_CONTROL_UI_BASE_PATH=/openclaw/ pnpm ui:build

用于本地开发（单独的开发服务器）：

代码：pnpm ui:dev # 首次运行时自动安装 UI 依赖

然后将 UI 指向你的 Gateway 网关 WS URL（例如 ws://127.0.0.1:18789）。

调试/测试：开发服务器 + 远程 Gateway 网关

控制 UI 是静态文件；WebSocket 目标是可配置的，可以与 HTTP 源不同。当你想要在本地使用 Vite 开发服务器但 Gateway 网关在其他地方运行时，这很方便。

- 启动 UI 开发服务器：pnpm ui:dev

- 打开类似以下的 URL：

代码：http://localhost:5173/?gatewayUrl=ws://<gateway-host>:18789

可选的一次性认证（如需要）：

代码：http://localhost:5173/?gatewayUrl=wss://<gateway-host>:18789&token=<gateway-token>

注意：

- gatewayUrl 在加载后存储在 localStorage 中并从 URL 中移除。
- token 存储在 localStorage 中；password 仅保留在内存中。
- 当 Gateway 网关在 TLS 后面时（Tailscale Serve、HTTPS 代理等），使用 wss://。

远程访问设置详情：远程访问。

2. 仪表板（控制 UI）

仪表板（控制 UI）

Gateway 网关仪表板是默认在 / 提供的浏览器控制 UI
（通过 gateway.controlUi.basePath 覆盖）。

快速打开（本地 Gateway 网关）：

-

关键参考：

- 控制 UI 了解使用方法和 UI 功能。
- Tailscale 了解 Serve/Funnel 自动化。
- Web 界面 了解绑定模式和安全注意事项。

认证通过 connect.params.auth（token 或密码）在 WebSocket 握手时强制执行。

参见 Gateway 网关配置 中的 gateway.auth。

安全注意事项：控制 UI 是一个管理界面（聊天、配置、执行审批）。

不要公开暴露它。UI 在首次加载后将 token 存储在 localStorage 中。

优先使用 localhost、Tailscale Serve 或 SSH 隧道。

快速路径（推荐）

- 新手引导后，CLI 现在会自动打开带有你的 token 的仪表板，并打印相同的带 token 链接。
- 随时重新打开：`openclaw dashboard`（复制链接，如果可能则打开浏览器，如果是无头环境则显示 SSH 提示）。
- token 保持本地（仅查询参数）；UI 在首次加载后移除它并保存到 `localStorage`。

Token 基础（本地 vs 远程）

- Localhost：打开 `openclaw dashboard` 并使用带 token 的链接（`?token=...``）。
- Token 来源：`gateway.auth.token`（或 `OPENCLAW_GATEWAY_TOKEN`）；UI 在首次加载后存储它。
- 非 localhost：使用 Tailscale Serve（如果 `gateway.auth.allowTailscale: true` 则无需 token）、带 token 的 tailnet 绑定，或 SSH 隧道。参见 Web 界面。

如果你看到"unauthorized" / 1008

- 运行 `openclaw dashboard` 获取新的带 token 链接。
- 确保 Gateway 网关可达（本地：`openclaw status`；远程：SSH 隧道 `ssh -N -L 18789:127.0.0.1:18789 user@host` 然后打开`
- 在仪表板设置中，粘贴你在 `gateway.auth.token`（或 `OPENCLAW_GATEWAY_TOKEN`）中配置的相同 token。

3. Web（Gateway 网关）

Web（Gateway 网关）

Gateway 网关从与 Gateway 网关 WebSocket 相同的端口提供一个小型浏览器 Control UI（Vite + Lit）：

- 默认：`
- 可选前缀：设置 `gateway.controlUi.basePath`（例如 `/openclaw`）

功能详见 Control UI。

本页重点介绍绑定模式、安全和面向 Web 的界面。

Webhooks

当 `hooks.enabled=true` 时，Gateway 网关还在同一 HTTP 服务器上公开一个小型 webhook 端点。

参见 Gateway 网关配置 → hooks 了解认证 + 载荷。

配置（默认开启）

当资源存在时（dist/control-ui），Control UI 默认启用。

你可以通过配置控制它：

代码：{

代码： gateway: {

代码： controlUi: { enabled: true, basePath: "/openclaw" }, // basePath 可选

代码： },

代码： }

Tailscale 访问

集成 Serve（推荐）

保持 Gateway 网关在本地回环上，让 Tailscale Serve 代理它：

代码：{

代码： gateway: {

代码： bind: "loopback",

代码： tailscale: { mode: "serve" },

代码： },

代码： }

然后启动 Gateway 网关：

代码： openclaw gateway

打开：

- gateway.controlUi.basePath`)

Tailnet 绑定 + 令牌

代码：{

代码： gateway: {

代码： bind: "tailnet",

代码： controlUi: { enabled: true },

代码： auth: { mode: "token", token: "your-token" },

代码： },

代码： }

然后启动 Gateway 网关（非本地回环绑定需要令牌）：

代码： openclaw gateway

打开：

- gateway.controlUi.basePath`)

公共互联网 (Funnel)

代码： {

代码： gateway: {

代码： bind: "loopback",

代码： tailscale: { mode: "funnel" },

代码： auth: { mode: "password" }, // 或 OPENCLAW_GATEWAY_PASSWORD

代码： },

代码： }

安全注意事项

- Gateway 网关认证默认是必需的（令牌/密码或 Tailscale 身份头）。
- 非本地回环绑定仍然需要共享令牌/密码（gateway.auth 或环境变量）。
- 向导默认生成 Gateway 网关令牌（即使在本地回环上）。
- UI 发送 connect.params.auth.token 或 connect.params.auth.password。
- 使用 Serve 时，当 gateway.auth.allowTailscale 为 true 时，Tailscale 身份头可以满足认证（无需令牌/密码）。设置 gateway.auth.allowTailscale: false 以要求显式凭证。参见 Tailscale 和 安全。
- gateway.tailscale.mode: "funnel" 需要 gateway.auth.mode: "password"（共享密码）。

构建 UI

Gateway 网关从 dist/control-ui 提供静态文件。使用以下命令构建：

代码：pnpm ui:build # 首次运行时自动安装 UI 依赖

4. TUI（终端 UI）

TUI（终端 UI）

快速开始

- 启动 Gateway 网关。

代码：openclaw gateway

- 打开 TUI。

代码：openclaw tui

- 输入消息并按 Enter。

远程 Gateway 网关：

代码：openclaw tui --url ws://<host>:<port> --token <gateway-token>

如果你的 Gateway 网关使用密码认证，请使用 --password。

你看到的内容

- 标题栏：连接 URL、当前智能体、当前会话。
- 聊天日志：用户消息、助手回复、系统通知、工具卡片。
- 状态行：连接/运行状态（连接中、运行中、流式传输中、空闲、错误）。
- 页脚：连接状态 + 智能体 + 会话 + 模型 + think/verbose/reasoning + token 计数 + 投递状态。
- 输入：带自动完成的文本编辑器。

心智模型：智能体 + 会话

- 智能体是唯一的标识符（例如 main、research）。Gateway 网关公开列表。
- 会话属于当前智能体。
- 会话键存储为 agent:<agentId>:<sessionKey>。
- 如果你输入 /session main，TUI 会将其扩展为 agent:<currentAgent>:main。
- 如果你输入 /session agent:other:main，你会显式切换到该智能体会话。

- 会话范围：
- per-sender（默认）：每个智能体有多个会话。
- global：TUI 始终使用 global 会话（选择器可能为空）。
- 当前智能体 + 会话始终在页脚中可见。

发送 + 投递

- 消息发送到 Gateway 网关；默认情况下不投递到提供商。
- 开启投递：
- /deliver on
- 或设置面板
- 或使用 openclaw tui --deliver 启动

选择器 + 覆盖层

- 模型选择器：列出可用模型并设置会话覆盖。
- 智能体选择器：选择不同的智能体。
- 会话选择器：仅显示当前智能体的会话。
- 设置：切换投递、工具输出展开和思考可见性。

键盘快捷键

- Enter：发送消息
- Esc：中止活动运行
- Ctrl+C：清除输入（按两次退出）
- Ctrl+D：退出
- Ctrl+L：模型选择器
- Ctrl+G：智能体选择器
- Ctrl+P：会话选择器
- Ctrl+O：切换工具输出展开
- Ctrl+T：切换思考可见性（重新加载历史）

斜杠命令

核心：

- /help
- /status
- /agent <id> (或 /agents)
- /session <key> (或 /sessions)
- /model <provider/model> (或 /models)

会话控制：

- /think <off|minimal|low|medium|high>
- /verbose <on|full|off>
- /reasoning <on|off|stream>
- /usage <off|tokens|full>
- /elevated <on|off|ask|full> (别名：/elev)
- /activation <mention|always>
- /deliver <on|off>

会话生命周期：

- /new 或 /reset (重置会话)
- /abort (中止活动运行)
- /settings
- /exit

其他 Gateway 网关斜杠命令 (例如 /context) 会转发到 Gateway 网关并显示为系统输出。参见斜杠命令。

本地 shell 命令

- 以 ! 为前缀的行会在 TUI 主机上运行本地 shell 命令。
- TUI 每个会话会提示一次以允许本地执行；拒绝会在该会话中禁用！。
- 命令在 TUI 工作目录中以全新的非交互式 shell 运行 (无持久化 cd/环境变量)。
- 单独的 ! 会作为普通消息发送；前导空格不会触发本地执行。

工具输出

- 工具调用显示为带有参数 + 结果的卡片。

- Ctrl+O 在折叠/展开视图之间切换。
- 工具运行时，部分更新会流式传输到同一张卡片。

历史 + 流式传输

- 连接时，TUI 加载最新历史（默认 200 条消息）。
- 流式响应原地更新直到完成。
- TUI 还监听智能体工具事件以获得更丰富的工具卡片。

连接详情

- TUI 以 mode: "tui" 向 Gateway 网关注册。
- 重新连接会显示系统消息；事件间隙会在日志中显示。

选项

- --url <url> : Gateway 网关 WebSocket URL（默认为配置或 ws://127.0.0.1:<port>）
- --token <token> : Gateway 网关令牌（如果需要）
- --password <password> : Gateway 网关密码（如果需要）
- --session <key> : 会话键（默认：main，或范围为全局时为 global）
- --deliver : 将助手回复投递到提供商（默认关闭）
- --thinking <level> : 覆盖发送的思考级别
- --timeout-ms <ms> : 智能体超时（毫秒）（默认为 agents.defaults.timeoutSeconds）

故障排除

发送消息后没有输出：

- 在 TUI 中运行 /status 以确认 Gateway 网关已连接且处于空闲/忙碌状态。
- 检查 Gateway 网关日志：openclaw logs --follow。
- 确认智能体可以运行：openclaw status 和 openclaw models status。
- 如果你期望消息出现在聊天渠道中，请启用投递（/deliver on 或 --deliver）。
- --history-limit <n> : 要加载的历史条目数（默认 200）

故障排除

- disconnected : 确保 Gateway 网关正在运行且你的 --url/--token/--password 正确。

- 选择器中没有智能体：检查 openclaw agents list 和你的路由配置。
- 会话选择器为空：你可能处于全局范围或还没有会话。

5. WebChat (Gateway 网关 WebSocket UI)

WebChat (Gateway 网关 WebSocket UI)

状态：macOS/iOS SwiftUI 聊天 UI 直接与 Gateway 网关 WebSocket 通信。

它是什么

- Gateway 网关的原生聊天 UI（无嵌入式浏览器，无本地静态服务器）。
- 使用与其他渠道相同的会话和路由规则。
- 确定性路由：回复始终返回到 WebChat。

快速开始

- 启动 Gateway 网关。
- 打开 WebChat UI (macOS/iOS 应用) 或控制 UI 聊天标签页。
- 确保已配置 Gateway 网关认证（默认需要，即使在 loopback 上）。

工作原理（行为）

- UI 连接到 Gateway 网关 WebSocket 并使用 chat.history、chat.send 和 chat.inject。
- chat.inject 直接将助手注释追加到转录并广播到 UI（无智能体运行）。
- 历史记录始终从 Gateway 网关获取（无本地文件监听）。
- 如果 Gateway 网关不可达，WebChat 为只读模式。

远程使用

- 远程模式通过 SSH/Tailscale 隧道传输 Gateway 网关 WebSocket。
- 你不需要运行单独的 WebChat 服务器。

配置参考 (WebChat)

完整配置：配置

渠道选项：

- 没有专用的 webchat. 块。WebChat 使用下面的 Gateway 网关端点 + 认证设置。

相关的全局选项：

- gateway.port 、 gateway.bind ： WebSocket 主机/端口。
- gateway.auth.mode 、 gateway.auth.token 、 gateway.auth.password ： WebSocket 认证。
- gateway.remote.url 、 gateway.remote.token 、 gateway.remote.password ： 远程 Gateway 网关目标。
- session. ： 会话存储和主键默认值。

第十章：平台专项说明

1. Android 应用（节点）

Android 应用（节点）

支持概览

- 角色：配套节点应用（Android 不托管 Gateway 网关）。
- 需要 Gateway 网关：是（在 macOS、Linux 或通过 WSL2 的 Windows 上运行）。
- 安装：入门指南 + 配对。
- Gateway 网关：操作手册 + 配置。
- 协议：Gateway 网关协议（节点 + 控制平面）。

系统控制

系统控制（launchd/systemd）位于 Gateway 网关主机上。参见 Gateway 网关。

连接操作手册

Android 节点应用（mDNS/NSD + WebSocket） Gateway 网关

Android 直接连接到 Gateway 网关 WebSocket（默认 ws://<host>:18789）并使用 Gateway 网关拥有的配对。

前置条件

- 你可以在"主"机器上运行 Gateway 网关。
- Android 设备/模拟器可以访问 Gateway 网关 WebSocket：
- 使用 mDNS/NSD 的同一局域网，或
- 使用 Wide-Area Bonjour / unicast DNS-SD 的同一 Tailscale tailnet（见下文），或
- 手动 Gateway 网关主机/端口（回退方案）
- 你可以在 Gateway 网关机器上运行 CLI（openclaw）（或通过 SSH）。

1) 启动 Gateway 网关

代码：openclaw gateway --port 18789 --verbose

在日志中确认你看到类似内容：

- listening on ws://0.0.0.0:18789

对于仅 tailnet 设置（推荐用于维也纳 伦敦），将 Gateway 网关绑定到 tailnet IP：

- 在 Gateway 网关主机的 ~/.openclaw/openclaw.json 中设置 gateway.bind: "tailnet"。
- 重启 Gateway 网关 / macOS 菜单栏应用。

2) 验证发现（可选）

从 Gateway 网关机器：

代码：`dns-sd -B _openclaw-gw._tcp local`。

更多调试说明：Bonjour。

通过 unicast DNS-SD 的 Tailnet（维也纳 伦敦）发现

Android NSD/mDNS 发现无法跨网络。如果你的 Android 节点和 Gateway 网关在不同网络但通过 Tailscale 连接，请改用 Wide-Area Bonjour / unicast DNS-SD：

- 在 Gateway 网关主机上设置 DNS-SD 区域（示例 openclaw.internal.）并发布 _openclaw-gw._tcp 记录。
- 配置 Tailscale split DNS，将你选择的域指向该 DNS 服务器。

详情和示例 CoreDNS 配置：Bonjour。

3) 从 Android 连接

在 Android 应用中：

- 应用通过前台服务（持久通知）保持 Gateway 网关连接活动。
- 打开设置。
- 在发现的 Gateway 网关下，选择你的 Gateway 网关并点击连接。
- 如果 mDNS 被阻止，使用高级 → 手动 Gateway 网关（主机 + 端口）并连接（手动）。

首次成功配对后，Android 在启动时自动重连：

- 手动端点（如果启用），否则
- 上次发现的 Gateway 网关（尽力而为）。

4) 批准配对（CLI）

在 Gateway 网关机器上：

代码：openclaw nodes pending

代码：openclaw nodes approve <requestId>

配对详情：Gateway 网关配对。

5) 验证节点已连接

- 通过节点状态：

代码：openclaw nodes status

- 通过 Gateway 网关：

代码：openclaw gateway call node.list --params "{}"

6) 聊天 + 历史

Android 节点的 Chat 面板使用 Gateway 网关的主会话键（main），因此历史和回复与 WebChat 和其他客户端共享：

- 历史：chat.history
- 发送：chat.send
- 推送更新（尽力而为）：chat.subscribe → event:"chat"

7) Canvas + 摄像头

Gateway 网关 Canvas 主机（推荐用于 web 内容）

如果你想让节点显示智能体可以在磁盘上编辑的真实 HTML/CSS/JS，请将节点指向 Gateway 网关 canvas 主机。

注意：节点使用 canvasHost.port（默认 18793）上的独立 canvas 主机。

- 在 Gateway 网关主机上创建 ~/.openclaw/workspace/canvas/index.html。
- 将节点导航到它（局域网）：

代码：openclaw nodes invoke --node "<Android Node>" --command canvas.navigate --params '{"url":"http://<gateway-hostname>.local:18793/__openclaw__/canvas/"}'

Tailnet（可选）：如果两个设备都在 Tailscale 上，使用 MagicDNS 名称或 tailnet IP 而不是 .local，例如`

此服务器将实时重载客户端注入 HTML 并在文件更改时重新加载。

A2UI 主机位于`

Canvas 命令（仅前台）：

- canvas.eval、canvas.snapshot、canvas.navigate（使用 {"url":""} 或 {"url":"/"} 返回默认脚手架）。canvas.snapshot 返回 { format, base64 }（默认 format="jpeg"）。
- A2UI：canvas.a2ui.push、canvas.a2ui.reset（canvas.a2ui.pushJSONL 遗留别名）

摄像头命令（仅前台；权限限制）：

- camera.snap (jpg)
- camera.clip (mp4)

参见 Camera 节点 了解参数和 CLI 助手。

2. 在 DigitalOcean 上运行 OpenClaw

在 DigitalOcean 上运行 OpenClaw

目标

以 \$6/月（或使用预留定价 \$4/月）在 DigitalOcean 上运行持久的 OpenClaw Gateway 网关。

如果你想要 \$0/月的选项且不介意 ARM + 特定提供商的设置，请参阅 Oracle Cloud 指南。

成本比较（2026）

提供商	方案	配置	价格/月	备注
Oracle Cloud	Always Free ARM	最高 4 OCPU、24GB RAM	\$0	ARM，容量有限 / 注册有坑
Hetzner	CX22	2 vCPU、4GB RAM	€3.79 (~\$4)	最便宜的付费选项
DigitalOcean	Basic	1 vCPU、1GB RAM	\$6	界面简单，文档完善
Vultr	Cloud Compute	1 vCPU、1GB RAM	\$6	多地区可选
Linode	Nanode	1 vCPU、1GB RAM	\$5	现为 Akamai 旗下

选择提供商：

- DigitalOcean：最简单的用户体验 + 可预测的设置（本指南）
- Hetzner：性价比高（参见 Hetzner 指南）
- Oracle Cloud：可以 \$0/月，但更麻烦且仅限 ARM（参见 Oracle 指南）

前提条件

- DigitalOcean 账户（注册可获 \$200 免费额度）
- SSH 密钥对（或愿意使用密码认证）
- 约 20 分钟

1) 创建 Droplet

- 登录 DigitalOcean
- 点击 Create → Droplets
- 选择：
- Region：离你（或你的用户）最近的地区
- Image：Ubuntu 24.04 LTS
- Size：Basic → Regular → \$6/mo（1 vCPU、1GB RAM、25GB SSD）
- Authentication：SSH 密钥（推荐）或密码
- 点击 Create Droplet
- 记下 IP 地址

2) 通过 SSH 连接

代码：ssh root@YOUR_DROPLET_IP

3) 安装 OpenClaw

代码：# Update system

代码：apt update && apt upgrade -y

代码：# Install Node.js 22

代码：curl -fsSL https://deb.nodesource.com/setup_22.x | bash -

代码：apt install -y nodejs

代码：# Install OpenClaw

代码：curl -fsSL https://openclaw.ai/install.sh | bash

代码：# Verify

代码：openclaw --version

4) 运行新手引导

代码：openclaw onboard --install-daemon

向导将引导你完成：

- 模型认证（API 密钥或 OAuth）
- 渠道设置（Telegram、WhatsApp、Discord 等）
- Gateway 网关令牌（自动生成）
- 守护进程安装（systemd）

5) 验证 Gateway 网关

代码：# Check status

代码：openclaw status

代码：# Check service

代码：systemctl --user status openclaw-gateway.service

代码：# View logs

代码：journalctl --user -u openclaw-gateway.service -f

6) 访问控制面板

Gateway 网关默认绑定到 loopback。要访问控制界面：

选项 A：SSH 隧道（推荐）

代码：# From your local machine

代码：ssh -L 18789:localhost:18789 root@YOUR_DROPLET_IP

代码：# Then open: http://localhost:18789

选项 B：Tailscale Serve（HTTPS，仅 loopback）

代码：# On the droplet

代码：curl -fsSL https://tailscale.com/install.sh | sh

代码：tailscale up

代码：# Configure Gateway to use Tailscale Serve

代码：openclaw config set gateway.tailscale.mode serve

代码：openclaw gateway restart

打开：`

注意事项：

- Serve 保持 Gateway 网关仅 loopback 并通过 Tailscale 身份头进行认证。
- 要改为需要令牌/密码，请设置 gateway.auth.allowTailscale: false 或使用 gateway.auth.mode: "password"。

选项 C：Tailnet 绑定（不使用 Serve）

代码：openclaw config set gateway.bind tailnet

代码：openclaw gateway restart

打开：`

7) 连接你的渠道

Telegram

代码：openclaw pairing list telegram

代码：openclaw pairing approve telegram <CODE>

WhatsApp

代码：openclaw channels login whatsapp

代码：# Scan QR code

参见渠道了解其他提供商。

1GB RAM 的优化

\$6 的 droplet 只有 1GB RAM。为了保持运行流畅：

添加 swap（推荐）

代码：fallocate -l 2G /swapfile

代码：chmod 600 /swapfile

代码：mkswap /swapfile

代码：swapon /swapfile

代码：echo '/swapfile none swap sw 0 0' >> /etc/fstab

使用更轻量的模型

如果遇到 OOM，考虑：

- 使用基于 API 的模型（Claude、GPT）而不是本地模型
- 将 `agents.defaults.model.primary` 设置为更小的模型

监控内存

代码：free -h

代码：htop

持久化

所有状态存储在：

- `~/.openclaw/` — 配置、凭证、会话数据
- `~/.openclaw/workspace/` — 工作区（SOUL.md、记忆等）

这些在重启后保留。定期备份：

代码：tar -czvf openclaw-backup.tar.gz ~/.openclaw ~/.openclaw/workspace

Oracle Cloud 免费替代方案

Oracle Cloud 提供 Always Free ARM 实例，比这里任何付费选项都强大得多 — 每月 \$0。

你将获得	配置
4 OCPUs	ARM Ampere A1
24GB RAM	绰绰有余
200GB 存储	块存储卷
永久免费	不收取信用卡费用

注意事项：

- 注册可能有点麻烦（失败了就重试）
- ARM 架构 — 大多数东西都能工作，但有些二进制文件需要 ARM 构建

完整设置指南请参阅 Oracle Cloud。关于注册技巧和注册流程故障排除，请参阅此社区指南。

故障排除

Gateway 网关无法启动

代码：openclaw gateway status
代码：openclaw doctor --non-interactive
代码：journalctl -u openclaw --no-pager -n 50

端口已被使用

代码：lsof -i:18789
代码：kill <PID>

内存不足

代码：# Check memory
代码：free -h

代码：# Add more swap
代码：# Or upgrade to \$12/mo droplet (2GB RAM)

另请参阅

- [Hetzner 指南](#) — 更便宜、更强大
- [Docker 安装](#) — 容器化设置
- [Tailscale](#) — 安全远程访问
- [配置](#) — 完整配置参考

3. 平台

平台

OpenClaw 核心使用 TypeScript 编写。Node 是推荐的运行时。

不推荐 Bun 用于 Gateway 网关（WhatsApp/Telegram 存在 bug）。

配套应用适用于 macOS（菜单栏应用）和移动节点（iOS/Android）。Windows 和 Linux 配套应用已在计划中，但 Gateway 网关目前已完全支持。

Windows 原生配套应用也在计划中；推荐通过 WSL2 使用 Gateway 网关。

选择你的操作系统

- macOS : macOS
- iOS : iOS
- Android : Android
- Windows : Windows
- Linux : Linux

VPS 和托管

- VPS 中心 : VPS 托管
- Fly.io : Fly.io
- Hetzner (Docker) : Hetzner
- GCP (Compute Engine) : GCP
- exe.dev (VM + HTTPS 代理) : exe.dev

常用链接

- [安装指南](#) : 入门指南

- Gateway 网关运行手册：Gateway 网关
- Gateway 网关配置：配置
- 服务状态：openclaw gateway status

Gateway 网关服务安装（CLI）

使用以下任一方式（均支持）：

- 向导（推荐）：openclaw onboard --install-daemon
- 直接安装：openclaw gateway install
- 配置流程：openclaw configure → 选择 Gateway service
- 修复/迁移：openclaw doctor（提供安装或修复服务）

服务目标取决于操作系统：

- macOS：LaunchAgent（bot.molt.gateway 或 bot.molt.<profile>；旧版 com.openclaw.）
- Linux/WSL2：systemd 用户服务（openclaw-gateway[-<profile>].service）

4. iOS 应用（节点）

iOS 应用（节点）

可用性：内部预览。iOS 应用尚未公开发发。

功能

- 通过 WebSocket（LAN 或 tailnet）连接到 Gateway 网关。
- 暴露节点能力：Canvas、屏幕快照、相机捕获、位置、对话模式、语音唤醒。
- 接收 node.invoke 命令并报告节点状态事件。

要求

- Gateway 网关运行在另一台设备上（macOS、Linux 或通过 WSL2 的 Windows）。
- 网络路径：
- 通过 Bonjour 的同一 LAN，或
- 通过单播 DNS-SD 的 Tailnet（示例域：openclaw.internal.），或
- 手动主机/端口（备选）。

快速开始（配对 + 连接）

- 启动 Gateway 网关：

代码：openclaw gateway --port 18789

- 在 iOS 应用中，打开设置并选择一个已发现的 Gateway 网关（或启用手动主机并输入主机/端口）。

- 在 Gateway 网关主机上批准配对请求：

代码：openclaw nodes pending

代码：openclaw nodes approve <requestId>

- 验证连接：

代码：openclaw nodes status

代码：openclaw gateway call node.list --params "{}"

发现路径

Bonjour (LAN)

Gateway 网关在 local. 上广播 _openclaw-gw._tcp。iOS 应用会自动列出这些。

Tailnet (跨网络)

如果 mDNS 被阻止，使用单播 DNS-SD 区域（选择一个域；示例：openclaw.internal.）和 Tailscale 分割 DNS。

参见 Bonjour 了解 CoreDNS 示例。

手动主机/端口

在设置中，启用手动主机并输入 Gateway 网关主机 + 端口（默认 18789）。

Canvas + A2UI

iOS 节点渲染一个 WKWebView canvas。使用 node.invoke 来驱动它：

代码：openclaw nodes invoke --node "iOS Node" --command canvas.navigate --params '{"url":"http://<gateway-host>:18793/__openclaw__/canvas/"}'

注意事项：

- Gateway 网关 canvas 主机服务于 /openclaw/canvas/ 和 /openclaw/a2ui/。
- 当广播了 canvas 主机 URL 时，iOS 节点在连接时自动导航到 A2UI。
- 使用 canvas.navigate 和 {"url":""} 返回内置脚手架。

Canvas eval / snapshot

代码：openclaw nodes invoke --node "iOS Node" --command canvas.eval --params
 '{"javascript":'+"(() => { const {ctx} = window.__openclaw;
 ctx.clearRect(0,0,innerWidth,innerHeight); ctx.lineWidth=6; ctx.strokeStyle=\"#ff2d55\";
 ctx.beginPath(); ctx.moveTo(40,40); ctx.lineTo(innerWidth-40, innerHeight-40); ctx.stroke();
 return \"ok\"; })()}'

代码：openclaw nodes invoke --node "iOS Node" --command canvas.snapshot --params
 '{"maxWidth":900,"format":"jpeg"}'

语音唤醒 + 对话模式

- 语音唤醒和对话模式在设置中可用。
- iOS 可能会暂停后台音频；当应用不活跃时，将语音功能视为尽力而为。

常见错误

- NODE_BACKGROUND_UNAVAILABLE：将 iOS 应用带到前台（canvas/相机/屏幕命令需要它）。
- A2UI_HOST_NOT_CONFIGURED：Gateway 网关未广播 canvas 主机 URL；检查 Gateway 网关配置 中的 canvasHost。
- 配对提示从未出现：运行 openclaw nodes pending 并手动批准。
- 重新安装后重连失败：钥匙串配对令牌已被清除；重新配对节点。

相关文档

- 配对
- 设备发现
- Bonjour

5. Linux 应用

Linux 应用

Gateway 网关在 Linux 上完全支持。Node 是推荐的运行时。

不推荐 Bun 用于 Gateway 网关（WhatsApp/Telegram 存在 bug）。

原生 Linux 配套应用已在计划中。如果你想帮助构建，欢迎贡献。

新手快速路径（VPS）

- 安装 Node 22+
- `npm i -g openclaw@latest`
- `openclaw onboard --install-daemon`
- 从你的笔记本电脑：`ssh -N -L 18789:127.0.0.1:18789 <user>@<host>`
- 打开` 并粘贴你的令牌

分步 VPS 指南：exe.dev

安装

- 入门指南
- 安装与更新
- 可选流程：Bun（实验性）、Nix、Docker

Gateway 网关

- Gateway 网关运行手册
- 配置

Gateway 网关服务安装（CLI）

使用以下任一方式：

代码：`openclaw onboard --install-daemon`

或：

代码：`openclaw gateway install`

或：

代码：`openclaw configure`

出现提示时选择 Gateway service。

修复/迁移：

代码：openclaw doctor

系统控制（systemd 用户单元）

OpenClaw 默认安装 systemd 用户服务。对于共享或常驻服务器使用系统服务。完整的单元示例和指南在 Gateway 网关运行手册 中。

最小设置：

创建 ~/.config/systemd/user/openclaw-gateway[-<profile>].service：

代码：[Unit]

代码：Description=OpenClaw Gateway (profile: <profile>, v<version>)

代码：After=network-online.target

代码：Wants=network-online.target

代码：[Service]

代码：ExecStart=/usr/local/bin/openclaw gateway --port 18789

代码：Restart=always

代码：RestartSec=5

代码：[Install]

代码：WantedBy=default.target

启用它：

代码：systemctl --user enable --now openclaw-gateway[-<profile>].service

6. macOS 上的 Gateway 网关（外部 launchd）

macOS 上的 Gateway 网关（外部 launchd）

OpenClaw.app 不再捆绑 Node/Bun 或 Gateway 网关运行时。macOS 应用期望有一个外部的 openclaw CLI 安装，不会将 Gateway 网关作为子进程启动，而是管理一个每用户的 launchd 服务来保持 Gateway 网关运行（或者如果已有本地 Gateway 网关正在运行，则连接到现有的）。

安装 CLI（本地模式必需）

你需要在 Mac 上安装 Node 22+，然后全局安装 openclaw：

代码：npm install -g openclaw@<version>

macOS 应用的安装 CLI按钮通过 npm/pnpm 运行相同的流程（不推荐使用 bun 作为 Gateway 网关运行时）。

Launchd（Gateway 网关作为 LaunchAgent）

标签：

- bot.molt.gateway（或 bot.molt.<profile>；旧版 com.openclaw. 可能仍然存在）

Plist 位置（每用户）：

- ~/Library/LaunchAgents/bot.molt.gateway.plist
（或 ~/Library/LaunchAgents/bot.molt.<profile>.plist）

管理者：

- macOS 应用在本地图式下拥有 LaunchAgent 的安装/更新权限。
- CLI 也可以安装它：openclaw gateway install。

行为：

- "OpenClaw Active"启用/禁用 LaunchAgent。
- 应用退出不会停止 Gateway 网关（launchd 保持其存活）。
- 如果 Gateway 网关已经在配置的端口上运行，应用会连接到它而不是启动新的。

日志：

- launchd stdout/err：/tmp/openclaw/openclaw-gateway.log

版本兼容性

macOS 应用会检查 Gateway 网关版本与其自身版本是否匹配。如果不兼容，请更新全局 CLI 以匹配应用版本。

冒烟测试

代码：openclaw --version

代码：OPENCLAW_SKIP_CHANNELS=1 \

代码：OPENCLAW_SKIP_CANVAS_HOST=1 \

代码：openclaw gateway --port 18999 --bind loopback

然后：

代码：openclaw gateway call health --url ws://127.0.0.1:18999 --timeout 3000

7. Canvas (macOS 应用)

Canvas (macOS 应用)

macOS 应用使用 WKWebView 嵌入一个智能体控制的 Canvas 面板。它是一个用于 HTML/CSS/JS、A2UI 和小型交互式界面的轻量级可视化工作区。

Canvas 存储位置

Canvas 状态存储在 Application Support 下：

- ~/Library/Application Support/OpenClaw/canvas/<session>/...

Canvas 面板通过自定义 URL 方案提供这些文件：

- openclaw-canvas://<session>/<path>

示例：

- openclaw-canvas://main/ → <canvasRoot>/main/index.html
- openclaw-canvas://main/assets/app.css → <canvasRoot>/main/assets/app.css
- openclaw-canvas://main/widgets/todo/ → <canvasRoot>/main/widgets/todo/index.html

如果根目录下没有 index.html，应用会显示一个内置脚手架页面。

面板行为

- 无边框、可调整大小的面板，锚定在菜单栏（或鼠标光标）附近。
- 记住每个会话的大小/位置。
- 当本地 canvas 文件更改时自动重新加载。
- 一次只显示一个 Canvas 面板（根据需要切换会话）。

可以从设置 → 允许 Canvas 禁用 Canvas。禁用时，canvas 节点命令返回 CANVAS_DISABLED。

智能体 API 接口

Canvas 通过 Gateway 网关 WebSocket 暴露，因此智能体可以：

- 显示/隐藏面板
- 导航到路径或 URL
- 执行 JavaScript
- 捕获快照图像

CLI 示例：

代码：openclaw nodes canvas present --node <id>

代码：openclaw nodes canvas navigate --node <id> --url "/"

代码：openclaw nodes canvas eval --node <id> --js "document.title"

代码：openclaw nodes canvas snapshot --node <id>

注意事项：

- canvas.navigate 接受本地 canvas 路径、http(s) URL 和 file:// URL。
- 如果传递 "/"，Canvas 会显示本地脚手架或 index.html。

Canvas 中的 A2UI

A2UI 由 Gateway 网关 canvas 主机托管并在 Canvas 面板内渲染。

当 Gateway 网关广播 Canvas 主机时，macOS 应用在首次打开时自动导航到 A2UI 主机页面。

默认 A2UI 主机 URL：

代码：http://<gateway-host>:18793/__openclaw__/a2ui/

A2UI 命令 (v0.8)

Canvas 目前接受 A2UI v0.8 服务器→客户端消息：

- beginRendering
- surfaceUpdate
- dataModelUpdate

- deleteSurface

createSurface (v0.9) 不受支持。

CLI 示例：

代码：cat > /tmp/a2ui-v0.8.jsonl <<'EOFA2'

代码：{"surfaceUpdate":{"surfaceId":"main","components":[{"id":"root","component":{"Column":{"children":{"explicitList":["title","content"]}}},{id":"title","component":{"Text":{"text":{"literalString":"Canvas (A2UI v0.8)"},"usageHint":"h1"}}},{id":"content","component":{"Text":{"text":{"literalString":"If you can read this, A2UI push works."},"usageHint":"body"}}}]}}

代码：{"beginRendering":{"surfaceId":"main","root":"root"}}

代码：EOFA2

代码：openclaw nodes canvas a2ui push --jsonl /tmp/a2ui-v0.8.jsonl --node <id>

快速测试：

代码：openclaw nodes canvas a2ui push --node <id> --text "Hello from A2UI"

从 Canvas 触发智能体运行

Canvas 可以通过深层链接触发新的智能体运行：

- openclaw://agent?...

示例（在 JS 中）：

代码：window.location.href = "openclaw://agent?message=Review%20this%20design";

除非提供有效密钥，否则应用会提示确认。

安全注意事项

- Canvas 方案阻止目录遍历；文件必须位于会话根目录下。
- 本地 Canvas 内容使用自定义方案（不需要 loopback 服务器）。
- 仅在显式导航时允许外部 http(s) URL。

8. macOS 上的 Gateway 网关生命周期

macOS 上的 Gateway 网关生命周期

macOS 应用默认通过 launchd 管理 Gateway 网关，不会将 Gateway 网关作为子进程生成。它首先尝试连接到配置端口上已运行的 Gateway 网关；如果无法访问，它会通过外部 openclaw CLI（无嵌入式运行时）启用 launchd 服务。这为你提供了可靠的登录时自动启动和崩溃后重启。

子进程模式（由应用直接生成 Gateway 网关）目前未使用。

如果你需要与 UI 更紧密的耦合，请在终端中手动运行 Gateway 网关。

默认行为 (launchd)

- 应用安装标记为 bot.molt.gateway 的按用户 LaunchAgent（使用 --profile/OPENCLAW_PROFILE 时为 bot.molt.<profile>；支持旧版 com.openclaw.）。
- 当启用本地模式时，应用确保 LaunchAgent 已加载，并在需要时启动 Gateway 网关。
- 日志写入 launchd Gateway 网关日志路径（在调试设置中可见）。

常用命令：

代码：launchctl kickstart -k gui/\$UID/bot.molt.gateway

代码：launchctl bootout gui/\$UID/bot.molt.gateway

运行命名配置文件时，将标签替换为 bot.molt.<profile>。

未签名的开发构建

scripts/restart-mac.sh --no-sign 用于在没有签名密钥时的快速本地构建。为了防止 launchd 指向未签名的中继二进制文件，它：

- 写入 ~/.openclaw/disable-launchagent。

已签名运行的 scripts/restart-mac.sh 会在标记存在时清除此覆盖。要手动重置：

代码：rm ~/.openclaw/disable-launchagent

仅连接模式

要强制 macOS 应用永不安装或管理 launchd，请使用

--attach-only（或 --no-launchd）启动它。这会设置 ~/.openclaw/disable-launchagent，

因此应用只会连接到已运行的 Gateway 网关。你可以在调试设置中切换相同的行为。

远程模式

远程模式永远不会启动本地 Gateway 网关。应用使用到远程主机的 SSH 隧道并通过该隧道连接。

为什么我们更喜欢 launchd

- 登录时自动启动。
- 内置的重启/KeepAlive 语义。
- 可预测的日志和监管。

如果将来再次需要真正的子进程模式，它应该被记录为单独的、明确的仅开发模式。

9. macOS 开发者设置

macOS 开发者设置

本指南涵盖从源代码构建和运行 OpenClaw macOS 应用程序的必要步骤。

前置条件

在构建应用之前，确保你已安装以下内容：

- Xcode 26.2+：Swift 开发所需。
- Node.js 22+ & pnpm：Gateway 网关、CLI 和打包脚本所需。

1. 安装依赖

安装项目范围的依赖：

代码：`pnpm install`

2. 构建和打包应用

要构建 macOS 应用并将其打包到 `dist/OpenClaw.app`，运行：

代码：`./scripts/package-mac-app.sh`

如果你没有 Apple Developer ID 证书，脚本将自动使用 ad-hoc 签名 (-) 。

有关开发运行模式、签名标志和 Team ID 故障排除，请参阅 macOS 应用 README：

注意：Ad-hoc 签名的应用可能会触发安全提示。如果应用立即崩溃并显示"Abort trap 6"，请参阅故障排除部分。

3. 安装 CLI

macOS 应用期望全局安装 openclaw CLI 来管理后台任务。

安装方法（推荐）：

- 打开 OpenClaw 应用。
- 转到 General 设置标签页。
- 点击 "Install CLI"。

或者，手动安装：

代码：`npm install -g openclaw@<version>`

故障排除

构建失败：工具链或 SDK 不匹配

macOS 应用构建期望最新的 macOS SDK 和 Swift 6.2 工具链。

系统依赖（必需）：

- 软件更新中可用的最新 macOS 版本（Xcode 26.2 SDK 所需）
- Xcode 26.2（Swift 6.2 工具链）

检查：

代码：`xcodebuild -version`

代码：`xcrun swift --version`

如果版本不匹配，更新 macOS/Xcode 并重新运行构建。

授予权限时应用崩溃

如果在尝试允许语音识别或麦克风访问时应用崩溃，可能是由于 TCC 缓存损坏或签名不匹配。

修复：

- 重置 TCC 权限：

代码： `tccutil reset All bot.molt.mac.debug`

- 如果这不起作用，在 `scripts/package-mac-app.sh` 中临时更改 `BUNDLE_ID` 以强制 macOS 从"全新状态"开始。

Gateway 网关无限期"Starting..."

如果 Gateway 网关状态一直停留在"Starting..."，检查是否有僵尸进程占用端口：

代码： `openclaw gateway status`

代码： `openclaw gateway stop`

代码：# 如果你没有使用 LaunchAgent（开发模式/手动运行），找到监听器：

代码： `lsof -nP -iTCP:18789 -sTCP:LISTEN`

如果手动运行占用了端口，停止该进程（Ctrl+C）。作为最后手段，杀死你找到的 PID。

10. macOS 上的健康检查

macOS 上的健康检查

如何从菜单栏应用查看关联渠道是否健康。

菜单栏

- 状态圆点现在反映 Baileys 健康状态：
- 绿色：已关联 + socket 最近已打开。
- 橙色：正在连接/重试。
- 红色：已登出或探测失败。
- 第二行显示"linked · auth 12m"或显示失败原因。
- "Run Health Check"菜单项触发按需探测。

设置

- 通用选项卡新增健康卡片，显示：关联认证时间、会话存储路径/数量、上次检查时间、上次错误/状态码，以及运行健康检查/显示日志按钮。
- 使用缓存快照，因此 UI 立即加载，离线时优雅降级。

- 渠道选项卡显示渠道状态 + WhatsApp/Telegram 的控制（登录二维码、登出、探测、上次断开/错误）。

探测工作原理

- 应用每约 60 秒和按需时通过 ShellExecutor 运行 `openclaw health --json`。探测加载凭证并报告状态，不发送消息。
- 分别缓存上次成功的快照和上次错误以避免闪烁；显示每个的时间戳。

有疑问时

- 你仍然可以使用 Gateway 网关健康 中的 CLI 流程（`openclaw status`、`openclaw status --deep`、`openclaw health --json`），并在 `/tmp/openclaw/openclaw-.log` 中跟踪 `web-heartbeat` / `web-reconnect`。

11. 菜单栏图标状态

菜单栏图标状态

作者：steipete · 更新时间：2025-12-06 · 范围：macOS 应用（apps/macos）

- 空闲：正常图标动画（眨眼、偶尔摆动）。
- 暂停：状态项使用 `appearsDisabled`；无动画。
- 语音触发（大耳朵）：语音唤醒检测器在听到唤醒词时调用 `AppState.triggerVoiceEars(ttl: nil)`，在捕获语音期间保持 `earBoostActive=true`。耳朵放大（1.9 倍），显示圆形耳孔以提高可读性，然后在 1 秒静音后通过 `stopVoiceEars()` 恢复。仅由应用内语音管道触发。
- 工作中（智能体运行中）：`AppState.isWorking=true`
驱动"尾巴/腿部快速摆动"微动画：工作进行中腿部摆动加快并略有偏移。目前在 WebChat 智能体运行时切换；在接入其他长时间任务时请添加相同的切换逻辑。

接入点

- 语音唤醒：运行时/测试器在触发时调用 `AppState.triggerVoiceEars(ttl: nil)`，在 1 秒静音后调用 `stopVoiceEars()` 以匹配捕获窗口。
- 智能体活动：在工作区间前后设置 `AppStateStore.shared.setWorking(true/false)`（已在 WebChat 智能体调用中完成）。保持区间简短，并在 `defer` 块中重置以避免动画卡住。

形状与尺寸

- 基础图标在

CritterIconRenderer.makeIcon(blink:legWiggle:earWiggle:earScale:earHoles:) 中绘制。

- 耳朵缩放默认为 1.0；语音增强时设置 earScale=1.9 并切换 earHoles=true，不改变整体框架（18×18 pt 模板图像渲染到 36×36 px Retina 后备存储）。
- 快速摆动使用最高约 1.0 的腿部摆幅并带有轻微的水平抖动；它与现有的空闲摆动叠加。

行为说明

- 耳朵/工作状态没有外部 CLI/代理切换；保持仅由应用自身信号控制，以避免意外的状态抖动。
- 保持 TTL 较短（<10 秒），以便在任务挂起时图标能快速恢复到基准状态。

12. 日志 (macOS)

日志 (macOS)

滚动诊断文件日志 (Debug 面板)

OpenClaw 通过 swift-log（默认使用统一日志）路由 macOS 应用日志，并且在需要持久化捕获时可以将本地轮转文件日志写入磁盘。

- 详细级别：Debug 面板 → Logs → App logging → Verbosity
- 启用：Debug 面板 → Logs → App logging → "Write rolling diagnostics log (JSONL)"
- 位置：~/Library/Logs/OpenClaw/diagnostics.jsonl（自动轮转；旧文件以 .1、.2、... 为后缀）
- 清除：Debug 面板 → Logs → App logging → "Clear"

注意事项：

- 此功能默认关闭。仅在主动调试时启用。
- 该文件包含敏感信息；分享前请先审查内容。

macOS 上统一日志的隐私数据

统一日志会屏蔽大部分负载内容，除非子系统选择启用 privacy-off。根据 Peter 关于 macOS 日志隐私机制（2025）的文章，这通过 /Library/Preferences/Logging/Subsystems/ 中以子系统名称为键的 plist 文件来控制。只有新的日志条目才会应用该标志，因此请在复现问题之前启用它。

为 OpenClaw 启用（`bot.molt`）

- 先将 plist 写入临时文件，然后以 root 身份原子性地安装：

代码：cat <<'EOF' >/tmp/bot.molt.plist

代码：<?xml version="1.0" encoding="UTF-8"?>

代码：<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">

代码：<plist version="1.0">

代码：<dict>

代码：<key>DEFAULT-OPTIONS</key>

代码：<dict>

代码：<key>Enable-Private-Data</key>

代码：<true/>

代码：</dict>

代码：</dict>

代码：</plist>

代码：EOF

代码：sudo install -m 644 -o root -g wheel /tmp/bot.molt.plist
/Library/Preferences/Logging/Subsystems/bot.molt.plist

- 无需重启；logd 会很快检测到该文件，但只有新的日志行才会包含隐私负载。
- 使用现有的辅助脚本查看更丰富的输出，例如 ./scripts/clawlog.sh --category WebChat --last 5m。

调试后禁用

- 移除覆盖配置：sudo rm /Library/Preferences/Logging/Subsystems/bot.molt.plist。
- 可选择运行 sudo log config --reload 强制 logd 立即丢弃覆盖配置。
- 请注意此数据可能包含电话号码和消息正文；仅在确实需要额外详细信息时才保留该 plist 文件。

13. 菜单栏状态逻辑

菜单栏状态逻辑

显示内容

- 我们在菜单栏图标和菜单的第一行状态行中展示当前智能体的工作状态。
- 工作活跃时隐藏健康状态；当所有会话空闲时恢复显示。
- 菜单中的"节点"区块仅列出设备（通过 node.list 配对的节点），不包括客户端/在线状态条目。
- 当提供商用量快照可用时，"用量"部分会显示在上下文下方。

状态模型

- 会话：事件携带 runId（每次运行）以及载荷中的 sessionKey。"main" 会话的键为 main；如果不存在，则回退到最近更新的会话。
- 优先级：main 始终优先。如果 main 处于活跃状态，立即显示其状态。如果 main 空闲，则显示最近活跃的非 main 会话。活动进行中不会来回切换；仅在当前会话进入空闲或 main 变为活跃时才切换。
- 活动类型：
 - job：高层命令执行（state: started|streaming|done|error）。
 - tool：phase: start|result，包含 toolName 和 meta/args。

IconState 枚举 (Swift)

- idle
- workingMain(ActivityKind)
- workingOther(ActivityKind)
- overridden(ActivityKind)（调试覆盖）

ActivityKind → 图标符号

- exec →
- read →
- write →
- edit →
- attach →
- 默认 →

视觉映射

- idle：正常小动物图标。
- workingMain：带图标符号的徽章，完整色调，腿部"工作"动画。

- workingOther：带图标符号的徽章，柔和色调，无快跑动画。
- overridden：无论活动状态如何，使用所选的图标符号/色调。

状态行文本（菜单）

- 工作活跃时：<会话角色> · <活动标签>
- 示例：Main · exec: npm test、Other · read: apps/macos/Sources/OpenClaw/AppState.swift。
- 空闲时：回退显示健康摘要。

事件接收

- 来源：控制渠道 agent 事件（ControlChannel.handleAgentEvent）。
- 解析字段：
- stream: "job"，包含 data.state 用于启动/停止。
- stream: "tool"，包含 data.phase、name，可选 meta/args。
- 标签：
- exec：args.command 的第一行。
- read/write：缩短的路径。
- edit：路径加上从 meta/diff 计数推断的变更类型。
- 回退：工具名称。

调试覆盖

- 设置 调试 "图标覆盖" 选择器：
- 系统（自动）（默认）
- 工作中：main（按工具类型）
- 工作中：other（按工具类型）
- 空闲
- 通过 @AppStorage("iconOverride") 存储；映射到 IconState.overridden。

测试清单

- 触发 main 会话任务：验证图标立即切换且状态行显示 main 标签。
- main 空闲时触发非 main 会话任务：图标/状态显示非 main；保持稳定直到完成。

- 在 other 活跃时启动 main：图标立即切换到 main。
- 快速连续工具调用：确保徽章不会闪烁（工具结果的 TTL 宽限期）。
- 所有会话空闲后健康行重新出现。

14. Peekaboo Bridge (macOS UI 自动化)

Peekaboo Bridge (macOS UI 自动化)

OpenClaw 可以将 PeekabooBridge 作为本地的、权限感知的 UI 自动化代理进行托管。这使得 peekaboo CLI 能够驱动 UI 自动化，同时复用 macOS 应用的 TCC 权限。

这是什么（以及不是什么）

- 宿主：OpenClaw.app 可以作为 PeekabooBridge 宿主。
- 客户端：使用 peekaboo CLI（无需单独的 openclaw ui ... 界面）。
- 界面：视觉叠加层保留在 Peekaboo.app 中；OpenClaw 只是一个轻量代理宿主。

启用桥接

在 macOS 应用中：

- 设置 → 启用 Peekaboo Bridge

启用后，OpenClaw 会启动一个本地 UNIX 套接字服务器。如果禁用，宿主会停止，peekaboo 将回退到其他可用宿主。

客户端发现顺序

Peekaboo 客户端通常按以下顺序尝试宿主：

- Peekaboo.app（完整用户体验）
- Claude.app（如已安装）
- OpenClaw.app（轻量代理）

使用 peekaboo bridge status --verbose

查看当前活跃的宿主及使用的套接字路径。你可以通过以下方式覆盖：

安全与权限

- 桥接会验证调用方的代码签名；强制执行 TeamID 白名单（Peekaboo 宿主 TeamID + OpenClaw 应用 TeamID）。

- 请求在约 10 秒后超时。
- 如果缺少所需权限，桥接会返回清晰的错误信息，而不是启动系统设置。

快照行为（自动化）

快照存储在内存中，并在短暂窗口期后自动过期。如果需要更长的保留时间，请从客户端重新捕获。

故障排除

- 如果 peekaboo 报告"bridge client is not authorized"，请确保客户端已正确签名，或仅在调试模式下使用 PEEKABOO_ALLOW_UNSIGNED_SOCKET_CLIENTS=1 运行宿主。
- 如果未找到宿主，请打开其中一个宿主应用（Peekaboo.app 或 OpenClaw.app）并确认已授予权限。

15. macOS 权限（TCC）

macOS 权限（TCC）

macOS 权限授予是脆弱的。TCC 将权限授予与应用的代码签名、Bundle 标识符和磁盘路径关联。如果其中任何一项发生变化，macOS 会将该应用视为新应用，可能会丢弃或隐藏权限提示。

稳定权限的要求

- 相同路径：从固定位置运行应用（对于 OpenClaw，为 dist/OpenClaw.app）。
- 相同 Bundle 标识符：更改 Bundle ID 会创建新的权限身份。
- 已签名的应用：未签名或临时签名的构建不会持久化权限。
- 一致的签名：使用真实的 Apple Development 或 Developer ID 证书，以确保签名在多次构建之间保持稳定。

临时签名每次构建都会生成新的身份。macOS 会忘记之前的授权，提示可能完全消失，直到清除过期条目为止。

权限提示消失时的恢复清单

- 退出应用。
- 在系统设置 -> 隐私与安全性中移除该应用条目。
- 从相同路径重新启动应用并重新授予权限。

- 如果提示仍未出现，使用 `tccutil` 重置 TCC 条目后重试。
- 某些权限仅在完全重启 macOS 后才会重新出现。

重置示例（根据需要替换 Bundle ID）：

代码：`sudo tccutil reset Accessibility bot.molt.mac`

代码：`sudo tccutil reset ScreenCapture bot.molt.mac`

代码：`sudo tccutil reset AppleEvents`

如果你正在测试权限，请始终使用真实证书签名。临时签名的构建仅适用于不需要权限的快速本地运行。

16. OpenClaw macOS 发布 (Sparkle)

OpenClaw macOS 发布 (Sparkle)

本应用现已支持 Sparkle 自动更新。发布构建必须经过 Developer ID 签名、压缩，并发布包含签名的 `appcast` 条目。

前提条件

- 已安装 Developer ID Application 证书（示例：Developer ID Application: <Developer Name> (<TEAMID>))。
- 环境变量 `SPARKLE_PRIVATE_KEY_FILE` 已设置为 Sparkle `ed25519` 私钥路径（公钥已嵌入 `Info.plist`）。如果缺失，请检查 `~/profile`。
- 用于 `xcrun notarytool` 的公证凭据（钥匙串配置文件或 API 密钥），以实现通过 Gatekeeper 安全分发的 DMG/zip。
- 我们使用名为 `openclaw-notary` 的钥匙串配置文件，由 `shell` 配置文件中的 App Store Connect API 密钥环境变量创建：
- `APP_STORE_CONNECT_API_KEY_P8`、`APP_STORE_CONNECT_KEY_ID`、`APP_STORE_CONNECT_ISSUER_ID`
- `echo "$APP_STORE_CONNECT_API_KEY_P8" | sed 's/\n/g' > /tmp/openclaw-notary.p8`
- `xcrun notarytool store-credentials "openclaw-notary" --key /tmp/openclaw-notary.p8 --key-id "$APP_STORE_CONNECT_KEY_ID" --issuer "$APP_STORE_CONNECT_ISSUER_ID"`
- 已安装 `pnpm` 依赖（`pnpm install --config.node-linker=hoisted`）。
- Sparkle 工具通过 SwiftPM 自动获取，位于 `apps/macos/.build/artifacts/sparkle/Sparkle/bin/`（`sign_update`、`generate_appcast`）。

等)。

构建与打包

注意事项：

- APP_BUILD 映射到 CFBundleVersion/sparkle:version；保持纯数字且单调递增（不含 -beta），否则 Sparkle 会将其视为相同版本。
- 默认为当前架构 (\$(uname -m))。对于发布/通用构建，设置 BUILD_ARCHS="arm64 x86_64"（或 BUILD_ARCHS=all）。
- 使用 scripts/package-mac-dist.sh 生成发布产物（zip + DMG + 公证）。使用 scripts/package-mac-app.sh 进行本地/开发打包。

代码：# 从仓库根目录运行；设置发布 ID 以启用 Sparkle 订阅源。

代码：# APP_BUILD 必须为纯数字且单调递增，以便 Sparkle 正确比较。

代码：BUNDLE_ID=bot.molt.mac \

代码：APP_VERSION=2026.1.27-beta.1 \

代码：APP_BUILD="\$(git rev-list --count HEAD)" \

代码：BUILD_CONFIG=release \

代码：SIGN_IDENTITY="Developer ID Application: <Developer Name> (<TEAMID>)" \

代码：scripts/package-mac-app.sh

代码：# 打包用于分发的 zip（包含资源分支以支持 Sparkle 增量更新）

代码：ditto -c -k --sequesterRsrc --keepParent dist/OpenClaw.app
dist/OpenClaw-2026.1.27-beta.1.zip

代码：# 可选：同时构建适合用户使用的样式化 DMG（拖拽到 /Applications）

代码：scripts/create-dmg.sh dist/OpenClaw.app dist/OpenClaw-2026.1.27-beta.1.dmg

代码：# 推荐：构建 + 公证/装订 zip + DMG

代码：# 首先，创建一次钥匙串配置文件：

代码：# xcrun notarytool store-credentials "openclaw-notary" \

代码：# --apple-id "<apple-id>" --team-id "<team-id>" --password
"<app-specific-password>"

代码：NOTARIZE=1 NOTARYTOOL_PROFILE=openclaw-notary \

代码：BUNDLE_ID=bot.molt.mac \

代码：APP_VERSION=2026.1.27-beta.1 \

代码：APP_BUILD="\$(git rev-list --count HEAD)" \

代码：BUILD_CONFIG=release \

代码：SIGN_IDENTITY="Developer ID Application: <Developer Name> (<TEAMID>)" \

代码：scripts/package-mac-dist.sh

代码：# 可选：随发布一起提供 dSYM

代码：ditto -c -k --keepParent apps/macos/.build/release/OpenClaw.app.dSYM
dist/OpenClaw-2026.1.27-beta.1.dSYM.zip

Appcast 条目

使用发布说明生成器，以便 Sparkle 渲染格式化的 HTML 说明：

代码：SPARKLE_PRIVATE_KEY_FILE=/path/to/ed25519-private-key
scripts/make_appcast.sh dist/OpenClaw-2026.1.27-beta.1.zip
<https://raw.githubusercontent.com/openclaw/openclaw/main/appcast.xml>

从 CHANGELOG.md（通过 scripts/changelog-to-html.sh）生成 HTML 发布说明，并将其嵌入 appcast 条目。

发布时，将更新后的 appcast.xml 与发布资源（zip + dSYM）一起提交。

发布与验证

- 将 OpenClaw-2026.1.27-beta.1.zip（和 OpenClaw-2026.1.27-beta.1.dSYM.zip）上传到标签 v2026.1.27-beta.1 对应的 GitHub 发布。
- 确保原始 appcast URL 与内置的订阅源匹配：
- 完整性检查：
- ``curl -I`` 返回 200。
- `curl -I <enclosure url>` 在资源上传后返回 200。
- 在之前的公开构建版本上，从 About 选项卡运行"Check for Updates..."，验证 Sparkle 能正常安装新构建。

完成定义：已签名的应用 + appcast

已发布，从旧版本的更新流程正常工作，且发布资源已附加到 GitHub 发布。

17. 远程 OpenClaw（macOS 远程主机）

远程 OpenClaw (macOS 远程主机)

此流程让 macOS 应用作为运行在另一台主机（桌面/服务器）上的 OpenClaw Gateway 网关的完整远程控制。这是应用的 Remote over

SSH（远程运行）功能。所有功能——健康检查、语音唤醒转发和 Web Chat——都重用来自 `_Settings → General_` 的相同远程 SSH 配置。

模式

- Local (this Mac)：一切都在笔记本电脑上运行。不涉及 SSH。
- Remote over SSH（默认）：OpenClaw 命令在远程主机上执行。mac 应用使用 `-o BatchMode` 加上你选择的身份/密钥打开 SSH 连接，并进行本地端口转发。
- Remote direct (ws/wss)：无 SSH 隧道。mac 应用直接连接到 Gateway 网关 URL（例如，通过 Tailscale Serve 或公共 HTTPS 反向代理）。

远程传输

远程模式支持两种传输方式：

- SSH 隧道（默认）：使用 `ssh -N -L ...` 将 Gateway 网关端口转发到 localhost。Gateway 网关会将节点的 IP 视为 127.0.0.1，因为隧道是 loopback。
- Direct (ws/wss)：直接连接到 Gateway 网关 URL。Gateway 网关看到真实的客户端 IP。

远程主机上的先决条件

- 安装 Node + pnpm 并构建/安装 OpenClaw CLI (`pnpm install && pnpm build && pnpm link --global`)。
- 确保 openclaw 在非交互式 shell 的 PATH 中（如需要，请符号链接到 `/usr/local/bin` 或 `/opt/homebrew/bin`）。
- 使用密钥认证打开 SSH。我们推荐使用 Tailscale IP 以实现离开局域网时的稳定可达性。

macOS 应用设置

- 打开 `_Settings → General_`。
- 在 OpenClaw runs 下，选择 Remote over SSH 并设置：
- Transport：SSH tunnel 或 Direct (ws/wss)。
- SSH target：user@host（可选 :port）。
- 如果 Gateway 网关在同一局域网上并广播 Bonjour，从发现列表中选择它以自动填充此字段。

- Gateway URL（仅 Direct）：wss://gateway.example.ts.net（或本地/局域网使用 ws://...）。
- Identity file（高级）：你的密钥路径。
- Project root（高级）：用于命令的远程 checkout 路径。
- CLI path（高级）：可运行的 openclaw 入口点/二进制文件的可选路径（广播时自动填充）。
- 点击 Test remote。成功表示远程 openclaw status --json 正确运行。失败通常意味着 PATH/CLI 问题；退出码 127 表示远程找不到 CLI。
- 健康检查和 Web Chat 现在将自动通过此 SSH 隧道运行。

Web Chat

- SSH 隧道：Web Chat 通过转发的 WebSocket 控制端口（默认 18789）连接到 Gateway 网关。
- Direct (ws/wss)：Web Chat 直接连接到配置的 Gateway 网关 URL。
- 不再有单独的 WebChat HTTP 服务器。

权限

- 远程主机需要与本地相同的 TCC 批准（自动化、辅助功能、屏幕录制、麦克风、语音识别、通知）。在该机器上运行新手引导以一次性授予它们。
- 节点通过 node.list / node.describe 广播其权限状态，以便智能体知道哪些可用。

安全注意事项

- 优先在远程主机上使用 loopback 绑定，并通过 SSH 或 Tailscale 连接。
- 如果你将 Gateway 网关绑定到非 loopback 接口，请要求令牌/密码认证。
- 参见安全和 Tailscale。

WhatsApp 登录流程（远程）

- 在远程主机上运行 openclaw channels login --verbose。用手机上的 WhatsApp 扫描二维码。
- 如果认证过期，在该主机上重新运行登录。健康检查会显示关联问题。

故障排除

- exit 127 / not found：openclaw 不在非登录 shell 的 PATH 中。将其添加到 /etc/paths、你的 shell rc，或符号链接到 /usr/local/bin//opt/homebrew/bin。

- Health probe failed：检查 SSH 可达性、PATH，以及 Baileys 是否已登录（`openclaw status --json`）。
- Web Chat 卡住：确认 Gateway 网关正在远程主机上运行，转发的端口与 Gateway 网关 WS 端口匹配；UI 需要健康的 WS 连接。
- 节点 IP 显示 127.0.0.1：使用 SSH 隧道时是预期的。如果你想让 Gateway 网关看到真实的客户端 IP，请将 Transport 切换到 Direct (ws/wss)。
- Voice Wake：触发短语在远程模式下自动转发；不需要单独的转发器。

通知声音

通过带有 `openclaw` 和 `node.invoke` 的脚本为每个通知选择声音，例如：

代码：`openclaw nodes notify --node <id> --title "Ping" --body "Remote gateway ready" --sound Glass`

应用中不再有全局"默认声音"开关；调用者为每个请求选择声音（或无声音）。

18. Mac 签名（调试构建）

Mac 签名（调试构建）

此应用通常从 `scripts/package-mac-app.sh` 构建，该脚本目前会：

- 设置稳定的调试 Bundle 标识符：`ai.openclaw.mac.debug`
- 使用该 Bundle ID 写入 `Info.plist`（可通过 `BUNDLE_ID=...` 覆盖）
- 调用 `scripts/codesign-mac-app.sh` 对主二进制文件和应用包进行签名，使 macOS 将每次重新构建视为相同的已签名包，并保留 TCC 权限（通知、辅助功能、屏幕录制、麦克风、语音）。要获得稳定的权限，请使用真实签名身份；临时签名是可选的且不稳定（参阅 macOS 权限）。
- 默认使用 `CODESIGN_TIMESTAMP=auto`；为 Developer ID 签名启用受信任的时间戳。设置 `CODESIGN_TIMESTAMP=off` 可跳过时间戳（离线调试构建）。
- 将构建元数据注入 `Info.plist`：`OpenClawBuildTimestamp`（UTC）和 `OpenClawGitCommit`（短哈希），以便"关于"面板可以显示构建信息、git 信息和调试/发布渠道。
- 打包需要 Node 22+：脚本会运行 TS 构建和 Control UI 构建。
- 从环境变量中读取 `SIGN_IDENTITY`。将 `export SIGN_IDENTITY="Apple Development: Your Name (TEAMID)"`（或你的 Developer ID Application 证书）添加到 shell 配置文件中，以始终使用你的证书签名。临时签名需要通过 `ALLOW_ADHOC_SIGNING=1` 或 `SIGN_IDENTITY="-"` 显式启用（不建议用于权限测试）。

- 签名后运行 Team ID 审计，如果应用包内的任何 Mach-O 文件由不同的 Team ID 签名则会失败。设置 SKIP_TEAM_ID_CHECK=1 可跳过此检查。

用法

代码：# 从仓库根目录

代码：scripts/package-mac-app.sh # 自动选择身份；未找到时报错

代码：SIGN_IDENTITY="Developer ID Application: Your Name"

scripts/package-mac-app.sh # 真实证书

代码：ALLOW_ADHOC_SIGNING=1 scripts/package-mac-app.sh #

临时签名（权限不会持久化）

代码：SIGN_IDENTITY="-" scripts/package-mac-app.sh # 显式临时签名（同样的限制）

代码：DISABLE_LIBRARY_VALIDATION=1 scripts/package-mac-app.sh # 仅限开发的 Sparkle Team ID 不匹配解决方案

临时签名注意事项

使用 SIGN_IDENTITY="-"（临时签名）签名时，脚本会自动禁用强化运行时（--options runtime）。这是为了防止应用在尝试加载不共享相同 Team ID 的嵌入式框架（如 Sparkle）时崩溃。临时签名还会破坏 TCC 权限持久化；参阅 macOS 权限 了解恢复步骤。

关于面板的构建元数据

package-mac-app.sh 会在包中标记以下信息：

- OpenClawBuildTimestamp：打包时的 ISO8601 UTC 时间
- OpenClawGitCommit：短 git 哈希（不可用时为 unknown）

"关于"选项卡读取这些键以显示版本、构建日期、git 提交以及是否为调试构建（通过 #if DEBUG）。代码更改后运行打包程序以刷新这些值。

原因

TCC 权限与 Bundle 标识符和代码签名绑定。使用不断变化的 UUID 的未签名调试构建会导致 macOS 在每次重新构建后忘记授权。对二进制文件进行签名（默认临时签名）并保持固定的 Bundle ID/路径（dist/OpenClaw.app）可以在构建之间保留授权，与 VibeTunnel 的方案一致。

19. Skills (macOS)

Skills (macOS)

macOS 应用通过 Gateway 网关展示 OpenClaw Skills；它不会在本地解析 Skills。

数据来源

- skills.status (Gateway 网关) 返回所有 Skills 以及资格和缺失的要求 (包括内置 Skills 的允许列表阻止情况)。
- 要求来源于每个 SKILL.md 中的 metadata.openclaw.requires。

安装操作

- metadata.openclaw.install 定义安装选项 (brew/node/go/uv)。
- 应用调用 skills.install 在 Gateway 网关主机上运行安装器。
- 当提供多个安装器时，Gateway 网关仅展示一个首选安装器 (如果可用则使用 brew，否则使用来自 skills.install 的 node 管理器，默认 npm)。

环境变量/API 密钥

- 应用将密钥存储在 ~/.openclaw/openclaw.json 的 skills.entries.<skillKey> 下。
- skills.update 更新 enabled、apiKey 和 env。

远程模式

- 安装 + 配置更新发生在 Gateway 网关主机上 (不是本地 Mac)。

20. 语音浮层生命周期 (macOS)

语音浮层生命周期 (macOS)

受众：macOS 应用贡献者。目标：在唤醒词与按键说话重叠时保持语音浮层行为可预测。

当前意图

- 如果浮层已因唤醒词显示，此时用户按下热键，热键会话会接管现有文本而非重置。浮层在热键按住期间保持显示。用户松开时：如果有去除空白后的文本则发送，否则关闭。
- 单独使用唤醒词时仍在静音后自动发送；按键说话在松开时立即发送。

已实现 (2025 年 12 月 9 日)

- 浮层会话现在为每次捕获（唤醒词或按键说话）携带一个令牌。当令牌不匹配时，部分/最终/发送/关闭/音量更新会被丢弃，避免过时回调。
- 按键说话会接管任何可见的浮层文本作为前缀（因此在唤醒浮层显示时按下热键会保留文本并追加新语音）。它最多等待 1.5 秒获取最终转录结果，然后回退到当前文本。
- 提示音/浮层日志以 info 级别输出，分类为 `voicewake.overlay`、`voicewake.ptt` 和 `voicewake.chime`（会话开始、部分、最终、发送、关闭、提示音原因）。

后续步骤

- `VoiceSessionCoordinator` (actor)
- 同一时间只拥有一个 `VoiceSession`。
- API（基于令牌）：`beginWakeCapture`、`beginPushToTalk`、`updatePartial`、`endCapture`、`cancel`、`applyCooldown`。
- 丢弃携带过时令牌的回调（防止旧识别器重新打开浮层）。
- `VoiceSession`（模型）
- 字段：`token`、`source` (`wakeWord|pushToTalk`)、已提交/临时文本、提示音标志、计时器（自动发送、空闲）、`overlayMode` (`display|editing|sending`)、冷却截止时间。
- 浮层绑定
- `VoiceSessionPublisher` (`ObservableObject`) 将活跃会话镜像到 SwiftUI。
- `VoiceWakeOverlayView` 仅通过 `publisher` 渲染；绝不直接修改全局单例。
- 浮层用户操作 (`sendNow`、`dismiss`、`edit`) 携带会话令牌回调到 `coordinator`。
- 统一发送路径
- `endCapture` 时：如果去除空白后文本为空 → 关闭；否则 `performSend(session:)`（播放一次发送提示音、转发、关闭）。
- 按键说话：无延迟；唤醒词：可选自动发送延迟。
- 按键说话结束后对唤醒运行时施加短暂冷却，防止唤醒词立即重新触发。
- 日志
- `Coordinator` 在子系统 `bot.molt`、分类 `voicewake.overlay` 和 `voicewake.chime` 下输出 `.info` 级别日志。
- 关键事件：`session_started`、`adopted_by_push_to_talk`、`partial`、`finalized`、`send`、`dismiss`、`cancel`、`cooldown`。

调试清单

- 复现浮层粘滞问题时流式查看日志：

代码：`sudo log stream --predicate 'subsystem == "bot.molt" AND category CONTAINS "voicewake"' --level info --style compact`

- 验证只有一个活跃会话令牌；过时回调应被 coordinator 丢弃。
- 确保按键说话松开时始终使用活跃令牌调用 `endCapture`；如果文本为空，预期 `dismiss` 且不播放提示音或发送。

迁移步骤（建议）

- 添加 `VoiceSessionCoordinator`、`VoiceSession` 和 `VoiceSessionPublisher`。
- 重构 `VoiceWakeRuntime`，使其创建/更新/结束会话，而非直接操作 `VoiceWakeOverlayController`。
- 重构 `VoicePushToTalk`，使其接管现有会话并在松开时调用 `endCapture`；施加运行时冷却。
- 将 `VoiceWakeOverlayController` 连接到 publisher；移除来自 runtime/PTT 的直接调用。
- 添加会话接管、冷却和空文本关闭的集成测试。

21. 语音唤醒与按键通话

语音唤醒与按键通话

模式

- 唤醒词模式（默认）：常驻语音识别器等待触发词（`swabbleTriggerWords`）。匹配时开始捕获，显示带有部分文本的悬浮窗，并在静默后自动发送。
- 按键通话（按住右 Option 键）：按住右 Option 键立即开始捕获——无需触发词。按住时显示悬浮窗；松开后延迟片刻再最终转发，以便你可以调整文本。

运行时行为（唤醒词）

- 语音识别器位于 `VoiceWakeRuntime` 中。
- 仅当唤醒词和下一个词之间有明显停顿（约 0.55 秒间隔）时才触发。悬浮窗/提示音可以在命令开始前的停顿时就启动。
- 静默窗口：语音流畅时为 2.0 秒，如果只听到触发词则为 5.0 秒。
- 硬性停止：120 秒，防止会话失控。
- 会话间去抖动：350 毫秒。
- 悬浮窗通过 `VoiceWakeOverlayController` 驱动，带有已提交/临时状态的颜色区分。
- 发送后，识别器干净地重启以监听下一个触发词。

生命周期不变量

- 如果启用了语音唤醒且权限已授予，唤醒词识别器应该处于监听状态（除非正在进行显式的按键通话捕获）。
- 悬浮窗可见性（包括通过 X 按钮手动关闭）绝不能阻止识别器恢复。

悬浮窗卡住的故障模式（之前的问题）

之前，如果悬浮窗卡在可见状态且你手动关闭它，语音唤醒可能会显得"失效"，因为运行时的重启尝试可能被悬浮窗可见性阻止，且没有安排后续重启。

加固措施：

- 唤醒运行时重启不再被悬浮窗可见性阻止。
- 悬浮窗关闭完成时通过 VoiceSessionCoordinator 触发 VoiceWakeRuntime.refresh(...)，因此手动点击 X 关闭总是会恢复监听。

按键通话细节

- 热键检测使用全局 .flagsChanged 监视器检测右 Option 键 (keyCode 61 + .option)。我们只观察事件（不拦截）。
- 捕获管道位于 VoicePushToTalk
中：立即启动语音识别，将部分结果流式传输到悬浮窗，并在松开时调用 VoiceWakeForwarder。
- 按键通话开始时，我们暂停唤醒词运行时以避免音频采集冲突；松开后自动重启。
- 权限：需要麦克风 + 语音识别权限；查看事件需要辅助功能/输入监控批准。
- 外接键盘：某些键盘可能无法按预期暴露右 Option 键——如果用户报告未响应，提供备用快捷键。

面向用户的设置

- 语音唤醒开关：启用唤醒词运行时。
- 按住 Cmd+Fn 说话：启用按键通话监视器。在 macOS < 26 上禁用。
- 语言和麦克风选择器、实时电平指示器、触发词表、测试器（仅本地；不转发）。
- 麦克风选择器在设备断开时保留上次选择，显示断开提示，并临时回退到系统默认设备直到设备恢复。
- 声音：触发检测和发送时的提示音；默认为 macOS "Glass" 系统声音。你可以为每个事件选择任何 NSSound 可加载的文件（例如

MP3/WAV/AIFF) 或选择无声音。

转发行为

- 启用语音唤醒时，转录文本被转发到活动的 Gateway 网关/智能体（与 Mac 应用其他部分使用相同的本地/远程模式）。
- 回复被投递到上次使用的主提供商（WhatsApp/Telegram/Discord/WebChat）。如果投递失败，错误会被记录，运行记录仍可通过 WebChat/会话日志查看。

转发负载

- `VoiceWakeForwarder.prefixedTranscript(_:)`
在发送前添加机器提示前缀。唤醒词和按键通话路径共享此方法。

快速验证

- 开启按键通话，按住 `Cmd+Fn`，说话，松开：悬浮窗应显示部分结果然后发送。
- 按住时，菜单栏耳朵图标应保持放大状态（使用 `triggerVoiceEars(ttl:nil)`）；松开后恢复。

22. WebChat (macOS 应用)

WebChat (macOS 应用)

macOS 菜单栏应用将 WebChat UI 嵌入为原生 SwiftUI 视图。它连接到 Gateway 网关，默认使用所选智能体的主会话（带有会话切换器用于其他会话）。

- 本地模式：直接连接到本地 Gateway 网关 WebSocket。
- 远程模式：通过 SSH 转发 Gateway 网关控制端口，并使用该隧道作为数据平面。

启动和调试

- 手动：Lobster 菜单 → "Open Chat"。
- 测试时自动打开：

代码：`dist/OpenClaw.app/Contents/MacOS/OpenClaw --webchat`

- 日志：`./scripts/clawlog.sh`（子系统 `bot.molt`，类别 `WebChatSwiftUI`）。

工作原理

- 数据平面：Gateway 网关 WS 方法 `chat.history`、`chat.send`、`chat.abort`、`chat.inject` 和事件 `chat`、`agent`、`presence`、`tick`、`health`。

- 会话：默认为主会话（main，或当范围为全局时为 global）。UI 可以在会话之间切换。
- 新手引导使用专用会话，以将首次运行设置分开。

安全面

- 远程模式仅通过 SSH 转发 Gateway 网关 WebSocket 控制端口。

已知限制

- UI 针对聊天会话优化（不是完整的浏览器沙箱）。

23. OpenClaw macOS IPC 架构

OpenClaw macOS IPC 架构

当前模型：一个本地 Unix 套接字将节点主机服务连接到 macOS 应用，用于 exec 审批 + system.run。存在一个 openclaw-mac 调试 CLI 用于发现/连接检查；智能体操作仍通过 Gateway 网关 WebSocket 和 node.invoke 流转。UI 自动化使用 PeekabooBridge。

目标

- 单个 GUI 应用实例拥有所有面向 TCC 的工作（通知、屏幕录制、麦克风、语音、AppleScript）。
- 小型自动化接口：Gateway 网关 + 节点命令，加上用于 UI 自动化的 PeekabooBridge。
- 可预测的权限：始终是同一个签名的 bundle ID，由 launchd 启动，因此 TCC 授权保持有效。

工作原理

Gateway 网关 + 节点传输

- 应用运行 Gateway 网关（本地模式）并作为节点连接到它。
- 智能体操作通过 node.invoke 执行（例如 system.run、system.notify、canvas。）。

节点服务 + 应用 IPC

- 一个无头节点主机服务连接到 Gateway 网关 WebSocket。
- system.run 请求通过本地 Unix 套接字转发到 macOS 应用。
- 应用在 UI 上下文中执行 exec，必要时提示，并返回输出。

图示（SCI）：

代码：Agent -> Gateway -> Node Service (WS)

代码：| IPC (UDS + token + HMAC + TTL)

代码：v

代码：Mac App (UI + TCC + system.run)

PeekabooBridge (UI 自动化)

- UI 自动化使用名为 bridge.sock 的单独 UNIX 套接字和 PeekabooBridge JSON 协议。
- 主机优先顺序（客户端侧）：Peekaboo.app → Claude.app → OpenClaw.app → 本地执行。
- 安全性：桥接主机需要允许的 TeamID；仅 DEBUG 的同 UID 逃逸通道由 PEEKABOO_ALLOW_UNSIGNED_SOCKET_CLIENTS=1 保护（Peekaboo 约定）。
- 参见：PeekabooBridge 用法了解详情。

操作流程

- 重启/重建：SIGN_IDENTITY="Apple Development: <Developer Name> (<TEAMID>)"
scripts/restart-mac.sh
- 终止现有实例
- Swift 构建 + 打包
- 写入/引导/启动 LaunchAgent
- 单实例：如果具有相同 bundle ID 的另一个实例正在运行，应用会提前退出。

加固注意事项

- 优先要求所有特权接口的 TeamID 匹配。
- PeekabooBridge：PEEKABOO_ALLOW_UNSIGNED_SOCKET_CLIENTS=1（仅 DEBUG）可能允许同 UID 调用者用于本地开发。
- 所有通信仅保持本地；不暴露网络套接字。
- TCC 提示仅源自 GUI 应用包；在重建时保持签名的 bundle ID 稳定。
- IPC 加固：套接字模式 0600、令牌、对等 UID 检查、HMAC 质询/响应、短 TTL。

24. OpenClaw macOS 配套应用（菜单栏 + Gateway 网关代理）

OpenClaw macOS 配套应用（菜单栏 + Gateway 网关代理）

macOS 应用是 OpenClaw 的菜单栏配套应用。它拥有权限，在本地管理/附加到 Gateway 网关（launchd 或手动），并作为节点向智能体暴露 macOS 功能。

功能

- 在菜单栏中显示原生通知和状态。
- 拥有 TCC 提示（通知、辅助功能、屏幕录制、麦克风、语音识别、自动化/AppleScript）。
- 运行或连接到 Gateway 网关（本地或远程）。
- 暴露 macOS 专用工具（Canvas、相机、屏幕录制、system.run）。
- 在远程模式下启动本地节点主机服务（launchd），在本地模式下停止它。
- 可选地托管 PeekabooBridge 用于 UI 自动化。
- 根据请求通过 npm/pnpm 安装全局 CLI（openclaw）（不建议使用 bun 作为 Gateway 网关运行时）。

本地 vs 远程模式

- 本地（默认）：如果存在运行中的本地 Gateway 网关，应用附加到它；否则通过 openclaw gateway install 启用 launchd 服务。
- 远程：应用通过 SSH/Tailscale 连接到 Gateway 网关，从不启动本地进程。

应用启动本地节点主机服务，以便远程 Gateway 网关可以访问此 Mac。

应用不会将 Gateway 网关作为子进程生成。

Launchd 控制

应用管理一个标记为 bot.molt.gateway 的每用户 LaunchAgent（使用 `--profile/OPENCLAW_PROFILE` 时为 bot.molt.<profile>；旧版 com.openclaw. 仍会卸载）。

代码：launchctl kickstart -k gui/\$UID/bot.molt.gateway

代码：launchctl bootout gui/\$UID/bot.molt.gateway

运行命名配置文件时，将标签替换为 bot.molt.<profile>。

如果 LaunchAgent 未安装，从应用中启用它或运行 openclaw gateway install。

节点功能（mac）

macOS 应用将自身呈现为一个节点。常用命令：

- Canvas : canvas.present 、 canvas.navigate 、 canvas.eval 、 canvas.snapshot 、 canvas.a2ui.
- 相机 : camera.snap 、 camera.clip
- 屏幕 : screen.record
- 系统 : system.run 、 system.notify

节点报告一个 permissions 映射，以便智能体可以决定什么是允许的。

节点服务 + 应用 IPC :

- 当无头节点主机服务运行时（远程模式），它作为节点连接到 Gateway 网关 WS。
- system.run 在 macOS 应用中执行（UI/TCC 上下文）通过本地 Unix 套接字；提示 + 输出保留在应用内。

图示（SCI）：

代码：Gateway -> Node Service (WS)

代码：| IPC (UDS + token + HMAC + TTL)

代码：v

代码：Mac App (UI + TCC + system.run)

Exec 审批 (system.run)

system.run 由 macOS 应用中的 Exec 审批控制（设置 → Exec approvals）。安全 + 询问 + 允许列表本地存储在 Mac 上：

代码：~/openclaw/exec-approvals.json

示例：

代码：{

代码： "version": 1,

代码： "defaults": {

代码： "security": "deny",

代码： "ask": "on-miss"

代码： },

代码： "agents": {

代码： "main": {

代码： "security": "allowlist",

代码： "ask": "on-miss",

代码： "allowlist": [{ "pattern": "/opt/homebrew/bin/rg" }]

代码： }

代码： }

代码： }

注意事项：

- allowlist 条目是解析后二进制路径的 glob 模式。
- 在提示中选择 "Always Allow" 会将该命令添加到允许列表。
- system.run 环境覆盖会被过滤（删除 PATH、DYLD_、LD_、NODE_OPTIONS、PYTHON、PERL、RUBYOPT），然后与应用的环境合并。

深度链接

应用为本地操作注册 openclaw:// URL 方案。

``openclaw://agent``

触发 Gateway 网关 agent 请求。

代码： `open 'openclaw://agent?message=Hello%20from%20deep%20link'`

查询参数：

- message（必需）
- sessionKey（可选）
- thinking（可选）
- deliver / to / channel（可选）
- timeoutSeconds（可选）
- key（可选无人值守模式密钥）

安全：

- 没有 key 时，应用会提示确认。
- 有有效的 key 时，运行是无人值守的（用于个人自动化）。

新手引导流程（典型）

- 安装并启动 OpenClaw.app。
- 完成权限清单（TCC 提示）。
- 确保本地模式处于活动状态且 Gateway 网关正在运行。
- 如果你想要终端访问，安装 CLI。

构建和开发工作流程（原生）

- `cd apps/macos && swift build`
- `swift run OpenClaw`（或 Xcode）
- 打包应用：`scripts/package-mac-app.sh`

调试 Gateway 网关连接（macOS CLI）

使用调试 CLI 来执行与 macOS 应用使用的相同的 Gateway 网关 WebSocket 握手和发现逻辑，而无需启动应用。

代码：`cd apps/macos`

代码：`swift run openclaw-mac connect --json`

代码：`swift run openclaw-mac discover --timeout 3000 --json`

Connect 选项：

- `--url <ws://host:port>`：覆盖配置
- `--mode <local|remote>`：从配置解析（默认：配置或 local）
- `--probe`：强制进行新的健康探测
- `--timeout <ms>`：请求超时（默认：15000）
- `--json`：用于比较的结构化输出

Discovery 选项：

- `--include-local`：包含会被过滤为"本地"的 Gateway 网关
- `--timeout <ms>`：总体发现窗口（默认：2000）
- `--json`：用于比较的结构化输出

提示：与 `openclaw gateway discover --json` 比较，查看 macOS 应用的发现管道（NWBrowser + tailnet DNS-SD 回退）是否与 Node CLI 基于 `dns-sd`

的发现不同。

远程连接管道（SSH 隧道）

当 macOS 应用在远程模式下运行时，它会打开一个 SSH 隧道，以便本地 UI 组件可以像在 localhost 上一样与远程 Gateway 网关通信。

控制隧道（Gateway 网关 WebSocket 端口）

- 目的：健康检查、状态、Web Chat、配置和其他控制平面调用。
- 本地端口：Gateway 网关端口（默认 18789），始终稳定。
- 远程端口：远程主机上的相同 Gateway 网关端口。
- 行为：无随机本地端口；应用复用现有的健康隧道或在需要时重启它。
- SSH 形式：ssh -N -L <local>:127.0.0.1:<remote>，带有 BatchMode + ExitOnForwardFailure + keepalive 选项。
- IP 报告：SSH 隧道使用 loopback，因此 Gateway 网关将看到节点 IP 为 127.0.0.1。如果你想要显示真实的客户端 IP，请使用 Direct (ws/wss) 传输（参见 macOS 远程访问）。

有关设置步骤，请参阅 macOS 远程访问。有关协议详情，请参阅 Gateway 网关协议。

相关文档

- Gateway 网关运维手册
- Gateway 网关（macOS）
- macOS 权限
- Canvas

25. 在 Oracle Cloud（OCI）上运行 OpenClaw

在 Oracle Cloud（OCI）上运行 OpenClaw

目标

在 Oracle Cloud 的 Always Free ARM 层上运行持久化的 OpenClaw Gateway 网关。

Oracle 的免费层非常适合 OpenClaw（特别是如果你已经有 OCI 账户），但有一些权衡：

- ARM 架构（大多数东西都能工作，但某些二进制文件可能仅支持 x86）

- 容量和注册可能比较麻烦

成本比较 (2026)

提供商	方案	配置	价格/月	说明
Oracle Cloud	Always Free ARM	最多 4 OCPU, 24GB RAM	\$0	ARM, 容量有限
Hetzner	CX22	2 vCPU, 4GB RAM	~\$4	最便宜的付费选项
DigitalOcean	Basic	1 vCPU, 1GB RAM	\$6	易用的 UI, 文档完善
Vultr	Cloud Compute	1 vCPU, 1GB RAM	\$6	多个地区
Linode	Nanode	1 vCPU, 1GB RAM	\$5	现为 Akamai 的一部分

先决条件

- Oracle Cloud 账户 (注册) —— 如果遇到问题请参阅社区注册指南
- Tailscale 账户 (在 tailscale.com 免费)
- 约 30 分钟

1) 创建 OCI 实例

- 登录 Oracle Cloud Console
- 导航到 Compute → Instances → Create Instance
- 配置 :
 - Name: openclaw
 - Image: Ubuntu 24.04 (aarch64)
 - Shape: VM.Standard.A1.Flex (Ampere ARM)
 - OCPUs: 2 (或最多 4)
 - Memory: 12 GB (或最多 24 GB)
 - Boot volume: 50 GB (最多 200 GB 免费)
 - SSH key: 添加你的公钥
 - 点击 Create
 - 记录公网 IP 地址

提示：如果实例创建失败并显示"Out of capacity"，尝试不同的可用区域或稍后重试。免费层容量有限。

2) 连接并更新

代码：# 通过公网 IP 连接

代码：ssh ubuntu@YOUR_PUBLIC_IP

代码：# 更新系统

代码：sudo apt update && sudo apt upgrade -y

代码：sudo apt install -y build-essential

注意：build-essential 是某些依赖项 ARM 编译所必需的。

3) 配置用户和主机名

代码：# 设置主机名

代码：sudo hostnamectl set-hostname openclaw

代码：# 为 ubuntu 用户设置密码

代码：sudo passwd ubuntu

代码：# 启用 lingering（注销后保持用户服务运行）

代码：sudo loginctl enable-linger ubuntu

4) 安装 Tailscale

代码：curl -fsSL https://tailscale.com/install.sh | sh

代码：sudo tailscale up --ssh --hostname=openclaw

这会启用 Tailscale SSH，所以你可以从 tailnet 上的任何设备通过 ssh openclaw 连接——不需要公网 IP。

验证：

代码：tailscale status

从现在开始，通过 Tailscale 连接：ssh ubuntu@openclaw（或使用 Tailscale IP）。

5) 安装 OpenClaw

代码：curl -fsSL https://openclaw.ai/install.sh | bash

代码：source ~/.bashrc

当提示"How do you want to hatch your bot?"时，选择 "Do this later"。

注意：如果你遇到 ARM 原生构建问题，在使用 Homebrew 之前先从系统包开始（例如 sudo apt install -y build-essential）。

6) 配置 Gateway 网关（loopback + 令牌认证）并启用 Tailscale Serve

使用令牌认证作为默认值。它是可预测的，避免需要任何"不安全认证"的控制 UI 标志。

代码：# 在 VM 上保持 Gateway 网关私有

代码：openclaw config set gateway.bind loopback

代码：# 要求 Gateway 网关 + 控制 UI 的认证

代码：openclaw config set gateway.auth.mode token

代码：openclaw doctor --generate-gateway-token

代码：# 通过 Tailscale Serve 暴露（HTTPS + tailnet 访问）

代码：openclaw config set gateway.tailscale.mode serve

代码：openclaw config set gateway.trustedProxies '["127.0.0.1"]'

代码：systemctl --user restart openclaw-gateway

7) 验证

代码：# 检查版本

代码：openclaw --version

代码：# 检查守护进程状态

代码：systemctl --user status openclaw-gateway

代码：# 检查 Tailscale Serve

代码：tailscale serve status

代码：# 测试本地响应

代码：curl http://localhost:18789

8) 锁定 VCN 安全

现在一切正常工作了，锁定 VCN 以阻止除 Tailscale 之外的所有流量。OCI 的虚拟云网络充当网络边缘的防火墙——流量在到达你的实例之前就被阻止。

- 在 OCI Console 中转到 Networking → Virtual Cloud Networks
- 点击你的 VCN → Security Lists → Default Security List
- 移除除以下之外的所有入站规则：
- 0.0.0.0/0 UDP 41641 (Tailscale)
- 保留默认出站规则（允许所有出站）

这会在网络边缘阻止端口 22 上的 SSH、HTTP、HTTPS 和其他所有内容。从现在开始，你只能通过 Tailscale 连接。

访问控制 UI

从你 Tailscale 网络上的任何设备：

代码：<https://openclaw.<tailnet-name>.ts.net/>

将 <tailnet-name> 替换为你的 tailnet 名称（在 tailscale status 中可见）。

不需要 SSH 隧道。Tailscale 提供：

- HTTPS 加密（自动证书）
- 通过 Tailscale 身份认证
- 从 tailnet 上的任何设备（笔记本电脑、手机等）访问

安全：VCN + Tailscale（推荐基线）

通过锁定 VCN（仅开放 UDP 41641）并将 Gateway 网关绑定到 loopback，你获得了强大的纵深防御：公共流量在网络边缘被阻止，管理访问通过你的 tailnet 进行。

此设置通常消除了纯粹为了阻止互联网范围的 SSH

暴力破解而需要额外的基于主机的防火墙规则的需求——但你仍应保持操作系统更新，运行 openclaw security audit，并验证你没有意外地在公共接口上监听。

已经受保护的内容

传统步骤	是否需要？	原因
-----	-----	-----
UFW 防火墙	否	VCN 在流量到达实例之前就阻止了
fail2ban	否	如果端口 22 在 VCN 被阻止则无暴力破解
sshd 加固	否	Tailscale SSH 不使用 sshd
禁用 root 登录	否	Tailscale 使用 Tailscale 身份，而不是系统用户
仅 SSH 密钥认证	否	Tailscale 通过你的 tailnet 认证
IPv6 加固	通常不需要	取决于你的 VCN/子网设置；验证实际分配/暴露的内容

仍然推荐

- 凭证权限： `chmod 700 ~/.openclaw`
- 安全审计： `openclaw security audit`
- 系统更新： 定期 `sudo apt update && sudo apt upgrade`
- 监控 Tailscale： 在 Tailscale 管理控制台 中查看设备

验证安全态势

代码：# 确认没有公共端口在监听

代码： `sudo ss -tlnp | grep -v '127.0.0.1\|:::1'`

代码：# 验证 Tailscale SSH 处于活动状态

代码： `tailscale status | grep -q 'offers: ssh' && echo "Tailscale SSH active"`

代码：# 可选：完全禁用 sshd

代码： `sudo systemctl disable --now ssh`

备用方案：SSH 隧道

如果 Tailscale Serve 不工作，使用 SSH 隧道：

代码：# 从你的本地机器（通过 Tailscale）

代码： `ssh -L 18789:127.0.0.1:18789 ubuntu@openclaw`

然后打开`

故障排除

实例创建失败 ("Out of capacity")

免费层 ARM 实例很受欢迎。尝试：

- 不同的可用性域
- 在非高峰时段（清晨）重试
- 选择 shape 时使用 "Always Free" 过滤器

Tailscale 无法连接

代码：# 检查状态

代码：sudo tailscale status

代码：# 重新认证

代码：sudo tailscale up --ssh --hostname=openclaw --reset

Gateway 网关无法启动

代码：openclaw gateway status

代码：openclaw doctor --non-interactive

代码：journalctl --user -u openclaw-gateway -n 50

无法访问控制 UI

代码：# 验证 Tailscale Serve 正在运行

代码：tailscale serve status

代码：# 检查 Gateway 网关是否在监听

代码：curl http://localhost:18789

代码：# 如需要则重启

代码：systemctl --user restart openclaw-gateway

ARM 二进制文件问题

某些工具可能没有 ARM 构建。检查：

代码：uname -m # 应该显示 aarch64

大多数 npm 包工作正常。对于二进制文件，寻找 linux-arm64 或 aarch64 版本。

持久化

所有状态存储在：

- ~/.openclaw/ — 配置、凭证、会话数据
- ~/.openclaw/workspace/ — 工作区 (SOUL.md、记忆、产物)

定期备份：

代码：tar -czvf openclaw-backup.tar.gz ~/.openclaw ~/.openclaw/workspace

另请参阅

- Gateway 网关远程访问 — 其他远程访问模式
- Tailscale 集成 — 完整的 Tailscale 文档
- Gateway 网关配置 — 所有配置选项
- DigitalOcean 指南 — 如果你想要付费 + 更容易注册
- Hetzner 指南 — 基于 Docker 的替代方案

26. 在 Raspberry Pi 上运行 OpenClaw

在 Raspberry Pi 上运行 OpenClaw

目标

在 Raspberry Pi 上运行持久、常驻的 OpenClaw Gateway 网关，一次性成本约 \$35-80（无月费）。

适用于：

- 24/7 个人 AI 助手
- 家庭自动化中心
- 低功耗、随时可用的 Telegram/WhatsApp 机器人

硬件要求

Pi 型号	内存	是否可用？	说明
-----	-----	-----	-----
Pi 5	4GB/8GB	最佳	最快，推荐
Pi 4	4GB	良好	大多数用户的最佳选择
Pi 4	2GB	可以	可用，添加交换空间
Pi 4	1GB	紧张	使用交换空间可行，最小配置
Pi 3B+	1GB	慢	可用但较慢
Pi Zero 2 W	512MB		不推荐

最低配置：1GB 内存，1 核，500MB 磁盘

推荐：2GB+ 内存，64 位系统，16GB+ SD 卡（或 USB SSD）

你需要准备

- Raspberry Pi 4 或 5（推荐 2GB+）
- MicroSD 卡（16GB+）或 USB SSD（性能更好）
- 电源（推荐官方 Pi 电源）
- 网络连接（以太网或 WiFi）
- 约 30 分钟

1) 刷写系统

使用 Raspberry Pi OS Lite (64-bit) — 无头服务器不需要桌面。

- 下载 Raspberry Pi Imager
- 选择系统：Raspberry Pi OS Lite (64-bit)
- 点击齿轮图标（）预配置：
- 设置主机名：gateway-host
- 启用 SSH
- 设置用户名/密码
- 配置 WiFi（如果不使用以太网）
- 刷写到你的 SD 卡 / USB 驱动器

- 插入并启动 Pi

2) 通过 SSH 连接

代码：ssh user@gateway-host

代码：# 或使用 IP 地址

代码：ssh user@192.168.x.x

3) 系统设置

代码：# 更新系统

代码：sudo apt update && sudo apt upgrade -y

代码：# 安装必要软件包

代码：sudo apt install -y git curl build-essential

代码：# 设置时区（对 cron/提醒很重要）

代码：sudo timedatectl set-timezone America/Chicago # 改成你的时区

4) 安装 Node.js 22（ARM64）

代码：# 通过 NodeSource 安装 Node.js

代码：curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -

代码：sudo apt install -y nodejs

代码：# 验证

代码：node --version # 应显示 v22.x.x

代码：npm --version

5) 添加交换空间（2GB 或更少内存时很重要）

交换空间可防止内存不足崩溃：

代码：# 创建 2GB 交换文件

代码：sudo fallocate -l 2G /swapfile

代码：sudo chmod 600 /swapfile

代码：sudo mkswap /swapfile

代码：sudo swapon /swapfile

代码：# 永久生效

代码：echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

代码：# 优化低内存（降低 swappiness）

代码：echo 'vm.swappiness=10' | sudo tee -a /etc/sysctl.conf

代码：sudo sysctl -p

6) 安装 OpenClaw

选项 A：标准安装（推荐）

代码：curl -fsSL https://openclaw.ai/install.sh | bash

选项 B：可修改安装（用于调试）

代码：git clone https://github.com/openclaw/openclaw.git

代码：cd openclaw

代码：npm install

代码：npm run build

代码：npm link

可修改安装让你可以直接访问日志和代码 — 对调试 ARM 特定问题很有用。

7) 运行新手引导

代码：openclaw onboard --install-daemon

按照向导操作：

- Gateway 网关模式：Local
- 认证：推荐 API 密钥（OAuth 在无头 Pi 上可能不稳定）
- 渠道：Telegram 最容易上手
- 守护进程：是（systemd）

8) 验证安装

代码：# 检查状态

代码：openclaw status

代码：# 检查服务

代码：sudo systemctl status openclaw

代码：# 查看日志

代码：journalctl -u openclaw -f

9) 访问仪表板

由于 Pi 是无头的，使用 SSH 隧道：

代码：# 从你的笔记本电脑/台式机

代码：ssh -L 18789:localhost:18789 user@gateway-host

代码：# 然后在浏览器中打开

代码：open http://localhost:18789

或使用 Tailscale 实现常驻访问：

代码：# 在 Pi 上

代码：curl -fsSL https://tailscale.com/install.sh | sh

代码：sudo tailscale up

代码：# 更新配置

代码：openclaw config set gateway.bind tailnet

代码：sudo systemctl restart openclaw

性能优化

使用 USB SSD（巨大改进）

SD 卡速度慢且会磨损。USB SSD 可大幅提升性能：

代码：# 检查是否从 USB 启动

代码：lsblk

设置请参见 Pi USB 启动指南。

减少内存使用

```
代码：# 禁用 GPU 内存分配（无头模式）
代码：echo 'gpu_mem=16' | sudo tee -a /boot/config.txt
代码：# 如不需要则禁用蓝牙
代码：sudo systemctl disable bluetooth
```

监控资源

```
代码：# 检查内存
代码：free -h

代码：# 检查 CPU 温度
代码：vcgencmd measure_temp

代码：# 实时监控
代码：htop
```

ARM 特定说明

二进制兼容性

大多数 OpenClaw 功能在 ARM64 上可用，但某些外部二进制文件可能需要 ARM 构建：

工具	ARM64 状态	说明
Node.js	运行良好	
WhatsApp (Baileys)	纯 JS	无问题
Telegram	纯 JS	无问题
gog (Gmail CLI)		检查是否有 ARM 版本
Chromium (browser)		sudo apt install chromium-browser

如果某个 skill 失败，检查其二进制文件是否有 ARM 构建。许多 Go/Rust 工具有；有些没有。

32 位 vs 64 位

始终使用 64 位系统。Node.js 和许多现代工具需要它。使用以下命令检查：

代码：uname -m

代码：# 应显示：aarch64（64 位）而不是 armv7l（32 位）

推荐的模型设置

由于 Pi 只是 Gateway 网关（模型在云端运行），使用基于 API 的模型：

代码：{

代码："agents": {

代码："defaults": {

代码："model": {

代码："primary": "anthropic/claude-sonnet-4-20250514",

代码："fallbacks": ["openai/gpt-4o-mini"]

代码：}

代码：}

代码：}

代码：}

不要尝试在 Pi 上运行本地 LLM — 即使是小模型也太慢了。让 Claude/GPT 来做繁重的工作。

开机自启

新手引导向导会设置这个，但要验证：

代码：# 检查服务是否已启用

代码：sudo systemctl is-enabled openclaw

代码：# 如果没有则启用

代码：sudo systemctl enable openclaw

代码：# 开机启动

代码：sudo systemctl start openclaw

故障排除

内存不足 (OOM)

代码：# 检查内存

代码：free -h

代码：# 添加更多交换空间（见步骤 5）

代码：# 或减少 Pi 上运行的服务

性能慢

- 使用 USB SSD 代替 SD 卡
- 禁用未使用的服务：sudo systemctl disable cups bluetooth avahi-daemon
- 检查 CPU 降频：vcgencmd get_throttled（应返回 0x0）

服务无法启动

代码：# 检查日志

代码：journalctl -u openclaw --no-pager -n 100

代码：# 常见修复：重新构建

代码：cd ~/openclaw # 如果使用可修改安装

代码：npm run build

代码：sudo systemctl restart openclaw

ARM 二进制问题

如果某个 skill 失败并显示"exec format error"：

- 检查该二进制文件是否有 ARM64 构建
- 尝试从源代码构建
- 或使用支持 ARM 的 Docker 容器

WiFi 断开

对于使用 WiFi 的无头 Pi：

代码：# 禁用 WiFi 电源管理

代码：sudo iwconfig wlan0 power off

代码：# 永久生效

代码：echo 'wireless-power off' | sudo tee -a /etc/network/interfaces

成本对比

设置	一次性成本	月费	说明
-----	-----	-----	-----
Pi 4 (2GB)	~\$45	\$0	+ 电费 (约 \$5/年)
Pi 4 (4GB)	~\$55	\$0	推荐
Pi 5 (4GB)	~\$60	\$0	最佳性能
Pi 5 (8GB)	~\$80	\$0	过剩但面向未来
DigitalOcean	\$0	\$6/月	\$72/年
Hetzner	\$0	€3.79/月	约 \$50/年

回本期：与云 VPS 相比，Pi 约 6-12 个月内回本。

另请参阅

- Linux 指南 — 通用 Linux 设置
- DigitalOcean 指南 — 云替代方案
- Hetzner 指南 — Docker 设置
- Tailscale — 远程访问
- 节点 — 将你的笔记本电脑/手机与 Pi Gateway 网关配对

27. Windows (WSL2)

Windows (WSL2)

Windows 上的 OpenClaw 推荐通过 WSL2（推荐 Ubuntu）。CLI + Gateway 网关在 Linux 内运行，这保持了运行时的一致性并使工具兼容性大大提高（Node/Bun/pnpm、Linux 二进制文件、Skills）。原生 Windows 可能更棘手。WSL2 给你完整的 Linux 体验——一条命令安装：wsl --install。

原生 Windows 配套应用已在计划中。

安装 (WSL2)

- 入门指南 (在 WSL 内使用)
- 安装和更新
- 官方 WSL2 指南 (Microsoft) :

Gateway 网关

- Gateway 网关操作手册
- 配置

Gateway 网关服务安装 (CLI)

在 WSL2 内：

代码：openclaw onboard --install-daemon

或：

代码：openclaw gateway install

或：

代码：openclaw configure

出现提示时选择 Gateway service。

修复/迁移：

代码：openclaw doctor

高级：通过 LAN 暴露 WSL 服务 (portproxy)

WSL 有自己的虚拟网络。如果另一台机器需要访问在 WSL 内运行的服务 (SSH、本地 TTS 服务器或 Gateway 网关)，你必须将 Windows 端口转发到当前的 WSL IP。WSL IP 在重启后会改变，因此你可能需要刷新转发规则。

示例 (以管理员身份运行 PowerShell)：

代码：\$Distro = "Ubuntu-24.04"

代码：\$ListenPort = 2222

代码：\$TargetPort = 22

代码：\$WslIp = (wsl -d \$Distro -- hostname -I).Trim().Split(" ")[0]

代码：if (-not \$WslIp) { throw "WSL IP not found." }

代码：netsh interface portproxy add v4tov4 listenaddress=0.0.0.0 listenport=\$ListenPort `

代码：connectaddress=\$WslIp connectport=\$TargetPort

允许端口通过 Windows 防火墙（一次性）：

代码：New-NetFirewallRule -DisplayName "WSL SSH \$ListenPort" -Direction Inbound `

代码：-Protocol TCP -LocalPort \$ListenPort -Action Allow

在 WSL 重启后刷新 portproxy：

代码：netsh interface portproxy delete v4tov4 listenport=\$ListenPort listenaddress=0.0.0.0 | Out-Null

代码：netsh interface portproxy add v4tov4 listenport=\$ListenPort listenaddress=0.0.0.0 `

代码：connectaddress=\$WslIp connectport=\$TargetPort | Out-Null

注意事项：

- 从另一台机器 SSH 目标是 Windows 主机 IP（示例：ssh user@windows-host -p 2222）。
- 远程节点必须指向可访问的 Gateway 网关 URL（不是 127.0.0.1）；使用 openclaw status --all 确认。
- 使用 listenaddress=0.0.0.0 进行 LAN 访问；127.0.0.1 仅保持本地访问。
- 如果你想自动化，注册一个计划任务在登录时运行刷新步骤。

WSL2 分步安装

1) 安装 WSL2 + Ubuntu

打开 PowerShell（管理员）：

代码：wsl --install

代码：# Or pick a distro explicitly:

代码：wsl --list --online

代码：wsl --install -d Ubuntu-24.04

如果 Windows 要求则重启。

2) 启用 systemd (Gateway 网关安装所需)

在你的 WSL 终端中：

代码：`sudo tee /etc/wsl.conf >/dev/null <<'EOF'`

代码：`[boot]`

代码：`systemd=true`

代码：`EOF`

然后从 PowerShell：

代码：`wsl --shutdown`

重新打开 Ubuntu，然后验证：

代码：`systemctl --user status`

3) 安装 OpenClaw (在 WSL 内)

在 WSL 内按照 Linux 入门指南流程：

代码：`git clone https://github.com/openclaw/openclaw.git`

代码：`cd openclaw`

代码：`pnpm install`

代码：`pnpm ui:build # auto-installs UI deps on first run`

代码：`pnpm build`

代码：`openclaw onboard`

完整指南：入门指南

Windows 配套应用

我们还没有 Windows 配套应用。如果你想让它实现，欢迎贡献。